

Spring Boot Reference Documentation

Table of Contents

1. Getting Started
2. Installing Spring Boot
 - 2.1. Installation Instructions for the Java Developer
 - 2.1.1. Maven Installation
 - 2.1.2. Gradle Installation
 - 2.2. Installing the Spring Boot CLI
 - 2.2.1. Manual Installation
 - 2.2.2. Installation with SDKMAN!
 - 2.2.3. OSX Homebrew Installation
 - 2.2.4. MacPorts Installation
 - 2.2.5. Command-line Completion
 - 2.2.6. Windows Scoop Installation
3. System Requirements
 - 3.1. Servlet Containers
 - 3.2. GraalVM Native Images
4. Upgrading Spring Boot
 - 4.1. Upgrading From 1.x
 - 4.2. Upgrading From 2.x
 - 4.3. Upgrading From 3.x
 - 4.4. Upgrading to a New Feature Release
 - 4.5. Upgrading the Spring Boot CLI
5. Tutorial
6. Developing Your First Spring Boot Application
 - 6.1. Prerequisites
 - 6.1.1. Maven
 - 6.1.2. Gradle
 - 6.2. Setting Up the Project With Maven
 - 6.3. Setting Up the Project With Gradle
 - 6.4. Adding Classpath Dependencies
 - 6.4.1. Maven
 - 6.4.2. Gradle
 - 6.5. Writing the Code
 - 6.5.1. The `@RestController` and `@RequestMapping` Annotations
 - 6.5.2. The `@SpringBootApplication` Annotation
 - 6.5.3. The “main” Method
 - 6.6. Running the Example
 - 6.6.1. Maven
 - 6.6.2. Gradle
 - 6.7. Creating an Executable Jar
 - 6.7.1. Maven
 - 6.7.2. Gradle
7. Tutorials
8. Reference Documentation
9. Auditing
 - 9.1. Custom Auditing
10. Cloud Foundry Support
 - 10.1. Disabling Extended Cloud Foundry Actuator Support
 - 10.2. Cloud Foundry Self-signed Certificates
 - 10.3. Custom Context Path
11. Enabling Production-ready Features
12. Endpoints
 - 12.1. Controlling Access to Endpoints
 - 12.1.1. Limiting Access
 - 12.2. Exposing Endpoints
 - 12.3. Security
 - 12.3.1. Cross Site Request Forgery Protection
 - 12.4. Configuring Endpoints
 - 12.5. Sanitize Sensitive Values
 - 12.6. Hypermedia for Actuator Web Endpoints
 - 12.7. CORS Support
 - 12.8. JSON
 - 12.9. Implementing Custom Endpoints
 - 12.9.1. Receiving Input
 - Input Type Conversion
 - 12.9.2. Custom Web Endpoints

- Web Endpoint Request Predicates
 - Path
 - HTTP method
 - Consumes
 - Produces
 - Web Endpoint Response Status
 - Web Endpoint Range Requests
 - Web Endpoint Security
- 12.10. Health Information
 - 12.10.1. Auto-configured HealthIndicators
 - 12.10.2. Writing Custom HealthIndicators
 - 12.10.3. Reactive Health Indicators
 - 12.10.4. Auto-configured ReactiveHealthIndicators
 - 12.10.5. Health Groups
 - 12.10.6. DataSource Health
- 12.11. Kubernetes Probes
 - 12.11.1. Checking External State With Kubernetes Probes
 - 12.11.2. Application Lifecycle and Probe States
- 12.12. Application Information
 - 12.12.1. Auto-configured InfoContributors
 - 12.12.2. Custom Application Information
 - 12.12.3. Git Commit Information
 - 12.12.4. Build Information
 - 12.12.5. Java Information
 - 12.12.6. OS Information
 - 12.12.7. Process Information
 - 12.12.8. SSL Information
 - 12.12.9. Writing Custom InfoContributors
- 12.13. Software Bill of Materials (SBOM)
 - 12.13.1. Other SBOM formats
 - 12.13.2. Additional SBOMs
- 13. Recording HTTP Exchanges
 - 13.1. Custom HTTP Exchange Recording
- 14. Production-ready Features
- 15. Monitoring and Management over JMX
 - 15.1. Customizing MBean Names
 - 15.2. Disabling JMX Endpoints
- 16. Loggers
 - 16.1. Configure a Logger
 - 16.2. OpenTelemetry
- 17. Metrics
 - 17.1. Getting Started
 - 17.2. Supported Monitoring Systems
 - 17.2.1. AppOptics
 - 17.2.2. Atlas
 - 17.2.3. Datadog
 - 17.2.4. Dynatrace
 - v2 API
 - v1 API (Legacy)
 - Version-independent Settings
 - 17.2.5. Elastic
 - 17.2.6. Ganglia
 - 17.2.7. Graphite
 - 17.2.8. Humio
 - 17.2.9. Influx
 - 17.2.10. JMX
 - 17.2.11. KairosDB
 - 17.2.12. New Relic
 - 17.2.13. OTLP
 - 17.2.14. Prometheus
 - 17.2.15. Simple
 - 17.2.16. Stackdriver
 - 17.2.17. StatsD
 - 17.3. Supported Metrics and Meters
 - 17.3.1. JVM Metrics
 - 17.3.2. System Metrics
 - 17.3.3. Application Startup Metrics

- 17.3.4. Logger Metrics
- 17.3.5. Task Execution and Scheduling Metrics
- 17.3.6. JMS Metrics
- 17.3.7. Spring MVC Metrics
- 17.3.8. Spring WebFlux Metrics
- 17.3.9. Jersey Server Metrics
- 17.3.10. SSL Bundle Metrics
- 17.3.11. HTTP Client Metrics
- 17.3.12. Tomcat Metrics
- 17.3.13. Cache Metrics
- 17.3.14. Spring Batch Metrics
- 17.3.15. Spring GraphQL Metrics
- 17.3.16. DataSource Metrics
- 17.3.17. Hibernate Metrics
- 17.3.18. Spring Data Repository Metrics
- 17.3.19. RabbitMQ Metrics
- 17.3.20. Spring Integration Metrics
- 17.3.21. Kafka Metrics
- 17.3.22. MongoDB Metrics
 - MongoDB Command Metrics
 - MongoDB Connection Pool Metrics
- 17.3.23. Neo4j Metrics
- 17.3.24. Jetty Metrics
- 17.3.25. Redis Metrics
- 17.4. Registering Custom Metrics
- 17.5. Customizing Individual Metrics
 - 17.5.1. Common Tags
 - 17.5.2. Per-meter Properties
- 17.6. Metrics Endpoint
- 17.7. Integration with Micrometer Observation
- 18. Monitoring and Management Over HTTP
 - 18.1. Customizing the Management Endpoint Paths
 - 18.2. Customizing the Management Server Port
 - 18.3. Configuring Management-specific SSL
 - 18.4. Customizing the Management Server Address
 - 18.5. Disabling HTTP Endpoints
- 19. Observability
 - 19.1. Context Propagation
 - 19.2. Common Tags
 - 19.3. Preventing Observations
 - 19.4. Micrometer Observation Annotations support
 - 19.5. OpenTelemetry Support
 - 19.5.1. Environment variables
 - 19.5.2. Logging
 - 19.5.3. Metrics
 - Metrics via the OpenTelemetry API and SDK
 - 19.5.4. Tracing
- 20. Process Monitoring
 - 20.1. Extending Configuration
 - 20.2. Programmatically Enabling Process Monitoring
- 21. Tracing
 - 21.1. Supported Tracers
 - 21.2. Getting Started
 - 21.3. Logging Correlation IDs
 - 21.4. Propagating Traces
 - 21.5. Tracer Implementations
 - 21.5.1. OpenTelemetry With OTLP
 - 21.5.2. OpenTelemetry With Zipkin
 - 21.5.3. OpenZipkin Brave With Zipkin
 - 21.6. Integration with Micrometer Observation
 - 21.7. Creating Custom Spans
 - 21.8. Baggage
 - 21.9. Tests
- 22. Data
- 23. Working with NoSQL Technologies
 - 23.1. Redis
 - 23.1.1. Connecting to Redis

- 23.2. MongoDB
 - 23.2.1. Connecting to a MongoDB Database
 - 23.2.2. MongoTemplate
 - 23.2.3. Spring Data MongoDB Repositories
- 23.3. Neo4j
 - 23.3.1. Connecting to a Neo4j Database
 - 23.3.2. Spring Data Neo4j Repositories
- 23.4. Elasticsearch
 - 23.4.1. Connecting to Elasticsearch Using REST clients
 - Connecting to Elasticsearch Using Rest5Client
 - Connecting to Elasticsearch Using ElasticsearchClient
 - Connecting to Elasticsearch using ReactiveElasticsearchClient
 - 23.4.2. Connecting to Elasticsearch by Using Spring Data
 - 23.4.3. Spring Data Elasticsearch Repositories
- 23.5. Cassandra
 - 23.5.1. Connecting to Cassandra
 - 23.5.2. Spring Data Cassandra Repositories
- 23.6. Couchbase
 - 23.6.1. Connecting to Couchbase
 - 23.6.2. Spring Data Couchbase Repositories
- 23.7. LDAP
 - 23.7.1. Connecting to an LDAP Server
 - 23.7.2. Spring Data LDAP Repositories
 - 23.7.3. Embedded In-memory LDAP Server
 - SSL
- 24. SQL Databases
 - 24.1. Configure a DataSource
 - 24.1.1. Embedded Database Support
 - 24.1.2. Connection to a Production Database
 - 24.1.3. DataSource Configuration
 - 24.1.4. Supported Connection Pools
 - 24.1.5. Connection to a JNDI DataSource
 - 24.2. Using JdbcTemplate
 - 24.3. Using JdbcClient
 - 24.4. JPA and Spring Data JPA
 - 24.4.1. Entity Classes
 - 24.4.2. Spring Data JPA Repositories
 - 24.4.3. Spring Data Envers Repositories
 - 24.4.4. Creating and Dropping JPA Databases
 - 24.4.5. Open EntityManager in View
 - 24.5. Spring Data JDBC
 - 24.6. Using H2's Web Console
 - 24.6.1. Changing the H2 Console's Path
 - 24.6.2. Accessing the H2 Console in a Secured Application
 - 24.7. Using jOOQ
 - 24.7.1. Code Generation
 - 24.7.2. Using DSLContext
 - 24.7.3. jOOQ SQL Dialect
 - 24.7.4. Customizing jOOQ
 - 24.8. Using R2DBC
 - 24.8.1. Embedded Database Support
 - 24.8.2. Using DatabaseClient
 - 24.8.3. Spring Data R2DBC Repositories
- 25. Aspect-Oriented Programming
- 26. Development-time Services
 - 26.1. Docker Compose Support
 - 26.1.1. Prerequisites
 - 26.1.2. Service Connections
 - 26.1.3. SSL support
 - 26.1.4. Custom Images
 - 26.1.5. Skipping Specific Containers
 - 26.1.6. Using a Specific Compose File
 - 26.1.7. Waiting for Container Readiness
 - 26.1.8. Controlling the Docker Compose Lifecycle
 - 26.1.9. Activating Docker Compose Profiles
 - 26.1.10. Using Docker Compose in Tests
 - 26.2. Testcontainers Support

- 26.2.1. Using Testcontainers at Development Time
 - Contributing Dynamic Properties at Development Time
 - Importing Testcontainers Declaration Classes
 - Using DevTools with Testcontainers at Development Time
- 27. Creating Your Own Auto-configuration
 - 27.1. Understanding Auto-configured Beans
 - 27.2. Locating Auto-configuration Candidates
 - 27.2.1. Deprecating and Replacing Auto-configuration Classes
 - 27.3. Condition Annotations
 - 27.3.1. Class Conditions
 - 27.3.2. Bean Conditions
 - 27.3.3. Property Conditions
 - 27.3.4. Resource Conditions
 - 27.3.5. Web Application Conditions
 - 27.3.6. SpEL Expression Conditions
 - 27.4. Testing your Auto-configuration
 - 27.4.1. Simulating a Web Context
 - 27.4.2. Overriding the Classpath
 - 27.5. Creating Your Own Starter
 - 27.5.1. Naming
 - 27.5.2. Configuration keys
 - 27.5.3. The “autoconfigure” Module
 - 27.5.4. Starter Module
- 28. Externalized Configuration
 - 28.1. Accessing Command Line Properties
 - 28.2. JSON Application Properties
 - 28.3. External Application Properties
 - 28.3.1. Optional Locations
 - 28.3.2. Wildcard Locations
 - 28.3.3. Profile Specific Files
 - 28.3.4. Importing Additional Data
 - Using “Fixed” and “Import Relative” Locations
 - Property Ordering
 - 28.3.5. Importing Extensionless Files
 - 28.3.6. Using Environment Variables
 - 28.3.7. Using Configuration Trees
 - 28.3.8. Property Placeholders
 - 28.3.9. Working With Multi-Document Files
 - 28.3.10. Activation Properties
 - 28.4. Encrypting Properties
 - 28.5. Working With YAML
 - 28.5.1. Mapping YAML to Properties
 - 28.5.2. Directly Loading YAML
 - 28.6. Configuring Random Values
 - 28.7. Configuring System Environment Properties
 - 28.8. Type-safe Configuration Properties
 - 28.8.1. JavaBean Properties Binding
 - 28.8.2. Constructor Binding
 - 28.8.3. Enabling @ConfigurationProperties-annotated Types
 - 28.8.4. Using @ConfigurationProperties-annotated Types
 - 28.8.5. Third-party Configuration
 - 28.8.6. Relaxed Binding
 - Binding Maps
 - Binding From Environment Variables
 - Binding Maps From Environment Variables
 - Caching
 - 28.8.7. Merging Complex Types
 - 28.8.8. Properties Conversion
 - Converting Durations
 - Converting Periods
 - Converting Data Sizes
 - Converting Base64 Data
 - 28.8.9. @ConfigurationProperties Validation
 - 28.8.10. @ConfigurationProperties vs. @Value
- 29. Core Features
- 30. Internationalization
- 31. JSON

- 31.1. Jackson 3
 - 31.1.1. Custom Serializers and Deserializers
 - 31.1.2. Mixins
- 31.2. Jackson 2
- 31.3. Gson
- 31.4. JSON-B
- 31.5. Kotlin Serialization
- 32. Kotlin Support
 - 32.1. Requirements
 - 32.2. Null-safety
 - 32.3. Kotlin API
 - 32.3.1. runApplication
 - 32.3.2. Extensions
 - 32.4. Dependency Management
 - 32.5. @ConfigurationProperties
 - 32.6. Testing
 - 32.7. Resources
 - 32.7.1. Further Reading
 - 32.7.2. Examples
- 33. Logging
 - 33.1. Log Format
 - 33.2. Console Output
 - 33.2.1. Color-coded Output
 - 33.3. File Output
 - 33.4. File Rotation
 - 33.5. Log Levels
 - 33.6. Log Groups
 - 33.7. Using a Log Shutdown Hook
 - 33.8. Custom Log Configuration
 - 33.9. Structured Logging
 - 33.9.1. Elastic Common Schema
 - 33.9.2. Graylog Extended Log Format (GELF)
 - 33.9.3. Logstash JSON format
 - 33.9.4. Customizing Structured Logging JSON
 - 33.9.5. Customizing Structured Logging Stack Traces
 - 33.9.6. Supporting Other Structured Logging Formats
 - 33.10. Logback Extensions
 - 33.10.1. Profile-specific Configuration
 - 33.10.2. Environment Properties
 - 33.11. Log4j2 Extensions
 - 33.11.1. Profile-specific Configuration
 - 33.11.2. Environment Properties Lookup
 - 33.11.3. Log4j2 System Properties
- 34. Profiles
 - 34.1. Adding Active Profiles
 - 34.2. Profile Groups
 - 34.3. Programmatically Setting Profiles
 - 34.4. Profile-specific Configuration Files
- 35. SpringApplication
 - 35.1. Startup Failure
 - 35.2. Lazy Initialization
 - 35.3. Customizing the Banner
 - 35.4. Customizing SpringApplication
 - 35.5. Fluent Builder API
 - 35.6. Application Availability
 - 35.6.1. Liveness State
 - 35.6.2. Readiness State
 - 35.6.3. Managing the Application Availability State
 - 35.7. Application Events and Listeners
 - 35.8. Web Environment
 - 35.9. Accessing Application Arguments
 - 35.10. Using the ApplicationRunner or CommandLineRunner
 - 35.11. Application Exit
 - 35.12. Admin Features
 - 35.13. Application Startup tracking
 - 35.14. Virtual threads
- 36. SSL

- 36.1. Configuring SSL With Java KeyStore Files
- 36.2. Configuring SSL With PEM-encoded Certificates
- 36.3. Applying SSL Bundles
- 36.4. Using SSL Bundles
- 36.5. Reloading SSL bundles
- 37. Task Execution and Scheduling
- 38. Reference
- 39. Caching
 - 39.1. Supported Cache Providers
 - 39.1.1. Generic
 - 39.1.2. JCache (JSR-107)
 - 39.1.3. Hazelcast
 - 39.1.4. Infinispan
 - 39.1.5. Couchbase
 - 39.1.6. Redis
 - 39.1.7. Caffeine
 - 39.1.8. Cache2k
 - 39.1.9. Simple
 - 39.1.10. None
 - 39.2. Testing
- 40. Sending Email
- 41. Hazelcast
- 42. IO
- 43. Distributed Transactions With JTA
 - 43.1. Using a Jakarta EE Managed Transaction Manager
 - 43.2. Mixing XA and Non-XA JMS Connections
 - 43.3. Supporting an Embedded Transaction Manager
- 44. Quartz Scheduler
- 45. Calling REST Services
 - 45.1. WebClient
 - 45.1.1. WebClient Runtime
 - 45.1.2. Global HTTP Connector Configuration
 - 45.1.3. WebClient Customization
 - 45.1.4. WebClient SSL Support
 - 45.2. RestClient
 - 45.2.1. RestClient Customization
 - 45.2.2. RestClient SSL Support
 - 45.3. RestTemplate
 - 45.3.1. RestTemplate Customization
 - 45.3.2. RestTemplate SSL Support
 - 45.4. HTTP Client Detection for RestClient and RestTemplate
 - 45.4.1. Global HTTP Client Configuration
 - 45.5. API Versioning
 - 45.6. HTTP Service Interface Clients
 - 45.6.1. Importing HTTP Services
 - 45.6.2. Service Client Groups
 - 45.6.3. Configuration Properties
 - 45.6.4. Customization
 - 45.6.5. Advanced Configuration
 - 45.7. Applying Global Configuration to All HTTP Clients
- 46. Spring Batch
 - 46.1. Running Spring Batch Jobs on Startup
- 47. Validation
- 48. Web Services
 - 48.1. Calling Web Services with WebServiceTemplate
- 49. AMQP
 - 49.1. RabbitMQ Support
 - 49.2. Sending a Message
 - 49.3. Sending a Message To A Stream
 - 49.4. Receiving a Message
- 50. Messaging
- 51. JMS
 - 51.1. ActiveMQ "Classic" Support
 - 51.2. ActiveMQ Artemis Support
 - 51.3. Using a JNDI ConnectionFactory
 - 51.4. Sending a Message
 - 51.5. Receiving a Message

- 52. Apache Kafka Support
 - 52.1. Sending a Message
 - 52.2. Receiving a Message
 - 52.3. Kafka Streams
 - 52.4. Additional Kafka Properties
 - 52.5. Testing with Embedded Kafka
- 53. Apache Pulsar Support
 - 53.1. Connecting to Pulsar
 - 53.1.1. Authentication
 - 53.1.2. SSL
 - 53.2. Connecting to Pulsar Administration
 - 53.2.1. Authentication
 - 53.3. Sending a Message
 - 53.4. Receiving a Message
 - 53.5. Reading a Message
 - 53.6. Transaction Support
 - 53.7. Additional Pulsar Properties
- 54. RSocket
 - 54.1. RSocket Strategies Auto-configuration
 - 54.2. RSocket Server Auto-configuration
 - 54.3. Spring Messaging RSocket Support
 - 54.4. Calling RSocket Services with RSocketRequester
- 55. Spring Integration
- 56. WebSockets
- 57. AOT Cache
 - 57.1. AOT Cache
 - 57.2. CDS
- 58. Ahead-of-Time Processing With the JVM
- 59. Checkpoint and Restore With the JVM
- 60. Cloud Native Buildpacks
- 61. Dockerfiles
 - 61.1. AOT Cache
 - 61.2. CDS
- 62. Efficient Container Images
 - 62.1. Layering Docker Images
- 63. Container Images
- 64. Efficient Deployments
 - 64.1. Unpacking the Executable jar
- 65. Packaging Spring Boot Applications
- 66. Advanced Native Images Topics
 - 66.1. Nested Configuration Properties
 - 66.2. Converting a Spring Boot Executable Jar
 - 66.2.1. Using Buildpacks
 - 66.2.2. Using GraalVM native-image
 - 66.3. Using the Tracing Agent
 - 66.3.1. Launch the Application Directly
 - 66.4. Custom Hints
 - 66.4.1. Testing Custom Hints
 - 66.4.2. Providing Hints Statically
 - 66.5. Known Limitations
- 67. GraalVM Native Images
- 68. Introducing GraalVM Native Images
 - 68.1. Key Differences with JVM Deployments
 - 68.2. Understanding Spring Ahead-of-Time Processing
 - 68.2.1. Source Code Generation
 - 68.2.2. Hint File Generation
 - 68.2.3. Proxy Class Generation
- 69. Testing
- 70. Testing Spring Applications
- 71. Testing Spring Boot Applications
 - 71.1. Detecting Web Application Type
 - 71.2. Detecting Test Configuration
 - 71.3. Using the Test Configuration Main Method
 - 71.4. Excluding Test Configuration
 - 71.5. Using Application Arguments
 - 71.6. Testing With a Mock Environment
 - 71.7. Testing With a Running Server

- 71.8. Customizing RestTestClient
- 71.9. Customizing WebTestClient
- 71.10. Using JMX
- 71.11. Using Observations
- 71.12. Using Metrics
- 71.13. Using Tracing
- 71.14. Mocking and Spying Beans
- 71.15. Auto-configured Tests
- 71.16. Auto-configured JSON Tests
- 71.17. Auto-configured Spring MVC Tests
- 71.18. Auto-configured Spring WebFlux Tests
- 71.19. Auto-configured Spring GraphQL Tests
- 71.20. Auto-configured Data Cassandra Tests
- 71.21. Auto-configured Data Couchbase Tests
- 71.22. Auto-configured Data Elasticsearch Tests
- 71.23. Auto-configured Data JPA Tests
- 71.24. Auto-configured JDBC Tests
- 71.25. Auto-configured Data JDBC Tests
- 71.26. Auto-configured Data R2DBC Tests
- 71.27. Auto-configured jOOQ Tests
- 71.28. Auto-configured Data MongoDB Tests
- 71.29. Auto-configured Data Neo4j Tests
- 71.30. Auto-configured Data Redis Tests
- 71.31. Auto-configured Data LDAP Tests
- 71.32. Auto-configured REST Clients
- 71.33. Auto-configured Web Clients
- 71.34. Auto-configured Spring REST Docs Tests
 - 71.34.1. Auto-configured Spring REST Docs Tests With Mock MVC
 - 71.34.2. Auto-configured Spring REST Docs Tests With WebTestClient
- 71.35. Auto-configured Spring Web Services Tests
 - 71.35.1. Auto-configured Spring Web Services Client Tests
 - 71.35.2. Auto-configured Spring Web Services Server Tests
- 71.36. Additional Auto-configuration and Slicing
- 71.37. User Configuration and Slicing
- 71.38. Using Spock to Test Spring Boot Applications
- 72. Test Modules
- 73. Test Scope Dependencies
- 74. Test Utilities
 - 74.1. ConfigDataApplicationContextInitializer
 - 74.2. TestPropertyValues
 - 74.3. OutputCaptureExtension
 - 74.4. TestRestTemplate
- 75. Testcontainers
 - 75.1. Using Spring Beans
 - 75.2. Using the JUnit Extension
 - 75.3. Importing Container Configuration Interfaces
 - 75.4. Lifecycle of Managed Containers
 - 75.5. Service Connections
 - 75.5.1. SSL with Service Connections
 - 75.6. Dynamic Properties
- 76. Auto-configuration
 - 76.1. Gradually Replacing Auto-configuration
 - 76.2. Disabling Specific Auto-configuration Classes
 - 76.3. Auto-configuration Packages
- 77. Build Systems
 - 77.1. Dependency Management
 - 77.2. Maven
 - 77.3. Gradle
 - 77.4. Ant
 - 77.5. Starters
- 78. Configuration Classes
 - 78.1. Importing Additional Configuration Classes
 - 78.2. Importing XML Configuration
- 79. Developer Tools
 - 79.1. Diagnosing Classloading Issues
 - 79.2. Property Defaults
 - 79.3. Automatic Restart

- 79.3.1. Logging Changes in Condition Evaluation
- 79.3.2. Excluding Resources
- 79.3.3. Watching Additional Paths
- 79.3.4. Disabling Restart
- 79.3.5. Using a Trigger File
- 79.3.6. Customizing the Restart Classloader
- 79.3.7. Known Limitations
- 79.4. LiveReload
- 79.5. Global Settings
 - 79.5.1. Configuring File System Watcher
- 79.6. Remote Applications
 - 79.6.1. Running the Remote Client Application
 - 79.6.2. Remote Update
- 80. Developing with Spring Boot
- 81. Packaging Your Application for Production
- 82. Running Your Application
 - 82.1. Running From an IDE
 - 82.2. Running as a Packaged Application
 - 82.3. Using the Maven Plugin
 - 82.4. Using the Gradle Plugin
 - 82.5. Hot Swapping
- 83. Spring Beans and Dependency Injection
- 84. Structuring Your Code
 - 84.1. Using the “default” Package
 - 84.2. Locating the Main Application Class
- 85. Using the @SpringBootApplication Annotation
- 86. Graceful Shutdown
 - 86.1. Rejecting Requests During the Grace Period
 - 86.2. Disabling Graceful Shutdown
- 87. Web
- 88. Reactive Web Applications
 - 88.1. The “Spring WebFlux Framework”
 - 88.1.1. Spring WebFlux Auto-configuration
 - 88.1.2. Spring WebFlux Conversion Service
 - 88.1.3. HTTP Codecs with HttpMessageReaders and HttpMessageWriters
 - 88.1.4. Static Content
 - 88.1.5. Welcome Page
 - 88.1.6. Template Engines
 - 88.1.7. Error Handling
 - Custom Error Pages
 - 88.1.8. Web Filters
 - 88.1.9. API Versioning
 - 88.2. Embedded Reactive Server Support
 - 88.2.1. Customizing Reactive Servers
 - Programmatic Customization
 - Customizing ConfigurableReactiveWebServerFactory Directly
 - 88.3. Reactive Server Resources Configuration
- 89. Servlet Web Applications
 - 89.1. The “Spring Web MVC Framework”
 - 89.1.1. Spring MVC Auto-configuration
 - 89.1.2. Spring MVC Conversion Service
 - 89.1.3. HttpMessageConverters
 - 89.1.4. MessageCodesResolver
 - 89.1.5. Static Content
 - 89.1.6. Welcome Page
 - 89.1.7. Custom Favicon
 - 89.1.8. Path Matching and Content Negotiation
 - 89.1.9. ConfigurableWebBindingInitializer
 - 89.1.10. Template Engines
 - 89.1.11. Error Handling
 - Custom Error Pages
 - Mapping Error Pages Outside of Spring MVC
 - Error Handling in a WAR Deployment
 - 89.1.12. CORS Support
 - 89.1.13. API Versioning
 - 89.2. JAX-RS and Jersey
 - 89.3. Embedded Servlet Container Support

- 89.3.1. Servlets, Filters, and Listeners
 - Registering Servlets, Filters, and Listeners as Spring Beans
- 89.3.2. Servlet Context Initialization
 - Init Parameters
 - Scanning for Servlets, Filters, and listeners
- 89.3.3. The ServletWebServerApplicationContext
- 89.3.4. Customizing Embedded Servlet Containers
 - SameSite Cookies
 - Character Encoding
 - Programmatic Customization
 - Customizing ConfigurableServletWebServerFactory Directly
- 89.3.5. JSP Limitations
- 90. Spring for GraphQL
 - 90.1. GraphQL Schema
 - 90.2. GraphQL RuntimeWiring
 - 90.3. Querydsl and QueryByExample Repositories Support
 - 90.4. Transports
 - 90.4.1. HTTP and WebSocket
 - 90.4.2. RSocket
 - 90.5. Exception Handling
 - 90.6. GraphQL and Schema Printer
- 91. Spring HATEOAS
- 92. Spring Security
 - 92.1. MVC Security
 - 92.2. WebFlux Security
 - 92.3. OAuth2
 - 92.3.1. Client
 - OAuth2 Client Registration for Common Providers
 - 92.3.2. Resource Server
 - 92.3.3. Authorization Server
 - 92.4. SAML 2.0
 - 92.4.1. Relying Party
- 93. Spring Session
- 94. How-To Guides
- 95. Actuator
 - 95.1. Change the HTTP Port or Address of the Actuator Endpoints
 - 95.2. Customizing Sanitization
 - 95.3. Map Health Indicators to Micrometer Metrics
- 96. AOT Cache
 - 96.1. Packaging an Application Using AOT Cache and Buildpacks
 - 96.2. Packaging an Application Using AOT Cache and Dockerfiles
 - 96.3. Preventing Remote Services Interaction During the Training Run
- 97. Ahead-of-Time Processing
 - 97.1. Conditions
- 98. Spring Boot Application
 - 98.1. Create Your Own FailureAnalyzer
 - 98.2. Troubleshoot Auto-configuration
 - 98.3. Customize the Environment or ApplicationContext Before It Starts
 - 98.4. Build an ApplicationContext Hierarchy (Adding a Parent or Root Context)
 - 98.5. Create a Non-web Application
- 99. Batch Applications
 - 99.1. Specifying a Batch Data Source
 - 99.2. Specifying a Batch Transaction Manager
 - 99.3. Specifying a Batch Task Executor
 - 99.4. Running From the Command Line
 - 99.5. Restarting a Stopped or Failed Job
- 100. Build
 - 100.1. Generate Build Information
 - 100.2. Generate Git Information
 - 100.3. Generate a CycloneDX SBOM
 - 100.4. Customize Dependency Versions
 - 100.5. Create an Executable JAR with Maven
 - 100.6. Use a Spring Boot Application as a Dependency
 - 100.7. Extract Specific Libraries When an Executable Jar Runs
 - 100.8. Create a Non-executable JAR with Exclusions
 - 100.9. Remote Debug a Spring Boot Application Started with Maven
 - 100.10. Build an Executable Archive From Ant without Using spring-boot-antlib

- 101. Data Access
 - 101.1. Configure a Custom DataSource
 - 101.2. Configure Two DataSources
 - 101.3. Use Spring Data Repositories
 - 101.4. Separate @Entity Definitions from Spring Configuration
 - 101.5. Filter Scanned @Entity Definitions
 - 101.6. Configure JPA Properties
 - 101.7. Configure Hibernate Naming Strategy
 - 101.8. Configure Hibernate Second-Level Caching
 - 101.9. Use Dependency Injection in Hibernate Components
 - 101.10. Use a Custom EntityManagerFactory
 - 101.11. Using Multiple EntityManagerFactories
 - 101.12. Use a Traditional persistence.xml File
 - 101.13. Use Spring Data JPA and Mongo Repositories
 - 101.14. Customize Spring Data's Web Support
 - 101.15. Expose Spring Data Repositories as REST Endpoint
 - 101.16. Configure a Component that is Used by JPA
 - 101.17. Configure jOOQ with Two DataSources
- 102. Database Initialization
 - 102.1. Initialize a Database Using Hibernate
 - 102.2. Initialize a Database Using Basic SQL Scripts
 - 102.3. Initialize a Spring Batch Database
 - 102.4. Use a Higher-level Database Migration Tool
 - 102.4.1. Execute Flyway Database Migrations on Startup
 - 102.4.2. Execute Liquibase Database Migrations on Startup
 - 102.4.3. Use Flyway for Test-only Migrations
 - 102.4.4. Use Liquibase for Test-only Migrations
 - 102.5. Depend Upon an Initialized Database
 - 102.5.1. Detect a Database Initializer
 - 102.5.2. Detect a Bean That Depends On Database Initialization
- 103. Deploying to the Cloud
 - 103.1. Cloud Foundry
 - 103.1.1. Binding to Services
 - 103.2. Kubernetes
 - 103.2.1. Kubernetes Container Lifecycle
 - 103.3. Heroku
 - 103.4. OpenShift
 - 103.5. Amazon Web Services (AWS)
 - 103.5.1. Amazon Elastic Container Service (ECS)
 - 103.5.2. AWS Elastic Beanstalk
 - Using the Tomcat Platform
 - Using the Java SE Platform
 - 103.6. CloudCaptain and Amazon Web Services
 - 103.7. Azure
 - 103.8. Google Cloud
- 104. Deploying Spring Boot Applications
- 105. Installing Spring Boot Applications
 - 105.1. Installation as a systemd Service
 - 105.2. Microsoft Windows Services
- 106. Traditional Deployment
 - 106.1. Create a Deployable War File
 - 106.2. Convert an Existing Application to Spring Boot
 - 106.3. Deploying a WAR to WebLogic
- 107. Docker Compose
 - 107.1. Customizing the JDBC URL
 - 107.2. Sharing Services Between Multiple Applications
- 108. Hot Swapping
 - 108.1. Reload Static Content
 - 108.2. Reload Templates without Restarting the Container
 - 108.2.1. Thymeleaf Templates
 - 108.2.2. FreeMarker Templates
 - 108.2.3. Groovy Templates
 - 108.3. Fast Application Restarts
 - 108.4. Reload Java Classes without Restarting the Container
- 109. HTTP Clients
 - 109.1. Configure RestTemplate to Use a Proxy
 - 109.2. Configure the TcpClient used by a Reactor Netty-based WebClient

- 110. How-to Guides
- 111. Jersey
 - 111.1. Secure Jersey Endpoints with Spring Security
 - 111.2. Use Jersey Alongside Another Web Framework
- 112. Logging
 - 112.1. Configure Logback for Logging
 - 112.1.1. Configure Logback for File-only Output
 - 112.2. Configure Log4j for Logging
 - 112.2.1. Use YAML or JSON to Configure Log4j 2
 - 112.2.2. Use Composite Configuration to Configure Log4j 2
- 113. Messaging
 - 113.1. Disable Transacted JMS Session
- 114. Developing Your First GraalVM Native Application
 - 114.1. Sample Application
 - 114.2. Building a Native Image Using Buildpacks
 - 114.2.1. System Requirements
 - 114.2.2. Using Maven
 - 114.2.3. Using Gradle
 - 114.2.4. Running the example
 - 114.3. Building a Native Image using Native Build Tools
 - 114.3.1. Prerequisites
 - Linux and macOS
 - Windows
 - 114.3.2. Using Maven
 - 114.3.3. Using Gradle
 - 114.3.4. Running the Example
- 115. GraalVM Native Applications
- 116. Testing GraalVM Native Images
 - 116.1. Testing Ahead-of-Time Processing With the JVM
 - 116.2. Testing With Native Build Tools
 - 116.2.1. Using Maven
 - 116.2.2. Using Gradle
- 117. NoSQL
 - 117.1. Use Jedis Instead of Lettuce
- 118. Properties and Configuration
 - 118.1. Automatically Expand Properties at Build Time
 - 118.1.1. Automatic Property Expansion Using Maven
 - 118.1.2. Automatic Property Expansion Using Gradle
 - 118.2. Externalize the Configuration of SpringApplication
 - 118.3. Change the Location of External Properties of an Application
 - 118.4. Use 'Short' Command Line Arguments
 - 118.5. Use YAML for External Properties
 - 118.6. Set the Active Spring Profiles
 - 118.7. Set the Default Profile Name
 - 118.8. Change Configuration Depending on the Environment
 - 118.9. Discover Built-in Options for External Properties
- 119. Security
 - 119.1. Switch Off the Spring Boot Security Configuration
 - 119.2. Change the UserDetailsService and Add User Accounts
 - 119.3. Enable HTTPS When Running Behind a Proxy Server
- 120. Spring MVC
 - 120.1. Write a JSON REST Service
 - 120.2. Write an XML REST Service
 - 120.3. Customize the Jackson JsonMapper
 - 120.4. Customize the @ResponseBody Rendering
 - 120.5. Handling Multipart File Uploads
 - 120.6. Switch Off the Spring MVC DispatcherServlet
 - 120.7. Switch Off the Default MVC Configuration
 - 120.8. Customize ViewResolvers
 - 120.9. Customize the 'whitelabel' Error Page
- 121. Testing
 - 121.1. Testing With Spring Security
 - 121.2. Structure javadoc:org.springframework.context.annotation.Configuration[format=annotation] Classes for Inclusion in Slice Tests
- 122. Embedded Web Servers
 - 122.1. Use Another Web Server
 - 122.2. Disabling the Web Server
 - 122.3. Change the HTTP Port

- 122.4. Use a Random Unassigned HTTP Port
- 122.5. Discover the HTTP Port at Runtime
- 122.6. Build URI Against the Test Web Server
- 122.7. Enable HTTP Response Compression
- 122.8. Configure SSL
 - 122.8.1. Using PEM-encoded files
 - 122.8.2. Using SSL Bundles
 - 122.8.3. Configure Server Name Indication
- 122.9. Configure HTTP/2
 - 122.9.1. HTTP/2 With Tomcat
 - 122.9.2. HTTP/2 With Jetty
 - 122.9.3. HTTP/2 With Reactor Netty
- 122.10. Configure the Web Server
- 122.11. Add a Servlet, Filter, or Listener to an Application
 - 122.11.1. Add a Servlet, Filter, or Listener by Using a Spring Bean
 - Disable Registration of a Servlet or Filter
 - 122.11.2. Add Servlets, Filters, and Listeners by Using Classpath Scanning
- 122.12. Configure Access Logging
- 122.13. Running Behind a Front-end Proxy Server
 - 122.13.1. Customize Tomcat's Proxy Configuration
- 122.14. Enable Multiple Connectors with Tomcat
- 122.15. Enable Tomcat's MBean Registry
- 122.16. Create WebSocket Endpoints Using @ServerEndpoint
- 123. Build Tool Plugins
- 124. Spring Boot AntLib Module
 - 124.1. Spring Boot Ant Tasks
 - 124.1.1. Using the "exejar" Task
 - 124.1.2. Examples
 - 124.2. Using the "findmainclass" Task
 - 124.2.1. Examples
- 125. Build Tool Plugins
- 126. Supporting Other Build Systems
 - 126.1. Repackaging Archives
 - 126.2. Nested Libraries
 - 126.3. Finding a Main Class
 - 126.4. Example Repackage Implementation
- 127. Spring Boot CLI
- 128. Spring Boot CLI
- 129. Installing the CLI
- 130. Using the CLI
 - 130.1. Initialize a New Project
 - 130.2. Using the Embedded Shell
- 131. Appendix
 - Appendix A: Common Application Properties
 - Appendix B: Auto-configuration Classes
 - 132. Managed Dependency Coordinates
 - Appendix C: Dependency Versions
 - 133. Version Properties
 - Appendix D: Deprecated Application Properties
 - Appendix E: Test Auto-configuration Annotations
- 134. Test Slices

1. Getting Started

2. Installing Spring Boot

Spring Boot can be used with “classic” Java development tools or installed as a command line tool. Either way, you need [Java SDK v17](https://www.java.com) (https://www.java.com) or higher. Before you begin, you should check your current Java installation by using the following command:

```
$ java -version
```

SHELL

If you are new to Java development or if you want to experiment with Spring Boot, you might want to try the [Spring Boot CLI](#) (Command Line Interface) first. Otherwise, read on for “classic” installation instructions.

2.1. Installation Instructions for the Java Developer

You can use Spring Boot in the same way as any standard Java library. To do so, include the appropriate `spring-boot-*.jar` files on your classpath. Spring Boot does not require any special tools integration, so you can use any IDE or text editor. Also, there is nothing special about a Spring Boot application, so you can run and debug a Spring Boot application as you would any other Java program.

Although you *could* copy Spring Boot jars, we generally recommend that you use a build tool that supports dependency management (such as Maven or Gradle).

2.1.1. Maven Installation

Spring Boot is compatible with Apache Maven 3.6.3 or later. If you do not already have Maven installed, you can follow the instructions at maven.apache.org.



On many operating systems, Maven can be installed with a package manager. If you use OSX Homebrew, try `brew install maven`. Ubuntu users can run `sudo apt-get install maven`. Windows users with [Chocolatey](https://chocolatey.org/) (https://chocolatey.org/) can run `choco install maven` from an elevated (administrator) prompt.

Spring Boot dependencies use the `org.springframework.boot` group id. Typically, your Maven POM file inherits from the `spring-boot-starter-parent` project and declares dependencies to one or more [starters](#). Spring Boot also provides an optional [Maven plugin](#) to create executable jars.

More details on getting started with Spring Boot and Maven can be found in the [maven-plugin:getting-started.html](#) section of the Maven plugin’s reference guide.

2.1.2. Gradle Installation

Spring Boot is compatible with Gradle 8.x (8.14 or later) or 9.x. If you do not already have Gradle installed, you can follow the instructions at gradle.org.

Spring Boot dependencies can be declared by using the `org.springframework.boot` group. Typically, your project declares dependencies to one or more [starters](#). Spring Boot provides a useful [Gradle plugin](#) that can be used to simplify dependency declarations and to create executable jars.

Gradle Wrapper

The Gradle Wrapper provides a nice way of “obtaining” Gradle when you need to build a project. It is a small script and library that you commit alongside your code to bootstrap the build process. See [{url-gradle-docs}/gradle_wrapper.html](#) for details.

More details on getting started with Spring Boot and Gradle can be found in the [gradle-plugin:getting-started.html](#) section of the Gradle plugin’s reference guide.

2.2. Installing the Spring Boot CLI

The Spring Boot CLI (Command Line Interface) is a command line tool that you can use to quickly prototype with Spring.

You do not need to use the CLI to work with Spring Boot, but it is a quick way to get a Spring application off the ground without an IDE.

2.2.1. Manual Installation

You can download the Spring CLI distribution from one of the following locations:

- `{url-artifact-repository}/org/springframework/boot/spring-boot-cli/{version-spring-boot}/spring-boot-cli-{version-spring-boot}-bin.zip[spring-boot-cli-{version-spring-boot}-bin.zip]`
- `{url-artifact-repository}/org/springframework/boot/spring-boot-cli/{version-spring-boot}/spring-boot-cli-{version-spring-boot}-bin.tar.gz[spring-boot-cli-{version-spring-boot}-bin.tar.gz]`

Once downloaded, follow the `{url-github-raw}/cli/spring-boot-tools/spring-boot-cli/src/main/content/INSTALL.txt[INSTALL.txt]` instructions from the unpacked archive. In summary, there is a `spring` script (`spring.bat` for Windows) in a `bin/` directory in the `.zip` file. Alternatively, you can use `java -jar` with the `.jar` file (the script helps you to be sure that the classpath is set correctly).

2.2.2. Installation with SDKMAN!

SDKMAN! (The Software Development Kit Manager) can be used for managing multiple versions of various binary SDKs, including Groovy and the Spring Boot CLI. Get SDKMAN! from sdkman.io and install Spring Boot by using the following commands:

```
$ sdk install springboot
$ spring --version
Spring CLI v{version-spring-boot}
```

SHELL

If you develop features for the CLI and want access to the version you built, use the following commands:

```
$ sdk install springboot dev /path/to/spring-boot/spring-boot-cli/target/spring-boot-cli-{version-spring-boot}-bin/spring-{version-spring-boot}/
$ sdk default springboot dev
$ spring --version
Spring CLI v{version-spring-boot}
```

SHELL

The preceding instructions install a local instance of `spring` called the `dev` instance. It points at your target build location, so every time you rebuild Spring Boot, `spring` is up-to-date.

You can see it by running the following command:

```
$ sdk ls springboot
```

SHELL

```
=====
Available Springboot Versions
=====
> + dev
* {version-spring-boot}

=====
+ - local version
* - installed
> - currently in use
=====
```

2.2.3. OSX Homebrew Installation

If you are on a Mac and use [Homebrew](https://brew.sh/) (https://brew.sh/), you can install the Spring Boot CLI by using the following commands:

```
$ brew tap spring-io/tap
$ brew install spring-boot
```

SHELL

Homebrew installs `spring` to `/usr/local/bin`.



If you do not see the formula, your installation of brew might be out-of-date. In that case, run `brew update` and try again.

2.2.4. MacPorts Installation

If you are on a Mac and use [MacPorts](https://www.macports.org/) (https://www.macports.org/), you can install the Spring Boot CLI by using the following command:

```
$ sudo port install spring-boot-cli
```

SHELL

2.2.5. Command-line Completion

The Spring Boot CLI includes scripts that provide command completion for the [BASH](https://en.wikipedia.org/wiki/Bash_%28Unix_shell%29) (https://en.wikipedia.org/wiki/Bash_%28Unix_shell%29) and [zsh](https://en.wikipedia.org/wiki/Z_shell) (https://en.wikipedia.org/wiki/Z_shell) shells. You can `source` the script named `spring` (`_spring` for `zsh`) or put it in your personal or system-wide bash completion initialization. On a Debian system, the system-wide scripts are in `<installation location>/shell-completion/<bash|zsh>` and all scripts in that directory are executed when a new shell starts. For example, to run the script manually if you have installed by using SDKMAN!, use the following commands:

```
$ . ~/.sdkman/candidates/springboot/current/shell-completion/bash/spring
$ spring <HIT TAB HERE>
  encodepassword help init shell version
```

SHELL



If you install the Spring Boot CLI by using Homebrew or MacPorts, the command-line completion scripts are automatically registered with your shell.

2.2.6. Windows Scoop Installation

If you are on a Windows and use [Scoop](https://scoop.sh/) (https://scoop.sh/), you can install the Spring Boot CLI by using the following commands:

```
$ scoop bucket add extras
$ scoop install springboot
```

SHELL

Scoop installs `spring` to `~\scoop\apps\springboot\current\bin`.



If you do not see the app manifest, your installation of scoop might be out-of-date. In that case, run `scoop update` and try again.

3. System Requirements

Spring Boot {version-spring-boot} requires at least [Java 17](https://www.java.com) (https://www.java.com) and is compatible with versions up to and including Java 25. {url-spring-framework-docs}/[Spring Framework {version-spring-framework}] or above is also required.

Support for third-party projects may have additional or higher requirements. Refer to the documentation for that support for further details.

Explicit build support is provided for the following build tools:

Build Tool	Version
Maven	3.6.3 or later
Gradle	Gradle 8.x (8.14 or later) and 9.x

3.1. Servlet Containers

Spring Boot supports the following embedded servlet containers:

Name	Servlet Version
Tomcat 11.0.x	6.1
Jetty 12.1.x	6.1

You can also deploy Spring Boot applications to any Servlet 6.1+ compatible container.

3.2. GraalVM Native Images

Spring Boot applications can be [converted into a Native Image](#) using GraalVM {version-graal} or above.

Images can be created using the [native build tools](https://github.com/graalvm/native-build-tools) (https://github.com/graalvm/native-build-tools) Gradle/Maven plugins or `native-image` tool provided by GraalVM. You can also create native images using the [native-image Paketo buildpack](https://github.com/paketo-buildpacks/native-image) (https://github.com/paketo-buildpacks/native-image).

The following versions are supported:

Name	Version
GraalVM Community	{version-graal}
Native Build Tools	{version-native-build-tools}

4. Upgrading Spring Boot

Instructions for how to upgrade from earlier versions of Spring Boot are provided on the project [{url-github-wiki}\[wiki\]](#). Follow the links in the [{url-github-wiki}#release-notes\[release notes\]](#) section to find the version that you want to upgrade to.

Upgrading instructions are always the first item in the release notes. If you are more than one release behind, please make sure that you also review the release notes of the versions that you jumped.

4.1. Upgrading From 1.x

If you are upgrading from the 1.x release of Spring Boot, check the [{url-github-wiki}/Spring-Boot-2.0-Migration-Guide\[migration guide\]](#) on the project wiki that provides detailed upgrade instructions to upgrade to Spring Boot 2.x. Check also the [{url-github-wiki}#release-notes\[release notes\]](#) for a list of “new and noteworthy” features for each release.

4.2. Upgrading From 2.x

If you are upgrading from the 2.x release of Spring Boot, check the [{url-github-wiki}/Spring-Boot-3.0-Migration-Guide\[migration guide\]](#) on the project wiki that provides detailed upgrade instructions. Check also the [{url-github-wiki}#release-notes\[release notes\]](#) for a list of “new and noteworthy” features for each release.

4.3. Upgrading From 3.x

If you are upgrading from the 3.x release of Spring Boot, check the [{url-github-wiki}/Spring-Boot-4.0-Migration-Guide\[migration guide\]](#) on the project wiki that provides detailed upgrade instructions. Check also the [{url-github-wiki}#release-notes\[release notes\]](#) for a list of “new and noteworthy” features for each release.

4.4. Upgrading to a New Feature Release

When upgrading to a new feature release, some properties may have been renamed or removed. Spring Boot provides a way to analyze your application’s environment and print diagnostics at startup, but also temporarily migrate properties at runtime for you. To enable that feature, add the following dependency to your project:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-properties-migrator</artifactId>
  <scope>runtime</scope>
</dependency>
```

XML



Properties that are added late to the environment, such as when using `javaDoc:org.springframework.context.annotation.PropertySource[format=annotation]`, will not be taken into account.



Once you finish the migration, please make sure to remove this module from your project’s dependencies.

4.5. Upgrading the Spring Boot CLI

To upgrade an existing CLI installation, use the appropriate package manager command (for example, `brew upgrade`). If you manually installed the CLI, follow the [standard instructions](#), remembering to update your `PATH` environment variable to remove any older references.

6. Developing Your First Spring Boot Application

This section describes how to develop a small “Hello World!” web application that highlights some of Spring Boot’s key features. You can choose between Maven or Gradle as the build system.



The spring.io (<https://spring.io>) website contains many “Getting Started” [guides](https://spring.io/guides) (<https://spring.io/guides>) that use Spring Boot. If you need to solve a specific problem, check there first.

You can shortcut the steps below by going to start.spring.io and choosing the “Web” starter from the dependencies searcher. Doing so generates a new project structure so that you can [start coding right away](#). Check the [start.spring.io user guide](https://github.com/spring-io/start.spring.io/blob/main/USING.adoc) (<https://github.com/spring-io/start.spring.io/blob/main/USING.adoc>) for more details.

6.1. Prerequisites

Before we begin, open a terminal and run the following commands to ensure that you have a valid version of Java installed:

```
$ java -version
openjdk version "17.0.16" 2025-07-15 LTS
OpenJDK Runtime Environment (build 17.0.16+12-LTS)
OpenJDK 64-Bit Server VM (build 17.0.16+12-LTS, mixed mode, sharing)
```

SHELL



This sample needs to be created in its own directory. Subsequent instructions assume that you have created a suitable directory and that it is your current directory.

6.1.1. Maven

If you want to use Maven, ensure that you have Maven installed:

```
$ mvn -v
Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)
Maven home: /Users/developer/.sdkman/candidates/maven/current
Java version: 17.0.16, vendor: BellSoft, runtime: /Users/developer/.sdkman/candidates/java/17.0.16-librca
```

SHELL

6.1.2. Gradle

If you want to use Gradle, ensure that you have Gradle installed:

```
$ gradle --version

-----
Gradle 8.14.3
-----

Build time: 2025-07-04 13:15:44 UTC
Revision: e5ee1df3d88b8ca3a8074787a94f373e3090e1db

Kotlin: 2.0.21
Groovy: 3.0.24
Ant: Apache Ant(TM) version 1.10.15 compiled on August 25 2024
Launcher JVM: 17.0.16 (BellSoft 17.0.16+12-LTS)
Daemon JVM: /Users/developer/.sdkman/candidates/java/17.0.16-librca (no JDK specified, using current Java home)
OS: Mac OS X 15.7.1 aarch64
```

SHELL

6.2. Setting Up the Project With Maven

We need to start by creating a Maven `pom.xml` file. The `pom.xml` is the recipe that is used to build your project. Open your favorite text editor and add the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>myproject</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>{version-spring-boot}</version>
  </parent>

  <!-- Additional lines to be added here... -->

</project>
```

XML

You can test it by running `mvn package` (for now, you can ignore the “jar will be empty - no content was marked for inclusion!” warning).



At this point, you could import the project into an IDE (most modern Java IDEs include built-in support for Maven). For simplicity, we continue to use a plain text editor for this example.

6.3. Setting Up the Project With Gradle

We need to start by creating a Gradle `build.gradle` file. The `build.gradle` is the build script that is used to build your project. Open your favorite text editor and add the following:

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '{version-spring-boot}'
}

apply plugin: 'io.spring.dependency-management'

group = 'com.example'
version = '0.0.1-SNAPSHOT'

java {
    sourceCompatibility = '17'
}

repositories {
    mavenCentral()
}

dependencies {
}
```

GRADLE

The preceding listing should give you a working build. You can test it by running `gradle classes`.



At this point, you could import the project into an IDE (most modern Java IDEs include built-in support for Gradle). For simplicity, we continue to use a plain text editor for this example.

6.4. Adding Classpath Dependencies

Spring Boot provides a number of starters that let you add jars to your classpath. Starters provide dependencies that you are likely to need when developing a specific type of application.

6.4.1. Maven

Most Spring Boot applications use the `spring-boot-starter-parent` in the `parent` section of the POM. The `spring-boot-starter-parent` is a special starter that provides useful Maven defaults. It also provides a `dependency-management` section so that you can omit `version` tags for “blessed” dependencies.

Since we are developing a web application, we add a `spring-boot-starter-web` dependency. Before that, we can look at what we currently have by running the following command:

```
$ mvn dependency:tree

[INFO] com.example:myproject:jar:0.0.1-SNAPSHOT
```

SHELL

The `mvn dependency:tree` command prints a tree representation of your project dependencies. You can see that `spring-boot-starter-parent` provides no dependencies by itself. To add the necessary dependencies, edit your `pom.xml` and add the `spring-boot-starter-web` dependency immediately below the `parent` section:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

XML

If you run `mvn dependency:tree` again, you see that there are now a number of additional dependencies, including the Tomcat web server and Spring Boot itself.

6.4.2. Gradle

Most Spring Boot applications use the `org.springframework.boot` Gradle plugin. This plugin provides useful defaults and Gradle tasks. The `io.spring.dependency-management` Gradle plugin provides `dependency management` so that you can omit `version` tags for “blessed” dependencies.

Since we are developing a web application, we add a `spring-boot-starter-web` dependency. Before that, we can look at what we currently have by running the following command:

```
$ gradle dependencies

> Task :dependencies
```

SHELL

```
-----
Root project 'myproject'
-----
```

The `gradle dependencies` command prints a tree representation of your project dependencies. Right now, the project has no dependencies. To add the necessary dependencies, edit your `build.gradle` and add the `spring-boot-starter-web` dependency in the `dependencies` section:

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
}
```

GRADLE

If you run `gradle dependencies` again, you see that there are now a number of additional dependencies, including the Tomcat web server and Spring Boot itself.

6.5. Writing the Code

To finish our application, we need to create a single Java file. By default, Maven and Gradle compile sources from `src/main/java`, so you need to create that directory structure and then add a file named `src/main/java/com/example/MyApplication.java` to contain the following code:

```
include-code::MyApplication[]
```

Although there is not much code here, quite a lot is going on. We step through the important parts in the next few sections.

6.5.1. The `@RestController` and `@RequestMapping` Annotations

The first annotation on our `MyApplication` class is `javax.annotation.processing.processor.annotation.RestController[format=annotation]`. This is known as a *stereotype* annotation. It provides hints for people reading the code and for Spring that the class plays a specific role. In this case, our class is a web `javax.annotation.processing.processor.annotation.Controller[format=annotation]`, so Spring considers it when handling incoming web requests.

The `javax.annotation.processing.processor.annotation.RequestMapping[format=annotation]` annotation provides “routing” information. It tells Spring that any HTTP request with the `/` path should be mapped to the `home` method. The `javax.annotation.processing.processor.annotation.RestController[format=annotation]` annotation tells Spring to render the resulting string directly back to the caller.



The `javax.annotation.processing.processor.annotation.RestController[format=annotation]` and `javax.annotation.processing.processor.annotation.RequestMapping[format=annotation]` annotations are Spring MVC annotations (they are not specific to Spring Boot). See the [MVC section](https://springframework.org/docs/5.3.x/web/webmvc.html) in the Spring Reference Documentation for more details.

6.5.2. The `@SpringBootApplication` Annotation

The second class-level annotation is `javax.annotation.processing.processor.annotation.SpringBootApplication[format=annotation]`. This annotation is known as a *meta-annotation*, it combines `javax.annotation.processing.processor.annotation.SpringBootConfiguration[format=annotation]`, `javax.annotation.processing.processor.annotation.EnableAutoConfiguration[format=annotation]` and `javax.annotation.processing.processor.annotation.ComponentScan[format=annotation]`.

Of those, the annotation we’re most interested in here is `javax.annotation.processing.processor.annotation.EnableAutoConfiguration[format=annotation]`. `javax.annotation.processing.processor.annotation.EnableAutoConfiguration[format=annotation]` tells Spring Boot to “guess” how you want to configure Spring, based on the jar dependencies that you have added. Since `spring-boot-starter-web` added Tomcat and Spring MVC, the auto-configuration assumes that you are developing a web application and sets up Spring accordingly.

Starters and Auto-configuration

Auto-configuration is designed to work well with starters, but the two concepts are not directly tied. You are free to pick and choose jar dependencies outside of the starters. Spring Boot still does its best to auto-configure your application.

6.5.3. The “main” Method

The final part of our application is the `main` method. This is a standard method that follows the Java convention for an application entry point. Our main method delegates to Spring Boot’s `javax.annotation.processing.processor.annotation.SpringApplication[]` class by calling `run`. `javax.annotation.processing.processor.annotation.SpringApplication[]` bootstraps our application, starting Spring, which, in turn, starts the auto-configured Tomcat web server. We need to pass `MyApplication.class` as an argument to the `run` method to tell `javax.annotation.processing.processor.annotation.SpringApplication[]` which is the primary Spring component. The `args` array is also passed through to expose any command-line arguments.

6.6. Running the Example

6.6.1. Maven

At this point, your application should work. Since you used the `spring-boot-starter-parent` POM, you have a useful `run` goal that you can use to start the application. Type `mvn spring-boot:run` from the root project directory to start the application. You should see output similar to the following:

```
$ mvn spring-boot:run
[INFO] Scanning for projects...
[INFO]
[INFO] --- spring-boot-maven-plugin:2.1.6:run (default-cli) ---
[INFO]
[INFO] Starting 1 application
[INFO]
[INFO] . ____ _ _ _ _
[INFO]  /  _ \ | | | |
[INFO] (  ___/ | | | |
[INFO] \  _/  | | | |
[INFO]  _/    | | | |
[INFO] _____|_|_|_|_|
[INFO] :: Spring Boot :: (v{version-spring-boot})
[INFO]
[INFO]
[INFO] (log output here)
[INFO]
[INFO] Started MyApplication in 0.906 seconds (process running for 6.514)
```

If you open a web browser to `localhost:8080`, you should see the following output:

```
Hello World!
```

To gracefully exit the application, press `ctrl-c`.

6.6.2. Gradle

At this point, your application should work. Since you used the `org.springframework.boot` Gradle plugin, you have a useful `bootRun` goal that you can use to start the application. Type `gradle bootRun` from the root project directory to start the application. You should see output similar to the following:

```
$ gradle bootRun
```

If you open a web browser to `localhost:8080`, you should see the following output:

Hello World!

To gracefully exit the application, press `ctrl-c`.

We finish our example by creating a completely self-contained executable jar file that we could run in production. Executable jars (sometimes called “uber jars” or “fat jars”) are archives containing your compiled classes along with all of the jar dependencies that your code needs to run.

Java does not provide a standard way to load nested jar files (jar files that are themselves contained within a jar). This can be problematic if you are looking to distribute a self-contained application.

To solve this problem, many developers use “uber” jars. An uber jar packages all the classes from all the application’s dependencies into a single archive. The problem with this approach is that it becomes hard to see which libraries are in your application. It can also be problematic if the same filename is used (but with different content) in multiple jars.

Spring Boot takes a different approach and lets you actually nest jars directly.

6.7.1. Maven

To create an executable jar, we need to add the `spring-boot-maven-plugin` to our `pom.xml`. To do so, insert the following lines just below the `dependencies` section:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

The `spring-boot-starter-parent` POM includes `<executions>` configuration to bind the `repackage` goal. If you do not use the parent POM, you need to declare this configuration yourself. See the [plugin documentation](#) for details.

Save your `pom.xml` and run `mvn package` from the command line, as follows:

```
$ mvn package
```

If you look in the `target` directory, you should see `myproject-0.0.1-SNAPSHOT.jar`. The file should be around 18 MB in size. If you want to peek inside, you can use `jar tvf`, as follows:

```
$ jar tvf target/myproject-0.0.1-SNAPSHOT.jar
```

You should also see a much smaller file named `myproject-0.0.1-SNAPSHOT.jar.original` in the `target` directory. This is the original jar file that Maven created before it was repackaged by Spring Boot.

To run that application, use the `java -jar` command, as follows:

```
$ java -jar target/myproject-0.0.1-SNAPSHOT.jar
```

```

      .  ____          ( _\    ____ .  __\ .  __\
 ( (    /___  \  |__| |__|  \  |__|  \  |__|  \  |__|  \
 \ \   /___  \  |__| |__|  \  |__|  \  |__|  \  |__|  \
  ' |___  \  |__| |__|  \  |__|  \  |__|  \  |__|  \
=====|_|=====|_|_/_/_/_/
:: Spring Boot :: (v{version-spring-boot})
.....
.....  . . . (log output here)
.....
..... Started MyApplication in 0.999 seconds (process running for 1.253)

```

As before, to exit the application, press `ctrl-c`.

6.7.2. Gradle

To create an executable jar, we need to run `gradle bootJar` from the command line, as follows:

```
$ gradle bootJar
```

```

BUILD SUCCESSFUL in 639ms
3 actionable tasks: 3 executed

```

If you look in the `build/libs` directory, you should see `myproject-0.0.1-SNAPSHOT.jar`. The file should be around 18 MB in size. If you want to peek inside, you can use `jar tvf`, as follows:

```
$ jar tvf build/libs/myproject-0.0.1-SNAPSHOT.jar
```

To run that application, use the `java -jar` command, as follows:

```
$ java -jar build/libs/myproject-0.0.1-SNAPSHOT.jar
```

```

      .  ____          ( _\    ____ .  __\ .  __\
 ( (    /___  \  |__| |__|  \  |__|  \  |__|  \  |__|  \
 \ \   /___  \  |__| |__|  \  |__|  \  |__|  \  |__|  \
  ' |___  \  |__| |__|  \  |__|  \  |__|  \  |__|  \
=====|_|=====|_|_/_/_/_/
:: Spring Boot :: (v{version-spring-boot})
.....
.....  . . . (log output here)
.....
..... Started MyApplication in 0.999 seconds (process running for 1.253)

```

As before, to exit the application, press `ctrl-c`.

7. Tutorials

This section provides tutorials to help you get started using Spring Boot.

8. Reference Documentation

9. Auditing

Once Spring Security is in play, Spring Boot Actuator has a flexible audit framework that publishes events (by default, “authentication success”, “failure” and “access denied” exceptions). This feature can be very useful for reporting and for implementing a lock-out policy based on authentication failures.

You can enable auditing by providing a bean of type `javax.annotation.processing.processor.AuditEventRepository[]` in your application's configuration. For convenience, Spring Boot offers an `javax.annotation.processing.processor.InMemoryAuditEventRepository[]`. `javax.annotation.processing.processor.InMemoryAuditEventRepository[]` has limited capabilities, and we recommend using it only for development environments. For production environments, consider creating your own alternative `javax.annotation.processing.processor.AuditEventRepository[]` implementation.

9.1. Custom Auditing

To customize published security events, you can provide your own implementations of `javax.annotation.processing.processor.AbstractAuthenticationAuditListener[]` and `javax.annotation.processing.processor.AbstractAuthorizationAuditListener[]`.

You can also use the audit services for your own business events. To do so, either inject the `javax.annotation.processing.processor.AuditEventRepository[]` bean into your own components and use that directly or publish an `javax.annotation.processing.processor.AuditApplicationEvent[]` with the Spring `javax.annotation.processing.processor.ApplicationEventPublisher[]` (by implementing `javax.annotation.processing.processor.ApplicationEventPublisherAware[]`).

10. Cloud Foundry Support

Spring Boot's `spring-boot-cloudfoundry` module (part of `spring-boot-starter-cloudfoundry`) includes additional support that is activated when you deploy to a compatible Cloud Foundry instance. The `/cloudfoundryapplication` path provides an alternative secured route to all `javax.annotation.security.RunAsAuthenticated.class` beans.

The extended support lets Cloud Foundry management UIs (such as the web application that you can use to view deployed applications) be augmented with Spring Boot actuator information. For example, an application status page can include full health information instead of the typical “running” or “stopped” status.



The `/cloudfoundryapplication` path is not directly accessible to regular users. To use the endpoint, you must pass a valid UAA token with the request.

10.1. Disabling Extended Cloud Foundry Actuator Support

If you want to fully disable the `/cloudfoundryapplication` endpoints, you can add the following setting to your `application.properties` file:

```
management:
  cloudfoundry:
    enabled: false
```

10.2. Cloud Foundry Self-signed Certificates

By default, the security verification for `/cloudfoundryapplication` endpoints makes SSL calls to various Cloud Foundry services. If your Cloud Foundry UAA or Cloud Controller services use self-signed certificates, you need to set the following property:

```
management:
  cloudfoundry:
    skip-ssl-validation: true
```

10.3. Custom Context Path

If the server's context-path has been configured to anything other than `/`, the Cloud Foundry endpoints are not available at the root of the application. For example, if `server.servlet.context-path=/app`, Cloud Foundry endpoints are available at `/app/cloudfoundryapplication/*`.

If you expect the Cloud Foundry endpoints to always be available at `/cloudfoundryapplication/*`, regardless of the server's context-path, you need to explicitly configure that in your application. The configuration differs, depending on the web server in use. For Tomcat, you can add the following configuration:

```
include-code::MyCloudFoundryConfiguration[]
```

If you're using a Webflux based application, you can use the following configuration:

```
include-code::MyReactiveCloudFoundryConfiguration[]
```

11. Enabling Production-ready Features

The {code-spring-boot}/module/spring-boot-actuator[spring-boot-actuator] module provides all of Spring Boot's production-ready features. The recommended way to enable the features is to add a dependency on the spring-boot-starter-actuator starter.

Definition of Actuator

An actuator is a manufacturing term that refers to a mechanical device for moving or controlling something. Actuators can generate a large amount of motion from a small change.

To add the actuator to a Maven-based project, add the following starter dependency:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

XML

For Gradle, use the following declaration:

```
dependencies {
  implementation 'org.springframework.boot:spring-boot-starter-actuator'
}
```

GRADLE

12. Endpoints

Actuator endpoints let you monitor and interact with your application. Spring Boot includes a number of built-in endpoints and lets you add your own. For example, the `health` endpoint provides basic application health information.

You can [control access](#) to each individual endpoint and [expose them \(make them remotely accessible\) over HTTP or JMX](#). An endpoint is considered to be available when access to it is permitted and it is exposed. The built-in endpoints are auto-configured only when they are available. Most applications choose exposure over HTTP, where the ID of the endpoint and a prefix of `/actuator` is mapped to a URL. For example, by default, the `health` endpoint is mapped to `/actuator/health`.



To learn more about the Actuator's endpoints and their request and response formats, see the [API documentation](#).

The following technology-agnostic endpoints are available:

ID	Description
<code>auditevents</code>	Exposes audit events information for the current application. Requires an <code>javadoc:org.springframework.boot.actuate.audit.AuditEventRepository[]</code> bean.
<code>beans</code>	Displays a complete list of all the Spring beans in your application.
<code>caches</code>	Exposes available caches.
<code>conditions</code>	Shows the conditions that were evaluated on configuration and auto-configuration classes and the reasons why they did or did not match.
<code>configprops</code>	Displays a collated list of all <code>javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]</code> . Subject to sanitization .
<code>env</code>	Exposes properties from Spring's <code>javadoc:org.springframework.core.env.ConfigurableEnvironment[]</code> . Subject to sanitization .
<code>flyway</code>	Shows any Flyway database migrations that have been applied. Requires one or more <code>javadoc:org.flywaydb.core.Flyway[]</code> beans.
<code>health</code>	Shows application health information.
<code>httpexchanges</code>	Displays HTTP exchange information (by default, the last 100 HTTP request-response exchanges). Requires an <code>javadoc:org.springframework.boot.actuate.web.exchanges.HttpExchangeRepository[]</code> bean.
<code>info</code>	Displays arbitrary application info.
<code>integrationgraph</code>	Shows the Spring Integration graph. Requires a dependency on <code>spring-integration-core</code> .
<code>loggers</code>	Shows and modifies the configuration of loggers in the application.
<code>liquibase</code>	Shows any Liquibase database migrations that have been applied. Requires one or more <code>javadoc:{url-liquibase-javado}/liquibase.Liquibase[]</code> beans.
<code>metrics</code>	Shows “metrics” information for the current application to diagnose the metrics the application has recorded.
<code>mappings</code>	Displays a collated list of all <code>javadoc:org.springframework.web.bind.annotation.RequestMapping[format=annotation]</code> paths.
<code>quartz</code>	Shows information about Quartz Scheduler jobs. Subject to sanitization .
<code>scheduledtasks</code>	Displays the scheduled tasks in your application.
<code>sessions</code>	Allows retrieval and deletion of user sessions from a Spring Session-backed session store. Requires a servlet-based web application that uses Spring Session.
<code>shutdown</code>	Lets the application be gracefully shutdown. Only works when using jar packaging. Disabled by default.
<code>startup</code>	Shows the startup steps data collected by the <code>javadoc:org.springframework.core.metrics.ApplicationStartup[]</code> . Requires the <code>javadoc:org.springframework.boot.SpringApplication[]</code> to be configured with a <code>javadoc:org.springframework.boot.context.metrics.buffering.BufferingApplicationStartup[]</code> .
<code>threaddump</code>	Performs a thread dump.

If your application is a web application (Spring MVC, Spring WebFlux, or Jersey), you can use the following additional endpoints:

ID	Description
<code>heapdump</code>	Returns a heap dump file. On a HotSpot JVM, an <code>HPROF</code> -format file is returned. On an OpenJ9 JVM, a <code>PHD</code> -format file is returned.
<code>logfile</code>	Returns the contents of the logfile (if the <code>logging.file.name</code> or the <code>logging.file.path</code> property has been set). Supports the use of the HTTP <code>Range</code> header to retrieve part of the log file's content.

ID	Description
prometheus	Exposes metrics in a format that can be scraped by a Prometheus server. Requires a dependency on micrometer-registry-prometheus.

12.1. Controlling Access to Endpoints

By default, access to all endpoints except for `shutdown` and `heapdump` is unrestricted. To configure the permitted access to an endpoint, use its `management.endpoint.<id>.access` property. The following example allows unrestricted access to the `shutdown` endpoint:

```
management:
  endpoint:
    shutdown:
      access: unrestricted
```

If you prefer access to be opt-in rather than opt-out, set the `configprop:management.endpoints.access.default[]` property to `none` and use individual endpoint `access` properties to opt back in. The following example allows read-only access to the `loggers` endpoint and denies access to all other endpoints:

```
management:
  endpoints:
    access:
      default: none
  endpoint:
    loggers:
      access: read-only
```



Inaccessible endpoints are removed entirely from the application context. If you want to change only the technologies over which an endpoint is exposed, use the [include and exclude properties](#) instead.

12.1.1. Limiting Access

Application-wide endpoint access can be limited using the `configprop:management.endpoints.access.max-permitted[]` property. This property takes precedence over the default access or an individual endpoint's access level. Set it to `none` to make all endpoints inaccessible. Set it to `read-only` to only allow read access to endpoints.

For `javadoc:org.springframework.boot.actuate.endpoint.annotation.Endpoint[format=annotation]`, `javadoc:org.springframework.boot.actuate.endpoint.jmx.annotation.JmxEndpoint[format=annotation]`, and `javadoc:org.springframework.boot.actuate.endpoint.web.annotation.WebEndpoint[format=annotation]`, read access equates to the endpoint methods annotated with `javadoc:org.springframework.boot.actuate.endpoint.annotation.ReadOperation[format=annotation]`. For `javadoc:org.springframework.boot.actuate.endpoint.web.annotation.ControllerEndpoint[format=annotation]` and `javadoc:org.springframework.boot.actuate.endpoint.web.annotation.RestControllerEndpoint[format=annotation]`, read access equates to request mappings that can handle `GET` and `HEAD` requests. For `javadoc:org.springframework.boot.actuate.endpoint.web.annotation.ServletEndpoint[format=annotation]`, read access equates to `GET` and `HEAD` requests.

12.2. Exposing Endpoints

By default, only the health endpoint is exposed over HTTP and JMX. Since Endpoints may contain sensitive information, you should carefully consider when to expose them.

To change which endpoints are exposed, use the following technology-specific `include` and `exclude` properties:

Property	Default
configprop:management.endpoints.jmx.exposure.exclude[]	
configprop:management.endpoints.jmx.exposure.include[]	health
configprop:management.endpoints.web.exposure.exclude[]	
configprop:management.endpoints.web.exposure.include[]	health

The `include` property lists the IDs of the endpoints that are exposed. The `exclude` property lists the IDs of the endpoints that should not be exposed. The `exclude` property takes precedence over the `include` property. You can configure both the `include` and the `exclude` properties with a list of endpoint IDs.

For example, to only expose the `health` and `info` endpoints over JMX, use the following property:

```
management:
  endpoints:
    jmx:
      exposure:
        include: "health,info"
```

* can be used to select all endpoints. For example, to expose everything over HTTP except the `env` and `beans` endpoints, use the following properties:

```
management:
  endpoints:
    web:
      exposure:
        include: "*"
        exclude: "env,beans"
```



* has a special meaning in YAML, so be sure to add quotation marks if you want to include (or exclude) all endpoints.



If your application is exposed publicly, we strongly recommend that you also [secure your endpoints](#).



If you want to implement your own strategy for when endpoints are exposed, you can register an `javax.servlet.http.HttpServletRequest` bean.

12.3. Security

For security purposes, only the `/health` endpoint is exposed over HTTP by default. You can use the `management.endpoints.web.exposure.include[]` property to configure the endpoints that are exposed.



Before setting the `management.endpoints.web.exposure.include`, ensure that the exposed actuators do not contain sensitive information, are secured by placing them behind a firewall, or are secured by something like Spring Security.

If Spring Security is on the classpath and no other `javax.servlet.http.HttpServletRequest` bean is present, all actuators other than `/health` are secured by Spring Boot auto-configuration. If you define a custom `javax.servlet.http.HttpServletRequest` bean, Spring Boot auto-configuration backs off and lets you fully control the actuator access rules.

If you wish to configure custom security for HTTP endpoints (for example, to allow only users with a certain role to access them), Spring Boot provides some convenient `javax.servlet.http.HttpServletRequest` objects that you can use in combination with Spring Security.

A typical Spring Security configuration might look something like the following example:

```
include-code::typical/MySecurityConfiguration[]
```

The preceding example uses `EndpointRequest.toAnyEndpoint()` to match a request to any endpoint and then ensures that all have the `ENDPOINT_ADMIN` role. Several other matcher methods are also available on `javax.servlet.http.HttpServletRequest`. See the [API documentation](#) for details.

If you deploy applications behind a firewall, you may prefer that all your actuator endpoints can be accessed without requiring authentication. You can do so by changing the `management.endpoints.web.exposure.include[]` property, as follows:

```
management:
  endpoints:
    web:
      exposure:
        include: "*"

```

Additionally, if Spring Security is present, you would need to add custom security configuration that allows unauthenticated access to the endpoints, as the following example shows:

```
include-code::exposeall/MySecurityConfiguration[]
```



In both of the preceding examples, the configuration applies only to the actuator endpoints. Since Spring Boot's security configuration backs off completely in the presence of any `javax.servlet.http.HttpServletRequest` bean, you need to configure an additional `javax.servlet.http.HttpServletRequest` bean with rules that apply to the rest of the application.

12.3.1. Cross Site Request Forgery Protection

Since Spring Boot relies on Spring Security's defaults, CSRF protection is turned on by default. This means that the actuator endpoints that require a `POST` (shutdown and loggers endpoints), a `PUT`, or a `DELETE` get a 403 (forbidden) error when the default security configuration is in use.



We recommend disabling CSRF protection completely only if you are creating a service that is used by non-browser clients.

You can find additional information about CSRF protection in the [{url-spring-security-docs}/features/exploits/csrf.html](#) [Spring Security Reference Guide].

12.4. Configuring Endpoints

Endpoints automatically cache responses to read operations that do not take any parameters. To configure the amount of time for which an endpoint caches a response, use its `cache.time-to-live` property. The following example sets the time-to-live of the `beans` endpoint's cache to 10 seconds:

```
management:
  endpoint:
    beans:
      cache:
        time-to-live: "10s"
```



The `management.endpoint.<name>` prefix uniquely identifies the endpoint that is being configured.

12.5. Sanitize Sensitive Values

Information returned by the `/env`, `/configprops` and `/quartz` endpoints can be sensitive, so by default values are always fully sanitized (replaced by `*****`).

Values can only be viewed in an unsanitized form when:

- The `show-values` property has been set to something other than `never`
- No custom [SanitizingFunction](#) beans apply

The `show-values` property can be configured for sanitizable endpoints to one of the following values:

- `never` - values are always fully sanitized (replaced by `*****`)
- `always` - values are shown to all users (as long as no `javadoc.org.springframework.boot.actuate.endpoint.SanitizingFunction[]` bean applies)
- `when-authorized` - values are shown only to authorized users (as long as no `javadoc.org.springframework.boot.actuate.endpoint.SanitizingFunction[]` bean applies)

For HTTP endpoints, a user is considered to be authorized if they have authenticated and have the roles configured by the endpoint's `roles` property. By default, any authenticated user is authorized.

For JMX endpoints, all users are always authorized.

The following example allows all users with the `admin` role to view values from the `/env` endpoint in their original form. Unauthorized users, or users without the `admin` role, will see only sanitized values.

```
management:
  endpoint:
    env:
      show-values: when-authorized
      roles: "admin"
```



This example assumes that no [SanitizingFunction](#) beans have been defined.

12.6. Hypermedia for Actuator Web Endpoints

A “discovery page” is added with links to all the endpoints. The “discovery page” is available on `/actuator` by default.

To disable the “discovery page”, add the following property to your application properties:

```
management:
  endpoints:
    web:
      discovery:
        enabled: false
```

When a custom management context path is configured, the “discovery page” automatically moves from `/actuator` to the root of the management context. For example, if the management context path is `/management`, the discovery page is available from `/management`. When the management context path is set to `/`, the discovery page is disabled to prevent the possibility of a clash with other mappings.

12.7. CORS Support

[Cross-origin resource sharing](https://en.wikipedia.org/wiki/Cross-origin_resource_sharing) (https://en.wikipedia.org/wiki/Cross-origin_resource_sharing) (CORS) is a [W3C specification](https://www.w3.org/TR/cors/) (<https://www.w3.org/TR/cors/>) that lets you specify in a flexible way what kind of cross-domain requests are authorized. If you use Spring MVC or Spring WebFlux, you can configure Actuator's web endpoints to support such scenarios.

CORS support is disabled by default and is only enabled once you have set the `configprop:management.endpoints.web.cors.allowed-origins[]` property. The following configuration permits GET and POST calls from the `example.com` domain:

```
management:
  endpoints:
    web:
      cors:
        allowed-origins: "https://example.com"
        allowed-methods: "GET,POST"
```



See `javadoc:org.springframework.boot.actuate.autoconfigure.endpoint.web.CorsEndpointProperties[]` for a complete list of options.

12.8. JSON

When working with JSON, Jackson is used for serialization and deserialization. By default, an isolated `javax.tools.jackson.databind.json.JsonMapper[]` is used. This isolation means that it does not share the same configuration as the application's `JsonMapper` and it is not affected by `spring.jackson.*` properties. To disable this behavior and configure Actuator to use the application's `JsonMapper`, set `configprop:management.endpoints.jackson.isolated-json-mapper[]` to `false`. Alternatively, you can define your own `javax.org.springframework.boot.actuate.endpoint.jackson.Endpoint[JsonMapper[]]` bean that produces a `JsonMapper` that meets your needs. Actuator will then use it for JSON processing.

12.9. Implementing Custom Endpoints

If you add a `javax.org.springframework.context.annotation.Bean[format=annotation]` annotated with `javax.org.springframework.boot.actuate.endpoint.annotation.Endpoint[format=annotation]`, any methods annotated with `javax.org.springframework.boot.actuate.endpoint.annotation.ReadOperation[format=annotation]`, `javax.org.springframework.boot.actuate.endpoint.annotation.WriteOperation[format=annotation]`, or `javax.org.springframework.boot.actuate.endpoint.annotation.DeleteOperation[format=annotation]` are automatically exposed over JMX and, in a web application, over HTTP as well. Endpoints can be exposed over HTTP by using Jersey, Spring MVC, or Spring WebFlux. If both Jersey and Spring MVC are available, Spring MVC is used.

The following example exposes a read operation that returns a custom object:

```
include-code::MyEndpoint[tag=read]
```

You can also write technology-specific endpoints by using `javax.org.springframework.boot.actuate.endpoint.jmx.annotation.JmxEndpoint[format=annotation]` or `javax.org.springframework.boot.actuate.endpoint.web.annotation.WebEndpoint[format=annotation]`. These endpoints are restricted to their respective technologies. For example, `javax.org.springframework.boot.actuate.endpoint.web.annotation.WebEndpoint[format=annotation]` is exposed only over HTTP and not over JMX.

You can write technology-specific extensions by using `javax.org.springframework.boot.actuate.endpoint.web.annotation.EndpointWebExtension[format=annotation]` and `javax.org.springframework.boot.actuate.endpoint.jmx.annotation.EndpointJmxExtension[format=annotation]`. These annotations let you provide technology-specific operations to augment an existing endpoint. An endpoint may have at most one extension of each type.

Finally, if you need access to web-framework-specific functionality, you can implement servlet or Spring `javax.org.springframework.stereotype.Controller[format=annotation]` and `javax.org.springframework.web.bind.annotation.RestController[format=annotation]` endpoints at the cost of them not being available over JMX or when using a different web framework.

12.9.1. Receiving Input

Operations on an endpoint receive input through their parameters. When exposed over the web, the values for these parameters are taken from the URL's query parameters and from the JSON request body. When exposed over JMX, the parameters are mapped to the parameters of the MBean's operations. Parameters are required by default. They can be made optional by annotating them with JSpecify's `javax.specify.annotations.Nullable[format=annotation]`. Kotlin null safety is also supported.

You can map each root property in the JSON request body to a parameter of the endpoint. Consider the following JSON request body:

```
{
  "name": "test",
  "counter": 42
}
```

JSON

You can use this to invoke a write operation that takes `String name` and `int counter` parameters, as the following example shows:

```
include-code::.../MyEndpoint[tag=write]
```



Because endpoints are technology agnostic, only simple types can be specified in the method signature. In particular, declaring a single parameter with a `javax.liquibase.report.CustomData[]` type that defines a `name` and `counter` properties is not supported.



To let the input be mapped to the operation method's parameters, Java code that implements an endpoint should be compiled with `-parameters`. For Kotlin code, please review <https://docs.spring.io/spring-framework/docs/5.2.x/reference/html/52x-languages/kotlin/classes-interfaces.html> [the recommendation] of the Spring Framework reference. This will happen automatically if you use Spring Boot's Gradle plugin or if you use Maven and `spring-boot-starter-parent`.

Input Type Conversion

The parameters passed to endpoint operation methods are, if necessary, automatically converted to the required type. Before calling an operation method, the input received over JMX or HTTP is converted to the required types by using an instance of `javax.org.springframework.boot.convert.ApplicationConversionService[]` as well as any `javax.org.springframework.core.convert.converter.Converter[]` or `javax.org.springframework.core.convert.converter.GenericConverter[]` beans qualified with `javax.org.springframework.boot.actuate.endpoint.annotation.EndpointConverter[format=annotation]`.

12.9.2. Custom Web Endpoints

Operations on an `javax.org.springframework.boot.actuate.endpoint.annotation.Endpoint[format=annotation]`, `javax.org.springframework.boot.actuate.endpoint.web.annotation.WebEndpoint[format=annotation]`, or `javax.org.springframework.boot.actuate.endpoint.web.annotation.EndpointWebExtension[format=annotation]` are automatically exposed over HTTP using Jersey, Spring MVC, or Spring WebFlux. If both Jersey and Spring MVC are available, Spring MVC is used.

Web Endpoint Request Predicates

A request predicate is automatically generated for each operation on a web-exposed endpoint.

Path

The path of the predicate is determined by the ID of the endpoint and the base path of the web-exposed endpoints. The default base path is `/actuator`. For example, an endpoint with an ID of `sessions` uses `/actuator/sessions` as its path in the predicate.

You can further customize the path by annotating one or more parameters of the operation method with `javax.annotation.processing.processor.annotation.selector.Selector[format=annotation]`. Such a parameter is added to the path predicate as a path variable. The variable's value is passed into the operation method when the endpoint operation is invoked. If you want to capture all remaining path elements, you can add `@Selector(Match=ALL_REMAINING)` to the last parameter and make it a type that is conversion-compatible with a `String[]`.

HTTP method

The HTTP method of the predicate is determined by the operation type, as shown in the following table:

Operation	HTTP method
<code>javax.annotation.processing.processor.annotation.selector.ReadOperation[format=annotation]</code>	GET
<code>javax.annotation.processing.processor.annotation.selector.WriteOperation[format=annotation]</code>	POST
<code>javax.annotation.processing.processor.annotation.selector.DeleteOperation[format=annotation]</code>	DELETE

Consumes

For a `javax.annotation.processing.processor.annotation.selector.WriteOperation[format=annotation]` (HTTP `POST`) that uses the request body, the `consumes` clause of the predicate is `application/vnd.spring-boot.actuator.v2+json, application/json`. For all other operations, the `consumes` clause is empty.

Produces

The `produces` clause of the predicate can be determined by the `produces` attribute of the `javax.annotation.processing.processor.annotation.selector.DeleteOperation[format=annotation]`, `javax.annotation.processing.processor.annotation.selector.ReadOperation[format=annotation]`, and `javax.annotation.processing.processor.annotation.selector.WriteOperation[format=annotation]` annotations. The attribute is optional. If it is not used, the `produces` clause is determined automatically.

If the operation method returns `void` or `javax.lang.java.lang.Void[]`, the `produces` clause is empty. If the operation method returns a `javax.annotation.processing.processor.annotation.selector.Resource[]`, the `produces` clause is `application/octet-stream`. For all other operations, the `produces` clause is `application/vnd.spring-boot.actuator.v2+json, application/json`.

Web Endpoint Response Status

The default response status for an endpoint operation depends on the operation type (read, write, or delete) and what, if anything, the operation returns.

If a `javax.annotation.processing.processor.annotation.selector.ReadOperation[format=annotation]` returns a value, the response status will be 200 (OK). If it does not return a value, the response status will be 404 (Not Found).

If a `javax.annotation.processing.processor.annotation.selector.WriteOperation[format=annotation]` or `javax.annotation.processing.processor.annotation.selector.DeleteOperation[format=annotation]` returns a value, the response status will be 200 (OK). If it does not return a value, the response status will be 204 (No Content).

If an operation is invoked without a required parameter or with a parameter that cannot be converted to the required type, the operation method is not called, and the response status will be 400 (Bad Request).

Web Endpoint Range Requests

You can use an HTTP range request to request part of an HTTP resource. When using Spring MVC or Spring Web Flux, operations that return a `javax.annotation.processing.processor.annotation.selector.Resource[]` automatically support range requests.



Range requests are not supported when using Jersey.

Web Endpoint Security

An operation on a web endpoint or a web-specific endpoint extension can receive the current `javax.annotation.processing.processor.annotation.selector.Principal[]` or `javax.annotation.processing.processor.annotation.selector.SecurityContext[]` as a method parameter. The former is typically used in conjunction with `javax.annotation.processing.processor.annotation.selector.Nullable[format=annotation]` to provide different behavior for authenticated and unauthenticated users. The latter is typically used to perform authorization checks by using its `isUserInRole(String)` method.

12.10. Health Information

You can use health information to check the status of your running application. It is often used by monitoring software to alert someone when a production system goes down. The information exposed by the `health` endpoint depends on the `configprop:management.endpoint.health.show-details[]` and `configprop:management.endpoint.health.show-components[]` properties, which can be configured with one of the following values:

Name	Description
<code>never</code>	Details are never shown.
<code>when-authorized</code>	Details are shown only to authorized users. Authorized roles can be configured by using <code>management.endpoint.health.roles</code> .
<code>always</code>	Details are shown to all users.

The default value is `never` . A user is considered to be authorized when they are in one or more of the endpoint's roles. If the endpoint has no configured roles (the default), all authenticated users are considered to be authorized. You can configure the roles by using the `configprop:management.endpoint.health.roles[]` property.



If you have secured your application and wish to use `always` , your security configuration must permit access to the health endpoint for both authenticated and unauthenticated users.

Health information is collected from the content of a `javadoc:org.springframework.boot.health.registry.HealthContributorRegistry[]` (by default, all `javadoc:org.springframework.boot.health.contributor.HealthContributor[]` instances defined in your `javadoc:org.springframework.context.ApplicationContext[]`). Spring Boot includes a number of auto-configured `javadoc:org.springframework.boot.health.contributor.HealthContributor[]` beans, and you can also write your own.

A `javadoc:org.springframework.boot.health.contributor.HealthContributor[]` can be either a `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` or a `javadoc:org.springframework.boot.health.contributor.CompositeHealthContributor[]` . A `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` provides actual health information, including a `javadoc:org.springframework.boot.health.contributor.Status[]` . A `javadoc:org.springframework.boot.health.contributor.CompositeHealthContributor[]` provides a composite of other `javadoc:org.springframework.boot.health.contributor.HealthContributor[]` instances. Taken together, contributors form a tree structure to represent the overall system health.

By default, the final system health is derived by a `javadoc:org.springframework.boot.health.actuate.endpoint.StatusAggregator[]` , which sorts the statuses from each `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` based on an ordered list of statuses. The first status in the sorted list is used as the overall health status. If no `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` returns a status that is known to the `javadoc:org.springframework.boot.health.actuate.endpoint.StatusAggregator[]` , an `UNKNOWN` status is used.



You can use the `javadoc:org.springframework.boot.health.registry.HealthContributorRegistry[]` to register and unregister health indicators at runtime.

12.10.1. Auto-configured HealthIndicators

When appropriate, Spring Boot auto-configures the `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` beans listed in the following table. You can also enable or disable selected indicators by configuring `management.health.key.enabled` , with the `key` listed in the following table:

Key	Name	Description
cassandra	<code>javadoc:org.springframework.boot.cassandra.health.CassandraDriverHealthIndicator[]</code>	Checks that a Cassandra database is up.
couchbase	<code>javadoc:org.springframework.boot.couchbase.health.CouchbaseHealthIndicator[]</code>	Checks that a Couchbase cluster is up.
db	<code>javadoc:org.springframework.boot.jdbc.health.DataSourceHealthIndicator[]</code>	Checks that a connection to <code>javadoc:javax.sql.DataSource[]</code> can be obtained.
diskspace	<code>javadoc:org.springframework.boot.health.application.DiskSpaceHealthIndicator[]</code>	Checks for low disk space.
elasticsearch	<code>javadoc:org.springframework.boot.elasticsearch.health.ElasticsearchRestClientHealthIndicator[]</code>	Checks that an Elasticsearch cluster is up.
hazelcast	<code>javadoc:org.springframework.boot.hazelcast.health.HazelcastHealthIndicator[]</code>	Checks that a Hazelcast server is up.
jms	<code>javadoc:org.springframework.boot.jms.health.JmsHealthIndicator[]</code>	Checks that a JMS broker is up.
ldap	<code>javadoc:org.springframework.boot.ldap.health.LdapHealthIndicator[]</code>	Checks that an LDAP server is up.
mail	<code>javadoc:org.springframework.boot.mail.health.MailHealthIndicator[]</code>	Checks that a mail server is up.
mongo	<code>javadoc:org.springframework.boot.mongodb.health.MongoHealthIndicator[]</code>	Checks that a Mongo database is up.
neo4j	<code>javadoc:org.springframework.boot.neo4j.health.Neo4jHealthIndicator[]</code>	Checks that a Neo4j database is up.
ping	<code>javadoc:org.springframework.boot.health.contributor.PingHealthIndicator[]</code>	Always responds with UP.
rabbit	<code>javadoc:org.springframework.boot.amqp.health.RabbitHealthIndicator[]</code>	Checks that a Rabbit server is up.
redis	<code>javadoc:org.springframework.boot.data.redis.health.DataRedisHealthIndicator[]</code>	Checks that a Redis server is up.
ssl	<code>javadoc:org.springframework.boot.health.application.SslHealthIndicator[]</code>	Checks that SSL certificates are ok.



You can disable them all by setting the `configprop:management.health.defaults.enabled[]` property.



The `ssl` `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` has a "warning threshold" property named `configprop:management.health.ssl.certificate-validity-warning-threshold[]` . You can use this threshold to give yourself enough lead time to rotate the soon-to-be-expired certificate. If an SSL certificate will become invalid within the period defined by this threshold, the `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` will report this in the details section of its response where `details.validChains.certificates.[*].validity.status` will have the value `WILL_EXPIRE_SOON` .

Additional `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` beans are enabled by default:

Key	Name	Description
livenessstate	javadoc:org.springframework.boot.health.application.LivenessStateHealthIndicator[]	Exposes the “Liveness” application availability state.
readinessstate	javadoc:org.springframework.boot.health.application.ReadinessStateHealthIndicator[]	Exposes the “Readiness” application availability state.

These can be disabled by using the `configprop:management.endpoint.health.probes.enabled[]` configuration property.

12.10.2. Writing Custom HealthIndicators

To provide custom health information, you can register Spring beans that implement the `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` interface. You need to provide an implementation of the `health()` method and return a `javadoc:org.springframework.boot.health.contributor.Health[]` response. The `javadoc:org.springframework.boot.health.contributor.Health[]` response should include a status and can optionally include additional details to be displayed. The following code shows a sample `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` implementation:

include-code::MyHealthIndicator[]



The identifier for a given `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` is the name of the bean without the `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` suffix, if it exists. In the preceding example, the health information is available in an entry named `my`.



Health indicators are usually called over HTTP and need to respond before any connection timeouts. Spring Boot will log a warning message for any health indicator that takes longer than 10 seconds to respond. If you want to configure this threshold, you can use the `configprop:management.endpoint.health.logging.slow-indicator-threshold[]` property.

In addition to Spring Boot's predefined `javadoc:org.springframework.boot.health.contributor.Status[]` types, `javadoc:org.springframework.boot.health.contributor.Health[]` can return a custom `javadoc:org.springframework.boot.health.contributor.Status[]` that represents a new system state. In such cases, you also need to provide a custom implementation of the `javadoc:org.springframework.boot.health.actuate.endpoint.StatusAggregator[]` interface, or you must configure the default implementation by using the `configprop:management.endpoint.health.status.order[]` configuration property.

For example, assume a new `javadoc:org.springframework.boot.health.contributor.Status[]` with a code of `FATAL` is being used in one of your `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` implementations. To configure the severity order, add the following property to your application properties:

```
management:
  endpoint:
    health:
      status:
        order: "fatal,down,out-of-service,unknown,up"
```

The HTTP status code in the response reflects the overall health status. By default, `OUT_OF_SERVICE` and `DOWN` map to 503. Any unmapped health statuses, including `UP`, map to 200. You might also want to register custom status mappings if you access the health endpoint over HTTP. Configuring a custom mapping disables the defaults mappings for `DOWN` and `OUT_OF_SERVICE`. If you want to retain the default mappings, you must explicitly configure them, alongside any custom mappings. For example, the following property maps `FATAL` to 503 (service unavailable) and retains the default mappings for `DOWN` and `OUT_OF_SERVICE`:

```
management:
  endpoint:
    health:
      status:
        http-mapping:
          down: 503
          fatal: 503
          out-of-service: 503
```



If you need more control, you can define your own `javadoc:org.springframework.boot.health.actuate.endpoint.HttpCodeStatusMapper[]` bean.

The following table shows the default status mappings for the built-in statuses:

Status	Mapping
DOWN	SERVICE_UNAVAILABLE (503)
OUT_OF_SERVICE	SERVICE_UNAVAILABLE (503)
UP	No mapping by default, so HTTP status is 200
UNKNOWN	No mapping by default, so HTTP status is 200

12.10.3. Reactive Health Indicators

For reactive applications, such as those that use Spring WebFlux, `javadoc:org.springframework.boot.health.contributor.ReactiveHealthContributor[]` provides a non-blocking contract for getting application health. Similar to a traditional `javadoc:org.springframework.boot.health.contributor.HealthContributor[]`, health information is collected from the content of a `javadoc:org.springframework.boot.health.registry.ReactiveHealthContributorRegistry[]` (by default, all `javadoc:org.springframework.boot.health.contributor.HealthContributor[]` and

`javadoc:org.springframework.boot.health.contributor.ReactiveHealthContributor[]` instances defined in your `javadoc:org.springframework.context.ApplicationContext[]`). Regular `javadoc:org.springframework.boot.health.contributor.HealthContributor[]` instances that do not check against a reactive API are executed on the elastic scheduler.



In a reactive application, you should use the `javadoc:org.springframework.boot.health.registry.ReactiveHealthContributorRegistry[]` to register and unregister health indicators at runtime. If you need to register a regular `javadoc:org.springframework.boot.health.contributor.HealthContributor[]`, you should wrap it with `ReactiveHealthContributor#adapt`.

To provide custom health information from a reactive API, you can register Spring beans that implement the `javadoc:org.springframework.boot.health.contributor.ReactiveHealthIndicator[]` interface. The following code shows a sample `javadoc:org.springframework.boot.health.contributor.ReactiveHealthIndicator[]` implementation:

```
include-code::MyReactiveHealthIndicator[]
```



To handle the error automatically, consider extending from `javadoc:org.springframework.boot.health.contributor.AbstractReactiveHealthIndicator[]`.

12.10.4. Auto-configured `ReactiveHealthIndicators`

When appropriate, Spring Boot auto-configures the following `javadoc:org.springframework.boot.health.contributor.ReactiveHealthIndicator[]` beans:

Key	Name	Description
cassandra	<code>javadoc:org.springframework.boot.cassandra.health.CassandraDriverReactiveHealthIndicator[]</code>	Checks that a Cassandra database is up.
couchbase	<code>javadoc:org.springframework.boot.couchbase.health.CouchbaseReactiveHealthIndicator[]</code>	Checks that a Couchbase cluster is up.
elasticsearch	<code>javadoc:org.springframework.boot.data.elasticsearch.health.DataElasticsearchReactiveHealthIndicator[]</code>	Checks that an Elasticsearch cluster is up.
mongo	<code>javadoc:org.springframework.boot.mongodb.health.MongoReactiveHealthIndicator[]</code>	Checks that a Mongo database is up.
neo4j	<code>javadoc:org.springframework.boot.neo4j.health.Neo4jReactiveHealthIndicator[]</code>	Checks that a Neo4j database is up.
redis	<code>javadoc:org.springframework.boot.data.redis.health.DataRedisReactiveHealthIndicator[]</code>	Checks that a Redis server is up.



If necessary, reactive indicators replace the regular ones. Also, any `javadoc:org.springframework.boot.health.contributor.HealthIndicator[]` that is not handled explicitly is wrapped automatically.

12.10.5. Health Groups

It is sometimes useful to organize health indicators into groups that you can use for different purposes.

To create a health indicator group, you can use the `management.endpoint.health.group.<name>` property and specify a list of health indicator IDs to `include` or `exclude`. For example, to create a group that includes only database indicators you can define the following:

```
management:
  endpoint:
    health:
      group:
        custom:
          include: "db"
```

You can then check the result by hitting `localhost:8080/actuator/health/custom`.

Similarly, to create a group that excludes the database indicators from the group and includes all the other indicators, you can define the following:

```
management:
  endpoint:
    health:
      group:
        custom:
          exclude: "db"
```

By default, startup will fail if a health group includes or excludes a health indicator that does not exist. To disable this behavior set `configprop:management.endpoint.health.validate-group-membership[]` to `false`.

By default, groups inherit the same `javadoc:org.springframework.boot.health.actuate.endpoint.StatusAggregator[]` and `javadoc:org.springframework.boot.health.actuate.endpoint.HttpCodeStatusMapper[]` settings as the system health. However, you can also define these on a per-group basis. You can also override the `show-details` and `roles` properties if required:

```
management:
  endpoint:
    health:
      group:
        custom:
          show-details: "when-authorized"
          roles: "admin"
          status:
            order: "fatal,up"
          http-mapping:
            fatal: 500
            out-of-service: 500
```



You can use `@Qualifier("groupname")` if you need to register custom `javado:org.springframework.boot.health.actuate.endpoint.StatusAggregator[]` or `javado:org.springframework.boot.health.actuate.endpoint.HttpCodeStatusMapper[]` beans for use with the group.

A health group can also include/exclude a `javado:org.springframework.boot.health.contributor.CompositeHealthContributor[]`. You can also include/exclude only a certain component of a `javado:org.springframework.boot.health.contributor.CompositeHealthContributor[]`. This can be done using the fully qualified name of the component as follows:

```
management.endpoint.health.group.custom.include="test/primary"
management.endpoint.health.group.custom.exclude="test/primary/b"
```

PROPERTIES

In the example above, the `custom` group will include the `javado:org.springframework.boot.health.contributor.HealthContributor[]` with the name `primary` which is a component of the composite `test`. Here, `primary` itself is a composite and the `javado:org.springframework.boot.health.contributor.HealthContributor[]` with the name `b` will be excluded from the `custom` group.

Health groups can be made available at an additional path on either the main or management port. This is useful in cloud environments such as Kubernetes, where it is quite common to use a separate management port for the actuator endpoints for security purposes. Having a separate port could lead to unreliable health checks because the main application might not work properly even if the health check is successful. The health group can be configured with an additional path as follows:

```
management.endpoint.health.group.live.additional-path="server:/healthz"
```

PROPERTIES

This would make the `live` health group available on the main server port at `/healthz`. The prefix is mandatory and must be either `server:` (represents the main server port) or `management:` (represents the management port, if configured.) The path must be a single path segment.

12.10.6. DataSource Health

The `javado:javax.sql.DataSource[]` health indicator shows the health of both standard data sources and routing data source beans. The health of a routing data source includes the health of each of its target data sources. In the health endpoint's response, each of a routing data source's targets is named by using its routing key. If you prefer not to include routing data sources in the indicator's output, set `configprop:management.health.db.ignore-routing-data-sources[]` to `true`.

12.11. Kubernetes Probes

Applications deployed on Kubernetes can provide information about their internal state with [Container Probes](https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#container-probes) (https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#container-probes). Depending on [your Kubernetes configuration](https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/) (https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/), the kubelet calls those probes and reacts to the result.

By default, Spring Boot manages your [Application Availability](#) state. If deployed in a Kubernetes environment, actuator gathers the "Liveness" and "Readiness" information from the `javado:org.springframework.boot.availability.ApplicationAvailability[]` interface and uses that information in dedicated [health indicators](#): `javado:org.springframework.boot.health.application.LivenessStateHealthIndicator[]` and `javado:org.springframework.boot.health.application.ReadinessStateHealthIndicator[]`. These indicators are shown on the global health endpoint (`/actuator/health`). They are also exposed as separate HTTP Probes by using [health groups](#): `/actuator/health/liveness` and `/actuator/health/readiness`.

You can then configure your Kubernetes infrastructure with the following endpoint information:

```
livenessProbe:
  httpGet:
    path: "/actuator/health/liveness"
    port: <actuator-port>
  failureThreshold: ...
  periodSeconds: ...

readinessProbe:
  httpGet:
    path: "/actuator/health/readiness"
    port: <actuator-port>
  failureThreshold: ...
  periodSeconds: ...
```

YAML



`<actuator-port>` should be set to the port that the actuator endpoints are available on. It could be the main web server port or a separate management port if the `"management.server.port"` property has been set.

These health groups are automatically enabled. You can disable them by using the `configprop:management.endpoint.health.probes.enabled[]` configuration property.



If an application takes longer to start than the configured liveness period, Kubernetes mentions the `"startupProbe"` as a possible solution. Generally speaking, the `"startupProbe"` is not necessarily needed here, as the `"readinessProbe"` fails until all startup tasks are done. This means your application will not receive traffic until it is ready. However, if your application takes a long time to start, consider using a `"startupProbe"` to make sure that Kubernetes won't kill your application while it is in the process of starting. See the section that describes [how probes behave during the application lifecycle](#).

If your Actuator endpoints are deployed on a separate management context, the endpoints do not use the same web infrastructure (port, connection pools, framework components) as the main application. In this case, a probe check could be successful even if the main application does not work properly (for example, it cannot accept new connections). For this reason, it is a good idea to make the `liveness` and `readiness` health groups available on the main server port. This can be done by setting the following property:

```
management.endpoint.health.probes.add-additional-paths=true
```

PROPERTIES

This would make the `liveness` group available at `/livez` and the `readiness` group available at `/readyz` on the main server port. Paths can be customized using the `additional-path` property on each group, see [health groups](#) for details.

12.11.1. Checking External State With Kubernetes Probes

Actuator configures the “liveness” and “readiness” probes as Health Groups. This means that all the [health groups features](#) are available for them. You can, for example, configure additional Health Indicators:

```
management:
  endpoint:
    health:
      group:
        readiness:
          include: "readinessState,customCheck"
```

By default, Spring Boot does not add other health indicators to these groups.

The “liveness” probe should not depend on health checks for external systems. If the [liveness state of an application](#) is broken, Kubernetes tries to solve that problem by restarting the application instance. This means that if an external system (such as a database, a Web API, or an external cache) fails, Kubernetes might restart all application instances and create cascading failures.

As for the “readiness” probe, the choice of checking external systems must be made carefully by the application developers. For this reason, Spring Boot does not include any additional health checks in the readiness probe. If the [readiness state of an application instance](#) is unready, Kubernetes does not route traffic to that instance. Some external systems might not be shared by application instances, in which case they could be included in a readiness probe. Other external systems might not be essential to the application (the application could have circuit breakers and fallbacks), in which case they definitely should not be included. Unfortunately, an external system that is shared by all application instances is common, and you have to make a judgement call: Include it in the readiness probe and expect that the application is taken out of service when the external service is down or leave it out and deal with failures higher up the stack, perhaps by using a circuit breaker in the caller.



If all instances of an application are unready, a Kubernetes Service with `type=ClusterIP` or `NodePort` does not accept any incoming connections. There is no HTTP error response (503 and so on), since there is no connection. A service with `type=LoadBalancer` might or might not accept connections, depending on the provider. A service that has an explicit [ingress](#) (<https://kubernetes.io/docs/concepts/services-networking/ingress/>) also responds in a way that depends on the implementation—the ingress service itself has to decide how to handle the “connection refused” from downstream. HTTP 503 is quite likely in the case of both load balancer and ingress.

Also, if an application uses Kubernetes [autoscaling](#) (<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>), it may react differently to applications being taken out of the load-balancer, depending on its autoscaler configuration.

12.11.2. Application Lifecycle and Probe States

An important aspect of the Kubernetes Probes support is its consistency with the application lifecycle. There is a significant difference between the `javadoc:org.springframework.boot.availability.AvailabilityState[]` (which is the in-memory, internal state of the application) and the actual probe (which exposes that state). Depending on the phase of application lifecycle, the probe might not be available.

Spring Boot publishes [application events during startup and shutdown](#), and probes can listen to such events and expose the `javadoc:org.springframework.boot.availability.AvailabilityState[]` information.

The following tables show the `javadoc:org.springframework.boot.availability.AvailabilityState[]` and the state of HTTP connectors at different stages.

When a Spring Boot application starts:

Startup phase	LivenessState	ReadinessState	HTTP server	Notes
Starting	BROKEN	REFUSING_TRAFFIC	Not started	Kubernetes checks the “liveness” Probe and restarts the application if it takes too long.
Started	CORRECT	REFUSING_TRAFFIC	Refuses requests	The application context is refreshed. The application performs startup tasks and does not receive traffic yet.
Ready	CORRECT	ACCEPTING_TRAFFIC	Accepts requests	Startup tasks are finished. The application is receiving traffic.

When a Spring Boot application shuts down:

Shutdown phase	Liveness State	Readiness State	HTTP server	Notes
Running	CORRECT	ACCEPTING_TRAFFIC	Accepts requests	Shutdown has been requested.
Graceful shutdown	CORRECT	REFUSING_TRAFFIC	New requests are rejected	If enabled, graceful shutdown processes in-flight requests . HTTP probes also stop accepting traffic, so the availability states are not readily available externally.
Shutdown complete	N/A	N/A	Server is shut down	The application context is closed and the application is shut down.



See [how-to:deployment/cloud.html](#) for more information about Kubernetes deployment. In particular, it describes how to use the **preStop** hook to give your application time to shut down gracefully before Kubernetes kills it.

12.12. Application Information

Application information exposes various information collected from all `javadoc:org.springframework.boot.actuate.info.InfoContributor[]` beans defined in your `javadoc:org.springframework.context.ApplicationContext[]`. Spring Boot includes a number of auto-configured `javadoc:org.springframework.boot.actuate.info.InfoContributor[]` beans, and you can write your own.

12.12.1. Auto-configured InfoContributors

When appropriate, Spring auto-configures the following `javadoc:org.springframework.boot.actuate.info.InfoContributor[]` beans:

ID	Name	Description	Prerequisites
build	<code>javadoc:org.springframework.boot.actuate.info.BuildInfoContributor[]</code>	Exposes build information.	A <code>META-INF/build-info.properties</code> resource.
env	<code>javadoc:org.springframework.boot.actuate.info.EnvironmentInfoContributor[]</code>	Exposes any property from the <code>javadoc:org.springframework.core.env.Environment[]</code> whose name starts with <code>info..</code>	None.
git	<code>javadoc:org.springframework.boot.actuate.info.GitInfoContributor[]</code>	Exposes git information.	A <code>git.properties</code> resource.
java	<code>javadoc:org.springframework.boot.actuate.info.JavaInfoContributor[]</code>	Exposes Java runtime information.	None.
os	<code>javadoc:org.springframework.boot.actuate.info.OsInfoContributor[]</code>	Exposes Operating System information.	None.
process	<code>javadoc:org.springframework.boot.actuate.info.ProcessInfoContributor[]</code>	Exposes process information.	None.
ssl	<code>javadoc:org.springframework.boot.actuate.info.SslInfoContributor[]</code>	Exposes SSL certificate information.	An SSL Bundle configured.

Whether an individual contributor is enabled is controlled by its `management.info.<id>.enabled` property. Different contributors have different defaults for this property, depending on their prerequisites and the nature of the information that they expose.

With no prerequisites to indicate that they should be enabled, the `env`, `java`, `os`, and `process` contributors are disabled by default. The `ssl` contributor has a prerequisite of having an [SSL Bundle](#) configured but it is disabled by default. Each can be enabled by setting its `management.info.<id>.enabled` property to `true`.

The `build` and `git` info contributors are enabled by default. Each can be disabled by setting its `management.info.<id>.enabled` property to `false`. Alternatively, to disable every contributor that is usually enabled by default, set the `configprop:management.info.defaults.enabled[]` property to `false`.

12.12.2. Custom Application Information

When the `env` contributor is enabled, you can customize the data exposed by the `info.*` Spring properties. All `javadoc:org.springframework.core.env.Environment[]` properties under the `info` key are automatically exposed. For example, you could add the following settings to your `application.properties` file:

```
info:
  app:
    encoding: "UTF-8"
    java:
      source: "17"
      target: "17"
```

Rather than hardcoding those values, you could also [expand info properties at build time](#).

Assuming you use Maven, you could rewrite the preceding example as follows:



```
info:
  app:
    encoding: "@project.build.sourceEncoding@"
    java:
      source: "@java.version@"
      target: "@java.version@"
```

12.12.3. Git Commit Information

Another useful feature of the `info` endpoint is its ability to publish information about the state of your `git` source code repository when the project was built. If a `javadoc:org.springframework.boot.info.GitProperties[]` bean is available, you can use the `info` endpoint to expose these properties.



A `javadoc:org.springframework.boot.info.GitProperties[]` bean is auto-configured if a `git.properties` file is available at the root of the classpath. See [how-to:build.html](#) for more detail.

By default, the endpoint exposes `git.branch`, `git.commit.id`, and `git.commit.time` properties, if present. If you do not want any of these properties in the endpoint response, they need to be excluded from the `git.properties` file. If you want to display the full git information (that is, the full content of `git.properties`), use the `configprop:management.info.git.mode[]` property, as follows:

```
management:
  info:
    git:
      mode: "full"
```

To disable the git commit information from the `info` endpoint completely, set the `configprop:management.info.git.enabled[]` property to `false`, as follows:

```
management:
  info:
    git:
      enabled: false
```

12.12.4. Build Information

If a `javadoc:org.springframework.boot.info.BuildProperties[]` bean is available, the `info` endpoint can also publish information about your build. This happens if a `META-INF/build-info.properties` file is available in the classpath.



The Maven and Gradle plugins can both generate that file. See [how-to:build.html](#) for more details.

12.12.5. Java Information

The `info` endpoint publishes information about your Java runtime environment, see `javadoc:org.springframework.boot.info.JavaInfo[]` for more details.

12.12.6. OS Information

The `info` endpoint publishes information about your Operating System, see `javadoc:org.springframework.boot.info.OsInfo[]` for more details.

12.12.7. Process Information

The `info` endpoint publishes information about your process, see `javadoc:org.springframework.boot.info.ProcessInfo[]` for more details.

12.12.8. SSL Information

The `info` endpoint publishes information about your SSL certificates (that are configured through [SSL Bundles](#)), see `javadoc:org.springframework.boot.info.SslInfo[]` for more details.

12.12.9. Writing Custom InfoContributors

To provide custom application information, you can register Spring beans that implement the `javadoc:org.springframework.boot.actuate.info.InfoContributor[]` interface.

The following example contributes an `example` entry with a single value:

```
include-code::MyInfoContributor[]
```

If you reach the `info` endpoint, you should see a response that contains the following additional entry:

```
{
  "example": {
    "key" : "value"
  }
}
```

JSON

12.13. Software Bill of Materials (SBOM)

The `sbom` endpoint exposes the [Software Bill of Materials](#) (https://en.wikipedia.org/wiki/Software_supply_chain). CycloneDX SBOMs can be auto-detected, but other formats can be manually configured, too.

The `sbom` actuator endpoint will then expose an SBOM called "application", which describes the contents of your application.



To automatically generate a CycloneDX SBOM at project build time, please see the [how-to:build.html](#) section.

12.13.1. Other SBOM formats

If you want to publish an SBOM in a different format, there are some configuration properties which you can use.

The configuration property `configprop:management.endpoint.sbom.application.location[]` sets the location for the application SBOM. For example, setting this to `classpath:sbom.json` will use the contents of the `/sbom.json` resource on the classpath.

The media type for SBOMs in CycloneDX, SPDX and Syft format is detected automatically. To override the auto-detected media type, use the configuration property `configprop:management.endpoint.sbom.application.media-type[]`.

12.13.2. Additional SBOMs

The actuator endpoint can handle multiple SBOMs. To add SBOMs, use the configuration property `configprop:management.endpoint.sbom.additional[]`, as shown in this example:

```
management:
  endpoint:
    sbom:
      additional:
        system:
          location: "optional:file:/system.spdx.json"
          media-type: "application/spdx+json"
```

This will add an SBOM called "system", which is stored in `/system.spdx.json`. The `optional:` prefix can be used to prevent a startup failure if the file doesn't exist.

13. Recording HTTP Exchanges

You can enable recording of HTTP exchanges by providing a bean of type `javax.servlet.http.HttpServletRequest` in your application's configuration. For convenience, Spring Boot offers `javax.servlet.http.HttpServletRequest`, which, by default, stores the last 100 request-response exchanges. `javax.servlet.http.HttpServletRequest` is limited compared to tracing solutions, and we recommend using it only for development environments. For production environments, we recommend using a production-ready tracing or observability solution, such as Zipkin or OpenTelemetry. Alternatively, you can create your own `javax.servlet.http.HttpServletRequest`.

You can use the `httpexchanges` endpoint to obtain information about the request-response exchanges that are stored in the `javax.servlet.http.HttpServletRequest`.

13.1. Custom HTTP Exchange Recording

To customize the items that are included in each recorded exchange, use the `configprops:management.httpexchanges.recording.include[]` configuration property.

To disable recording entirely, set `configprops:management.httpexchanges.recording.enabled[]` to `false`.

14. Production-ready Features

Spring Boot includes a number of additional features to help you monitor and manage your application when you push it to production. You can choose to manage and monitor your application by using HTTP endpoints or with JMX. Auditing, health, and metrics gathering can also be automatically applied to your application.

15. Monitoring and Management over JMX

Java Management Extensions (JMX) provide a standard mechanism to monitor and manage applications. By default, this feature is not enabled. You can turn it on by setting the `configprop:spring.jmx.enabled[]` configuration property to `true`. Spring Boot exposes the most suitable `javadoc:javax.management.MBeanServer[]` as a bean with an ID of `mbeanServer`. Any of your beans that are annotated with Spring JMX annotations (`@org.springframework.jmx.export.annotation.ManagedResource`, `javadoc:org.springframework.jmx.export.annotation.ManagedAttribute[format=annotation]`, or `javadoc:org.springframework.jmx.export.annotation.ManagedOperation[format=annotation]`) are exposed to it.

If your platform provides a standard `javadoc:javax.management.MBeanServer[]`, Spring Boot uses that and defaults to the VM `javadoc:javax.management.MBeanServer[]`, if necessary. If all that fails, a new `javadoc:javax.management.MBeanServer[]` is created.

 **spring.jmx.enabled** affects only the management beans provided by Spring. Enabling management beans provided by other libraries (for example `{url-log4j2-docs}/jmx.html[Log4j2]` or `{url-quartz-javadoc}/constant-values.html#org.quartz.impl.StdSchedulerFactory.PROP_SCHED_JMX_EXPORT[Quartz]`) is independent.

See the `{code-spring-boot-autoconfigure-src}/jmx/JmxAutoConfiguration.java[ JmxAutoConfiguration ]` class for more details.

By default, Spring Boot also exposes management endpoints as JMX MBeans under the `org.springframework.boot` domain. To take full control over endpoint registration in the JMX domain, consider registering your own `javadoc:org.springframework.boot.actuate.endpoint.jmx.EndpointObjectNameFactory[]` implementation.

15.1. Customizing MBean Names

The name of the MBean is usually generated from the `id` of the endpoint. For example, the `health` endpoint is exposed as `org.springframework.boot:type=Endpoint,name=Health`.

If your application contains more than one Spring `javadoc:org.springframework.context.ApplicationContext[]`, you may find that names clash. To solve this problem, you can set the `configprop:spring.jmx.unique-names[]` property to `true` so that MBean names are always unique.

You can also customize the JMX domain under which endpoints are exposed. The following settings show an example of doing so in `application.properties`:

```
spring:
  jmx:
    unique-names: true
management:
  endpoints:
    jmx:
      domain: "com.example.myapp"
```

15.2. Disabling JMX Endpoints

If you do not want to expose endpoints over JMX, you can set the `configprop:management.endpoints.jmx.exposure.exclude[]` property to `*`, as the following example shows:

```
management:
  endpoints:
    jmx:
      exposure:
        exclude: "*"

```

16. Loggers

Spring Boot Actuator includes the ability to view and configure the log levels of your application at runtime. You can view either the entire list or an individual logger's configuration, which is made up of both the explicitly configured logging level as well as the effective logging level given to it by the logging framework. These levels can be one of:

- TRACE
- DEBUG
- INFO
- WARN
- ERROR
- FATAL
- OFF
- null

null indicates that there is no explicit configuration.

16.1. Configure a Logger

To configure a given logger, POST a partial entity to the resource's URI, as the following example shows:

```
{
  "configuredLevel": "DEBUG"
}
```

JSON

To “reset” the specific level of the logger (and use the default configuration instead), you can pass a value of null as the configuredLevel .

16.2. OpenTelemetry

By default, logging via OpenTelemetry is not configured. You have to provide the location of the OpenTelemetry logs endpoint to configure it:

```
management:
  opentelemetry:
    logging:
      export:
        otlp:
          endpoint: "https://otlp.example.com:4318/v1/logs"
```



The OpenTelemetry Logback appender and Log4j appender are not part of Spring Boot. For more details, see the [OpenTelemetry Logback appender](https://github.com/open-telemetry/opentelemetry-java-instrumentation/tree/main/instrumentation/logback/logback-appender-1.0/library) (https://github.com/open-telemetry/opentelemetry-java-instrumentation/tree/main/instrumentation/logback/logback-appender-1.0/library) or the [OpenTelemetry Log4j2 appender](https://github.com/open-telemetry/opentelemetry-java-instrumentation/tree/main/instrumentation/log4j/log4j-appender-2.17/library) (https://github.com/open-telemetry/opentelemetry-java-instrumentation/tree/main/instrumentation/log4j/log4j-appender-2.17/library) in the [OpenTelemetry Java instrumentation GitHub repository](https://github.com/open-telemetry/opentelemetry-java-instrumentation) (https://github.com/open-telemetry/opentelemetry-java-instrumentation).



You have to configure the appender in your logback-spring.xml or log4j2-spring.xml configuration to get OpenTelemetry logging working.

The OpenTelemetryAppender for both Logback and Log4j requires access to an javadoc:io.opentelemetry.api.OpenTelemetry[] instance to function properly. This instance must be set programmatically during application startup, which can be done like this:

```
include-code::OpenTelemetryAppenderInitializer[]
```

17. Metrics

Spring Boot Actuator provides dependency management and auto-configuration for [Micrometer](#), an application metrics facade that supports [various monitoring systems](#), including:

- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#) (in-memory)
- [actuator/metrics.html](#)
- [actuator/metrics.html](#)



To learn more about Micrometer's capabilities, see its [reference documentation](#), in particular the [concepts](#) section.

17.1. Getting Started

Spring Boot auto-configures a composite `io.micrometer.core.instrument.MeterRegistry[]` and adds a registry to the composite for each of the supported implementations that it finds on the classpath. Having a dependency on `micrometer-registry-{system}` in your runtime classpath is enough for Spring Boot to configure the registry.

Most registries share common features. For instance, you can disable a particular registry even if the Micrometer registry implementation is on the classpath. The following example disables Datadog:

```
management:
  datadog:
    metrics:
      export:
        enabled: false
```

You can also disable all registries unless stated otherwise by the registry-specific property, as the following example shows:

```
management:
  defaults:
    metrics:
      export:
        enabled: false
```

Spring Boot also adds any auto-configured registries to the global static composite registry on the `io.micrometer.core.instrument.Metrics[]` class, unless you explicitly tell it not to:

```
management:
  metrics:
    use-global-registry: false
```

You can register any number of `org.springframework.boot.micrometer.metrics.autoconfigure.MeterRegistryCustomizer[]` beans to further configure the registry, such as applying common tags, before any meters are registered with the registry:

`include-code::commontags/MyMeterRegistryConfiguration[]`

You can apply customizations to particular registry implementations by being more specific about the generic type:

`include-code::specificitytype/MyMeterRegistryConfiguration[]`

Spring Boot also [configures built-in instrumentation](#) that you can control through configuration or dedicated annotation markers.

17.2. Supported Monitoring Systems

This section briefly describes each of the supported monitoring systems.

17.2.1. AppOptics

By default, the AppOptics registry periodically pushes metrics to `api.appoptics.com/v1/measurements`. To export metrics to SaaS {url-micrometer-docs-implementations}/appOptics[AppOptics], your API token must be provided:

```
management:
  appoptics:
    metrics:
      export:
        api-token: "YOUR_TOKEN"
```

17.2.2. Atlas

By default, metrics are exported to {url-micrometer-docs-implementations}/atlas[Atlas] running on your local machine. You can provide the location of the [Atlas server](https://github.com/Netflix/atlas) (<https://github.com/Netflix/atlas>):

```
management:
  atlas:
    metrics:
      export:
        uri: "https://atlas.example.com:7101/api/v1/publish"
```

17.2.3. Datadog

A Datadog registry periodically pushes metrics to [datadoghq](https://www.datadoghq.com) (<https://www.datadoghq.com>). To export metrics to {url-micrometer-docs-implementations}/datadog[Datadog], you must provide your API key:

```
management:
  datadog:
    metrics:
      export:
        api-key: "YOUR_KEY"
```

If you additionally provide an application key (optional), then metadata such as meter descriptions, types, and base units will also be exported:

```
management:
  datadog:
    metrics:
      export:
        api-key: "YOUR_API_KEY"
        application-key: "YOUR_APPLICATION_KEY"
```

By default, metrics are sent to the Datadog US [site](https://docs.datadoghq.com/getting_started/site) (https://docs.datadoghq.com/getting_started/site) (`api.datadoghq.com`). If your Datadog project is hosted on one of the other sites, or you need to send metrics through a proxy, configure the URI accordingly:

```
management:
  datadog:
    metrics:
      export:
        uri: "https://api.datadoghq.eu"
```

You can also change the interval at which metrics are sent to Datadog:

```
management:
  datadog:
    metrics:
      export:
        step: "30s"
```

17.2.4. Dynatrace

Dynatrace offers two metrics ingest APIs, both of which are implemented for {url-micrometer-docs-implementations}/dynatrace[Micrometer]. You can find the Dynatrace documentation on Micrometer metrics ingest {url-dynatrace-docs-shortlink}/micrometer-metrics-ingest[here]. Configuration properties in the `v1` namespace apply only when exporting to the {url-dynatrace-docs-shortlink}/api-metrics[Timeseries v1 API]. Configuration properties in the `v2` namespace apply only when exporting to the {url-dynatrace-docs-shortlink}/api-metrics-v2-post-datapoints[Metrics v2 API]. Note that this integration can export only to either the `v1` or `v2` version of the API at a time, with `v2` being preferred. If the `device-id` (required for `v1` but not used in `v2`) is set in the `v1` namespace, metrics are exported to the `v1` endpoint. Otherwise, `v2` is assumed.

v2 API

You can use the v2 API in two ways.

Auto-configuration

Dynatrace auto-configuration is available for hosts that are monitored by the OneAgent or by the Dynatrace Operator for Kubernetes.

Local OneAgent: If a OneAgent is running on the host, metrics are automatically exported to the {url-dynatrace-docs-shortlink}/local-api[local OneAgent ingest endpoint]. The ingest endpoint forwards the metrics to the Dynatrace backend.

Dynatrace Kubernetes Operator: When running in Kubernetes with the Dynatrace Operator installed, the registry will automatically pick up your endpoint URI and API token from the operator instead.

This is the default behavior and requires no special setup beyond a dependency on `io.micrometer:micrometer-registry-dynatrace`.

Manual Configuration

If no auto-configuration is available, the endpoint of the `{url-dynatrace-docs-shortlink}/api-metrics-v2-post-datapoints` [Metrics v2 API] and an API token are required. The `{url-dynatrace-docs-shortlink}/api-authentication` [API token] must have the “Ingest metrics” (`metrics.ingest`) permission set. We recommend limiting the scope of the token to this one permission. You must ensure that the endpoint URI contains the path (for example, `/api/v2/metrics/ingest`):

The URL of the Metrics API v2 ingest endpoint is different according to your deployment option:

- SaaS: `https://{your-environment-id}.live.dynatrace.com/api/v2/metrics/ingest`
- Managed deployments: `https://{your-domain}/e/{your-environment-id}/api/v2/metrics/ingest`

The example below configures metrics export using the `example` environment id:

```
management:
  dynatrace:
    metrics:
      export:
        uri: "https://example.live.dynatrace.com/api/v2/metrics/ingest"
        api-token: "YOUR_TOKEN"
```

When using the Dynatrace v2 API, the following optional features are available (more details can be found in the `{url-dynatrace-docs-shortlink}/micrometer-metrics-ingest#dt-configuration-properties` [Dynatrace documentation]):

- Metric key prefix: Sets a prefix that is prepended to all exported metric keys.
- Enrich with Dynatrace metadata: If a OneAgent or Dynatrace operator is running, enrich metrics with additional metadata (for example, about the host, process, or pod).
- Default dimensions: Specify key-value pairs that are added to all exported metrics. If tags with the same key are specified with Micrometer, they overwrite the default dimensions.
- Use Dynatrace Summary instruments: In some cases the Micrometer Dynatrace registry created metrics that were rejected. In Micrometer 1.9.x, this was fixed by introducing Dynatrace-specific summary instruments. Setting this toggle to `false` forces Micrometer to fall back to the behavior that was the default before 1.9.x. It should only be used when encountering problems while migrating from Micrometer 1.8.x to 1.9.x.
- Export meter metadata: Starting from Micrometer 1.12.0, the Dynatrace exporter will also export meter metadata, such as unit and description by default. Use the `export-meter-metadata` toggle to turn this feature off.

It is possible to not specify a URI and API token, as shown in the following example. In this scenario, the automatically configured endpoint is used:

```
management:
  dynatrace:
    metrics:
      export:
        # Specify uri and api-token here if not using the local OneAgent endpoint.
        v2:
          metric-key-prefix: "your.key.prefix"
          enrich-with-dynatrace-metadata: true
          default-dimensions:
            key1: "value1"
            key2: "value2"
          use-dynatrace-summary-instruments: true # (default: true)
          export-meter-metadata: true             # (default: true)
```

v1 API (Legacy)

The Dynatrace v1 API metrics registry pushes metrics to the configured URI periodically by using the `{url-dynatrace-docs-shortlink}/api-metrics` [Timeseries v1 API]. For backwards-compatibility with existing setups, when `device-id` is set (required for v1, but not used in v2), metrics are exported to the Timeseries v1 endpoint. To export metrics to `{url-micrometer-docs-implementations}/dynatrace` [Dynatrace], your API token, device ID, and URI must be provided:

```
management:
  dynatrace:
    metrics:
      export:
        uri: "https://{your-environment-id}.live.dynatrace.com"
        api-token: "YOUR_TOKEN"
        v1:
          device-id: "YOUR_DEVICE_ID"
```

For the v1 API, you must specify the base environment URI without a path, as the v1 endpoint path is added automatically.

Version-independent Settings

In addition to the API endpoint and token, you can also change the interval at which metrics are sent to Dynatrace. The default export interval is `60s`. The following example sets the export interval to 30 seconds:


```
management:
  dynatrace:
    metrics:
      export:
        step: "30s"
```

You can find more information on how to set up the Dynatrace exporter for Micrometer in the {url-micrometer-docs-implementations}/dynatrace[Micrometer documentation] and the {url-dynatrace-docs-shortlink}/micrometer-metrics-ingest[Dynatrace documentation].

17.2.5. Elastic

By default, metrics are exported to {url-micrometer-docs-implementations}/elastic[Elastic] running on your local machine. You can provide the location of the Elastic server to use by using the following property:

```
management:
  elastic:
    metrics:
      export:
        host: "https://elastic.example.com:8086"
```

17.2.6. Ganglia

By default, metrics are exported to {url-micrometer-docs-implementations}/ganglia[Ganglia] running on your local machine. You can provide the [Ganglia server](http://ganglia.sourceforge.net) (http://ganglia.sourceforge.net) host and port, as the following example shows:

```
management:
  ganglia:
    metrics:
      export:
        host: "ganglia.example.com"
        port: 9649
```

17.2.7. Graphite

By default, metrics are exported to {url-micrometer-docs-implementations}/graphite[Graphite] running on your local machine. You can provide the [Graphite server](https://graphiteapp.org) (https://graphiteapp.org) host and port, as the following example shows:

```
management:
  graphite:
    metrics:
      export:
        host: "graphite.example.com"
        port: 9004
```

Micrometer provides a default `javadoc:io.micrometer.core.instrument.util.HierarchicalNameMapper[]` that governs how a dimensional meter ID is {url-micrometer-docs-implementations}/graphite#_hierarchical_name_mapping[mapped to flat hierarchical names].



To take control over this behavior, define your `javadoc:io.micrometer.graphite.GraphiteMeterRegistry[]` and supply your own `javadoc:io.micrometer.core.instrument.util.HierarchicalNameMapper[]`. Auto-configured `javadoc:io.micrometer.graphite.GraphiteConfig[]` and `javadoc:io.micrometer.core.instrument.Clock[]` beans are provided unless you define your own:

```
include-code::MyGraphiteConfiguration[]
```

17.2.8. Humio

By default, the Humio registry periodically pushes metrics to cloud.humio.com. To export metrics to SaaS {url-micrometer-docs-implementations}/humio[Humio], you must provide your API token:

```
management:
  humio:
    metrics:
      export:
        api-token: "YOUR_TOKEN"
```

You should also configure one or more tags to identify the data source to which metrics are pushed:

```
management:
  humio:
    metrics:
      export:
        tags:
          alpha: "a"
          bravo: "b"
```

17.2.9. Influx

By default, metrics are exported to an {url-micrometer-docs-implementations}/influx[Influx] v1 instance running on your local machine with the default configuration. To export metrics to InfluxDB v2, configure the `org`, `bucket`, and authentication `token` for writing metrics. You can provide the location of the [Influx server](https://www.influxdata.com) (https://www.influxdata.com) to use by using:

```
management:
  influx:
    metrics:
      export:
        uri: "https://influx.example.com:8086"
```

17.2.10. JMX

Micrometer provides a hierarchical mapping to {url-micrometer-docs-implementations}/jmx[JMX], primarily as a cheap and portable way to view metrics locally. By default, metrics are exported to the `metrics` JMX domain. You can provide the domain to use by using:

```
management:
  jmx:
    metrics:
      export:
        domain: "com.example.app.metrics"
```

Micrometer provides a default `javadoc:io.micrometer.core.instrument.util.HierarchicalNameMapper[]` that governs how a dimensional meter ID is {url-micrometer-docs-implementations}/jmx#_hierarchical_name_mapping[mapped to flat hierarchical names].



To take control over this behavior, define your `javadoc:io.micrometer.jmx.JmxMeterRegistry[]` and supply your own `javadoc:io.micrometer.core.instrument.util.HierarchicalNameMapper[]`. Auto-configured `javadoc:io.micrometer.jmx.JmxConfig[]` and `javadoc:io.micrometer.core.instrument.Clock[]` beans are provided unless you define your own:

```
include-code::MyJmxConfiguration[]
```

17.2.11. KairosDB

By default, metrics are exported to {url-micrometer-docs-implementations}/kairos[KairosDB] running on your local machine. You can provide the location of the [KairosDB server](https://kairosdb.github.io/) (https://kairosdb.github.io/) to use by using:

```
management:
  kairos:
    metrics:
      export:
        uri: "https://kairosdb.example.com:8080/api/v1/datapoints"
```

17.2.12. New Relic

A New Relic registry periodically pushes metrics to {url-micrometer-docs-implementations}/new-relic[New Relic]. To export metrics to [New Relic](https://newrelic.com/) (https://newrelic.com), you must provide your API key and account ID:

```
management:
  newrelic:
    metrics:
      export:
        api-key: "YOUR_KEY"
        account-id: "YOUR_ACCOUNT_ID"
```

You can also change the interval at which metrics are sent to New Relic:

```
management:
  newrelic:
    metrics:
      export:
        step: "30s"
```

By default, metrics are published through REST calls, but you can also use the Java Agent API if you have it on the classpath:

```
management:
  newrelic:
    metrics:
      export:
        client-provider-type: "insights-agent"
```

Finally, you can take full control by defining your own `javadoc:io.micrometer.newrelic.NewRelicClientProvider[]` bean.

17.2.13. OTLP

By default, metrics are exported over the {url-micrometer-docs-implementations}/otlp[OpenTelemetry protocol (OTLP)] to a consumer running on your local machine. To export to another location, provide the location of the [OTLP metrics endpoint](https://opentelemetry.io/) (https://opentelemetry.io/) using `configprop:management.otlp.metrics.export.url[]`:

```
management:
  otlp:
    metrics:
      export:
        url: "https://otlp.example.com:4318/v1/metrics"
```

Custom headers, for example for authentication, can also be provided using `configprop:management.otlp.metrics.export.headers.*[]` properties.

If an `OtlpMetricsSender` bean is available, it will be configured on the `OtlpMeterRegistry` that Spring Boot auto-configures.

17.2.14. Prometheus

{url-micrometer-docs-implementations}/prometheus[Prometheus] expects to scrape or poll individual application instances for metrics. Spring Boot provides an actuator endpoint at `/actuator/prometheus` to present a [Prometheus scrape](https://prometheus.io) (https://prometheus.io) with the appropriate format.



By default, the endpoint is not available and must be exposed. See [exposing endpoints](#) for more details.

The following example `scrape_config` adds to `prometheus.yml`:

YAML

```
scrape_configs:
- job_name: "spring"
  metrics_path: "/actuator/prometheus"
  static_configs:
  - targets: ["HOST:PORT"]
```

[Prometheus Exemplars](https://prometheus.io/docs/prometheus/latest/feature_flags/#exemplars-storage) (https://prometheus.io/docs/prometheus/latest/feature_flags/#exemplars-storage) are also supported. To enable this feature, a `javadoc:io.prometheus.metrics.tracer.common.SpanContext[]` bean should be present. If you're using the deprecated Prometheus simpleclient support and want to enable that feature, a `javadoc:io.prometheus.client.exemplars.tracer.common.SpanContextSupplier[]` bean should be present. If you use {url-micrometer-tracing-docs}[Micrometer Tracing], this will be auto-configured for you, but you can always create your own if you want. Please check the [Prometheus Docs](https://prometheus.io/docs/prometheus/latest/feature_flags/#exemplars-storage) (https://prometheus.io/docs/prometheus/latest/feature_flags/#exemplars-storage), since this feature needs to be explicitly enabled on Prometheus' side, and it is only supported using the [OpenMetrics](https://github.com/OpenObservability/OpenMetrics/blob/v1.0.0/specification/OpenMetrics.md#exemplars) (https://github.com/OpenObservability/OpenMetrics/blob/v1.0.0/specification/OpenMetrics.md#exemplars) format.

For ephemeral or batch jobs that may not exist long enough to be scraped, you can use [Prometheus Pushgateway](https://github.com/prometheus/pushgateway) (https://github.com/prometheus/pushgateway) support to expose the metrics to Prometheus.

To enable Prometheus Pushgateway support, add the following dependency to your project:

XML

```
<dependency>
<groupId>io.prometheus</groupId>
<artifactId>prometheus-metrics-exporter-pushgateway</artifactId>
</dependency>
```

When the Prometheus Pushgateway dependency is present on the classpath and the `configprop:management.prometheus.metrics.export.pushgateway.enabled[]` property is set to `true`, a `javadoc:org.springframework.boot.micrometer.metrics.export.prometheus.PrometheusPushGatewayManager[]` bean is auto-configured. This manages the pushing of metrics to a Prometheus Pushgateway.

You can tune the `javadoc:org.springframework.boot.micrometer.metrics.export.prometheus.PrometheusPushGatewayManager[]` by using properties under `management.prometheus.metrics.export.pushgateway`. For advanced configuration, you can also provide your own `javadoc:org.springframework.boot.micrometer.metrics.export.prometheus.PrometheusPushGatewayManager[]` bean.

17.2.15. Simple

Micrometer ships with a simple, in-memory backend that is automatically used as a fallback if no other registry is configured. This lets you see what metrics are collected in the [metrics endpoint](#).

The in-memory backend disables itself as soon as you use any other available backend. You can also disable it explicitly:

```
management:
  simple:
    metrics:
      export:
        enabled: false
```

17.2.16. Stackdriver

The Stackdriver registry periodically pushes metrics to [Stackdriver](https://cloud.google.com/stackdriver/) (https://cloud.google.com/stackdriver/). To export metrics to SaaS {url-micrometer-docs-implementations}/stackdriver[Stackdriver], you must provide your Google Cloud project ID:

```
management:
  stackdriver:
    metrics:
      export:
        project-id: "my-project"
```

You can also change the interval at which metrics are sent to Stackdriver:

```
management:
  stackdriver:
    metrics:
      export:
        step: "30s"
```

17.2.17. StatsD

The StatsD registry eagerly pushes metrics over UDP to a StatsD agent. By default, metrics are exported to a `{url-micrometer-docs-implementations}/statsD[StatsD]` agent running on your local machine. You can provide the StatsD agent host, port, and protocol to use by using:

```
management:
  statsd:
    metrics:
      export:
        host: "statsd.example.com"
        port: 9125
        protocol: "udp"
```

You can also change the StatsD line protocol to use (it defaults to Datadog):

```
management:
  statsd:
    metrics:
      export:
        flavor: "etsy"
```

17.3. Supported Metrics and Meters

Spring Boot provides automatic meter registration for a wide variety of technologies. In most situations, the defaults provide sensible metrics that can be published to any of the supported monitoring systems.

17.3.1. JVM Metrics

Auto-configuration enables JVM Metrics by using core Micrometer classes. JVM metrics are published under the `jvm.` meter name.

The following JVM metrics are provided:

- Various memory and buffer pool details
- Statistics related to garbage collection
- Thread utilization
- [Virtual threads statistics](https://docs.micrometer.io/micrometer/reference/reference/jvm.html#_java_21_metrics) (https://docs.micrometer.io/micrometer/reference/reference/jvm.html#_java_21_metrics) (for this, `io.micrometer:micrometer-java21` has to be on the classpath)
- The number of classes loaded and unloaded
- JVM version information
- JIT compilation time

17.3.2. System Metrics

Auto-configuration enables system metrics by using core Micrometer classes. System metrics are published under the `system.`, `process.`, and `disk.` meter names.

The following system metrics are provided:

- CPU metrics
- File descriptor metrics
- Uptime metrics (both the amount of time the application has been running and a fixed gauge of the absolute start time)
- Disk space available

17.3.3. Application Startup Metrics

Auto-configuration exposes application startup time metrics:

- `application.started.time`: time taken to start the application.
- `application.ready.time`: time taken for the application to be ready to service requests.

Metrics are tagged by the fully qualified name of the application class.

17.3.4. Logger Metrics

Auto-configuration enables the event metrics for both Logback and Log4j2. The details are published under the `log4j2.events.` or `logback.events.` meter names.

17.3.5. Task Execution and Scheduling Metrics

Auto-configuration enables the instrumentation of all available `javadoc:org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor[]` and `javadoc:org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler[]` beans, as long as the underlying `javadoc:java.util.concurrent.ThreadPoolExecutor[]` is available. Metrics are tagged by the name of the executor, which is derived from the bean name.

17.3.6. JMS Metrics

Auto-configuration enables the instrumentation of all available `javadoc:org.springframework.jms.core.JmsTemplate[]` beans and `javadoc:org.springframework.jms.annotation.JmsListener[format=annotation]` annotated methods. This will produce `"jms.message.publish"` and `"jms.message.process"` metrics respectively. See the [\[url-spring-framework-docs/integration/observability.html#observability-jms\]](https://spring-framework-docs/integration/observability.html#observability-jms) (Spring Framework reference documentation for more information on produced observations).



`javadoc:org.springframework.jms.core.JmsClient[]` and `javadoc:org.springframework.jms.core.JmsMessagingTemplate[]` that uses a `javadoc:org.springframework.jms.core.JmsTemplate[]` bean are also instrumented.

17.3.7. Spring MVC Metrics

Auto-configuration enables the instrumentation of all requests handled by Spring MVC controllers and functional handlers. By default, metrics are generated with the name, `http.server.requests`. You can customize the name by setting the `configprop:management.observations.http.server.requests.name[]` property.

See the [{url-spring-framework-docs}/integration/observability.html#observability.http-server.servlet](#) (Spring Framework reference documentation for more information on produced observations).

To add to the default tags, provide a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` that extends `javadoc:org.springframework.http.server.observation.DefaultServerRequestObservationConvention[]` from the `org.springframework.http.server.observation` package. To replace the default tags, provide a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` that implements `javadoc:org.springframework.http.server.observation.ServerRequestObservationConvention[]`.



In some cases, exceptions handled in web controllers are not recorded as request metrics tags. Applications can opt in and record exceptions by [setting handled exceptions as request attributes](#).

By default, all requests are handled. To customize the filter, provide a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` that implements `FilterRegistrationBean<ServerHttpObservationFilter>`.

17.3.8. Spring WebFlux Metrics

Auto-configuration enables the instrumentation of all requests handled by Spring WebFlux controllers and functional handlers. By default, metrics are generated with the name, `http.server.requests`. You can customize the name by setting the `configprop:management.observations.http.server.requests.name[]` property.

See the [{url-spring-framework-docs}/integration/observability.html#observability.http-server.reactive](#) (Spring Framework reference documentation for more information on produced observations).

To add to the default tags, provide a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` that extends `javadoc:org.springframework.http.server.reactive.observation.DefaultServerRequestObservationConvention[]` from the `org.springframework.http.server.reactive.observation` package. To replace the default tags, provide a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` that implements `javadoc:org.springframework.http.server.reactive.observation.ServerRequestObservationConvention[]`.



In some cases, exceptions handled in controllers and handler functions are not recorded as request metrics tags. Applications can opt in and record exceptions by [setting handled exceptions as request attributes](#).

17.3.9. Jersey Server Metrics

Auto-configuration enables the instrumentation of all requests handled by the Jersey JAX-RS implementation. By default, metrics are generated with the name, `http.server.requests`. You can customize the name by setting the `configprop:management.observations.http.server.requests.name[]` property.

By default, Jersey server metrics are tagged with the following information:

Tag	Description
<code>exception</code>	The simple class name of any exception that was thrown while handling the request.
<code>method</code>	The request's method (for example, <code>GET</code> or <code>POST</code>)
<code>outcome</code>	The request's outcome, based on the status code of the response. 1xx is <code>INFORMATIONAL</code> , 2xx is <code>SUCCESS</code> , 3xx is <code>REDIRECTION</code> , 4xx is <code>CLIENT_ERROR</code> , and 5xx is <code>SERVER_ERROR</code>
<code>status</code>	The response's HTTP status code (for example, <code>200</code> or <code>500</code>)
<code>uri</code>	The request's URI template prior to variable substitution, if possible (for example, <code>/api/person/{id}</code>)

To customize the tags, provide a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` that implements `javadoc:io.micrometer.core.instrument.binder.jersey.server.JerseyObservationConvention[]`.

17.3.10. SSL Bundle Metrics

Spring Boot Actuator publishes expiry metrics about SSL bundles. The metric `ssl.chain.expiry` gauges the expiry date of each certificate chain in seconds. This number will be negative if the chain has already expired. This metric is tagged with the following information:

Tag	Description
<code>bundle</code>	The name of the bundle which contains the certificate chain
<code>certificate</code>	The serial number (in hex format) of the certificate which is the soonest to expire in the chain
<code>chain</code>	The name of the certificate chain.

17.3.11. HTTP Client Metrics

Spring Boot Actuator manages the instrumentation of `javax.servlet.http.HttpServlet`, `javax.servlet.http.HttpServlet`, `javax.servlet.http.HttpServlet` and `javax.servlet.http.HttpServlet`. For that, you have to inject the auto-configured builder and use it to create instances:

- `javax.servlet.http.HttpServlet` for `javax.servlet.http.HttpServlet`
- `javax.servlet.http.HttpServlet` for `javax.servlet.http.HttpServlet`
- `javax.servlet.http.HttpServlet` for `javax.servlet.http.HttpServlet`

You can also manually apply the customizers responsible for this instrumentation, namely `javax.servlet.http.HttpServlet`, `javax.servlet.http.HttpServlet` and `javax.servlet.http.HttpServlet`.

By default, metrics are generated with the name, `http.client.requests`. You can customize the name by setting the `configprop:management.observations.http.client.requests.name` property.

See the [{url-spring-framework-docs}/integration/observability.html#observability.http-client](#) [Spring Framework reference documentation for more information on produced observations].

To customize the tags when using `javax.servlet.http.HttpServlet` or `javax.servlet.http.HttpServlet`, provide a `javax.servlet.http.HttpServlet` that implements `javax.servlet.http.HttpServlet` from the `org.springframework.http.client.observability` package. To customize the tags when using `javax.servlet.http.HttpServlet`, provide a `javax.servlet.http.HttpServlet` that implements `javax.servlet.http.HttpServlet` from the `org.springframework.web.reactive.function.client` package.

17.3.12. Tomcat Metrics

Auto-configuration enables the instrumentation of Tomcat only when an MBean `javax.servlet.http.HttpServlet` is enabled. By default, the MBean registry is disabled, but you can enable it by setting `configprop:server.tomcat.mbeanregistry.enabled` to `true`.

Tomcat metrics are published under the `tomcat` meter name.

17.3.13. Cache Metrics

Auto-configuration enables the instrumentation of all available `javax.servlet.http.HttpServlet` instances on startup, with metrics prefixed with `cache`. Cache instrumentation is standardized for a basic set of metrics. Additional, cache-specific metrics are also available.

The following cache libraries are supported:

- Cache2k
- Caffeine
- Hazelcast
- Any compliant JCache (JSR-107) implementation
- Redis



Metrics should be enabled for the auto-configuration to pick them up. Refer to the documentation of the cache library you are using for more details.

Metrics are tagged by the name of the cache and by the name of the `javax.servlet.http.HttpServlet`, which is derived from the bean name.



Only caches that are configured on startup are bound to the registry. For caches not defined in the cache's configuration, such as caches created on the fly or programmatically after the startup phase, an explicit registration is required. A `javax.servlet.http.HttpServlet` bean is made available to make that process easier.

17.3.14. Spring Batch Metrics

See the [{url-spring-batch-docs}/monitoring-and-metrics.html](#) [Spring Batch reference documentation].

17.3.15. Spring GraphQL Metrics

See the [{url-spring-graphql-docs}/observability.html](#) [Spring GraphQL reference documentation].

17.3.16. DataSource Metrics

Auto-configuration enables the instrumentation of all available `javax.servlet.http.HttpServlet` objects with metrics prefixed with `jdbc.connections`. Data source instrumentation results in gauges that represent the currently active, idle, maximum allowed, and minimum allowed connections in the pool.

Metrics are also tagged by the name of the `javax.servlet.http.HttpServlet` computed based on the bean name.



By default, Spring Boot provides metadata for all supported data sources. You can add additional `javax.servlet.http.HttpServlet` beans if your favorite data source is not supported. See `javax.servlet.http.HttpServlet` for examples.

Also, Hikari-specific metrics are exposed with a `hikaricp` prefix. Each metric is tagged by the name of the pool (you can control it with `spring.datasource.name`).

17.3.17. Hibernate Metrics

If `org.hibernate.orm:hibernate-micrometer` is on the classpath, all available Hibernate `javax.persistence.EntityManagerFactory` instances that have statistics enabled are instrumented with a metric named `hibernate`.

Metrics are also tagged by the name of the `javax.persistence.EntityManagerFactory`, which is derived from the bean name.

To enable statistics, the standard JPA property `hibernate.generate_statistics` must be set to `true`. You can enable that on the auto-configured `javax.persistence.EntityManagerFactory`:

```
spring:
  jpa:
    properties:
      "[hibernate.generate_statistics]": true
```

17.3.18. Spring Data Repository Metrics

Auto-configuration enables the instrumentation of all Spring Data `org.springframework.data.repository.Repository` method invocations. By default, metrics are generated with the name, `spring.data.repository.invocations`. You can customize the name by setting the `configprop:management.metrics.data.repository.metric-name` property.

The `javax.io.micrometer.core.annotation.Timed` annotation from the `io.micrometer.core.annotation` package is supported on `org.springframework.data.repository.Repository` interfaces and methods. If you do not want to record metrics for all `org.springframework.data.repository.Repository` invocations, you can set `configprop:management.metrics.data.repository.autotime.enabled` to `false` and exclusively use `javax.io.micrometer.core.annotation.Timed` annotations instead.



A `javax.io.micrometer.core.annotation.Timed` annotation with `longTask = true` enables a long task timer for the method. Long task timers require a separate metric name and can be stacked with a short task timer.

By default, repository invocation related metrics are tagged with the following information:

Tag	Description
repository	The simple class name of the source <code>org.springframework.data.repository.Repository</code> .
method	The name of the <code>org.springframework.data.repository.Repository</code> method that was invoked.
state	The result state (<code>SUCCESS</code> , <code>ERROR</code> , <code>CANCELED</code> , or <code>RUNNING</code>).
exception	The simple class name of any exception that was thrown from the invocation.

To replace the default tags, provide a `org.springframework.context.annotation.Bean` that implements `org.springframework.boot.data.metrics.RepositoryTagsProvider`.

17.3.19. RabbitMQ Metrics

Auto-configuration enables the instrumentation of all available RabbitMQ connection factories with a metric named `rabbitmq`.

17.3.20. Spring Integration Metrics

Spring Integration automatically provides `{url-spring-integration-docs}/metrics.html#micrometer-integration` [Micrometer support] whenever a `javax.io.micrometer.core.instrument.MeterRegistry` bean is available. Metrics are published under the `spring.integration` meter name.

17.3.21. Kafka Metrics

Auto-configuration registers a `org.springframework.kafka.core.MicrometerConsumerListener` and `org.springframework.kafka.core.MicrometerProducerListener` for the auto-configured consumer factory and producer factory, respectively. It also registers a `org.springframework.kafka.streams.KafkaStreamsMicrometerListener` for `org.springframework.kafka.config.StreamsBuilderFactoryBean`. For more detail, see the `{url-spring-kafka-docs}/kafka/micrometer.html#micrometer-native` [Micrometer Native Metrics] section of the Spring Kafka documentation.

17.3.22. MongoDB Metrics

This section briefly describes the available metrics for MongoDB.

MongoDB Command Metrics

Auto-configuration registers a `javax.io.micrometer.core.instrument.binder.mongodb.MongoMetricsCommandListener` with the auto-configured `javax.mongodb-driver-sync-javadoc/com.mongodb.client.MongoClient`.

A timer metric named `mongodb.driver.commands` is created for each command issued to the underlying MongoDB driver. Each metric is tagged with the following information by default:

Tag	Description
command	The name of the command issued.
cluster.id	The identifier of the cluster to which the command was sent.

Tag	Description
<code>server.address</code>	The address of the server to which the command was sent.
<code>status</code>	The outcome of the command (<code>SUCCESS</code> or <code>FAILED</code>).

To replace the default metric tags, define a `javado:io.micrometer.core.instrument.binder.mongodb.MongoCommandTagsProvider[]` bean, as the following example shows:

include-code::MyCommandTagsProviderConfiguration[]

To disable the auto-configured command metrics, set the following property:

```
management:
  metrics:
    mongodb:
      command:
        enabled: false
```

MongoDB Connection Pool Metrics

Auto-configuration registers a `javado:io.micrometer.core.instrument.binder.mongodb.MongoMetricsConnectionPoolListener[]` with the auto-configured `javado:{url-mongodb-driver-sync-javadoc}/com.mongodb.client.MongoClient[]`.

The following gauge metrics are created for the connection pool:

- `mongodb.driver.pool.size` reports the current size of the connection pool, including idle and in-use members.
- `mongodb.driver.pool.checkedout` reports the count of connections that are currently in use.
- `mongodb.driver.pool.waitqueuesize` reports the current size of the wait queue for a connection from the pool.

Each metric is tagged with the following information by default:

Tag	Description
<code>cluster.id</code>	The identifier of the cluster to which the connection pool corresponds.
<code>server.address</code>	The address of the server to which the connection pool corresponds.

To replace the default metric tags, define a `javado:io.micrometer.core.instrument.binder.mongodb.MongoConnectionPoolTagsProvider[]` bean:

include-code::MyConnectionPoolTagsProviderConfiguration[]

To disable the auto-configured connection pool metrics, set the following property:

```
management:
  metrics:
    mongodb:
      connectionpool:
        enabled: false
```

17.3.23. Neo4j Metrics

Auto-configuration registers a `javado:org.neo4j.driver.observation.micrometer.MicrometerObservationProvider[]` for the auto-configured `javado:org.neo4j.driver.Driver[]`.

To override this behavior, you can register a `javado:org.springframework.boot.neo4j.autoconfigure.ConfigBuilderCustomizer[]` bean with an order higher than zero.

17.3.24. Jetty Metrics

Auto-configuration binds metrics for Jetty's `javado:org.eclipse.jetty.util.thread.ThreadPool[]` by using Micrometer's `javado:io.micrometer.core.instrument.binder.jetty.JettyServerThreadPoolMetrics[]`. Metrics for Jetty's `javado:org.eclipse.jetty.server.Connector[]` instances are bound by using Micrometer's `javado:io.micrometer.core.instrument.binder.jetty.JettyConnectionMetrics[]` and, when `configprop:server.ssl.enabled[]` is set to `true`, Micrometer's `javado:io.micrometer.core.instrument.binder.jetty.JettySslHandshakeMetrics[]`.

17.3.25. Redis Metrics

Auto-configuration registers a `javado:io.lettuce.core.tracing.MicrometerTracing[]` for the auto-configured `javado:org.springframework.data.redis.connection.lettuce.LettuceConnectionFactory[]`. For more detail, see the `{url-lettuce-docs}/advanced-usage/#observability[Observability section]` of the Lettuce documentation.

17.4. Registering Custom Metrics

To register custom metrics, inject `javado:io.micrometer.core.instrument.MeterRegistry[]` into your component:

include-code::MyBean[]

If your metrics depend on other beans, we recommend that you use a `javado:io.micrometer.core.instrument.binder.MeterBinder[]` to register them:

include-code::MyMeterBinderConfiguration[]

Using a `javadoc:io.micrometer.core.instrument.binder.MeterBinder[]` ensures that the correct dependency relationships are set up and that the bean is available when the metric's value is retrieved. A `javadoc:io.micrometer.core.instrument.binder.MeterBinder[]` implementation can also be useful if you find that you repeatedly instrument a suite of metrics across components or applications.



By default, metrics from all `javadoc:io.micrometer.core.instrument.binder.MeterBinder[]` beans are automatically bound to the Spring-managed `javadoc:io.micrometer.core.instrument.MeterRegistry[]`.

17.5. Customizing Individual Metrics

If you need to apply customizations to specific `javadoc:io.micrometer.core.instrument.Meter[]` instances, you can use the `javadoc:io.micrometer.core.instrument.config.MeterFilter[]` interface.

For example, if you want to rename the `mytag.region` tag to `mytag.area` for all meter IDs beginning with `com.example`, you can do the following:

include-code::MyMetricsFilterConfiguration[]



By default, all `javadoc:io.micrometer.core.instrument.config.MeterFilter[]` beans are automatically bound to the Spring-managed `javadoc:io.micrometer.core.instrument.MeterRegistry[]`. Make sure to register your metrics by using the Spring-managed `javadoc:io.micrometer.core.instrument.MeterRegistry[]` and not any of the static methods on `javadoc:io.micrometer.core.instrument.Metrics[]`. These use the global registry that is not Spring-managed.

17.5.1. Common Tags

Common tags are generally used for dimensional drill-down on the operating environment, such as host, instance, region, stack, and others. Commons tags are applied to all meters and can be configured, as the following example shows:

```
management:
  metrics:
    tags:
      region: "us-east-1"
      stack: "prod"
```

The preceding example adds `region` and `stack` tags to all meters with a value of `us-east-1` and `prod`, respectively.



The order of common tags is important if you use Graphite. As the order of common tags cannot be guaranteed by using this approach, Graphite users are advised to define a custom `javadoc:io.micrometer.core.instrument.config.MeterFilter[]` instead.

17.5.2. Per-meter Properties

In addition to `javadoc:io.micrometer.core.instrument.config.MeterFilter[]` beans, you can apply a limited set of customization on a per-meter basis using properties. Per-meter customizations are applied, using Spring Boot's `javadoc:org.springframework.boot.micrometer.metrics.autoconfigure.PropertiesMeterFilter[]`, to any meter IDs that start with the given name. The following example filters out any meters that have an ID starting with `example.remote`.

```
management:
  metrics:
    enable:
      example:
        remote: false
```

The following properties allow per-meter customization:

Table 1. Per-meter customizations

Property	Description
configprop:management.metrics.enable[]	Whether to accept meters with certain IDs. Meters that are not accepted are filtered from the <code>javadoc:io.micrometer.core.instrument.MeterRegistry[]</code> .
configprop:management.metrics.distribution.percentiles-histogram[]	Whether to publish a histogram suitable for computing aggregable (across dimension) percentile approximations.
configprop:management.metrics.distribution.minimum-expected-value[], configprop:management.metrics.distribution.maximum-expected-value[]	Publish fewer histogram buckets by clamping the range of expected values.
configprop:management.metrics.distribution.percentiles[]	Publish percentile values computed in your application
configprop:management.metrics.distribution.expiry[], configprop:management.metrics.distribution.buffer-length[]	Give greater weight to recent samples by accumulating them in ring buffers which rotate after a configurable expiry, with a configurable buffer length.
configprop:management.metrics.distribution.slo[]	Publish a cumulative histogram with buckets defined by your service-level objectives.

For more details on the concepts behind `percentiles-histogram`, `percentiles`, and `slo`, see the `{url-micrometer-docs-concepts}/histogram-quantiles.html`[Histograms and percentiles] section of the Micrometer documentation.

17.6. Metrics Endpoint

Spring Boot provides a `metrics` endpoint that you can use diagnostically to examine the metrics collected by an application. The endpoint is not available by default and must be exposed. See [exposing endpoints](#) for more details.

Navigating to `/actuator/metrics` displays a list of available meter names. You can drill down to view information about a particular meter by providing its name as a selector—for example, `/actuator/metrics/jvm.memory.max`.



The name you use here should match the name used in the code, not the name after it has been naming-convention normalized for a monitoring system to which it is shipped. In other words, if `jvm.memory.max` appears as `jvm_memory_max` in Prometheus because of its snake case naming convention, you should still use `jvm.memory.max` as the selector when inspecting the meter in the `metrics` endpoint.

You can also add any number of `tag=KEY:VALUE` query parameters to the end of the URL to dimensionally drill down on a meter—for example, `/actuator/metrics/jvm.memory.max?tag=area:nonheap`.



The reported measurements are the *sum* of the statistics of all meters that match the meter name and any tags that have been applied. In the preceding example, the returned **Value** statistic is the sum of the maximum memory footprints of the “Code Cache”, “Compressed Class Space”, and “Metaspace” areas of the heap. If you wanted to see only the maximum size for the “Metaspace”, you could add an additional `tag=id:Metaspace`—that is, `/actuator/metrics/jvm.memory.max?tag=area:nonheap&tag=id:Metaspace`.

17.7. Integration with Micrometer Observation

A `javadoc:io.micrometer.core.instrument.observation.DefaultMeterObservationHandler[]` is automatically registered on the `javadoc:io.micrometer.observation.ObservationRegistry[]`, which creates metrics for every completed observation.

18. Monitoring and Management Over HTTP

If you are developing a web application, Spring Boot Actuator auto-configures all enabled endpoints to be exposed over HTTP. The default convention is to use the `id` of the endpoint with a prefix of `/actuator` as the URL path. For example, `health` is exposed as `/actuator/health`.



Actuator is supported natively with Spring MVC, Spring WebFlux, and Jersey. If both Jersey and Spring MVC are available, Spring MVC is used.



Jackson is a required dependency in order to get the correct JSON responses as documented in the [API documentation](#). Jackson 3 should be used for Spring MVC and Spring WebFlux. Jersey does not yet have a Jackson 3 module, so you will need to use Jackson 2.

18.1. Customizing the Management Endpoint Paths

Sometimes, it is useful to customize the prefix for the management endpoints. For example, your application might already use `/actuator` for another purpose. You can use the `configprop:management.endpoints.web.base-path[]` property to change the prefix for your management endpoint, as the following example shows:

```
management:
  endpoints:
    web:
      base-path: "/manage"
```

The preceding `application.properties` example changes the endpoint from `/actuator/{id}` to `/manage/{id}` (for example, `/manage/info`).



Unless the management port has been configured to [expose endpoints by using a different HTTP port](#), `management.endpoints.web.base-path` is relative to `server.servlet.context-path` (for servlet web applications) or `spring.webflux.base-path` (for reactive web applications). If `management.server.port` is configured, `management.endpoints.web.base-path` is relative to `management.server.base-path`.

If you want to map endpoints to a different path, you can use the `configprop:management.endpoints.web.path-mapping[]` property.

The following example remaps `/actuator/health` to `/healthcheck`:

```
management:
  endpoints:
    web:
      base-path: "/"
      path-mapping:
        health: "healthcheck"
```

18.2. Customizing the Management Server Port

Exposing management endpoints by using the default HTTP port is a sensible choice for cloud-based deployments. If, however, your application runs inside your own data center, you may prefer to expose endpoints by using a different HTTP port.

You can set the `configprop:management.server.port[]` property to change the HTTP port, as the following example shows:

```
management:
  server:
    port: 8081
```



On Cloud Foundry, by default, applications receive requests only on port 8080 for both HTTP and TCP routing. If you want to use a custom management port on Cloud Foundry, you need to explicitly set up the application's routes to forward traffic to the custom port.

18.3. Configuring Management-specific SSL

When configured to use a custom port, you can also configure the management server with its own SSL by using the various `management.server.ssl.*` properties. For example, doing so lets a management server be available over HTTP while the main application uses HTTPS, as the following property settings show:

```
server:
  port: 8443
  ssl:
    enabled: true
    key-store: "classpath:store.jks"
    key-password: "secret"
management:
  server:
    port: 8080
    ssl:
      enabled: false
```

Alternatively, both the main server and the management server can use SSL, but with different key stores, as follows:

```
server:
  port: 8443
  ssl:
    enabled: true
    key-store: "classpath:main.jks"
    key-password: "secret"
management:
  server:
    port: 8080
  ssl:
    enabled: true
    key-store: "classpath:management.jks"
    key-password: "secret"
```

18.4. Customizing the Management Server Address

You can customize the address on which the management endpoints are available by setting the `configprop:management.server.address[]` property. Doing so can be useful if you want to listen only on an internal or ops-facing network or to listen only for connections from `localhost`.



You can listen on a different address only when the port differs from the main server port.

The following example `application.properties` does not allow remote management connections:

```
management:
  server:
    port: 8081
    address: "127.0.0.1"
```

18.5. Disabling HTTP Endpoints

If you do not want to expose endpoints over HTTP, you can set the management port to `-1`, as the following example shows:

```
management:
  server:
    port: -1
```

You can also achieve this by using the `configprop:management.endpoints.web.exposure.exclude[]` property, as the following example shows:

```
management:
  endpoints:
    web:
      exposure:
        exclude: "*"

```

19. Observability

Observability is the ability to observe the internal state of a running system from the outside. It consists of the three pillars: logging, metrics and traces.

For metrics and traces, Spring Boot uses {url-micrometer-docs}/observation[Micrometer Observation]. To create your own observations (which will lead to metrics and traces), you can inject an `javadoc:io.micrometer.observation.ObservationRegistry[]`.

include-code::MyCustomObservation[]



Low cardinality tags will be added to metrics and traces, while high cardinality tags will only be added to traces.

Beans of type `javadoc:io.micrometer.observation.ObservationPredicate[]`, `javadoc:io.micrometer.observation.GlobalObservationConvention[]`, `javadoc:io.micrometer.observation.ObservationFilter[]` and `javadoc:io.micrometer.observation.ObservationHandler[]` will be automatically registered on the `javadoc:io.micrometer.observation.ObservationRegistry[]`. You can additionally register any number of `javadoc:org.springframework.boot.micrometer.observation.autoconfigure.ObservationRegistryCustomizer[]` beans to further configure the registry.



Observability for JDBC can be configured using a separate project. The [Datasource Micrometer project](https://github.com/jdbc-observations/datasource-micrometer) (<https://github.com/jdbc-observations/datasource-micrometer>) provides a Spring Boot starter which automatically creates observations when JDBC operations are invoked. Read more about it [in the reference documentation](https://jdbc-observations.github.io/datasource-micrometer/docs/current/docs/html/) (<https://jdbc-observations.github.io/datasource-micrometer/docs/current/docs/html/>).



Observability for R2DBC is built into Spring Boot. To enable it, add the `io.r2dbc:r2dbc-proxy` dependency to your project.

19.1. Context Propagation

Observability support relies on the [Context Propagation library](https://github.com/micrometer-metrics/context-propagation) (<https://github.com/micrometer-metrics/context-propagation>) for forwarding the current observation across threads and reactive pipelines. By default, `javadoc:java.lang.ThreadLocal[]` values are not automatically reinstated in reactive operators. This behavior is controlled with the `configprop:spring.reactor.context-propagation[]` property, which can be set to `auto` to enable automatic propagation.

If you're working with `javadoc:org.springframework.scheduling.annotation.Async[format=annotation]` methods and the `javadoc:org.springframework.core.task.AsyncTaskExecutor[]` is auto-configured, you have to opt-in for context propagation using the `configprop:spring.task.execution.propagate-context[]` property.

If you are configuring the `javadoc:org.springframework.core.task.AsyncTaskExecutor[]` yourself, then you need to register a `javadoc:org.springframework.core.task.support.ContextPropagatingTaskDecorator[]` bean, as shown in the following example:

include-code::ContextPropagationConfiguration[]

For more details about observations please see the {url-micrometer-docs}/observation[Micrometer Observation documentation].

19.2. Common Tags

Common tags are generally used for dimensional drill-down on the operating environment, such as host, instance, region, stack, and others. Common tags are applied to all observations as low cardinality tags and can be configured, as the following example shows:

```
management:
  observations:
    key-values:
      region: "us-east-1"
      stack: "prod"
```

The preceding example adds `region` and `stack` tags to all observations with a value of `us-east-1` and `prod`, respectively.

19.3. Preventing Observations

If you'd like to prevent some observations from being reported, you can use the `configprop:management.observations.enable[]` properties:

```
management:
  observations:
    enable:
      denied:
        prefix: false
      another:
        denied:
          prefix: false
```

The preceding example will prevent all observations with a name starting with `denied.prefix` or `another.denied.prefix`.



If you want to prevent Spring Security from reporting observations, set the property `configprop:management.observations.enable.spring.security[]` to `false`.

If you need greater control over the prevention of observations, you can register beans of type `javadoc:io.micrometer.observation.ObservationPredicate[]`. Observations are only reported if all the `javadoc:io.micrometer.observation.ObservationPredicate[]` beans return `true` for that observation.

include-code::MyObservationPredicate[]

The preceding example will prevent all observations whose name contains "denied".

19.4. Micrometer Observation Annotations support

To enable scanning of observability annotations like `javadoc:io.micrometer.observation.annotation.Observed[format=annotation]`, `javadoc:io.micrometer.core.annotation.Timed[format=annotation]`, `javadoc:io.micrometer.core.annotation.Counted[format=annotation]`, `javadoc:io.micrometer.core.aop.MeterTag[format=annotation]` and `javadoc:io.micrometer.tracing.annotation.NewSpan[format=annotation]`, set the `configprop:management.observations.annotations.enabled[]` property to `true`. A dependency on `org.aspectj:aspectjweaver`, which is part of `spring-boot-starter-aspectj`, is also required. This feature is supported by Micrometer directly. Please refer to the [{url-micrometer-docs-concepts}/timers.html#_the_timed_annotation\[Micrometer\]](#), [{url-micrometer-docs-observation}/components.html#micrometer-observation-annotations\[Micrometer Observation\]](#) and [{url-micrometer-tracing-docs}/api.html#_aspect_oriented_programming\[Micrometer Tracing\]](#) reference docs.



When you annotate methods or classes which are already instrumented (for example, [Spring Data repositories](#) or [Spring MVC controllers](#)), you will get duplicate observations. In that case you can either disable the automatic instrumentation using [properties](#) or an `javadoc:io.micrometer.observation.ObservationPredicate[]` and rely on your annotations, or you can remove your annotations.

19.5. OpenTelemetry Support



There are several ways to support [OpenTelemetry](https://opentelemetry.io/) (https://opentelemetry.io/) in your application. You can use the [OpenTelemetry Java Agent](https://opentelemetry.io/docs/zero-code/java/agent/) (https://opentelemetry.io/docs/zero-code/java/agent/) or the [OpenTelemetry Spring Boot Starter](https://opentelemetry.io/docs/zero-code/java/spring-boot-starter/) (https://opentelemetry.io/docs/zero-code/java/spring-boot-starter/), which are supported by the OTEL community; the metrics and traces use the semantic conventions defined by OTEL libraries. This documentation describes OpenTelemetry as officially supported by the Spring team, using Micrometer and the OTLP exporter; the metrics and traces use the semantic conventions described in the Spring projects documentation, such as [\[Spring Framework\]](https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/observability/integration/observability.html).

Spring Boot's actuator module includes basic support for OpenTelemetry.

It provides a bean of type `javado:io.opentelemetry.api.OpenTelemetry[]`, and if there are beans of type `javado:io.opentelemetry.sdk.trace.SdkTracerProvider[]`, `javado:io.opentelemetry.context.propagation.ContextPropagators[]`, `javado:io.opentelemetry.sdk.logs.SdkLoggerProvider[]` or `javado:io.opentelemetry.sdk.metrics.SdkMeterProvider[]` in the application context, they automatically get registered. Additionally, it provides a `javado:io.opentelemetry.sdk.resources.Resource[]` bean. The attributes of the auto-configured `javado:io.opentelemetry.sdk.resources.Resource[]` can be configured via the `configprop:management.opentelemetry.resource-attributes[]` configuration property. Auto-configured attributes will be merged with attributes from the `OTEL_RESOURCE_ATTRIBUTES` and `OTEL_SERVICE_NAME` environment variables, with attributes configured through the configuration property taking precedence over those from the environment variables.

If you have defined your own `io.opentelemetry.sdk.resources.Resource[]` bean, this will no longer be the case.



Spring Boot does not provide automatic exporting of OpenTelemetry metrics or logs. Exporting OpenTelemetry traces is only auto-configured when used together with [Micrometer Tracing](#).

19.5.1. Environment variables

Spring Boot supports the following environment variables to configure the OpenTelemetry resource:

- OTEL_RESOURCE_ATTRIBUTES (<https://opentelemetry.io/docs/specs/otel/configuration/sdk-environment-variables/#general-sdk-configuration>)
- OTEL_SERVICE_NAME (<https://opentelemetry.io/docs/specs/otel/configuration/sdk-environment-variables/#general-sdk-configuration>)



The `OTEL_RESOURCE_ATTRIBUTES` environment variable consists of a list of key-value pairs. For example: `key1=value1,key2=value2,key3=spring%20boot` . All attribute values are treated as strings, and any characters outside the baggage-octet range must be **percent-encoded**.

Micrometer also supports the following environment variables to configure the metrics export over OTLP:

- [OTEL_EXPORTER_OTLP_ENDPOINT](https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/#otel_exporter_otlp_endpoint) (https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/#otel_exporter_otlp_endpoint)
- [OTEL_EXPORTER_OTLP_METRICS_ENDPOINT](https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/#otel_exporter_otlp_metrics_endpoint) (https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/#otel_exporter_otlp_metrics_endpoint)
- [OTEL_EXPORTER_OTLP_HEADERS](https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/#otel_exporter_otlp_headers) (https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/#otel_exporter_otlp_headers)
- [OTEL_EXPORTER_OTLP_METRICS_HEADERS](https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/#otel_exporter_otlp_metrics_headers) (https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/#otel_exporter_otlp_metrics_headers)

Other environment variables as described in [the OpenTelemetry documentation](https://opentelemetry.io/docs/specs/otel/configuration/sdk-environment-variables/) (<https://opentelemetry.io/docs/specs/otel/configuration/sdk-environment-variables/>) are not supported.

If you want all environment variables specified by OpenTelemetry's SDK to be effective, you have to supply your own `OpenTelemetry` bean.



Doing this will switch off Spring Boot's OpenTelemetry auto-configuration and may break the built-in observability functionality.

First, add a dependency to `io.opentelemetry:opentelemetry-sdk-extension-autoconfigure` to get [OpenTelemetry's zero-code SDK autoconfigure module](https://opentelemetry.io/docs/languages/java/configuration/#zero-code-sdk-autoconfigure) (<https://opentelemetry.io/docs/languages/java/configuration/#zero-code-sdk-autoconfigure>), then add this configuration:

```
include-code::AutoConfiguredOpenTelemetrySdkConfiguration[]
```

19.5.2. Logging

The `javadoc:org.springframework.boot.opentelemetry.autoconfigure.logging.OpenTelemetryLoggingAutoConfiguration[]` configures OpenTelemetry's `javadoc:io.opentelemetry.sdk.logs.SdkLoggerProvider[]`. Exporting logs via OTLP is supported through the `javadoc:org.springframework.boot.opentelemetry.autoconfigure.logging.otlp.OtlpLoggingAutoConfiguration[]`, which enables OTLP log exporting over HTTP or gRPC.



If you need to apply advanced customizations to OTLP log record exporters, consider registering `javadoc:org.springframework.boot.opentelemetry.autoconfigure.logging.otlp.OtlpHttpLogRecordExporterBuilderCustomizer[]` or `javadoc:org.springframework.boot.opentelemetry.autoconfigure.logging.otlp.OtlpGrpcLogRecordExporterBuilderCustomizer[]` beans. These will be invoked before the creation of the `javadoc:io.opentelemetry.exporter.otlp.http.logs.OtlpHttpLogRecordExporter[]` or `javadoc:io.opentelemetry.exporter.otlp.logs.OtlpGrpcLogRecordExporter[]`. The customizers take precedence over anything applied by the auto-configuration.

However, while there is a `SdkLoggerProvider` bean, Spring Boot doesn't support bridging logs to this bean out of the box. This can be done with 3rd-party log bridges, as described in the [Logging with OpenTelemetry](#) section.

19.5.3. Metrics

The choice of metrics in the Spring portfolio is Micrometer, which means that metrics are not collected and exported through the OpenTelemetry's `javadoc:io.opentelemetry.sdk.metrics.SdkMeterProvider[]`. Spring Boot doesn't provide a `SdkMeterProvider` bean.

However, Micrometer metrics can be exported via OTLP to any OpenTelemetry capable backend using the `javadoc:io.micrometer.registry.otlp.OtlpMeterRegistry[]`, as described in the [Metrics with OTLP](#) section.



Micrometer's OTLP registry doesn't use the `Resource` bean, but setting `OTEL_RESOURCE_ATTRIBUTES`, `OTEL_SERVICE_NAME` or `configprop:management.opentelemetry.resource-attributes[]` works.

Metrics via the OpenTelemetry API and SDK

If you or a dependency you include make use of OpenTelemetry's `javadoc:io.opentelemetry.api.metrics.MeterProvider[]`, those metrics are not exported.

We strongly recommend that you report your metrics with Micrometer. If a dependency you include uses OpenTelemetry's `MeterProvider`, you can include this configuration in your application to configure a `MeterProvider` bean, which you then have to wire into your dependency:

```
include-code::OpenTelemetryMetricsConfiguration[]
```

This configuration also enables metrics export via OTLP over HTTP.

19.5.4. Tracing

If Micrometer tracing is used, the `javadoc:org.springframework.boot.micrometer.tracing.opentelemetry.autoconfigure.OpenTelemetryTracingAutoConfiguration[]` configures OpenTelemetry's `javadoc:io.opentelemetry.sdk.trace.SdkTracerProvider[]`. Exporting traces through OTLP is enabled by the `javadoc:org.springframework.boot.micrometer.tracing.opentelemetry.autoconfigure.otlp.OtlpTracingAutoConfiguration[]`, which supports exporting traces with OTLP over HTTP or gRPC.

We strongly recommend using the Micrometer Observation or Tracing API instead of using the OpenTelemetry API directly.

20. Process Monitoring

In the `spring-boot` module, you can find two classes to create files that are often useful for process monitoring:

- `javadoc:org.springframework.boot.context.ApplicationPidFileWriter[]` creates a file that contains the application PID (by default, in the application directory with a file name of `application.pid`).
- `javadoc:org.springframework.boot.web.server.context.WebServerPortFileWriter[]` creates a file (or files) that contain the ports of the running web server (by default, in the application directory with a file name of `application.port`).

By default, these writers are not activated, but you can enable them:

- [actuator/process-monitoring.html](#)
- [actuator/process-monitoring.html](#)

20.1. Extending Configuration

In the `META-INF/spring.factories` file, you can activate the listener (or listeners) that writes a PID file:

```
org.springframework.context.ApplicationListener=\
org.springframework.boot.context.ApplicationPidFileWriter,\
org.springframework.boot.web.server.context.WebServerPortFileWriter
```

20.2. Programmatically Enabling Process Monitoring

You can also activate a listener by invoking the `SpringApplication.addListeners(...)` method and passing the appropriate `javadoc:java.io.Writer[]` object. This method also lets you customize the file name and path in the `javadoc:java.io.Writer[]` constructor.

21. Tracing

Spring Boot Actuator provides dependency management and auto-configuration for [Micrometer Tracing](#), a facade for popular tracer libraries.



To learn more about Micrometer Tracing capabilities, see its [reference documentation](#).

21.1. Supported Tracers

Spring Boot ships auto-configuration for the following tracers:

- [OpenTelemetry](#) (<https://opentelemetry.io/>) with [Zipkin](#) (<https://zipkin.io/>) or [OTLP](#) (<https://opentelemetry.io/docs/reference/specification/protocol/>).
- [OpenZipkin Brave](#) (<https://github.com/openzipkin/brave>) with [Zipkin](#) (<https://zipkin.io/>).

21.2. Getting Started

We need an example application that we can use to get started with tracing. For our purposes, the simple “Hello World!” web application that’s covered in the [tutorial: first-application/index.html](#) section will suffice. We’re going to use the Brave tracer with Zipkin as trace backend.

To recap, our main application code looks like this:

```
include-code::MyApplication[]
```



There's an added logger statement in the `home()` method, which will be important later.

Now we have to add the `org.springframework.boot:spring-boot-starter-zipkin` dependency.

Then add the following application properties:

```
management:
  tracing:
    sampling:
      probability: 1.0
```

By default, Spring Boot samples only 10% of requests to prevent overwhelming the trace backend. This property switches it to 100% so that every request is sent to the trace backend.

To collect and visualize the traces, we need a running trace backend. We use Zipkin as our trace backend here. The [Zipkin Quickstart guide](#) (<https://zipkin.io/pages/quickstart>) provides instructions how to start Zipkin locally.

After Zipkin is running, you can start your application.

If you open a web browser to `localhost:8080`, you should see the following output:

```
Hello World!
```

Behind the scenes, an observation has been created for the HTTP request, which in turn gets bridged to Brave, which reports a new trace to Zipkin.

Now open the Zipkin UI at `localhost:9411` and press the "Run Query" button to list all collected traces. You should see one trace. Press the "Show" button to see the details of that trace.

21.3. Logging Correlation IDs

Correlation IDs provide a helpful way to link lines in your log files to spans/traces. If you are using Micrometer Tracing, Spring Boot will include correlation IDs in your logs by default.

The default correlation ID is built from `traceId` and `spanId` [MDC](#) (<https://logback.qos.ch/manual/mdc.html>) values. For example, if Micrometer Tracing has added an MDC `traceId` of `803B448A0489F84084905D3093480352` and an MDC `spanId` of `3425F23BB2432450` the log output will include the correlation ID `[803B448A0489F84084905D3093480352-3425F23BB2432450]`.

If you prefer to use a different format for your correlation ID, you can use the `configprop:logging.pattern.correlation[]` property to define one. For example, the following will provide a correlation ID for Logback in format previously used by Spring Cloud Sleuth:

```
logging:
  pattern:
    correlation: "[${spring.application.name:},%X{traceId:-},%X{spanId:-}] "
  include-application-name: false
```



In the example above, `configprop:logging.include-application-name[]` is set to `false` to avoid the application name being duplicated in the log messages (`configprop:logging.pattern.correlation[]` already contains it). It's also worth mentioning that `configprop:logging.pattern.correlation[]` contains a trailing space so that it is separated from the logger name that comes right after it by default.



Correlation IDs rely on context propagation. Please read [this documentation for more details](#).

21.4. Propagating Traces

To automatically propagate traces over the network, use the auto-configured `RestTemplateBuilder`, `RestClient.Builder` or `WebClient.Builder` to construct the client.



If you create the `javax.xml.bind.annotation.XmlRootElement`, the `javax.xml.bind.annotation.XmlRootElement` or the `javax.xml.bind.annotation.XmlRootElement` without using the auto-configured builders, automatic trace propagation won't work!

21.5. Tracer Implementations

As Micrometer Tracer supports multiple tracer implementations, there are multiple dependency combinations possible with Spring Boot. The combinations OpenTelemetry with OTLP and Brave with Zipkin are common and have dedicated starters.

21.5.1. OpenTelemetry With OTLP

Tracing with OpenTelemetry and reporting using OTLP requires the following dependencies:

- `org.springframework.boot:spring-boot-starter-opentelemetry`

Use the `management.opentelemetry.tracing.export.otlp.*` configuration properties to configure reporting using OTLP.



If you need to apply advanced customizations to OTLP span exporters, consider registering `javax.xml.bind.annotation.XmlRootElement` or `javax.xml.bind.annotation.XmlRootElement` beans. These will be invoked before the creation of the `OtlpHttpSpanExporter` or `OtlpGrpcSpanExporter`. The customizers take precedence over anything applied by the auto-configuration.

21.5.2. OpenTelemetry With Zipkin

Tracing with OpenTelemetry and reporting to Zipkin requires the following dependencies:

- `org.springframework.boot:spring-boot-micrometer-tracing-opentelemetry` - Spring Boot's support for Micrometer Tracing over OpenTelemetry.
- `io.micrometer:micrometer-tracing-bridge-otel` - bridges the Micrometer Observation API to OpenTelemetry.
- `org.springframework.boot:spring-boot-zipkin` - Spring Boot's support for Zipkin.
- `io.opentelemetry:opentelemetry-exporter-zipkin` - OpenTelemetry exporter that reports traces to Zipkin.

Use the `management.tracing.export.zipkin.*` configuration properties to configure reporting to Zipkin.

21.5.3. OpenZipkin Brave With Zipkin

Tracing with OpenZipkin Brave and reporting to Zipkin requires the following dependencies:

- `org.springframework.boot:spring-boot-starter-zipkin`

Use the `management.tracing.export.zipkin.*` configuration properties to configure reporting to Zipkin.

21.6. Integration with Micrometer Observation

A `javax.xml.bind.annotation.XmlRootElement` is automatically registered on the `javax.xml.bind.annotation.XmlRootElement`, which creates spans for every completed observation.

21.7. Creating Custom Spans

You can create your own spans by starting an observation. For this, inject `javax.xml.bind.annotation.XmlRootElement` into your component:

```
include-code::CustomObservation[]
```

This will create an observation named "some-operation" with the tag "some-tag=some-value".



If you want to create a span without creating a metric, you need to use the `{url-micrometer-tracing-docs}/api[lower-level Tracer API]` from Micrometer.

21.8. Baggage

You can create baggage with the `javax.xml.bind.annotation.XmlRootElement` API:

```
include-code::CreatingBaggage[]
```

This example creates baggage named `baggage1` with the value `value1`. The baggage is automatically propagated over the network if you're using W3C propagation. If you're using B3 propagation, baggage is not automatically propagated. To manually propagate baggage over the network, use the `configprop.management.tracing.baggage.remote-fields[]` configuration property (this works for W3C, too). For the example above, setting this property to `baggage1` results in an HTTP header `baggage1: value1`.

If you want to propagate the baggage to the MDC, use the `configprop.management.tracing.baggage.correlation.fields[]` configuration property. For the example above, setting this property to `baggage1` results in an MDC entry named `baggage1`.

21.9. Tests

Tracing components which are reporting data are not auto-configured when using `javax.xml.bind.annotation.XmlRootElement`. See [testing/spring-boot-applications.html](#) for more details.

22. Data

Spring Boot integrates with a number of data technologies, both SQL and NoSQL.

23. Working with NoSQL Technologies

Spring Data provides additional projects that help you access a variety of NoSQL technologies, including:

- {url-spring-data-cassandra-site}[Cassandra]
- {url-spring-data-couchbase-site}[Couchbase]
- {url-spring-data-elasticsearch-site}[Elasticsearch]
- {url-spring-data-geode-site}[Geode]
- {url-spring-data-ldap-site}[LDAP]
- {url-spring-data-mongodb-site}[MongoDB]
- {url-spring-data-neo4j-site}[Neo4j]
- {url-spring-data-redis-site}[Redis]

Of these, Spring Boot provides auto-configuration for Cassandra, Couchbase, Elasticsearch, LDAP, MongoDB, Neo4j and Redis. Additionally, {url-spring-boot-for-apache-geode-site}[Spring Boot for Apache Geode] provides {url-spring-boot-for-apache-geode-docs}#geode-repositories[auto-configuration for Apache Geode]. You can make use of the other projects, but you must configure them yourself. See the appropriate reference documentation at {url-spring-data-site}.

Spring Boot also provides auto-configuration for the InfluxDB client but it is deprecated in favor of [the new InfluxDB Java client](https://github.com/influxdata/influxdb-client-java) (https://github.com/influxdata/influxdb-client-java) that provides its own Spring Boot integration.

23.1. Redis

[Redis](https://redis.io/) (https://redis.io/) is a cache, message broker, and richly-featured key-value store. Spring Boot offers basic auto-configuration for the [Lettuce](https://github.com/redis/lettuce/) (https://github.com/redis/lettuce/) and [Jedis](https://github.com/redis/jedis/) (https://github.com/redis/jedis/) client libraries and the abstractions on top of them provided by [Spring Data Redis](https://github.com/spring-projects/spring-data-redis) (https://github.com/spring-projects/spring-data-redis).

There is a `spring-boot-starter-data-redis` starter for collecting the dependencies in a convenient way. By default, it uses [Lettuce](https://github.com/redis/lettuce/) (https://github.com/redis/lettuce/). That starter handles both traditional and reactive applications.



We also provide a `spring-boot-starter-data-redis-reactive` starter for consistency with the other stores with reactive support.

23.1.1. Connecting to Redis

You can inject an auto-configured `javadoec.org.springframework.data.redis.connection.RedisConnectionFactory[]`, `javadoec.org.springframework.data.redis.core.StringRedisTemplate[]`, or vanilla `javadoec.org.springframework.data.redis.core.RedisTemplate[]` instance as you would any other Spring Bean. The following listing shows an example of such a bean:

include-code::MyBean[]

By default, the instance tries to connect to a Redis server at `localhost:6379`. You can specify custom connection details using `spring.data.redis.*` properties, as shown in the following example:

```
spring:
  data:
    redis:
      host: "localhost"
      port: 6379
      database: 0
      username: "user"
      password: "secret"
```

You can also specify the url of the Redis server directly. When setting the url, the host, port, username and password properties are ignored. This is shown in the following example:

```
spring:
  data:
    redis:
      url: "redis://user:secret@localhost:6379"
      database: 0
```



You can also register an arbitrary number of beans that implement `javadoec.org.springframework.boot.data.redis.autoconfigure.LettuceClientConfigurationBuilderCustomizer[]` for more advanced customizations. `javadoec.io.lettuce.core.resource.ClientResources[]` can also be customized using `javadoec.org.springframework.boot.data.redis.autoconfigure.ClientResourcesBuilderCustomizer[]`. If you use Jedis, `javadoec.org.springframework.boot.data.redis.autoconfigure.JedisClientConfigurationBuilderCustomizer[]` is also available.

Alternatively, you can register a bean of type `javadoec.org.springframework.data.redis.connection.RedisStandaloneConfiguration[]`, `javadoec.org.springframework.data.redis.connection.RedisSentinelConfiguration[]`, `javadoec.org.springframework.data.redis.connection.RedisClusterConfiguration[]`, or `javadoec.org.springframework.data.redis.connection.RedisStaticMasterReplicaConfiguration[]` to take full control over the configuration.



master/replica is not supported by Jedis.

If you add your own `javax.annotation.processing.processor.Bean[format=annotation]` of any of the auto-configured types, it replaces the default (except in the case of `javax.annotation.processing.processor.RedisTemplate[]`, when the exclusion is based on the bean name, `redisTemplate`, not its type).

By default, a pooled connection factory is auto-configured if `commons-pool2` is on the classpath.

The auto-configured `javax.annotation.processing.processor.RedisConnectionFactory[]` can be configured to use SSL for communication with the server by setting the properties as shown in this example:

```
spring:
  data:
    redis:
      ssl:
        enabled: true
```

Custom SSL trust material can be configured in an [SSL bundle](#) and applied to the `javax.annotation.processing.processor.RedisConnectionFactory[]` as shown in this example:

```
spring:
  data:
    redis:
      ssl:
        bundle: "example"
```

23.2. MongoDB

[MongoDB](https://www.mongodb.com/) (<https://www.mongodb.com/>) is an open-source NoSQL document database that uses a JSON-like schema instead of traditional table-based relational data. Spring Boot offers several conveniences for working with MongoDB, including the `spring-boot-starter-data-mongodb` and `spring-boot-starter-data-mongodb-reactive` starters.

23.2.1. Connecting to a MongoDB Database

To access MongoDB databases, you can inject an auto-configured `javax.annotation.processing.processor.MongoDatabaseFactory[]`. By default, the instance tries to connect to a MongoDB server at `mongodb://localhost/test`. The following example shows how to connect to a MongoDB database:

include-code::MyBean[]

If you have defined your own `javax.annotation.processing.processor.MongoClient[]`, it will be used to auto-configure a suitable `javax.annotation.processing.processor.MongoDatabaseFactory[]`.

The auto-configured `javax.annotation.processing.processor.MongoClient[]` is created using a `javax.annotation.processing.processor.MongoClientSettings[]` bean. If you have defined your own `javax.annotation.processing.processor.MongoClientSettings[]`, it will be used without modification and the `spring.data.mongodb` properties will be ignored. Otherwise a `javax.annotation.processing.processor.MongoClientSettings[]` will be auto-configured and will have the `spring.data.mongodb` properties applied to it. In either case, you can declare one or more `javax.annotation.processing.processor.MongoClientSettingsBuilderCustomizer[]` beans to fine-tune the `javax.annotation.processing.processor.MongoClientSettings[]` configuration. Each will be called in order with the `javax.annotation.processing.processor.MongoClientSettings$Builder[]` that is used to build the `javax.annotation.processing.processor.MongoClientSettings[]`.

You can set the `spring.mongodb.uri` property to change the URL and configure additional settings such as the *replica set*, as shown in the following example:

```
spring:
  mongodb:
    uri: "mongodb://user:secret@mongoserver1.example.com:27017,mongoserver2.example.com:23456/test"
```

Alternatively, you can specify connection details using discrete properties. For example, you might declare the following settings in your `application.properties`:

```
spring:
  mongodb:
    host: "mongoserver1.example.com"
    port: 27017
    additional-hosts:
      - "mongoserver2.example.com:23456"
    database: "test"
    username: "user"
    password: "secret"
```

The auto-configured `javax.annotation.processing.processor.MongoClient[]` can be configured to use SSL for communication with the server by setting the properties as shown in this example:

```
spring:
  mongodb:
    uri: "mongodb://user:secret@mongoserver1.example.com:27017,mongoserver2.example.com:23456/test"
    ssl:
      enabled: true
```

Custom SSL trust material can be configured in an [SSL bundle](#) and applied to the `javax.annotation.processing.processor.MongoClient[]` as shown in this example:

```
spring:
  mongodb:
    uri: "mongodb://user:secret@mongoserver1.example.com:27017,mongoserver2.example.com:23456/test"
    ssl:
      bundle: "example"
```



If `configprop:spring.mongodb.port[]` is not specified, the default of **27017** is used. You could delete this line from the example shown earlier.

You can also specify the port as part of the host address by using the `host:port` syntax. This format should be used if you need to change the port of an **additional-hosts** entry.



If you do not use Spring Data MongoDB, you can inject a `javado:org.springframework.data.mongodb.driver.sync-javadoc/com.mongodb.client.MongoClient[]` bean instead of using `javado:org.springframework.data.mongodb.MongoDatabaseFactory[]`. If you want to take complete control of establishing the MongoDB connection, you can also declare your own `javado:org.springframework.data.mongodb.MongoDatabaseFactory[]` or `javado:org.springframework.data.mongodb.driver-sync-javadoc/com.mongodb.client.MongoClient[]` bean.



If you are using the reactive driver, Netty is required for SSL. The auto-configuration configures this factory automatically if Netty is available and the factory to use has not been customized already.

23.2.2. MongoTemplate

{url-spring-data-mongodb-site}[Spring Data MongoDB] provides a `javado:org.springframework.data.mongodb.core.MongoTemplate[]` class that is very similar in its design to Spring's `javado:org.springframework.jdbc.core.JdbcTemplate[]`. As with `javado:org.springframework.jdbc.core.JdbcTemplate[]`, Spring Boot auto-configures a bean for you to inject the template, as follows:

```
include-code::MyBean[]
```

See the `javado:org.springframework.data.mongodb.core.MongoOperations[]` API documentation for complete details.

23.2.3. Spring Data MongoDB Repositories

Spring Data includes repository support for MongoDB. As with the JPA repositories discussed earlier, the basic principle is that queries are constructed automatically, based on method names.

In fact, both Spring Data JPA and Spring Data MongoDB share the same common infrastructure. You could take the JPA example from earlier and, assuming that `City` is now a MongoDB data class rather than a JPA `javado:jakarta.persistence.Entity[format=annotation]`, it works in the same way, as shown in the following example:

```
include-code::CityRepository[]
```

Repositories and documents are found through scanning. By default, the [auto-configuration packages](#) are scanned. You can customize the locations to look for repositories and documents by using `javado:org.springframework.data.mongodb.repository.config.EnableMongoRepositories[format=annotation]` and `javado:org.springframework.boot.persistence.autoconfigure.EntityScan[format=annotation]` respectively.



For complete details of Spring Data MongoDB, including its rich object mapping technologies, see its {url-spring-data-mongodb-docs}[reference documentation].

23.3. Neo4j

[Neo4j](https://neo4j.com/) (https://neo4j.com/) is an open-source NoSQL graph database that uses a rich data model of nodes connected by first class relationships, which is better suited for connected big data than traditional RDBMS approaches. Spring Boot offers several conveniences for working with Neo4j, including the `spring-boot-starter-data-neo4j` starter.

23.3.1. Connecting to a Neo4j Database

To access a Neo4j server, you can inject an auto-configured `javado:org.neo4j.driver.Driver[]`. By default, the instance tries to connect to a Neo4j server at `localhost:7687` using the Bolt protocol. The following example shows how to inject a Neo4j `javado:org.neo4j.driver.Driver[]` that gives you access, amongst other things, to a `javado:org.neo4j.driver.Session[]`:

```
include-code::MyBean[]
```

You can configure various aspects of the driver using `spring.neo4j.*` properties. The following example shows how to configure the uri and credentials to use:

```
spring:
  neo4j:
    uri: "bolt://my-server:7687"
    authentication:
      username: "neo4j"
      password: "secret"
```

The auto-configured `javado:org.neo4j.driver.Driver[]` is created using `org.neo4j.driver.Config$ConfigBuilder`. To fine-tune its configuration, declare one or more `javado:org.springframework.boot.neo4j.autoconfigure.ConfigBuilderCustomizer[]` beans. Each will be called in order with the `org.neo4j.driver.Config$ConfigBuilder` that is used to build the `javado:org.neo4j.driver.Driver[]`.

23.3.2. Spring Data Neo4j Repositories

Spring Data includes repository support for Neo4j. For complete details of Spring Data Neo4j, see the {url-spring-data-neo4j-docs}[reference documentation].

Spring Data Neo4j shares the common infrastructure with Spring Data JPA as many other Spring Data modules do. You could take the JPA example from earlier and define `City` as Spring Data Neo4j `javax.persistence.Entity` rather than `javax.persistence.Entity` and the repository abstraction works in the same way, as shown in the following example:

```
include-code::CityRepository[]
```

The `spring-boot-starter-data-neo4j` starter enables the repository support as well as transaction management. Spring Boot supports both classic and reactive Neo4j repositories, using the `javax.persistence.Entity` or `javax.persistence.Entity` beans. When Project Reactor is available on the classpath, the reactive style is also auto-configured.

Repositories and entities are found through scanning. By default, the [auto-configuration packages](#) are scanned. You can customize the locations to look for repositories and entities by using `javax.persistence.Entity` and `javax.persistence.Entity` respectively.



In an application using the reactive style, a `javax.transaction.Transactional` is not auto-configured. To enable transaction management, the following bean must be defined in your configuration:

```
include-code::MyNeo4jConfiguration[]
```

23.4. Elasticsearch

[Elasticsearch](https://www.elastic.co/products/elasticsearch) (<https://www.elastic.co/products/elasticsearch>) is an open source, distributed, RESTful search and analytics engine. Spring Boot offers basic auto-configuration for Elasticsearch clients.

Spring Boot supports several clients:

- The official low-level REST client
- The official Java API client
- The `javax.persistence.Entity` provided by Spring Data Elasticsearch

Spring Boot provides a dedicated starter, `spring-boot-starter-data-elasticsearch`.

23.4.1. Connecting to Elasticsearch Using REST clients

Elasticsearch ships two different REST clients that you can use to query a cluster: the [low-level client](https://www.elastic.co/docs/reference/elasticsearch/clients/java/transport/rest5-client/) (<https://www.elastic.co/docs/reference/elasticsearch/clients/java/transport/rest5-client/>) and the [Java API client](https://www.elastic.co/docs/reference/elasticsearch/clients/java/) (<https://www.elastic.co/docs/reference/elasticsearch/clients/java/>). The Java API client is provided by the `co.elastic.clients:elasticsearch-java` module and the low-level client is provided by the `co.elastic.clients:elasticsearch-rest5-client` module. Additionally, Spring Boot provides support for a reactive client from the `org.springframework.data:spring-data-elasticsearch` module. By default, the clients will target `localhost:9200`. You can use `spring.elasticsearch.*` properties to further tune how the clients are configured, as shown in the following example:

```
spring:
  elasticsearch:
    uris: "https://search.example.com:9200"
    socket-timeout: "10s"
    username: "user"
    password: "secret"
```

Connecting to Elasticsearch Using Rest5Client

If you have `co.elastic.clients:elasticsearch-rest5-client` on the classpath, Spring Boot will auto-configure and register a `javax.persistence.Entity` bean. In addition to the properties described previously, to fine-tune the `javax.persistence.Entity` you can register an arbitrary number of beans that implement `javax.persistence.Entity` for more advanced customizations. To take full control over the client's configuration, define a `javax.persistence.Entity` bean.

Additionally, a `javax.persistence.Entity` can be auto-configured to automatically discover nodes from a running Elasticsearch cluster and set them on the `javax.persistence.Entity` bean. You can further tune how `javax.persistence.Entity` is configured, as shown in the following example:

```
spring:
  elasticsearch:
    restclient:
      sniffer:
        enabled: true
        interval: "10m"
        delay-after-failure: "30s"
```

Connecting to Elasticsearch Using ElasticsearchClient

If you use the `spring-boot-starter-elasticsearch` or have added `co.elastic.clients:elasticsearch-java` to the classpath, Spring Boot will auto-configure and register an `javax.persistence.Entity` bean.

The `javax.persistence.Entity` uses a transport that depends upon the previously described `javax.persistence.Entity`. Therefore, the properties described previously can be used to configure the `javax.persistence.Entity`. Furthermore, you can define a `javax.persistence.Entity` bean to take further control of the behavior of the transport.

Connecting to Elasticsearch using ReactiveElasticsearchClient

{url-spring-data-elasticsearch-site}[Spring Data Elasticsearch] ships `javadoc:org.springframework.data.elasticsearch.client.elc.ReactiveElasticsearchClient[]` for querying Elasticsearch instances in a reactive fashion. If you have Spring Data Elasticsearch and Reactor on the classpath, Spring Boot will auto-configure and register a `javadoc:org.springframework.data.elasticsearch.client.elc.ReactiveElasticsearchClient[]`.

The `javadoc:org.springframework.data.elasticsearch.client.elc.ReactiveElasticsearchClient[]` uses a transport that depends upon the previously described `javadoc:co.elastic.clients.transport.rest5_client.low_level.Rest5Client[]`. Therefore, the properties described previously can be used to configure the `javadoc:org.springframework.data.elasticsearch.client.elc.ReactiveElasticsearchClient[]`. Furthermore, you can define a `javadoc:co.elastic.clients.transport.rest5_client.Rest5ClientOptions[]` bean to take further control of the behavior of the transport.

23.4.2. Connecting to Elasticsearch by Using Spring Data

To connect to Elasticsearch, an `javadoc:co.elastic.clients.elasticsearch.ElasticsearchClient[]` bean must be defined, auto-configured by Spring Boot or manually provided by the application (see previous sections). With this configuration in place, an `javadoc:org.springframework.data.elasticsearch.client.elc.ElasticsearchTemplate[]` can be injected like any other Spring bean, as shown in the following example:

```
include-code::MyBean[]
```

In the presence of `spring-data-elasticsearch` and Reactor, Spring Boot can also auto-configure a [ReactiveElasticsearchClient](#) and a `javadoc:org.springframework.data.elasticsearch.client.elc.ReactiveElasticsearchTemplate[]` as beans. They are the reactive equivalent of the other REST clients.

23.4.3. Spring Data Elasticsearch Repositories

Spring Data includes repository support for Elasticsearch. As with the JPA repositories discussed earlier, the basic principle is that queries are constructed for you automatically based on method names.

In fact, both Spring Data JPA and Spring Data Elasticsearch share the same common infrastructure. You could take the JPA example from earlier and, assuming that `City` is now an Elasticsearch `javadoc:org.springframework.data.elasticsearch.annotations.Document[format=annotation]` class rather than a JPA `javadoc:jakarta.persistence.Entity[format=annotation]`, it works in the same way.

Repositories and documents are found through scanning. By default, the [auto-configuration packages](#) are scanned. You can customize the locations to look for repositories and documents by using `javadoc:org.springframework.data.elasticsearch.repository.config.EnableElasticsearchRepositories[format=annotation]` and `javadoc:org.springframework.boot.persistence.autoconfigure.EntityScan[format=annotation]` respectively.



For complete details of Spring Data Elasticsearch, see the {url-spring-data-elasticsearch-docs}[reference documentation].

Spring Boot supports both classic and reactive Elasticsearch repositories, using the `javadoc:org.springframework.data.elasticsearch.client.elc.ElasticsearchTemplate[]` or `javadoc:org.springframework.data.elasticsearch.client.elc.ReactiveElasticsearchTemplate[]` beans. Most likely those beans are auto-configured by Spring Boot given the required dependencies are present.

If you wish to use your own template for backing the Elasticsearch repositories, you can add your own `javadoc:org.springframework.data.elasticsearch.client.elc.ElasticsearchTemplate[]` or `javadoc:org.springframework.data.elasticsearch.core.ElasticsearchOperations[]` `javadoc:org.springframework.context.annotation.Bean[format=annotation]`, as long as it is named `"elasticsearchTemplate"`. Same applies to `javadoc:org.springframework.data.elasticsearch.client.elc.ReactiveElasticsearchTemplate[]` and `javadoc:org.springframework.data.elasticsearch.core.ReactiveElasticsearchOperations[]`, with the bean name `"reactiveElasticsearchTemplate"`.

You can choose to disable the repositories support with the following property:

```
spring:
  data:
    elasticsearch:
      repositories:
        enabled: false
```

23.5. Cassandra

[Cassandra](https://cassandra.apache.org/) (https://cassandra.apache.org/) is an open source, distributed database management system designed to handle large amounts of data across many commodity servers. Spring Boot offers auto-configuration for Cassandra and the abstractions on top of it provided by {url-spring-data-cassandra-site}[Spring Data Cassandra]. There is a `spring-boot-starter-data-cassandra` starter for collecting the dependencies in a convenient way.

23.5.1. Connecting to Cassandra

You can inject an auto-configured `javadoc:org.springframework.data.cassandra.core.cql.CqlTemplate[]`, `javadoc:org.springframework.data.cassandra.core.CassandraTemplate[]`, or a `Cassandra CqlSession` instance as you would with any other Spring Bean. The `spring.cassandra.*` properties can be used to customize the connection. Generally, you provide `keyspace-name` and `contact-points` as well the local datacenter name, as shown in the following example:

```
spring:
  cassandra:
    keyspace-name: "mykeyspace"
    contact-points: "cassandrahost1:9042,cassandrahost2:9042"
    local-datacenter: "datacenter1"
```

If the port is the same for all your contact points you can use a shortcut and only specify the host names, as shown in the following example:


```
spring:
  cassandra:
    keyspace-name: "mykeyspace"
    contact-points: "cassandrahost1,cassandrahost2"
    local-datacenter: "datacenter1"
```



Those two examples are identical as the port default to **9042**. If you need to configure the port, use `spring.cassandra.port`.

The auto-configured `CqlSession` can be configured to use SSL for communication with the server by setting the properties as shown in this example:

```
spring:
  cassandra:
    keyspace-name: "mykeyspace"
    contact-points: "cassandrahost1,cassandrahost2"
    local-datacenter: "datacenter1"
    ssl:
      enabled: true
```

Custom SSL trust material can be configured in an [SSL bundle](#) and applied to the `CqlSession` as shown in this example:

```
spring:
  cassandra:
    keyspace-name: "mykeyspace"
    contact-points: "cassandrahost1,cassandrahost2"
    local-datacenter: "datacenter1"
    ssl:
      bundle: "example"
```



The Cassandra driver has its own configuration infrastructure that loads an `application.conf` at the root of the classpath.

Spring Boot does not look for such a file by default but can load one using `spring.cassandra.config`. If a property is both present in `spring.cassandra.*` and the configuration file, the value in `spring.cassandra.*` takes precedence.

For more advanced driver customizations, you can register an arbitrary number of beans that implement `javado:org.springframework.boot.cassandra.autoconfigure.DriverConfigLoaderBuilderCustomizer[]`. The `CqlSession` can be customized with a bean of type `javado:org.springframework.boot.cassandra.autoconfigure.CqlSessionBuilderCustomizer[]`.



If you use `CqlSessionBuilder` to create multiple `CqlSession` beans, keep in mind the builder is mutable so make sure to inject a fresh copy for each session.

The following code listing shows how to inject a Cassandra bean:

```
include-code::MyBean[]
```

If you add your own `javado:org.springframework.context.annotation.Bean[format=annotation]` of type `javado:org.springframework.data.cassandra.core.CassandraTemplate[]`, it replaces the default.

23.5.2. Spring Data Cassandra Repositories

Spring Data includes basic repository support for Cassandra. Currently, this is more limited than the JPA repositories discussed earlier and needs `javado:org.springframework.data.cassandra.repository.Query[format=annotation]` annotated finder methods.

Repositories and entities are found through scanning. By default, the [auto-configuration packages](#) are scanned. You can customize the locations to look for repositories and entities by using `javado:org.springframework.data.cassandra.repository.config.EnableCassandraRepositories[format=annotation]` and `javado:org.springframework.boot.persistence.autoconfigure.EntityScan[format=annotation]` respectively.



For complete details of Spring Data Cassandra, see the `{url-spring-data-cassandra-docs}[reference documentation]`.

23.6. Couchbase

[Couchbase](https://www.couchbase.com/) (<https://www.couchbase.com/>) is an open-source, distributed, multi-model NoSQL document-oriented database that is optimized for interactive applications. Spring Boot offers auto-configuration for Couchbase and the abstractions on top of it provided by [Spring Data Couchbase](https://github.com/spring-projects/spring-data-couchbase) (<https://github.com/spring-projects/spring-data-couchbase>). There are `spring-boot-starter-data-couchbase` and `spring-boot-starter-data-couchbase-reactive` starters for collecting the dependencies in a convenient way.

23.6.1. Connecting to Couchbase

You can get a `javado:com.couchbase.client.java.Cluster[]` by adding the Couchbase SDK and some configuration. The `spring.couchbase.*` properties can be used to customize the connection. Generally, you provide the [connection string](https://docs.couchbase.com/dotnet-sdk/current/howtos/managing-connections.html) (<https://docs.couchbase.com/dotnet-sdk/current/howtos/managing-connections.html>) and credentials for authentication. Basic authentication with username and password can be configured as shown in the following example:

```
spring:
  couchbase:
    connection-string: "couchbase://192.168.1.123"
    username: "user"
    password: "secret"
```

Client certificates (<https://docs.couchbase.com/server/current/manage/manage-security/configure-client-certificates.html>) can be used for authentication instead of username and password. The location and password for a Java KeyStore containing client certificates can be configured as shown in the following example:

```
spring:
  couchbase:
    connection-string: "couchbase://192.168.1.123"
    env:
      ssl:
        enabled: true
    authentication:
      jks:
        location: "classpath:client.p12"
        password: "secret"
```

PEM-encoded certificates and a private key can be configured as shown in the following example:

```
spring:
  couchbase:
    connection-string: "couchbase://192.168.1.123"
    env:
      ssl:
        enabled: true
    authentication:
      pem:
        certificates: "classpath:client.crt"
        private-key: "classpath:client.key"
```

It is also possible to customize some of the `javadoc:com.couchbase.client.java.env.ClusterEnvironment[]` settings. For instance, the following configuration changes the timeout to open a new `javadoc:com.couchbase.client.java.Bucket[]` and enables SSL support with a reference to a configured SSL bundle:

```
spring:
  couchbase:
    env:
      timeouts:
        connect: "3s"
      ssl:
        bundle: "example"
```



Check the `spring.couchbase.env.*` properties for more details. To take more control, one or more `javadoc:org.springframework.boot.couchbase.autoconfigure.ClusterEnvironmentBuilderCustomizer[]` beans can be used.

23.6.2. Spring Data Couchbase Repositories

Spring Data includes repository support for Couchbase.

Repositories and documents are found through scanning. By default, the auto-configuration packages are scanned. You can customize the locations to look for repositories and documents by using `javadoc:org.springframework.data.couchbase.repository.config.EnableCouchbaseRepositories[format=annotation]` and `javadoc:org.springframework.boot.persistence.autoconfigure.EntityScan[format=annotation]` respectively.

For complete details of Spring Data Couchbase, see the `{url-spring-data-couchbase-docs}`[reference documentation].

You can inject an auto-configured `javadoc:org.springframework.data.couchbase.core.CouchbaseTemplate[]` instance as you would with any other Spring Bean, provided a `javadoc:org.springframework.data.couchbase.CouchbaseClientFactory[]` bean is available. This happens when a `javadoc:com.couchbase.client.java.Cluster[]` is available, as described above, and a bucket name has been specified:

```
spring:
  data:
    couchbase:
      bucket-name: "my-bucket"
```

The following examples shows how to inject a `javadoc:org.springframework.data.couchbase.core.CouchbaseTemplate[]` bean:

include-code::MyBean[]

There are a few beans that you can define in your own configuration to override those provided by the auto-configuration:

- A `javadoc:org.springframework.data.couchbase.core.mapping.CouchbaseMappingContext[]` `javadoc:org.springframework.context.annotation.Bean[format=annotation]` with a name of `couchbaseMappingContext`.
- A `javadoc:org.springframework.data.convert.CustomConversions[]` `javadoc:org.springframework.context.annotation.Bean[format=annotation]` with a name of `couchbaseCustomConversions`.
- A `javadoc:org.springframework.data.couchbase.core.CouchbaseTemplate[]` `javadoc:org.springframework.context.annotation.Bean[format=annotation]` with a name of `couchbaseTemplate`.

To avoid hard-coding those names in your own config, you can reuse `javadoc:org.springframework.data.couchbase.config.BeanNames[]` provided by Spring Data Couchbase. For instance, you can customize the converters to use, as follows:

include-code::MyCouchbaseConfiguration[]

23.7. LDAP

LDAP (https://en.wikipedia.org/wiki/Lightweight_Directory_Access_Protocol) (Lightweight Directory Access Protocol) is an open, vendor-neutral, industry standard application protocol for accessing and maintaining distributed directory information services over an IP network. Spring Boot offers auto-configuration for any compliant LDAP server as well as support for the embedded in-memory LDAP server from **UnboundID** (<https://ldap.com/unboundid-ldap-sdk-for-java/>).

LDAP abstractions are provided by **Spring Data LDAP** (<https://github.com/spring-projects/spring-data-ldap>). There is a `spring-boot-starter-data-ldap` starter for collecting the dependencies in a convenient way.

23.7.1. Connecting to an LDAP Server

To connect to an LDAP server, make sure you declare a dependency on the `spring-boot-starter-data-ldap` starter or `spring-ldap-core` and then declare the URLs of your server in your application.properties, as shown in the following example:

```
spring:
  ldap:
    urls: "ldap://myserver:1235"
    username: "admin"
    password: "secret"
```

If you need to customize connection settings, you can use the `spring.ldap.base` and `spring.ldap.base-environment` properties.

An `javax.annotation.processing.ProcessingEnvironment` is auto-configured based on these settings. If a `javax.annotation.processing.ProcessingEnvironment` bean is available, it is associated to the auto-configured `javax.annotation.processing.ProcessingEnvironment`. If you need to customize it, for instance to use a `javax.annotation.processing.ProcessingEnvironment`, you can still inject the auto-configured `javax.annotation.processing.ProcessingEnvironment`. Make sure to flag your customized `javax.annotation.processing.ProcessingEnvironment` as `javax.annotation.processing.ProcessingEnvironment` so that the auto-configured `javax.annotation.processing.ProcessingEnvironment` uses it.

23.7.2. Spring Data LDAP Repositories

Spring Data includes repository support for LDAP.

Repositories and documents are found through scanning. By default, the [auto-configuration packages](#) are scanned. You can customize the locations to look for repositories and documents by using `javax.annotation.processing.ProcessingEnvironment` and `javax.annotation.processing.ProcessingEnvironment` respectively.



For complete details of Spring Data LDAP, see the `{url-spring-data-ldap-docs}` [reference documentation].

You can also inject an auto-configured `javax.annotation.processing.ProcessingEnvironment` instance as you would with any other Spring Bean, as shown in the following example:

```
include-code::MyBean[]
```

23.7.3. Embedded In-memory LDAP Server

For testing purposes, Spring Boot supports auto-configuration of an in-memory LDAP server from **UnboundID** (<https://ldap.com/unboundid-ldap-sdk-for-java/>). To configure the server, add a dependency to `com.unboundid:unboundid-ldapsdk` and declare a `configprop:spring.ldap.embedded.base-dn` property, as follows:

```
spring:
  ldap:
    embedded:
      base-dn: "dc=spring,dc=io"
```

It is possible to define multiple base-dn values, however, since distinguished names usually contain commas, they must be defined using the correct notation.

In yaml files, you can use the yaml list notation. In properties files, you must include the index as part of the property name:



```
spring.ldap.embedded.base-dn:
- "dc=spring,dc=io"
- "dc=vmware,dc=com"
```

By default, the server starts on a random port and triggers the regular LDAP support. There is no need to specify a `configprop:spring.ldap.urls` property.

If there is a `schema.ldif` file on your classpath, it is used to initialize the server. If you want to load the initialization script from a different resource, you can also use the `configprop:spring.ldap.embedded.ldif` property.

By default, a standard schema is used to validate LDIF files. You can turn off validation altogether by setting the `configprop:spring.ldap.embedded.validation.enabled` property. If you have custom attributes, you can use `configprop:spring.ldap.embedded.validation.schema` to define your custom attribute types or object classes.

SSL

The in-memory LDAP server supports SSL (LDAPS). To enable SSL, configure the [SSL bundle](#) to use by setting the `configprop:spring.ldap.embedded.ssl.bundle` property, as shown in the following example:

```
spring:
  ldap:
    embedded:
      ssl:
        bundle: "example"
```

24. SQL Databases

The {url-spring-framework-site}[Spring Framework] provides extensive support for working with SQL databases, from direct JDBC access using `jdbc:org.springframework.jdbc.core.simple.JdbcClient[]` or `jdbc:org.springframework.jdbc.core.JdbcTemplate[]` to complete “object relational mapping” technologies such as Hibernate. {url-spring-data-site}[Spring Data] provides an additional level of functionality: creating `jdbc:org.springframework.data.repository.Repository[]` implementations directly from interfaces and using conventions to generate queries from your method names.

24.1. Configure a DataSource

Java's `jdbc:javax.sql.DataSource[]` interface provides a standard method of working with database connections. Traditionally, a `jdbc:javax.sql.DataSource[]` uses a URL along with some credentials to establish a database connection.



See the [how-to:data-access.html](#) section of the “How-to Guides” for more advanced examples, typically to take full control over the configuration of the DataSource.

24.1.1. Embedded Database Support

It is often convenient to develop applications by using an in-memory embedded database. Obviously, in-memory databases do not provide persistent storage. You need to populate your database when your application starts and be prepared to throw away data when your application ends.



The “How-to Guides” section includes a [section on how to initialize a database](#).

Spring Boot can auto-configure embedded [H2](https://www.h2database.com) (<https://www.h2database.com>), [HSQL](https://hsqldb.org/) (<https://hsqldb.org/>), and [Derby](https://db.apache.org/derby/) (<https://db.apache.org/derby/>) (deprecated) databases. You need not provide any connection URLs. You need only include a build dependency to the embedded database that you want to use. If there are multiple embedded databases on the classpath, set the `configprop:spring.datasource.embedded-database-connection[]` configuration property to control which one is used. Setting the property to `none` disables auto-configuration of an embedded database.



If you are using this feature in your tests, you may notice that the same database is reused by your whole test suite regardless of the number of application contexts that you use. If you want to make sure that each context has a separate embedded database, you should set `spring.datasource.generate-unique-name` to `true`.

For example, the typical POM dependencies would be as follows:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.hsqldb</groupId>
  <artifactId>hsqldb</artifactId>
  <scope>runtime</scope>
</dependency>
```

XML



You need a dependency on `spring-jdbc` for an embedded database to be auto-configured. In this example, it is pulled in transitively through `spring-boot-starter-data-jpa`.



If, for whatever reason, you do configure the connection URL for an embedded database, take care to ensure that the database's automatic shutdown is disabled. If you use H2, you should use `DB_CLOSE_ON_EXIT=FALSE` to do so. If you use HSQLDB, you should ensure that `shutdown=true` is not used. Disabling the database's automatic shutdown lets Spring Boot control when the database is closed, thereby ensuring that it happens once access to the database is no longer needed.

24.1.2. Connection to a Production Database

Production database connections can also be auto-configured by using a pooling `jdbc:javax.sql.DataSource[]`.

24.1.3. DataSource Configuration

DataSource configuration is controlled by external configuration properties in `spring.datasource.*`. For example, you might declare the following section in `application.properties`:

```
spring:
  datasource:
    url: "jdbc:mysql://localhost/test"
    username: "dbuser"
    password: "dbpass"
```



You should at least specify the URL by setting the `configprop:spring.datasource.url[]` property. Otherwise, Spring Boot tries to auto-configure an embedded database.



Spring Boot can deduce the JDBC driver class for most databases from the URL. If you need to specify a specific class, you can use the `configprop:spring.datasource.driver-class-name[]` property.



For a pooling `jdbc:javax.sql.DataSource[]` to be created, we need to be able to verify that a valid `jdbc:java.sql.Driver[]` class is available, so we check for that before doing anything. In other words, if you set `spring.datasource.driver-class-name=com.mysql.jdbc.Driver`, then that class has to be loadable.

See `javado:org.springframework.boot.jdbc.autoconfigure.DataSourceProperties[]` API documentation for more of the supported options. These are the standard options that work regardless of the actual implementation. It is also possible to fine-tune implementation-specific settings by using their respective prefix (`spring.datasource.hikari.*` , `spring.datasource.tomcat.*` , `spring.datasource.dbcp2.*` , and `spring.datasource.oracleucp.*`). See the documentation of the connection pool implementation you are using for more details.

For instance, if you use the `{url-tomcat-docs}/jdbc-pool.html#Common_Attributes[Tomcat connection pool]`, you could customize many additional settings, as shown in the following example:

```
spring:
  datasource:
    tomcat:
      max-wait: 10000
      max-active: 50
      test-on-borrow: true
```

This will set the pool to wait 10000ms before throwing an exception if no connection is available, limit the maximum number of connections to 50 and validate the connection before borrowing it from the pool.

24.1.4. Supported Connection Pools

Spring Boot uses the following algorithm for choosing a specific implementation:

1. We prefer HikariCP (<https://github.com/brettwooldridge/HikariCP>) for its performance and concurrency. If HikariCP is available, we always choose it.
2. Otherwise, if the Tomcat pooling `javado:javax.sql.DataSource[]` is available, we use it.
3. Otherwise, if Commons DBCP2 (<https://commons.apache.org/proper/commons-dbc2/>) is available, we use it.
4. If none of HikariCP, Tomcat, and DBCP2 are available and if Oracle UCP is available, we use it.



If you use the `spring-boot-starter-jdbc` or `spring-boot-starter-data-jpa` starters, you automatically get a dependency to HikariCP.

You can bypass that algorithm completely and specify the connection pool to use by setting the `configprop:spring.datasource.type[]` property. This is especially important if you run your application in a Tomcat container, as `tomcat-jdbc` is provided by default.

Additional connection pools can always be configured manually, using `javado:org.springframework.boot.jdbc.DataSourceBuilder[]`. If you define your own `javado:javax.sql.DataSource[]` bean, auto-configuration does not occur. The following connection pools are supported by `javado:org.springframework.boot.jdbc.DataSourceBuilder[]`:

- HikariCP
- Tomcat pooling `javado:javax.sql.DataSource[]`
- Commons DBCP2
- Oracle UCP & `OracleDataSource`
- Spring Framework's `javado:org.springframework.jdbc.datasource.SimpleDriverDataSource[]`
- H2 `javado:org.h2.jdbcx.JdbcDataSource[]`
- PostgreSQL `javado:org.postgresql.ds.PGSimpleDataSource[]`
- C3P0
- Vibur

24.1.5. Connection to a JNDI DataSource

If you deploy your Spring Boot application to an Application Server, you might want to configure and manage your `DataSource` by using your Application Server's built-in features and access it by using JNDI.

The `configprop:spring.datasource.jndi-name[]` property can be used as an alternative to the `configprop:spring.datasource.url[]`, `configprop:spring.datasource.username[]`, and `configprop:spring.datasource.password[]` properties to access the `javado:javax.sql.DataSource[]` from a specific JNDI location. For example, the following section in `application.properties` shows how you can access a JBoss AS defined `javado:javax.sql.DataSource[]`:

```
spring:
  datasource:
    jndi-name: "java:jboss/datasources/customers"
```

24.2. Using JdbcTemplate

Spring's `javado:org.springframework.jdbc.core.JdbcTemplate[]` and `javado:org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate[]` classes are auto-configured, and you can autowire them directly into your own beans, as shown in the following example:

```
include-code::MyBean[]
```

You can customize some properties of the template by using the `spring.jdbc.template.*` properties, as shown in the following example:

```
spring:
  jdbc:
    template:
      max-rows: 500
```

If tuning of SQL exceptions is required, you can define your own `SQLExceptionTranslator` bean so that it is associated with the auto-configured `JdbcTemplate`.



The `jdbc.org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate[]` reuses the same `jdbc.org.springframework.jdbc.core.JdbcTemplate[]` instance behind the scenes. If more than one `jdbc.org.springframework.jdbc.core.JdbcTemplate[]` is defined and no primary candidate exists, the `jdbc.org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate[]` is not auto-configured.

24.3. Using JdbcClient

Spring's `jdbc.org.springframework.jdbc.core.simple.JdbcClient[]` is auto-configured based on the presence of a `jdbc.org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate[]`. You can inject it directly in your own beans as well, as shown in the following example:

```
include-code::MyBean[]
```

If you rely on auto-configuration to create the underlying `jdbc.org.springframework.jdbc.core.JdbcTemplate[]`, any customization using `spring.jdbc.template.*` properties is taken into account in the client as well.

24.4. JPA and Spring Data JPA

The Java Persistence API is a standard technology that lets you “map” objects to relational databases. The `spring-boot-starter-data-jpa` POM provides a quick way to get started. It provides the following key dependencies:

- Hibernate: One of the most popular JPA implementations.
- Spring Data JPA: Helps you to implement JPA-based repositories.
- Spring ORM: Core ORM support from the Spring Framework.



We do not go into too many details of JPA or `{url-spring-data-site}[Spring Data]` here. You can follow the [Accessing Data with JPA](https://spring.io/guides/gs/accessing-data-jpa/) (<https://spring.io/guides/gs/accessing-data-jpa/>) guide from spring.io and read the `{url-spring-data-jpa-site}[Spring Data JPA]` and [Hibernate](https://hibernate.org/orm/documentation/) (<https://hibernate.org/orm/documentation/>) reference documentation.

24.4.1. Entity Classes

Traditionally, JPA “Entity” classes are specified in a `persistence.xml` file. With Spring Boot, this file is not necessary and “Entity Scanning” is used instead. By default the [auto-configuration packages](#) are scanned.

Any classes annotated with `jdbc.org.jakarta.persistence.Entity[format=annotation]`, `jdbc.org.jakarta.persistence.Embeddable[format=annotation]`, or `jdbc.org.jakarta.persistence.MappedSuperclass[format=annotation]` are considered. A typical entity class resembles the following example:

```
include-code::City[]
```



You can customize entity scanning locations by using the `jdbc.org.springframework.boot.persistence.autoconfigure.EntityScan[format=annotation]` annotation. See the [how-to:data-access.html](#) section of the “How-to Guides”.

24.4.2. Spring Data JPA Repositories

`{url-spring-data-jpa-site}[Spring Data JPA]` repositories are interfaces that you can define to access data. JPA queries are created automatically from your method names. For example, a `CityRepository` interface might declare a `findAllByState(String state)` method to find all the cities in a given state.

For more complex queries, you can annotate your method with Spring Data's `jdbc.org.springframework.data.jpa.repository.Query[]` annotation.

Spring Data repositories usually extend from the `jdbc.org.springframework.data.repository.Repository[]` or `jdbc.org.springframework.data.repository.CrudRepository[]` interfaces. If you use auto-configuration, the [auto-configuration packages](#) are searched for repositories.



You can customize the locations to look for repositories using `jdbc.org.springframework.data.jpa.repository.config.EnableJpaRepositories[format=annotation]`.

The following example shows a typical Spring Data repository interface definition:

```
include-code::CityRepository[]
```

Spring Data JPA repositories support three different modes of bootstrapping: default, deferred, and lazy. To enable deferred or lazy bootstrapping, set the `configprop:spring.data.jpa.repositories.bootstrap-mode[]` property to `deferred` or `lazy` respectively. When using deferred or lazy bootstrapping, the auto-configured `jdbc.org.springframework.boot.jpa.EntityManagerFactoryBuilder[]` will use the context's `jdbc.org.springframework.core.task.AsyncTaskExecutor[]`, if any, as the bootstrap executor. If more than one exists, the one named `applicationTaskExecutor` will be used.



When using deferred or lazy bootstrapping, make sure to defer any access to the JPA infrastructure after the application context bootstrap phase. You can use `jdbc.org.springframework.beans.factory.SmartInitializingSingleton[]` to invoke any initialization that requires the JPA infrastructure. For JPA components (such as converters) that are created as Spring beans, use `jdbc.org.springframework.beans.factory.ObjectProvider[]` to delay the resolution of dependencies, if any.



We have barely scratched the surface of Spring Data JPA. For complete details, see the [Spring Data JPA reference documentation](#).

24.4.3. Spring Data Envers Repositories

If [Spring Data Envers](#) is available, JPA repositories are auto-configured to support typical Envers queries.

To use Spring Data Envers, make sure your repository extends from `org.springframework.data.repository.history.RevisionRepository` as shown in the following example:

```
include-code::CountryRepository[]
```



For more details, check the [Spring Data Envers reference documentation](#).

24.4.4. Creating and Dropping JPA Databases

By default, JPA databases are automatically created **only** if you use an embedded database, that is H2, HSQL, or Derby (deprecated). You can explicitly configure JPA settings by using `spring.jpa.*` properties. For example, to create and drop tables you can add the following line to your `application.properties`:

```
spring:
  jpa:
    hibernate.ddl-auto: "create-drop"
```



Hibernate's own internal property name for this (if you happen to remember it better) is `hibernate.hbm2ddl.auto`. You can set it, along with other Hibernate native properties, by using `spring.jpa.properties.*` (the prefix is stripped before adding them to the entity manager). The following line shows an example of setting JPA properties for Hibernate:

```
spring:
  jpa:
    properties:
      hibernate:
        "globally_quoted_identifiers": "true"
```

The line in the preceding example passes a value of `true` for the `hibernate.globally_quoted_identifiers` property to the Hibernate entity manager.

By default, the DDL execution (or validation) is deferred until the `org.springframework.context.ApplicationContext` has started.

24.4.5. Open EntityManager in View

If you are running a web application, Spring Boot by default registers `org.springframework.orm.jpa.support.OpenEntityManagerInViewInterceptor` to apply the “Open EntityManager in View” pattern, to allow for lazy loading in web views. If you do not want this behavior, you should set `spring.jpa.open-in-view` to `false` in your `application.properties`.

24.5. Spring Data JDBC

Spring Data includes repository support for JDBC and will automatically generate SQL for the methods on `org.springframework.data.repository.CrudRepository`. For more advanced queries, a `org.springframework.data.jdbc.repository.query.Query` annotation is provided.

Spring Boot will auto-configure Spring Data's JDBC repositories when the necessary dependencies are on the classpath. They can be added to your project with a single dependency on `spring-boot-starter-data-jdbc`. If necessary, you can take control of Spring Data JDBC's configuration by adding the `org.springframework.data.jdbc.repository.config.EnableJdbcRepositories` annotation or an `org.springframework.data.jdbc.repository.config.AbstractJdbcConfiguration` subclass to your application.

If you're using Spring Data JDBC with [ahead-of-time processing](#) (targeting either the JVM or a native image), some additional configuration is recommended. To prevent the need for a DB connection during AOT processing, define a `JdbcDialect` bean that's appropriate for your application's database. For example, if you're using Postgres, define a `JdbcPostgresDialect` bean.



For complete details of Spring Data JDBC, see the [reference documentation](#).

24.6. Using H2's Web Console

The [H2 database](https://www.h2database.com) (<https://www.h2database.com>) provides a [browser-based console](https://www.h2database.com/html/quickstart.html#h2_console) (https://www.h2database.com/html/quickstart.html#h2_console) that Spring Boot can auto-configure for you. The console is auto-configured when the following conditions are met:

- You are developing a servlet-based web application.
- `org.springframework.boot:spring-boot-h2console` is on the classpath.
- You are using [Spring Boot's developer tools](#).



If you are not using Spring Boot's developer tools but would still like to make use of H2's console, you can configure the `configprop:spring.h2.console.enabled` property with a value of `true`.



The H2 console is only intended for use during development, so you should take care to ensure that `spring.h2.console.enabled` is not set to `true` in production.

24.6.1. Changing the H2 Console's Path

By default, the console is available at `/h2-console`. You can customize the console's path by using the `configprop:spring.h2.console.path[]` property.

24.6.2. Accessing the H2 Console in a Secured Application

H2 Console uses frames and, as it is intended for development only, does not implement CSRF protection measures. If your application uses Spring Security, you need to configure it to

- disable CSRF protection for requests against the console,
- set the header `X-Frame-Options` to `SAMEORIGIN` on responses from the console.

More information on `{url-spring-security-docs}/features/exploits/csrf.html[CSRF]` and the header `{url-spring-security-docs}/features/exploits/headers.html#headers-frame-options[X-Frame-Options]` can be found in the Spring Security Reference Guide.

In simple setups, a `javado:org.springframework.security.web.SecurityFilterChain[]` like the following can be used:

include-code::DevProfileSecurityConfiguration[tag=!customizer]



The H2 console is only intended for use during development. In production, disabling CSRF protection or allowing frames for a website may create severe security risks.



`PathRequest.toH2Console()` returns the correct request matcher also when the console's path has been customized.

24.7. Using jOOQ

jOOQ Object Oriented Querying (jOOQ (<https://www.jooq.org/>)) is a popular product from [Data Geekery](https://www.datageekery.com/) (<https://www.datageekery.com/>) which generates Java code from your database and lets you build type-safe SQL queries through its fluent API. Both the commercial and open source editions can be used with Spring Boot. jOOQ requires Java 21 or later.

24.7.1. Code Generation

In order to use jOOQ type-safe queries, you need to generate Java classes from your database schema. You can follow the instructions in the `{url-jooq-docs}/#jooq-in-7-steps-step3[jOOQ user manual]`. If you use the `jooq-codegen-maven` plugin and you also use the `spring-boot-starter-parent` “parent POM”, you can safely omit the plugin's `<version>` tag. You can also use Spring Boot-defined version variables (such as `h2.version`) to declare the plugin's database dependency. The following listing shows an example:

```
<plugin>
  <groupId>org.jooq</groupId>
  <artifactId>jooq-codegen-maven</artifactId>
  <executions>
    ...
  </executions>
  <dependencies>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <version>${h2.version}</version>
    </dependency>
  </dependencies>
  <configuration>
    <jdbc>
      <driver>org.h2.Driver</driver>
      <url>jdbc:h2:~/yourdatabase</url>
    </jdbc>
    <generator>
      ...
    </generator>
  </configuration>
</plugin>
```

XML

24.7.2. Using DSLContext

The fluent API offered by jOOQ is initiated through the `javado:org.jooq.DSLContext[]` interface. Spring Boot auto-configures a `javado:org.jooq.DSLContext[]` as a Spring Bean and connects it to your application `javado:javax.sql.DataSource[]`. To use the `javado:org.jooq.DSLContext[]`, you can inject it, as shown in the following example:

include-code::MyBean[tag=!method]



The jOOQ manual tends to use a variable named `create` to hold the `javado:org.jooq.DSLContext[]`.

You can then use the `javado:org.jooq.DSLContext[]` to construct your queries, as shown in the following example:

include-code::MyBean[tag=method]

24.7.3. jOOQ SQL Dialect

Unless the `configprop:spring.jooq.sql-dialect[]` property has been configured, Spring Boot determines the SQL dialect to use for your datasource. If Spring Boot could not detect the dialect, it uses `DEFAULT`.



Spring Boot can only auto-configure dialects supported by the open source version of jOOQ.

24.7.4. Customizing jOOQ

More advanced customizations can be achieved by defining your own `javadoc:org.springframework.boot.jooq.autoconfigure.DefaultConfigurationCustomizer[]` bean that will be invoked prior to creating the `javadoc:org.jooq.Configuration[]` `javadoc:org.springframework.context.annotation.Bean[format=annotation]`. This takes precedence to anything that is applied by the auto-configuration.

You can also create your own `javadoc:org.jooq.Configuration[]` `javadoc:org.springframework.context.annotation.Bean[format=annotation]` if you want to take complete control of the jOOQ configuration.

24.8. Using R2DBC

The Reactive Relational Database Connectivity ([R2DBC](https://r2dbc.io) (<https://r2dbc.io>)) project brings reactive programming APIs to relational databases. R2DBC's `javadoc:io.r2dbc.spi.Connection[]` provides a standard method of working with non-blocking database connections. Connections are provided by using a `javadoc:io.r2dbc.spi.ConnectionFactory[]`, similar to a `javadoc:javax.sql.DataSource[]` with jdbc.

`javadoc:io.r2dbc.spi.ConnectionFactory[]` configuration is controlled by external configuration properties in `spring.r2dbc.*`. For example, you might declare the following section in `application.properties`:

```
spring:
  r2dbc:
    url: "r2dbc:postgresql://localhost/test"
    username: "dbuser"
    password: "dbpass"
```



You do not need to specify a driver class name, since Spring Boot obtains the driver from R2DBC's Connection Factory discovery.



At least the url should be provided. Information specified in the URL takes precedence over individual properties, that is `name`, `username`, `password` and pooling options.



The “How-to Guides” section includes a [section on how to initialize a database](#).

To customize the connections created by a `javadoc:io.r2dbc.spi.ConnectionFactory[]`, that is, set specific parameters that you do not want (or cannot) configure in your central database configuration, you can use a `javadoc:org.springframework.boot.r2dbc.autoconfigure.ConnectionFactoryOptionsBuilderCustomizer[]` `javadoc:org.springframework.context.annotation.Bean[format=annotation]`. The following example shows how to manually override the database port while the rest of the options are taken from the application configuration:

```
include-code::MyR2dbcConfiguration[]
```

The following examples show how to set some PostgreSQL connection options:

```
include-code::MyPostgresR2dbcConfiguration[]
```

When a `javadoc:io.r2dbc.spi.ConnectionFactory[]` bean is available, the regular JDBC `javadoc:javax.sql.DataSource[]` auto-configuration backs off. If you want to retain the JDBC `javadoc:javax.sql.DataSource[]` auto-configuration, and are comfortable with the risk of using the blocking JDBC API in a reactive application, add `@Import(DataSourceAutoConfiguration.class)` on a `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class in your application to re-enable it.

24.8.1. Embedded Database Support

Similarly to [the JDBC support](#), Spring Boot can automatically configure an embedded database for reactive usage. You need not provide any connection URLs. You need only include a build dependency to the embedded database that you want to use, as shown in the following example:

```
<dependency>
  <groupId>io.r2dbc</groupId>
  <artifactId>r2dbc-h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

XHTML



If you are using this feature in your tests, you may notice that the same database is reused by your whole test suite regardless of the number of application contexts that you use. If you want to make sure that each context has a separate embedded database, you should set `spring.r2dbc.generate-unique-name` to `true`.

24.8.2. Using DatabaseClient

A `javadoc:org.springframework.r2dbc.core.DatabaseClient[]` bean is auto-configured, and you can autowire it directly into your own beans, as shown in the following example:

```
include-code::MyBean[]
```

24.8.3. Spring Data R2DBC Repositories

[Spring Data R2DBC](https://spring.io/projects/spring-data-r2dbc) (<https://spring.io/projects/spring-data-r2dbc>) repositories are interfaces that you can define to access data. Queries are created automatically from your method names. For example, a `CityRepository` interface might declare a `findAllByState(String state)` method to find all the cities in a given state.

For more complex queries, you can annotate your method with Spring Data's `javadoc:org.springframework.data.r2dbc.repository.Query[format=annotation]` annotation.

Spring Data repositories usually extend from the `javadoc:org.springframework.data.repository.Repository[]` or `javadoc:org.springframework.data.repository.CrudRepository[]` interfaces. If you use auto-configuration, the [auto-configuration packages](#) are searched for repositories.

The following example shows a typical Spring Data repository interface definition:

include-code::CityRepository[]



We have barely scratched the surface of Spring Data R2DBC. For complete details, see the [Spring Data R2DBC reference documentation](#).

25. Aspect-Oriented Programming

Spring Boot provides auto-configuration for aspect-oriented programming (AOP). You can learn more about AOP with Spring in the [{url-spring-framework-docs}/core/aop-api.html](#)[Spring Framework reference documentation].

By default, Spring Boot's auto-configuration configures Spring AOP to use CGLib proxies. To use JDK proxies instead, set `spring.aop.proxy-target-class` to `false`.

If `AspectJ` is on the classpath, Spring Boot's auto-configuration will automatically enable `AspectJ` auto proxy such that `org.springframework.context.annotation.EnableAspectJAutoProxy` is not required.

26. Development-time Services

Development-time services provide external dependencies needed to run the application while developing it. They are only supposed to be used while developing and are disabled when the application is deployed.

Spring Boot offers support for two development time services, Docker Compose and Testcontainers. The next sections will provide more details about them.

26.1. Docker Compose Support

Docker Compose is a popular technology that can be used to define and manage multiple containers for services that your application needs. A `compose.yml` file is typically created next to your application which defines and configures service containers.

A typical workflow with Docker Compose is to run `docker compose up`, work on your application with it connecting to started services, then run `docker compose down` when you are finished.

The `spring-boot-docker-compose` module can be included in a project to provide support for working with containers using Docker Compose. Add the module dependency to your build, as shown in the following listings for Maven and Gradle:

MavenXML

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-docker-compose</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

GradleGRADLE

```
dependencies {
    developmentOnly("org.springframework.boot:spring-boot-docker-compose")
}
```

When this module is included as a dependency Spring Boot will do the following:

- Search for a `compose.yml` and other common compose filenames in your working directory
- Call `docker compose up` with the discovered `compose.yml`
- Create service connection beans for each supported container
- Call `docker compose stop` when the application is shutdown

If the Docker Compose services are already running when starting the application, Spring Boot will only create the service connection beans for each supported container. It will not call `docker compose up` again and it will not call `docker compose stop` when the application is shutdown.



Repackaged archives do not contain Spring Boot's Docker Compose by default. If you want to use this support, you need to include it. When using the Maven plugin, set the `excludeDockerCompose` property to `false`. When using the Gradle plugin, [configure the task's classpath to include the `developmentOnly` configuration](#).

26.1.1. Prerequisites

You need to have the `docker` and `docker compose` (or `docker-compose`) CLI applications on your path. The minimum supported Docker Compose version is 2.2.0.

26.1.2. Service Connections

A service connection is a connection to any remote service. Spring Boot's auto-configuration can consume the details of a service connection and use them to establish a connection to a remote service. When doing so, the connection details take precedence over any connection-related configuration properties.

When using Spring Boot's Docker Compose support, service connections are established to the port mapped by the container.



Docker compose is usually used in such a way that the ports inside the container are mapped to ephemeral ports on your computer. For example, a Postgres server may run inside the container using port 5432 but be mapped to a totally different port locally. The service connection will always discover and use the locally mapped port.

Service connections are established by using the image name of the container. The following service connections are currently supported:

Connection Details	Matched on
<code>javadoc:org.springframework.boot.activemq.autoconfigure.ActiveMQConnectionDetails[]</code>	Containers named "symptoma/activemq" or "apache/activemq-classic"
<code>javadoc:org.springframework.boot.artemis.autoconfigure.ArtemisConnectionDetails[]</code>	Containers named "apache/activemq-artemis"
<code>javadoc:org.springframework.boot.cassandra.autoconfigure.CassandraConnectionDetails[]</code>	Containers named "cassandra"
<code>javadoc:org.springframework.boot.elasticsearch.autoconfigure.ElasticsearchConnectionDetails[]</code>	Containers named "elasticsearch"
<code>javadoc:org.springframework.boot.hazelcast.autoconfigure.HazelcastConnectionDetails[]</code>	Containers named "hazelcast/hazelcast".

Connection Details	Matched on
javadoc:org.springframework.boot.jdbc.autoconfigure.JdbcConnectionDetails[]	Containers named "clickhouse/clickhouse-server", "gvenzl/oracle-free", "gvenzl/oracle-xe", "mariadb", "mssql/server", "mysql", or "postgres"
javadoc:org.springframework.boot ldap.autoconfigure.LdapConnectionDetails[]	Containers named "osixia/openldap", "lldap/lldap"
javadoc:org.springframework.boot.mongodb.autoconfigure.MongoConnectionDetails[]	Containers named "mongo"
javadoc:org.springframework.boot.neo4j.autoconfigure.Neo4jConnectionDetails[]	Containers named "neo4j"
javadoc:org.springframework.boot.opentelemetry.autoconfigure.logging.otlp.OtlpLoggingConnectionDetails[]	Containers named "otel/opentelemetry-collector-contrib", "grafana/otel-lgtm"
javadoc:org.springframework.boot.micrometer.metrics.autoconfigure.export.otlp.OtlpMetricsConnectionDetails[]	Containers named "otel/opentelemetry-collector-contrib", "grafana/otel-lgtm"
javadoc:org.springframework.boot.micrometer.tracing.opentelemetry.autoconfigure.otlp.OtlpTracingConnectionDetails[]	Containers named "otel/opentelemetry-collector-contrib", "grafana/otel-lgtm"
javadoc:org.springframework.boot.pulsar.autoconfigure.PulsarConnectionDetails[]	Containers named "apache/pulsar/pulsar"
javadoc:org.springframework.boot.r2dbc.autoconfigure.R2dbcConnectionDetails[]	Containers named "clickhouse/clickhouse-server", "gvenzl/oracle-free", "gvenzl/oracle-xe", "mariadb", "mssql/server", "mysql", or "postgres"
javadoc:org.springframework.boot.amqp.autoconfigure.RabbitConnectionDetails[]	Containers named "rabbitmq"
javadoc:org.springframework.boot.data.redis.autoconfigure.DataRedisConnectionDetails[]	Containers named "redis", "redis/redis-stack" or "redis/redis-stack-server"
javadoc:org.springframework.boot.zipkin.autoconfigure.ZipkinConnectionDetails[]	Containers named "openzipkin/zipkin".

26.1.3. SSL support

Some images come with SSL enabled out of the box, or maybe you want to enable SSL for the container to mirror your production setup. Spring Boot supports SSL configuration for supported service connections. Please note that you still have to enable SSL on the service which is running inside the container yourself, this feature only configures SSL on the client side in your application.

SSL is supported for the following service connections:

- Cassandra
- Couchbase
- Elasticsearch
- Kafka
- MongoDB
- RabbitMQ
- Redis

To enable SSL support for a service, you can use [service labels](https://docs.docker.com/reference/compose-file/services/#labels) (https://docs.docker.com/reference/compose-file/services/#labels).

For JKS based keystores and truststores, you can use the following container labels:

- org.springframework.boot.sslbundle.jks.key.alias
- org.springframework.boot.sslbundle.jks.key.password
- org.springframework.boot.sslbundle.jks.options.ciphers
- org.springframework.boot.sslbundle.jks.options.enabled-protocols
- org.springframework.boot.sslbundle.jks.protocol
- org.springframework.boot.sslbundle.jks.keystore.type
- org.springframework.boot.sslbundle.jks.keystore.provider
- org.springframework.boot.sslbundle.jks.keystore.location
- org.springframework.boot.sslbundle.jks.keystore.password
- org.springframework.boot.sslbundle.jks.truststore.type
- org.springframework.boot.sslbundle.jks.truststore.provider
- org.springframework.boot.sslbundle.jks.truststore.location
- org.springframework.boot.sslbundle.jks.truststore.password

These labels mirror the properties available for [SSL bundles](#).

For PEM based keystores and truststores, you can use the following container labels:

- `org.springframework.boot.sslbundle.pem.key.alias`
- `org.springframework.boot.sslbundle.pem.key.password`
- `org.springframework.boot.sslbundle.pem.options.ciphers`
- `org.springframework.boot.sslbundle.pem.options.enabled-protocols`
- `org.springframework.boot.sslbundle.pem.protocol`
- `org.springframework.boot.sslbundle.pem.keystore.type`
- `org.springframework.boot.sslbundle.pem.keystore.certificate`
- `org.springframework.boot.sslbundle.pem.keystore.private-key`
- `org.springframework.boot.sslbundle.pem.keystore.private-key-password`
- `org.springframework.boot.sslbundle.pem.truststore.type`
- `org.springframework.boot.sslbundle.pem.truststore.certificate`
- `org.springframework.boot.sslbundle.pem.truststore.private-key`
- `org.springframework.boot.sslbundle.pem.truststore.private-key-password`

These labels mirror the properties available for [SSL bundles](#).

The following example enables SSL for a redis container:

```
services:
  redis:
    image: 'redis:latest'
    ports:
      - '6379'
    secrets:
      - ssl-ca
      - ssl-key
      - ssl-cert
    command: 'redis-server --tls-port 6379 --port 0 --tls-cert-file /run/secrets/ssl-cert --tls-key-file /run/secrets/ssl-key --tls-ca-cert-file /run/secrets/ssl-ca'
    labels:
      - 'org.springframework.boot.sslbundle.pem.keystore.certificate=client.crt'
      - 'org.springframework.boot.sslbundle.pem.keystore.private-key=client.key'
      - 'org.springframework.boot.sslbundle.pem.truststore.certificate=ca.crt'
secrets:
  ssl-ca:
    file: 'ca.crt'
  ssl-key:
    file: 'server.key'
  ssl-cert:
    file: 'server.crt'
```

YAML

26.1.4. Custom Images

Sometimes you may need to use your own version of an image to provide a service. You can use any custom image as long as it behaves in the same way as the standard image. Specifically, any environment variables that the standard image supports must also be used in your custom image.

If your image uses a different name, you can use a label in your `compose.yml` file so that Spring Boot can provide a service connection. Use a label named `org.springframework.boot.service-connection` to provide the service name.

For example:

```
services:
  redis:
    image: 'mycompany/mycustomredis:7.0'
    ports:
      - '6379'
    labels:
      org.springframework.boot.service-connection: redis
```

YAML

26.1.5. Skipping Specific Containers

If you have a container image defined in your `compose.yml` that you don't want connected to your application you can use a label to ignore it. Any container with labeled with `org.springframework.boot.ignore` will be ignored by Spring Boot.

For example:

```
services:
  redis:
    image: 'redis:7.0'
    ports:
      - '6379'
    labels:
      org.springframework.boot.ignore: true
```

YAML

26.1.6. Using a Specific Compose File

If your compose file is not in the same directory as your application, or if it's named differently, you can use `configprop:spring.docker.compose.file[]` in your `application.properties` or `application.yml` to point to a different file. Properties can be defined as an exact path or a path that's relative to your application.

For example:

```
spring:
  docker:
    compose:
      file: "../my-compose.yml"
```

26.1.7. Waiting for Container Readiness

Containers started by Docker Compose may take some time to become fully ready. The recommended way of checking for readiness is to add a `healthcheck` section under the service definition in your `compose.yml` file.

Since it's not uncommon for `healthcheck` configuration to be omitted from `compose.yml` files, Spring Boot also checks directly for service readiness. By default, a container is considered ready when a TCP/IP connection can be established to its mapped port.

You can disable this on a per-container basis by adding a `org.springframework.boot.readiness-check.tcp.disable` label in your `compose.yml` file.

For example:

```
services:
  redis:
    image: 'redis:7.0'
    ports:
      - '6379'
    labels:
      org.springframework.boot.readiness-check.tcp.disable: true
```

YAML

You can also change timeout values in your `application.properties` or `application.yml` file:

```
spring:
  docker:
    compose:
      readiness:
        tcp:
          connect-timeout: 10s
          read-timeout: 5s
```

The overall timeout can be configured using `configprop:spring.docker.compose.readiness.timeout[]`.

26.1.8. Controlling the Docker Compose Lifecycle

By default Spring Boot calls `docker compose up` when your application starts and `docker compose stop` when it's shut down. If you prefer to have different lifecycle management you can use the `configprop:spring.docker.compose.lifecycle-management[]` property.

The following values are supported:

- `none` - Do not start or stop Docker Compose
- `start-only` - Start Docker Compose when the application starts and leave it running
- `start-and-stop` - Start Docker Compose when the application starts and stop it when the JVM exits

In addition you can use the `configprop:spring.docker.compose.start.command[]` property to change whether `docker compose up` or `docker compose start` is used. The `configprop:spring.docker.compose.stop.command[]` allows you to configure if `docker compose down` or `docker compose stop` is used.

You can also pass additional arguments to Docker Compose commands. The `configprop:spring.docker.compose.arguments[]` property allows you to specify arguments that are passed to all Docker Compose commands. The `configprop:spring.docker.compose.start.arguments[]` property allows you to specify arguments that are passed only to the up (or start) command, while the `configprop:spring.docker.compose.stop.arguments[]` property allows you to specify arguments that are passed only to the down (or stop) command.

The following example shows how lifecycle management can be configured:


```
spring:
  docker:
    compose:
      lifecycle-management: start-and-stop
      arguments:
        - "--project-name=myapp"
        - "--progress=auto"
      start:
        command: up
        arguments:
          - "--build"
          - "--force-recreate"
      stop:
        command: down
        timeout: 1m
        arguments:
          - "--volumes"
          - "--remove-orphans"
```

26.1.9. Activating Docker Compose Profiles

Docker Compose profiles are similar to Spring profiles in that they let you adjust your Docker Compose configuration for specific environments. If you want to activate a specific Docker Compose profile you can use the `configprop:spring.docker.compose.profiles.active[]` property in your `application.properties` or `application.yml` file:

```
spring:
  docker:
    compose:
      profiles:
        active: "myprofile"
```

26.1.10. Using Docker Compose in Tests

By default, Spring Boot's Docker Compose support is disabled when running tests.

To enable Docker Compose support in tests, set `configprop:spring.docker.compose.skip.in-tests[]` to `false`.

When using Gradle, you also need to change the configuration of the `spring-boot-docker-compose` dependency from `developmentOnly` to `testAndDevelopmentOnly`:

Gradle

```
dependencies {
    testAndDevelopmentOnly("org.springframework.boot:spring-boot-docker-compose")
}
```

GRADLE

26.2. Testcontainers Support

As well as [using Testcontainers for integration testing](#), it's also possible to use them at development time. The next sections will provide more details about that.

26.2.1. Using Testcontainers at Development Time

This approach allows developers to quickly start containers for the services that the application depends on, removing the need to manually provision things like database servers. Using Testcontainers in this way provides functionality similar to Docker Compose, except that your container configuration is in Java rather than YAML.

To use Testcontainers at development time you need to launch your application using your “test” classpath rather than “main”. This will allow you to access all declared test dependencies and give you a natural place to write your test configuration.

To create a test launchable version of your application you should create an “Application” class in the `src/test` directory. For example, if your main application is in `src/main/java/com/example/MyApplication.java`, you should create `src/test/java/com/example/TestMyApplication.java`

The `TestMyApplication` class can use the `SpringApplication.from(...)` method to launch the real application:

```
include-code::launch/TestMyApplication[]
```

You'll also need to define the `javadoc:org.testcontainers.containers.Container[]` instances that you want to start along with your application. To do this, you need to make sure that the `spring-boot-testcontainers` module has been added as a `test` dependency. Once that has been done, you can create a `javadoc:org.springframework.boot.test.context.TestConfiguration[format=annotation]` class that declares `javadoc:org.springframework.context.annotation.Bean[format=annotation]` methods for the containers you want to start.

You can also annotate your `javadoc:org.springframework.context.annotation.Bean[format=annotation]` methods with `javadoc:org.springframework.boot.testcontainers.service.connection.ServiceConnection[format=annotation]` in order to create `javadoc:org.springframework.boot.autoconfigure.service.connection.ConnectionDetails[]` beans. See [the service connections](#) section for details of the supported technologies.

A typical Testcontainers configuration would look like this:

```
include-code::test/MyContainersConfiguration[]
```



The lifecycle of `javadoc:org.testcontainers.containers.Container[]` beans is automatically managed by Spring Boot. Containers will be started and stopped automatically.



You can use the `configprop:spring.testcontainers.beans.startup[]` property to change how containers are started. By default `sequential` startup is used, but you may also choose `parallel` if you wish to start multiple containers in parallel.

Once you have defined your test configuration, you can use the `with(...)` method to attach it to your test launcher:

```
include-code::test/TestMyApplication[]
```

You can now launch `TestMyApplication` as you would any regular Java `main` method application to start your application and the containers that it needs to run.



You can use the Maven goal `spring-boot:test-run` or the Gradle task `bootTestRun` to do this from the command line.

Contributing Dynamic Properties at Development Time

If you want to contribute dynamic properties at development time from your `javado:org.testcontainers.containers.Container[]` `javado:org.springframework.context.annotation.Bean[format=annotation]` methods, define an additional `javado:org.springframework.test.context.DynamicPropertyRegistrar[]` bean. The registrar should be defined using a `javado:org.springframework.context.annotation.Bean[format=annotation]` method that injects the container from which the properties will be sourced as a parameter. This arrangement ensures that container has been started before the properties are used.

A typical configuration would look like this:

```
include-code::MyContainersConfiguration[]
```



Using a `javado:org.springframework.boot.testcontainers.service.connection.ServiceConnection[format=annotation]` is recommended whenever possible, however, dynamic properties can be a useful fallback for technologies that don't yet have `javado:org.springframework.boot.testcontainers.service.connection.ServiceConnection[format=annotation]` support.

Importing Testcontainers Declaration Classes

A common pattern when using Testcontainers is to declare `javado:org.testcontainers.containers.Container[]` instances as static fields. Often these fields are defined directly on the test class. They can also be declared on a parent class or on an interface that the test implements.

For example, the following `MyContainers` interface declares `mongo` and `neo4j` containers:

```
include-code::MyContainers[]
```

If you already have containers defined in this way, or you just prefer this style, you can import these declaration classes rather than defining your containers as `javado:org.springframework.context.annotation.Bean[format=annotation]` methods. To do so, add the `javado:org.springframework.boot.testcontainers.context.ImportTestcontainers[format=annotation]` annotation to your test configuration class:

```
include-code::MyContainersConfiguration[]
```



If you don't intend to use the [service connections feature](#) but want to use `@DynamicPropertySource` instead, remove the `javado:org.springframework.boot.testcontainers.service.connection.ServiceConnection[format=annotation]` annotation from the `javado:org.testcontainers.containers.Container[]` fields. You can also add `javado:org.springframework.test.context.DynamicPropertySource[format=annotation]` annotated methods to your declaration class.

Using DevTools with Testcontainers at Development Time

When using devtools, you can annotate beans and bean methods with `javado:org.springframework.boot.devtools.restart.RestartScope[format=annotation]`. Such beans won't be recreated when the devtools restart the application. This is especially useful for `javado:org.testcontainers.containers.Container[]` beans, as they keep their state despite the application restart.

```
include-code::MyContainersConfiguration[]
```



If you're using Gradle and want to use this feature, you need to change the configuration of the `spring-boot-devtools` dependency from `developmentOnly` to `testAndDevelopmentOnly`. With the default scope of `developmentOnly`, the `bootTestRun` task will not pick up changes in your code, as the devtools are not active.

27. Creating Your Own Auto-configuration

If you work in a company that develops shared libraries, or if you work on an open-source or commercial library, you might want to develop your own auto-configuration. Auto-configuration classes can be bundled in external jars and still be picked up by Spring Boot.

Auto-configuration can be associated to a “starter” that provides the auto-configuration code as well as the typical libraries that you would use with it. We first cover what you need to know to build your own auto-configuration and then we move on to the [typical steps required to create a custom starter](#).

27.1. Understanding Auto-configured Beans

Classes that implement auto-configuration are annotated with `javax.annotation.processing.Generated` and `org.springframework.boot.autoconfigure.AutoConfiguration` (format=annotation). This annotation itself is meta-annotated with `org.springframework.context.annotation.Configuration` (format=annotation), making auto-configurations standard `org.springframework.context.annotation.Configuration` (format=annotation) classes. Additional `org.springframework.context.annotation.Conditional` (format=annotation) annotations are used to constrain when the auto-configuration should apply. Usually, auto-configuration classes use `org.springframework.boot.autoconfigure.condition.ConditionalOnClass` (format=annotation) and `org.springframework.boot.autoconfigure.condition.ConditionalOnMissingBean` (format=annotation) annotations. This ensures that auto-configuration applies only when relevant classes are found and when you have not declared your own `org.springframework.context.annotation.Configuration` (format=annotation).

You can browse the source code of `{code-spring-boot-autoconfigure-src}[ spring-boot-autoconfigure ]` to see the core `org.springframework.boot.autoconfigure.AutoConfiguration` (format=annotation) classes that Spring provides (see the `{code-spring-boot}/core/spring-boot-autoconfigure/src/main/resources/META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` [`META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`] file). You can also look at the equivalent file in other modules to see the auto-configurations that they provide.

27.2. Locating Auto-configuration Candidates

Spring Boot checks for the presence of a `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` file within your published jar. The file should list your configuration classes, with one class name per line, as shown in the following example:

```
com.mycorp.libx.autoconfigure.LibXAutoConfiguration
com.mycorp.libx.autoconfigure.LibXWebAutoConfiguration
```



You can add comments to the imports file using the `#` character.



In the unusual case that an auto-configuration class is not a top-level class, its class name should use `$` to separate it from its containing class, for example `com.example.Outer$NestedAutoConfiguration`.



Auto-configurations must be loaded *only* by being named in the imports file. Make sure that they are defined in a specific package space and that they are never the target of component scanning. Furthermore, auto-configuration classes should not enable component scanning to find additional components. Specific `org.springframework.context.annotation.Import` (format=annotation) annotations should be used instead.

If your configuration needs to be applied in a specific order, you can use the `before`, `beforeName`, `after` and `afterName` attributes on the `org.springframework.boot.autoconfigure.AutoConfiguration` (format=annotation) annotation or the dedicated `org.springframework.boot.autoconfigure.AutoConfigureBefore` (format=annotation) and `org.springframework.boot.autoconfigure.AutoConfigureAfter` (format=annotation) annotations. For example, if you provide web-specific configuration, your class may need to be applied after `org.springframework.boot.webmvc.autoconfigure.WebMvcAutoConfiguration`.

If you want to order certain auto-configurations that should not have any direct knowledge of each other, you can also use `org.springframework.boot.autoconfigure.AutoConfigureOrder` (format=annotation). That annotation has the same semantic as the regular `org.springframework.core.annotation.Order` (format=annotation) annotation but provides a dedicated order for auto-configuration classes.

As with standard `org.springframework.context.annotation.Configuration` (format=annotation) classes, the order in which auto-configuration classes are applied only affects the order in which their beans are defined. The order in which those beans are subsequently created is unaffected and is determined by each bean's dependencies and any `org.springframework.context.annotation.DependsOn` (format=annotation) relationships.

27.2.1. Deprecating and Replacing Auto-configuration Classes

You may need to occasionally deprecate auto-configuration classes and offer an alternative. For example, you may want to change the package name where your auto-configuration class resides.

Since auto-configuration classes may be referenced in `before` / `after` ordering and `excludes`, you'll need to add an additional file that tells Spring Boot how to deal with replacements. To define replacements, create a `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.replacements` file indicating the link between the old class and the new one.

For example:

```
com.mycorp.libx.autoconfigure.LibXAutoConfiguration=com.mycorp.libx.autoconfigure.core.LibXAutoConfiguration
```



The `AutoConfiguration.imports` file should also be updated to *only* reference the replacement class.

27.3. Condition Annotations

You almost always want to include one or more `javax.annotation.processing.Generated` annotations on your auto-configuration class. The `javax.annotation.processing.Generated` annotation is one common example that is used to allow developers to override auto-configuration if they are not happy with your defaults.

Spring Boot includes a number of `javax.annotation.processing.Generated` annotations that you can reuse in your own code by annotating `javax.annotation.processing.Generated` classes or individual `javax.annotation.processing.Generated` methods. These annotations include:

- [features/developing-auto-configuration.html](#)
- [features/developing-auto-configuration.html](#)
- [features/developing-auto-configuration.html](#)
- [features/developing-auto-configuration.html](#)
- [features/developing-auto-configuration.html](#)
- [features/developing-auto-configuration.html](#)

27.3.1. Class Conditions

The `javax.annotation.processing.Generated` and `javax.annotation.processing.Generated` annotations let `javax.annotation.processing.Generated` classes be included based on the presence or absence of specific classes. Due to the fact that annotation metadata is parsed by using [ASM](https://asm.ow2.io/) (https://asm.ow2.io/), you can use the `value` attribute to refer to the real class, even though that class might not actually appear on the running application classpath. You can also use the `name` attribute if you prefer to specify the class name by using a `java.lang.String[]` value.

This mechanism does not apply the same way to `javax.annotation.processing.Generated` methods where typically the return type is the target of the condition: before the condition on the method applies, the JVM will have loaded the class and potentially processed method references which will fail if the class is not present.

To handle this scenario, a separate `javax.annotation.processing.Generated` class can be used to isolate the condition, as shown in the following example:

include-code::MyAutoConfiguration[]



If you use `javax.annotation.processing.Generated` or `javax.annotation.processing.Generated` as a part of a meta-annotation to compose your own composed annotations, you must use `name` as referring to the class in such a case is not handled.

27.3.2. Bean Conditions

The `javax.annotation.processing.Generated` and `javax.annotation.processing.Generated` annotations let a bean be included based on the presence or absence of specific beans. You can use the `value` attribute to specify beans by type or `name` to specify beans by name. The `search` attribute lets you limit the `javax.annotation.processing.Generated` hierarchy that should be considered when searching for beans.

When placed on a `javax.annotation.processing.Generated` method, the target type defaults to the return type of the method, as shown in the following example:

include-code::MyAutoConfiguration[]

In the preceding example, the `someService` bean is going to be created if no bean of type `SomeService` is already contained in the `javax.annotation.processing.Generated`.



You need to be very careful about the order in which bean definitions are added, as these conditions are evaluated based on what has been processed so far. For this reason, we recommend using only `javax.annotation.processing.Generated` and `javax.annotation.processing.Generated` annotations on auto-configuration classes (since these are guaranteed to load after any user-defined bean definitions have been added).



`javax.annotation.processing.Generated` and `javax.annotation.processing.Generated` do not prevent `javax.annotation.processing.Generated` classes from being created. The only difference between using these conditions at the class level and marking each contained `javax.annotation.processing.Generated` method with the annotation is that the former prevents registration of the `javax.annotation.processing.Generated` class as a bean if the condition does not match.



When declaring a `javax.annotation.processing.Generated` method, provide as much type information as possible in the method's return type. For example, if your bean's concrete class implements an interface the bean method's return type should be the concrete class and not the interface. Providing as much type information as possible in `javax.annotation.processing.Generated` methods is particularly important when using bean conditions as their evaluation can only rely upon to type information that is available in the method signature.

27.3.3. Property Conditions

The `javadoc:org.springframework.boot.autoconfigure.condition.ConditionalOnProperty[format=annotation]` annotation lets configuration be included based on a Spring Environment property. Use the `prefix` and `name` attributes to specify the property that should be checked. By default, any property that exists and is not equal to `false` is matched. There is also a dedicated `javadoc:org.springframework.boot.autoconfigure.condition.ConditionalOnBooleanProperty[format=annotation]` annotation specifically made for boolean properties. With both annotations you can also create more advanced checks by using the `havingValue` and `matchIfMissing` attributes.

If multiple names are given in the `name` attribute, all of the properties have to pass the test for the condition to match.

27.3.4. Resource Conditions

The `org.springframework.boot.autoconfigure.condition.ConditionalOnResource(format=annotation)` annotation lets configuration be included only when a specific resource is present. Resources can be specified by using the usual Spring conventions, as shown in the following example: `file:/home/user/test.dat`.

27.3.5. Web Application Conditions

The javadoc:org.springframework.boot.autoconfigure.condition.ConditionalOnWebApplication[format=annotation] and javadoc:org.springframework.boot.autoconfigure.condition.ConditionalOnNotWebApplication[format=annotation] annotations let configuration be included depending on whether the application is a web application. A servlet-based web application is any application that uses a Spring javadoc:org.springframework.web.context.WebApplicationContext[], defines a session scope, or has a javadoc:org.springframework.web.context.ConfigurableWebEnvironment[]. A reactive web application is any application that uses a javadoc:org.springframework.boot.web.context.reactive.ReactiveWebApplicationContext[], or has a javadoc:org.springframework.boot.web.context.reactive.ConfigurableReactiveWebEnvironment[].

The `org.springframework.boot.autoconfigure.condition.ConditionalOnWarDeployment` and `org.springframework.boot.autoconfigure.condition.ConditionalOnNotWarDeployment` annotations let configuration be included depending on whether the application is a traditional WAR application that is deployed to a servlet container. This condition will not match for applications that are run with an embedded web server.

27.3.6. SpEL Expression Conditions

The `javax.annotation.processing.annotation.Generated` annotation lets configuration be included based on the result of a {url-spring-framework-docs}/core/expressions.html}[SpEL expression].



Referencing a bean in the expression will cause that bean to be initialized very early in context refresh processing. As a result, the bean won't be eligible for post-processing (such as configuration properties binding) and its state may be incomplete.

27.4. Testing your Auto-configuration

An auto-configuration can be affected by many factors: user configuration (`@Bean` definition and `java.lang.annotation.Annotation` customization), condition evaluation (presence of a particular library), and others. Concretely, each test should create a well defined `java.lang.annotation.Annotation` that represents a combination of those customizations. `java.lang.annotation.Annotation` provides a great way to achieve that.



javadoc:org.springframework.boot.test.context.runner.ApplicationContextRunner[] doesn't work when running the tests in a native image.

`javaxadoc.org.springframework.boot.test.context.runner.ApplicationContextRunner[]` is usually defined as a field of the test class to gather the base, common configuration. The following example makes sure that `MyServiceAutoConfiguration` is always invoked:

```
include-code::MyServiceAutoConfigurationTests[tag=runner]
```



If multiple auto-configurations have to be defined, there is no need to order their declarations as they are invoked in the exact same order as when running the application.

Each test can use the runner to represent a particular use case. For instance, the sample below invokes a user configuration (`UserConfiguration`) and checks that the auto-configuration backs off properly. Invoking `run` provides a callback context that can be used with `AssertJ`.

```
include-code::MyServiceAutoConfigurationTests[tag=test-user-config]
```

It is also possible to easily customize the `javadoc:org.springframework.core.env.Environment[]`, as shown in the following example:

include-code::MyServiceAutoConfigurationTests[tag=test-env]

The runner can also be used to display the `javax.annotation.processing.processorcondition.ConditionEvaluationReport[]`. The report can be printed at `INFO` or `DEBUG` level. The following example shows how to use the `javax.annotation.processing.processorcondition.ConditionEvaluationReportLoggingListener[]` to print the report in auto-configuration tests.

```
include-code::MyConditionEvaluationReportingTests[]
```

27.4.1. Simulating a Web Context

If you need to test an auto-configuration that only operates in a servlet or reactive web application context, use the `javadoc:org.springframework.boot.test.context.runner.WebApplicationContextRunner[]` or `javadoc:org.springframework.boot.test.context.runner.ReactiveWebApplicationContextRunner[]` respectively.

27.4.2. Overriding the Classpath

It is also possible to test what happens when a particular class and/or package is not present at runtime. Spring Boot ships with a `org.springframework.boot.test.context.FilteredClassLoader` that can easily be used by the runner. In the following example, we assert that if `MyService` is not present, the auto-configuration is properly disabled:


```
include-code::../MyServiceAutoConfigurationTests[tag=test-classloader]
```

27.5. Creating Your Own Starter

A typical Spring Boot starter contains code to auto-configure and customize the infrastructure of a given technology, let's call that "acme". To make it easily extensible, a number of configuration keys in a dedicated namespace can be exposed to the environment. Finally, a single "starter" dependency is provided to help users get started as easily as possible.

Concretely, a custom starter can contain the following:

- The `acme-spring-boot` module that contains the auto-configuration code for "acme" as well as any API to use the feature.
- The `acme-spring-boot-starter` module that provides a dependency to the other starters required by "acme", `acme-spring-boot`, and potentially additional dependencies that are typically useful. In a nutshell, adding the starter should provide everything needed to start using that library.

This separation in two modules is in no way necessary. If "acme" has several flavors, options or optional features, then it is better to separate the auto-configuration as you can clearly express the fact some features are optional. Besides, you have the ability to craft a starter that provides an opinion about those optional dependencies. At the same time, others can rely only on `acme-spring-boot` and craft their own starter with different opinions.

If the auto-configuration is relatively straightforward and does not have optional features, merging the two modules in the starter is definitely an option.

If "acme" has a basic set of dependencies that are required to work, but you want to express a more opinionated view, then having a separate starter is a good option.

When testing "acme" features, you may need test-specific auto-configurations. For example, you could offer a way to replace external dependencies with in-memory alternatives. A separate test-scoped starter can be created for this purpose, following the same principles.

27.5.1. Naming

You should make sure to provide a proper namespace for your starter. Do not start your module names with `spring-boot`, even if you use a different Maven `groupId`. We may offer official support for the thing you auto-configure in the future.

As a rule of thumb, you should name a combined module after the starter. For example, assume that you are creating a starter for "acme" and that you name the auto-configure module `acme-spring-boot` and the starter `acme-spring-boot-starter`. If you only have one module that combines the two, name it `acme-spring-boot-starter`.

If "acme" also has a test-scoped starter, name it `acme-spring-boot-starter-test`.

27.5.2. Configuration keys

If your starter provides configuration keys, use a unique namespace for them. In particular, do not include your keys in the namespaces that Spring Boot uses (such as `server`, `management`, `spring`, and so on). If you use the same namespace, we may modify these namespaces in the future in ways that break your modules. As a rule of thumb, prefix all your keys with a namespace that you own (for example `acme`).

Make sure that configuration keys are documented by adding field Javadoc for each property, as shown in the following example:

```
include-code::AcmeProperties[]
```



You should only use plain text with `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` field Javadoc, since they are not processed before being added to the JSON.

If you use `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` with record class then record components' descriptions should be provided via class-level Javadoc tag `@param` (there are no explicit instance fields in record classes to put regular field-level Javadocs on).

Here are some rules we follow internally to make sure descriptions are consistent:

- Do not start the description by "The" or "A".
- For `boolean` types, start the description with "Whether" or "Enable".
- For collection-based types, start the description with "Comma-separated list"
- Use `javadoc:java.time.Duration[]` rather than `long` and describe the default unit if it differs from milliseconds, such as "If a duration suffix is not specified, seconds will be used".
- Do not provide the default value in the description unless it has to be determined at runtime.

Make sure to [trigger meta-data generation](#) so that IDE assistance is available for your keys as well. You may want to review the generated metadata (`META-INF/spring-configuration-metadata.json`) to make sure your keys are properly documented. Using your own starter in a compatible IDE is also a good idea to validate that quality of the metadata.

27.5.3. The "autoconfigure" Module

The `autoconfigure` module contains everything that is necessary to get started with the library. It may also contain configuration key definitions (such as `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]`) and any callback interface that can be used to further customize how the components are initialized.



You should mark the dependencies to the library as optional so that you can include the `autoconfigure` module in your projects more easily. If you do it that way, the library is not provided and, by default, Spring Boot backs off.

Spring Boot uses an annotation processor to collect the conditions on auto-configurations in a metadata file (`META-INF/spring-autoconfigure-metadata.properties`). If that file is present, it is used to eagerly filter auto-configurations that do not match, which will improve startup time.

When building with Maven, configure the compiler plugin (3.12.0 or later) to add `spring-boot-autoconfigure-processor` to the annotation processor paths:

```

<project>
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <configuration>
          <annotationProcessorPaths>
            <path>
              <groupId>org.springframework.boot</groupId>
              <artifactId>spring-boot-autoconfigure-processor</artifactId>
            </path>
          </annotationProcessorPaths>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>

```

With Gradle, the dependency should be declared in the `annotationProcessor` configuration, as shown in the following example:

```

dependencies {
    annotationProcessor "org.springframework.boot:spring-boot-autoconfigure-processor"
}

```

27.5.4. Starter Module

The starter is really an empty jar. Its only purpose is to provide the necessary dependencies to work with the library. You can think of it as an opinionated view of what is required to get started.

Do not make assumptions about the project in which your starter is added. If the library you are auto-configuring typically requires other starters, mention them as well. Providing a proper set of *default* dependencies may be hard if the number of optional dependencies is high, as you should avoid including dependencies that are unnecessary for a typical usage of the library. In other words, you should not include optional dependencies.



Either way, your starter must reference the core Spring Boot starter (**spring-boot-starter**) directly or indirectly (there is no need to add it if your starter relies on another starter). If a project is created with only your custom starter, Spring Boot's core features will be honoured by the presence of the core starter.

28. Externalized Configuration

Spring Boot lets you externalize your configuration so that you can work with the same application code in different environments. You can use a variety of external configuration sources including Java properties files, YAML files, environment variables, and command-line arguments.

Property values can be injected directly into your beans by using the `javax.annotation.Value` annotation, accessed through Spring's `org.springframework.core.env.Environment` abstraction, or be bound to structured objects through `org.springframework.boot.context.properties.ConfigurationProperties`.

Spring Boot uses a very particular `org.springframework.core.env.PropertySource` order that is designed to allow sensible overriding of values. Later property sources can override the values defined in earlier ones.

Sources are considered in the following order:

1. Default properties (specified by setting `org.springframework.boot.SpringApplication.setDefaultProperties`).
2. `org.springframework.context.annotation.PropertySource` annotations on your `org.springframework.context.annotation.Configuration` classes. Please note that such property sources are not added to the `org.springframework.core.env.Environment` until the application context is being refreshed. This is too late to configure certain properties such as `logging.*` and `spring.main.*` which are read before refresh begins.
3. Config data (such as `application.properties` files).
4. A `org.springframework.boot.env.RandomValuePropertySource` that has properties only in `random.*`.
5. OS environment variables.
6. Java System properties (`System.getProperties()`).
7. JNDI attributes from `java:comp/env`.
8. `org.jakarta.servlet.ServletContext` init parameters.
9. `org.jakarta.servlet.ServletConfig` init parameters.
10. Properties from `SPRING_APPLICATION_JSON` (inline JSON embedded in an environment variable or system property).
11. Command line arguments.
12. `properties` attribute on your tests. Available on `org.springframework.boot.test.context.SpringBootTest` and the test annotations for testing a particular slice of your application.
13. `org.springframework.test.context.DynamicPropertySource` annotations in your tests.
14. `org.springframework.test.context.TestPropertySource` annotations on your tests.
15. Devtools global settings properties in the `$HOME/.config/spring-boot` directory when devtools is active.

Config data files are considered in the following order:

1. Application properties packaged inside your jar (`application.properties` and YAML variants).
2. Profile-specific application properties packaged inside your jar (`application-{profile}.properties` and YAML variants).
3. Application properties outside of your packaged jar (`application.properties` and YAML variants).
4. Profile-specific application properties outside of your packaged jar (`application-{profile}.properties` and YAML variants).



It is recommended to stick with one format for your entire application. If you have configuration files with both `.properties` and YAML format in the same location, `.properties` takes precedence.



If you use environment variables rather than system properties, most operating systems disallow period-separated key names, but you can use underscores instead (for example, `configprop:spring.config.name` instead of `configprop:spring.config.name[]`). See [features/external-config.html](#) for details.



If your application runs in a servlet container or application server, then JNDI properties (in `java:comp/env`) or servlet context initialization parameters can be used instead of, or as well as, environment variables or system properties.

To provide a concrete example, suppose you develop a `org.springframework.stereotype.Component` that uses a `name` property, as shown in the following example:

```
include-code::MyBean[]
```

On your application classpath (for example, inside your jar) you can have an `application.properties` file that provides a sensible default property value for `name`. When running in a new environment, an `application.properties` file can be provided outside of your jar that overrides the `name`. For one-off testing, you can launch with a specific command line switch (for example, `java -jar app.jar --name="Spring"`).



The `env` and `configprops` endpoints can be useful in determining why a property has a particular value. You can use these two endpoints to diagnose unexpected property values. See the [Production ready features](#) section for details.

28.1. Accessing Command Line Properties

By default, `javadoc:org.springframework.boot.SpringApplication[]` converts any command line option arguments (that is, arguments starting with `--`, such as `--server.port=9000`) to a property and adds them to the Spring `javadoc:org.springframework.core.env.Environment[]`. As mentioned previously, command line properties always take precedence over file-based property sources.

If you do not want command line properties to be added to the `javadoc:org.springframework.core.env.Environment[]`, you can disable them by using `SpringApplication.setAddCommandLineProperties(false)`.

28.2. JSON Application Properties

Environment variables and system properties often have restrictions that mean some property names cannot be used. To help with this, Spring Boot allows you to encode a block of properties into a single JSON structure.

When your application starts, any `spring.application.json` or `SPRING_APPLICATION_JSON` properties will be parsed and added to the `javadoc:org.springframework.core.env.Environment[]`.

For example, the `SPRING_APPLICATION_JSON` property can be supplied on the command line in a UN*X shell as an environment variable:

```
$ SPRING_APPLICATION_JSON='{ "my": { "name": "test" } }' java -jar myapp.jar
```

SHELL

In the preceding example, you end up with `my.name=test` in the Spring `javadoc:org.springframework.core.env.Environment[]`.

The same JSON can also be provided as a system property:

```
$ java -Dspring.application.json='{ "my": { "name": "test" } }' -jar myapp.jar
```

SHELL

Or you could supply the JSON by using a command line argument:

```
$ java -jar myapp.jar --spring.application.json='{ "my": { "name": "test" } }'
```

SHELL

If you are deploying to a classic Application Server, you could also use a JNDI variable named `java:comp/env/spring.application.json`.



Although `null` values from the JSON will be added to the resulting property source, the `javadoc:org.springframework.core.env.PropertySourcesPropertyResolver[]` treats `null` properties as missing values. This means that the JSON cannot override properties from lower order property sources with a `null` value.

28.3. External Application Properties

Spring Boot will automatically find and load `application.properties` and `application.yaml` files from the following locations when your application starts:

1. From the classpath
 - a. The classpath root
 - b. The classpath `/config` package
2. From the current directory
 - a. The current directory
 - b. The `config/` subdirectory in the current directory
 - c. Immediate child directories of the `config/` subdirectory

The list is ordered by precedence (with values from lower items overriding earlier ones). Documents from the loaded files are added as `javadoc:org.springframework.core.env.PropertySource[]` instances to the Spring `javadoc:org.springframework.core.env.Environment[]`.

If you do not like `application` as the configuration file name, you can switch to another file name by specifying a `configprop:spring.config.name[]` environment property. For example, to look for `myproject.properties` and `myproject.yaml` files you can run your application as follows:

```
$ java -jar myproject.jar --spring.config.name=myproject
```

SHELL

You can also refer to an explicit location by using the `configprop:spring.config.location[]` environment property. This property accepts a comma-separated list of one or more locations to check.

The following example shows how to specify two distinct files:

```
$ java -jar myproject.jar --spring.config.location=\
optional:classpath:/default.properties,\
optional:classpath:/override.properties
```

SHELL



Use the prefix `optional:` if the locations are optional and you do not mind if they do not exist.



`spring.config.name`, `spring.config.location`, and `spring.config.additional-location` are used very early to determine which files have to be loaded. They must be defined as an environment property (typically an OS environment variable, a system property, or a command-line argument).

If `spring.config.location` contains directories (as opposed to files), they should end in `/`. At runtime they will be appended with the names generated from `spring.config.name` before being loaded. Files specified in `spring.config.location` are imported directly.



Both directory and file location values are also expanded to check for [profile-specific files](#). For example, if you have a `spring.config.location` of `classpath:myconfig.properties`, you will also find appropriate `classpath:myconfig-<profile>.properties` files are loaded.

In most situations, each `spring.config.location[]` item you add will reference a single file or directory. Locations are processed in the order that they are defined and later ones can override the values of earlier ones.

If you have a complex location setup, and you use profile-specific configuration files, you may need to provide further hints so that Spring Boot knows how they should be grouped. A location group is a collection of locations that are all considered at the same level. For example, you might want to group all classpath locations, then all external locations. Items within a location group should be separated with `;`. See the example in the [features/external-config.html](#) section for more details.

Locations configured by using `spring.config.location` replace the default locations. For example, if `spring.config.location` is configured with the value `optional:classpath:/custom-config/,optional:file:./custom-config/`, the complete set of locations considered is:

1. `optional:classpath:custom-config/`
2. `optional:file:./custom-config/`

If you prefer to add additional locations, rather than replacing them, you can use `spring.config.additional-location`. Properties loaded from additional locations can override those in the default locations. For example, if `spring.config.additional-location` is configured with the value `optional:classpath:/custom-config/,optional:file:./custom-config/`, the complete set of locations considered is:

1. `optional:classpath:/;optional:classpath:/config/`
2. `optional:file:./;optional:file:./config/;optional:file:./config/*/`
3. `optional:classpath:custom-config/`
4. `optional:file:./custom-config/`

This search ordering lets you specify default values in one configuration file and then selectively override those values in another. You can provide default values for your application in `application.properties` (or whatever other basename you choose with `spring.config.name`) in one of the default locations. These default values can then be overridden at runtime with a different file located in one of the custom locations.

28.3.1. Optional Locations

By default, when a specified config data location does not exist, Spring Boot will throw a `java.lang.IllegalArgumentException: ConfigDataLocationNotFoundException` and your application will not start.

If you want to specify a location, but you do not mind if it does not always exist, you can use the `optional:` prefix. You can use this prefix with the `spring.config.location` and `spring.config.additional-location` properties, as well as with [spring.config.import](#) declarations.

For example, a `spring.config.import` value of `optional:file:./myconfig.properties` allows your application to start, even if the `myconfig.properties` file is missing.

If you want to ignore all `java.lang.IllegalArgumentException: ConfigDataLocationNotFoundException` errors and always continue to start your application, you can use the `spring.config.on-not-found` property. Set the value to `ignore` using `SpringApplication.setDefaultProperties(...)` or with a system/environment variable.

28.3.2. Wildcard Locations

If a config file location includes the `*` character for the last path segment, it is considered a wildcard location. Wildcards are expanded when the config is loaded so that immediate subdirectories are also checked. Wildcard locations are particularly useful in an environment such as Kubernetes when there are multiple sources of config properties.

For example, if you have some Redis configuration and some MySQL configuration, you might want to keep those two pieces of configuration separate, while requiring that both those are present in an `application.properties` file. This might result in two separate `application.properties` files mounted at different locations such as `/config/redis/application.properties` and `/config/mysql/application.properties`. In such a case, having a wildcard location of `config/*/`, will result in both files being processed.

By default, Spring Boot includes `config/*/` in the default search locations. It means that all subdirectories of the `/config` directory outside of your jar will be searched.

You can use wildcard locations yourself with the `spring.config.location` and `spring.config.additional-location` properties.



A wildcard location must contain only one `*` and end with `*/` for search locations that are directories or `*/<filename>` for search locations that are files. Locations with wildcards are sorted alphabetically based on the absolute path of the file names.



Wildcard locations only work with external directories. You cannot use a wildcard in a `classpath:` location.

28.3.3. Profile Specific Files

As well as `application` property files, Spring Boot will also attempt to load profile-specific files using the naming convention `application-{profile}`. For example, if your application activates a profile named `prod` and uses YAML files, then both `application.yaml` and `application-prod.yaml` will be considered.

Profile-specific properties are loaded from the same locations as standard `application.properties`, with profile-specific files always overriding the non-specific ones. If several profiles are specified, a last-wins strategy applies. For example, if profiles `prod, live` are specified by the `configprop:spring.profiles.active[]` property, values in `application-prod.properties` can be overridden by those in `application-live.properties`.

The last-wins strategy applies at the location group level. A `configprop:spring.config.location[]` of `classpath:/cfg/,classpath:/ext/` will not have the same override rules as `classpath:/cfg/;classpath:/ext/`.

For example, continuing our `prod, live` example above, we might have the following files:

```
/cfg
  application-live.properties
/ext
  application-live.properties
  application-prod.properties
```



When we have a `configprop:spring.config.location[]` of `classpath:/cfg/,classpath:/ext/` we process all `/cfg` files before all `/ext` files:

1. `/cfg/application-live.properties`
2. `/ext/application-prod.properties`
3. `/ext/application-live.properties`

When we have `classpath:/cfg/;classpath:/ext/` instead (with a `;` delimiter) we process `/cfg` and `/ext` at the same level:

1. `/ext/application-prod.properties`
2. `/cfg/application-live.properties`
3. `/ext/application-live.properties`

The `javadoc:org.springframework.core.env.Environment[]` has a set of default profiles (by default, `[default]`) that are used if no active profiles are set. In other words, if no profiles are explicitly activated, then properties from `application-default` are considered.



Properties files are only ever loaded once. If you have already directly imported a profile specific property files then it will not be imported a second time.

28.3.4. Importing Additional Data

Application properties may import further config data from other locations using the `configprop:spring.config.import[]` property. Imports are processed as they are discovered, and are treated as additional documents inserted immediately below the one that declares the import.

For example, you might have the following in your classpath `application.properties` file:

```
spring:
  application:
    name: "myapp"
  config:
    import: "optional:file:./dev.properties"
```

This will trigger the import of a `dev.properties` file in current directory (if such a file exists). Values from the imported `dev.properties` will take precedence over the file that triggered the import. In the above example, the `dev.properties` could redefine `spring.application.name` to a different value.

An import will only be imported once no matter how many times it is declared.

Using "Fixed" and "Import Relative" Locations

Imports may be specified as *fixed* or *import relative* locations. A fixed location always resolves to the same underlying resource, regardless of where the `configprop:spring.config.import[]` property is declared. An import relative location resolves relative to the file that declares the `configprop:spring.config.import[]` property.

A location starting with a forward slash (`/`) or a URL style prefix (`file:`, `classpath:`, etc.) is considered fixed. All other locations are considered import relative.



optional: prefixes are not considered when determining if a location is fixed or import relative.

As an example, say we have a `/demo` directory containing our `application.jar` file. We might add a `/demo/application.properties` file with the following content:

```
spring.config.import=optional:core/core.properties
```

PROPERTIES

This is an import relative location and so will attempt to load the file `/demo/core/core.properties` if it exists.

If `/demo/core/core.properties` has the following content:

```
spring.config.import=optional:extra/extra.properties
```

PROPERTIES

It will attempt to load `/demo/core/extra/extra.properties`. The `optional:extra/extra.properties` is relative to `/demo/core/core.properties` so the full directory is `/demo/core/ + extra/extra.properties`.

Property Ordering

The order an import is defined inside a single document within the properties/yaml file does not matter. For instance, the two examples below produce the same result:

```
spring:
  config:
    import: "my.properties"
my:
  property: "value"
```

```
my:
  property: "value"
spring:
  config:
    import: "my.properties"
```

In both of the above examples, the values from the `my.properties` file will take precedence over the file that triggered its import.

Several locations can be specified under a single `spring.config.import` key. Locations will be processed in the order that they are defined, with later imports taking precedence.



When appropriate, [Profile-specific variants](#) are also considered for import. The example above would import both `my.properties` as well as any `my-<profile>.properties` variants.

Spring Boot includes pluggable API that allows various different location addresses to be supported. By default you can import Java Properties, YAML and [configuration trees](#).



Third-party jars can offer support for additional technologies (there is no requirement for files to be local). For example, you can imagine config data being from external stores such as Consul, Apache ZooKeeper or Netflix Archaius.

If you want to support your own locations, see the [javadoc:org.springframework.boot.context.config.ConfigDataLocationResolver\[\]](#) and [javadoc:org.springframework.boot.context.config.ConfigDataLoader\[\]](#) classes in the `org.springframework.boot.context.config` package.

28.3.5. Importing Extensionless Files

Some cloud platforms cannot add a file extension to volume mounted files. To import these extensionless files, you need to give Spring Boot a hint so that it knows how to load them. You can do this by putting an extension hint in square brackets.

For example, suppose you have a `/etc/config/myconfig` file that you wish to import as yaml. You can import it from your `application.properties` using the following:

```
spring:
  config:
    import: "file:/etc/config/myconfig[.yaml]"
```

28.3.6. Using Environment Variables

When running applications on a cloud platform (such as Kubernetes) you often need to read config values that the platform supplies. You can either use environment variables for such purpose, or you can use [configuration trees](#).

You can even store whole configurations in properties or yaml format in (multiline) environment variables and load them using the `env:` prefix. Assume there's an environment variable called `MY_CONFIGURATION` with this content:

```
my.name=Service1
my.cluster=Cluster1
```

PROPERTIES

Using the `env:` prefix it is possible to import all properties from this variable:

```
spring:
  config:
    import: "env:MY_CONFIGURATION"
```



This feature also supports [specifying the extension](#). The default extension is `.properties`.

28.3.7. Using Configuration Trees

Storing config values in environment variables has drawbacks, especially if the value is supposed to be kept secret.

As an alternative to environment variables, many cloud platforms now allow you to map configuration into mounted data volumes. For example, Kubernetes can volume mount both [ConfigMaps](#) (<https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/#populate-a-volume-with-data-stored-in-a-configmap>) and [Secrets](#) (<https://kubernetes.io/docs/concepts/configuration/secret/#using-secrets-as-files-from-a-pod>).

There are two common volume mount patterns that can be used:

1. A single file contains a complete set of properties (usually written as YAML).
2. Multiple files are written to a directory tree, with the filename becoming the 'key' and the contents becoming the 'value'.

For the first case, you can import the YAML or Properties file directly using `spring.config.import` as described [above](#). For the second case, you need to use the `configtree:` prefix so that Spring Boot knows it needs to expose all the files as properties.

As an example, let's imagine that Kubernetes has mounted the following volume:

```
etc/
  config/
    myapp/
      username
      password
```

The contents of the `username` file would be a config value, and the contents of `password` would be a secret.

To import these properties, you can add the following to your `application.properties` or `application.yaml` file:

```
spring:
  config:
    import: "optional:configtree:/etc/config/"
```

You can then access or inject `myapp.username` and `myapp.password` properties from the `java.lang.String[]` in the usual way.



The names of the folders and files under the config tree form the property name. In the above example, to access the properties as `username` and `password`, you can set `spring.config.import` to `optional:configtree:/etc/config/myapp`.



Filenames with dot notation are also correctly mapped. For example, in the above example, a file named `myapp.username` in `/etc/config` would result in a `myapp.username` property in the `java.lang.String[]`.



Configuration tree values can be bound to both string `java.lang.String[]` and `byte[]` types depending on the contents expected.

If you have multiple config trees to import from the same parent folder you can use a wildcard shortcut. Any `configtree:` location that ends with `/*` will import all immediate children as config trees. As with a non-wildcard import, the names of the folders and files under each config tree form the property name.

For example, given the following volume:

```
etc/
  config/
    dbconfig/
      db/
        username
        password
    mqconfig/
      mq/
        username
        password
```

You can use `configtree:/etc/config/*` as the import location:

```
spring:
  config:
    import: "optional:configtree:/etc/config/*"
```

This will add `db.username`, `db.password`, `mq.username` and `mq.password` properties.



Directories loaded using a wildcard are sorted alphabetically. If you need a different order, then you should list each location as a separate import

Configuration trees can also be used for Docker secrets. When a Docker swarm service is granted access to a secret, the secret gets mounted into the container. For example, if a secret named `db.password` is mounted at location `/run/secrets/`, you can make `db.password` available to the Spring environment using the following:

```
spring:
  config:
    import: "optional:configtree:/run/secrets/"
```

28.3.8. Property Placeholders

The values in `application.properties` and `application.yaml` are filtered through the existing `java.lang.String[]` when they are used, so you can refer back to previously defined values (for example, from System properties or environment variables). The standard `${name}` property-placeholder syntax can be used anywhere within a value. Property placeholders can also specify a default value using a `:` to separate the default value from the property name, for example `${name:default}`.

The use of placeholders with and without defaults is shown in the following example:

```
app:
  name: "MyApp"
  description: "${app.name} is a Spring Boot application written by ${username:Unknown}"
```

Assuming that the `username` property has not been set elsewhere, `app.description` will have the value `MyApp is a Spring Boot application written by Unknown`.



You should always refer to property names in the placeholder using their canonical form (kebab-case using only lowercase letters). This will allow Spring Boot to use the same logic as it does when [relaxed binding](#) `javax.annotation.processing.ConfigurationProperties(format=annotation)`.

For example, `${demo.item-price}` will pick up `demo.item-price` and `demo.itemPrice` forms from the `application.properties` file, as well as `DEMO_ITEMPRICE` from the system environment. If you used `${demo.itemPrice}` instead, `demo.item-price` and `DEMO_ITEMPRICE` would not be considered.



You can also use this technique to create “short” variants of existing Spring Boot properties. See the [how-to:properties-and-configuration.html](#) section in “How-to Guides” for details.

28.3.9. Working With Multi-Document Files

Spring Boot allows you to split a single physical file into multiple logical documents which are each added independently. Documents are processed in order, from top to bottom. Later documents can override the properties defined in earlier ones.

For `application.yaml` files, the standard YAML multi-document syntax is used. Three consecutive hyphens represent the end of one document, and the start of the next.

For example, the following file has two logical documents:

```
spring:
  application:
    name: "MyApp"
---
spring:
  application:
    name: "MyCloudApp"
  config:
    activate:
      on-cloud-platform: "kubernetes"
```

YAML

For `application.properties` files a special `#---` or `!---` comment is used to mark the document splits:

```
spring.application.name=MyApp
#---
spring.application.name=MyCloudApp
spring.config.activate.on-cloud-platform=kubernetes
```

PROPERTIES



Property file separators must not have any leading whitespace and must have exactly three hyphen characters. The lines immediately before and after the separator must not be same comment prefix.



Multi-document property files are often used in conjunction with activation properties such as `spring.config.activate.on-profile`. See the [next section](#) for details.



Multi-document property files cannot be loaded by using the `javax.annotation.processing.PropertySource(format=annotation)` or `javax.test.annotation.TestPropertySource(format=annotation)` annotations.

28.3.10. Activation Properties

It is sometimes useful to only activate a given set of properties when certain conditions are met. For example, you might have properties that are only relevant when a specific profile is active.

You can conditionally activate a properties document using `spring.config.activate.*`.

The following activation properties are available:

Table 2. activation properties

Property	Note
on-profile	A profile expression that must match for the document to be active, or a list of profile expressions of which at least one must match for the document to be active.
on-cloud-platform	The <code>javax.annotation.processing.CloudPlatform[]</code> that must be detected for the document to be active.

For example, the following specifies that the second document is only active when running on Kubernetes, and only when either the “prod” or “staging” profiles are active:

```
myprop:
  "always-set"
---
spring:
  config:
    activate:
      on-cloud-platform: "kubernetes"
      on-profile: "prod | staging"
myotherprop: "sometimes-set"
```

28.4. Encrypting Properties

Spring Boot does not provide any built-in support for encrypting property values, however, it does provide the hook points necessary to modify values contained in the `Spring javadoc:org.springframework.core.env.Environment[]`. The `javadoc:org.springframework.boot.EnvironmentPostProcessor[]` interface allows you to manipulate the `javadoc:org.springframework.core.env.Environment[]` before the application starts. See [how-to:application.html](#) for details.

If you need a secure way to store credentials and passwords, the [Spring Cloud Vault](#) (<https://cloud.spring.io/spring-cloud-vault/>) project provides support for storing externalized configuration in [HashiCorp Vault](#) (<https://www.vaultproject.io/>).

28.5. Working With YAML

[YAML](#) (<https://yaml.org>) is a superset of JSON and, as such, is a convenient format for specifying hierarchical configuration data. The `javadoc:org.springframework.boot.SpringApplication[]` class automatically supports YAML as an alternative to properties whenever you have the [SnakeYAML](#) (<https://github.com/snakeyaml/snakeyaml>) library on your classpath.



If you use starters, SnakeYAML is automatically provided by `spring-boot-starter`.

28.5.1. Mapping YAML to Properties

YAML documents need to be converted from their hierarchical format to a flat structure that can be used with the `Spring javadoc:org.springframework.core.env.Environment[]`. For example, consider the following YAML document:

```
environments:
  dev:
    url: "https://dev.example.com"
    name: "Developer Setup"
  prod:
    url: "https://another.example.com"
    name: "My Cool App"
```

YAML

In order to access these properties from the `javadoc:org.springframework.core.env.Environment[]`, they would be flattened as follows:

```
environments.dev.url=https://dev.example.com
environments.dev.name=Developer Setup
environments.prod.url=https://another.example.com
environments.prod.name=My Cool App
```

PROPERTIES

Likewise, YAML lists also need to be flattened. They are represented as property keys with `[index]` dereferencers. For example, consider the following YAML:

```
my:
  servers:
    - "dev.example.com"
    - "another.example.com"
```

YAML

The preceding example would be transformed into these properties:

```
my.servers[0]=dev.example.com
my.servers[1]=another.example.com
```

PROPERTIES



Properties that use the `[index]` notation can be bound to Java `javadoc:java.util.List[]` or `javadoc:java.util.Set[]` objects using Spring Boot's `javadoc:org.springframework.boot.context.properties.bind.Binder[]` class. For more details see the [features/external-config.html](#) section below.



YAML files cannot be loaded by using the `javadoc:org.springframework.context.annotation.PropertySource[format=annotation]` or `javadoc:org.springframework.test.context.TestPropertySource[format=annotation]` annotations. So, in the case that you need to load values that way, you need to use a properties file.

28.5.2. Directly Loading YAML

Spring Framework provides two convenient classes that can be used to load YAML documents. The `javadoc:org.springframework.beans.factory.config.YamlPropertiesFactoryBean[]` loads YAML as `javadoc:java.util.Properties[]` and the `javadoc:org.springframework.beans.factory.config.YamlMapFactoryBean[]` loads YAML as a `javadoc:java.util.Map[]`.

You can also use the `javadoc:org.springframework.boot.env.YamlPropertySourceLoader[]` class if you want to load YAML as a `Spring javadoc:org.springframework.core.env.PropertySource[]`.

28.6. Configuring Random Values

The `javadoc:org.springframework.boot.env.RandomValuePropertySource[]` is useful for injecting random values (for example, into secrets or test cases). It can produce integers, longs, uuids, or strings, as shown in the following example:

```
my:
  secret: "${random.value}"
  number: "${random.int}"
  bignumber: "${random.long}"
  uuid: "${random.uuid}"
  number-less-than-ten: "${random.int(10)}"
  number-in-range: "${random.int[1024,65536]}"
```


The `random.int*` syntax is `OPEN value (,max) CLOSE` where the `OPEN,CLOSE` are any character and `value,max` are integers. If `max` is provided, then `value` is the minimum value and `max` is the maximum value (exclusive).

28.7. Configuring System Environment Properties

Spring Boot supports setting a prefix for environment properties. This is useful if the system environment is shared by multiple Spring Boot applications with different configuration requirements. The prefix for system environment properties can be set directly on `javadoc.org.springframework.boot.SpringApplication[]` by calling the `setEnvironmentPrefix(...)` method before the application is run.

For example, if you set the prefix to `input`, a property such as `remote.timeout` will be resolved as `INPUT_REMOTE_TIMEOUT` in the system environment.



The prefix *only* applies to system environment properties. The example above would continue to use `remote.timeout` when reading properties from other sources.

28.8. Type-safe Configuration Properties

Using the `@Value("${property}")` annotation to inject configuration properties can sometimes be cumbersome, especially if you are working with multiple properties or your data is hierarchical in nature. Spring Boot provides an alternative method of working with properties that lets strongly typed beans govern and validate the configuration of your application.



See also the [differences between javadoc.org.springframework.beans.factory.annotation.Value\[format=annotation\]](#) and type-safe configuration properties].

28.8.1. JavaBean Properties Binding

It is possible to bind a bean declaring standard JavaBean properties as shown in the following example:

```
include-code::MyProperties[]
```

The preceding POJO defines the following properties:

- `my.service.enabled`, with a value of `false` by default.
- `my.service.remote-address`, with a type that can be coerced from `javadoc.java.lang.String[]`.
- `my.service.security.username`, with a nested "security" object whose name is determined by the name of the property. In particular, the type is not used at all there and could have been `javadoc.org.springframework.boot.security.autoconfigure.SecurityProperties[]`.
- `my.service.security.password`.
- `my.service.security.roles`, with a collection of `javadoc.java.lang.String[]` that defaults to `USER`.



To use a reserved keyword in the name of a property, such as `my.service.import`, use the `javadoc.org.springframework.boot.context.properties.bind.Name[format=annotation]` annotation on the property's field.



The properties that map to `javadoc.org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` classes available in Spring Boot, which are configured through properties files, YAML files, environment variables, and other mechanisms, are public API but the accessors (getters/setters) of the class itself are not meant to be used directly.

Such arrangement relies on a default empty constructor and getters and setters are usually mandatory, since binding is through standard Java Beans property descriptors, just like in Spring MVC. A setter may be omitted in the following cases:



- Pre-initialized Maps and Collections, as long as they are initialized with a mutable implementation (like the `roles` field in the preceding example).
- Pre-initialized nested POJOs (like the `Security` field in the preceding example). If you want the binder to create the instance on the fly by using its default constructor, you need a setter.

Some people use Project Lombok to add getters and setters automatically. Make sure that Lombok does not generate any particular constructor for such a type, as it is used automatically by the container to instantiate the object.

Finally, only standard Java Bean properties are considered and binding on static properties is not supported.

28.8.2. Constructor Binding

The example in the previous section can be rewritten in an immutable fashion as shown in the following example:

```
include-code::MyProperties[]
```

In this setup, the presence of a single parameterized constructor implies that constructor binding should be used. This means that the binder will find a constructor with the parameters that you wish to have bound. If your class has multiple constructors, the `javadoc.org.springframework.boot.context.properties.bind.ConstructorBinding[format=annotation]` annotation can be used to specify which constructor to use for constructor binding.

To opt-out of constructor binding for a class, the parameterized constructor must be annotated with `javadoc.org.springframework.beans.factory.annotation.Autowired[format=annotation]` or made `private`. Kotlin developers can use an empty primary constructor to opt-out of constructor binding.

For example:


```
include-code::primaryconstructor/MyProperties[]
```

Constructor binding can be used with records. Unless your record has multiple constructors, there is no need to use `javadoc:org.springframework.boot.context.properties.bind.ConstructorBinding[format=annotation]`.

Nested members of a constructor bound class (such as `Security` in the example above) will also be bound through their constructor.

Default values can be specified using `javadoc:org.springframework.boot.context.properties.bind.DefaultValue[format=annotation]` on constructor parameters and record components. The conversion service will be applied to coerce the annotation's `javadoc:java.lang.String[]` value to the target type of a missing property.

Referring to the previous example, if no properties are bound to `Security`, the `MyProperties` instance will contain a `null` value for `security`. To make it contain a non-null instance of `Security` even when no properties are bound to it (when using Kotlin, this will require the `username` and `password` parameters of `Security` to be declared as nullable as they do not have default values), use an empty `javadoc:org.springframework.boot.context.properties.bind.DefaultValue[format=annotation]` annotation:

```
include-code::nonnull/MyProperties[tag=*
```



To use constructor binding the class must be enabled using `javadoc:org.springframework.boot.context.properties.EnableConfigurationProperties[format=annotation]` or configuration property scanning. You cannot use constructor binding with beans that are created by the regular Spring mechanisms (for example `javadoc:org.springframework.stereotype.Component[format=annotation]` beans, beans created by using `javadoc:org.springframework.context.annotation.Bean[format=annotation]` methods or beans loaded by using `javadoc:org.springframework.context.annotation.Import[format=annotation]`)



To use constructor binding the class must be compiled with `-parameters`. This will happen automatically if you use Spring Boot's Gradle plugin or if you use Maven and `spring-boot-starter-parent`.



The use of `javadoc:java.util.Optional[]` with `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` is not recommended as it is primarily intended for use as a return type. As such, it is not well-suited to configuration property injection. For consistency with properties of other types, if you do declare an `javadoc:java.util.Optional[]` property and it has no value, `null` rather than an empty `javadoc:java.util.Optional[]` will be bound.



To use a reserved keyword in the name of a property, such as `my.service.import`, use the `javadoc:org.springframework.boot.context.properties.bind.Name[format=annotation]` annotation on the constructor parameter.

28.8.3. Enabling @ConfigurationProperties-annotated Types

Spring Boot provides infrastructure to bind `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` types and register them as beans. You can either enable configuration properties on a class-by-class basis or enable configuration property scanning that works in a similar manner to component scanning.

Sometimes, classes annotated with `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` might not be suitable for scanning, for example, if you're developing your own auto-configuration or you want to enable them conditionally. In these cases, specify the list of types to process using the `javadoc:org.springframework.boot.context.properties.EnableConfigurationProperties[format=annotation]` annotation. This can be done on any `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class, as shown in the following example:

```
include-code::MyConfiguration[] include-code::SomeProperties[]
```

To use configuration property scanning, add the `javadoc:org.springframework.boot.context.properties.ConfigurationPropertiesScan[format=annotation]` annotation to your application. Typically, it is added to the main application class that is annotated with `javadoc:org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` but it can be added to any `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class. By default, scanning will occur from the package of the class that declares the annotation. If you want to define specific packages to scan, you can do so as shown in the following example:

```
include-code::MyApplication[]
```



When the `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` bean is registered using configuration property scanning or through `javadoc:org.springframework.boot.context.properties.EnableConfigurationProperties[format=annotation]`, the bean has a conventional name: `<prefix>-<fqqn>`, where `<prefix>` is the environment key prefix specified in the `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` annotation and `<fqqn>` is the fully qualified name of the bean. If the annotation does not provide any prefix, only the fully qualified name of the bean is used.

Assuming that it is in the `com.example.app` package, the bean name of the `SomeProperties` example above is `some.properties-com.example.app.SomeProperties`.

We recommend that `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` only deal with the environment and, in particular, does not inject other beans from the context. For corner cases, setter injection can be used or any of the `*Aware` interfaces provided by the framework (such as `javadoc:org.springframework.context.EnvironmentAware[]` if you need access to the `javadoc:org.springframework.core.env.Environment[]`). If you still want to inject other beans using the constructor, the configuration properties bean must be annotated with `javadoc:org.springframework.stereotype.Component[format=annotation]` and use JavaBean-based property binding.

28.8.4. Using @ConfigurationProperties-annotated Types

This style of configuration works particularly well with the `javadoc:org.springframework.boot.SpringApplication[]` external YAML configuration, as shown in the following example:

```
my:
  service:
    remote-address: 192.168.1.1
    security:
      username: "admin"
      roles:
        - "USER"
        - "ADMIN"
```

To work with `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans, you can inject them in the same way as any other bean, as shown in the following example:

```
include-code::MyService[]
```



Using `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` also lets you generate metadata files that can be used by IDEs to offer auto-completion for your own keys. See the [appendix](#) for details.

28.8.5. Third-party Configuration

As well as using `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` to annotate a class, you can also use it on public `javadoc:org.springframework.context.annotation.Bean[format=annotation]` methods. Doing so can be particularly useful when you want to bind properties to third-party components that are outside of your control.

To configure a bean from the `javadoc:org.springframework.core.env.Environment[]` properties, add `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` to its bean registration, as shown in the following example:

```
include-code::ThirdPartyConfiguration[]
```

Any JavaBean property defined with the `another` prefix is mapped onto that `AnotherComponent` bean in manner similar to the preceding `SomeProperties` example.

28.8.6. Relaxed Binding

Spring Boot uses some relaxed rules for binding `javadoc:org.springframework.core.env.Environment[]` properties to `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans, so there does not need to be an exact match between the `javadoc:org.springframework.core.env.Environment[]` property name and the bean property name. Common examples where this is useful include dash-separated environment properties (for example, `context-path` binds to `contextPath`), and capitalized environment properties (for example, `PORT` binds to `port`).

As an example, consider the following `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` class:

```
include-code::MyPersonProperties[]
```

With the preceding code, the following properties names can all be used:

Table 3. relaxed binding

Property	Note
<code>my.main-project.person.first-name</code>	Kebab case, which is recommended for use in <code>.properties</code> and YAML files.
<code>my.main-project.person.firstName</code>	Standard camel case syntax.
<code>my.main-project.person.first_name</code>	Underscore notation, which is an alternative format for use in <code>.properties</code> and YAML files.
<code>MY_MAINPROJECT_PERSON_FIRSTNAME</code>	Upper case format, which is recommended when using system environment variables.



The `prefix` value for the annotation *must* be in kebab case (lowercase and separated by `-`, such as `my.main-project.person`).

Table 4. relaxed binding rules per property source

Property Source	Simple	List
Properties Files	Camel case, kebab case, or underscore notation	Standard list syntax using <code>[]</code> or comma-separated values
YAML Files	Camel case, kebab case, or underscore notation	Standard YAML list syntax or comma-separated values
Environment Variables	Upper case format with underscore as the delimiter (see features/external-config.html).	Numeric values surrounded by underscores (see features/external-config.html)
System properties	Camel case, kebab case, or underscore notation	Standard list syntax using <code>[]</code> or comma-separated values



We recommend that, when possible, properties are stored in lower-case kebab format, such as `my.person.first-name=Rod`.

Binding Maps

When binding to `java.util.Map[]` properties you may need to use a special bracket notation so that the original `key` value is preserved. If the key is not surrounded by `[]`, any characters that are not alpha-numeric, `-` or `.` are removed.

For example, consider binding the following properties to a `Map<String,String>`:

```
my:
  map:
    "[/key1]": "value1"
    "[/key2]": "value2"
    "/key3": "value3"
```



For YAML files, the brackets need to be surrounded by quotes for the keys to be parsed properly.

The properties above will bind to a `java.util.Map[]` with `/key1`, `/key2` and `key3` as the keys in the map. The slash has been removed from `key3` because it was not surrounded by square brackets.

When binding to scalar values, keys with `.` in them do not need to be surrounded by `[]`. Scalar values include enums and all types in the `java.lang` package except for `java.lang.Object[]`. Binding `a.b=c` to `Map<String,String>` will preserve the `.` in the key and return a Map with the entry `{"a.b"="c"}`. For any other types you need to use the bracket notation if your `key` contains a `.`. For example, binding `a.b=c` to `Map<String, Object>` will return a Map with the entry `{"a"="{\"b\"=\"c\"}"}` whereas `[a.b]=c` will return a Map with the entry `{"a.b"="c"}`.

Binding From Environment Variables

Most operating systems impose strict rules around the names that can be used for environment variables. For example, Linux shell variables can contain only letters (`a` to `z` or `A` to `Z`), numbers (`0` to `9`) or the underscore character (`_`). By convention, Unix shell variables will also have their names in UPPERCASE.

Spring Boot's relaxed binding rules are, as much as possible, designed to be compatible with these naming restrictions.

To convert a property name in the canonical-form to an environment variable name you can follow these rules:

- Replace dots (`.`) with underscores (`_`).
- Remove any dashes (`-`).
- Convert to uppercase.

For example, the configuration property `spring.main.log-startup-info` would be an environment variable named `SPRING_MAIN_LOGSTARTUPINFO`.

Environment variables can also be used when binding to object lists. To bind to a `java.util.List[]`, the element number should be surrounded with underscores in the variable name.

For example, the configuration property `my.service[0].other` would use an environment variable named `MY_SERVICE_0_OTHER`.

Support for binding from environment variables is applied to the `systemEnvironment` property source and to any additional property source whose name ends with `-systemEnvironment`.

Binding Maps From Environment Variables

When Spring Boot binds an environment variable to a property class, it lowercases the environment variable name before binding. Most of the time this detail isn't important, except when binding to `java.util.Map[]` properties.

The keys in the `java.util.Map[]` are always in lowercase, as seen in the following example:

```
include-code::MyMapsProperties[]
```

When setting `MY_PROPS_VALUES_KEY=value`, the `values` `java.util.Map[]` contains a `{"key"="value"}` entry.

Only the environment variable **name** is lower-cased, not the value. When setting `MY_PROPS_VALUES_KEY=VALUE`, the `values` `java.util.Map[]` contains a `{"key"="VALUE"}` entry.

Caching

Relaxed binding uses a cache to improve performance. By default, this caching is only applied to immutable property sources. To customize this behavior, for example to enable caching for mutable property sources, use `java.org.springframework.boot.context.properties.source.ConfigurationPropertyCaching[]`.

28.8.7. Merging Complex Types

When lists are configured in more than one place, overriding works by replacing the entire list.

For example, assume a `MyPojo` object with `name` and `description` attributes that are `null` by default. The following example exposes a list of `MyPojo` objects from `MyProperties`:

```
include-code::list/MyProperties[]
```

Consider the following configuration:

```

my:
  list:
    - name: "my name"
      description: "my description"
---
spring:
  config:
    activate:
      on-profile: "dev"
my:
  list:
    - name: "my another name"

```

If the `dev` profile is not active, `MyProperties.list` contains one `MyPojo` entry, as previously defined. If the `dev` profile is enabled, however, the `list` *still* contains only one entry (with a name of `my another name` and a description of `null`). This configuration *does not* add a second `MyPojo` instance to the list, and it does not merge the items.

When a `java.util.List[]` is specified in multiple profiles, the one with the highest priority (and only that one) is used. Consider the following example:

```

my:
  list:
    - name: "my name"
      description: "my description"
    - name: "another name"
      description: "another description"
---
spring:
  config:
    activate:
      on-profile: "dev"
my:
  list:
    - name: "my another name"

```

In the preceding example, if the `dev` profile is active, `MyProperties.list` contains *one* `MyPojo` entry (with a name of `my another name` and a description of `null`). For YAML, both comma-separated lists and YAML lists can be used for completely overriding the contents of the list.

For `java.util.Map[]` properties, you can bind with property values drawn from multiple sources. However, for the same property in multiple sources, the one with the highest priority is used. The following example exposes a `Map<String, MyPojo>` from `MyProperties`:

include-code::map/MyProperties[]

Consider the following configuration:

```

my:
  map:
    key1:
      name: "my name 1"
      description: "my description 1"
---
spring:
  config:
    activate:
      on-profile: "dev"
my:
  map:
    key1:
      name: "dev name 1"
    key2:
      name: "dev name 2"
      description: "dev description 2"

```

If the `dev` profile is not active, `MyProperties.map` contains one entry with key `key1` (with a name of `my name 1` and a description of `my description 1`). If the `dev` profile is enabled, however, `map` contains two entries with keys `key1` (with a name of `dev name 1` and a description of `my description 1`) and `key2` (with a name of `dev name 2` and a description of `dev description 2`).



The preceding merging rules apply to properties from all property sources, and not just files.

28.8.8. Properties Conversion

Spring Boot attempts to coerce the external application properties to the right type when it binds to the `org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans. If you need custom type conversion, you can provide a `org.springframework.core.convert.ConversionService[]` bean (with a bean named `conversionService`) or custom property editors (through a `org.springframework.beans.factory.config.CustomEditorConfigurer[]` bean) or custom converters (with bean definitions annotated as `org.springframework.boot.context.properties.ConfigurationPropertiesBinding[format=annotation]`).



Beans used for property conversion are requested very early during the application lifecycle so make sure to limit the dependencies that your `org.springframework.core.convert.ConversionService[]` is using. Typically, any dependency that you require may not be fully initialized at creation time.



You may want to rename your custom `javadoc:org.springframework.core.convert.ConversionService[]` if it is not required for configuration keys coercion and only rely on custom converters qualified with `javadoc:org.springframework.boot.context.properties.ConfigurationPropertiesBinding[format=annotation]`. When qualifying a `@Bean` method with `@ConfigurationPropertiesBinding`, the method should be **static** to avoid “bean is not eligible for getting processed by all BeanPostProcessors” warnings.

Converting Durations

Spring Boot has dedicated support for expressing durations. If you expose a `javadoc:java.time.Duration[]` property, the following formats in application properties are available:

- A regular `long` representation (using milliseconds as the default unit unless a `javadoc:org.springframework.boot.convert.DurationUnit[format=annotation]` has been specified)
- The standard ISO-8601 format `{apiref-openjdk}/java.base/java/time/Duration.html#parse(java.lang.CharSequence)` (used by `javadoc:java.time.Duration[]`)
- A more readable format where the value and the unit are coupled (`10s` means 10 seconds)

Consider the following example:

```
include-code::javabeanbinding/MyProperties[]
```

To specify a session timeout of 30 seconds, `30`, `PT30S` and `30s` are all equivalent. A read timeout of 500ms can be specified in any of the following form: `500`, `PT0.5S` and `500ms`.

You can also use any of the supported units. These are:

- `ns` for nanoseconds
- `us` for microseconds
- `ms` for milliseconds
- `s` for seconds
- `m` for minutes
- `h` for hours
- `d` for days

The default unit is milliseconds and can be overridden using `javadoc:org.springframework.boot.convert.DurationUnit[format=annotation]` as illustrated in the sample above.

If you prefer to use constructor binding, the same properties can be exposed, as shown in the following example:

```
include-code::constructorbinding/MyProperties[]
```



If you are upgrading a `javadoc:java.lang.Long[]` property, make sure to define the unit (using `javadoc:org.springframework.boot.convert.DurationUnit[format=annotation]`) if it is not milliseconds. Doing so gives a transparent upgrade path while supporting a much richer format.

Converting Periods

In addition to durations, Spring Boot can also work with `javadoc:java.time.Period[]` type. The following formats can be used in application properties:

- A regular `int` representation (using days as the default unit unless a `javadoc:org.springframework.boot.convert.PeriodUnit[format=annotation]` has been specified)
- The standard ISO-8601 format `{apiref-openjdk}/java.base/java/time/Period.html#parse(java.lang.CharSequence)` (used by `javadoc:java.time.Period[]`)
- A simpler format where the value and the unit pairs are coupled (`1y3d` means 1 year and 3 days)

The following units are supported with the simple format:

- `y` for years
- `m` for months
- `w` for weeks
- `d` for days



The `javadoc:java.time.Period[]` type never actually stores the number of weeks, it is a shortcut that means “7 days”.

Converting Data Sizes

Spring Framework has a `javadoc:org.springframework.util.unit.DataSize[]` value type that expresses a size in bytes. If you expose a `javadoc:org.springframework.util.unit.DataSize[]` property, the following formats in application properties are available:

- A regular `long` representation (using bytes as the default unit unless a `javadoc:org.springframework.boot.convert.DataSizeUnit[format=annotation]` has been specified)
- A more readable format where the value and the unit are coupled (`10MB` means 10 megabytes)

Consider the following example:

```
include-code::javabeanbinding/MyProperties[]
```

To specify a buffer size of 10 megabytes, `10` and `10MB` are equivalent. A size threshold of 256 bytes can be specified as `256` or `256B`.

You can also use any of the supported units. These are:

- B for bytes
- KB for kilobytes
- MB for megabytes
- GB for gigabytes
- TB for terabytes

The default unit is bytes and can be overridden using `javadoc:org.springframework.boot.convert.DataSizeUnit[format=annotation]` as illustrated in the sample above.

If you prefer to use constructor binding, the same properties can be exposed, as shown in the following example:

include-code::constructorbinding/MyProperties[]



If you are upgrading a `javadoc:java.lang.Long[]` property, make sure to define the unit (using `javadoc:org.springframework.boot.convert.DataSizeUnit[format=annotation]`) if it is not bytes. Doing so gives a transparent upgrade path while supporting a much richer format.

Converting Base64 Data

Spring Boot supports resolving binary data that have been Base64 encoded. If you expose a `Resource` property, the base64 encoded text can be provided as the value with a `base64:` prefix, as shown in the following example:

```
my:
  property: base64:SGVsbG8gV29ybGQ=
```



The `Resource` property can also be used to provide the path to the resource, making it more versatile.

28.8.9. @ConfigurationProperties Validation

Spring Boot attempts to validate `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` classes whenever they are annotated with Spring's `javadoc:org.springframework.validation.annotation.Validated[format=annotation]` annotation. You can use JSR-303 `jakarta.validation` constraint annotations directly on your configuration class. To do so, ensure that a compliant JSR-303 implementation is on your classpath and then add constraint annotations to your fields, as shown in the following example:

include-code::MyProperties[]



You can also trigger validation by annotating the `javadoc:org.springframework.context.annotation.Bean[format=annotation]` method that creates the configuration properties with `javadoc:org.springframework.validation.annotation.Validated[format=annotation]`.

To cascade validation to nested properties the associated field must be annotated with `javadoc:jakarta.validation.Valid[format=annotation]`. The following example builds on the preceding `MyProperties` example:

include-code::nested/MyProperties[]

You can also add a custom Spring `javadoc:org.springframework.validation.Validator[]` by creating a bean definition called `configurationPropertiesValidator`. The `javadoc:org.springframework.context.annotation.Bean[format=annotation]` method should be declared `static`. The configuration properties validator is created very early in the application's lifecycle, and declaring the `javadoc:org.springframework.context.annotation.Bean[format=annotation]` method as static lets the bean be created without having to instantiate the `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class. Doing so avoids any problems that may be caused by early instantiation.



The `spring-boot-actuator` module includes an endpoint that exposes all `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans. Point your web browser to `/actuator/configprops` or use the equivalent JMX endpoint. See the [Production ready features](#) section for details.

28.8.10. @ConfigurationProperties vs. @Value

The `javadoc:org.springframework.beans.factory.annotation.Value[format=annotation]` annotation is a core container feature, and it does not provide the same features as type-safe configuration properties. The following table summarizes the features that are supported by `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` and `javadoc:org.springframework.beans.factory.annotation.Value[format=annotation]`:

Feature	@ConfigurationProperties	@Value
Relaxed binding	Yes	Limited (see note below)
Meta-data support	Yes	No
SpEL evaluation	No	Yes



If you do want to use `javax.annotation.Value`, we recommend that you refer to property names using their canonical form (kebab-case using only lowercase letters). This will allow Spring Boot to use the same logic as it does when [relaxed binding](#) `ConfigurationProperties`.

For example, `@Value("${demo.item-price}")` will pick up `demo.item-price` and `demo.itemPrice` forms from the `application.properties` file, as well as `DEMO_ITEMPRICE` from the system environment. If you used `@Value("${demo.itemPrice}")` instead, `demo.item-price` and `DEMO_ITEMPRICE` would not be considered.

If you define a set of configuration keys for your own components, we recommend you group them in a POJO annotated with `ConfigurationProperties`. Doing so will provide you with structured, type-safe object that you can inject into your own beans.

SpEL expressions from [application property files](#) are not processed at time of parsing these files and populating the environment. However, it is possible to write a SpEL expression in `Value`. If the value of a property from an application property file is a SpEL expression, it will be evaluated when consumed through `Value`.

29. Core Features

This section dives into the details of Spring Boot. Here you can learn about the key features that you may want to use and customize. If you have not already done so, you might want to read the [tutorial/index.html](#) and [using/index.html](#) sections, so that you have a good grounding of the basics.

30. Internationalization

Spring Boot supports localized messages so that your application can cater to users of different language preferences. By default, Spring Boot looks for the presence of a `messages` resource bundle at the root of the classpath.



The auto-configuration applies when the default properties file for the configured resource bundle is available (`messages.properties` by default). If your resource bundle contains only language-specific properties files, you are required to add the default. If no properties file is found that matches any of the configured base names, there will be no auto-configured `org.springframework.context.MessageSource[]`.

The basename of the resource bundle as well as several other attributes can be configured using the `spring.messages` namespace, as shown in the following example:

```
spring:
  messages:
    basename: "messages, config.i18n.messages"
    common-messages: "classpath:my-common-messages.properties"
    fallback-to-system-locale: false
```



The `configprop:spring.messages.basename[]` property supports a list of locations, either a package qualifier or a resource resolved from the classpath root. The `configprop:spring.messages.common-messages[]` property supports a list of property file resources.

See `org.springframework.boot.autoconfigure.context.MessageSourceProperties[]` for more supported options.

31. JSON

Spring Boot provides integration with the following JSON mapping libraries:

- Jackson 3
- Jackson 2
- Gson
- JSON-B
- Kotlin Serialization

Jackson 3 is the preferred and default library.

Support for Jackson 2 is deprecated and will be removed in a future Spring Boot 4.x release. It is provided purely to ease the migration from Jackson 2 to Jackson 3 and should not be relied up in the longer term.

31.1. Jackson 3

Auto-configuration for Jackson 3 is provided and Jackson is part of `spring-boot-starter-json`. When Jackson is on the classpath a `javax.json.JsonMapper` bean is automatically configured. Several configuration properties are provided for [customizing the configuration of the `javax.json.JsonMapper`](#).

31.1.1. Custom Serializers and Deserializers

If you use Jackson to serialize and deserialize JSON data, you might want to write your own `javax.json.JsonSerializer` and `javax.json.JsonDeserializer` classes. Custom serializers are usually [registered with Jackson through a module](#) (<https://github.com/FasterXML/jackson-docs/wiki/JacksonHowToCustomSerializers>), but Spring Boot provides an alternative `javax.json.JsonComponent` annotation that makes it easier to directly register Spring Beans.

You can use the `javax.json.JsonComponent` annotation directly on `javax.json.JsonSerializer`, `javax.json.JsonDeserializer` or `javax.json.JsonKeyDeserializer` implementations. You can also use it on classes that contain serializers/deserializers as inner classes, as shown in the following example:

```
include-code::MyJacksonComponent[]
```

All `javax.json.JsonComponent` beans in the `javax.json.JsonContext` are automatically registered with Jackson. Because `javax.json.JsonComponent` is meta-annotated with `javax.json.JsonComponent`, the usual component-scanning rules apply.

Spring Boot also provides `javax.json.JsonObjectSerializer` and `javax.json.JsonObjectDeserializer` base classes that provide useful alternatives to the standard Jackson versions when serializing objects. See `javax.json.JsonObjectSerializer` and `javax.json.JsonObjectDeserializer` in the API documentation for details.

The example above can be rewritten to use `javax.json.JsonObjectSerializer` and `javax.json.JsonObjectDeserializer` as follows:

```
include-code::object/MyJacksonComponent[]
```

31.1.2. Mixins

Jackson has support for mixins that can be used to mix additional annotations into those already declared on a target class. Spring Boot's Jackson auto-configuration will scan your application's packages for classes annotated with `javax.json.JsonMixin` and register them with the auto-configured `javax.json.JsonMapper`. The registration is performed by Spring Boot's `javax.json.JsonMixinModule`.

31.2. Jackson 2

Deprecated auto-configuration for Jackson 2 is provided by the `spring-boot-jackson2` module. When this module is on the classpath a `com.fasterxml.jackson.databind.ObjectMapper` bean is automatically configured. Several `spring.jackson2.*` configuration properties are provided for customizing the configuration. To take more control, define one or more `javax.json.Json2ObjectMapperBuilderCustomizer` beans.

When both Jackson 3 and Jackson 2 are present, various configuration properties can be used to indicate that Jackson 2 is preferred:

- `configprop:spring.graphql.rsocket.preferred-json-mapper`
- `configprop:spring.http.codecs.preferred-json-mapper` (used by Spring WebFlux and reactive HTTP clients)
- `configprop:spring.http.converters.preferred-json-mapper` (used by Spring MVC and imperative HTTP clients)
- `configprop:spring.rsocket.preferred-mapper`
- `configprop:spring.websocket.messaging.preferred-json-mapper`

In each case, set the relevant property to `jackson2` to indicate that Jackson 2 is preferred.

31.3. Gson

Auto-configuration for Gson is provided. When Gson is on the classpath a `com.google.gson.Gson` bean is automatically configured. Several `spring.gson.*` configuration properties are provided for customizing the configuration. To take more control, one or more `javax.json.GsonBuilderCustomizer` beans can be used.

31.4. JSON-B

Auto-configuration for JSON-B is provided. When the JSON-B API and an implementation are on the classpath a `jakarta.json.bind.Jsonb[]` bean will be automatically configured. The preferred JSON-B implementation is Eclipse Yasson for which dependency management is provided.

31.5. Kotlin Serialization

Auto-configuration for Kotlin Serialization is provided. When `kotlinx-serialization-json` is on the classpath a `Json`

(<https://kotlinlang.org/api/kotlinx.serialization/kotlinx-serialization-json/kotlinx.serialization.json/-json/>) bean is automatically configured. Several `spring.kotlinx.serialization.json.*` configuration properties are provided for customizing the configuration.

32. Kotlin Support

Kotlin (<https://kotlinlang.org>) is a statically-typed language targeting the JVM (and other platforms) which allows writing concise and elegant code while providing [{url-kotlin-docs}/java-interop.html](#) [interoperability] with existing libraries written in Java.

Spring Boot provides Kotlin support by leveraging the support in other Spring projects such as Spring Framework, Spring Data, and Reactor. See the [{url-spring-framework-docs}/languages/kotlin.html](#) [Spring Framework Kotlin support documentation] for more information.

The easiest way to start with Spring Boot and Kotlin is to follow [this comprehensive tutorial](https://spring.io/guides/tutorials/spring-boot-kotlin/) (<https://spring.io/guides/tutorials/spring-boot-kotlin/>). You can create new Kotlin projects by using [start.spring.io](https://start.spring.io/#language=kotlin) (<https://start.spring.io/#language=kotlin>). Feel free to join the #spring channel of [Kotlin Slack](https://slack.kotlinlang.org/) (<https://slack.kotlinlang.org/>) or ask a question with the `spring` and `kotlin` tags on [Stack Overflow](https://stackoverflow.com/questions/tagged/spring+kotlin) (<https://stackoverflow.com/questions/tagged/spring+kotlin>) if you need support.

32.1. Requirements

Spring Boot requires at least Kotlin 2.2.x and manages a suitable Kotlin version through dependency management. To use Kotlin, `org.jetbrains.kotlin:kotlin-stdlib` and `org.jetbrains.kotlin:kotlin-reflect` must be present on the classpath.

Kotlin 2.2.x introduces [new defaulting rules for propagating annotations to parameters, fields, and properties](https://kotlinlang.org/docs/whatsnew22.html#new-defaulting-rules-for-use-site-annotation-targets) (<https://kotlinlang.org/docs/whatsnew22.html#new-defaulting-rules-for-use-site-annotation-targets>). In order to avoid related warnings and use what will likely become the Kotlin default behavior in an upcoming version, it is recommended to configure the `-Xannotation-default-target=param-property` compiler flag.

Since [Kotlin classes are final by default](https://discuss.kotlinlang.org/t/classes-final-by-default/166) (<https://discuss.kotlinlang.org/t/classes-final-by-default/166>), you are likely to want to configure [{url-kotlin-docs}/compiler-plugins.html#spring-support](#) [kotlin-spring] plugin in order to automatically open Spring-annotated classes so that they can be proxied.

[Jackson's Kotlin module](https://github.com/FasterXML/jackson-module-kotlin) (<https://github.com/FasterXML/jackson-module-kotlin>) is required for serializing / deserializing JSON data in Kotlin. It is automatically registered when found on the classpath.



These dependencies and plugins are provided by default if one bootstraps a Kotlin project on [start.spring.io](https://start.spring.io/#language=kotlin) (<https://start.spring.io/#language=kotlin>).

32.2. Null-safety

One of Kotlin's key features is [{url-kotlin-docs}/null-safety.html](#) [null-safety]. It deals with `null` values at compile time rather than deferring the problem to runtime and encountering a `java.lang.NullPointerException`. This helps to eliminate a common source of bugs without paying the cost of wrappers like `java.util.Optional`. Kotlin also allows using functional constructs with nullable values as described in this [comprehensive guide to null-safety in Kotlin](https://www.baeldung.com/kotlin-null-safety) (<https://www.baeldung.com/kotlin-null-safety>).

Although Java does not let you express null-safety in its type-system, most Spring projects provide [{url-spring-framework-docs}/core/null-safety.html](#) [null-safety] via [JSpecify](https://jspecify.dev/) (<https://jspecify.dev/>) annotations.

As of Kotlin 2.1, Kotlin enforces strict handling of nullability annotations from the `org.jspecify.annotations` package.

32.3. Kotlin API

32.3.1. runApplication

Spring Boot provides an idiomatic way to run an application with `runApplication<MyApplication>(*args)` as shown in the following example:

```
import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
class MyApplication

fun main(args: Array<String>) {
    runApplication<MyApplication>(*args)
}
```

KOTLIN

This is a drop-in replacement for `SpringApplication.run(MyApplication::class.java, *args)`. It also allows customization of the application as shown in the following example:

```
runApplication<MyApplication>(*args) {
    setBannerMode(OFF)
}
```

KOTLIN

32.3.2. Extensions

Kotlin [{url-kotlin-docs}/extensions.html](#) [extensions] provide the ability to extend existing classes with additional functionality. The Spring Boot Kotlin API makes use of these extensions to add new Kotlin specific conveniences to existing APIs.

`org.springframework.boot.testclient.TestRestTemplate` extensions, similar to those provided by Spring Framework for `org.springframework.web.client.RestOperations` in Spring Framework, are provided. Among other things, the extensions make it possible to take advantage of Kotlin reified type parameters.

32.4. Dependency Management

In order to avoid mixing different versions of Kotlin dependencies on the classpath, Spring Boot imports the Kotlin BOM.

With Maven, the Kotlin version can be customized by setting the `kotlin.version` property and plugin management is provided for `kotlin-maven-plugin`. With Gradle, the Spring Boot plugin automatically aligns the `kotlin.version` with the version of the Kotlin plugin.

Spring Boot also manages the version of Coroutines dependencies by importing the Kotlin Coroutines BOM. The version can be customized by setting the `kotlin-coroutines.version` property.



`org.jetbrains.kotlinx:kotlinx-coroutines-reactor` dependency is provided by default if one bootstraps a Kotlin project with at least one reactive dependency on [start.spring.io](https://start.spring.io/#!language=kotlin) (<https://start.spring.io/#!language=kotlin>).

32.5. @ConfigurationProperties

`javax.annotation.processing.ConfigurationProperties[format=annotation]` when used in combination with [constructor binding](#) supports data classes with immutable `val` properties as shown in the following example:

```
@ConfigurationProperties("example.kotlin")
data class KotlinExampleProperties(
    val name: String,
    val description: String,
    val myService: MyService) {

    data class MyService(
        val apiToken: String,
        val uri: URI
    )
}
```

KOTLIN

Due to the limitations of their interoperability with Java, support for value classes is limited. In particular, relying upon a value class's default value will not work with configuration property binding. In such cases, a data class should be used instead.



To generate [your own metadata](#) using the annotation processor, `{url-kotlin-docs}/kapt.html` [`kapt` should be configured] with the `spring-boot-configuration-processor` dependency. Note that some features (such as detecting the default value or deprecated items) are not working due to limitations in the model `kapt` provides.

32.6. Testing

While it is possible to use JUnit 4 to test Kotlin code, JUnit 6 is provided by default and is recommended. JUnit 6 enables a test class to be instantiated once and reused for all of the class's tests. This makes it possible to use `javax.annotation.processing.BeforeAll[format=annotation]` and `javax.annotation.processing.AfterAll[format=annotation]` annotations on non-static methods, which is a good fit for Kotlin.

To mock Kotlin classes, [MockK](https://mockk.io/) (<https://mockk.io/>) is recommended. If you need the `MockK` equivalent of the Mockito specific `@MockitoBean` and `javax.annotation.processing.test.context.bean.override.mockito.MockitoSpyBean[format=annotation]` annotations, you can use [SpringMockK](https://github.com/Ninja-Squad/springmockk) (<https://github.com/Ninja-Squad/springmockk>) which provides similar `@MockkBean` and `@SpykBean` annotations.

32.7. Resources

32.7.1. Further Reading

- [{url-kotlin-docs}\[Kotlin language reference\]](#)
- [Kotlin Slack](https://kotlinlang.slack.com/) (<https://kotlinlang.slack.com/>) (with a dedicated `#spring` channel)
- [Stack Overflow with spring and kotlin tags](https://stackoverflow.com/questions/tagged/spring+kotlin) (<https://stackoverflow.com/questions/tagged/spring+kotlin>)
- [Try Kotlin in your browser](https://try.kotlinlang.org/) (<https://try.kotlinlang.org/>)
- [Kotlin blog](https://blog.jetbrains.com/kotlin/) (<https://blog.jetbrains.com/kotlin/>)
- [Awesome Kotlin](https://kotlin.link/) (<https://kotlin.link/>)
- [Tutorial: building web applications with Spring Boot and Kotlin](https://spring.io/guides/tutorials/spring-boot-kotlin/) (<https://spring.io/guides/tutorials/spring-boot-kotlin/>)
- [Developing Spring Boot applications with Kotlin](https://spring.io/blog/2016/02/15/developing-spring-boot-applications-with-kotlin) (<https://spring.io/blog/2016/02/15/developing-spring-boot-applications-with-kotlin>)
- [A Geospatial Messenger with Kotlin, Spring Boot and PostgreSQL](https://spring.io/blog/2016/03/20/a-geospatial-messenger-with-kotlin-spring-boot-and-postgresql) (<https://spring.io/blog/2016/03/20/a-geospatial-messenger-with-kotlin-spring-boot-and-postgresql>)
- [Introducing Kotlin support in Spring Framework 5.0](https://spring.io/blog/2017/01/04/introducing-kotlin-support-in-spring-framework-5-0) (<https://spring.io/blog/2017/01/04/introducing-kotlin-support-in-spring-framework-5-0>)
- [Spring Framework 5 Kotlin APIs, the functional way](https://spring.io/blog/2017/08/01/spring-framework-5-kotlin-apis-the-functional-way) (<https://spring.io/blog/2017/08/01/spring-framework-5-kotlin-apis-the-functional-way>)

32.7.2. Examples

- [spring-boot-kotlin-demo](https://github.com/sdeleuze/spring-boot-kotlin-demo) (<https://github.com/sdeleuze/spring-boot-kotlin-demo>): regular Spring Boot + Spring Data JPA project
- [mixit](https://github.com/mixitconf/mixit) (<https://github.com/mixitconf/mixit>): Spring Boot 2 + WebFlux + Reactive Spring Data MongoDB
- [spring-kotlin-fullstack](https://github.com/sdeleuze/spring-kotlin-fullstack) (<https://github.com/sdeleuze/spring-kotlin-fullstack>): WebFlux Kotlin fullstack example with Kotlin2js for frontend instead of JavaScript or TypeScript
- [spring-petclinic-kotlin](https://github.com/spring-petclinic/spring-petclinic-kotlin) (<https://github.com/spring-petclinic/spring-petclinic-kotlin>): Kotlin version of the Spring PetClinic Sample Application
- [spring-kotlin-deepdive](https://github.com/sdeleuze/spring-kotlin-deepdive) (<https://github.com/sdeleuze/spring-kotlin-deepdive>): a step by step migration for Boot 1.0 + Java to Boot 2.0 + Kotlin
- [spring-boot-coroutines-demo](https://github.com/sdeleuze/spring-boot-coroutines-demo) (<https://github.com/sdeleuze/spring-boot-coroutines-demo>): Coroutines sample project

33. Logging

Spring Boot uses **Commons Logging** (<https://commons.apache.org/logging/>) for all internal logging but leaves the underlying log implementation open. Default configurations are provided for {apiref-openjdk}/java.logging/java/util/logging/package-summary.html[Java Util Logging], **Log4j2** (<https://logging.apache.org/log4j/2.x/>), and **Logback** (<https://logback.qos.ch/>). In each case, loggers are pre-configured to use console output with optional file output also available.

By default, if you use the starters, Logback is used for logging. Appropriate Logback routing is also included to ensure that dependent libraries that use Java Util Logging, Commons Logging, Log4j, or SLF4J all work correctly.



There are a lot of logging frameworks available for Java. Do not worry if the above list seems confusing. Generally, you do not need to change your logging dependencies and the Spring Boot defaults work just fine.



When you deploy your application to a servlet container or application server, logging performed with the Java Util Logging API is not routed into your application's logs. This prevents logging performed by the container or other applications that have been deployed to it from appearing in your application's logs.

33.1. Log Format

The default log output from Spring Boot resembles the following example:

```
link:ROOT:partial$logging/logging-format.txt[role=include]
```

The following items are output:

- Date and Time: Millisecond precision and easily sortable.
- Log Level: `ERROR`, `WARN`, `INFO`, `DEBUG`, or `TRACE`.
- Process ID.
- A `---` separator to distinguish the start of actual log messages.
- Application name: Enclosed in square brackets (logged by default only if `configprop:spring.application.name[]` is set)
- Application group: Enclosed in square brackets (logged by default only if `configprop:spring.application.group[]` is set)
- Thread name: Enclosed in square brackets (may be truncated for console output).
- Correlation ID: If tracing is enabled (not shown in the sample above)
- Logger name: This is usually the source class name (often abbreviated).
- The log message.



Logback does not have a `FATAL` level. It is mapped to `ERROR`.



If you have a `configprop:spring.application.name[]` property but don't want it logged you can set `configprop:logging.include-application-name[]` to `false`.



If you have a `configprop:spring.application.group[]` property but don't want it logged you can set `configprop:logging.include-application-group[]` to `false`.



For more details about correlation IDs, please [see this documentation](#).

33.2. Console Output

The default log configuration echoes messages to the console as they are written. By default, `ERROR`-level, `WARN`-level, and `INFO`-level messages are logged. You can also enable a “debug” mode by starting your application with a `--debug` flag.

```
$ java -jar myapp.jar --debug
```

SHELL



You can also specify `debug=true` in your `application.properties`.

When the debug mode is enabled, a selection of core loggers (embedded container, Hibernate, and Spring Boot) are configured to output more information. Enabling the debug mode does *not* configure your application to log all messages with `DEBUG` level.

Alternatively, you can enable a “trace” mode by starting your application with a `--trace` flag (or `trace=true` in your `application.properties`). Doing so enables trace logging for a selection of core loggers (embedded container, Hibernate schema generation, and the whole Spring portfolio).

If you want to disable console-based logging, you can set the `configprop:logging.console.enabled[]` property to `false`.

33.2.1. Color-coded Output

If your terminal supports ANSI, color output is used to aid readability. You can set `spring.output.ansi.enabled` to a `java:org.springframework.boot.ansi.AnsiOutput$Enabled[supported value]` to override the auto-detection.

Color coding is configured by using the `%clr` conversion word. In its simplest form, the converter colors the output according to the log level, as shown in the following example:

```
%clr(%5p)
```

The following table describes the mapping of log levels to colors:

Level	Color
FATAL	Red
ERROR	Red
WARN	Yellow
INFO	Green
DEBUG	Green
TRACE	Green

Alternatively, you can specify the color or style that should be used by providing it as an option to the conversion. For example, to make the text yellow, use the following setting:

```
%clr(%d{yyyy-MM-dd'T'HH:mm:ss.SSSXXX}){yellow}
```

The following colors and styles are supported:

- blue
- cyan
- faint
- green
- magenta
- red
- yellow

33.3. File Output

By default, Spring Boot logs only to the console and does not write log files. If you want to write log files in addition to the console output, you need to set a `configprop:logging.file.name[]` or `configprop:logging.file.path[]` property (for example, in your `application.properties`). If both properties are set, `logging.file.path` is ignored and only `logging.file.name` is used.

The following table shows how the `logging.*` properties can be used together:

Table 5. Logging properties

configprop:logging.file.name[]	configprop:logging.file.path[]	Description
(none)	(none)	Console only logging.
Specific file (for example, <code>my.log</code>)	(none)	Writes to the location specified by <code>logging.file.name</code> . The location can be absolute or relative to the current directory.
(none)	Specific directory (for example, <code>/var/log</code>)	Writes <code>spring.log</code> to the directory specified by <code>logging.file.path</code> . The directory can be absolute or relative to the current directory.
Specific file	Specific directory	Writes to the location specified by <code>logging.file.name</code> and ignores <code>logging.file.path</code> . The location can be absolute or relative to the current directory.

Log files rotate when they reach 10 MB and, as with console output, `ERROR`-level, `WARN`-level, and `INFO`-level messages are logged by default. Note that Log4j2 requires `logging.file.path` to be set with such configuration.



Logging properties are independent of the actual logging infrastructure. As a result, specific configuration keys (such as `logback.configurationFile` for Logback) are not managed by Spring Boot.

33.4. File Rotation

If you are using the Logback, it is possible to fine-tune log rotation settings using your `application.properties` or `application.yaml` file. For all other logging system, you will need to configure rotation settings directly yourself (for example, if you use Log4j2 then you could add a `log4j2.xml` or `log4j2-spring.xml` file).

The following rotation policy properties are supported:

Name	Description
configprop:logging.logback.rollingpolicy.file-name-pattern[]	The filename pattern used to create log archives.
configprop:logging.logback.rollingpolicy.clean-history-on-start[]	If log archive cleanup should occur when the application starts.
configprop:logging.logback.rollingpolicy.max-file-size[]	The maximum size of log file before it is archived.
configprop:logging.logback.rollingpolicy.total-size-cap[]	The maximum amount of size log archives can take before being deleted.
configprop:logging.logback.rollingpolicy.max-history[]	The maximum number of archive log files to keep (defaults to 7).

33.5. Log Levels

All the supported logging systems can have the logger levels set in the Spring javadoc:org.springframework.core.env.Environment[] (for example, in application.properties) by using logging.level.<logger-name>=<level> where level is one of TRACE, DEBUG, INFO, WARN, ERROR, FATAL, or OFF. The root logger can be configured by using logging.level.root .

The following example shows potential logging settings in application.properties :

```
logging:
  level:
    root: "warn"
    org.springframework.web: "debug"
    org.hibernate: "error"
```

It is also possible to set logging levels using environment variables. For example, LOGGING_LEVEL_ORG_SPRINGFRAMEWORK_WEB=DEBUG will set org.springframework.web to DEBUG .



The above approach will only work for package level logging. Since relaxed binding [always converts environment variables to lowercase](#), it is not possible to configure logging for an individual class in this way. If you need to configure logging for a class, you can use [the SPRING_APPLICATION_JSON](#) variable.

33.6. Log Groups

It is often useful to be able to group related loggers together so that they can all be configured at the same time. For example, you might commonly change the logging levels for *all* Tomcat related loggers, but you can not easily remember top level packages.

To help with this, Spring Boot allows you to define logging groups in your Spring javadoc:org.springframework.core.env.Environment[]. For example, here is how you could define a “tomcat” group by adding it to your application.properties :

```
logging:
  group:
    tomcat: "org.apache.catalina,org.apache.coyote,org.apache.tomcat"
```

Once defined, you can change the level for all the loggers in the group with a single line:

```
logging:
  level:
    tomcat: "trace"
```

Spring Boot includes the following pre-defined logging groups that can be used out-of-the-box:

Name	Loggers
web	org.springframework.core.codec , org.springframework.http , org.springframework.web , org.springframework.boot.actuate.endpoint.web , org.springframework.boot.web.servlet.ServletContextInitializerBeans
sql	org.springframework.jdbc.core , org.hibernate.SQL , javadoc:org.jooq.tools.LoggerListener[]

33.7. Using a Log Shutdown Hook

In order to release logging resources when your application terminates, a shutdown hook that will trigger log system cleanup when the JVM exits is provided. This shutdown hook is registered automatically unless your application is deployed as a war file. If your application has complex context hierarchies the shutdown hook may not meet your needs. If it does not, disable the shutdown hook and investigate the options provided directly by the underlying logging system. For example, Logback offers [context selectors](#) (https://logback.qos.ch/manual/loggingSeparation.html) which allow each Logger to be created in its own context. You can use the configprop:logging.register-shutdown-hook[] property to disable the shutdown hook. Setting it to false will disable the registration. You can set the property in your application.properties or application.yaml file:

```
logging:
  register-shutdown-hook: false
```


33.8. Custom Log Configuration

The various logging systems can be activated by including the appropriate libraries on the classpath and can be further customized by providing a suitable configuration file in the root of the classpath or in a location specified by the following Spring javadoc:org.springframework.core.env.Environment[] property: configprop:logging.config[].

You can force Spring Boot to use a particular logging system by using the org.springframework.boot.logging.LoggingSystem system property. The value should be the fully qualified class name of a javadoc:org.springframework.boot.logging.LoggingSystem[] implementation. You can also disable Spring Boot's logging configuration entirely by using a value of none .



Since logging is initialized **before** the javadoc:org.springframework.context.ApplicationContext[] is created, it is not possible to control logging from javadoc:org.springframework.context.annotation.PropertySources[format=annotation] in Spring javadoc:org.springframework.context.annotation.Configuration[format=annotation] files. The only way to change the logging system or disable it entirely is through System properties.

Depending on your logging system, the following files are loaded:

Logging System	Customization
Logback	logback-spring.xml , logback-spring.groovy , logback.xml ,or logback.groovy
Log4j2	log4j2-spring.xml or log4j2.xml
JDK (Java Util Logging)	logging.properties



When possible, we recommend that you use the -spring variants for your logging configuration (for example, logback-spring.xml rather than logback.xml). If you use standard configuration locations, Spring cannot completely control log initialization.



There are known classloading issues with Java Util Logging that cause problems when running from an 'executable jar'. We recommend that you avoid it when running from an 'executable jar' if at all possible.

To help with the customization, some other properties are transferred from the Spring javadoc:org.springframework.core.env.Environment[] to System properties. This allows the properties to be consumed by logging system configuration. For example, setting logging.file.name in application.properties or LOGGING_FILE_NAME as an environment variable will result in the LOG_FILE System property being set. The properties that are transferred are described in the following table:

Spring Environment	System Property	Comments
configprop:logging.exception-conversion-word[]	LOG_EXCEPTION_CONVERSION_WORD	The conversion word used when logging exceptions.
configprop:logging.file.name[]	LOG_FILE	If defined, it is used in the default log configuration.
configprop:logging.file.path[]	LOG_PATH	If defined, it is used in the default log configuration.
configprop:logging.pattern.console[]	CONSOLE_LOG_PATTERN	The log pattern to use on the console (stdout).
configprop:logging.pattern.dateformat[]	LOG_DATEFORMAT_PATTERN	Appender pattern for log date format.
configprop:logging.charset.console[]	CONSOLE_LOG_CHARSET	The charset to use for console logging.
configprop:logging.threshold.console[]	CONSOLE_LOG_THRESHOLD	The log level threshold to use for console logging.
configprop:logging.pattern.file[]	FILE_LOG_PATTERN	The log pattern to use in a file (if LOG_FILE is enabled).
configprop:logging.charset.file[]	FILE_LOG_CHARSET	The charset to use for file logging (if LOG_FILE is enabled).
configprop:logging.threshold.file[]	FILE_LOG_THRESHOLD	The log level threshold to use for file logging.
configprop:logging.pattern.level[]	LOG_LEVEL_PATTERN	The format to use when rendering the log level (default %5p).
configprop:logging.structured.format.console[]	CONSOLE_LOG_STRUCTURED_FORMAT	The structured logging format to use for console logging.
configprop:logging.structured.format.file[]	FILE_LOG_STRUCTURED_FORMAT	The structured logging format to use for file logging.
PID	PID	The current process ID (discovered if possible and when not already defined as an OS environment variable).

If you use Logback, the following properties are also transferred:

Spring Environment	System Property	Comments
configprop:logging.logback.rollingpolicy.file-name-pattern[]	LOGBACK_ROLLINGPOLICY_FILE_NAME_PATTERN	Pattern for rolled-over log file names (default \${LOG_FILE}-%d{yyyy-MM-dd}-%i.gz).

Spring Environment	System Property	Comments
configprop:logging.logback.rollingpolicy.clean-history-on-start[]	LOGBACK_ROLLINGPOLICY_CLEAN_HISTORY_ON_START	Whether to clean the archive log files on startup.
configprop:logging.logback.rollingpolicy.max-file-size[]	LOGBACK_ROLLINGPOLICY_MAX_FILE_SIZE	Maximum log file size.
configprop:logging.logback.rollingpolicy.total-size-cap[]	LOGBACK_ROLLINGPOLICY_TOTAL_SIZE_CAP	Total size of log backups to be kept.
configprop:logging.logback.rollingpolicy.max-history[]	LOGBACK_ROLLINGPOLICY_MAX_HISTORY	Maximum number of archive log files to keep.

All the supported logging systems can consult System properties when parsing their configuration files. See the default configurations in `spring-boot.jar` for examples:

- `{code-spring-boot}/core/spring-boot/src/main/resources/org/springframework/boot/logging/logback/defaults.xml`[Logback]
- `{code-spring-boot}/core/spring-boot/src/main/resources/org/springframework/boot/logging/log4j2/log4j2.xml`[Log4j 2]
- `{code-spring-boot}/core/spring-boot/src/main/resources/org/springframework/boot/logging/java/logging-file.properties`[Java Util logging]



If you want to use a placeholder in a logging property, you should use [Spring Boot's syntax](#) and not the syntax of the underlying framework. Notably, if you use Logback, you should use `:` as the delimiter between a property name and its default value and not use `: - .`



You can add MDC and other ad-hoc content to log lines by overriding only the `LOG_LEVEL_PATTERN` (or `logging.pattern.level` with Logback). For example, if you use `logging.pattern.level=user:%X{user} %5p`, then the default log format contains an MDC entry for "user", if it exists, as shown in the following example.

```
2019-08-30 12:30:04.031 user:someone INFO 22174 --- [nio-8080-exec-0] demo.Controller
Handling authenticated request
```

33.9. Structured Logging

Structured logging is a technique where the log output is written in a well-defined, often machine-readable format. Spring Boot supports structured logging and has support for the following JSON formats out of the box:

- Elastic Common Schema (ECS)
- Graylog Extended Log Format (GELF)
- Logstash

To enable structured logging, set the property `configprop:logging.structured.format.console[]` (for console output) or `configprop:logging.structured.format.file[]` (for file output) to the id of the format you want to use.

If you are using Custom Log Configuration, update your configuration to respect `CONSOLE_LOG_STRUCTURED_FORMAT` and `FILE_LOG_STRUCTURED_FORMAT` system properties. Take `CONSOLE_LOG_STRUCTURED_FORMAT` for example:

Logback

```
<!-- replace your encoder with StructuredLogEncoder -->
<encoder class="org.springframework.boot.logging.logback.StructuredLogEncoder">
  <format>${CONSOLE_LOG_STRUCTURED_FORMAT}</format>
  <charset>${CONSOLE_LOG_CHARSET}</charset>
</encoder>
```

XML

You can also refer to the default configurations included in Spring Boot:

- `{code-spring-boot}/core/spring-boot/src/main/resources/org/springframework/boot/logging/logback/structured-console-appender.xml`[Logback Structured Console Appender]
- `{code-spring-boot}/core/spring-boot/src/main/resources/org/springframework/boot/logging/logback/structured-file-appender.xml`[Logback Structured File Appender]

Log4j2

```
<!-- replace your PatternLayout with StructuredLogLayout -->
<StructuredLogLayout format="${sys:CONSOLE_LOG_STRUCTURED_FORMAT}" charset="${sys:CONSOLE_LOG_CHARSET}"/>
```

XML

You can also refer to the default configurations included in Spring Boot:

- `{code-spring-boot}/core/spring-boot/src/main/resources/org/springframework/boot/logging/log4j2/log4j2.xml`[Log4j2 Console Appender]
- `{code-spring-boot}/core/spring-boot/src/main/resources/org/springframework/boot/logging/log4j2/log4j2-file.xml`[Log4j2 Console and File Appender]

33.9.1. Elastic Common Schema

[Elastic Common Schema](https://www.elastic.co/guide/en/ecs/8.11/ecs-reference.html) (https://www.elastic.co/guide/en/ecs/8.11/ecs-reference.html) is a JSON based logging format.

To enable the Elastic Common Schema log format, set the appropriate `format` property to `ecs` :

```
logging:
  structured:
    format:
      console: ecs
      file: ecs
```

A log line looks like this:

```
{ "@timestamp": "2024-01-01T10:15:00.067462556Z", "log": { "level": "INFO", "logger": "org.example.Application", "process": { "pid": 39599, "thread": { "name": "main" } }, "service": { "name": "simple" }, "message": "No active profile set, falling back to 1 default profile: \\"default\\""}, "ecs": { "version": "8.11" } }
```

JSON

This format also adds every key value pair contained in the MDC to the JSON object. You can also use the [SLF4J fluent logging API](https://www.slf4j.org/manual.html#fluent) (<https://www.slf4j.org/manual.html#fluent>) to add key value pairs to the logged JSON object with the [addKeyValue](https://www.slf4j.org/apidocs/org/slf4j/spi/LoggingEventBuilder.html#addKeyValue(java.lang.String,java.lang.Object)) ([https://www.slf4j.org/apidocs/org/slf4j/spi/LoggingEventBuilder.html#addKeyValue\(java.lang.String,java.lang.Object\)](https://www.slf4j.org/apidocs/org/slf4j/spi/LoggingEventBuilder.html#addKeyValue(java.lang.String,java.lang.Object))) method.

The `service` values can be customized using `logging.structured.ecs.service` properties:

```
logging:
  structured:
    ecs:
      service:
        name: MyService
        version: 1.0
        environment: Production
        node-name: Primary
```



`configprop:logging.structured.ecs.service.name[]` will default to `configprop:spring.application.name[]` if not specified.



`configprop:logging.structured.ecs.service.version[]` will default to `configprop:spring.application.version[]` if not specified.

33.9.2. Graylog Extended Log Format (GELF)

[Graylog Extended Log Format](https://go2docs.graylog.org/current/getting_in_log_data/gelf.html) (https://go2docs.graylog.org/current/getting_in_log_data/gelf.html) is a JSON based logging format for the Graylog log analytics platform.

To enable the Graylog Extended Log Format, set the appropriate `format` property to `gelf`:

```
logging:
  structured:
    format:
      console: gelf
      file: gelf
```

A log line looks like this:

```
{ "version": "1.1", "short_message": "No active profile set, falling back to 1 default profile: \\"default\\""}, "timestamp": 1725958035.857, "level": 6, "_level_name": "INFO", "_process_pid": 47649, "_process_thread_name": "main", "_log_logger": "org.example.Application" }
```

JSON

This format also adds every key value pair contained in the MDC to the JSON object. You can also use the [SLF4J fluent logging API](https://www.slf4j.org/manual.html#fluent) (<https://www.slf4j.org/manual.html#fluent>) to add key value pairs to the logged JSON object with the [addKeyValue](https://www.slf4j.org/apidocs/org/slf4j/spi/LoggingEventBuilder.html#addKeyValue(java.lang.String,java.lang.Object)) ([https://www.slf4j.org/apidocs/org/slf4j/spi/LoggingEventBuilder.html#addKeyValue\(java.lang.String,java.lang.Object\)](https://www.slf4j.org/apidocs/org/slf4j/spi/LoggingEventBuilder.html#addKeyValue(java.lang.String,java.lang.Object))) method.

Several fields can be customized using `logging.structured.gelf` properties:

```
logging:
  structured:
    gelf:
      host: MyService
      service:
        version: 1.0
```



`configprop:logging.structured.gelf.host[]` will default to `configprop:spring.application.name[]` if not specified.



`configprop:logging.structured.gelf.service.version[]` will default to `configprop:spring.application.version[]` if not specified.

33.9.3. Logstash JSON format

The [Logstash JSON format](https://github.com/logfellow/logstash-logback-encoder?tab=readme-ov-file#standard-fields) (<https://github.com/logfellow/logstash-logback-encoder?tab=readme-ov-file#standard-fields>) is a JSON based logging format.

To enable the Logstash JSON log format, set the appropriate `format` property to `logstash`:

```
logging:
  structured:
    format:
      console: logstash
      file: logstash
```

A log line looks like this:

```
{"@timestamp":"2024-01-01T10:15:00.111037681+02:00","@version":"1","message":"No active profile set, falling back to 1 default profile:
\default\"","logger_name":"org.example.Application","thread_name":"main","level":"INFO","level_value":20000}
```

JSON

This format also adds every key value pair contained in the MDC to the JSON object. You can also use the [SLF4J fluent logging API](https://www.slf4j.org/manual.html#fluent) (<https://www.slf4j.org/manual.html#fluent>) to add key value pairs to the logged JSON object with the [addKeyValue](https://www.slf4j.org/apidocs/org/slf4j/spi/LoggingEventBuilder.html#addKeyValue(java.lang.String,java.lang.Object)) ([https://www.slf4j.org/apidocs/org/slf4j/spi/LoggingEventBuilder.html#addKeyValue\(java.lang.String,java.lang.Object\)](https://www.slf4j.org/apidocs/org/slf4j/spi/LoggingEventBuilder.html#addKeyValue(java.lang.String,java.lang.Object))) method.

If you add [markers](https://www.slf4j.org/api/org/slf4j/Marker.html) (<https://www.slf4j.org/api/org/slf4j/Marker.html>), these will show up in a `tags` string array in the JSON.

33.9.4. Customizing Structured Logging JSON

Spring Boot attempts to pick sensible defaults for the JSON names and values output for structured logging. Sometimes, however, you may want to make small adjustments to the JSON for your own needs. For example, it's possible that you might want to change some of the names to match the expectations of your log ingestion system. You might also want to filter out certain members since you don't find them useful.

The following properties allow you to change the way that structured logging JSON is written:

Property	Description
<code>configprop:logging.structured.json.include[]</code> & <code>configprop:logging.structured.json.exclude[]</code>	Filters specific paths from the JSON
<code>configprop:logging.structured.json.rename[]</code>	Renames a specific member in the JSON
<code>configprop:logging.structured.json.add[]</code>	Adds additional members to the JSON

For example, the following will exclude `log.level`, rename `process.id` to `procid` and add a fixed `corpname` field:

```
logging:
  structured:
    json:
      exclude: log.level
      rename:
        process.id: procid
      add:
        corpname: mycorp
```



For more advanced customizations, you can use the `org.springframework.boot.logging.structured.StructuredLoggingJsonMembersCustomizer[]` interface. You can reference one or more implementations using the `configprop:logging.structured.json.customizer[]` property. You can also declare implementations by listing them in a `META-INF/spring.factories` file.

33.9.5. Customizing Structured Logging Stack Traces

Complete stack traces are included in the JSON output whenever a message is logged with an exception. This amount of information may be costly to process by your log ingestion system, so you may want to tune the way that stack traces are printed.

To do this, you can use one or more of the following properties:

Property	Description
<code>configprop:logging.structured.json.stacktrace.root[]</code>	Use <code>last</code> to print the root item last (same as Java) or <code>first</code> to print the root item first.
<code>configprop:logging.structured.json.stacktrace.max-length[]</code>	The maximum length that should be printed
<code>configprop:logging.structured.json.stacktrace.max-throwable-depth[]</code>	The maximum number of frames to print per stack trace (including common and suppressed frames)
<code>configprop:logging.structured.json.stacktrace.include-common-frames[]</code>	If common frames should be included or removed
<code>configprop:logging.structured.json.stacktrace.include-hashes[]</code>	If a hash of the stack trace should be included

For example, the following will use root first stack traces, limit their length, and include hashes.

```
logging:
  structured:
    json:
      stacktrace:
        root: first
        max-length: 1024
        include-common-frames: true
        include-hashes: true
```



If you need complete control over stack trace printing you can set `configprop:logging.structured.json.stacktrace.printer[]` to the name of a `javadoc:org.springframework.boot.logging.StackTracePrinter[]` implementation. You can also set it to `logging-system` to force regular logging system stack trace output to be used.

Your `StackTracePrinter` implementation can also include a constructor argument that accepts a `javadoc:org.springframework.boot.logging.StandardStackTracePrinter[]` if it wishes to apply further customization to the stack trace printer created from the properties.

33.9.6. Supporting Other Structured Logging Formats

The structured logging support in Spring Boot is extensible, allowing you to define your own custom format. To do this, implement the `javadoc:org.springframework.boot.logging.structured.StructuredLogFormatter[]` interface. The generic type argument has to be `javadoc:ch.qos.logback.classic.spi.ILoggingEvent[]` when using Logback and `javadoc:org.apache.logging.log4j.core.LogEvent[]` when using Log4j2 (that means your implementation is tied to a specific logging system). Your implementation is then called with the log event and returns the `javadoc:java.lang.String[]` to be logged, as seen in this example:

```
include-code::MyCustomFormat[]
```

As you can see in the example, you can return any format, it doesn't have to be JSON.

To enable your custom format, set the property `configprop:logging.structured.format.console[]` or `configprop:logging.structured.format.file[]` to the fully qualified class name of your implementation.

Your implementation can use some constructor parameters, which are injected automatically. Please see the JavaDoc of `javadoc:org.springframework.boot.logging.structured.StructuredLogFormatter[]` for more details.

33.10. Logback Extensions

Spring Boot includes a number of extensions to Logback that can help with advanced configuration. You can use these extensions in your `logback-spring.xml` configuration file.



Because the standard `logback.xml` configuration file is loaded too early, you cannot use extensions in it. You need to either use `logback-spring.xml` or define a `configprop:logging.config[]` property.



The extensions cannot be used with Logback's [configuration scanning](https://logback.qos.ch/manual/configuration.html#autoScan) (<https://logback.qos.ch/manual/configuration.html#autoScan>). If you attempt to do so, making changes to the configuration file results in an error similar to one of the following being logged:

```
ERROR in ch.qos.logback.core.joran.spi.Interpreter@4:71 - no applicable action for [springProperty], current ElementPath is [[configuration][springProperty]]
ERROR in ch.qos.logback.core.joran.spi.Interpreter@4:71 - no applicable action for [springProfile], current ElementPath is [[configuration][springProfile]]
```

33.10.1. Profile-specific Configuration

The `<springProfile>` tag lets you optionally include or exclude sections of configuration based on the active Spring profiles. Profile sections are supported anywhere within the `<configuration>` element. Use the `name` attribute to specify which profile accepts the configuration. The `<springProfile>` tag can contain a profile name (for example `staging`) or a profile expression. A profile expression allows for more complicated profile logic to be expressed, for example `production & (eu-central | eu-west)`. Check the [Spring Framework reference guide](https://docs.spring.io/spring-framework/docs/core/beans/environment.html#beans-definition-profiles-java) for more details. The following listing shows three sample profiles:

```
<springProfile name="staging">
  <!-- configuration to be enabled when the "staging" profile is active -->
</springProfile>

<springProfile name="dev | staging">
  <!-- configuration to be enabled when the "dev" or "staging" profiles are active -->
</springProfile>

<springProfile name="!production">
  <!-- configuration to be enabled when the "production" profile is not active -->
</springProfile>
```

XML

33.10.2. Environment Properties

The `<springProperty>` tag lets you expose properties from the Spring `javadoc:org.springframework.core.env.Environment[]` for use within Logback. Doing so can be useful if you want to access values from your `application.properties` file in your Logback configuration. The tag works in a similar way to Logback's standard `<property>` tag. However, rather than specifying a direct `value`, you specify the `source` of the property (from the `javadoc:org.springframework.core.env.Environment[]`). If you need to store the property somewhere other than in `local` scope, you can use the `scope` attribute. If you need a fallback value (in case the property is not set in the `javadoc:org.springframework.core.env.Environment[]`), you can use the `defaultValue` attribute. The following example shows how to expose properties for use within Logback:

```
<springProperty scope="context" name="fluentHost" source="myapp.fluentd.host"
  defaultValue="localhost"/>
<appender name="FLUENT" class="ch.qos.logback.more.appenders.DataFluentAppender">
  <remoteHost>${fluentHost}</remoteHost>
  ...
</appender>
```



The `source` must be specified in kebab case (such as `my.property-name`). However, properties can be added to the `javadoc:org.springframework.core.env.Environment[]` by using the relaxed rules.

33.11. Log4j2 Extensions

Spring Boot includes a number of extensions to Log4j2 that can help with advanced configuration. You can use these extensions in any `log4j2-spring.xml` configuration file.



Because the standard `log4j2.xml` configuration file is loaded too early, you cannot use extensions in it. You need to either use `log4j2-spring.xml` or define a `configprop:logging.config[]` property.



The extensions supersede the [Spring Boot support](https://logging.apache.org/log4j/2.x/log4j-spring-boot.html) (https://logging.apache.org/log4j/2.x/log4j-spring-boot.html) provided by Log4J. You should make sure not to include the `org.apache.logging.log4j:log4j-spring-boot` module in your build.

33.11.1. Profile-specific Configuration

The `<SpringProfile>` tag lets you optionally include or exclude sections of configuration based on the active Spring profiles. Profile sections are supported anywhere within the `<Configuration>` element. Use the `name` attribute to specify which profile accepts the configuration. The `<SpringProfile>` tag can contain a profile name (for example `staging`) or a profile expression. A profile expression allows for more complicated profile logic to be expressed, for example `production & (eu-central | eu-west)`. Check the [framework-docs/core/beans/environment.html#beans-definition-profiles-java](https://framework-docs.org/core/beans/environment.html#beans-definition-profiles-java) [Spring Framework reference guide] for more details. The following listing shows three sample profiles:

```
<SpringProfile name="staging">
  <!-- configuration to be enabled when the "staging" profile is active -->
</SpringProfile>

<SpringProfile name="dev | staging">
  <!-- configuration to be enabled when the "dev" or "staging" profiles are active -->
</SpringProfile>

<SpringProfile name="!production">
  <!-- configuration to be enabled when the "production" profile is not active -->
</SpringProfile>
```

33.11.2. Environment Properties Lookup

If you want to refer to properties from your Spring `javadoc:org.springframework.core.env.Environment[]` within your Log4j2 configuration you can use `spring:` prefixed [lookups](https://logging.apache.org/log4j/2.x/manual/lookups.html) (https://logging.apache.org/log4j/2.x/manual/lookups.html). Doing so can be useful if you want to access values from your `application.properties` file in your Log4j2 configuration.

The following example shows how to set Log4j2 properties named `applicationName` and `applicationGroup` that read `spring.application.name` and `spring.application.group` from the Spring `javadoc:org.springframework.core.env.Environment[]`:

```
<Properties>
  <Property name="applicationName">${spring:spring.application.name}</Property>
  <Property name="applicationGroup">${spring:spring.application.group}</Property>
</Properties>
```



The lookup key should be specified in kebab case (such as `my.property-name`).

33.11.3. Log4j2 System Properties

Log4j2 supports a number of [System Properties](https://logging.apache.org/log4j/2.x/manual/systemproperties.html) (https://logging.apache.org/log4j/2.x/manual/systemproperties.html) that can be used to configure various items. For example, the `log4j2.skipJansi` system property can be used to configure if the `javadoc:org.apache.logging.log4j.core.appender.ConsoleAppender[]` will try to use a [Jansi](https://github.com/fusesource/jansi) (https://github.com/fusesource/jansi) output stream on Windows.

All system properties that are loaded after the Log4j2 initialization can be obtained from the Spring `javadoc:org.springframework.core.env.Environment[]`. For example, you could add `log4j2.skipJansi=false` to your `application.properties` file to have the `javadoc:org.apache.logging.log4j.core.appender.ConsoleAppender[]` use Jansi on Windows.



The Spring `javadoc:org.springframework.core.env.Environment[]` is only considered when system properties and OS environment variables do not contain the value being loaded.



System properties that are loaded during early Log4j2 initialization cannot reference the Spring `javadoc:org.springframework.core.env.Environment[]`. For example, the property Log4j2 uses to allow the default Log4j2 implementation to be chosen is used before the Spring Environment is available.

34. Profiles

Spring Profiles provide a way to segregate parts of your application configuration and make it be available only in certain environments. Any `javax.annotation.Component`, `javax.annotation.Configuration` or `javax.annotation.ConfigurationProperties` can be marked with `javax.annotation.Profile` to limit when it is loaded, as shown in the following example:

```
include-code::ProductionConfiguration[]
```



If `javax.annotation.ConfigurationProperties` beans are registered through `javax.annotation.EnableConfigurationProperties` instead of automatic scanning, the `javax.annotation.Profile` annotation needs to be specified on the `javax.annotation.Configuration` class that has the `javax.annotation.EnableConfigurationProperties` annotation. In the case where `javax.annotation.ConfigurationProperties` are scanned, `javax.annotation.Profile` can be specified on the `javax.annotation.ConfigurationProperties` class itself.

You can use a `spring.profiles.active` `javax.annotation.Environment` property to specify which profiles are active. You can specify the property in any of the ways described earlier in this chapter. For example, you could include it in your `application.properties`, as shown in the following example:

```
spring:
  profiles:
    active: "dev,hsqldb"
```

You could also specify it on the command line by using the following switch: `--spring.profiles.active=dev,hsqldb`.

If no profile is active, a default profile is enabled. The name of the default profile is `default` and it can be tuned using the `spring.profiles.default` `javax.annotation.Environment` property, as shown in the following example:

```
spring:
  profiles:
    default: "none"
```

`spring.profiles.active` and `spring.profiles.default` can only be used in non-profile-specific documents. This means they cannot be included in [profile specific files](#) or [documents activated](#) by `spring.config.activate.on-profile`.

For example, the second document configuration is invalid:

```
# this document is valid
spring:
  profiles:
    active: "prod"
---
# this document is invalid
spring:
  config:
    activate:
      on-profile: "prod"
  profiles:
    active: "metrics"
```

The `spring.profiles.active` property follows the same ordering rules as other properties. The highest `javax.annotation.PropertySource` wins. This means that you can specify active profiles in `application.properties` and then **replace** them by using the command line switch.



See [the “Externalized Configuration”](#) for more details on the order in which property sources are considered.



By default, profile names in Spring Boot may contain letters, numbers, or permitted characters (`-`, `_`, `.`, `+`, `@`). In addition, they can only start and end with a letter or number.

This restriction helps to prevent common parsing issues. If, however, you prefer more flexible profile names you can set `spring.profiles.validate` to `false` in your `application.properties` or `application.yaml` file:

```
spring:
  profiles:
    validate: false
```


34.1. Adding Active Profiles

Sometimes, it is useful to have properties that **add** to the active profiles rather than replace them. The `configprop:spring.profiles.include[]` property can be used to add active profiles on top of those activated by the `configprop:spring.profiles.active[]` property. The `javadoc:org.springframework.boot.SpringApplication[]` entry point also has a Java API for setting additional profiles. See the `setAdditionalProfiles()` method in `javadoc:org.springframework.boot.SpringApplication[]`.

For example, when an application with the following properties is run, the common and local profiles will be activated even when it runs using the `--spring.profiles.active` switch:

```
spring:
  profiles:
    include:
      - "common"
      - "local"
```



Included profiles are added before any `configprop:spring.profiles.active[]` profiles.



The `configprop:spring.profiles.include[]` property is processed for each property source, as such the usual [complex type merging rules](#) for lists do not apply.



Similar to `spring.profiles.active`, `spring.profiles.include` can only be used in non-profile-specific documents. This means it cannot be included in [profile specific files](#) or [documents activated](#) by `spring.config.activate.on-profile`.

Profile groups, which are described in the [next section](#) can also be used to add active profiles if a given profile is active.

34.2. Profile Groups

Occasionally the profiles that you define and use in your application are too fine-grained and become cumbersome to use. For example, you might have `proddb` and `prodmq` profiles that you use to enable database and messaging features independently.

To help with this, Spring Boot lets you define profile groups. A profile group allows you to define a logical name for a related group of profiles.

For example, we can create a `production` group that consists of our `proddb` and `prodmq` profiles.

```
spring:
  profiles:
    group:
      production:
        - "proddb"
        - "prodmq"
```

Our application can now be started using `--spring.profiles.active=production` to activate the `production`, `proddb` and `prodmq` profiles in one hit.



Similar to `spring.profiles.active` and `spring.profiles.include`, `spring.profiles.group` can only be used in non-profile-specific documents. This means it cannot be included in [profile specific files](#) or [documents activated](#) by `spring.config.activate.on-profile`.

34.3. Programmatically Setting Profiles

You can programmatically set active profiles by calling `SpringApplication.setAdditionalProfiles(...)` before your application runs. It is also possible to activate profiles by using Spring's `javadoc:org.springframework.core.env.ConfigurableEnvironment[]` interface.

34.4. Profile-specific Configuration Files

Profile-specific variants of both `application.properties` (or `application.yaml`) and files referenced through `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` are considered as files and loaded. See [features/external-config.html](#) for details.

35. SpringApplication

The `org.springframework.boot.SpringApplication[]` class provides a convenient way to bootstrap a Spring application that is started from a `main()` method. In many situations, you can delegate to the static `org.springframework.boot.SpringApplication#run(java.lang.Class,java.lang.String...[])` method, as shown in the following example:

```
include-code::MyApplication[]
```

When your application starts, you should see something similar to the following output:

```
link:ROOT:partial$application/spring-application.txt[role=include]
```

By default, `INFO` logging messages are shown, including some relevant startup details, such as the user that launched the application. If you need a log level other than `INFO`, you can set it, as described in [features/logging.html](#). The application version is determined using the implementation version from the main application class's package. Startup information logging can be turned off by setting `spring.main.log-startup-info` to `false`. This will also turn off logging of the application's active profiles.



To add additional logging during startup, you can override `logStartupInfo(boolean)` in a subclass of `org.springframework.boot.SpringApplication[]`.

35.1. Startup Failure

If your application fails to start, registered `org.springframework.boot.diagnostics.FailureAnalyzer[]` beans get a chance to provide a dedicated error message and a concrete action to fix the problem. For instance, if you start a web application on port `8080` and that port is already in use, you should see something similar to the following message:

```
*****
APPLICATION FAILED TO START
*****
```

Description:

Embedded servlet container failed to start. Port 8080 was already in use.

Action:

Identify and stop the process that is listening on port 8080 or configure this application to listen on another port.



Spring Boot provides numerous `org.springframework.boot.diagnostics.FailureAnalyzer[]` implementations, and you can [add your own](#).

If no failure analyzers are able to handle the exception, you can still display the full conditions report to better understand what went wrong. To do so, you need to [enable the debug property](#) or [enable DEBUG logging](#) for `org.springframework.boot.autoconfigure.logging.ConditionEvaluationReportLoggingListener[]`.

For instance, if you are running your application by using `java -jar`, you can enable the `debug` property as follows:

```
$ java -jar myproject-0.0.1-SNAPSHOT.jar --debug
```

SHELL

35.2. Lazy Initialization

`org.springframework.boot.SpringApplication[]` allows an application to be initialized lazily. When lazy initialization is enabled, beans are created as they are needed rather than during application startup. As a result, enabling lazy initialization can reduce the time that it takes your application to start. In a web application, enabling lazy initialization will result in many web-related beans not being initialized until an HTTP request is received.

A downside of lazy initialization is that it can delay the discovery of a problem with the application. If a misconfigured bean is initialized lazily, a failure will no longer occur during startup and the problem will only become apparent when the bean is initialized. Care must also be taken to ensure that the JVM has sufficient memory to accommodate all of the application's beans and not just those that are initialized during startup. For these reasons, lazy initialization is not enabled by default and it is recommended that fine-tuning of the JVM's heap size is done before enabling lazy initialization.

Lazy initialization can be enabled programmatically using the `lazyInitialization` method on `org.springframework.boot.builder.SpringApplicationBuilder[]` or the `setLazyInitialization` method on `org.springframework.boot.SpringApplication[]`. Alternatively, it can be enabled using the `configprop:spring.main.lazy-initialization[]` property as shown in the following example:

```
spring:
  main:
    lazy-initialization: true
```



If you want to disable lazy initialization for certain beans while using lazy initialization for the rest of the application, you can explicitly set their `lazy` attribute to `false` using the `@Lazy(false)` annotation.

35.3. Customizing the Banner

The banner that is printed on start up can be changed by adding a `banner.txt` file to your classpath or by setting the `configprop:spring.banner.location[]` property to the location of such a file. If the file has an encoding other than UTF-8, you can set `spring.banner.charset`.

Inside your `banner.txt` file, you can use any key available in the `org.springframework.core.env.Environment[]` as well as any of the following placeholders:

Table 6. Banner variables

Variable	Description
<code>\${application.version}</code>	The version number of your application, as declared in <code>MANIFEST.MF</code> . For example, <code>Implementation-Version: 1.0</code> is printed as <code>1.0</code> .
<code>\${application.formatted-version}</code>	The version number of your application, as declared in <code>MANIFEST.MF</code> and formatted for display (surrounded with brackets and prefixed with <code>v</code>). For example <code>(v1.0)</code> .
<code>\${spring-boot.version}</code>	The Spring Boot version that you are using. For example <code>{version-spring-boot}</code> .
<code>\${spring-boot.formatted-version}</code>	The Spring Boot version that you are using, formatted for display (surrounded with brackets and prefixed with <code>v</code>). For example <code>(v{version-spring-boot})</code> .
<code>\${Ansi.NAME}</code> (or <code>\${AnsiColor.NAME}</code> , <code>\${AnsiBackground.NAME}</code> , <code>\${AnsiStyle.NAME}</code>)	Where <code>NAME</code> is the name of an ANSI escape code. See <code>javadoc:org.springframework.boot.ansi.AnsiPropertySource[]</code> for details.
<code>\${application.title}</code>	The title of your application, as declared in <code>MANIFEST.MF</code> . For example <code>Implementation-Title: MyApp</code> is printed as <code>MyApp</code> .



The `SpringApplication.setBanner(...)` method can be used if you want to generate a banner programmatically. Use the `javadoc:org.springframework.boot.Banner[]` interface and implement your own `printBanner()` method.

You can also use the `configprop:spring.main.banner-mode[]` property to determine if the banner has to be printed on `javadoc:java.lang.System#out[]` (`console`), sent to the configured logger (`log`), or not produced at all (`off`).

The printed banner is registered as a singleton bean under the following name: `springBootBanner`.



The `application.title`, `application.version`, and `application.formatted-version` properties are only available if you are using `java -jar` or `java -cp` with Spring Boot launchers. The values will not be resolved if you are running an unpacked jar and starting it with `java -cp <classpath> <mainclass>` or running your application as a native image.

To use the `application.*` properties, launch your application as a packed jar using `java -jar` or as an unpacked jar using `java org.springframework.boot.loader.launch.JarLauncher`. This will initialize the `application.*` banner properties before building the classpath and launching your app.

35.4. Customizing SpringApplication

If the `javadoc:org.springframework.boot.SpringApplication[]` defaults are not to your taste, you can instead create a local instance and customize it. For example, to turn off the banner, you could write:

include-code::MyApplication[]



The constructor arguments passed to `javadoc:org.springframework.boot.SpringApplication[]` are configuration sources for Spring beans. In most cases, these are references to `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` classes, but they could also be direct references `javadoc:org.springframework.stereotype.Component[format=annotation]` classes.

It is also possible to configure the `javadoc:org.springframework.boot.SpringApplication[]` by using an `application.properties` file. See [features/external-config.html](#) for details.

For a complete list of the configuration options, see the `javadoc:org.springframework.boot.SpringApplication[]` API documentation.

35.5. Fluent Builder API

If you need to build an `javadoc:org.springframework.context.ApplicationContext[]` hierarchy (multiple contexts with a parent/child relationship) or if you prefer using a fluent builder API, you can use the `javadoc:org.springframework.boot.builder.SpringApplicationBuilder[]`.

The `javadoc:org.springframework.boot.builder.SpringApplicationBuilder[]` lets you chain together multiple method calls and includes `parent` and `child` methods that let you create a hierarchy, as shown in the following example:

include-code::MyApplication[tag=*)



There are some restrictions when creating an `javadoc:org.springframework.context.ApplicationContext[]` hierarchy. For example, Web components **must** be contained within the child context, and the same `javadoc:org.springframework.core.env.Environment[]` is used for both parent and child contexts. See the `javadoc:org.springframework.boot.builder.SpringApplicationBuilder[]` API documentation for full details.

35.6. Application Availability

When deployed on platforms, applications can provide information about their availability to the platform using infrastructure such as [Kubernetes Probes](#) (<https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>). Spring Boot includes out-of-the box support for the commonly used “liveness” and “readiness” availability states. If you are using Spring Boot’s “actuator” support then these states are exposed as health endpoint groups.

In addition, you can also obtain availability states by injecting the `javadoc:org.springframework.boot.availability.ApplicationAvailability[]` interface into your own beans.

35.6.1. Liveness State

The “Liveness” state of an application tells whether its internal state allows it to work correctly, or recover by itself if it is currently failing. A broken “Liveness” state means that the application is in a state that it cannot recover from, and the infrastructure should restart the application.



In general, the “Liveness” state should not be based on external checks, such as [health checks](#). If it did, a failing external system (a database, a Web API, an external cache) would trigger massive restarts and cascading failures across the platform.

The internal state of Spring Boot applications is mostly represented by the `org.springframework.context.ApplicationContext[]`. If the application context has started successfully, Spring Boot assumes that the application is in a valid state. An application is considered live as soon as the context has been refreshed, see [Spring Boot application lifecycle and related Application Events](#).

35.6.2. Readiness State

The “Readiness” state of an application tells whether the application is ready to handle traffic. A failing “Readiness” state tells the platform that it should not route traffic to the application for now. This typically happens during startup, while `org.springframework.boot.CommandLineRunner[]` and `org.springframework.boot.ApplicationRunner[]` components are being processed, or at any time if the application decides that it is too busy for additional traffic.

An application is considered ready as soon as application and command-line runners have been called, see [Spring Boot application lifecycle and related Application Events](#).



Tasks expected to run during startup should be executed by `org.springframework.boot.CommandLineRunner[]` and `org.springframework.boot.ApplicationRunner[]` components instead of using Spring component lifecycle callbacks such as `org.springframework.annotation.PostConstruct`.

35.6.3. Managing the Application Availability State

Application components can retrieve the current availability state at any time, by injecting the `org.springframework.boot.availability.ApplicationAvailability[]` interface and calling methods on it. More often, applications will want to listen to state updates or update the state of the application.

For example, we can export the “Readiness” state of the application to a file so that a Kubernetes “exec Probe” can look at this file:

```
include-code::MyReadinessStateExporter[]
```

We can also update the state of the application, when the application breaks and cannot recover:

```
include-code::MyLocalCacheVerifier[]
```

Spring Boot provides [Kubernetes HTTP probes for “Liveness” and “Readiness” with Actuator Health Endpoints](#). You can get more guidance about [deploying Spring Boot applications on Kubernetes in the dedicated section](#).

35.7. Application Events and Listeners

In addition to the usual Spring Framework events, such as `org.springframework.context.event.ContextRefreshedEvent[]`, a `org.springframework.boot.SpringApplication[]` sends some additional application events.



Some events are actually triggered before the `org.springframework.context.ApplicationContext[]` is created, so you cannot register a listener on those as a `org.springframework.context.annotation.Bean`. You can register them with the `SpringApplication.addListeners(...)` method or the `SpringApplicationBuilder.listeners(...)` method.

If you want those listeners to be registered automatically, regardless of the way the application is created, you can add a `META-INF/spring.factories` file to your project and reference your listener(s) by using the `org.springframework.context.ApplicationListener[]` key, as shown in the following example:

```
org.springframework.context.ApplicationListener=com.example.project.MyListener
```

Application events are sent in the following order, as your application runs:

1. An `org.springframework.boot.context.event.ApplicationStartingEvent[]` is sent at the start of a run but before any processing, except for the registration of listeners and initializers.
2. An `org.springframework.boot.context.event.ApplicationEnvironmentPreparedEvent[]` is sent when the `org.springframework.core.env.Environment[]` to be used in the context is known but before the context is created.
3. An `org.springframework.boot.context.event.ApplicationContextInitializedEvent[]` is sent when the `org.springframework.context.ApplicationContext[]` is prepared and `ApplicationContextInitializers` have been called but before any bean definitions are loaded.
4. An `org.springframework.boot.context.event.ApplicationPreparedEvent[]` is sent just before the refresh is started but after bean definitions have been loaded.
5. An `org.springframework.boot.context.event.ApplicationStartedEvent[]` is sent after the context has been refreshed but before any application and command-line runners have been called.
6. An `org.springframework.boot.availability.AvailabilityChangeEvent[]` is sent right after with `org.springframework.boot.availability.LivenessState#CORRECT[]` to indicate that the application is considered as live.
7. An `org.springframework.boot.context.event.ApplicationReadyEvent[]` is sent after any [application and command-line runners](#) have been called.
8. An `org.springframework.boot.availability.AvailabilityChangeEvent[]` is sent right after with `org.springframework.boot.availability.ReadinessState#ACCEPTING_TRAFFIC[]` to indicate that the application is ready to service requests.

9. An `org.springframework.boot.context.event.ApplicationFailedEvent[]` is sent if there is an exception on startup.

The above list only includes `SpringApplicationEvent`s that are tied to a `org.springframework.boot.SpringApplication[]`. In addition to these, the following events are also published after `org.springframework.boot.context.event.ApplicationPreparedEvent[]` and before `org.springframework.boot.context.event.ApplicationStartedEvent[]`:

- A `org.springframework.boot.web.server.context.WebServerInitializedEvent[]` is sent after the `org.springframework.boot.web.server.WebServer[]` is ready. `org.springframework.boot.web.server.servlet.context.ServletWebServerInitializedEvent[]` and `org.springframework.boot.web.server.reactive.context.ReactiveWebServerInitializedEvent[]` are the servlet and reactive variants respectively.
- A `org.springframework.context.event.ContextRefreshedEvent[]` is sent when an `org.springframework.context.ApplicationContext[]` is refreshed.



You often need not use application events, but it can be handy to know that they exist. Internally, Spring Boot uses events to handle a variety of tasks.



Event listeners should not run potentially lengthy tasks as they execute in the same thread by default. Consider using [application and command-line runners](#) instead.

Application events are sent by using Spring Framework's event publishing mechanism. Part of this mechanism ensures that an event published to the listeners in a child context is also published to the listeners in any ancestor contexts. As a result of this, if your application uses a hierarchy of `org.springframework.boot.SpringApplication[]` instances, a listener may receive multiple instances of the same type of application event.

To allow your listener to distinguish between an event for its context and an event for a descendant context, it should request that its application context is injected and then compare the injected context with the context of the event. The context can be injected by implementing `org.springframework.context.ApplicationContextAware[]` or, if the listener is a bean, by using `org.springframework.beans.factory.annotation.Autowired[format=annotation]`.

35.8. Web Environment

A `org.springframework.boot.SpringApplication[]` attempts to create the right type of `org.springframework.context.ApplicationContext[]` on your behalf. The algorithm used to determine a `org.springframework.boot.WebApplicationType[]` is the following:

- If Spring MVC is present, an `org.springframework.boot.web.server.servlet.context.AnnotationConfigServletWebServerApplicationContext[]` is used
- If Spring MVC is not present and Spring WebFlux is present, an `org.springframework.boot.web.server.reactive.context.AnnotationConfigReactiveWebServerApplicationContext[]` is used
- Otherwise, `org.springframework.context.annotation.AnnotationConfigApplicationContext[]` is used

This means that if you are using Spring MVC and the new `org.springframework.web.reactive.function.client.WebClient[]` from Spring WebFlux in the same application, Spring MVC will be used by default. You can override that easily by calling `setWebApplicationType(WebApplicationType)`.

It is also possible to take complete control of the `org.springframework.context.ApplicationContext[]` type that is used by calling `setApplicationContextFactory(...)`.



It is often desirable to call `setWebApplicationType(WebApplicationType.NONE)` when using `org.springframework.boot.SpringApplication[]` within a JUnit test.

35.9. Accessing Application Arguments

If you need to access the application arguments that were passed to `SpringApplication.run(...)`, you can inject a `org.springframework.boot.ApplicationArguments[]` bean. The `org.springframework.boot.ApplicationArguments[]` interface provides access to both the raw `String[]` arguments as well as parsed `option` and `non-option` arguments, as shown in the following example:

```
include-code::MyBean[]
```



Spring Boot also registers a `org.springframework.core.env.CommandLinePropertySource[]` with the Spring `org.springframework.core.env.Environment[]`. This lets you also inject single application arguments by using the `org.springframework.beans.factory.annotation.Value[format=annotation]` annotation.

35.10. Using the ApplicationRunner or CommandLineRunner

If you need to run some specific code once the `org.springframework.boot.SpringApplication[]` has started, you can implement the `org.springframework.boot.ApplicationRunner[]` or `org.springframework.boot.CommandLineRunner[]` interfaces. Both interfaces work in the same way and offer a single `run` method, which is called just before `SpringApplication.run(...)` completes.



This contract is well suited for tasks that should run after application startup but before it starts accepting traffic.

The `org.springframework.boot.CommandLineRunner[]` interface provides access to application arguments as a string array, whereas the `org.springframework.boot.ApplicationRunner[]` uses the `org.springframework.boot.ApplicationArguments[]` interface discussed earlier. The following example shows a `org.springframework.boot.CommandLineRunner[]` with a `run` method:

```
include-code::MyCommandLineRunner[]
```

If several `org.springframework.boot.CommandLineRunner[]` or `org.springframework.boot.ApplicationRunner[]` beans are defined that must be called in a specific order, you can additionally implement the `org.springframework.core.Ordered[]` interface or use the `org.springframework.core.annotation.Order[]` annotation.

35.11. Application Exit

Each `javadoc:org.springframework.boot.SpringApplication[]` registers a shutdown hook with the JVM to ensure that the `javadoc:org.springframework.context.ApplicationContext[]` closes gracefully on exit. All the standard Spring lifecycle callbacks (such as the `javadoc:org.springframework.beans.factory.DisposableBean[]` interface or the `javadoc:jakarta.annotation.PreDestroy[format=annotation]` annotation) can be used.

In addition, beans may implement the `javadoc:org.springframework.boot.ExitCodeGenerator[]` interface if they wish to return a specific exit code when `SpringApplication.exit()` is called. This exit code can then be passed to `System.exit()` to return it as a status code, as shown in the following example:

```
include-code::MyApplication[]
```

Also, the `javadoc:org.springframework.boot.ExitCodeGenerator[]` interface may be implemented by exceptions. When such an exception is encountered, Spring Boot returns the exit code provided by the implemented `getExitCode()` method.

If there is more than one `javadoc:org.springframework.boot.ExitCodeGenerator[]`, the first non-zero exit code that is generated is used. To control the order in which the generators are called, additionally implement the `javadoc:org.springframework.core.Ordered[]` interface or use the `javadoc:org.springframework.core.annotation.Order[]` annotation.

35.12. Admin Features

It is possible to enable admin-related features for the application by specifying the `configprop:spring.application.admin.enabled[]` property. This exposes the `javadoc:org.springframework.boot.admin.SpringApplicationAdminMXBean[]` on the platform `javadoc:javax.management.MBeanServer[]`. You could use this feature to administer your Spring Boot application remotely. This feature could also be useful for any service wrapper implementation.



If you want to know on which HTTP port the application is running, get the property with a key of `local.server.port`.

35.13. Application Startup tracking

During the application startup, the `javadoc:org.springframework.boot.SpringApplication[]` and the `javadoc:org.springframework.context.ApplicationContext[]` perform many tasks related to the application lifecycle, the beans lifecycle or even processing application events. With `javadoc:org.springframework.core.metrics.ApplicationStartup[]`, Spring Framework ([url-spring-framework-docs/core/beans/context-introduction.html#context-functionality-startup](https://docs.spring.io/spring-framework/docs/core/beans/context-introduction.html#context-functionality-startup)) allows you to track the application startup sequence with `javadoc:org.springframework.core.metrics.StartupStep[]` objects. This data can be collected for profiling purposes, or just to have a better understanding of an application startup process.

You can choose an `javadoc:org.springframework.core.metrics.ApplicationStartup[]` implementation when setting up the `javadoc:org.springframework.boot.SpringApplication[]` instance. For example, to use the `javadoc:org.springframework.boot.context.metrics.buffering.BufferingApplicationStartup[]`, you could write:

```
include-code::MyApplication[]
```

The first available implementation, `javadoc:org.springframework.core.metrics.jfr.FlightRecorderApplicationStartup[]` is provided by Spring Framework. It adds Spring-specific startup events to a Java Flight Recorder session and is meant for profiling applications and correlating their Spring context lifecycle with JVM events (such as allocations, GCs, class loading...). Once configured, you can record data by running the application with the Flight Recorder enabled:

```
$ java -XX:StartFlightRecording:filename=recording.jfr,duration=10s -jar demo.jar
```

SHELL

Spring Boot ships with the `javadoc:org.springframework.boot.context.metrics.buffering.BufferingApplicationStartup[]` variant; this implementation is meant for buffering the startup steps and draining them into an external metrics system. Applications can ask for the bean of type `javadoc:org.springframework.boot.context.metrics.buffering.BufferingApplicationStartup[]` in any component.

Spring Boot can also be configured to expose a [startup endpoint](#) that provides this information as a JSON document.

35.14. Virtual threads

Virtual threads require Java 21 or later. For the best experience, Java 24 or later is strongly recommended. To enable virtual threads, set the `configprop:spring.threads.virtual.enabled[]` property to `true`.

Before turning on this option for your application, you should consider [reading the official Java virtual threads documentation](https://docs.oracle.com/en/java/javase/24/core/virtual-threads.html) (<https://docs.oracle.com/en/java/javase/24/core/virtual-threads.html>). In some cases, applications can experience lower throughput because of "Pinned Virtual Threads"; this page also explains how to detect such cases with JDK Flight Recorder or the `jcmd` CLI.



If virtual threads are enabled, properties which configure thread pools don't have an effect anymore. That's because virtual threads are scheduled on a JVM wide platform thread pool and not on dedicated thread pools.



One side effect of virtual threads is that they are daemon threads. A JVM will exit if all of its threads are daemon threads. This behavior can be a problem when you rely on `javadoc:org.springframework.scheduling.annotation.Scheduled[format=annotation]` beans, for example, to keep your application alive. If you use virtual threads, the scheduler thread is a virtual thread and therefore a daemon thread and won't keep the JVM alive. This not only affects scheduling and can be the case with other technologies too. To keep the JVM running in all cases, it is recommended to set the property `configprop:spring.main.keep-alive[]` to `true`. This ensures that the JVM is kept alive, even if all threads are virtual threads.

36. SSL

Spring Boot provides the ability to configure SSL trust material that can be applied to several types of connections in order to support secure communications. Configuration properties with the prefix `spring.ssl.bundle` can be used to specify named sets of trust material and associated information.

36.1. Configuring SSL With Java KeyStore Files

Configuration properties with the prefix `spring.ssl.bundle.jks` can be used to configure bundles of trust material created with the Java `keytool` utility and stored in Java KeyStore files in the JKS or PKCS12 format. Each bundle has a user-provided name that can be used to reference the bundle.

When used to secure an embedded web server, a `keystore` is typically configured with a Java KeyStore containing a certificate and private key as shown in this example:

```
spring:
  ssl:
    bundle:
      jks:
        mybundle:
          key:
            alias: "application"
          keystore:
            location: "classpath:application.p12"
            password: "secret"
            type: "PKCS12"
```

When used to secure a client-side connection, a `truststore` is typically configured with a Java KeyStore containing the server certificate as shown in this example:

```
spring:
  ssl:
    bundle:
      jks:
        mybundle:
          truststore:
            location: "classpath:server.p12"
            password: "secret"
```



Rather than the location to a file, its [Base64 encoded content](#) can be provided. If you chose this option, the value of the property should start with `base64:`.

See [javadoc.org.springframework.boot.autoconfigure.ssl.JksSslBundleProperties\[\]](https://docs.spring.io/spring-boot/docs/current/api/org.springframework.boot.autoconfigure.ssl.JksSslBundleProperties.html) for the full set of supported properties.



If you're using environment variables to configure the bundle, the name of the bundle is [always converted to lowercase](#).

36.2. Configuring SSL With PEM-encoded Certificates

Configuration properties with the prefix `spring.ssl.bundle.pem` can be used to configure bundles of trust material in the form of PEM-encoded text. Each bundle has a user-provided name that can be used to reference the bundle.

When used to secure an embedded web server, a `keystore` is typically configured with a certificate and private key as shown in this example:

```
spring:
  ssl:
    bundle:
      pem:
        mybundle:
          keystore:
            certificate: "classpath:application.crt"
            private-key: "classpath:application.key"
```

When used to secure a client-side connection, a `truststore` is typically configured with the server certificate as shown in this example:

```
spring:
  ssl:
    bundle:
      pem:
        mybundle:
          truststore:
            certificate: "classpath:server.crt"
```

Rather than the location to a file, its [Base64 encoded content](#) can be provided. If you chose this option, the value of the property should start with **base64:** .

PEM content can also be used directly for both the **certificate** and **private-key** properties. If the property values contain **BEGIN** and **END** markers then they will be treated as PEM content rather than a resource location.

The following example shows how a truststore certificate can be defined:



```
spring:
  ssl:
    bundle:
      pem:
        mybundle:
          truststore:
            certificate: |
              -----BEGIN CERTIFICATE-----
              MIID1zCCAr+gAwIBAgIUNMSQQv8IzVQsgSmmdPQNqyzWs4wDQYJKoZIhvcNAQEL
              BQAwesELMAkGA1UEBhMCWFGxEjAQBgNVBAgMCVN0YXR1TmFtZTERMA8GA1UEBwwI
              ...
              VOIJjcmYjEZbTvpjFKznvaFiOUv+8L7jHQ1/Yf+9c3C8gSjdUfv88m17pqYXd+Ds
              HEmfMNNjht130UyjNCITmLVXyy5p35vWmdf95U3uEbJSnNVtXH8qRmN9oK9mUpDb
              ngX6JBjI7fw7tXoqlSLHniBODM88fU1QSho8
              -----END CERTIFICATE-----
```

See `javadoc:org.springframework.boot.autoconfigure.ssl.PemSslBundleProperties[]` for the full set of supported properties.



If you're using environment variables to configure the bundle, the name of the bundle is [always converted to lowercase](#).

36.3. Applying SSL Bundles

Once configured using properties, SSL bundles can be referred to by name in configuration properties for various types of connections that are auto-configured by Spring Boot. See the sections on [embedded web servers](#), [data technologies](#), and [REST clients](#) for further information.

36.4. Using SSL Bundles

Spring Boot auto-configures a bean of type `javadoc:org.springframework.boot.ssl.SslBundles[]` that provides access to each of the named bundles configured using the `spring.ssl.bundle` properties.

An `javadoc:org.springframework.boot.ssl.SslBundle[]` can be retrieved from the auto-configured `javadoc:org.springframework.boot.ssl.SslBundles[]` bean and used to create objects that are used to configure SSL connectivity in client libraries. The `javadoc:org.springframework.boot.ssl.SslBundle[]` provides a layered approach of obtaining these SSL objects:

- `getStores()` provides access to the key store and trust store `javadoc:java.security.KeyStore[]` instances as well as any required key store password.
- `getManagers()` provides access to the `javadoc:javax.net.ssl.KeyManagerFactory[]` and `javadoc:javax.net.ssl.TrustManagerFactory[]` instances as well as the `javadoc:javax.net.ssl.KeyManager[]` and `javadoc:javax.net.ssl.TrustManager[]` arrays that they create.
- `createSslContext()` provides a convenient way to obtain a new `javadoc:javax.net.ssl.SSLContext[]` instance.

In addition, the `javadoc:org.springframework.boot.ssl.SslBundle[]` provides details about the key being used, the protocol to use and any option that should be applied to the SSL engine.

The following example shows retrieving an `javadoc:org.springframework.boot.ssl.SslBundle[]` and using it to create an `javadoc:javax.net.ssl.SSLContext[]`:

```
include-code::MyComponent[]
```

36.5. Reloading SSL bundles

SSL bundles can be reloaded when the key material changes. The component consuming the bundle has to be compatible with reloadable SSL bundles. Currently the following components are compatible:

- Tomcat web server
- Netty web server

To enable reloading, you need to opt-in via a configuration property as shown in this example:

```
spring:
  ssl:
    bundle:
      pem:
        mybundle:
          reload-on-update: true
          keystore:
            certificate: "file:/some/directory/application.crt"
            private-key: "file:/some/directory/application.key"
```

A file watcher is then watching the files and if they change, the SSL bundle will be reloaded. This in turn triggers a reload in the consuming component, e.g. Tomcat rotates the certificates in the SSL enabled connectors.

You can configure the quiet period (to make sure that there are no more changes) of the file watcher with the `configprop:spring.ssl.bundle.watch.file.quiet-period[]` property.

37. Task Execution and Scheduling

In the absence of an `javado:org.springframework.core.task.AsyncTaskExecutor` bean in the context, Spring Boot auto-configures an `javado:org.springframework.core.task.AsyncTaskExecutor`. When virtual threads are enabled (using Java 21+ and `configprop:spring.threads.virtual.enabled` set to `true`) this will be a `javado:org.springframework.core.task.SimpleAsyncTaskExecutor` that uses virtual threads. Otherwise, it will be a `javado:org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor` with sensible defaults.

The auto-configured `javado:org.springframework.core.task.AsyncTaskExecutor` is used for the following integrations unless a custom `javado:org.springframework.core.task.AsyncTaskExecutor` bean is defined:

- Execution of asynchronous tasks using `javado:org.springframework.scheduling.annotation.EnableAsync` [format=annotation], unless a bean of type `javado:org.springframework.scheduling.annotation.AsyncConfigurer` is defined.
- Asynchronous handling of `javado:org.springframework.scheduling.annotation.Async` return values from controller methods in Spring for GraphQL.
- Asynchronous request handling in Spring MVC.
- Support for blocking execution in Spring WebFlux.
- Utilized for inbound and outbound message channels in Spring WebSocket.
- Bootstrap executor for JPA, based on the bootstrap mode of JPA repositories.
- Bootstrap executor for `{url-spring-framework-docs}/core/beans/java/composing-configuration-classes.html#beans-java-startup-background` [background initialization] of beans in the `ApplicationContext`.

While this approach works in most scenarios, Spring Boot allows you to override the auto-configured `javado:org.springframework.core.task.AsyncTaskExecutor`. By default, when a custom `javado:org.springframework.core.task.AsyncTaskExecutor` bean is registered, the auto-configured `javado:org.springframework.core.task.AsyncTaskExecutor` backs off, and the custom `javado:org.springframework.core.task.AsyncTaskExecutor` is used for regular task execution (via `javado:org.springframework.scheduling.annotation.EnableAsync` [format=annotation]).

However, Spring MVC, Spring WebFlux, and Spring GraphQL all require a bean named `applicationTaskExecutor`. For Spring MVC and Spring WebFlux, this bean must be of type `javado:org.springframework.core.task.AsyncTaskExecutor`, whereas Spring GraphQL does not enforce this type requirement.

Spring WebSocket and JPA will use `javado:org.springframework.core.task.AsyncTaskExecutor` if either a single bean of this type is available or a bean named `applicationTaskExecutor` is defined.

Finally, the bootstrap executor of the `ApplicationContext` uses a bean named `applicationTaskExecutor` unless a bean named `bootstrapExecutor` is defined.

The following code snippet demonstrates how to register a custom `javado:org.springframework.core.task.AsyncTaskExecutor` to be used with Spring MVC, Spring WebFlux, Spring GraphQL, Spring WebSocket, JPA, and background initialization of beans.

```
include-code::application/MyTaskExecutorConfiguration[]
```



The `applicationTaskExecutor` bean will also be used for regular task execution if there is no `javado:org.springframework.context.annotation.Primary` [format=annotation] bean or a bean named `taskExecutor` of type `javado:org.springframework.core.task.AsyncTaskExecutor` or `javado:org.springframework.scheduling.annotation.AsyncConfigurer` present in the application context.



If neither the auto-configured `AsyncTaskExecutor` nor the `applicationTaskExecutor` bean is defined, the application defaults to a bean named `taskExecutor` for regular task execution (`javado:org.springframework.scheduling.annotation.EnableAsync` [format=annotation]), following Spring Framework's behavior. However, this bean will not be used for Spring MVC, Spring WebFlux, Spring GraphQL. It could, however, be used for Spring WebSocket or JPA if the bean's type is `javado:org.springframework.core.task.AsyncTaskExecutor`.

If your application needs multiple `Executor` beans for different integrations, such as one for regular task execution with `javado:org.springframework.scheduling.annotation.EnableAsync` [format=annotation] and other for Spring MVC, Spring WebFlux, Spring WebSocket and JPA, you can configure them as follows.

```
include-code::multiple/MyTaskExecutorConfiguration[]
```



The auto-configured `javado:org.springframework.boot.task.ThreadPoolTaskExecutorBuilder` or `javado:org.springframework.boot.task.SimpleAsyncTaskExecutorBuilder` allow you to easily create instances of type `javado:org.springframework.core.task.AsyncTaskExecutor` that replicate the default behavior of auto-configuration.

```
include-code::builder/MyTaskExecutorConfiguration[]
```

If a `taskExecutor` named bean is not an option, you can mark your bean as `javado:org.springframework.context.annotation.Primary` [format=annotation] or define an `javado:org.springframework.scheduling.annotation.AsyncConfigurer` bean to specify the `Executor` responsible for handling regular task execution with `javado:org.springframework.scheduling.annotation.EnableAsync` [format=annotation]. The following example demonstrates how to achieve this.

```
include-code::async/MyTaskExecutorConfiguration[]
```

To register a custom `javado:org.springframework.core.task.AsyncTaskExecutor` while keeping the auto-configured `javado:org.springframework.core.task.AsyncTaskExecutor`, you can create a custom `javado:org.springframework.core.task.AsyncTaskExecutor` bean and set the `defaultCandidate=false` attribute in its `javado:org.springframework.context.annotation.Bean` [format=annotation] annotation, as demonstrated in the following example:

```
include-code::defaultcandidate/MyTaskExecutorConfiguration[]
```


In that case, you will be able to autowire your custom `java.util.concurrent.Executor[]` into other components while retaining the auto-configured `javadoc:org.springframework.core.task.AsyncTaskExecutor[]`. However, remember to use the `javadoc:org.springframework.beans.factory.annotation.Qualifier[format=annotation]` annotation alongside `javadoc:org.springframework.beans.factory.annotation.Autowired[format=annotation]`.

If this is not possible for you, you can request Spring Boot to auto-configure an `javadoc:org.springframework.core.task.AsyncTaskExecutor[]` anyway, as follows:

```
spring:
  task:
    execution:
      mode: force
```

The auto-configured `javadoc:org.springframework.core.task.AsyncTaskExecutor[]` will be used automatically for all integrations, even if a custom `java.util.concurrent.Executor[]` bean is registered, including those marked as `javadoc:org.springframework.context.annotation.Primary[format=annotation]`. These integrations include:

- Asynchronous task execution (`javadoc:org.springframework.scheduling.annotation.EnableAsync[format=annotation]`), unless an `javadoc:org.springframework.scheduling.annotation.AsyncConfigurer[]` bean is present.
- Spring for GraphQL's asynchronous handling of `java.util.concurrent.Callable[]` return values from controller methods.
- Spring MVC's asynchronous request processing.
- Spring WebFlux's blocking execution support.
- Utilized for inbound and outbound message channels in Spring WebSocket.
- Bootstrap executor for JPA, based on the bootstrap mode of JPA repositories.
- Bootstrap executor for <https://spring-framework-docs.org/core/beans/java/composing-configuration-classes.html#beans-java-startup-background> [background initialization] of beans in the `ApplicationContext`, unless a bean named `bootstrapExecutor` is defined.



Depending on your target arrangement, you could set `configprop:spring.task.execution.mode[]` to `force` to auto-configure an `applicationTaskExecutor`, change your `java.util.concurrent.Executor[]` into an `javadoc:org.springframework.core.task.AsyncTaskExecutor[]` or define both an `javadoc:org.springframework.core.task.AsyncTaskExecutor[]` and an `javadoc:org.springframework.scheduling.annotation.AsyncConfigurer[]` wrapping your custom `java.util.concurrent.Executor[]`.



When `force` mode is enabled, `applicationTaskExecutor` will also be configured for regular task execution with `javadoc:org.springframework.scheduling.annotation.EnableAsync[format=annotation]`, even if a `javadoc:org.springframework.context.annotation.Primary[format=annotation]` bean or a bean named `taskExecutor` of type `java.util.concurrent.Executor[]` is present. The only way to override the `Executor` for regular tasks is by registering an `javadoc:org.springframework.scheduling.annotation.AsyncConfigurer[]` bean.

When a `javadoc:org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor[]` is auto-configured, the thread pool uses 8 core threads that can grow and shrink according to the load. Those default settings can be fine-tuned using the `spring.task.execution` namespace, as shown in the following example:

```
spring:
  task:
    execution:
      pool:
        max-size: 16
        queue-capacity: 100
        keep-alive: "10s"
```

This changes the thread pool to use a bounded queue so that when the queue is full (100 tasks), the thread pool increases to maximum 16 threads. Shrinking of the pool is more aggressive as threads are reclaimed when they are idle for 10 seconds (rather than 60 seconds by default).

A scheduler can also be auto-configured if it needs to be associated with scheduled task execution (using `javadoc:org.springframework.scheduling.annotation.EnableScheduling[format=annotation]` for instance).

If virtual threads are enabled (using Java 21+ and `configprop:spring.threads.virtual.enabled[]` set to `true`) this will be a `javadoc:org.springframework.scheduling.concurrent.SimpleAsyncTaskScheduler[]` that uses virtual threads. This `javadoc:org.springframework.scheduling.concurrent.SimpleAsyncTaskScheduler[]` will ignore any pooling related properties.

If virtual threads are not enabled, it will be a `javadoc:org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler[]` with sensible defaults. The `javadoc:org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler[]` uses one thread by default and its settings can be fine-tuned using the `spring.task.scheduling` namespace, as shown in the following example:

```
spring:
  task:
    scheduling:
      thread-name-prefix: "scheduling-"
    pool:
      size: 2
```

A `javadoc:org.springframework.boot.task.ThreadPoolTaskExecutorBuilder[]` bean, a `javadoc:org.springframework.boot.task.SimpleAsyncTaskExecutorBuilder[]` bean, a `javadoc:org.springframework.boot.task.ThreadPoolTaskSchedulerBuilder[]` bean and a `javadoc:org.springframework.boot.task.SimpleAsyncTaskSchedulerBuilder[]` are made available in the context if a custom executor or scheduler needs to be created. The `javadoc:org.springframework.boot.task.SimpleAsyncTaskExecutorBuilder[]` and `javadoc:org.springframework.boot.task.SimpleAsyncTaskSchedulerBuilder[]` beans are auto-configured to use virtual threads if they are enabled (using Java 21+ and `configprop:spring.threads.virtual.enabled[]` set to `true`).

38. Reference

This section provides information on using the features and capabilities of Spring Boot.

39. Caching

The Spring Framework provides support for transparently adding caching to an application. At its core, the abstraction applies caching to methods, thus reducing the number of executions based on the information available in the cache. The caching logic is applied transparently, without any interference to the invoker. For more details, check the [{url-spring-framework-docs}/integration/cache.html](#) [relevant section] of the Spring Framework reference documentation.

Spring Boot auto-configures the cache infrastructure as long as caching support is enabled by using the `javax.cache.annotation.EnableCaching` annotation.



Avoid adding `javax.cache.annotation.EnableCaching` to the main method's application class. Doing so makes caching a mandatory feature, including [when running a test suite](#).

To add caching to an operation of your service add the relevant annotation to its method, as shown in the following example:

```
include-code::MyMathService[]
```

This example demonstrates the use of caching on a potentially costly operation. Before invoking `computePiDecimal`, the abstraction looks for an entry in the `piDecimals` cache that matches the `precision` argument. If an entry is found, the content in the cache is immediately returned to the caller, and the method is not invoked. Otherwise, the method is invoked, and the cache is updated before returning the value.



You can also use the standard JSR-107 (JCache) annotations (such as `javax.cache.annotation.CacheResult`) transparently. However, we strongly advise you to not mix and match the Spring Cache and JCache annotations.

If you do not add any specific cache library, Spring Boot auto-configures a [simple provider](#) that uses concurrent maps in memory. When a cache is required (such as `piDecimals` in the preceding example), this provider creates it for you. The simple provider is not really recommended for production usage, but it is great for getting started and making sure that you understand the features. When you have made up your mind about the cache provider to use, please make sure to read its documentation to figure out how to configure the caches that your application uses. Nearly all providers require you to explicitly configure every cache that you use in the application. Some offer a way to customize the default caches defined by the `spring.cache.cache-names[]` property.



It is also possible to transparently [{url-spring-framework-docs}/integration/cache/annotations.html#cache-annotations-put\[update\]](#) or [{url-spring-framework-docs}/integration/cache/annotations.html#cache-annotations-evict\[evict\]](#) data from the cache.

39.1. Supported Cache Providers

The cache abstraction does not provide an actual store and relies on abstraction materialized by the `javax.cache.Cache` and `javax.cache.CacheManager` interfaces.

If you have not defined a bean of type `javax.cache.CacheManager` or a `javax.cache.interceptor.CacheResolver` named `cacheResolver` (see `javax.cache.annotation.CachingConfigurer`), Spring Boot tries to detect the following providers (in the indicated order):

1. [io/caching.html](#)
2. [io/caching.html](#) (EhCache 3, Hazelcast, Infinispan, and others)
3. [io/caching.html](#)
4. [io/caching.html](#)
5. [io/caching.html](#)
6. [io/caching.html](#)
7. [io/caching.html](#)
8. [io/caching.html](#)
9. [io/caching.html](#)



If the `javax.cache.CacheManager` is auto-configured by Spring Boot, it is possible to *force* a particular cache provider by setting the `spring.cache.type[]` property.



Use the `spring-boot-starter-cache` starter to quickly add basic caching dependencies. The starter brings in `spring-context-support`. If you add dependencies manually, you must include `spring-context-support` in order to use the JCache or Caffeine support.

If the `javax.cache.CacheManager` is auto-configured by Spring Boot, you can further tune its configuration before it is fully initialized by exposing a bean that implements the `javax.cache.boot.autoconfigure.CacheManagerCustomizer` interface. The following example sets a flag to say that `null` values should not be passed down to the underlying map:

```
include-code::MyCacheManagerConfiguration[]
```



In the preceding example, an auto-configured `javax.cache.concurrent.ConcurrentMapCacheManager` is expected. If that is not the case (either you provided your own config or a different cache provider was auto-configured), the customizer is not invoked at all. You can have as many customizers as you want, and you can also order them by using `javax.annotation.Order` or `javax.annotation.Ordered`.

39.1.1. Generic

Generic caching is used if the context defines *at least* one `javax.cache.Cache` bean. A `javax.cache.CacheManager` wrapping all beans of that type is created.

39.1.2. JCache (JSR-107)

JCache (<https://jcp.org/en/jsr/detail?id=107>) is bootstrapped through the presence of a `javax.cache.spi.CachingProvider` on the classpath (that is, a JSR-107 compliant caching library exists on the classpath), and the `javax.cache.jcache.JCacheCacheManager` is provided by the `spring-boot-starter-cache` starter. Various compliant libraries are available, and Spring Boot provides dependency management for Ehcache 3, Hazelcast, and Infinispan. Any other compliant library can be added as well.

It might happen that more than one provider is present, in which case the provider must be explicitly specified. Even if the JSR-107 standard does not enforce a standardized way to define the location of the configuration file, Spring Boot does its best to accommodate setting a cache with implementation details, as shown in the following example:

```
# Only necessary if more than one provider is present
spring:
  cache:
    jcache:
      provider: "com.example.MyCachingProvider"
      config: "classpath:example.xml"
```



When a cache library offers both a native implementation and JSR-107 support, Spring Boot prefers the JSR-107 support, so that the same features are available if you switch to a different JSR-107 implementation.



Spring Boot has [general support for Hazelcast](#). If a single `com.hazelcast.core.HazelcastInstance` is available, it is automatically reused for the `javax.cache.CacheManager` as well, unless the `config: spring.cache.jcache.config` property is specified.

There are two ways to customize the underlying `javax.cache.CacheManager`:

- Caches can be created on startup by setting the `config: spring.cache.cache-names` property. If a custom `javax.cache.configuration.Configuration` bean is defined, it is used to customize them.
- `javax.cache.configuration.CacheManagerCustomizer` beans are invoked with the reference of the `javax.cache.CacheManager` for full customization.



If a standard `javax.cache.CacheManager` bean is defined, it is wrapped automatically in an `javax.cache.CacheManager` implementation that the abstraction expects. No further customization is applied to it.

39.1.3. Hazelcast

Spring Boot has [general support for Hazelcast](#). If a `com.hazelcast.core.HazelcastInstance` has been auto-configured and `com.hazelcast:hazelcast-spring` is on the classpath, it is automatically wrapped in a `javax.cache.CacheManager`.



Hazelcast can be used as a JCache compliant cache or as a Spring `javax.cache.CacheManager` compliant cache. When setting `config: spring.cache.type` to `hazelcast`, Spring Boot will use the `javax.cache.CacheManager` based implementation. If you want to use Hazelcast as a JCache compliant cache, set `config: spring.cache.type` to `jcache`. If you have multiple JCache compliant cache providers and want to force the use of Hazelcast, you have to [explicitly set the JCache provider](#).

39.1.4. Infinispan

Infinispan (<https://infinispan.org/>) has no default configuration file location, so it must be specified explicitly. Otherwise, the default bootstrap is used.

```
spring:
  cache:
    infinispan:
      config: "infinispan.xml"
```

Caches can be created on startup by setting the `config: spring.cache.cache-names` property. If a custom `org.infinispan.configuration.cache.ConfigurationBuilder` bean is defined, it is used to customize the caches.

For more details, see [the documentation](https://infinispan.org/docs/stable/titles/spring/spring.html) (<https://infinispan.org/docs/stable/titles/spring/spring.html>).

39.1.5. Couchbase

If Spring Data Couchbase is available and Couchbase is [configured](#), a `org.springframework.data.couchbase.cache.CouchbaseCacheManager` is auto-configured. It is possible to create additional caches on startup by setting the `config: spring.cache.cache-names` property and cache defaults can be configured by using `spring.cache.couchbase.*` properties. For instance, the following configuration creates `cache1` and `cache2` caches with an entry *expiration* of 10 minutes:

```
spring:
  cache:
    cache-names: "cache1,cache2"
    couchbase:
      expiration: "10m"
```

If you need more control over the configuration, consider registering a `org.springframework.boot.cache.autoconfigure.CouchbaseCacheManagerBuilderCustomizer` bean. The following example shows a customizer that configures a specific entry expiration for `cache1` and `cache2`:

include-code::MyCouchbaseCacheManagerConfiguration[]

39.1.6. Redis

If [Redis](https://redis.io/) (https://redis.io/) is available and configured, a `javadoc:org.springframework.data.redis.cache.RedisCacheManager[]` is auto-configured. It is possible to create additional caches on startup by setting the `configprop:spring.cache.cache-names[]` property and cache defaults can be configured by using `spring.cache.redis.*` properties. For instance, the following configuration creates `cache1` and `cache2` caches with a *time to live* of 10 minutes:

```
spring:
  cache:
    cache-names: "cache1,cache2"
    redis:
      time-to-live: "10m"
```



By default, a key prefix is added so that, if two separate caches use the same key, Redis does not have overlapping keys and cannot return invalid values. We strongly recommend keeping this setting enabled if you create your own `javadoc:org.springframework.data.redis.cache.RedisCacheManager[]`.



You can take full control of the default configuration by adding a `javadoc:org.springframework.data.redis.cache.RedisCacheConfiguration[]` `javadoc:org.springframework.context.annotation.Bean[format=annotation]` of your own. This can be useful if you need to customize the default serialization strategy.

If you need more control over the configuration, consider registering a `javadoc:org.springframework.boot.cache.autoconfigure.RedisCacheManagerBuilderCustomizer[]` bean. The following example shows a customizer that configures a specific time to live for `cache1` and `cache2`:

include-code::MyRedisCacheManagerConfiguration[]

39.1.7. Caffeine

[Caffeine](https://github.com/ben-manes/caffeine) (https://github.com/ben-manes/caffeine) is a Java 8 rewrite of Guava's cache that supersedes support for Guava. If Caffeine is present, a `javadoc:org.springframework.cache.caffeine.CaffeineCacheManager[]` (provided by the `spring-boot-starter-cache` starter) is auto-configured. Caches can be created on startup by setting the `configprop:spring.cache.cache-names[]` property and can be customized by one of the following (in the indicated order):

1. A cache spec defined by `spring.cache.caffeine.spec`
2. A `javadoc:com.github.benmanes.caffeine.cache.CaffeineSpec[]` bean is defined
3. A `javadoc:com.github.benmanes.caffeine.cache.Caffeine[]` bean is defined

For instance, the following configuration creates `cache1` and `cache2` caches with a maximum size of 500 and a *time to live* of 10 minutes

```
spring:
  cache:
    cache-names: "cache1,cache2"
    caffeine:
      spec: "maximumSize=500,expireAfterAccess=600s"
```

If a `javadoc:com.github.benmanes.caffeine.cache.CacheLoader[]` bean is defined, it is automatically associated to the `javadoc:org.springframework.cache.caffeine.CaffeineCacheManager[]`. Since the `javadoc:com.github.benmanes.caffeine.cache.CacheLoader[]` is going to be associated with *all* caches managed by the cache manager, it must be defined as `CacheLoader<Object, Object>`. The auto-configuration ignores any other generic type.

39.1.8. Cache2k

[Cache2k](https://cache2k.org/) (https://cache2k.org/) is an in-memory cache. If the Cache2k spring integration is present, a `SpringCache2kCacheManager` is auto-configured.

Caches can be created on startup by setting the `configprop:spring.cache.cache-names[]` property. Cache defaults can be customized using a `javadoc:org.springframework.boot.cache.autoconfigure.Cache2kBuilderCustomizer[]` bean. The following example shows a customizer that configures the capacity of the cache to 200 entries, with an expiration of 5 minutes:

include-code::MyCache2kDefaultsConfiguration[]

39.1.9. Simple

If none of the other providers can be found, a simple implementation using a `javadoc:java.util.concurrent.ConcurrentHashMap[]` as the cache store is configured. This is the default if no caching library is present in your application. By default, caches are created as needed, but you can restrict the list of available caches by setting the `cache-names` property. For instance, if you want only `cache1` and `cache2` caches, set the `cache-names` property as follows:

```
spring:
  cache:
    cache-names: "cache1,cache2"
```

If you do so and your application uses a cache not listed, then it fails at runtime when the cache is needed, but not on startup. This is similar to the way the "real" cache providers behave if you use an undeclared cache.

39.1.10. None

If you need to use a no-op cache rather than the auto-configured cache manager in a certain environment, set the cache type to `none`, as shown in the following example:

```
spring:
  cache:
    type: "none"
```

39.2. Testing

It is generally useful to use a no-op implementation when running a test suite. This section lists a number of strategies that are useful for tests.

When a custom `javado:org.springframework.cache.CacheManager[]` is defined, the best option is to make sure that caching configuration is defined in an isolated `javado:org.springframework.context.annotation.Configuration[format=annotation]` class. Doing so makes sure that caching is not required by slice tests. For tests that enable a full context, such as `javado:org.springframework.boot.test.context.SpringBootTest[format=annotation]`, an explicit configuration overriding the regular configuration is required.

If caching is auto-configured, more options are available. Tests can be annotated with `javado:org.springframework.boot.test.autoconfigure.core.AutoConfigureCache[format=annotation]` to replace the auto-configured `javado:org.springframework.cache.CacheManager[]` by a no-op implementation.

```
include-code::MyIntegrationTests[]
```

Another option is to force a no-op implementation for the auto-configured `javado:org.springframework.cache.CacheManager[]`:

```
spring:
  cache:
    type: "none"
```

40. Sending Email

The Spring Framework provides an abstraction for sending email by using the `javax.mail.JavaMailSender` interface, and Spring Boot provides auto-configuration for it as well as a starter module.



See the <https://spring-framework-docs.org/integration/email.html> [reference documentation] for a detailed explanation of how you can use `javax.mail.JavaMailSender`.

If `spring.mail.host` and the relevant libraries (as defined by `spring-boot-starter-mail`) are available, a default `javax.mail.JavaMailSender` is created if none exists. The sender can be further customized by configuration items from the `spring.mail` namespace. See `javax.mail.Session` for more details.

In particular, certain default timeout values are infinite, and you may want to change that to avoid having a thread blocked by an unresponsive mail server, as shown in the following example:

```
spring:
  mail:
    properties:
      "[mail.smtp.connectiontimeout]": 5000
      "[mail.smtp.timeout]": 3000
      "[mail.smtp.writetimeout]": 5000
```

It is also possible to configure a `javax.mail.JavaMailSender` with an existing `javax.mail.Session` from JNDI:

```
spring:
  mail:
    jndi-name: "mail/Session"
```

When a `jndi-name` is set, it takes precedence over all other Session-related settings.

41. Hazelcast

If [Hazelcast](https://hazelcast.com/) (<https://hazelcast.com/>) is on the classpath and a suitable configuration is found, Spring Boot auto-configures a `javadoc:com.hazelcast.core.HazelcastInstance[]` that you can inject in your application.

Spring Boot first attempts to create a client by checking the following configuration options:

- The presence of a `javadoc:com.hazelcast.client.config.ClientConfig[]` bean.
- A configuration file defined by the `configprop:spring.hazelcast.config[]` property.
- The presence of the `hazelcast.client.config` system property.
- A `hazelcast-client.xml` in the working directory or at the root of the classpath.
- A `hazelcast-client.yaml` (or `hazelcast-client.yml`) in the working directory or at the root of the classpath.

If a client can not be created, Spring Boot attempts to configure an embedded server. If you define a `javadoc:com.hazelcast.config.Config[]` bean, Spring Boot uses it at is. If your configuration defines an instance name, Spring Boot tries to locate an existing instance rather than creating a new one.

You could also specify the Hazelcast configuration file to use through configuration, as shown in the following example:

```
spring:
  hazelcast:
    config: "classpath:config/my-hazelcast.xml"
```

Otherwise, Spring Boot tries to find the Hazelcast configuration from the default locations: `hazelcast.xml` in the working directory or at the root of the classpath, or a YAML counterpart in the same locations. We also check if the `hazelcast.config` system property is set. See the [Hazelcast documentation](https://docs.hazelcast.org/docs/latest/manual/html-single/) (<https://docs.hazelcast.org/docs/latest/manual/html-single/>) for more details.



By default, `javadoc:com.hazelcast.spring.context.SpringAware[format=annotation]` on Hazelcast components is supported. The `javadoc:com.hazelcast.core.ManagedContext[]` can be overridden by declaring a `javadoc:org.springframework.boot.hazelcast.autoconfigure.HazelcastConfigCustomizer[]` bean with an `javadoc:org.springframework.core.annotation.Order[format=annotation]` higher than zero.



Spring Boot also has [explicit caching support for Hazelcast](#). If caching is enabled, the `javadoc:com.hazelcast.core.HazelcastInstance[]` is automatically wrapped in a `javadoc:org.springframework.cache.CacheManager[]` implementation.

42. IO

Most applications will need to deal with input and output concerns at some point. Spring Boot provides utilities and integrations with a range of technologies to help when you need IO capabilities. This section covers standard IO features such as caching and validation as well as more advanced topics such as batch, scheduling, and distributed transactions. We will also cover calling remote REST or SOAP services and sending email.

43. Distributed Transactions With JTA

Spring Boot supports distributed JTA transactions across multiple XA resources by using a transaction manager retrieved from JNDI.

When a JTA environment is detected, Spring's `javax.transaction.jta.JtaTransactionManager` is used to manage transactions. Auto-configured JMS, DataSource, and JPA beans are upgraded to support XA transactions. You can use standard Spring idioms, such as `javax.transaction.Transactional`, to participate in a distributed transaction. If you are within a JTA environment and still want to use local transactions, you can set the `spring.jta.enabled` property to `false` to disable the JTA auto-configuration.

43.1. Using a Jakarta EE Managed Transaction Manager

If you package your Spring Boot application as a `war` or `ear` file and deploy it to a Jakarta EE application server, you can use your application server's built-in transaction manager. Spring Boot tries to auto-configure a transaction manager by looking at common JNDI locations (`java:comp/UserTransaction` , `java:comp/TransactionManager` , and so on). When using a transaction service provided by your application server, you generally also want to ensure that all resources are managed by the server and exposed over JNDI. Spring Boot tries to auto-configure JMS by looking for a `javax.jms.ConnectionFactory` at the JNDI path (`java:/JmsXA` or `java:/XAConnectionFactory`), and you can use the `spring.datasource.jndi-name` property to configure your `javax.sql.DataSource`.

43.2. Mixing XA and Non-XA JMS Connections

When using JTA, the primary `javax.jms.ConnectionFactory` bean is XA-aware and participates in distributed transactions. You can inject into your bean without needing to use any `javax.transaction.Transactional`:

```
include-code::primary/MyBean[]
```

In some situations, you might want to process certain JMS messages by using a non-XA `javax.jms.ConnectionFactory`. For example, your JMS processing logic might take longer than the XA timeout.

If you want to use a non-XA `javax.jms.ConnectionFactory`, you can the `nonXaJmsConnectionFactory` bean:

```
include-code::nonxa/MyBean[]
```

For consistency, the `jmsConnectionFactory` bean is also provided by using the bean alias `xaJmsConnectionFactory`:

```
include-code::xa/MyBean[]
```

43.3. Supporting an Embedded Transaction Manager

The `javax.transaction.jta.XAConnectionFactoryWrapper` and `javax.transaction.jta.XADataSourceWrapper` interfaces can be used to support embedded transaction managers. The interfaces are responsible for wrapping `javax.jms.XAConnectionFactory` and `javax.sql.XADataSource` beans and exposing them as regular `javax.jms.ConnectionFactory` and `javax.sql.DataSource` beans, which transparently enroll in the distributed transaction. DataSource and JMS auto-configuration use JTA variants, provided you have a `javax.transaction.jta.JtaTransactionManager` bean and appropriate XA wrapper beans registered within your `ApplicationContext`.

44. Quartz Scheduler

Spring Boot offers several conveniences for working with the [Quartz scheduler](https://www.quartz-scheduler.org/) (<https://www.quartz-scheduler.org/>), including the `spring-boot-starter-quartz` starter. If Quartz is available, a `javado:org.quartz.Scheduler[]` is auto-configured (through the `javado:org.springframework.scheduling.quartz.SchedulerFactoryBean[]` abstraction).

Beans of the following types are automatically picked up and associated with the `javado:org.quartz.Scheduler[]`:

- `javado:org.quartz.JobDetail[]`: defines a particular Job. `javado:org.quartz.JobDetail[]` instances can be built with the `javado:org.quartz.JobBuilder[]` API.
- `javado:org.quartz.Calendar[]`.
- `javado:org.quartz.Trigger[]`: defines when a particular job is triggered.

By default, an in-memory `javado:org.quartz.spi.JobStore[]` is used. However, it is possible to configure a JDBC-based store if a `javado:javax.sql.DataSource[]` bean is available in your application and if the `configprop:spring.quartz.job-store-type[]` property is configured accordingly, as shown in the following example:

```
spring:
  quartz:
    job-store-type: "jdbc"
```

When the JDBC store is used, the schema can be initialized on startup, as shown in the following example:

```
spring:
  quartz:
    jdbc:
      initialize-schema: "always"
```



By default, the database is detected and initialized by using the standard scripts provided with the Quartz library. These scripts drop existing tables, deleting all triggers on every restart. To use a custom script, set the `configprop:spring.quartz.jdbc.schema[]` property. Some of the standard scripts – such as those for SQL Server, Azure SQL, and Sybase – cannot be used without modification. In these cases, make a copy of the script and edit it as directed in the script's comments then set `configprop:spring.quartz.jdbc.schema[]` to use your customized script.

To have Quartz use a `javado:javax.sql.DataSource[]` other than the application's main `javado:javax.sql.DataSource[]`, declare a `javado:javax.sql.DataSource[]` bean, annotating its `javado:org.springframework.context.annotation.Bean[format=annotation]` method with `javado:org.springframework.boot.quartz.autoconfigure.QuartzDataSource[format=annotation]`. Doing so ensures that the Quartz-specific `javado:javax.sql.DataSource[]` is used by both the `javado:org.springframework.scheduling.quartz.SchedulerFactoryBean[]` and for schema initialization. Similarly, to have Quartz use a `javado:org.springframework.transaction.TransactionManager[]` other than the application's main `javado:org.springframework.transaction.TransactionManager[]` declare a `javado:org.springframework.transaction.TransactionManager[]` bean, annotating its `javado:org.springframework.context.annotation.Bean[format=annotation]` method with `javado:org.springframework.boot.quartz.autoconfigure.QuartzTransactionManager[format=annotation]`.

By default, jobs created by configuration will not overwrite already registered jobs that have been read from a persistent job store. To enable overwriting existing job definitions set the `configprop:spring.quartz.override-existing-jobs[]` property.

Quartz Scheduler configuration can be customized using `spring.quartz` properties and `javado:org.springframework.boot.quartz.autoconfigure.SchedulerFactoryBeanCustomizer[]` beans, which allow programmatic `javado:org.springframework.scheduling.quartz.SchedulerFactoryBean[]` customization. Advanced Quartz configuration properties can be customized using `spring.quartz.properties.*`.



In particular, an `javado:java.util.concurrent.Executor[]` bean is not associated with the scheduler as Quartz offers a way to configure the scheduler through `spring.quartz.properties`. If you need to customize the task executor, consider implementing `javado:org.springframework.boot.quartz.autoconfigure.SchedulerFactoryBeanCustomizer[]`.

Jobs can define setters to inject data map properties. Regular beans can also be injected in a similar manner, as shown in the following example:

```
include-code::MySampleJob[]
```

45. Calling REST Services

Spring Boot provides various convenient ways to call remote REST services. If you are developing a non-blocking reactive application and you're using Spring WebFlux, then you can use `javax.ws.rs.client.WebClient[]`. If you prefer imperative APIs then you can use `javax.ws.rs.client.RestClient[]` or `javax.ws.rs.client.RestTemplate[]`.

45.1. WebClient

If you have Spring WebFlux on your classpath we recommend that you use `javax.ws.rs.client.WebClient[]` to call remote REST services. The `javax.ws.rs.client.WebClient[]` interface provides a functional style API and is fully reactive. You can learn more about the `javax.ws.rs.client.WebClient[]` in the dedicated [url-spring-framework-docs/web/webflux-webclient.html](#) section in the Spring Framework docs.



If you are not writing a reactive Spring WebFlux application you can use the [RestClient](#) instead of a `javax.ws.rs.client.WebClient[]`. This provides a similar functional API, but is imperative rather than reactive.

Spring Boot creates and pre-configures a prototype `javax.ws.rs.client.WebClient$Builder[]` bean for you. It is strongly advised to inject it in your components and use it to create `javax.ws.rs.client.WebClient[]` instances. Spring Boot is configuring that builder to share HTTP resources and reflect codecs setup in the same fashion as the server ones (see [WebFlux HTTP codecs auto-configuration](#)), and more.

The following code shows a typical example:

```
include-code::MyService[]
```

45.1.1. WebClient Runtime

Spring Boot will auto-detect which `javax.ws.rs.client.WebClient$Builder[]` to use to drive `javax.ws.rs.client.WebClient[]` depending on the libraries available on the application classpath. In order of preference, the following clients are supported:

1. Reactor Netty
2. Jetty RS client
3. Apache HttpClient
4. JDK HttpClient

If multiple clients are available on the classpath, the most preferred client will be used.

The `spring-boot-starter-webflux` starter depends on `io.projectreactor.netty:reactor-netty` by default, which brings both server and client implementations. If you choose to use Jetty as a reactive server instead, you should add a dependency on the Jetty Reactive HTTP client library, `org.eclipse.jetty:jetty-reactive-httpclient`. Using the same technology for server and client has its advantages, as it will automatically share HTTP resources between client and server.

Developers can override the resource configuration for Jetty and Reactor Netty by providing a custom `javax.ws.rs.client.ReactorResourceFactory[]` or `javax.ws.rs.client.JettyResourceFactory[]` bean - this will be applied to both clients and servers.

If you wish to override that choice for the client, you can define your own `javax.ws.rs.client.WebClient$Builder[]` bean and have full control over the client configuration.

You can learn more about the [url-spring-framework-docs/web/webflux-webclient/client-builder.html](#) `WebClient` configuration options in the Spring Framework reference documentation.

45.1.2. Global HTTP Connector Configuration

If the auto-detected `javax.ws.rs.client.WebClient$Builder[]` does not meet your needs, you can use the `configprop:spring.http.clients.reactive.connector[]` property to pick a specific connector. For example, if you have Reactor Netty on your classpath, but you prefer Jetty's `javax.ws.rs.client.WebClient$Builder[]` you can add the following:

```
spring:
  http:
    clients:
      reactive:
        connector: jetty
```



You can also use [global configuration properties](#) which apply to all HTTP clients.

For more complex customizations, you can use `javax.ws.rs.client.WebClient$Builder[]` or declare your own `javax.ws.rs.client.WebClient$Builder[]` bean which will cause auto-configuration to back off. This can be useful when you need to customize some of the internals of the underlying HTTP library.

For example, the following will use a JDK client configured with a specific `javax.ws.rs.client.WebClient$Builder[]`:

```
include-code::MyConnectorHttpConfiguration[]
```

45.1.3. WebClient Customization

There are three main approaches to `javax.ws.rs.client.WebClient` customization, depending on how broadly you want the customizations to apply.

To make the scope of any customizations as narrow as possible, inject the auto-configured `javax.ws.rs.client.WebClient$Builder` and then call its methods as required. `javax.ws.rs.client.WebClient$Builder` instances are stateful: Any change on the builder is reflected in all clients subsequently created with it. If you want to create several clients with the same builder, you can also consider cloning the builder with `WebClient.Builder other = builder.clone();`.

To make an application-wide, additive customization to all `javax.ws.rs.client.WebClient$Builder` instances, you can declare `javax.ws.rs.client.WebClientCustomizer` beans and change the `javax.ws.rs.client.WebClient$Builder` locally at the point of injection.

Finally, you can fall back to the original API and use `WebClient.create()`. In that case, no auto-configuration or `javax.ws.rs.client.WebClientCustomizer` is applied.

45.1.4. WebClient SSL Support

If you need custom SSL configuration on the `javax.ws.rs.client.WebClient$Builder` used by the `javax.ws.rs.client.WebClient`, you can inject a `javax.ws.rs.client.WebClientSsl` instance that can be used with the builder's `apply` method.

The `javax.ws.rs.client.WebClientSsl` interface provides access to any [SSL bundles](#) that you have defined in your `application.properties` or `application.yml` file.

The following code shows a typical example:

```
include-code::MyService[]
```

45.2. RestClient

If you are not using Spring WebFlux or Project Reactor in your application we recommend that you use `javax.ws.rs.client.RestClient` to call remote REST services.

The `javax.ws.rs.client.RestClient` interface provides a functional style imperative API.

Spring Boot creates and pre-configures a prototype `javax.ws.rs.client.RestClient$Builder` bean for you. It is strongly advised to inject it in your components and use it to create `javax.ws.rs.client.RestClient` instances. Spring Boot is configuring that builder with `javax.ws.rs.client.WebClient$Builder` and an appropriate `javax.ws.rs.client.WebClient$Builder`.

The following code shows a typical example:

```
include-code::MyService[]
```

45.2.1. RestClient Customization

There are three main approaches to `javax.ws.rs.client.RestClient` customization, depending on how broadly you want the customizations to apply.

To make the scope of any customizations as narrow as possible, inject the auto-configured `javax.ws.rs.client.RestClient$Builder` and then call its methods as required. `javax.ws.rs.client.RestClient$Builder` instances are stateful: Any change on the builder is reflected in all clients subsequently created with it. If you want to create several clients with the same builder, you can also consider cloning the builder with `RestClient.Builder other = builder.clone();`.

To make an application-wide, additive customization to all `javax.ws.rs.client.RestClient$Builder` instances, you can declare `javax.ws.rs.client.RestClientCustomizer` beans and change the `javax.ws.rs.client.RestClient$Builder` locally at the point of injection.

Finally, you can fall back to the original API and use `RestClient.create()`. In that case, no auto-configuration or `javax.ws.rs.client.RestClientCustomizer` is applied.



You can also change the [global HTTP client configuration](#).

45.2.2. RestClient SSL Support

If you need custom SSL configuration on the `javax.ws.rs.client.WebClient$Builder` used by the `javax.ws.rs.client.RestClient`, you can inject a `javax.ws.rs.client.WebClientSsl` instance that can be used with the builder's `apply` method.

The `javax.ws.rs.client.WebClientSsl` interface provides access to any [SSL bundles](#) that you have defined in your `application.properties` or `application.yml` file.

The following code shows a typical example:

```
include-code::MyService[]
```

If you need to apply other customization in addition to an SSL bundle, you can use the `javax.ws.rs.client.WebClient$Builder` class with `javax.ws.rs.client.WebClient$Builder`:

```
include-code::settings/MyService[]
```

45.3. RestTemplate

Spring Framework's `javax.ws.rs.client.RestTemplate` class predates `javax.ws.rs.client.RestClient` and is the classic way that many applications use to call remote REST services. You might choose to use `javax.ws.rs.client.RestTemplate` when you have existing code that you don't want to migrate to `javax.ws.rs.client.RestClient`, or because you're already familiar with the `javax.ws.rs.client.RestTemplate` API.

Since `javax.ws.rs.client.RestTemplate` instances often need to be customized before being used, Spring Boot does not provide any single auto-configured `javax.ws.rs.client.RestTemplate` bean. It does, however, auto-configure a `javax.ws.rs.client.RestTemplateBuilder`, which can be used to create `javax.ws.rs.client.RestTemplate` instances when needed. The auto-configured `javax.ws.rs.client.RestTemplateBuilder` ensures that sensible `javax.ws.rs.client.HttpMessageConverters` and an appropriate `javax.ws.rs.client.ClientHttpRequestFactory` are applied to `javax.ws.rs.client.RestTemplate` instances.

The following code shows a typical example:

```
include-code::MyService[]
```

`javax.ws.rs.client.RestTemplateBuilder` includes a number of useful methods that can be used to quickly configure a `javax.ws.rs.client.RestTemplate`. For example, to add BASIC authentication support, you can use `builder.basicAuthentication("user", "password").build()`.

45.3.1. RestTemplate Customization

There are three main approaches to `javax.ws.rs.client.RestTemplate` customization, depending on how broadly you want the customizations to apply.

To make the scope of any customizations as narrow as possible, inject the auto-configured `javax.ws.rs.client.RestTemplateBuilder` and then call its methods as required. Each method call returns a new `javax.ws.rs.client.RestTemplateBuilder` instance, so the customizations only affect this use of the builder.

To make an application-wide, additive customization, use a `javax.ws.rs.client.RestTemplateCustomizer` bean. All such beans are automatically registered with the auto-configured `javax.ws.rs.client.RestTemplateBuilder` and are applied to any templates that are built with it.

The following example shows a customizer that configures the use of a proxy for all hosts except `192.168.0.5`:

```
include-code::MyRestTemplateCustomizer[]
```

Finally, you can define your own `javax.ws.rs.client.RestTemplateBuilder` bean. Doing so will replace the auto-configured builder. If you want any `javax.ws.rs.client.RestTemplateCustomizer` beans to be applied to your custom builder, as the auto-configuration would have done, configure it using a `javax.ws.rs.client.RestTemplateBuilderConfigurer`. The following example exposes a `javax.ws.rs.client.RestTemplateBuilder` that matches what Spring Boot's auto-configuration would have done, except that custom connect and read timeouts are also specified:

```
include-code::MyRestTemplateBuilderConfiguration[]
```

The most extreme (and rarely used) option is to create your own `javax.ws.rs.client.RestTemplateBuilder` bean without using a configurator. In addition to replacing the auto-configured builder, this also prevents any `javax.ws.rs.client.RestTemplateCustomizer` beans from being used.



You can also change the [global HTTP client configuration](#).

45.3.2. RestTemplate SSL Support

If you need custom SSL configuration on the `javax.ws.rs.client.RestTemplate`, you can apply an [SSL bundle](#) to the `javax.ws.rs.client.RestTemplateBuilder` as shown in this example:

```
include-code::MyService[]
```

45.4. HTTP Client Detection for RestClient and RestTemplate

Spring Boot will auto-detect which HTTP client to use with `javax.ws.rs.client.RestClient` and `javax.ws.rs.client.RestTemplate` depending on the libraries available on the application classpath. In order of preference, the following clients are supported:

1. Apache HttpClient
2. Jetty HttpClient
3. Reactor Netty HttpClient
4. JDK client (`java.net.http.HttpClient`)
5. Simple JDK client (`java.net.HttpURLConnection`)

If multiple clients are available on the classpath, and no global configuration is provided, the most preferred client will be used.

45.4.1. Global HTTP Client Configuration

If the auto-detected HTTP client does not meet your needs, you can use the `configprop:spring.http.clients.imperative.factory` property to pick a specific factory. For example, if you have Apache HttpClient on your classpath, but you prefer Jetty's `javax.ws.rs.client.HttpClient` you can add the following:

```
spring:
  http:
    clients:
      imperative:
        factory: jetty
```



You can also use [global configuration properties](#) which apply to all HTTP clients.

For more complex customizations, you can use `javado:org.springframework.boot.http.client.autoconfigure.ClientHttpRequestFactoryBuilderCustomizer[]` or declare your own `javado:org.springframework.boot.http.client.ClientHttpRequestFactoryBuilder[]` bean which will cause auto-configuration to back off. This can be useful when you need to customize some of the internals of the underlying HTTP library.

For example, the following will use a JDK client configured with a specific `javado:java.net.ProxySelector[]`:

```
include-code::MyClientHttpConfiguration[]
```

45.5. API Versioning

Both `WebClient` and `RestClient` support making versioned remote HTTP calls so that APIs can be evolved over time. Commonly this involves sending an HTTP header, a query parameter or URL path segment that indicates the version of the API that should be used.

You can configure API versioning using methods on `WebClient.Builder` or `RestClient.Builder`.



API versioning is also supported on the server-side. See the [Spring MVC](#) and [Spring WebFlux](#) sections for details.



The server-side API versioning configuration is not taken into account to auto-configure the client. Clients that should use an API versioning strategy, typically for testing, need to configure it explicitly.

45.6. HTTP Service Interface Clients

Instead of directly using a `javado:org.springframework.web.client.RestClient[]` or `javado:org.springframework.web.reactive.function.client.WebClient[]` to call an HTTP service, it's also possible to call them using annotated Java interfaces.

HTTP Service interfaces defines a service contract by using methods that are annotated with `javado:org.springframework.web.service.annotation.HttpExchange[format=annotation]`, or more typically the method specific variants (`javado:org.springframework.web.service.annotation.GetExchange[format=annotation]`, `javado:org.springframework.web.service.annotation.PostExchange[format=annotation]`, `javado:org.springframework.web.service.annotation.DeleteExchange[format=annotation]`, etc).

For example, the following code defines an HTTP Service for an “echo” API that will return a JSON object containing an echo of the request.

```
include-code::EchoService[]
```

More details about how to develop HTTP Service interface clients can be found in the [{url-spring-framework-docs}/integration/rest-clients.html#rest-http-service-client](#) [Spring Framework reference documentation].

45.6.1. Importing HTTP Services

In order to use an HTTP Service interface as client you need to import it. One way to achieve this is to use the `javado:org.springframework.web.service.registry.ImportHttpServices[format=annotation]` annotation, typically on your main application class. You can use the annotation to import specific classes, or scan for classes to import from specific packages.

For example, the following configuration will scan for HTTP Service interfaces in the `com.example.myclients` package:

```
include-code::MyApplication[]
```

45.6.2. Service Client Groups

Hard-coding absolute URLs in `javado:org.springframework.web.service.annotation.HttpExchange[format=annotation]` annotations is often not ideal in production applications. Instead, you will typically want to give the HTTP Service client a logical name in your code, and then lookup a URL from a property based on that name.

HTTP Service clients allow you to do this by registering them into named groups. An HTTP Service group is a collection of HTTP Service interfaces that all share common features.

For example, we may want to define an “echo” group to use for HTTP Service clients that call `https://echo.zuplo.io`.



HTTP Service groups can be used to define more than just URLs. For example, your group could define connection timeouts and SSL settings. You can also associate client customization logic to a group, such as adding code to insert required authorization headers.

To associate an HTTP Service interface with a group when using `javado:org.springframework.web.service.registry.ImportHttpServices[format=annotation]` you can use the `group` attribute.

For example, if we assume our example above is organized in such a way that all HTTP Service interfaces in the `com.example.myclients` package belong to the `echo` group. We first remove the hardcoded URL from the service interface:

```
include-code::EchoService[]
```


We can then write:

```
include-code::MyApplication[]
```

And finally we can then use a `base-url` property to link the `echo` group to an actual URL:

```
spring:
  http:
    serviceclient:
      echo:
        base-url: "https://echo.zuplo.io"
```



HTTP Service clients will be associated with a group named “default” if you don’t specify a group.



If you have multiple HTTP Service interfaces in the same package that need to be associated with different groups you can list them individually. The `javax.javadoc.org.springframework.web.service.registry.ImportHttpServices[format=annotation]` is repeatable and the **types** attributes allows you to import individual classes.

For example:

```
include-code::repeat/MyApplication[]
```

45.6.3. Configuration Properties

Configuration properties for HTTP Services can be specified under `spring.http.serviceclient.<group-name>`:

You can use properties to configure aspects such as:

- The base URL.
- Any default headers that should be sent.
- API versioning configuration.
- Redirect settings.
- Connection and read timeouts.
- SSL bundles to use.



You can also use [global configuration properties](#) which apply to all HTTP clients.

For example, the properties below will:

- Configure all HTTP clients to use a one second connect timeout (unless otherwise overridden).
- Configure HTTP Service clients in the “echo” group to:
 - Use a specific base URL.
 - Have a two second read timeout.
 - Insert API version information using an `X-Version` header.

```
spring:
  http:
    clients:
      connect-timeout: 1s
    serviceclient:
      echo:
        base-url: "https://echo.zuplo.io"
        read-timeout: 2s;
    apiversion:
      default: 1.0.0
    insert:
      header: X-Version
```

45.6.4. Customization

If you need to customize HTTP Service clients beyond basic properties, you can use an HTTP Service group configurator. For `RestClient` backed HTTP Service clients, you can declare a bean that implements `javax.javadoc.org.springframework.web.client.support.RestClientHttpServiceGroupConfigurer[]`. For `WebClient` backed HTTP Service clients you can declare a bean that implements `javax.javadoc.org.springframework.web.reactive.function.client.support.WebClientHttpServiceGroupConfigurer[]`.

Both work in the same way and will be automatically applied by Spring Boot’s auto-configuration.

For example, the following configuration would add a group customizer that adds an HTTP header to each outgoing request containing the group name:

```
include-code::MyHttpServiceGroupConfiguration[]
```

45.6.5. Advanced Configuration

As well as the `javax.annotation.processing.ImportHttpServices` annotation, Spring Framework also offers an `javax.annotation.processing.ImportHttpServiceRegistrar` class. You can `javax.annotation.processing.Import` your own extension of this class to perform programmatic configuration. For more details, see [{url-spring-framework-docs}/integration/rest-clients.html#rest-http-service-client-group-config](#) [Spring Framework reference documentation].

Regardless of which method you use to register HTTP Service clients, Spring Boot's support remains the same.

45.7. Applying Global Configuration to All HTTP Clients

Regardless of the underlying technology being used, all HTTP clients have common settings that can be configured.

These include:

- Connection Timeouts.
- Read Timeouts.
- How HTTP redirects should be handled.
- Which SSL bundle should be used when connecting.

These common settings are represented by the `org.springframework.boot.http.client.HttpClientSettings` class which can be passed into the `build(...)` methods of `org.springframework.boot.http.client.reactive.ClientHttpConnectorBuilder` and `org.springframework.boot.http.client.ClientHttpRequestFactoryBuilder`.

If you want to apply the same configuration to all auto-configured clients, you can use `spring.http.clients` properties to do so:

```
spring:
  http:
    clients:
      connect-timeout: 2s
      read-timeout: 1s
      redirects: dont-follow
```

46. Spring Batch

Spring Boot offers several conveniences for working with [Spring Batch](#), including running a Job on startup.

When building a batch application, the following stores can be auto-configured:

- In-memory
- JDBC

Each store has specific additional settings. For instance, it is possible to customize the tables prefix for the JDBC store, as shown in the following example:

```
spring:
  batch:
    jdbc:
      table-prefix: "CUSTOM_"
```

To disable Spring Boot's auto-configuration and take complete control of Spring Batch's configuration, add `javado:org.springframework.batch.core.configuration.annotation.EnableBatchProcessing[format=annotation]` to one of your `javado:org.springframework.context.annotation.Configuration[format=annotation]` classes or extend `javado:org.springframework.batch.core.configuration.support.DefaultBatchConfiguration[]`. This will cause the auto-configuration to back off, including initialization of Spring Batch's database schema if you're using the JDBC-based store. Spring Batch can then be configured using the `@Enable*JobRepository` annotation's attributes rather than the previously described configuration properties.

To learn more about manually configuring Spring Batch, see the API documentation of:

- `javado:org.springframework.batch.core.configuration.support.DefaultBatchConfiguration[]`
- `javado:org.springframework.batch.core.configuration.annotation.EnableBatchProcessing[format=annotation]`
- `javado:org.springframework.batch.core.configuration.annotation.EnableJdbcJobRepository[format=annotation]`
- `javado:org.springframework.batch.core.configuration.annotation.EnableMongoJobRepository[format=annotation]`

For more information about Spring Batch, see the [Spring Batch project page](#).

46.1. Running Spring Batch Jobs on Startup

When Spring Boot auto-configures Spring Batch, and if a single `javado:org.springframework.batch.core.job.Job[]` bean is found in the application context, it is executed on startup (see `javado:org.springframework.boot.batch.autoconfigure.JobLauncherApplicationRunner[]` for details). If multiple `javado:org.springframework.batch.core.job.Job[]` beans are found, the job that should be executed must be specified using `configprop:spring.batch.job.name[]`.

You can disable running a `javado:org.springframework.batch.core.job.Job[]` found in the application context, as shown in the following example:

```
spring:
  batch:
    job:
      enabled: false
```

See `{code-spring-boot-batch-src}/autoconfigure/BatchAutoConfiguration.java[ BatchAutoConfiguration ]` and `{code-spring-boot-batch-jdbc-src}/autoconfigure/BatchJdbcAutoConfiguration.java[ BatchJdbcAutoConfiguration ]` for more details.

47. Validation

The method validation feature supported by Bean Validation 1.1 is automatically enabled as long as a JSR-303 implementation (such as Hibernate validator) is on the classpath. This lets bean methods be annotated with `jakarta.validation` constraints on their parameters and/or on their return value. Target classes with such annotated methods need to be annotated with the `javax.validation.annotation.Validated` annotation at the type level for their methods to be searched for inline constraint annotations.

For instance, the following service triggers the validation of the first argument, making sure its size is between 8 and 10:

```
include-code::MyBean[]
```

The application's `javax.validation.MessageSource` is used when resolving `{parameters}` in constraint messages. This allows you to use [your application's messages.properties files](#) for Bean Validation messages. Once the parameters have been resolved, message interpolation is completed using Bean Validation's default interpolator.

To customize the `javax.validation.Configuration` used to build the `javax.validation.ValidatorFactory`, define a `javax.validation.bootstrap.autoconfigure.ValidationConfigurationCustomizer` bean. When multiple customizer beans are defined, they are called in order based on their `javax.validation.annotation.Order` annotation or `javax.validation.core.Ordered` implementation.

48. Web Services

Spring Boot provides Web Services auto-configuration so that all you must do is define your `javax.xml.ws.Endpoint` beans.

The `javax.xml.ws.Endpoint` beans can be easily accessed with the `spring-boot-starter-webservices` module.

`javax.xml.ws.Endpoint` and `javax.xml.ws.Endpoint` beans can be automatically created for your WSDLs and XSDs respectively. To do so, configure their location, as shown in the following example:

```
spring:
  webservices:
    wsdl-locations: "classpath:/wsdl"
```

48.1. Calling Web Services with `WebServiceTemplate`

If you need to call remote Web services from your application, you can use the `javax.xml.ws.WebServiceTemplate` class. Since `javax.xml.ws.WebServiceTemplate` instances often need to be customized before being used, Spring Boot does not provide any single auto-configured `javax.xml.ws.WebServiceTemplate` bean. It does, however, auto-configure a `javax.xml.ws.WebServiceTemplateBuilder`, which can be used to create `javax.xml.ws.WebServiceTemplate` instances when needed.

The following code shows a typical example:

```
include-code::MyService[]
```

By default, `javax.xml.ws.WebServiceTemplateBuilder` detects a suitable HTTP-based `javax.xml.ws.WebServiceMessageSender` using the available HTTP client libraries on the classpath. You can also customize read and connection timeouts for an individual builder as follows:

```
include-code::MyWebServiceTemplateConfiguration[]
```



You can also change the [global HTTP client configuration](#) used if not specific template customization code is applied.

49. AMQP

The Advanced Message Queuing Protocol (AMQP) is a platform-neutral, wire-level protocol for message-oriented middleware. The Spring AMQP project applies core Spring concepts to the development of AMQP-based messaging solutions. Spring Boot offers several conveniences for working with AMQP through RabbitMQ, including the `spring-boot-starter-amqp` starter.

49.1. RabbitMQ Support

RabbitMQ (<https://www.rabbitmq.com/>) is a lightweight, reliable, scalable, and portable message broker based on the AMQP protocol. Spring uses RabbitMQ to communicate through the AMQP protocol.

RabbitMQ configuration is controlled by external configuration properties in `spring.rabbitmq.*`. For example, you might declare the following section in `application.properties`:

```
spring:
  rabbitmq:
    host: "localhost"
    port: 5672
    username: "admin"
    password: "secret"
```

Alternatively, you could configure the same connection using the `addresses` attribute:

```
spring:
  rabbitmq:
    addresses: "amqp://admin:secret@localhost"
```



When specifying addresses that way, the `host` and `port` properties are ignored. If the address uses the `amqps` protocol, SSL support is enabled automatically.

See `javadoc:org.springframework.boot.amqp.autoconfigure.RabbitProperties[]` for more of the supported property-based configuration options. To configure lower-level details of the RabbitMQ `javadoc:com.rabbitmq.client.ConnectionFactory[]` that is used by Spring AMQP, define a `javadoc:org.springframework.boot.amqp.autoconfigure.ConnectionFactoryCustomizer[]` bean.

If a `javadoc:org.springframework.amqp.rabbit.connection.ConnectionNameStrategy[]` bean exists in the context, it will be automatically used to name connections created by the auto-configured `javadoc:org.springframework.amqp.rabbit.connection.CachingConnectionFactory[]`.

To make an application-wide, additive customization to the `javadoc:org.springframework.amqp.rabbit.core.RabbitTemplate[]`, use a `javadoc:org.springframework.boot.amqp.autoconfigure.RabbitTemplateCustomizer[]` bean.



See [Understanding AMQP, the protocol used by RabbitMQ](https://spring.io/blog/2010/06/14/understanding-amqp-the-protocol-used-by-rabbitmq/) (<https://spring.io/blog/2010/06/14/understanding-amqp-the-protocol-used-by-rabbitmq/>) for more details.

49.2. Sending a Message

Spring's `javadoc:org.springframework.amqp.core.AmqpTemplate[]` and `javadoc:org.springframework.amqp.core.AmqpAdmin[]` are auto-configured, and you can autowire them directly into your own beans, as shown in the following example:

include-code::MyBean[]



`javadoc:org.springframework.amqp.rabbit.core.RabbitMessagingTemplate[]` can be injected in a similar manner. If a `javadoc:org.springframework.amqp.support.converter.MessageConverter[]` bean is defined, it is associated automatically to the auto-configured `javadoc:org.springframework.amqp.core.AmqpTemplate[]`.

If necessary, any `javadoc:org.springframework.amqp.core.Queue[]` that is defined as a bean is automatically used to declare a corresponding queue on the RabbitMQ instance.

To retry operations, you can enable retries on the `javadoc:org.springframework.amqp.core.AmqpTemplate[]` (for example, in the event that the broker connection is lost):

```
spring:
  rabbitmq:
    template:
      retry:
        enabled: true
        initial-interval: "2s"
```

Retries are disabled by default. You can also customize the `javadoc:org.springframework.core.retry.RetryTemplate[]` programmatically by declaring a `javadoc:org.springframework.boot.amqp.autoconfigure.RabbitTemplateRetrySettingsCustomizer[]` bean.

If you need to create more `javadoc:org.springframework.amqp.rabbit.core.RabbitTemplate[]` instances or if you want to override the default, Spring Boot provides a `javadoc:org.springframework.boot.amqp.autoconfigure.RabbitTemplateConfigurer[]` bean that you can use to initialize a `javadoc:org.springframework.amqp.rabbit.core.RabbitTemplate[]` with the same settings as the factories used by the auto-configuration.

If there's a bean of type `javadoc:org.springframework.amqp.rabbit.support.micrometer.RabbitTemplateObservationConvention[]` in the context, it will automatically be configured on the `javadoc:org.springframework.amqp.rabbit.core.RabbitTemplate[]`.

49.3. Sending a Message To A Stream

To send a message to a particular stream, specify the name of the stream, as shown in the following example:

```
spring:
  rabbitmq:
    stream:
      name: "my-stream"
```

If a `javadoc:org.springframework.amqp.support.converter.MessageConverter[]`, `javadoc:org.springframework.rabbit.stream.support.converter.StreamMessageConverter[]`, `javadoc:org.springframework.rabbit.stream.producer.ProducerCustomizer[]` or `javadoc:org.springframework.rabbit.stream.micrometer.RabbitStreamTemplateObservationConvention[]` bean is defined, it is associated automatically to the auto-configured `javadoc:org.springframework.rabbit.stream.producer.RabbitStreamTemplate[]`.

If you need to create more `javadoc:org.springframework.rabbit.stream.producer.RabbitStreamTemplate[]` instances or if you want to override the default, Spring Boot provides a `javadoc:org.springframework.boot.amqp.autoconfigure.RabbitStreamTemplateConfigurer[]` bean that you can use to initialize a `javadoc:org.springframework.rabbit.stream.producer.RabbitStreamTemplate[]` with the same settings as the factories used by the auto-configuration.

49.4. Receiving a Message

When the Rabbit infrastructure is present, any bean can be annotated with `javadoc:org.springframework.amqp.rabbit.annotation.RabbitListener[format=annotation]` to create a listener endpoint. If no `javadoc:org.springframework.amqp.rabbit.listener.RabbitListenerContainerFactory[]` has been defined, a default `javadoc:org.springframework.amqp.rabbit.config.SimpleRabbitListenerContainerFactory[]` is automatically configured and you can switch to a direct container using the `configprop:spring.rabbitmq.listener.type[]` property. If a `javadoc:org.springframework.amqp.support.converter.MessageConverter[]`, a `javadoc:org.springframework.amqp.rabbit.retry.MessageRecoverer[]` or a `javadoc:org.springframework.amqp.rabbit.support.micrometer.RabbitListenerObservationConvention[]` bean is defined, it is automatically associated with the default factory.

The following sample component creates a listener endpoint on the `someQueue` queue:

include-code::MyBean[]



See `javadoc:org.springframework.amqp.rabbit.annotation.EnableRabbit[format=annotation]` for more details.

If you need to create more `javadoc:org.springframework.amqp.rabbit.listener.RabbitListenerContainerFactory[]` instances or if you want to override the default, Spring Boot provides a `javadoc:org.springframework.boot.amqp.autoconfigure.SimpleRabbitListenerContainerFactoryConfigurer[]` and a `javadoc:org.springframework.boot.amqp.autoconfigure.DirectRabbitListenerContainerFactoryConfigurer[]` that you can use to initialize a `javadoc:org.springframework.amqp.rabbit.config.SimpleRabbitListenerContainerFactory[]` and a `javadoc:org.springframework.amqp.rabbit.config.DirectRabbitListenerContainerFactory[]` with the same settings as the factories used by the auto-configuration.



It does not matter which container type you chose. Those two beans are exposed by the auto-configuration.

For instance, the following configuration class exposes another factory that uses a specific `javadoc:org.springframework.amqp.support.converter.MessageConverter[]`:

include-code::custom/MyRabbitConfiguration[]

Then you can use the factory in any `javadoc:org.springframework.amqp.rabbit.annotation.RabbitListener[format=annotation]`-annotated method, as follows:

include-code::custom/MyBean[]

You can enable retries to handle situations where your listener throws an exception. By default, `javadoc:org.springframework.amqp.rabbit.retry.RejectAndDontRequeueRecoverer[]` is used, but you can define a `javadoc:org.springframework.amqp.rabbit.retry.MessageRecoverer[]` of your own. When retries are exhausted, the message is rejected and either dropped or routed to a dead-letter exchange if the broker is configured to do so. By default, retries are disabled. You can also customize the `javadoc:org.springframework.core.retry.RetryPolicy[]` programmatically by declaring a `javadoc:org.springframework.boot.amqp.autoconfigure.RabbitListenerRetrySettingsCustomizer[]` bean.



By default, if retries are disabled and the listener throws an exception, the delivery is retried indefinitely. You can modify this behavior in two ways: Set the **defaultRequeueRejected** property to **false** so that zero re-deliveries are attempted or throw an `javadoc:org.springframework.amqp.AmqpRejectAndDontRequeueException[]` to signal the message should be rejected. The latter is the mechanism used when retries are enabled and the maximum number of delivery attempts is reached.

50. Messaging

The Spring Framework provides extensive support for integrating with messaging systems, from simplified use of the JMS API using `javax.jms.core.JmsClient[]` to a complete infrastructure to receive messages asynchronously. Spring AMQP provides a similar feature set for the Advanced Message Queuing Protocol. Spring Boot also provides auto-configuration options for `javax.jms.core.JmsClient[]` and RabbitMQ. Spring WebSocket natively includes support for STOMP messaging, and Spring Boot has support for that through starters and a small amount of auto-configuration. Spring Boot also has support for Apache Kafka and Apache Pulsar.

51.JMS

The `javax.jms.ConnectionFactory` interface provides a standard method of creating a `javax.jms.Connection` for interacting with a JMS broker. Although Spring needs a `javax.jms.ConnectionFactory` to work with JMS, you generally need not use it directly yourself and can instead rely on higher level messaging abstractions. (See the [{url-spring-framework-docs}/integration/jms.html](#) [relevant section] of the Spring Framework reference documentation for details.) Spring Boot also auto-configures the necessary infrastructure to send and receive messages.

51.1. ActiveMQ "Classic" Support

When **ActiveMQ "Classic"** (<https://activemq.apache.org/components/classic>) is available on the classpath, Spring Boot can configure a `javax.jms.ConnectionFactory`. If the broker is present, an embedded broker is automatically started and configured (provided no broker URL is specified through configuration and the embedded broker is not disabled in the configuration).



If you use `spring-boot-starter-activemq`, the necessary dependencies to connect to an ActiveMQ "Classic" instance are provided, as is the Spring infrastructure to integrate with JMS. Adding `org.apache.activemq:activemq-broker` to your application lets you use the embedded broker.

ActiveMQ "Classic" configuration is controlled by external configuration properties in `spring.activemq.*`.

If `activemq-broker` is on the classpath, ActiveMQ "Classic" is auto-configured to use the **VM transport** (<https://activemq.apache.org/vm-transport-reference.html>), which starts a broker embedded in the same JVM instance.

You can disable the embedded broker by configuring the `spring.activemq.embedded.enabled` property, as shown in the following example:

```
spring:
  activemq:
    embedded:
      enabled: false
```

The embedded broker will also be disabled if you configure the broker URL, as shown in the following example:

```
spring:
  activemq:
    broker-url: "tcp://192.168.1.210:9876"
    user: "admin"
    password: "secret"
```

If you want to take full control over the embedded broker, see [the ActiveMQ "Classic" documentation](https://activemq.apache.org/how-do-i-embed-a-broker-inside-a-connection.html) (<https://activemq.apache.org/how-do-i-embed-a-broker-inside-a-connection.html>) for further information.

By default, a `org.springframework.jms.connection.CachingConnectionFactory` wraps the native `javax.jms.ConnectionFactory` with sensible settings that you can control by external configuration properties in `spring.jms.*`:

```
spring:
  jms:
    cache:
      session-cache-size: 5
```

If you'd rather use native pooling, you can do so by adding a dependency to `org.messaginghub:pooled-jms` and configuring the `javax.messaginghub.pooled.jms.JmsPoolConnectionFactory` accordingly, as shown in the following example:

```
spring:
  activemq:
    pool:
      enabled: true
      max-connections: 50
```



See `org.springframework.boot.activemq.autoconfigure.ActiveMQProperties` for more of the supported options. You can also register an arbitrary number of beans that implement `org.springframework.boot.activemq.autoconfigure.ActiveMQConnectionFactoryCustomizer` for more advanced customizations.

By default, ActiveMQ "Classic" creates a destination if it does not yet exist so that destinations are resolved against their provided names.

51.2. ActiveMQ Artemis Support

Spring Boot can auto-configure a `javax.jms.ConnectionFactory` when it detects that **ActiveMQ Artemis** (<https://activemq.apache.org/components/artemis/>) is available on the classpath. If the broker is present, an embedded broker is automatically started and configured (unless the mode property has been explicitly set). The supported modes are `embedded` (to make explicit that an embedded broker is required and that an error should occur if the broker is not available on the classpath) and `native` (to connect to a broker using the `netty` transport protocol). When the latter is configured, Spring Boot configures a `javax.jms.ConnectionFactory` that connects to a broker running on the local machine with the default settings.



If you use `spring-boot-starter-artemis`, the necessary dependencies to connect to an existing ActiveMQ Artemis instance are provided, as well as the Spring infrastructure to integrate with JMS. Adding `org.apache.activemq:artemis-jakarta-server` to your application lets you use embedded mode.

ActiveMQ Artemis configuration is controlled by external configuration properties in `spring.artemis.*`. For example, you might declare the following section in `application.properties`:


```
spring:
  artemis:
    mode: native
    broker-url: "tcp://192.168.1.210:9876"
    user: "admin"
    password: "secret"
```

When embedding the broker, you can choose if you want to enable persistence and list the destinations that should be made available. These can be specified as a comma-separated list to create them with the default options, or you can define bean(s) of type `javax.jms.QueueConfiguration` or `javax.jms.TopicConfiguration`, for advanced queue and topic configurations, respectively.

By default, a `javax.jms.ConnectionFactory` wraps the native `javax.jms.ConnectionFactory` with sensible settings that you can control by external configuration properties in `spring.jms.*`:

```
spring:
  jms:
    cache:
      session-cache-size: 5
```

If you'd rather use native pooling, you can do so by adding a dependency on `org.messaginghub.pooled-jms` and configuring the `javax.jms.PoolConnectionFactory` accordingly, as shown in the following example:

```
spring:
  artemis:
    pool:
      enabled: true
      max-connections: 50
```

See `javax.jms.SpringArtemisAutoConfigure.ArtemisProperties` for more supported options.

No JNDI lookup is involved, and destinations are resolved against their names, using either the `name` attribute in the ActiveMQ Artemis configuration or the names provided through configuration.

51.3. Using a JNDI ConnectionFactory

If you are running your application in an application server, Spring Boot tries to locate a `javax.jms.ConnectionFactory` by using JNDI. By default, the `java:/JmsXA` and `java:/XAConnectionFactory` location are checked. You can use the `configprop:spring.jms.jndi-name` property if you need to specify an alternative location, as shown in the following example:

```
spring:
  jms:
    jndi-name: "java:/MyConnectionFactory"
```

51.4. Sending a Message

Spring's `javax.jms.core.JmsClient` is auto-configured, and you can autowire it directly into your own beans, as shown in the following example:

include-code::MyBean[]



`javax.jms.core.JmsMessagingTemplate` can be injected in a similar manner, and both use the traditional `javax.jms.core.JmsTemplate` that can be injected as well. If a `javax.jms.support.destination.DestinationResolver` or a `javax.jms.support.converter.MessageConverter` bean is defined, it is associated automatically to the auto-configured `javax.jms.core.JmsTemplate`.

51.5. Receiving a Message

When the JMS infrastructure is present, any bean can be annotated with `javax.jms.annotation.JmsListener[format=annotation]` to create a listener endpoint. If no `javax.jms.config.JmsListenerContainerFactory` has been defined, a default one is configured automatically. If a `javax.jms.support.destination.DestinationResolver`, a `javax.jms.support.converter.MessageConverter`, or a `javax.jms.ExceptionListener` beans are defined, they are associated automatically with the default factory.

In most scenarios, message listener containers should be configured against the native `javax.jms.ConnectionFactory`. This way each listener container has its own connection and this gives full responsibility to it in terms of local recovery. The auto-configuration uses `javax.jms.ConnectionFactoryUnwrapper` to unwrap the native connection factory from the auto-configured one.



The auto-configuration only unwraps `CachedConnectionFactory`.

By default, the default factory is transactional. If you run in an infrastructure where a `javax.jta.TransactionManager` is present, it is associated to the listener container by default. If not, the `sessionTransacted` flag is enabled. In that latter scenario, you can associate your local data store transaction to the processing of an incoming message by adding `javax.jta.annotation.Transactional[format=annotation]` on your listener method (or a delegate thereof). This ensures that the incoming message is acknowledged, once the local transaction has completed. This also includes sending response messages that have been performed on the same JMS session.

The following component creates a listener endpoint on the `someQueue` destination:

include-code::MyBean[]



See the `javax.jms.annotation.EnableJms` API documentation for more details.

If you need to create more `javax.jms.config.JmsListenerContainerFactory` instances or if you want to override the default, Spring Boot provides a `javax.jms.config.DefaultJmsListenerContainerFactoryConfigurer` that you can use to initialize a `javax.jms.config.DefaultJmsListenerContainerFactory` with the same settings as the one that is auto-configured.

For instance, the following example exposes another factory that uses a specific `javax.jms.support.converter.MessageConverter`:

include-code::custom/MyJmsConfiguration[]



In the example above, the customization uses `javax.jms.ConnectionFactoryUnwrapper` to associate the native connection factory to the message listener container the same way the auto-configured factory does.

Then you can use the factory in any `javax.jms.annotation.JmsListener`-annotated method as follows:

include-code::custom/MyBean[]

Analogous to `DefaultJmsListenerContainerFactoryConfigurer`, Spring Boot also provides a `javax.jms.config.SimpleJmsListenerContainerFactoryConfigurer` that you can use to initialize a `javax.jms.config.SimpleJmsListenerContainerFactory` and apply the related settings that auto-configuration provides.



In contrast to `javax.jms.listener.DefaultMessageListenerContainer` that uses a pull-based mechanism (polling) to process messages, `javax.jms.listener.SimpleMessageListenerContainer` uses a push-based mechanism that's very close to the spirit of the standalone JMS specification. To learn more about the differences between the two listener containers, consult their respective javadocs and [{url-spring-framework-docs}/integration/jms/using.html#jms-mdp](#) [Spring Framework reference documentation].

52. Apache Kafka Support

[Apache Kafka](https://kafka.apache.org/) (<https://kafka.apache.org/>) is supported by providing auto-configuration of the `spring-kafka` project.

Kafka configuration is controlled by external configuration properties in `spring.kafka.*`. For example, you might declare the following section in `application.properties`:

```
spring:
  kafka:
    bootstrap-servers: "localhost:9092"
    consumer:
      group-id: "myGroup"
```



To create a topic on startup, add a bean of type `javadoc:org.apache.kafka.clients.admin.NewTopic[]`. If the topic already exists, the bean is ignored.

See `javadoc:org.springframework.boot.kafka.autoconfigure.KafkaProperties[]` for more supported options.

52.1. Sending a Message

Spring's `javadoc:org.springframework.kafka.core.KafkaTemplate[]` is auto-configured, and you can autowire it directly in your own beans, as shown in the following example:

include-code::MyBean[]



If the property `configprop:spring.kafka.producer.transaction-id-prefix[]` is defined, a `javadoc:org.springframework.kafka.transaction.KafkaTransactionManager[]` is automatically configured. Also, if a `javadoc:org.springframework.kafka.support.converter.RecordMessageConverter[]` bean is defined, it is automatically associated to the auto-configured `javadoc:org.springframework.kafka.core.KafkaTemplate[]`.

If there's a bean of type `KafkaTemplateObservationConvention` in the context, it is automatically registered on the `KafkaTemplate`.

52.2. Receiving a Message

When the Apache Kafka infrastructure is present, any bean can be annotated with `javadoc:org.springframework.kafka.annotation.KafkaListener[format=annotation]` to create a listener endpoint. If no `javadoc:org.springframework.kafka.config.KafkaListenerContainerFactory[]` has been defined, a default one is automatically configured with keys defined in `spring.kafka.listener.*`.

The following component creates a listener endpoint on the `someTopic` topic:

include-code::MyBean[]

If a `javadoc:org.springframework.kafka.transaction.KafkaTransactionManager[]` bean is defined, it is automatically associated to the container factory. Similarly, if a `javadoc:org.springframework.kafka.listener.adapter.RecordFilterStrategy[]`, `javadoc:org.springframework.kafka.listener.CommonErrorHandler[]`, `javadoc:org.springframework.kafka.listener.AfterRollbackProcessor[]` or `javadoc:org.springframework.kafka.listener.ConsumerAwareRebalanceListener[]` bean is defined, it is automatically associated to the default factory.

Depending on the listener type, a `javadoc:org.springframework.kafka.support.converter.RecordMessageConverter[]` or `javadoc:org.springframework.kafka.support.converter.BatchMessageConverter[]` bean is associated to the default factory. If only a `javadoc:org.springframework.kafka.support.converter.RecordMessageConverter[]` bean is present for a batch listener, it is wrapped in a `javadoc:org.springframework.kafka.support.converter.BatchMessageConverter[]`.



A custom `javadoc:org.springframework.kafka.transaction.ChainedKafkaTransactionManager[]` must be marked `javadoc:org.springframework.context.annotation.Primary[format=annotation]` as it usually references the auto-configured `javadoc:org.springframework.kafka.transaction.KafkaTransactionManager[]` bean.

If there's a bean of type `KafkaListenerObservationConvention` in the context, it is automatically registered on the container factory.

52.3. Kafka Streams

Spring for Apache Kafka provides a factory bean to create a `javadoc:org.apache.kafka.streams.StreamsBuilder[]` object and manage the lifecycle of its streams. Spring Boot auto-configures the required `javadoc:org.springframework.kafka.config.KafkaStreamsConfiguration[]` bean as long as `kafka-streams` is on the classpath and Kafka Streams is enabled by the `javadoc:org.springframework.kafka.annotation.EnableKafkaStreams[format=annotation]` annotation.

Enabling Kafka Streams means that the application id and bootstrap servers must be set. The former can be configured using `spring.kafka.streams.application-id`, defaulting to `spring.application.name` if not set. The latter can be set globally or specifically overridden only for streams.

Several additional properties are available using dedicated properties; other arbitrary Kafka properties can be set using the `spring.kafka.streams.properties` namespace. See also [messaging/kafka.html](#) for more information.

To use the factory bean, wire `javadoc:org.apache.kafka.streams.StreamsBuilder[]` into your `javadoc:org.springframework.context.annotation.Bean[format=annotation]` as shown in the following example:

include-code::MyKafkaStreamsConfiguration[]

By default, the streams managed by the `javadoc:org.apache.kafka.streams.StreamsBuilder[]` object are started automatically. You can customize this behavior using the `configprop:spring.kafka.streams.auto-startup[]` property.



You can also register an arbitrary number of beans that implement `javadoc:org.springframework.kafka.config.StreamsBuilderFactoryBeanConfigurer[]` for more advanced customizations.

52.4. Additional Kafka Properties

The properties supported by auto configuration are shown in the [Integration Properties](#) section of the Appendix. Note that, for the most part, these properties (hyphenated or camelCase) map directly to the Apache Kafka dotted properties. See the Apache Kafka documentation for details.

Properties that don't include a client type (`producer` , `consumer` , `admin` , or `streams`) in their name are considered to be common and apply to all clients. Most of these common properties can be overridden for one or more of the client types, if needed.

Apache Kafka designates properties with an importance of HIGH, MEDIUM, or LOW. Spring Boot auto-configuration supports all HIGH importance properties, some selected MEDIUM and LOW properties, and any properties that do not have a default value.

Only a subset of the properties supported by Kafka are available directly through the `javadoc:org.springframework.boot.kafka.autoconfigure.KafkaProperties[]` class. If you wish to configure the individual client types with additional properties that are not directly supported, use the following properties:

```
spring:
  kafka:
    properties:
      "[prop.one]": "first"
    admin:
      properties:
        "[prop.two]": "second"
    consumer:
      properties:
        "[prop.three]": "third"
    producer:
      properties:
        "[prop.four]": "fourth"
    streams:
      properties:
        "[prop.five]": "fifth"
```

This sets the common `prop.one` Kafka property to `first` (applies to producers, consumers, admins, and streams), the `prop.two` admin property to `second` , the `prop.three` consumer property to `third` , the `prop.four` producer property to `fourth` and the `prop.five` streams property to `fifth` .

You can also configure the Spring Kafka `javadoc:org.springframework.kafka.support.serializer.JsonDeserializer[]` as follows:

```
spring:
  kafka:
    consumer:
      value-deserializer: "org.springframework.kafka.support.serializer.JsonDeserializer"
      properties:
        "[spring.json.value.default.type]": "com.example.Invoice"
        "[spring.json.trusted.packages]": "com.example.main,com.example.another"
```

Similarly, you can disable the `javadoc:org.springframework.kafka.support.serializer.JsonSerializer[]` default behavior of sending type information in headers:

```
spring:
  kafka:
    producer:
      value-serializer: "org.springframework.kafka.support.serializer.JsonSerializer"
      properties:
        "[spring.json.add.type.headers]": false
```



Properties set in this way override any configuration item that Spring Boot explicitly supports.

52.5. Testing with Embedded Kafka

Spring for Apache Kafka provides a convenient way to test projects with an embedded Apache Kafka broker. To use this feature, annotate a test class with `javadoc:org.springframework.kafka.test.context.EmbeddedKafka[format=annotation]` from the `spring-kafka-test` module. For more information, please see the [Spring for Apache Kafka {url-spring-kafka-docs}/testing.html#ekb\[reference manual\]](#).

To make Spring Boot auto-configuration work with the aforementioned embedded Apache Kafka broker, you need to remap a system property for embedded broker addresses (populated by the `javadoc:org.springframework.kafka.test.EmbeddedKafkaBroker[]`) into the Spring Boot configuration property for Apache Kafka. There are several ways to do that:

- Provide a system property to map embedded broker addresses into `configprop:spring.kafka.bootstrap-servers[]` in the test class:

```
include-code::property/MyTest[tag=*]
```

- Configure a property name on the `javadoc:org.springframework.kafka.test.context.EmbeddedKafka[format=annotation]` annotation:

```
include-code::annotation/MyTest[]
```

- Use a placeholder in configuration properties:

```
spring:
  kafka:
    bootstrap-servers: "${spring.embedded.kafka.brokers}"
```

53. Apache Pulsar Support

[Apache Pulsar](https://pulsar.apache.org/) (https://pulsar.apache.org/) is supported by providing auto-configuration of the {url-spring-pulsar-site}[Spring for Apache Pulsar] project.

Spring Boot will auto-configure and register the Spring for Apache Pulsar components when `org.springframework.pulsar:spring-pulsar` is on the classpath.

There is the `spring-boot-starter-pulsar` starter for conveniently collecting the dependencies for use.

53.1. Connecting to Pulsar

When you use the Pulsar starter, Spring Boot will auto-configure and register a `javadoc:org.apache.pulsar.client.api.PulsarClient[]` bean.

By default, the application tries to connect to a local Pulsar instance at `pulsar://localhost:6650`. This can be adjusted by setting the `configprop:spring.pulsar.client.service-url[]` property to a different value.



The value must be a valid [Pulsar Protocol](https://pulsar.apache.org/docs/client-libraries-java/#connection-urls) (https://pulsar.apache.org/docs/client-libraries-java/#connection-urls) URL

You can configure the client by specifying any of the `spring.pulsar.client.*` prefixed application properties.

If you need more control over the configuration, consider registering one or more `javadoc:org.springframework.pulsar.core.PulsarClientBuilderCustomizer[]` beans.

53.1.1. Authentication

To connect to a Pulsar cluster that requires authentication, you need to specify which authentication plugin to use by setting the `pluginClassName` and any parameters required by the plugin. You can set the parameters as a map of parameter names to parameter values. The following example shows how to configure the `AuthenticationOAuth2` plugin.

```
spring:
  pulsar:
    client:
      authentication:
        plugin-class-name: org.apache.pulsar.client.impl.auth.oauth2.AuthenticationOAuth2
        param:
          issuerUrl: https://auth.server.cloud/
          privateKey: file:///Users/some-key.json
          audience: urn:sn:acme:dev:my-instance
```

You need to ensure that names defined under `spring.pulsar.client.authentication.param.*` exactly match those expected by your auth plugin (which is typically camel cased). Spring Boot will not attempt any kind of relaxed binding for these entries.



For example, if you want to configure the issuer url for the `AuthenticationOAuth2` auth plugin you must use `spring.pulsar.client.authentication.param.issuerUrl`. If you use other forms, such as `issuerurl` or `issuer-url`, the setting will not be applied to the plugin.

This lack of relaxed binding also makes using environment variables for authentication parameters problematic because the case sensitivity is lost during translation. If you use environment variables for the parameters then you will need to follow {url-spring-pulsar-docs}/reference/pulsar/pulsar-client.html#client-authentication-env-vars[these steps] in the Spring for Apache Pulsar reference documentation for it to work properly.

53.1.2. SSL

By default, Pulsar clients communicate with Pulsar services in plain text. You can follow {url-spring-pulsar-docs}/reference/pulsar/pulsar-client.html#tls-encryption[these steps] in the Spring for Apache Pulsar reference documentation to enable TLS encryption.

For complete details on the client and authentication see the Spring for Apache Pulsar {url-spring-pulsar-docs}/reference/pulsar/pulsar-client.html[reference documentation].

53.2. Connecting to Pulsar Administration

Spring for Apache Pulsar's `javadoc:org.springframework.pulsar.core.PulsarAdministration[]` client is also auto-configured.

By default, the application tries to connect to a local Pulsar instance at `http://localhost:8080`. This can be adjusted by setting the `configprop:spring.pulsar.admin.service-url[]` property to a different value in the form `(http|https)://<host>:<port>`.

If you need more control over the configuration, consider registering one or more `javadoc:org.springframework.pulsar.core.PulsarAdminBuilderCustomizer[]` beans.

53.2.1. Authentication

When accessing a Pulsar cluster that requires authentication, the admin client requires the same security configuration as the regular Pulsar client. You can use the aforementioned [authentication configuration](#) by replacing `spring.pulsar.client.authentication` with `spring.pulsar.admin.authentication`.



To create a topic on startup, add a bean of type `javadoc:org.springframework.pulsar.core.PulsarTopic[]`. If the topic already exists, the bean is ignored.

53.3. Sending a Message

Spring's `javadoc:org.springframework.pulsar.core.PulsarTemplate[]` is auto-configured, and you can use it to send messages, as shown in the following example:

```
include-code::MyBean[]
```

The `javadoc:org.springframework.pulsar.core.PulsarTemplate[]` relies on a `javadoc:org.springframework.pulsar.core.PulsarProducerFactory[]` to create the underlying Pulsar producer. Spring Boot auto-configuration also provides this producer factory, which by default, caches the producers that it creates. You can configure the producer factory and cache settings by specifying any of the `spring.pulsar.producer.*` and `spring.pulsar.producer.cache.*` prefixed application properties.

If you need more control over the producer factory configuration, consider registering one or more `javadoc:org.springframework.pulsar.core.ProducerBuilderCustomizer[]` beans. These customizers are applied to all created producers. You can also pass in a `javadoc:org.springframework.pulsar.core.ProducerBuilderCustomizer[]` when sending a message to only affect the current producer.

If you need more control over the message being sent, you can pass in a `javadoc:org.springframework.pulsar.core.TypedMessageBuilderCustomizer[]` when sending a message.

53.4. Receiving a Message

When the Apache Pulsar infrastructure is present, any bean can be annotated with `javadoc:org.springframework.pulsar.annotation.PulsarListener[format=annotation]` to create a listener endpoint. The following component creates a listener endpoint on the `someTopic` topic:

```
include-code::MyBean[]
```

Spring Boot auto-configuration provides all the components necessary for `javadoc:org.springframework.pulsar.annotation.PulsarListener[]`, such as the `javadoc:org.springframework.pulsar.config.PulsarListenerContainerFactory[]` and the consumer factory it uses to construct the underlying Pulsar consumers. You can configure these components by specifying any of the `spring.pulsar.listener.*` and `spring.pulsar.consumer.*` prefixed application properties.

If you need more control over the configuration of the consumer factory, consider registering one or more `javadoc:org.springframework.pulsar.core.ConsumerBuilderCustomizer[]` beans. These customizers are applied to all consumers created by the factory, and therefore all `javadoc:org.springframework.pulsar.annotation.PulsarListener[format=annotation]` instances. You can also customize a single listener by setting the `consumerCustomizer` attribute of the `javadoc:org.springframework.pulsar.annotation.PulsarListener[format=annotation]` annotation.

If you need more control over the actual container factory configuration, consider registering one or more `PulsarContainerFactoryCustomizer<ConcurrentPulsarListenerContainerFactory>[]` beans.

53.5. Reading a Message

The Pulsar reader interface enables applications to manually manage cursors. When you use a reader to connect to a topic you need to specify which message the reader begins reading from when it connects to a topic.

When the Apache Pulsar infrastructure is present, any bean can be annotated with `javadoc:org.springframework.pulsar.annotation.PulsarReader[format=annotation]` to consume messages using a reader. The following component creates a reader endpoint that starts reading messages from the beginning of the `someTopic` topic:

```
include-code::MyBean[]
```

The `javadoc:org.springframework.pulsar.annotation.PulsarReader[format=annotation]` relies on a `javadoc:org.springframework.pulsar.core.PulsarReaderFactory[]` to create the underlying Pulsar reader. Spring Boot auto-configuration provides this reader factory which can be customized by setting any of the `spring.pulsar.reader.*` prefixed application properties.

If you need more control over the configuration of the reader factory, consider registering one or more `javadoc:org.springframework.pulsar.core.ReaderBuilderCustomizer[]` beans. These customizers are applied to all readers created by the factory, and therefore all `javadoc:org.springframework.pulsar.annotation.PulsarReader[format=annotation]` instances. You can also customize a single listener by setting the `readerCustomizer` attribute of the `javadoc:org.springframework.pulsar.annotation.PulsarReader[format=annotation]` annotation.

If you need more control over the actual container factory configuration, consider registering one or more `PulsarContainerFactoryCustomizer<DefaultPulsarReaderContainerFactory>[]` beans.



For more details on any of the above components and to discover other available features, see the [Spring for Apache Pulsar {url=spring-pulsar-docs}\[reference documentation\]](#).

53.6. Transaction Support

Spring for Apache Pulsar supports transactions when using `javadoc:org.springframework.pulsar.core.PulsarTemplate[]` and `javadoc:org.springframework.pulsar.annotation.PulsarListener[format=annotation]`.

Setting the `configprop:spring.pulsar.transaction.enabled[]` property to `true` will:

- Configure a `javadoc:org.springframework.pulsar.transaction.PulsarTransactionManager[]` bean
- Enable transaction support for `javadoc:org.springframework.pulsar.core.PulsarTemplate[]`
- Enable transaction support for `javadoc:org.springframework.pulsar.annotation.PulsarListener[format=annotation]` methods

The `transactional` attribute of `javadoc:org.springframework.pulsar.annotation.PulsarListener[format=annotation]` can be used to fine-tune when transactions should be used with listeners.

For more control of the Spring for Apache Pulsar transaction features you should define your own `javadoc:org.springframework.pulsar.core.PulsarTemplate[]` and/or `javadoc:org.springframework.pulsar.config.ConcurrentPulsarListenerContainerFactory[]` beans. You can also define a `javadoc:org.springframework.pulsar.transaction.PulsarAwareTransactionManager[]` bean if the default auto-configured `javadoc:org.springframework.pulsar.transaction.PulsarTransactionManager[]` is not suitable.

53.7. Additional Pulsar Properties

The properties supported by auto-configuration are shown in the [Integration Properties](#) section of the Appendix. Note that, for the most part, these properties (hyphenated or camelCase) map directly to the Apache Pulsar configuration properties. See the Apache Pulsar documentation for details.

Only a subset of the properties supported by Pulsar are available directly through the javadoc:`org.springframework.boot.pulsar.autoconfigure.PulsarProperties[]` class. If you wish to tune the auto-configured components with additional properties that are not directly supported, you can use the customizer supported by each aforementioned component.

54. RSocket

RSocket (<https://rsocket.io>) is a binary protocol for use on byte stream transports. It enables symmetric interaction models through async message passing over a single connection.

The `spring-messaging` module of the Spring Framework provides support for RSocket requesters and responders, both on the client and on the server side. See the [RSocket section](https://spring-framework-docs.org/rsocket.html#rsocket-spring) of the Spring Framework reference for more details, including an overview of the RSocket protocol.

54.1. RSocket Strategies Auto-configuration

Spring Boot auto-configures an `javadoc:org.springframework.messaging.rsocket.RSocketStrategies[]` bean that provides all the required infrastructure for encoding and decoding RSocket payloads. By default, the auto-configuration will try to configure the following (in order):

1. **CBOR** (<https://cbor.io/>) codecs with Jackson
2. JSON codecs with Jackson

The `spring-boot-starter-rsocket` starter provides both dependencies. See the [Jackson support section](#) to know more about customization possibilities.

Developers can customize the `javadoc:org.springframework.messaging.rsocket.RSocketStrategies[]` component by creating beans that implement the `javadoc:org.springframework.boot.rsocket.messaging.RSocketStrategiesCustomizer[]` interface. Note that their `javadoc:org.springframework.core.annotation.Order[format=annotation]` is important, as it determines the order of codecs.

54.2. RSocket Server Auto-configuration

Spring Boot provides RSocket server auto-configuration. The required dependencies are provided by the `spring-boot-starter-rsocket`.

Spring Boot allows exposing RSocket over WebSocket from a WebFlux server, or standing up an independent RSocket server. This depends on the type of application and its configuration.

For WebFlux application (that is of type `javadoc:org.springframework.boot.WebApplicationType#REACTIVE[]`), the RSocket server will be plugged into the Web Server only if the following properties match:

```
spring:
  rsocket:
    server:
      mapping-path: "/rsocket"
      transport: "websocket"
```



Plugging RSocket into a web server is only supported with Reactor Netty, as RSocket itself is built with that library.

Alternatively, an RSocket TCP or websocket server is started as an independent, embedded server. Besides the dependency requirements, the only required configuration is to define a port for that server:

```
spring:
  rsocket:
    server:
      port: 9898
```

54.3. Spring Messaging RSocket Support

Spring Boot will auto-configure the Spring Messaging infrastructure for RSocket.

This means that Spring Boot will create a `javadoc:org.springframework.messaging.rsocket.annotation.support.RSocketMessageHandler[]` bean that will handle RSocket requests to your application.



You can use <https://spring-framework-docs.org/web/webmvc/mvc-controller/ann-advice.html> [`@ControllerAdvice`] to handle exceptions.

54.4. Calling RSocket Services with RSocketRequester

Once the `javadoc:io.rsocket.RSocket[]` channel is established between server and client, any party can send or receive requests to the other.

As a server, you can get injected with an `javadoc:org.springframework.messaging.rsocket.RSocketRequester[]` instance on any handler method of an RSocket `javadoc:org.springframework.stereotype.Controller[format=annotation]`. As a client, you need to configure and establish an RSocket connection first. Spring Boot auto-configures an `javadoc:org.springframework.messaging.rsocket.RSocketRequester$Builder[]` for such cases with the expected codecs and applies any `javadoc:org.springframework.messaging.rsocket.RSocketConnectorConfigurer[]` bean.

The `javadoc:org.springframework.messaging.rsocket.RSocketRequester$Builder[]` instance is a prototype bean, meaning each injection point will provide you with a new instance. This is done on purpose since this builder is stateful and you should not create requesters with different setups using the same instance.

The following code shows a typical example:

```
include-code::MyService[]
```

55. Spring Integration

Spring Boot offers several conveniences for working with {url-spring-integration-site}[Spring Integration], including the `spring-boot-starter-integration` starter. Spring Integration provides abstractions over messaging and also other transports such as HTTP, TCP, and others. If Spring Integration is available on your classpath, it is initialized through the `javax.annotation.processing.EnableIntegration` annotation.

Spring Integration polling logic relies on the auto-configured `javax.annotation.processing.TaskScheduler`. The default `javax.annotation.processing.integration.scheduler.PollerMetadata` (poll unbounded number of messages every second) can be customized with `spring.integration.poller.*` configuration properties.

Spring Boot also configures some features that are triggered by the presence of additional Spring Integration modules. If `spring-integration-jmx` is also on the classpath, message processing statistics are published over JMX. If `spring-integration-jdbc` is available, the default database schema can be created on startup, as shown in the following line:

```
spring:
  integration:
    jdbc:
      initialize-schema: "always"
```

If `spring-integration-rsocket` is available, developers can configure an RSocket server using `spring.rsocket.server.*` properties and let it use `javax.annotation.processing.integration.rsocket.integration.RSocketEndpoint` or `javax.annotation.processing.integration.rsocket.outbound.RSocketOutboundGateway` components to handle incoming RSocket messages. This infrastructure can handle Spring Integration RSocket channel adapters and `javax.annotation.processing.messaging.handler.annotation.MessageMapping` handlers (given `spring.integration.rsocket.server.message-mapping-enabled` is configured).

Spring Boot can also auto-configure an `javax.annotation.processing.integration.rsocket.client.RSocketConnector` using configuration properties:

```
# Connecting to a RSocket server over TCP
spring:
  integration:
    rsocket:
      client:
        host: "example.org"
        port: 9898

# Connecting to a RSocket Server over WebSocket
spring:
  integration:
    rsocket:
      client:
        uri: "ws://example.org"
```

See the {code-spring-boot-integration-src}/autoconfigure/IntegrationAutoConfiguration.java[IntegrationAutoConfiguration] and `javax.annotation.processing.integration.autoconfigure.IntegrationProperties` classes for more details.

56. WebSockets

Spring Boot provides WebSockets auto-configuration for embedded Tomcat and Jetty. If you deploy a war file to a standalone container, Spring Boot assumes that the container is responsible for the configuration of its WebSocket support.

Spring Framework provides [url-spring-framework-docs/web/websocket.html](https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/websocket/) [rich WebSocket support] for MVC web applications that can be easily accessed through the `spring-boot-starter-websocket` module.

WebSocket support is also available for [url-spring-framework-docs/web/webflux-websocket.html](https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/webflux/websocket/) [reactive web applications] and requires to include the WebSocket API alongside `spring-boot-starter-webflux` :

```
<dependency>
  <groupId>jakarta.websocket</groupId>
  <artifactId>jakarta.websocket-api</artifactId>
</dependency>
```

XML

57. AOT Cache

AOT cache is a [JVM feature](https://openjdk.org/jeps/483) (https://openjdk.org/jeps/483) that can help reduce the startup time and memory footprint of Java applications.

If you're using Java < 24, you should read the sections about CDS. CDS is the predecessor of AOT cache, but works similarly.

Spring Boot supports both CDS and AOT cache, and it is recommended that you use AOT cache if it is available in the JVM version you are using (Java 24 or later).

57.1. AOT Cache



If you're using Java < 24, AOT cache is not available. You have to use CDS instead.

To use the AOT cache feature, you should first perform a training run on your application in extracted form:

```
$ java -Djarmode=tools -jar my-app.jar extract --destination application
$ cd application
$ java -XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh -jar my-app.jar
```

SHELL

This creates an `app.aot` cache file that can be reused as long as the application is not updated and the same Java version is used.

To use the cache file, you need to add an extra parameter when starting the application:

```
$ java -XX:AOTCache=app.aot -jar my-app.jar
```

SHELL



You have to use the cache file with the extracted form of the application, otherwise it has no effect.

57.2. CDS



If you're using Java 24 or later, please use AOT cache instead of CDS.

To use CDS, you should first perform a training run on your application in extracted form:

```
$ java -Djarmode=tools -jar my-app.jar extract --destination application
$ cd application
$ java -XX:ArchiveClassesAtExit=application.jsa -Dspring.context.exit=onRefresh -jar my-app.jar
```

SHELL

This creates an `application.jsa` archive file that can be reused as long as the application is not updated.

To use the archive file, you need to add an extra parameter when starting the application:

```
$ java -XX:SharedArchiveFile=application.jsa -jar my-app.jar
```

SHELL



You have to use the cache file with the extracted form of the application, otherwise it has no effect.



For more details about CDS, refer to the [Class Data Sharing documentation of the JDK](https://docs.oracle.com/en/java/javase/17/vm/class-data-sharing.html) (https://docs.oracle.com/en/java/javase/17/vm/class-data-sharing.html).

58. Ahead-of-Time Processing With the JVM

It's beneficial for the startup time to run your application using the AOT generated initialization code. First, you need to ensure that the jar you are building includes AOT generated code.



AOT cache and Spring's AOT can be combined to further improve startup time.

For Maven, this means that you should build with `-Pnative` to activate the `native` profile:

```
$ mvn -Pnative package
```

SHELL

For Gradle, you need to ensure that your build includes the `org.springframework.boot.aot` plugin.

When the JAR has been built, run it with `spring.aot.enabled` system property set to `true`. For example:

```
$ java -Dspring.aot.enabled=true -jar myapplication.jar
..... Starting AOT-processed MyApplication ...
```

SHELL

Beware that using the ahead-of-time processing has drawbacks. It implies the following restrictions:

- The classpath is fixed and fully defined at build time
- The beans defined in your application cannot change at runtime, meaning:
 - The Spring `javax.annotation.Profile` annotation and profile-specific configuration [have limitations](#).
 - Properties that change if a bean is created are not supported (for example, `javax.annotation.ConditionalOnProperty` and `.enabled` properties).

To learn more about ahead-of-time processing, please see the [packaging/native-image/introducing-graalvm-native-images.html](#) section.

59. Checkpoint and Restore With the JVM

Coordinated Restore at Checkpoint (<https://wiki.openjdk.org/display/crac/Main>) (CRaC) is an OpenJDK project that defines a new Java API to allow you to checkpoint and restore an application on the HotSpot JVM. It is based on CRIU (<https://github.com/checkpoint-restore/criu>), a project that implements checkpoint/restore functionality on Linux.

The principle is the following: you start your application almost as usual but with a CRaC enabled version of the JDK like BellSoft Liberica JDK with CRaC (<https://bell-sw.com/pages/downloads/?package=jdk-crac>) or Azul Zulu JDK with CRaC (<https://www.azul.com/downloads/?package=jdk-crac#zulu>). Then at some point, potentially after some workloads that will warm up your JVM by executing all common code paths, you trigger a checkpoint using an API call, a `jcmd` command, an HTTP endpoint, or a different mechanism.

A memory representation of the running JVM, including its warmness, is then serialized to disk, allowing a fast restoration at a later point, potentially on another machine with a similar operating system and CPU architecture. The restored process retains all the capabilities of the HotSpot JVM, including further JIT optimizations at runtime.

Based on the foundations provided by Spring Framework, Spring Boot provides support for checkpointing and restoring your application, and manages out-of-the-box the lifecycle of resources such as socket, files and thread pools on a limited scope (<https://github.com/spring-projects/spring-lifecycle-smoke-tests/blob/ci/STATUS.adoc>). Additional lifecycle management is expected for other dependencies and potentially for the application code dealing with such resources.

You can find more details about the two modes supported ("on demand checkpoint/restore of a running application" and "automatic checkpoint/restore at startup"), how to enable checkpoint and restore support and some guidelines in `{url-spring-framework-docs}/integration/checkpoint-restore.html` [the Spring Framework JVM Checkpoint Restore support documentation].

60. Cloud Native Buildpacks

Docker images can be built directly from your Maven or Gradle plugin using [Cloud Native Buildpacks](https://buildpacks.io) (<https://buildpacks.io>). If you've ever used an application platform such as Cloud Foundry or Heroku then you've probably used a buildpack. Buildpacks are the part of the platform that takes your application and converts it into something that the platform can actually run. For example, Cloud Foundry's Java buildpack will notice that you're pushing a `.jar` file and automatically add a relevant JRE.

With Cloud Native Buildpacks, you can create Docker compatible images that you can run anywhere. Spring Boot includes buildpack support directly for both Maven and Gradle. This means you can just type a single command and quickly get a sensible image into your locally running Docker daemon.

See the individual plugin documentation on how to use buildpacks with [Maven](#) and [Gradle](#).



The [Paketo Spring Boot buildpack](https://github.com/paketo-buildpacks/spring-boot) (<https://github.com/paketo-buildpacks/spring-boot>) supports the `layers.idx` file, so any [layer customization](#) that is applied to it will be reflected in the image created by the buildpacks.



In order to achieve reproducible builds and container image caching, buildpacks can manipulate the application resources metadata (such as the file "last modified" information). You should ensure that your application does not rely on that metadata at runtime. Spring Boot can use that information when serving static resources, but this can be disabled with `configprop:spring.web.resources.cache.use-last-modified[]`.

61. Dockerfiles

While it is possible to convert a Spring Boot uber jar into a Docker image with just a few lines in the `Dockerfile`, using the [layering feature](#) will result in an optimized image. When you create a jar containing the layers index file, the `spring-boot-jarmode-tools` jar will be added as a dependency to your jar. With this jar on the classpath, you can launch your application in a special mode which allows the bootstrap code to run something entirely different from your application, for example, something that extracts the layers.

Here's how you can launch your jar with a `tools` jar mode:

```
$ java -Djarmode=tools -jar my-app.jar
```

SHELL

This will provide the following output:

```
Usage:
  java -Djarmode=tools -jar my-app.jar

Available commands:
  extract      Extract the contents from the jar
  list-layers  List layers from the jar that can be extracted
  help        Help about any command
```

The `extract` command can be used to easily split the application into layers to be added to the `Dockerfile`. Here is an example of a `Dockerfile` using `jarmode`.

```
link:reference:partial$dockerfile[role=include]
# Start the application jar - this is not the uber jar used by the builder
# This jar only contains application code and references to the extracted jar files
# This layout is efficient to start up and AOT cache (and CDS) friendly
ENTRYPOINT ["java", "-jar", "application.jar"]
```

DOCKERFILE

Assuming the above `Dockerfile` is in the current directory, your Docker image can be built with `docker build .`, or optionally specifying the path to your application jar, as shown in the following example:

```
$ docker build --build-arg JAR_FILE=path/to/myapp.jar .
```

SHELL

This is a multi-stage `Dockerfile`. The builder stage extracts the directories that are needed later. Each of the `COPY` commands relates to the layers extracted by the `jarmode`.

Of course, a `Dockerfile` can be written without using the `jarmode`. You can use some combination of `unzip` and `mv` to move things to the right layer but `jarmode` simplifies that. Additionally, the layout created by the `jarmode` is AOT cache (and CDS) friendly out of the box.

61.1. AOT Cache



If you're using Java < 24, AOT cache is not available. You have to use CDS instead.

If you want to additionally enable the [AOT cache](#), you can use this `Dockerfile`:

```
link:reference:partial$dockerfile[role=include]
# Execute the AOT cache training run
RUN java -XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh -jar application.jar
# Start the application jar with AOT cache enabled - this is not the uber jar used by the builder
# This jar only contains application code and references to the extracted jar files
# This layout is efficient to start up and AOT cache friendly
ENTRYPOINT ["java", "-XX:AOTCache=app.aot", "-jar", "application.jar"]
```

DOCKERFILE

This is mostly the same as the above `Dockerfile`. As the last steps, it creates the AOT cache file by doing a training run and passes the AOT cache parameter to `java -jar`.

61.2. CDS



If you're using Java 24 or later, please use AOT cache instead of CDS.

If you want to additionally enable [CDS](#), you can use this `Dockerfile`:

```
link:reference:partial$dockerfile[role=include]
# Execute the CDS training run
RUN java -XX:ArchiveClassesAtExit=application.jsa -Dspring.context.exit=onRefresh -jar application.jar
# Start the application jar with CDS enabled - this is not the uber jar used by the builder
# This jar only contains application code and references to the extracted jar files
# This layout is efficient to start up and CDS friendly
ENTRYPOINT ["java", "-XX:SharedArchiveFile=application.jsa", "-jar", "application.jar"]
```

DOCKERFILE

This is mostly the same as the above `Dockerfile`. As the last steps, it creates the CDS archive by doing a training run and passes the CDS parameter to `java -jar`.

62. Efficient Container Images

It is easily possible to package a Spring Boot uber jar as a Docker image. However, there are various downsides to copying and running the uber jar as-is in the Docker image. There's always a certain amount of overhead when running an uber jar without unpacking it, and in a containerized environment this can be noticeable. The other issue is that putting your application's code and all its dependencies in one layer in the Docker image is not optimal. Since you probably recompile your code more often than you upgrade the version of Spring Boot you use, it's often better to separate things a bit more. If you put jar files in the layer before your application classes, Docker often only needs to change the very bottom layer and can pick others up from its cache.

62.1. Layering Docker Images

To make it easier to create optimized Docker images, Spring Boot supports adding a layer index file to the jar. It provides a list of layers and the parts of the jar that should be contained within them. The list of layers in the index is ordered based on the order in which the layers should be added to the Docker/OCI image. Out-of-the-box, the following layers are supported:

- `dependencies` (for regular released dependencies)
- `spring-boot-loader` (for everything under `org/springframework/boot/loader`)
- `snapshot-dependencies` (for snapshot dependencies)
- `application` (for application classes and resources)

The following shows an example of a `layers.idx` file:

```
- "dependencies":  
  - BOOT-INF/lib/library1.jar  
  - BOOT-INF/lib/library2.jar  
- "spring-boot-loader":  
  - org/springframework/boot/loader/launch/JarLauncher.class  
  - ... <other classes>  
- "snapshot-dependencies":  
  - BOOT-INF/lib/library3-SNAPSHOT.jar  
- "application":  
  - META-INF/MANIFEST.MF  
  - BOOT-INF/classes/a/b/C.class
```

YAML

This layering is designed to separate code based on how likely it is to change between application builds. Library code is less likely to change between builds, so it is placed in its own layers to allow tooling to re-use the layers from cache. Application code is more likely to change between builds so it is isolated in a separate layer.

Spring Boot also supports layering for war files with the help of a `layers.idx`.

For Maven, see the [packaging layered jar or war section](#) for more details on adding a layer index to the archive. For Gradle, see the [packaging layered jar or war section](#) of the Gradle plugin documentation.

63. Container Images

Spring Boot applications can be containerized [using Dockerfiles](#), or by [using Cloud Native Buildpacks](#) to create optimized docker compatible container images that you can run anywhere.

64. Efficient Deployments

64.1. Unpacking the Executable jar

You can run your application using the executable jar, but loading the classes from nested jars has a small startup cost. Depending on the size of the jar, running the application from an exploded structure is faster and recommended in production. Certain PaaS implementations may also choose to extract archives before they run. For example, Cloud Foundry operates this way.

Spring Boot supports extracting your application to a directory using different layouts. The default layout is the most efficient, and it is [AOT cache](#) (and [CDS](#)) friendly.

In this layout, the libraries are extracted to a `lib/` folder; and the application jar contains the application classes and a manifest which references the libraries in the `lib/` folder.

To unpack the executable jar, run this command:

```
$ java -Djarmode=tools -jar my-app.jar extract
```

SHELL

And then in production, you can run the extracted jar:

```
$ java -jar my-app/my-app.jar
```

SHELL

After startup, you should not expect any differences in execution time between running an executable jar and running an extracted jar.



Run `java -Djarmode=tools -jar my-app.jar help extract` to see all possible options.

65. Packaging Spring Boot Applications

Spring Boot supports several technologies for optimizing applications for deployment, including [GraalVM native images](#), [AOT cache](#), and [Checkpoint and Restore](#).

Spring Boot applications can be packaged in Docker containers using techniques described in [packaging/container-images/index.html](#).

66. Advanced Native Images Topics

66.1. Nested Configuration Properties

Reflection hints are automatically created for configuration properties by the Spring ahead-of-time engine. Nested configuration properties which are not inner classes, however, **must** be annotated with `javax.annotation.processing.Generated` or `org.springframework.boot.context.properties.NestedConfigurationProperty` [format=annotation], otherwise they won't be detected and will not be bindable.

```
include-code::MyProperties[]
```

where `Nested` is:

```
include-code::Nested[]
```

The example above produces configuration properties for `my.properties.name` and `my.properties.nested.number`. Without the `org.springframework.boot.context.properties.NestedConfigurationProperty` [format=annotation] annotation on the `nested` field, the `my.properties.nested.number` property would not be bindable in a native image. You can also annotate the getter method.

When using constructor binding, you have to annotate the field with `org.springframework.boot.context.properties.NestedConfigurationProperty` [format=annotation]:

```
include-code::MyPropertiesCtor[]
```

When using records, you have to annotate the parameter with `org.springframework.boot.context.properties.NestedConfigurationProperty` [format=annotation]:

```
include-code::MyPropertiesRecord[]
```

When using Kotlin, you need to annotate the parameter of a data class with `org.springframework.boot.context.properties.NestedConfigurationProperty` [format=annotation]:

```
include-code::MyPropertiesKotlin[]
```



Please use public getters and setters in all cases, otherwise the properties will not be bindable.

66.2. Converting a Spring Boot Executable Jar

It is possible to convert a Spring Boot executable jar into a native image as long as the jar contains the AOT generated assets. This can be useful for a number of reasons, including:

- You can keep your regular JVM pipeline and turn the JVM application into a native image on your CI/CD platform.
- As `native-image` does not support cross-compilation (<https://github.com/oracle/graal/issues/407>), you can keep an OS neutral deployment artifact which you convert later to different OS architectures.

You can convert a Spring Boot executable jar into a native image using Cloud Native Buildpacks, or using the `native-image` tool that is shipped with GraalVM.



Your executable jar must include AOT generated assets such as generated classes and JSON hint files.

66.2.1. Using Buildpacks

Spring Boot applications usually use Cloud Native Buildpacks through the Maven (`mvn spring-boot:build-image`) or Gradle (`gradle bootBuildImage`) integrations. You can, however, also use `{url-buildpacks-docs}/for-platform-operators/how-to/integrate-ci/pack/[ pack ]` to turn an AOT processed Spring Boot executable jar into a native container image.



You have to build your application with at least JDK 25, because Buildpacks use the same GraalVM native-image version as the Java version used for compilation.

First, make sure that a Docker daemon is available (see [Get Docker](https://docs.docker.com/installation/#installation) (<https://docs.docker.com/installation/#installation>) for more details). [Configure it to allow non-root user](https://docs.docker.com/engine/install/linux-postinstall/#manage-docker-as-a-non-root-user) (<https://docs.docker.com/engine/install/linux-postinstall/#manage-docker-as-a-non-root-user>) if you are on Linux.

You also need to install `pack` by following `{url-buildpacks-docs}/for-platform-operators/how-to/integrate-ci/pack/#install` [the installation guide on buildpacks.io].

Assuming an AOT processed Spring Boot executable jar built as `myproject-0.0.1-SNAPSHOT.jar` is in the `target` directory, run:

```
$ pack build --builder paketobuildpacks/builder-noble-java-tiny \
  --path target/myproject-0.0.1-SNAPSHOT.jar \
  --env 'BP_NATIVE_IMAGE=true' \
  my-application:0.0.1-SNAPSHOT
```

SHELL



You do not need to have a local GraalVM installation to generate an image in this way.

Once `pack` has finished, you can launch the application using `docker run`:

```
$ docker run --rm -p 8080:8080 docker.io/library/myproject:0.0.1-SNAPSHOT
```

SHELL

66.2.2. Using GraalVM native-image

Another option to turn an AOT processed Spring Boot executable jar into a native executable is to use the GraalVM `native-image` tool. For this to work, you'll need a GraalVM distribution on your machine. You can either download it manually on the [{url-download-liberica-nik}\[Liberica Native Image Kit page\]](#) or you can use a download manager like SDKMAN!.

Assuming an AOT processed Spring Boot executable jar built as `myproject-0.0.1-SNAPSHOT.jar` is in the `target` directory, run:

```
$ rm -rf target/native
$ mkdir -p target/native
$ cd target/native
$ jar -xvf ../myproject-0.0.1-SNAPSHOT.jar
$ native-image -H:Name=myproject @META-INF/native-image/argfile -cp ../BOOT-INF/classes:`find BOOT-INF/lib | tr '\n' ':'`
$ mv myproject ../
```

SHELL



These commands work on Linux or macOS machines, but you will need to adapt them for Windows.



The `@META-INF/native-image/argfile` might not be packaged in your jar. It is only included when reachability metadata overrides are needed.



The `native-image -cp` flag does not accept wildcards. You need to ensure that all jars are listed (the command above uses `find` and `tr` to do this).

66.3. Using the Tracing Agent

The GraalVM native image [{url-graal-docs-native-image}/metadata/AutomaticMetadataCollection\[tracing agent\]](#) allows you to intercept reflection, resources or proxy usage on the JVM in order to generate the related hints. Spring should generate most of these hints automatically, but the tracing agent can be used to quickly identify the missing entries.

When using the agent to generate hints for a native image, there are a couple of approaches:

- Launch the application directly and exercise it.
- Run application tests to exercise the application.

The first option is interesting for identifying the missing hints when a library or a pattern is not recognized by Spring.

The second option sounds more appealing for a repeatable setup, but by default the generated hints will include anything required by the test infrastructure. Some of these will be unnecessary when the application runs for real. To address this problem the agent supports an access-filter file that will cause certain data to be excluded from the generated output.

66.3.1. Launch the Application Directly

Use the following command to launch the application with the native image tracing agent attached:

```
$ java -Dspring.aot.enabled=true \
  -agentlib:native-image-agent=config-output-dir=/path/to/config-dir/ \
  -jar target/myproject-0.0.1-SNAPSHOT.jar
```

SHELL

Now you can exercise the code paths you want to have hints for and then stop the application with `ctrl-c`.

On application shutdown the native image tracing agent will write the hint files to the given config output directory. You can either manually inspect these files, or use them as input to the native image build process. To use them as input, copy them into the `src/main/resources/META-INF/native-image/` directory. The next time you build the native image, GraalVM will take these files into consideration.

There are more advanced options which can be set on the native image tracing agent, for example filtering the recorded hints by caller classes, etc. For further reading, please see [{url-graal-docs-native-image}/metadata/AutomaticMetadataCollection\[the official documentation\]](#).

66.4. Custom Hints

If you need to provide your own hints for reflection, resources, serialization, proxy usage and so on, you can use the `javadoc:org.springframework.aot.hint.RuntimeHintsRegistrar[]` API. Create a class that implements the `javadoc:org.springframework.aot.hint.RuntimeHintsRegistrar[]` interface, and then make appropriate calls to the provided `javadoc:org.springframework.aot.hint.RuntimeHints[]` instance:

```
include-code::MyRuntimeHints[]
```

You can then use `javadoc:org.springframework.context.annotation.ImportRuntimeHints[format=annotation]` on any `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class (for example your `javadoc:org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` annotated application class) to activate those hints.

If you have classes which need binding (mostly needed when serializing or deserializing JSON), you can use [{url-spring-framework-docs}/core/aot.html#aot.hints.register-reflection-for-binding\[@RegisterReflectionForBinding \]](#) on any bean. Most of the hints are automatically inferred, for example when accepting or returning data from a `javadoc:org.springframework.web.bind.annotation.RestController[format=annotation]` method. But when you work with `javadoc:org.springframework.web.reactive.function.client.WebClient[]`, `javadoc:org.springframework.web.client.RestClient[]` or `javadoc:org.springframework.web.client.RestTemplate[]` directly, you might need to use `javadoc:org.springframework.aot.hint.annotation.RegisterReflectionForBinding[format=annotation]`.

66.4.1. Testing Custom Hints

The `javadoc:org.springframework.aot.hint.predicate.RuntimeHintsPredicates[]` API can be used to test your hints. The API provides methods that build a `javadoc:java.util.function.Predicate[]` that can be used to test a `javadoc:org.springframework.aot.hint.RuntimeHints[]` instance.

If you're using AssertJ, your test would look like this:

```
include-code::MyRuntimeHintsTests[]
```

66.4.2. Providing Hints Statically

If you prefer, custom hints can be provided statically in one or more GraalVM JSON hint files. Such files should be placed in `src/main/resources/` within a `META-INF/native-image/**/` directory. The [hints generated during AOT processing](#) are written to a directory named `META-INF/native-image/{groupId}/{artifactId}/`. Place your static hint files in a directory that does not clash with this location, such as `META-INF/native-image/{groupId}/{artifactId}-additional-hints/`.

66.5. Known Limitations

GraalVM native images are an evolving technology and not all libraries provide support. The GraalVM community is helping by providing [reachability metadata](#) (<https://github.com/oracle/graalvm-reachability-metadata>) for projects that don't yet ship their own. Spring itself doesn't contain hints for 3rd party libraries and instead relies on the reachability metadata project.

If you encounter problems when generating native images for Spring Boot applications, please check the [{url-github-wiki}/Spring-Boot-with-GraalVM](#) [Spring Boot with GraalVM] page of the Spring Boot wiki. You can also contribute issues to the [spring-aot-smoke-tests](#) (<https://github.com/spring-projects/spring-aot-smoke-tests>) project on GitHub which is used to confirm that common application types are working as expected.

If you find a library which doesn't work with GraalVM, please raise an issue on the [reachability metadata project](#) (<https://github.com/oracle/graalvm-reachability-metadata>).

67. GraalVM Native Images

GraalVM Native Images (<https://www.graalvm.org/native-image/>) are standalone executables that can be generated by processing compiled Java applications ahead-of-time. Native Images generally have a smaller memory footprint and start faster than their JVM counterparts.

68. Introducing GraalVM Native Images

GraalVM Native Images provide a new way to deploy and run Java applications. Compared to the Java Virtual Machine, native images can run with a smaller memory footprint and with much faster startup times.

They are well suited to applications that are deployed using container images and are especially interesting when combined with "Function as a service" (FaaS) platforms.

Unlike traditional applications written for the JVM, GraalVM Native Image applications require ahead-of-time processing in order to create an executable. This ahead-of-time processing involves statically analyzing your application code from its main entry point.

A GraalVM Native Image is a complete, platform-specific executable. You do not need to ship a Java Virtual Machine in order to run a native image.



If you just want to get started and experiment with GraalVM you can jump to the [how-to-native-image/developing-your-first-application.html](#) section and return to this section later.

68.1. Key Differences with JVM Deployments

The fact that GraalVM Native Images are produced ahead-of-time means that there are some key differences between native and JVM based applications. The main differences are:

- Static analysis of your application is performed at build-time from the `main` entry point.
- Code that cannot be reached when the native image is created will be removed and won't be part of the executable.
- GraalVM is not directly aware of dynamic elements of your code and must be told about reflection, resources, serialization, and dynamic proxies.
- The application classpath is fixed at build time and cannot change.
- There is no lazy class loading, everything shipped in the executables will be loaded in memory on startup.
- There are some limitations around some aspects of Java applications that are not fully supported.

On top of those differences, Spring uses a process called [Spring Ahead-of-Time processing](#), which imposes further limitations. Please make sure to read at least the beginning of the next section to learn about those.



The [{url-graal-docs-native-image}/metadata/Compatibility/\[Native Image Compatibility Guide\]](#) section of the GraalVM reference documentation provides more details about GraalVM limitations.

68.2. Understanding Spring Ahead-of-Time Processing

Typical Spring Boot applications are quite dynamic and configuration is performed at runtime. In fact, the concept of Spring Boot auto-configuration depends heavily on reacting to the state of the runtime in order to configure things correctly.

Although it would be possible to tell GraalVM about these dynamic aspects of the application, doing so would undo most of the benefit of static analysis. So instead, when using Spring Boot to create native images, a closed-world is assumed and the dynamic aspects of the application are restricted.

A closed-world assumption implies, besides [the limitations created by GraalVM itself](#), the following restrictions:

- The beans defined in your application cannot change at runtime, meaning:
 - The Spring `javax.annotation.Profile` annotation and profile-specific configuration [have limitations](#).
 - Properties that change if a bean is created are not supported (for example, `org.springframework.boot.autoconfigure.condition.ConditionalOnProperty` and `.enabled` properties).

When these restrictions are in place, it becomes possible for Spring to perform ahead-of-time processing during build-time and generate additional assets that GraalVM can use. A Spring AOT processed application will typically generate:

- Java source code
- Bytecode (for dynamic proxies, etc.)
- GraalVM JSON hint files in `META-INF/native-image/{groupId}/{artifactId}/` :
 - Resource hints (`resource-config.json`)
 - Reflection hints (`reflect-config.json`)
 - Serialization hints (`serialization-config.json`)
 - Java Proxy Hints (`proxy-config.json`)
 - JNI Hints (`jni-config.json`)

If the generated hints are not sufficient, you can also [provide your own](#).

68.2.1. Source Code Generation

Spring applications are composed of Spring Beans. Internally, Spring Framework uses two distinct concepts to manage beans. There are bean instances, which are the actual instances that have been created and can be injected into other beans. There are also bean definitions which are used to define attributes of a bean and how its instance should be created.

If we take a typical `javax.annotation.Configuration` class:

```
include-code::MyConfiguration[]
```

The bean definition is created by parsing the `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class and finding the `javadoc:org.springframework.context.annotation.Bean[format=annotation]` methods. In the above example, we're defining a `javadoc:org.springframework.beans.factory.config.BeanDefinition[]` for a singleton bean named `myBean`. We're also creating a `javadoc:org.springframework.beans.factory.config.BeanDefinition[]` for the `MyConfiguration` class itself.

When the `myBean` instance is required, Spring knows that it must invoke the `myBean()` method and use the result. When running on the JVM, `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class parsing happens when your application starts and `javadoc:org.springframework.context.annotation.Bean[format=annotation]` methods are invoked using reflection.

When creating a native image, Spring operates in a different way. Rather than parsing `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` classes and generating bean definitions at runtime, it does it at build-time. Once the bean definitions have been discovered, they are processed and converted into source code that can be analyzed by the GraalVM compiler.

The Spring AOT process would convert the configuration class above to code like this:

```
include-code::MyConfiguration__BeanDefinitions[]
```



The exact code generated may differ depending on the nature of your bean definitions.

You can see above that the generated code creates equivalent bean definitions to the `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class, but in a direct way that can be understood by GraalVM.

There is a bean definition for the `myConfiguration` bean, and one for `myBean`. When a `myBean` instance is required, a `javadoc:org.springframework.beans.factory.aot.BeanInstanceSupplier[]` is called. This supplier will invoke the `myBean()` method on the `myConfiguration` bean.



During Spring AOT processing, your application is started up to the point that bean definitions are available. Bean instances are not created during the AOT processing phase.

Spring AOT will generate code like this for all your bean definitions. It will also generate code when bean post-processing is required (for example, to call `javadoc:org.springframework.beans.factory.annotation.Autowired[format=annotation]` methods). An `javadoc:org.springframework.context.ApplicationContextInitializer[]` will also be generated which will be used by Spring Boot to initialize the `javadoc:org.springframework.context.ApplicationContext[]` when an AOT processed application is actually run.



Although AOT generated source code can be verbose, it is quite readable and can be helpful when debugging an application. Generated source files can be found in `target/spring-aot/main/sources` when using Maven and `build/generated/aotSources` with Gradle.

68.2.2. Hint File Generation

In addition to generating source files, the Spring AOT engine will also generate hint files that are used by GraalVM. Hint files contain JSON data that describes how GraalVM should deal with things that it can't understand by directly inspecting the code.

For example, you might be using a Spring annotation on a private method. Spring will need to use reflection in order to invoke private methods, even on GraalVM. When such situations arise, Spring can write a reflection hint so that GraalVM knows that even though the private method isn't called directly, it still needs to be available in the native image.

Hint files are generated under `META-INF/native-image` where they are automatically picked up by GraalVM.



Generated hint files can be found in `target/spring-aot/main/resources` when using Maven and `build/generated/aotResources` with Gradle.

68.2.3. Proxy Class Generation

Spring sometimes needs to generate proxy classes to enhance the code you've written with additional features. To do this, it uses the cglib library which directly generates bytecode.

When an application is running on the JVM, proxy classes are generated dynamically as the application runs. When creating a native image, these proxies need to be created at build-time so that they can be included by GraalVM.



Unlike source code generation, generated bytecode isn't particularly helpful when debugging an application. However, if you need to inspect the contents of the `.class` files using a tool such as `javap` you can find them in `target/spring-aot/main/classes` for Maven and `build/generated/aotClasses` for Gradle.

69. Testing

Spring Boot provides a number of utilities and annotations to help when testing your application.

Test support is provided by two general-purpose modules – `spring-boot-test` contains core items and `spring-boot-test-autoconfigure` supports auto-configuration for tests – and several focused `-test` modules that provide testing support for a particular feature.

Most developers use the `spring-boot-starter-test` starter, which imports both general-purpose Spring Boot test modules as well as JUnit Jupiter, AssertJ, Hamcrest, and a number of other useful libraries, and the focused `-test` modules that are applicable to their particular application.

If you have tests that use JUnit 4, JUnit 6's vintage engine can be used to run them. To use the vintage engine, add a dependency on `junit-vintage-engine`, as shown in the following example:



```
<dependency>
  <groupId>org.junit.vintage</groupId>
  <artifactId>junit-vintage-engine</artifactId>
  <scope>test</scope>
  <exclusions>
    <exclusion>
      <groupId>org.hamcrest</groupId>
      <artifactId>hamcrest-core</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

XML

`hamcrest-core` is excluded in favor of `org.hamcrest:hamcrest` that is part of `spring-boot-starter-test`.

70. Testing Spring Applications

One of the major advantages of dependency injection is that it should make your code easier to unit test. You can instantiate objects by using the `new` operator without even involving Spring. You can also use *mock objects* instead of real dependencies.

Often, you need to move beyond unit testing and start integration testing (with a Spring javadoc:org.springframework.context.ApplicationContext()). It is useful to be able to perform integration testing without requiring deployment of your application or needing to connect to other infrastructure.

The Spring Framework includes a dedicated test module for such integration testing. You can declare a dependency directly to `org.springframework:spring-test` or use the `spring-boot-starter-test` starter to pull it in transitively.

If you have not used the `spring-test` module before, you should start by reading the {url-spring-framework-docs}/testing.html[relevant section] of the Spring Framework reference documentation.

71. Testing Spring Boot Applications

A Spring Boot application is a `SpringApplication`, so nothing very special has to be done to test it beyond what you would normally do with a vanilla Spring context.



External properties, logging, and other features of Spring Boot are installed in the context by default only if you use `SpringApplication` to create it.

Spring Boot provides a `SpringBootTest` annotation, which can be used as an alternative to the standard `spring-test` `ContextConfiguration` annotation when you need Spring Boot features. The annotation works by [creating the `SpringApplication`](#) used in your tests through `SpringApplication`. In addition to `SpringBootTest` a number of other annotations are also provided for [testing more specific slices](#) of an application.



If you are using JUnit 4, do not forget to also add `@RunWith(SpringRunner.class)` to your test, otherwise the annotations will be ignored. If you are using JUnit 5, there is no need to add the equivalent `@ExtendWith(SpringExtension.class)` as `SpringBootTest` and the other `@*Test` annotations are already annotated with it.

By default, `SpringBootTest` will not start a server. You can use the `webEnvironment` attribute of `SpringBootTest` to further refine how your tests run:

- MOCK (Default)** : Loads a `SpringApplication` and provides a mock web environment. Embedded servers are not started when using this annotation. If a web environment is not available on your classpath, this mode transparently falls back to creating a regular non-web `SpringApplication`. It can be used in conjunction with `@AutoConfigureMockMvc`, or `WebTestClient` for mock-based testing of your web application.
- RANDOM_PORT** : Loads a `WebServerApplicationContext` and provides a real web environment. Embedded servers are started and listen on a random port.
- DEFINED_PORT** : Loads a `WebServerApplicationContext` and provides a real web environment. Embedded servers are started and listen on a defined port (from your `application.properties`) or on the default port of 8080.
- NONE** : Loads a `SpringApplication` but does not provide *any* web environment (mock or otherwise).



If your test is `Transactional`, it rolls back the transaction at the end of each test method by default. However, as using this arrangement with either `RANDOM_PORT` or `DEFINED_PORT` implicitly provides a real servlet environment, the HTTP client and server run in separate threads and, thus, in separate transactions. Any transaction initiated on the server does not roll back in this case.



`SpringBootTest` with `webEnvironment = WebEnvironment.RANDOM_PORT` will also start the management server on a separate random port if your application uses a different port for the management server.

71.1. Detecting Web Application Type

If Spring MVC is available, a regular MVC-based application context is configured. If you have only Spring WebFlux, we will detect that and configure a WebFlux-based application context instead.

If both are present, Spring MVC takes precedence. If you want to test a reactive web application in this scenario, you must set the `configprop:spring.main.web-application-type` property:

```
include-code::MyWebFluxTests[]
```

71.2. Detecting Test Configuration

If you are familiar with the Spring Test Framework, you may be used to using `@ContextConfiguration(classes=...)` in order to specify which Spring `Configuration` to load. Alternatively, you might have often used nested `Configuration` classes within your test.

When testing Spring Boot applications, this is often not required. Spring Boot's `@*Test` annotations search for your primary configuration automatically whenever you do not explicitly define one.

The search algorithm works up from the package that contains the test until it finds a class annotated with `SpringBootApplication` or `SpringBootTest`. As long as you [structure your code](#) in a sensible way, your main configuration is usually found.



If you use a [test annotation to test a more specific slice of your application](#), you should avoid adding configuration settings that are specific to a particular area on the [main method's application class](#).

The underlying component scan configuration of `SpringBootApplication` defines exclude filters that are used to make sure slicing works as expected. If you are using an explicit `ComponentScan` directive on your `SpringBootApplication`-annotated class, be aware that those filters will be disabled. If you are using slicing, you should define them again.

If you want to customize the primary configuration, you can use a nested `org.springframework.boot.test.context.TestConfiguration` class. Unlike a nested `org.springframework.context.annotation.Configuration` class, which would be used instead of your application's primary configuration, a nested `org.springframework.boot.test.context.TestConfiguration` class is used in addition to your application's primary configuration.



Spring's test framework caches application contexts between tests. Therefore, as long as your tests share the same configuration (no matter how it is discovered), the potentially time-consuming process of loading the context happens only once.

71.3. Using the Test Configuration Main Method

Typically the test configuration discovered by `org.springframework.boot.test.context.SpringBootTest` will be your main `org.springframework.boot.autoconfigure.SpringBootApplication`. In most well structured applications, this configuration class will also include the `main` method used to launch the application.

For example, the following is a very common code pattern for a typical Spring Boot application:

```
include-code::typical/MyApplication[]
```

In the example above, the `main` method doesn't do anything other than delegate to `org.springframework.boot.SpringApplication#run(java.lang.Class,java.lang.String...)`. It is, however, possible to have a more complex `main` method that applies customizations before calling `org.springframework.boot.SpringApplication#run(java.lang.Class,java.lang.String...)`.

For example, here is an application that changes the banner mode and sets additional profiles:

```
include-code::custom/MyApplication[]
```

Since customizations in the `main` method can affect the resulting `org.springframework.context.ApplicationContext`, it's possible that you might also want to use the `main` method to create the `org.springframework.context.ApplicationContext` used in your tests. By default, `org.springframework.boot.test.context.SpringBootTest` will not call your `main` method, and instead the class itself is used directly to create the `org.springframework.context.ApplicationContext`

If you want to change this behavior, you can change the `useMainMethod` attribute of `org.springframework.boot.test.context.SpringBootTest` to `org.springframework.boot.test.context.SpringBootTest$UseMainMethod#ALWAYS` or `org.springframework.boot.test.context.SpringBootTest$UseMainMethod#WHEN_AVAILABLE`. When set to `ALWAYS`, the test will fail if no `main` method can be found. When set to `WHEN_AVAILABLE` the `main` method will be used if it is available, otherwise the standard loading mechanism will be used.

For example, the following test will invoke the `main` method of `MyApplication` in order to create the `org.springframework.context.ApplicationContext`. If the main method sets additional profiles then those will be active when the `org.springframework.context.ApplicationContext` starts.

```
include-code::always/MyApplicationTests[]
```

71.4. Excluding Test Configuration

If your application uses component scanning (for example, if you use `org.springframework.boot.autoconfigure.SpringBootApplication` or `org.springframework.context.annotation.ComponentScan`), you may find top-level configuration classes that you created only for specific tests accidentally get picked up everywhere.

As we [have seen earlier](#), `org.springframework.boot.test.context.TestConfiguration` can be used on an inner class of a test to customize the primary configuration. `org.springframework.boot.test.context.TestConfiguration` can also be used on a top-level class. Doing so indicates that the class should not be picked up by scanning. You can then import the class explicitly where it is required, as shown in the following example:

```
include-code::MyTests[]
```



If you directly use `org.springframework.context.annotation.ComponentScan` (that is, not through `org.springframework.boot.autoconfigure.SpringBootApplication`) you need to register the `org.springframework.boot.context.TypeExcludeFilter` with it. See the `org.springframework.boot.context.TypeExcludeFilter` API documentation for details.



An imported `org.springframework.boot.test.context.TestConfiguration` is processed earlier than an inner-class `org.springframework.boot.test.context.TestConfiguration` and an imported `org.springframework.boot.test.context.TestConfiguration` will be processed before any configuration found through component scanning. Generally speaking, this difference in ordering has no noticeable effect but it is something to be aware of if you're relying on bean overriding.

71.5. Using Application Arguments

If your application expects [arguments](#), you can have `org.springframework.boot.test.context.SpringBootTest` inject them using the `args` attribute.

```
include-code::MyApplicationArgumentTests[]
```

71.6. Testing With a Mock Environment

By default, `org.springframework.boot.test.context.SpringBootTest` does not start the server but instead sets up a mock environment for testing web endpoints.

With Spring MVC, we can query our web endpoints using `{url-spring-framework-docs}/testing/mockmvc.html` [`MockMvc`]. The following integrations are available:

- The regular `{url-spring-framework-docs}/testing/mockmvc/hamcrest.html[ MockMvc ]` that uses Hamcrest.
- `{url-spring-framework-docs}/testing/mockmvc/assertj.html[ MockMvcTester ]` that wraps `javadoc:org.springframework.test.web.servlet.MockMvc[]` and uses AssertJ.
- `{url-spring-framework-docs}/testing/resttestclient.html[ RestTestClient ]` where `javadoc:org.springframework.test.web.servlet.MockMvc[]` is plugged in as the server to handle requests with.
- `{url-spring-framework-docs}/testing/webtestclient.html[ WebTestClient ]` where `javadoc:org.springframework.test.web.servlet.MockMvc[]` is plugged in as the server to handle requests with.

The following example showcases the available integrations:

```
include-code::MyMockMvcTests[]
```



If you want to focus only on the web layer and not start a complete `javadoc:org.springframework.context.ApplicationContext[]`, consider using `javadoc:org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest[format=annotation]` instead).

With Spring WebFlux endpoints, you can use `{url-spring-framework-docs}/testing/webtestclient.html[ WebTestClient ]` as shown in the following example:

```
include-code::MyMockWebTestClientTests[]
```



Testing within a mocked environment is usually faster than running with a full servlet container. However, since mocking occurs at the Spring MVC layer, code that relies on lower-level servlet container behavior cannot be directly tested with MockMvc.

For example, Spring Boot's error handling is based on the “error page” support provided by the servlet container. This means that, whilst you can test your MVC layer throws and handles exceptions as expected, you cannot directly test that a specific custom error page is rendered. If you need to test these lower-level concerns, you can start a fully running server as described in the next section.

71.7. Testing With a Running Server

If you need to start a full running server, we recommend that you use random ports. If you use `@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)`, an available port is picked at random each time your test runs.

The `javadoc:org.springframework.boot.test.web.server.LocalServerPort[format=annotation]` annotation can be used to inject the actual port used into your test.

Tests that need to make REST calls to the started server can autowire a `{url-spring-framework-docs}/testing/resttestclient.html[ RestTestClient ]` by annotating the test class with `javadoc:org.springframework.boot.resttestclient.autoconfigure.AutoConfigureRestTestClient[format=annotation]`.

The configured client resolves relative links to the running server and comes with a dedicated API for verifying responses, as shown in the following example:

```
include-code::MyRandomPortRestTestClientTests[]
```

If you prefer to use AssertJ, dedicated assertions are available from `javadoc:org.springframework.test.web.servlet.client.assertj.RestTestClientResponse[]`, as shown in the following example:

```
include-code::MyRandomPortRestTestClientAssertJTests[]
```

If you have `spring-webflux` on the classpath, you can also autowire a `{url-spring-framework-docs}/testing/webtestclient.html[ WebTestClient ]` by annotating the test class with `javadoc:org.springframework.boot.webtestclient.autoconfigure.AutoConfigureWebTestClient[format=annotation]`.

`WebTestClient` provides a similar API, as shown in the following example:

```
include-code::MyRandomPortWebTestClientTests[]
```



`javadoc:org.springframework.test.web.reactive.server.WebTestClient[]` can also be used with a mock environment, removing the need for a running server, by annotating your test class with `javadoc:org.springframework.boot.webtestclient.autoconfigure.AutoConfigureWebTestClient[format=annotation]` from `spring-boot-webflux-test`.



For certain sliced tests you may need to use the `javadoc:org.springframework.boot.web.server.test.AutoConfigureWebServer[format=annotation]` annotation to auto-configure an embedded web server.

The `spring-boot-resttestclient` module also provides a `javadoc:org.springframework.boot.resttestclient.TestRestTemplate[]` facility:

```
include-code::MyRandomPortTestRestTemplateTests[]
```

To use `TestRestTemplate` a dependency on `spring-boot-restclient` is also required. Take care when adding this dependency as it will enable auto-configuration for `RestClient.Builder`. If your main code uses `RestClient.Builder`, declare the `spring-boot-restclient` dependency so that it is on your application's main classpath and not only on its test classpath.

71.8. Customizing RestTestClient

To customize the `javadoc:org.springframework.test.web.servlet.client.RestTestClient[]` bean, configure a `javadoc:org.springframework.boot.resttestclient.autoconfigure.RestTestClientBuilderCustomizer[]` bean. Any such beans are called with the `javadoc:org.springframework.test.web.servlet.client.RestTestClient$Builder[]` that is used to create the `javadoc:org.springframework.test.web.servlet.client.RestTestClient[]`.

71.9. Customizing WebClient

To customize the `javado:org.springframework.test.web.reactive.server.WebTestClient[]` bean, configure a `javado:org.springframework.boot.webtestclient.autoconfigure.WebTestClientBuilderCustomizer[]` bean. Any such beans are called with the `javado:org.springframework.test.web.reactive.server.WebTestClient$Builder[]` that is used to create the `javado:org.springframework.test.web.reactive.server.WebTestClient[]`.

71.10. Using JMX

As the test context framework caches context, JMX is disabled by default to prevent identical components to register on the same domain. If such test needs access to an `javado:javax.management.MBeanServer[]`, consider marking it dirty as well:

```
include-code::MyJmxTests[]
```

71.11. Using Observations

If you annotate a [sliced test](#) with `javado:org.springframework.boot.micrometer.tracing.test.autoconfigure.AutoConfigureTracing[format=annotation]` from `spring-boot-micrometer-tracing-test` or with `javado:org.springframework.boot.micrometer.metrics.test.autoconfigure.AutoConfigureMetrics[format=annotation]` from `spring-boot-micrometer-metrics-test`, it auto-configures an `javado:io.micrometer.observation.ObservationRegistry[]`.

71.12. Using Metrics

Regardless of your classpath, meter registries, except the in-memory backed, are not auto-configured when using `javado:org.springframework.boot.test.context.SpringBootTest[format=annotation]`.

If you need to export metrics to a different backend as part of an integration test, annotate it with `javado:org.springframework.boot.micrometer.metrics.test.autoconfigure.AutoConfigureMetrics[format=annotation]`.

If you annotate a [sliced test](#) with `javado:org.springframework.boot.micrometer.metrics.test.autoconfigure.AutoConfigureMetrics[format=annotation]`, it auto-configures an in-memory `javado:io.micrometer.core.instrument.MeterRegistry[]`. Data exporting in sliced tests is not supported with the `javado:org.springframework.boot.micrometer.metrics.test.autoconfigure.AutoConfigureMetrics[format=annotation]` annotation.

71.13. Using Tracing

Regardless of your classpath, tracing components which are reporting data are not auto-configured when using `javado:org.springframework.boot.test.context.SpringBootTest[format=annotation]`.

If you need those components as part of an integration test, annotate the test with `javado:org.springframework.boot.micrometer.tracing.test.autoconfigure.AutoConfigureTracing[format=annotation]`.

If you have created your own reporting components (e.g. a custom `javado:io.opentelemetry.sdk.trace.export.SpanExporter[]` or `brave.handler.SpanHandler`) and you don't want them to be active in tests, you can use the `javado:org.springframework.boot.micrometer.tracing.autoconfigure.ConditionalOnEnabledTracingExport[format=annotation]` annotation to disable them.

If you annotate a [sliced test](#) with `javado:org.springframework.boot.micrometer.tracing.test.autoconfigure.AutoConfigureTracing[format=annotation]`, it auto-configures a no-op `javado:io.micrometer.tracing.Tracer[]`. Data exporting in sliced tests is not supported with the `javado:org.springframework.boot.micrometer.tracing.test.autoconfigure.AutoConfigureTracing[format=annotation]` annotation.

71.14. Mocking and Spying Beans

When running tests, it is sometimes necessary to mock certain components within your application context. For example, you may have a facade over some remote service that is unavailable during development. Mocking can also be useful when you want to simulate failures that might be hard to trigger in a real environment.

Spring Framework includes a `javado:org.springframework.test.context.bean.override.mockito.MockitoBean[format=annotation]` annotation that can be used to define a Mockito mock for a bean inside your `javado:org.springframework.context.ApplicationContext[]`. Additionally, `javado:org.springframework.test.context.bean.override.mockito.MockitoSpyBean[format=annotation]` can be used to define a Mockito spy. Learn more about these features in the [{url-spring-framework-docs}/testing/annotations/integration-spring/annotation-mockitobean.html](#) [Spring Framework documentation].

71.15. Auto-configured Tests

Spring Boot's auto-configuration system works well for applications but can sometimes be a little too much for tests. It often helps to load only the parts of the configuration that are required to test a “slice” of your application. For example, you might want to test that Spring MVC controllers are mapping URLs correctly, and you do not want to involve database calls in those tests, or you might want to test JPA entities, and you are not interested in the web layer when those tests run.

When combined with `spring-boot-test-autoconfigure`, Spring Boot's [test modules](#) include a number of annotations that can be used to automatically configure such “slices”. Each of them works in a similar way, providing a `@...Test` annotation that loads the `javado:org.springframework.context.ApplicationContext[]` and one or more `@AutoConfigure...` annotations that can be used to customize auto-configuration settings.



Each slice restricts component scan to appropriate components and loads a very restricted set of auto-configuration classes. If you need to exclude one of them, most `@...Test` annotations provide an `excludeAutoConfiguration` attribute. Alternatively, you can use `@ImportAutoConfiguration#exclude`.



Including multiple “slices” by using several `@...Test` annotations in one test is not supported. If you need multiple “slices”, pick one of the `@...Test` annotations and include the `@AutoConfigure...` annotations of the other “slices” by hand.



It is also possible to use the `@AutoConfigure...` annotations with the standard `javado:org.springframework.boot.test.context.SpringBootTest[format=annotation]` annotation. You can use this combination if you are not interested in “slicing” your application but you want some of the auto-configured test beans.

71.16. Auto-configured JSON Tests

To test that object JSON serialization and deserialization is working as expected, you can use the `javax.annotation.processing.processor.support.JsonTest` annotation from the `spring-boot-test-autoconfigure` module. `javax.annotation.processing.processor.support.JsonTest` auto-configures the available supported JSON mapper, which can be one of the following libraries:

- Jackson `javax.annotation.processing.processor.support.JsonMapper`, any `javax.annotation.processing.processor.support.JsonComponent` beans and any Jackson `javax.annotation.processing.processor.support.JsonModule`
- Jackson 2 (deprecated) `javax.annotation.processing.processor.support.ObjectMapper`, any `javax.annotation.processing.processor.support.JsonComponent` beans and any Jackson `javax.annotation.processing.processor.support.Module`
- Gson
- Jsonb



A list of the auto-configurations that are enabled by `javax.annotation.processing.processor.support.JsonTest` can be [found in the appendix](#).

If you need to configure elements of the auto-configuration, you can use the `javax.annotation.processing.processor.support.AutoConfigureJsonTesters` annotation.

Spring Boot includes AssertJ-based helpers that work with the `JSONAssert` and `JsonPath` libraries to check that JSON appears as expected. The `javax.annotation.processing.processor.support.JsonTester`, `javax.annotation.processing.processor.support.GsonTester`, `javax.annotation.processing.processor.support.JsonbTester`, and `javax.annotation.processing.processor.support.BasicJsonTester` classes can be used for Jackson, Gson, Jsonb, and Strings respectively. Any helper fields on the test class can be `javax.annotation.processing.processor.support.Autowired` when using `javax.annotation.processing.processor.support.JsonTest`. The following example shows a test class for Jackson:

```
include-code::MyJsonTests[]
```



JSON helper classes can also be used directly in standard unit tests. To do so, call the `initFields` method of the helper in your `javax.annotation.processing.processor.support.BeforeEach` method if you do not use `javax.annotation.processing.processor.support.JsonTest`.

If you use Spring Boot's AssertJ-based helpers to assert on a number value at a given JSON path, you might not be able to use `isEqualTo` depending on the type. Instead, you can use AssertJ's `satisfies` to assert that the value matches the given condition. For instance, the following example asserts that the actual number is a float value close to 0.15 within an offset of 0.01.

```
include-code::MyJsonAssertTests[tag=*
```

71.17. Auto-configured Spring MVC Tests

To test whether Spring MVC controllers are working as expected, use the `javax.annotation.processing.processor.support.WebMvcTest` annotation from the `spring-boot-webmvc-test` module. `javax.annotation.processing.processor.support.WebMvcTest` auto-configures the Spring MVC infrastructure and limits scanned beans to `javax.annotation.processing.processor.support.Controller`, `javax.annotation.processing.processor.support.ControllerAdvice`, `javax.annotation.processing.processor.support.JsonComponent`, `javax.annotation.processing.processor.support.JsonModule` (deprecated), `javax.annotation.processing.processor.support.converter.Converter`, `javax.annotation.processing.processor.support.converter.GenericConverter`, `javax.annotation.processing.processor.support.Filter`, `javax.annotation.processing.processor.support.HandlerInterceptor`, `javax.annotation.processing.processor.support.WebMvcConfigurer`, `javax.annotation.processing.processor.support.WebMvcRegistrations`, and `javax.annotation.processing.processor.support.HandlerMethodArgumentResolver`. Regular `javax.annotation.processing.processor.support.Component` and `javax.annotation.processing.processor.support.ConfigurationProperties` beans are not scanned when the `javax.annotation.processing.processor.support.WebMvcTest` annotation is used. `javax.annotation.processing.processor.support.EnableConfigurationProperties` can be used to include `javax.annotation.processing.processor.support.ConfigurationProperties` beans.



A list of the auto-configuration settings that are enabled by `javax.annotation.processing.processor.support.WebMvcTest` can be [found in the appendix](#).



If you need to register extra components, such as a `javax.annotation.processing.processor.support.JsonModule`, you can import additional configuration classes by using `javax.annotation.processing.processor.support.Import` on your test.

Often, `javax.annotation.processing.processor.support.WebMvcTest` is limited to a single controller and is used in combination with `javax.annotation.processing.processor.support.bean.override.mockito.MockitoBean` to provide mock implementations for required collaborators.

`javax.annotation.processing.processor.support.WebMvcTest` also auto-configures `javax.annotation.processing.processor.support.test.web.servlet.MockMvc`. Mock MVC offers a powerful way to quickly test MVC controllers without needing to start a full HTTP server. If AssertJ is available, the AssertJ support provided by `javax.annotation.processing.processor.support.test.web.servlet.assertj.MockMvcTester` is auto-configured as well. If you'd like to use `javax.annotation.processing.processor.support.test.web.servlet.client.RestTestClient` in your tests, annotate your test class with `javax.annotation.processing.processor.support.resttestclient.autoconfigure.AutoConfigureRestTestClient`. A `RestTestClient` that uses the Mock MVC infrastructure will then be auto-configured.



You can also auto-configure `javadoc.org.springframework.test.web.servlet.MockMvc[]` and `javadoc.org.springframework.test.web.servlet.assertj.MockMvcTester[]` in a non-`@WebMvcTest` (such as `javadoc.org.springframework.boot.test.context.SpringBootTest[format=annotation]`) by annotating it with `javadoc.org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc[format=annotation]`. The following example uses `javadoc.org.springframework.test.web.servlet.assertj.MockMvcTester[]`:

```
include-code::MyControllerTests[]
```



If you need to configure elements of the auto-configuration (for example, when servlet filters should be applied) you can use attributes in the `javadoc.org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc[format=annotation]` annotation.

If you use `HtmlUnit` and `Selenium`, auto-configuration also provides an `HtmlUnit javadoc.org.springframework.web.reactive.function.client.WebClient[]` bean and/or a `Selenium javadoc.org.openqa.selenium.WebDriver[]` bean. The following example uses `HtmlUnit`:

```
include-code::MyHtmlUnitTests[]
```



By default, Spring Boot puts `javadoc.org.openqa.selenium.WebDriver[]` beans in a special “scope” to ensure that the driver exits after each test and that a new instance is injected. If you do not want this behavior, you can add `@Scope(ConfigurableBeanFactory.SCOPE_SINGLETON)` to your `javadoc.org.openqa.selenium.WebDriver[] javadoc.org.springframework.context.annotation.Bean[format=annotation]` definition.



The `webDriver` scope created by Spring Boot will replace any user defined scope of the same name. If you define your own `webDriver` scope you may find it stops working when you use `javadoc.org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest[format=annotation]`.

If you have Spring Security on the classpath, `javadoc.org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest[format=annotation]` will also scan `javadoc.org.springframework.security.config.annotation.web.WebSecurityConfigurer[]` beans. Instead of disabling security completely for such tests, you can use Spring Security's test support. More details on how to use Spring Security's `javadoc.org.springframework.test.web.servlet.MockMvc[]` support can be found in this [how-to:testing.html](#) “How-to Guides” section.



Sometimes writing Spring MVC tests is not enough; Spring Boot can help you run [full end-to-end tests with an actual server](#).

71.18. Auto-configured Spring WebFlux Tests

To test that `{url-spring-framework-docs}/web-reactive.html`[Spring WebFlux] controllers are working as expected, you can use the `javadoc.org.springframework.boot.webflux.test.autoconfigure.WebFluxTest[format=annotation]` annotation from the `spring-boot-webflux-test` module. `javadoc.org.springframework.boot.webflux.test.autoconfigure.WebFluxTest[format=annotation]` auto-configures the Spring WebFlux infrastructure and limits scanned beans to `javadoc.org.springframework.stereotype.Controller[format=annotation]`, `javadoc.org.springframework.web.bind.annotation.ControllerAdvice[format=annotation]`, `javadoc.org.springframework.boot.jackson.JacksonComponent[format=annotation]`, `javadoc.org.springframework.boot.jackson2.JsonComponent[format=annotation]` (deprecated), `javadoc.org.springframework.core.convert.converter.Converter[]`, `javadoc.org.springframework.core.convert.converter.GenericConverter[]` and `javadoc.org.springframework.web.reactive.config.WebFluxConfigurer[]`. Regular `javadoc.org.springframework.stereotype.Component[format=annotation]` and `javadoc.org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans are not scanned when the `javadoc.org.springframework.boot.webflux.test.autoconfigure.WebFluxTest[format=annotation]` annotation is used. `javadoc.org.springframework.boot.context.properties.EnableConfigurationProperties[format=annotation]` can be used to include `javadoc.org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans.



A list of the auto-configurations that are enabled by `javadoc.org.springframework.boot.webflux.test.autoconfigure.WebFluxTest[format=annotation]` can be [found in the appendix](#).



If you need to register extra components, such as a `javadoc.tools.jackson.databind.JacksonModule[]`, you can import additional configuration classes using `javadoc.org.springframework.context.annotation.Import[format=annotation]` on your test.

Often, `javadoc.org.springframework.boot.webflux.test.autoconfigure.WebFluxTest[format=annotation]` is limited to a single controller and used in combination with the `javadoc.org.springframework.test.context.bean.override.mockito.MockitoBean[format=annotation]` annotation to provide mock implementations for required collaborators.

`javadoc.org.springframework.boot.webflux.test.autoconfigure.WebFluxTest[format=annotation]` also auto-configures `{url-spring-framework-docs}/testing/webtestclient.html`[`WebTestClient`], which offers a powerful way to quickly test WebFlux controllers without needing to start a full HTTP server.



You can also auto-configure `javadoc.org.springframework.test.web.reactive.server.WebTestClient[]` in a non-`@WebFluxTest` (such as `javadoc.org.springframework.boot.test.context.SpringBootTest[format=annotation]`) by annotating it with `javadoc.org.springframework.boot.webtestclient.autoconfigure.AutoConfigureWebTestClient[format=annotation]`.

The following example shows a class that uses both `javadoc.org.springframework.boot.webflux.test.autoconfigure.WebFluxTest[format=annotation]` and a `javadoc.org.springframework.test.web.reactive.server.WebTestClient[]`:

```
include-code::MyControllerTests[]
```



This setup is only supported by WebFlux applications as using `javadoc.org.springframework.test.web.reactive.server.WebTestClient[]` in a mocked web application only works with WebFlux at the moment.



javadoc:org.springframework.boot.webflux.test.autoconfigure.WebFluxTest[format=annotation] cannot detect routes registered through the functional web framework. For testing javadoc:org.springframework.web.reactive.function.server.RouterFunction[] beans in the context, consider importing your javadoc:org.springframework.web.reactive.function.server.RouterFunction[] yourself by using javadoc:org.springframework.context.annotation.Import[format=annotation] or by using javadoc:org.springframework.boot.test.context.SpringBootTest[format=annotation].



javadoc:org.springframework.boot.webflux.test.autoconfigure.WebFluxTest[format=annotation] cannot detect custom security configuration registered as a javadoc:org.springframework.context.annotation.Bean[format=annotation] of type javadoc:org.springframework.security.web.server.SecurityWebFilterChain[]. To include that in your test, you will need to import the configuration that registers the bean by using javadoc:org.springframework.context.annotation.Import[format=annotation] or by using javadoc:org.springframework.boot.test.context.SpringBootTest[format=annotation].



Sometimes writing Spring WebFlux tests is not enough; Spring Boot can help you run [full end-to-end tests with an actual server](#).

71.19. Auto-configured Spring GraphQL Tests

Spring GraphQL offers a dedicated testing support module; you'll need to add it to your project:

Maven

XML

```
<dependencies>
  <dependency>
    <groupId>org.springframework.graphql</groupId>
    <artifactId>spring-graphql-test</artifactId>
    <scope>test</scope>
  </dependency>
  <!-- Unless already present in the compile scope -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Gradle

GRADLE

```
dependencies {
    testImplementation("org.springframework.graphql:spring-graphql-test")
    // Unless already present in the implementation configuration
    testImplementation("org.springframework.boot:spring-boot-starter-webflux")
}
```

This testing module ships the {url-spring-graphql-docs}/testing.html#testing.graphqltester[GraphQLTester]. The tester is heavily used in test, so be sure to become familiar with using it. There are javadoc:org.springframework.graphql.test.tester.GraphQLTester[] variants and Spring Boot will auto-configure them depending on the type of tests:

- the javadoc:org.springframework.graphql.test.tester.ExecutionGraphQLServiceTester[] performs tests on the server side, without a client nor a transport
- the javadoc:org.springframework.graphql.test.tester.HttpGraphQLTester[] performs tests with a client that connects to a server, with or without a live server

Spring Boot helps you to test your {url-spring-graphql-docs}/controllers.html[Spring GraphQL Controllers] with the javadoc:org.springframework.boot.graphql.test.autoconfigure.GraphQLTest[format=annotation] annotation from the `spring-boot-graphql-test` module. javadoc:org.springframework.boot.graphql.test.autoconfigure.GraphQLTest[format=annotation] auto-configures the Spring GraphQL infrastructure, without any transport nor server being involved. This limits scanned beans to javadoc:org.springframework.stereotype.Controller[format=annotation], javadoc:org.springframework.graphql.execution.RuntimeWiringConfigurer[], javadoc:org.springframework.boot.jackson.JacksonComponent[], javadoc:org.springframework.boot.jackson2.JsonComponent[format=annotation] (deprecated), javadoc:org.springframework.core.convert.converter.Converter[], javadoc:org.springframework.core.convert.converter.GenericConverter[], javadoc:org.springframework.graphql.execution.DataFetcherExceptionResolver[], javadoc:org.springframework.graphql.execution.instrumentation.Instrumentation[] and javadoc:org.springframework.boot.graphql.autoconfigure.GraphQLSourceBuilderCustomizer[]. Regular javadoc:org.springframework.stereotype.Component[format=annotation] and javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation] beans are not scanned when the javadoc:org.springframework.boot.graphql.test.autoconfigure.GraphQLTest[format=annotation] annotation is used. javadoc:org.springframework.boot.context.properties.EnableConfigurationProperties[format=annotation] can be used to include javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation] beans.



A list of the auto-configurations that are enabled by javadoc:org.springframework.boot.graphql.test.autoconfigure.GraphQLTest[format=annotation] can be [found in the appendix](#).

Often, javadoc:org.springframework.boot.graphql.test.autoconfigure.GraphQLTest[format=annotation] is limited to a set of controllers and used in combination with the javadoc:org.springframework.test.context.bean.override.mockito.MockitoBean[format=annotation] annotation to provide mock implementations for required collaborators.

```
include-code::GreetingControllerTests[]
```

javadoc:org.springframework.boot.test.context.SpringBootTest[format=annotation] tests are full integration tests and involve the entire application. A javadoc:org.springframework.graphql.test.tester.HttpGraphQLTester[] bean can be added by annotating your test class with javadoc:org.springframework.boot.graphql.test.autoconfigure.tester.AutoConfigureHttpGraphQLTester[format=annotation] from the `spring-boot-graphql-test` module:

```
include-code::GraphQLIntegrationTests[]
```

The `javax.xml.bind.annotation.XmlRootElement` bean uses the relevant transport of the integration test. When using a random or defined port, the tester is configured against the live server. To bind the tester to `MockMvc`, make sure to annotate your test class with `javax.xml.bind.annotation.XmlRootElement`.

71.20. Auto-configured Data Cassandra Tests

You can use `javax.xml.bind.annotation.XmlRootElement` from the `spring-boot-data-cassandra-test` module to test Data Cassandra applications. By default, it configures a `javax.xml.bind.annotation.XmlRootElement`, scans for `javax.xml.bind.annotation.XmlRootElement` classes, and configures Spring Data Cassandra repositories. Regular `javax.xml.bind.annotation.XmlRootElement` and `javax.xml.bind.annotation.XmlRootElement` beans are not scanned when the `javax.xml.bind.annotation.XmlRootElement` annotation is used. `javax.xml.bind.annotation.XmlRootElement` can be used to include `javax.xml.bind.annotation.XmlRootElement` beans. (For more about using Cassandra with Spring Boot, see [data/nosql.html](#).)



A list of the auto-configuration settings that are enabled by `javax.xml.bind.annotation.XmlRootElement` can be [found in the appendix](#).

The following example shows a typical setup for using Cassandra tests in Spring Boot:

```
include-code::MyDataCassandraTests[]
```

71.21. Auto-configured Data Couchbase Tests

You can use `javax.xml.bind.annotation.XmlRootElement` from the `spring-boot-data-couchbase-test` module to test Data Couchbase applications. By default, it configures a `javax.xml.bind.annotation.XmlRootElement` or `javax.xml.bind.annotation.XmlRootElement`, scans for `javax.xml.bind.annotation.XmlRootElement` classes, and configures Spring Data Couchbase repositories. Regular `javax.xml.bind.annotation.XmlRootElement` and `javax.xml.bind.annotation.XmlRootElement` beans are not scanned when the `javax.xml.bind.annotation.XmlRootElement` annotation is used. `javax.xml.bind.annotation.XmlRootElement` can be used to include `javax.xml.bind.annotation.XmlRootElement` beans. (For more about using Couchbase with Spring Boot, see [data/nosql.html](#), earlier in this chapter.)



A list of the auto-configuration settings that are enabled by `javax.xml.bind.annotation.XmlRootElement` can be [found in the appendix](#).

The following example shows a typical setup for using Couchbase tests in Spring Boot:

```
include-code::MyDataCouchbaseTests[]
```

71.22. Auto-configured Data Elasticsearch Tests

You can use `javax.xml.bind.annotation.XmlRootElement` from the `spring-boot-data-elasticsearch-test` module to test Data Elasticsearch applications. By default, it configures a `javax.xml.bind.annotation.XmlRootElement`, scans for `javax.xml.bind.annotation.XmlRootElement` classes, and configures Spring Data Elasticsearch repositories. Regular `javax.xml.bind.annotation.XmlRootElement` and `javax.xml.bind.annotation.XmlRootElement` beans are not scanned when the `javax.xml.bind.annotation.XmlRootElement` annotation is used. `javax.xml.bind.annotation.XmlRootElement` can be used to include `javax.xml.bind.annotation.XmlRootElement` beans. (For more about using Elasticsearch with Spring Boot, see [data/nosql.html](#), earlier in this chapter.)



A list of the auto-configuration settings that are enabled by `javax.xml.bind.annotation.XmlRootElement` can be [found in the appendix](#).

The following example shows a typical setup for using Elasticsearch tests in Spring Boot:

```
include-code::MyDataElasticsearchTests[]
```

71.23. Auto-configured Data JPA Tests

You can use the `javax.xml.bind.annotation.XmlRootElement` annotation from the `spring-boot-data-jpa-test` module to test Data JPA applications. By default, it scans for `javax.xml.bind.annotation.XmlRootElement` classes and configures Spring Data JPA repositories. If an embedded database is available on the classpath, it configures one as well. SQL queries are logged by default by setting the `spring.jpa.show-sql` property to `true`. This can be disabled using the `showSql` attribute of the annotation.

Regular `javax.xml.bind.annotation.XmlRootElement` and `javax.xml.bind.annotation.XmlRootElement` beans are not scanned when the `javax.xml.bind.annotation.XmlRootElement` annotation is used. `javax.xml.bind.annotation.XmlRootElement` can be used to include `javax.xml.bind.annotation.XmlRootElement` beans.



A list of the auto-configuration settings that are enabled by `javax.xml.bind.annotation.XmlRootElement` can be [found in the appendix](#).

By default, data JPA tests are transactional and roll back at the end of each test. See the [{url-spring-framework-docs}/testing/testcontext-framework/tx.html#testcontext-tx-enabling-transactions\[relevant section\]](#) in the Spring Framework Reference Documentation for more details. If that is not what you want, you can disable transaction management for a test or for the whole class as follows:

```
include-code::MyNonTransactionalTests[]
```

Data JPA tests may also inject a `javax.persistence.EntityManager` bean, which provides an alternative to the standard JPA `EntityManager` that is specifically designed for tests.



`javax.persistence.EntityManager` can also be auto-configured to any of your Spring-based test class by adding `javax.persistence.EntityManager` annotation. When doing so, make sure that your test is running in a transaction, for instance by adding `javax.transaction.Transactional` annotation on your test class or method.

A `javax.persistence.jdbc.core.JdbcTemplate` is also available if you need that. The following example shows the `javax.persistence.jdbc.core.JdbcTemplate` annotation in use:

```
include-code::withoutdb/MyRepositoryTests[]
```

In-memory embedded databases generally work well for tests, since they are fast and do not require any installation. If, however, you prefer to run tests against a real database you can use the `javax.persistence.jdbc.core.JdbcTemplate` annotation, as shown in the following example:

```
include-code::withdb/MyRepositoryTests[]
```

71.24. Auto-configured JDBC Tests

`javax.persistence.jdbc.core.JdbcTemplate` from the `spring-boot-jdbc-test` module is similar to `javax.persistence.jdbc.core.JdbcTemplate` but is for tests that only require a `javax.sql.DataSource` and do not use Spring Data JDBC. By default, it configures an in-memory embedded database and a `javax.persistence.jdbc.core.JdbcTemplate`. Regular `javax.persistence.stereotype.Component` and `javax.persistence.context.properties.ConfigurationProperties` beans are not scanned when the `javax.persistence.jdbc.core.JdbcTemplate` annotation is used. `javax.persistence.context.properties.EnableConfigurationProperties` can be used to include `javax.persistence.context.properties.ConfigurationProperties` beans.



A list of the auto-configurations that are enabled by `javax.persistence.jdbc.core.JdbcTemplate` can be [found in the appendix](#).

By default, JDBC tests are transactional and roll back at the end of each test. See the [{url-spring-framework-docs}/testing/testcontext-framework/tx.html#testcontext-tx-enabling-transactions\[relevant section\]](#) in the Spring Framework Reference Documentation for more details. If that is not what you want, you can disable transaction management for a test or for the whole class, as follows:

```
include-code::MyTransactionalTests[]
```

If you prefer your test to run against a real database, you can use the `javax.persistence.jdbc.core.JdbcTemplate` annotation in the same way as for `javax.persistence.jdbc.core.JdbcTemplate`. (See [testing/spring-boot-applications.html](#).)

71.25. Auto-configured Data JDBC Tests

`javax.persistence.jdbc.core.JdbcTemplate` from the `spring-boot-data-jdbc-test` module is similar to `javax.persistence.jdbc.core.JdbcTemplate` but is for tests that use Spring Data JDBC repositories. By default, it configures an in-memory embedded database, a `javax.persistence.jdbc.core.JdbcTemplate`, and Spring Data JDBC repositories. Only `javax.persistence.data.jdbc.repository.config.AbstractJdbcConfiguration` subclasses are scanned when the `javax.persistence.jdbc.core.JdbcTemplate` annotation is used, regular `javax.persistence.stereotype.Component` and `javax.persistence.context.properties.ConfigurationProperties` beans are not scanned. `javax.persistence.context.properties.EnableConfigurationProperties` can be used to include `javax.persistence.context.properties.ConfigurationProperties` beans.



A list of the auto-configurations that are enabled by `javax.persistence.jdbc.core.JdbcTemplate` can be [found in the appendix](#).

By default, Data JDBC tests are transactional and roll back at the end of each test. See the [{url-spring-framework-docs}/testing/testcontext-framework/tx.html#testcontext-tx-enabling-transactions\[relevant section\]](#) in the Spring Framework Reference Documentation for more details. If that is not what you want, you can disable transaction management for a test or for the whole test class as [shown in the JDBC example](#).

If you prefer your test to run against a real database, you can use the `javax.persistence.jdbc.core.JdbcTemplate` annotation in the same way as for `javax.persistence.jdbc.core.JdbcTemplate`. (See [testing/spring-boot-applications.html](#).)

71.26. Auto-configured Data R2DBC Tests

`javax.persistence.jdbc.core.JdbcTemplate` from the `spring-boot-data-r2dbc-test` module is similar to `javax.persistence.jdbc.core.JdbcTemplate` but is for tests that use Spring Data R2DBC repositories. By default, it configures an in-memory embedded database, an `javax.persistence.data.r2dbc.core.R2dbcEntityTemplate`, and Spring Data R2DBC repositories. Regular `javax.persistence.stereotype.Component` and `javax.persistence.context.properties.ConfigurationProperties` beans

are not scanned when the `javax.annotation.processing.Generated` annotation is used. `javax.annotation.processing.Generated` can be used to include `javax.annotation.processing.Generated` beans.



A list of the auto-configurations that are enabled by `javax.annotation.processing.Generated` can be [found in the appendix](#).

By default, Data R2DBC tests are not transactional.

If you prefer your test to run against a real database, you can use the `javax.annotation.processing.Generated` annotation in the same way as for `javax.annotation.processing.Generated`. (See [testing/spring-boot-applications.html](#).)

71.27. Auto-configured jOOQ Tests

You can use `javax.annotation.processing.Generated` from `spring-boot-jooq-test` in a similar fashion as `javax.annotation.processing.Generated` but for jOOQ-related tests. As jOOQ relies heavily on a Java-based schema that corresponds with the database schema, the existing `javax.annotation.processing.Generated` is used. If you want to replace it with an in-memory database, you can use `javax.annotation.processing.Generated` to override those settings. (For more about using jOOQ with Spring Boot, see [data/sql.html](#).) Regular `javax.annotation.processing.Generated` and `javax.annotation.processing.Generated` beans are not scanned when the `javax.annotation.processing.Generated` annotation is used. `javax.annotation.processing.Generated` can be used to include `javax.annotation.processing.Generated` beans.



A list of the auto-configurations that are enabled by `javax.annotation.processing.Generated` can be [found in the appendix](#).

`javax.annotation.processing.Generated` configures a `javax.annotation.processing.Generated`. The following example shows the `javax.annotation.processing.Generated` annotation in use:

```
include-code::MyJooqTests[]
```

jOOQ tests are transactional and roll back at the end of each test by default. If that is not what you want, you can disable transaction management for a test or for the whole test class as [shown in the JDBC example](#).

71.28. Auto-configured Data MongoDB Tests

You can use `javax.annotation.processing.Generated` from the `spring-boot-data-mongodb-test` module to test MongoDB applications. By default, it configures a `javax.annotation.processing.Generated`, scans for `javax.annotation.processing.Generated` classes, and configures Spring Data MongoDB repositories. Regular `javax.annotation.processing.Generated` and `javax.annotation.processing.Generated` beans are not scanned when the `javax.annotation.processing.Generated` annotation is used. `javax.annotation.processing.Generated` can be used to include `javax.annotation.processing.Generated` beans. (For more about using MongoDB with Spring Boot, see [data/nosql.html](#).)



A list of the auto-configuration settings that are enabled by `javax.annotation.processing.Generated` can be [found in the appendix](#).

The following class shows the `javax.annotation.processing.Generated` annotation in use:

```
include-code::MyDataMongoDbTests[]
```

71.29. Auto-configured Data Neo4j Tests

You can use `javax.annotation.processing.Generated` from the `spring-boot-data-neo4j-test` module to test Neo4j applications. By default, it scans for `javax.annotation.processing.Generated` classes, and configures Spring Data Neo4j repositories. Regular `javax.annotation.processing.Generated` and `javax.annotation.processing.Generated` beans are not scanned when the `javax.annotation.processing.Generated` annotation is used. `javax.annotation.processing.Generated` can be used to include `javax.annotation.processing.Generated` beans. (For more about using Neo4j with Spring Boot, see [data/nosql.html](#).)



A list of the auto-configuration settings that are enabled by `javax.annotation.processing.Generated` can be [found in the appendix](#).

The following example shows a typical setup for using Neo4j tests in Spring Boot:

```
include-code::propagation/MyDataNeo4jTests[]
```

By default, Data Neo4j tests are transactional and roll back at the end of each test. See the [{url-spring-framework-docs}/testing/testcontext-framework/tx.html#testcontext-tx-enabling-transactions\[relevant section\]](#) in the Spring Framework Reference Documentation for more details. If that is not what you want, you can disable transaction management for a test or for the whole class, as follows:

```
include-code::nopropagation/MyDataNeo4jTests[]
```



Transactional tests are not supported with reactive access. If you are using this style, you must configure `javadoc:org.springframework.boot.data.neo4j.test.autoconfigure.DataNeo4jTest[format=annotation]` tests as described above.

71.30. Auto-configured Data Redis Tests

You can use `javadoc:org.springframework.boot.data.redis.test.autoconfigure.DataRedisTest[format=annotation]` from the `spring-boot-data-redis-test` module to test Data Redis applications. By default, it scans for `javadoc:org.springframework.data.redis.core.RedisHash[format=annotation]` classes and configures Spring Data Redis repositories. Regular `javadoc:org.springframework.stereotype.Component[format=annotation]` and `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans are not scanned when the `javadoc:org.springframework.boot.data.redis.test.autoconfigure.DataRedisTest[format=annotation]` annotation is used.

`javadoc:org.springframework.boot.context.properties.EnableConfigurationProperties[format=annotation]` can be used to include

`javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans. (For more about using Redis with Spring Boot, see [data/nosql.html](#).)



A list of the auto-configuration settings that are enabled by `javadoc:org.springframework.boot.data.redis.test.autoconfigure.DataRedisTest[format=annotation]` can be [found in the appendix](#).

The following example shows the `javadoc:org.springframework.boot.data.redis.test.autoconfigure.DataRedisTest[format=annotation]` annotation in use:

```
include-code::MyDataRedisTests[]
```

71.31. Auto-configured Data LDAP Tests

You can use `javadoc:org.springframework.boot.data.ldap.test.autoconfigure.DataLdapTest[format=annotation]` to test Data LDAP applications. By default, it configures an in-memory embedded LDAP (if available), configures an `javadoc:org.springframework.ldap.core.LdapTemplate[]`, scans for `javadoc:org.springframework.ldap.odm.annotations.Entry[format=annotation]` classes, and configures Spring Data LDAP repositories. Regular `javadoc:org.springframework.stereotype.Component[format=annotation]` and `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans are not scanned when the `javadoc:org.springframework.boot.data.ldap.test.autoconfigure.DataLdapTest[format=annotation]` annotation is used.

`javadoc:org.springframework.boot.context.properties.EnableConfigurationProperties[format=annotation]` can be used to include

`javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans. (For more about using LDAP with Spring Boot, see [data/nosql.html](#).)



A list of the auto-configuration settings that are enabled by `javadoc:org.springframework.boot.data.ldap.test.autoconfigure.DataLdapTest[format=annotation]` can be [found in the appendix](#).

The following example shows the `javadoc:org.springframework.boot.data.ldap.test.autoconfigure.DataLdapTest[format=annotation]` annotation in use:

```
include-code::inmemory/MyDataLdapTests[]
```

In-memory embedded LDAP generally works well for tests, since it is fast and does not require any developer installation. If, however, you prefer to run tests against a real LDAP server, you should exclude the embedded LDAP auto-configuration, as shown in the following example:

```
include-code::server/MyDataLdapTests[]
```

71.32. Auto-configured REST Clients

You can use the `javadoc:org.springframework.boot.restclient.test.autoconfigure.RestClientTest[format=annotation]` annotation from the `spring-boot-restclient-test` module to test REST clients. By default, it auto-configures Jackson, GSON, and Jsonb support, configures a `javadoc:org.springframework.boot.restclient.RestTemplateBuilder[]` and a `javadoc:org.springframework.web.client.RestClient$Builder[]`, and adds support for `javadoc:org.springframework.test.web.client.MockRestServiceServer[]`. Regular `javadoc:org.springframework.stereotype.Component[format=annotation]` and `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans are not scanned when the `javadoc:org.springframework.boot.restclient.test.autoconfigure.RestClientTest[format=annotation]` annotation is used.

`javadoc:org.springframework.boot.context.properties.EnableConfigurationProperties[format=annotation]` can be used to include

`javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` beans.



A list of the auto-configuration settings that are enabled by `javadoc:org.springframework.boot.restclient.test.autoconfigure.RestClientTest[format=annotation]` can be [found in the appendix](#).

The specific beans that you want to test should be specified by using the `value` or `components` attribute of `javadoc:org.springframework.boot.restclient.test.autoconfigure.RestClientTest[format=annotation]`.

When using a `javadoc:org.springframework.boot.restclient.RestTemplateBuilder[]` in the beans under test and `RestTemplateBuilder.rootUri(String rootUri)` has been called when building the `javadoc:org.springframework.web.client.RestTemplate[]`, then the root URI should be omitted from the `javadoc:org.springframework.test.web.client.MockRestServiceServer[]` expectations as shown in the following example:

```
include-code::MyRestTemplateServiceTests[]
```

When using a `javadoc:org.springframework.web.client.RestClient$Builder[]` in the beans under test, or when using a `javadoc:org.springframework.boot.restclient.RestTemplateBuilder[]` without calling `rootUri(String rootUri)`, the full URI must be used in the `javadoc:org.springframework.test.web.client.MockRestServiceServer[]` expectations as shown in the following example:

```
include-code::MyRestClientServiceTests[]
```

71.33. Auto-configured Web Clients

You can use the `javax.annotation.processing.processor.WebClientTest[format=annotation]` annotation from the `spring-boot-webclient-test` module to test code that uses `WebClient`. By default, it auto-configures Jackson, GSON, and Jsonb support, and configures a `javax.annotation.processing.processor.WebClient$Builder[]`. Regular `javax.annotation.processing.processor.Component[format=annotation]` and `javax.annotation.processing.processor.ConfigurationProperties[format=annotation]` beans are not scanned when the `javax.annotation.processing.processor.WebClientTest[format=annotation]` annotation is used. `javax.annotation.processing.processor.EnableConfigurationProperties[format=annotation]` can be used to include `javax.annotation.processing.processor.ConfigurationProperties[format=annotation]` beans.



A list of the auto-configuration settings that are enabled by `javax.annotation.processing.processor.WebClientTest[format=annotation]` can be found in the [appendix](#).

The specific beans that you want to test should be specified by using the `value` or `components` attribute of `javax.annotation.processing.processor.WebClientTest[format=annotation]`.

71.34. Auto-configured Spring REST Docs Tests

You can use the `javax.annotation.processing.processor.restdocs.test.autoconfigure.AutoConfigureRestDocs[format=annotation]` annotation from the `spring-boot-restdocs-` module to use `{url-spring-restdocs-site}[Spring REST Docs]` in your tests with Mock MVC or `WebTestClient`. It removes the need for the JUnit extension in Spring REST Docs.

`javax.annotation.processing.processor.restdocs.test.autoconfigure.AutoConfigureRestDocs[format=annotation]` can be used to override the default output directory (`target/generated-snippets` if you are using Maven or `build/generated-snippets` if you are using Gradle). It can also be used to configure the host, scheme, and port that appears in any documented URIs.

71.34.1. Auto-configured Spring REST Docs Tests With Mock MVC

`javax.annotation.processing.processor.restdocs.test.autoconfigure.AutoConfigureRestDocs[format=annotation]` customizes the `javax.annotation.processing.processor.test.web.servlet.MockMvc[]` bean to use Spring REST Docs when testing servlet-based web applications. You can inject it by using `javax.annotation.processing.processor.beans.factory.annotation.Autowired[format=annotation]` and use it in your tests as you normally would when using Mock MVC and Spring REST Docs, as shown in the following example:

```
include-code::hamcrest/MyUserDocumentationTests[]
```

If you prefer to use the AssertJ integration, `javax.annotation.processing.processor.test.web.servlet.assertj.MockMvcTester[]` is available as well, as shown in the following example:

```
include-code::assertj/MyUserDocumentationTests[]
```

Both reuses the same `javax.annotation.processing.processor.test.web.servlet.MockMvc[]` instance behind the scenes so any configuration to it applies to both.

If you require more control over Spring REST Docs configuration than offered by the attributes of `javax.annotation.processing.processor.restdocs.test.autoconfigure.AutoConfigureRestDocs[format=annotation]`, you can use a `javax.annotation.processing.processor.restdocs.test.autoconfigure.RestDocsMockMvcConfigurationCustomizer[]` bean, as shown in the following example:

```
include-code::MyRestDocsConfiguration[]
```

If you want to make use of Spring REST Docs support for a parameterized output directory, you can create a `javax.annotation.processing.processor.restdocs.mockmvc.RestDocumentationResultHandler[]` bean. The auto-configuration calls `alwaysDo` with this result handler; thereby causing each `javax.annotation.processing.processor.test.web.servlet.MockMvc[]` call to automatically generate the default snippets. The following example shows a `javax.annotation.processing.processor.restdocs.mockmvc.RestDocumentationResultHandler[]` being defined:

```
include-code::MyResultHandlerConfiguration[]
```

71.34.2. Auto-configured Spring REST Docs Tests With WebTestClient

`javax.annotation.processing.processor.restdocs.test.autoconfigure.AutoConfigureRestDocs[format=annotation]` can also be used with `javax.annotation.processing.processor.test.web.reactive.server.WebTestClient[]` when testing reactive web applications. You can inject it by using `javax.annotation.processing.processor.beans.factory.annotation.Autowired[format=annotation]` and use it in your tests as you normally would when using `javax.annotation.processing.processor.webflux.test.autoconfigure.WebFluxTest[format=annotation]` and Spring REST Docs, as shown in the following example:

```
include-code::MyUsersDocumentationTests[]
```

If you require more control over Spring REST Docs configuration than offered by the attributes of `javax.annotation.processing.processor.restdocs.test.autoconfigure.AutoConfigureRestDocs[format=annotation]`, you can use a `javax.annotation.processing.processor.restdocs.test.autoconfigure.RestDocsWebTestClientConfigurationCustomizer[]` bean, as shown in the following example:

```
include-code::MyRestDocsConfiguration[]
```

If you want to make use of Spring REST Docs support for a parameterized output directory, you can use a `javax.annotation.processing.processor.boot.webtestclient.autoconfigure.WebTestClientBuilderCustomizer[]` to configure a consumer for every entity exchange result. The following example shows such a `javax.annotation.processing.processor.boot.webtestclient.autoconfigure.WebTestClientBuilderCustomizer[]` being defined:

```
include-code::MyWebTestClientBuilderCustomizerConfiguration[]
```

71.35. Auto-configured Spring Web Services Tests

71.35.1. Auto-configured Spring Web Services Client Tests

You can use `javadoc:org.springframework.boot.webservices.test.autoconfigure.client.WebServiceClientTest[format=annotation]` from the `spring-boot-webservices-test` module to test applications that call web services using the Spring Web Services project. By default, it configures a `javadoc:org.springframework.ws.test.client.MockWebServiceServer[]` bean and automatically customizes your `javadoc:org.springframework.boot.webservices.client.WebServiceTemplateBuilder[]`. (For more about using Web Services with Spring Boot, see [io/webservices.html](#).)



A list of the auto-configuration settings that are enabled by `javadoc:org.springframework.boot.webservices.test.autoconfigure.client.WebServiceClientTest[format=annotation]` can be [found in the appendix](#).

The following example shows the `javadoc:org.springframework.boot.webservices.test.autoconfigure.client.WebServiceClientTest[format=annotation]` annotation in use:

```
include-code::MyWebServiceClientTests[]
```

71.35.2. Auto-configured Spring Web Services Server Tests

You can use `javadoc:org.springframework.boot.webservices.test.autoconfigure.server.WebServiceServerTest[format=annotation]` from the `spring-boot-webservices-test` module to test applications that implement web services using the Spring Web Services project. By default, it configures a `javadoc:org.springframework.ws.test.server.MockWebServiceClient[]` bean that can be used to call your web service endpoints. (For more about using Web Services with Spring Boot, see [io/webservices.html](#).)



A list of the auto-configuration settings that are enabled by `javadoc:org.springframework.boot.webservices.test.autoconfigure.server.WebServiceServerTest[format=annotation]` can be [found in the appendix](#).

The following example shows the `javadoc:org.springframework.boot.webservices.test.autoconfigure.server.WebServiceServerTest[format=annotation]` annotation in use:

```
include-code::MyWebServiceServerTests[]
```

71.36. Additional Auto-configuration and Slicing

Each slice provides one or more `@AutoConfigure...` annotations that namely defines the auto-configurations that should be included as part of a slice. Additional auto-configurations can be added on a test-by-test basis by creating a custom `@AutoConfigure...` annotation or by adding `javadoc:org.springframework.boot.autoconfigure.ImportAutoConfiguration[format=annotation]` to the test as shown in the following example:

```
include-code::MyJdbcTests[]
```



Make sure to not use the regular `javadoc:org.springframework.context.annotation.Import[format=annotation]` annotation to import auto-configurations as they are handled in a specific way by Spring Boot.

Alternatively, additional auto-configurations can be added for any use of a slice annotation by registering them in a file stored in `META-INF/spring` as shown in the following example:

META-INF/spring/org.springframework.boot.jdbc.test.autoconfigure.JdbcTest.imports

```
com.example.IntegrationAutoConfiguration
```

In this example, the `com.example.IntegrationAutoConfiguration` is enabled on every test annotated with `javadoc:org.springframework.boot.jdbc.test.autoconfigure.JdbcTest[format=annotation]`.



You can use comments with `#` in this file.



A slice or `@AutoConfigure...` annotation can be customized this way as long as it is meta-annotated with `javadoc:org.springframework.boot.autoconfigure.ImportAutoConfiguration[format=annotation]`.

71.37. User Configuration and Slicing

If you [structure your code](#) in a sensible way, your `javadoc:org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` class is [used by default](#) as the configuration of your tests.

It then becomes important not to litter the application's main class with configuration settings that are specific to a particular area of its functionality.

Assume that you are using Spring Data MongoDB, you rely on the auto-configuration for it, and you have enabled auditing. You could define your `javadoc:org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` as follows:

```
include-code::MyApplication[]
```

Because this class is the source configuration for the test, any slice test actually tries to enable Mongo auditing, which is definitely not what you want to do. A recommended approach is to move that area-specific configuration to a separate `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class at the same level as your application, as shown in the following example:

```
include-code::MyMongoConfiguration[]
```



Depending on the complexity of your application, you may either have a single `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class for your customizations or one class per domain area. The latter approach lets you enable it in one of your tests, if necessary, with the `javadoc:org.springframework.context.annotation.Import[format=annotation]` annotation. See [this how-to section](#) for more details on when you might want to enable specific `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` classes for slice tests.

Test slices exclude `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` classes from scanning. For example, for a `javadoc:org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest[format=annotation]`, the following configuration will not include the given `javadoc:org.springframework.web.servlet.config.annotation.WebMvcConfigurer[]` bean in the application context loaded by the test slice:

```
include-code::MyWebConfiguration[]
```

The configuration below will, however, cause the custom `javadoc:org.springframework.web.servlet.config.annotation.WebMvcConfigurer[]` to be loaded by the test slice.

```
include-code::MyWebMvcConfigurer[]
```

Another source of confusion is classpath scanning. Assume that, while you structured your code in a sensible way, you need to scan an additional package. Your application may resemble the following code:

```
include-code::scan/MyApplication[]
```

Doing so effectively overrides the default component scan directive with the side effect of scanning those two packages regardless of the slice that you chose. For instance, a `javadoc:org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest[format=annotation]` seems to suddenly scan components and user configurations of your application. Again, moving the custom directive to a separate class is a good way to fix this issue.



If this is not an option for you, you can create a `javadoc:org.springframework.boot.SpringBootConfiguration[format=annotation]` somewhere in the hierarchy of your test so that it is used instead. Alternatively, you can specify a source for your test, which disables the behavior of finding a default one.

71.38. Using Spock to Test Spring Boot Applications

Spock 2.4 or later can be used to test a Spring Boot application. To do so, add a dependency on a `-groovy-5.0` version of Spock's `spock-spring` module to your application's build. `spock-spring` integrates Spring's test framework into Spock. See [the documentation for Spock's Spring module](https://spockframework.org/spock/docs/2.4/modules.html#spring-module) (https://spockframework.org/spock/docs/2.4/modules.html#spring-module) for further details.

72. Test Modules

Spring Boot offers several focused, feature-specific `-test` modules:

Module	Purpose
<code>spring-boot-cache-test</code>	Testing applications that use Spring Framework's cache abstraction.
<code>spring-boot-data-cassandra-test</code>	Testing applications that use Spring Data Cassandra. Provides the <code>@DataCassandraTest</code> test slice.
<code>spring-boot-data-couchbase-test</code>	Testing applications that use Spring Data Couchbase. Provides the <code>@DataCouchbaseTest</code> test slice.
<code>spring-boot-data-elasticsearch-test</code>	Testing applications that use Spring Data Elasticsearch. Provides the <code>@DataElasticsearchTest</code> test slice.
<code>spring-boot-data-jdbc-test</code>	Testing applications that use Spring Data JDBC. Provides the <code>@DataJdbcTest</code> test slice.
<code>spring-boot-data-jpa-test</code>	Testing applications that use Spring Data JPA. Provides the <code>@DataJpaTest</code> test slice.
<code>spring-boot-data-ldap-test</code>	Testing applications that use Spring Data LDAP. Provides the <code>@DataLdapTest</code> test slice.
<code>spring-boot-mongodb-test</code>	Testing applications that use Spring Data MongoDB. Provides the <code>@DataMongoTest</code> test slice.
<code>spring-boot-data-neo4j-test</code>	Testing applications that use Spring Data JPA. Provides the <code>@DataNeo4jTest</code> test slice.
<code>spring-boot-data-r2dbc-test</code>	Testing applications that use Spring Data R2DBC. Provides the <code>@DataR2dbcTest</code> test slice.
<code>spring-boot-data-redis-test</code>	Testing applications that use Spring Data Redis. Provides the <code>@DataRedisTest</code> test slice.
<code>spring-boot-graphql-test</code>	Testing applications that use Spring GraphQL. Provides the <code>@GraphQLTest</code> test slice.
<code>spring-boot-jdbc-test</code>	Testing applications that using Spring JDBC. Provides the <code>@JdbcTest</code> test slice.
<code>spring-boot-jooq-test</code>	Testing applications that using jOOQ. Provides the <code>@JooqTest</code> test slice.
<code>spring-boot-jpa-test</code>	Testing applications that use JPA.
<code>spring-boot-micrometer-metrics-test</code>	Testing applications that use Micrometer Metrics.
<code>spring-boot-micrometer-tracing-test</code>	Testing applications that use Micrometer Tracing.
<code>spring-boot-restclient-test</code>	Testing applications that use REST clients. Provides the <code>@RestClientTest</code> test slice.
<code>spring-boot-security-test</code>	Testing applications that use Spring Security.
<code>spring-boot-webserver-test</code>	Testing applications that use <code>WebServer</code> implementations. Provides the <code>@AutoconfigureWebServer</code> annotation.
<code>spring-boot-webclient-test</code>	Testing applications that use <code>WebClient</code> . Provides the <code>@WebClientTest</code> test slice.
<code>spring-boot-webflux-test</code>	Testing applications that use Spring WebFlux. Provides the <code>@WebFluxTest</code> test slice.
<code>spring-boot-webmvc-test</code>	Testing applications that use Spring Web MVC. Provides the <code>@WebMvcTest</code> test slice.
<code>spring-boot-webservices-test</code>	Testing applications that use Spring Web Services. Provides the <code>@WebServiceClientTest</code> and <code>@WebServiceServerTest</code> test slices.

73. Test Scope Dependencies

The `spring-boot-starter-test` starter (in the `test` scope) contains the following provided libraries:

- [JUnit](https://junit.org) (<https://junit.org>): The de-facto standard for unit testing Java applications.
- [Spring Test](https://docs.spring.io/spring-framework/docs/testing/integration.html) & Spring Boot Test: Utilities and integration test support for Spring Boot applications.
- [AssertJ](https://assertj.github.io/doc/) (<https://assertj.github.io/doc/>): A fluent assertion library.
- [Hamcrest](https://github.com/hamcrest/JavaHamcrest) (<https://github.com/hamcrest/JavaHamcrest>): A library of matcher objects (also known as constraints or predicates).
- [Mockito](https://site.mockito.org/) (<https://site.mockito.org/>): A Java mocking framework.
- [JSONassert](https://github.com/skyscreamer/JSONassert) (<https://github.com/skyscreamer/JSONassert>): An assertion library for JSON.
- [JsonPath](https://github.com/jayway/json-path) (<https://github.com/jayway/json-path>): XPath for JSON.
- [Awaitility](https://github.com/awaitility/awaitility) (<https://github.com/awaitility/awaitility>): A library for testing asynchronous systems.

We generally find these common libraries to be useful when writing tests. If these libraries do not suit your needs, you can add additional test dependencies of your own.

74. Test Utilities

A few test utility classes that are generally useful when testing your application are packaged as part of `spring-boot`.

74.1. ConfigDataApplicationContextInitializer

`javadoc:org.springframework.boot.test.context.ConfigDataApplicationContextInitializer[]` is an `javadoc:org.springframework.context.ApplicationContextInitializer[]` that you can apply to your tests to load Spring Boot `application.properties` files. You can use it when you do not need the full set of features provided by `javadoc:org.springframework.boot.test.context.SpringBootTest[format=annotation]`, as shown in the following example:

include-code::MyConfigFileTests[]



Using `javadoc:org.springframework.boot.test.context.ConfigDataApplicationContextInitializer[]` alone does not provide support for `@Value("${...}")` injection. Its only job is to ensure that `application.properties` files are loaded into Spring's `javadoc:org.springframework.core.env.Environment[]`. For `javadoc:org.springframework.beans.factory.annotation.Value[format=annotation]` support, you need to either additionally configure a `javadoc:org.springframework.context.support.PropertySourcesPlaceholderConfigurer[]` or use `javadoc:org.springframework.boot.test.context.SpringBootTest[format=annotation]`, which auto-configures one for you.

74.2. TestPropertyValues

`javadoc:org.springframework.boot.test.util.TestPropertyValues[]` lets you quickly add properties to a `javadoc:org.springframework.core.env.ConfigurableEnvironment[]` or `javadoc:org.springframework.context.ConfigurableApplicationContext[]`. You can call it with `key=value` strings, as follows:

include-code::MyEnvironmentTests[]

74.3. OutputCaptureExtension

`javadoc:org.springframework.boot.test.system.OutputCaptureExtension[]` is a JUnit `javadoc:org.junit.jupiter.api.extension.Extension[]` that you can use to capture `javadoc:java.lang.System#out[]` and `javadoc:java.lang.System#err[]` output. To use it, add `@ExtendWith(OutputCaptureExtension.class)` and inject `javadoc:org.springframework.boot.test.system.CapturedOutput[]` as an argument to your test class constructor or test method as follows:

include-code::MyOutputCaptureTests[]

74.4. TestRestTemplate

`javadoc:org.springframework.boot.test.resttestclient.TestRestTemplate[]` is a convenience alternative to Spring's `javadoc:org.springframework.web.client.RestTemplate[]` that is useful in integration tests. It's provided by the `spring-boot-resttestclient` module. A dependency on `spring-boot-restclient` is also required. Take care when adding this dependency as it will enable auto-configuration for `RestClient.Builder`. If your main code uses `RestClient.Builder`, declare the `spring-boot-restclient` dependency so that it is on your application's main classpath and not only on its test classpath.

You can get a vanilla template or one that sends Basic HTTP authentication (with a username and password). In either case, the template is fault tolerant. This means that it behaves in a test-friendly way by not throwing exceptions on 4xx and 5xx errors. Instead, such errors can be detected through the returned `javadoc:org.springframework.http.ResponseEntity[]` and its status code.

If you need fluent API for assertions, consider using `javadoc:org.springframework.test.web.servlet.client.RestTestClient[]` that works with [mock environments](#) and [end-to-end tests](#).

If you are using Spring WebFlux, consider the `javadoc:org.springframework.test.web.reactive.server.WebTestClient[]` that provides a similar API and works with [mock environments](#), [WebFlux integration tests](#), and [end-to-end tests](#).

It is recommended, but not mandatory, to use the Apache HTTP Client (version 5.1 or better). If you have that on your classpath, the `javadoc:org.springframework.boot.test.resttestclient.TestRestTemplate[]` responds by configuring the client appropriately. If you do use Apache's HTTP client it is configured to ignore cookies (so the template is stateless).

`javadoc:org.springframework.boot.test.resttestclient.TestRestTemplate[]` can be instantiated directly in your integration tests, as shown in the following example:

include-code::MyTests[]

Alternatively, if you use the `javadoc:org.springframework.boot.test.context.SpringBootTest[format=annotation]` annotation with `WebEnvironment.RANDOM_PORT` or `WebEnvironment.DEFINED_PORT`, you can inject a fully configured `javadoc:org.springframework.boot.test.resttestclient.TestRestTemplate[]` by annotating the test class with `javadoc:org.springframework.boot.test.resttestclient.autoconfigure.AutoConfigureTestRestTemplate[format=annotation]`. If necessary, additional customizations can be applied through the `javadoc:org.springframework.boot.testclient.RestTemplateBuilder[]` bean.

Any URLs that do not specify a host and port automatically connect to the embedded server, as shown in the following example:

include-code::MySpringBootTest[]

75. Testcontainers

The [Testcontainers](https://www.testcontainers.org/) (<https://www.testcontainers.org/>) library provides a way to manage services running inside Docker containers. It integrates with JUnit, allowing you to write a test class that can start up a container before any of the tests run. Testcontainers is especially useful for writing integration tests that talk to a real backend service such as MySQL, MongoDB, Cassandra and others.

In following sections we will describe some of the methods you can use to integrate Testcontainers with your tests.

75.1. Using Spring Beans

The containers provided by Testcontainers can be managed by Spring Boot as beans.

To declare a container as a bean, add a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` method to your test configuration:

```
include-code::MyTestConfiguration[]
```

You can then inject and use the container by importing the configuration class in the test class:

```
include-code::MyIntegrationTests[]
```



This method of managing containers is often used in combination with service connection annotations.

75.2. Using the JUnit Extension

Testcontainers provides a JUnit extension which can be used to manage containers in your tests. The extension is activated by applying the `javadoc:org.testcontainers.junit.jupiter.Testcontainers[format=annotation]` annotation from Testcontainers to your test class.

You can then use the `javadoc:org.testcontainers.junit.jupiter.Container[format=annotation]` annotation on static container fields.

The `javadoc:org.testcontainers.junit.jupiter.Testcontainers[format=annotation]` annotation can be used on vanilla JUnit tests, or in combination with `javadoc:org.springframework.boot.test.context.SpringBootTest[format=annotation]`:

```
include-code::MyIntegrationTests[]
```

The example above will start up a Neo4j container before any of the tests are run. The lifecycle of the container instance is managed by Testcontainers, as described in [{url-testcontainers-docs}/test_framework_integration/junit_5/#extension](#) [their official documentation].



In most cases, you will additionally need to configure the application to connect to the service running in the container.

75.3. Importing Container Configuration Interfaces

A common pattern with Testcontainers is to declare the container instances as static fields in an interface.

For example, the following interface declares two containers, one named `mongo` of type `javadoc:{url-testcontainers-mongodb-javadoc}/org.testcontainers.mongodb.MongoDBContainer[]` and another named `neo4j` of type `javadoc:{url-testcontainers-neo4j-javadoc}/org.testcontainers.neo4j.Neo4jContainer[]`:

```
include-code::MyContainers[]
```

When you have containers declared in this way, you can reuse their configuration in multiple tests by having the test classes implement the interface.

It's also possible to use the same interface configuration in your Spring Boot tests. To do so, add `javadoc:org.springframework.boot.testcontainers.context.ImportTestcontainers[format=annotation]` to your test configuration class:

```
include-code::MyTestConfiguration[]
```

75.4. Lifecycle of Managed Containers

If you have used the annotations and extensions provided by Testcontainers, then the lifecycle of container instances is managed entirely by Testcontainers. Please refer to the [{url-testcontainers-docs}\[official Testcontainers documentation\]](#) for the information.

When the containers are managed by Spring as beans, then their lifecycle is managed by Spring:

- Container beans are created and started before all other beans.
- Container beans are stopped after the destruction of all other beans.

This process ensures that any beans, which rely on functionality provided by the containers, can use those functionalities. It also ensures that they are cleaned up whilst the container is still available.



When your application beans rely on functionality of containers, prefer configuring the containers as Spring beans to ensure the correct lifecycle behavior.



Having containers managed by Testcontainers instead of as Spring beans provides no guarantee of the order in which beans and containers will shutdown. It can happen that containers are shutdown before the beans relying on container functionality are cleaned up. This can lead to exceptions being thrown by client beans, for example, due to loss of connection.

Container beans are created and started once per application context managed by Spring's TestContext Framework. For details about how TestContext Framework manages the underlying application contexts and beans therein, please refer to the {url-spring-framework-docs}[Spring Framework documentation].

Container beans are stopped as part of the TestContext Framework's standard application context shutdown process. When the application context gets shutdown, the containers are shutdown as well. This usually happens after all tests using that specific cached application context have finished executing. It may also happen earlier, depending on the caching behavior configured in TestContext Framework.



A single test container instance can, and often is, retained across execution of tests from multiple test classes.

75.5. Service Connections

A service connection is a connection to any remote service. Spring Boot's auto-configuration can consume the details of a service connection and use them to establish a connection to a remote service. When doing so, the connection details take precedence over any connection-related configuration properties.

When using Testcontainers, connection details can be automatically created for a service running in a container by annotating the container field in the test class.

include-code::MyIntegrationTests[]

Thanks to javadoc:org.springframework.boot.testcontainers.service.connection.ServiceConnection[format=annotation], the above configuration allows Neo4j-related beans in the application to communicate with Neo4j running inside the Testcontainers-managed Docker container. This is done by automatically defining a javadoc:org.springframework.boot.neo4j.autoconfigure.Neo4jConnectionDetails[] bean which is then used by the Neo4j auto-configuration, overriding any connection-related configuration properties.



You'll need to add the `spring-boot-testcontainers` module as a test dependency in order to use service connections with Testcontainers.

Service connection annotations are processed by javadoc:org.springframework.boot.testcontainers.service.connection.ContainerConnectionDetailsFactory[] classes registered with `spring.factories` . A javadoc:org.springframework.boot.testcontainers.service.connection.ContainerConnectionDetailsFactory[] can create a javadoc:org.springframework.boot.autoconfigure.service.connection.ConnectionDetails[] bean based on a specific javadoc:org.testcontainers.containers.Container[] subclass, or the Docker image name.

The following service connection factories are provided in the `spring-boot-testcontainers` jar:

Connection Details	Matched on
javadoc:org.springframework.boot.activemq.autoconfigure.ActiveMQConnectionDetails[]	Containers named "symptoma/activemq" or javadoc:org.testcontainers.activemq.ActiveMQContainer[]
javadoc:org.springframework.boot.artemis.autoconfigure.ArtemisConnectionDetails[]	Containers of type javadoc:org.testcontainers.activemq.ArtemisContainer[]
javadoc:org.springframework.boot.cassandra.autoconfigure.CassandraConnectionDetails[]	Containers of type javadoc:org.testcontainers.cassandra.CassandraContainer[]
javadoc:org.springframework.boot.couchbase.autoconfigure.CouchbaseConnectionDetails[]	Containers of type javadoc:org.testcontainers.couchbase.CouchbaseContainer[]
javadoc:org.springframework.boot.elasticsearch.autoconfigure.ElasticsearchConnectionDetails[]	Containers of type javadoc:org.testcontainers.elasticsearch.ElasticsearchContainer[]
javadoc:org.springframework.boot.flyway.autoconfigure.FlywayConnectionDetails[]	Containers of type javadoc:{url-testcontainers-jdbc-javadoc}/org.testcontainers.containers.JdbcDatabaseContainer[]
javadoc:org.springframework.boot.jdbc.autoconfigure.JdbcConnectionDetails[]	Containers of type javadoc:{url-testcontainers-jdbc-javadoc}/org.testcontainers.containers.JdbcDatabaseContainer[]
javadoc:org.springframework.boot.kafka.autoconfigure.KafkaConnectionDetails[]	Containers of type javadoc:org.testcontainers.kafka.KafkaContai javadoc:org.testcontainers.kafka.ConfluentKafkaContainer[] or javadoc:org.testcontainers.redpanda.RedpandaContainer[]
javadoc:org.springframework.boot.ldap.autoconfigure.LdapConnectionDetails[]	Containers named "osixia/openldap" or of type javadoc:org.testcontainers.ldap.LLdapContainer[]
javadoc:org.springframework.boot.liquibase.autoconfigure.LiquibaseConnectionDetails[]	Containers of type javadoc:{url-testcontainers-jdbc-javadoc}/org.testcontainers.containers.JdbcDatabaseContainer[]

Connection Details	Matched on
<code>javadoc:org.springframework.boot.mongodb.autoconfigure.MongoConnectionDetails[]</code>	Containers of type <code>javadoc:{url-testcontainers-mongodb-javadoc}/org.testcontainers.mongodb.MongoDBContainer[]</code> or <code>javadoc:{url-testcontainers-mongodb-javadoc}/org.testcontainers.mongodb.MongoDBAtlasLocalConta</code>
<code>javadoc:org.springframework.boot.neo4j.autoconfigure.Neo4jConnectionDetails[]</code>	Containers of type <code>javadoc:{url-testcontainers-neo4j-javadoc}/org.testcontainers.neo4j.Neo4jContainer[]</code>
<code>javadoc:org.springframework.boot.opentelemetry.autoconfigure.logging.otlp.OtlpLoggingConnectionDetails[]</code>	Containers named "otel/opentelemetry-collector-contrib" or of ty <code>javadoc:org.testcontainers.grafana.LgtmStackContainer[]</code>
<code>javadoc:org.springframework.boot.micrometer.metrics.autoconfigure.export.otlp.OtlpMetricsConnectionDetails[]</code>	Containers named "otel/opentelemetry-collector-contrib" or of ty <code>javadoc:org.testcontainers.grafana.LgtmStackContainer[]</code>
<code>javadoc:org.springframework.boot.micrometer.tracing.opentelemetry.autoconfigure.otlp.OtlpTracingConnectionDetails[]</code>	Containers named "otel/opentelemetry-collector-contrib" or of ty <code>javadoc:org.testcontainers.grafana.LgtmStackContainer[]</code>
<code>javadoc:org.springframework.boot.pulsar.autoconfigure.PulsarConnectionDetails[]</code>	Containers of type <code>javadoc:{url-testcontainers-pulsar-javadoc}/org.testcontainers.pulsar.PulsarContainer[]</code>
<code>javadoc:org.springframework.boot.r2dbc.autoconfigure.R2dbcConnectionDetails[]</code>	Containers of type <code>javadoc:org.testcontainers.clickhouse.ClickHouseContainer[], javadoc:{url-testcontainers-mariadb-javadoc}/org.testcontainers.mariadb.MariaDBContainer[], javadoc:org.testcontainers.mssqlserver.MSSQLServerContainer javadoc:{url-testcontainers-mysql-javadoc}/org.testcontainers.mysql.MySQLContainer[], javadoc:org.testcontainers.oracle.OracleContainer[OracleContai (free)], javadoc:{url-testcontainers-oracle-xe-javadoc}/org.testcontainers.containers.OracleContainer[OracleC (XE)] or javadoc:{url-testcontainers-postgresql-javadoc}/org.testcontainers.postgresql.PostgreSQLContainer[]</code>
<code>javadoc:org.springframework.boot.amqp.autoconfigure.RabbitConnectionDetails[]</code>	Containers of type <code>javadoc:{url-testcontainers-rabbitmq-javadoc}/org.testcontainers.rabbitmq.RabbitMQContainer[]</code>
<code>javadoc:org.springframework.boot.data.redis.autoconfigure.DataRedisConnectionDetails[]</code>	Containers of type <code>javadoc:com.redis.testcontainers.RedisContai javadoc:com.redis.testcontainers.RedisStackContainer[],</code> or cont named "redis", "redis/redis-stack" or "redis/redis-stack-server"
<code>javadoc:org.springframework.boot.zipkin.autoconfigure.ZipkinConnectionDetails[]</code>	Containers named "openzipkin/zipkin"



By default all applicable connection details beans will be created for a given `javadoc:org.testcontainers.containers.Container[]`. For example, a `javadoc:{url-testcontainers-postgresql-javadoc}/org.testcontainers.postgresql.PostgreSQLContainer[]` will create both `javadoc:org.springframework.boot.jdbc.autoconfigure.JdbcConnectionDetails[]` and `javadoc:org.springframework.boot.r2dbc.autoconfigure.R2dbcConnectionDetails[]`.

If you want to create only a subset of the applicable types, you can use the `type` attribute of `javadoc:org.springframework.boot.testcontainers.service.connection.ServiceConnection[format=annotation]`.

By default `Container.getDockedImageName().getRepository()` is used to obtain the name used to find connection details. The repository portion of the Docker image name ignores any registry and the version. This works as long as Spring Boot is able to get the instance of the `javadoc:org.testcontainers.containers.Container[]`, which is the case when using a `static` field like in the example above.

If you're using a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` method, Spring Boot won't call the bean method to get the Docker image name, because this would cause eager initialization issues. Instead, the return type of the bean method is used to find out which connection detail should be used. This works as long as you're using typed containers such as `javadoc:{url-testcontainers-neo4j-javadoc}/org.testcontainers.neo4j.Neo4jContainer[]` or `javadoc:{url-testcontainers-rabbitmq-javadoc}/org.testcontainers.rabbitmq.RabbitMQContainer[]`. This stops working if you're using `javadoc:org.testcontainers.containers.GenericContainer[]`, for example with Redis as shown in the following example:

```
include-code::MyRedisConfiguration[]
```

Spring Boot can't tell from `javadoc:org.testcontainers.containers.GenericContainer[]` which container image is used, so the `name` attribute from `javadoc:org.springframework.boot.testcontainers.service.connection.ServiceConnection[format=annotation]` must be used to provide that hint.

You can also use the `name` attribute of `javadoc:org.springframework.boot.testcontainers.service.connection.ServiceConnection[format=annotation]` to override which connection detail will be used, for example when using custom images. If you are using the Docker image `registry.mycompany.com/mirror/myredis`, you'd use `@ServiceConnection(name="redis")` to ensure `javadoc:org.springframework.boot.data.redis.autoconfigure.DataRedisConnectionDetails[]` are created.

75.5.1. SSL with Service Connections

You can use the `javax.annotation.processing.processor.Path` annotation, `javax.annotation.processing.processor.Path`, `javax.annotation.processing.processor.Path`, `javax.annotation.processing.processor.Path` and `javax.annotation.processing.processor.Path` annotations on a supported container to enable SSL support for that service connection. Please note that you still have to enable SSL on the service which is running inside the Testcontainer yourself, the annotations only configure SSL on the client side in your application.

```
include-code::MyRedisWithSslIntegrationTests[]
```

The above code uses the `javax.annotation.processing.processor.Path` annotation to load the client certificate and key into the keystore and the `javax.annotation.processing.processor.Path` annotation to load the CA certificate into the truststore. This will authenticate the client against the server, and the CA certificate in the truststore makes sure that the server certificate is valid and trusted.

The `SecureRedisContainer` in this example is a custom subclass of `RedisContainer` which copies certificates to the correct places and invokes `redis-server` with commandline parameters enabling SSL.

The SSL annotations are supported for the following service connections:

- Cassandra
- Couchbase
- Elasticsearch
- Kafka
- MongoDB
- RabbitMQ
- Redis

The `ElasticsearchContainer` additionally supports automatic detection of server side SSL. To use this feature, annotate the container with `javax.annotation.processing.processor.Path`, as seen in the following example, and Spring Boot takes care of the client side SSL configuration for you:

```
include-code::MyElasticsearchWithSslIntegrationTests[]
```

75.6. Dynamic Properties

A slightly more verbose but also more flexible alternative to service connections is `javax.annotation.processing.processor.Path`. A static `javax.annotation.processing.processor.Path` method allows adding dynamic property values to the Spring Environment.

```
include-code::MyIntegrationTests[]
```

The above configuration allows Neo4j-related beans in the application to communicate with Neo4j running inside the Testcontainers-managed Docker container.

76. Auto-configuration

Spring Boot auto-configuration attempts to automatically configure your Spring application based on the jar dependencies that you have added. For example, if `HSQLDB` is on your classpath, and you have not manually configured any database connection beans, then Spring Boot auto-configures an in-memory database.

You need to opt-in to auto-configuration by adding the `javax.annotation.processing.processor.Path[format=annotation]` or `javax.annotation.processing.processor.Path[format=annotation]` annotations to one of your `javax.annotation.processing.processor.Path[format=annotation]` classes.



You should only ever add one `javax.annotation.processing.processor.Path[format=annotation]` or `javax.annotation.processing.processor.Path[format=annotation]` annotation. We generally recommend that you add one or the other to your primary `javax.annotation.processing.processor.Path[format=annotation]` class only.

76.1. Gradually Replacing Auto-configuration

Auto-configuration is non-invasive. At any point, you can start to define your own configuration to replace specific parts of the auto-configuration. For example, if you add your own `javax.sql.DataSource[]` bean, the default embedded database support backs away.

If you need to find out what auto-configuration is currently being applied, and why, start your application with the `--debug` switch. Doing so enables debug logs for a selection of core loggers and logs a conditions report to the console.

76.2. Disabling Specific Auto-configuration Classes

If you find that specific auto-configuration classes that you do not want are being applied, you can use the `exclude` attribute of `javax.annotation.processing.processor.Path[format=annotation]` to disable them, as shown in the following example:

```
include-code::MyApplication[]
```

If the class is not on the classpath, you can use the `excludeName` attribute of the annotation and specify the fully qualified name instead. If you prefer to use `javax.annotation.processing.processor.Path[format=annotation]` rather than `javax.annotation.processing.processor.Path[format=annotation]`, `exclude` and `excludeName` are also available. Finally, you can also control the list of auto-configuration classes to exclude by using the `configprop:spring.autoconfigure.exclude[]` property.



You can define exclusions both at the annotation level and by using the property.



Even though auto-configuration classes are `public`, the only aspect of the class that is considered public API is the name of the class which can be used for disabling the auto-configuration. The actual contents of those classes, such as nested configuration classes or bean methods are for internal use only and we do not recommend using those directly.

76.3. Auto-configuration Packages

Auto-configuration packages are the packages that various auto-configured features look in by default when scanning for things such as entities and Spring Data repositories. The `javax.annotation.processing.processor.Path[format=annotation]` annotation (either directly or through its presence on `javax.annotation.processing.processor.Path[format=annotation]`) determines the default auto-configuration package. Additional packages can be configured using the `javax.annotation.processing.processor.Path[format=annotation]` annotation.

77. Build Systems

It is strongly recommended that you choose a build system that supports [dependency management](#) and that can consume artifacts published to the Maven Central repository. We would recommend that you choose Maven or Gradle. It is possible to get Spring Boot to work with other build systems (Ant, for example), but they are not particularly well supported.

77.1. Dependency Management

Each release of Spring Boot provides a curated list of dependencies that it supports. In practice, you do not need to provide a version for any of these dependencies in your build configuration, as Spring Boot manages that for you. When you upgrade Spring Boot itself, these dependencies are upgraded as well in a consistent way.



You can still specify a version and override Spring Boot's recommendations if you need to do so.

The curated list contains all the Spring modules that you can use with Spring Boot as well as a refined list of third party libraries. The list is available as a standard Bills of Materials (`spring-boot-dependencies`) that can be used with both [Maven](#) and [Gradle](#).



Each release of Spring Boot is associated with a base version of the Spring Framework. We **highly** recommend that you do not specify its version.

77.2. Maven

To learn about using Spring Boot with Maven, see the documentation for Spring Boot's Maven plugin:

- [Reference](#)
- [API](#)

77.3. Gradle

To learn about using Spring Boot with Gradle, see the documentation for Spring Boot's Gradle plugin:

- [Reference](#)
- [API](#)

77.4. Ant

It is possible to build a Spring Boot project using Apache Ant+Ivy. The `spring-boot-antlib` “AntLib” module is also available to help Ant create executable jars.

To declare dependencies, a typical `ivy.xml` file looks something like the following example:

```
<ivy-module version="2.0">
  <info organisation="org.springframework.boot" module="spring-boot-sample-ant" />
  <configurations>
    <conf name="compile" description="everything needed to compile this module" />
    <conf name="runtime" extends="compile" description="everything needed to run this module" />
  </configurations>
  <dependencies>
    <dependency org="org.springframework.boot" name="spring-boot-starter"
      rev="{spring-boot.version}" conf="compile" />
  </dependencies>
</ivy-module>
```

XML

A typical `build.xml` looks like the following example:

```

<project
  xmlns:ivy="antlib:org.apache.ivy.ant"
  xmlns:spring-boot="antlib:org.springframework.boot.ant"
  name="myapp" default="build">

  <property name="spring-boot.version" value="{version-spring-boot}" />

  <target name="resolve" description="--> retrieve dependencies with ivy">
    <ivy:retrieve pattern="lib/[conf]/[artifact]-[type]-[revision].[ext]" />
  </target>

  <target name="classpaths" depends="resolve">
    <path id="compile.classpath">
      <fileset dir="lib/compile" includes="*.jar" />
    </path>
  </target>

  <target name="init" depends="classpaths">
    <mkdir dir="build/classes" />
  </target>

  <target name="compile" depends="init" description="compile">
    <javac srcdir="src/main/java" destdir="build/classes" classpathref="compile.classpath" />
  </target>

  <target name="build" depends="compile">
    <spring-boot:exejar destfile="build/myapp.jar" classes="build/classes">
      <spring-boot:lib>
        <fileset dir="lib/runtime" />
      </spring-boot:lib>
    </spring-boot:exejar>
  </target>
</project>

```



If you do not want to use the `spring-boot-antlib` module, see the [how-to:build.html](#) section of “How-to Guides”.

77.5. Starters

Starters are a set of convenient dependency descriptors that you can include in your application. You get a one-stop shop for all the Spring and related technologies that you need without having to hunt through sample code and copy-paste loads of dependency descriptors. For example, if you want to get started using Spring and JPA for database access, include the `spring-boot-starter-data-jpa` dependency in your project.

The starters contain a lot of the dependencies that you need to get a project up and running quickly and with a consistent, supported set of managed transitive dependencies.

What is in a name

All **official** starters follow a similar naming pattern; `spring-boot-starter-*`, where `*` is a particular type of application. This naming structure is intended to help when you need to find a starter. The Maven integration in many IDEs lets you search dependencies by name. For example, with the appropriate Eclipse or Spring Tools plugin installed, you can press `ctrl+space` in the POM editor and type “spring-boot-starter” for a complete list.

As explained in the [features/developing-auto-configuration.html](#) section, third party starters should not start with `spring-boot`, as it is reserved for official Spring Boot artifacts. Rather, a third-party starter typically starts with the name of the project. For example, a third-party starter project called `thirdpartyproject` would typically be named `thirdpartyproject-spring-boot-starter`.

The following application starters are provided by Spring Boot under the `org.springframework.boot` group:

Spring Boot application starters

`partial$starters/application-starters.adoc`

In addition to the application starters, the following starters can be used to add [production ready](#) features:

Spring Boot production starters

`partial$starters/production-starters.adoc`

Finally, Spring Boot also includes the following starters that can be used if you want to exclude or swap specific technical facets:

Spring Boot technical starters

`partial$starters/technical-starters.adoc`

To learn how to swap technical facets, please see the how-to documentation for [swapping web server](#) and [logging system](#).



For a list of additional community contributed starters, see the `{code-spring-boot-latest}/starter/README.adoc` [README file] in the `spring-boot-starters` module on GitHub.

78. Configuration Classes

Spring Boot favors Java-based configuration. Although it is possible to use `javado:org.springframework.boot.SpringApplication[]` with XML sources, we generally recommend that your primary source be a single `javado:org.springframework.context.annotation.Configuration[format=annotation]` class. Usually the class that defines the `main` method is a good candidate as the primary `javado:org.springframework.context.annotation.Configuration[format=annotation]`.



Many Spring configuration examples have been published on the Internet that use XML configuration. If possible, always try to use the equivalent Java-based configuration. Searching for **Enable*** annotations can be a good starting point.

78.1. Importing Additional Configuration Classes

You need not put all your `javado:org.springframework.context.annotation.Configuration[format=annotation]` into a single class. The `javado:org.springframework.context.annotation.Import[format=annotation]` annotation can be used to import additional configuration classes. Alternatively, you can use `javado:org.springframework.context.annotation.ComponentScan[format=annotation]` to automatically pick up all Spring components, including `javado:org.springframework.context.annotation.Configuration[format=annotation]` classes.

78.2. Importing XML Configuration

If you absolutely must use XML based configuration, we recommend that you still start with a `javado:org.springframework.context.annotation.Configuration[format=annotation]` class. You can then use an `javado:org.springframework.context.annotation.ImportResource[format=annotation]` annotation to load XML configuration files.

79. Developer Tools

Spring Boot includes an additional set of tools that can make the application development experience a little more pleasant. The `spring-boot-devtools` module can be included in any project to provide additional development-time features. To include devtools support, add the module dependency to your build, as shown in the following listings for Maven and Gradle:

Maven

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

XMIL

Gradle

```
dependencies {
    developmentOnly("org.springframework.boot:spring-boot-devtools")
}
```

GRADLE



Devtools might cause classloading issues, in particular in multi-module projects. [using/devtools.html](#) explains how to diagnose and solve them.



Developer tools are automatically disabled when running a fully packaged application. If your application is launched from `java -jar` or if it is started from a special classloader, then it is considered a “production application”. You can control this behavior by using the `spring.devtools.restart.enabled` system property. To enable devtools, irrespective of the classloader used to launch your application, set the `-Dspring.devtools.restart.enabled=true` system property. This must not be done in a production environment where running devtools is a security risk. To disable devtools, exclude the dependency or set the `-Dspring.devtools.restart.enabled=false` system property.



Flagging the dependency as optional in Maven or using the `developmentOnly` configuration in Gradle (as shown above) prevents devtools from being transitively applied to other modules that use your project.



Repackaged archives do not contain devtools by default. If you want to use a [certain remote devtools feature](#), you need to include it. When using the Maven plugin, opt-in for optional dependencies by setting the `includeOptional` property to `true`. You also need to set the `excludeDevtools` property to `false`. When using the Gradle plugin, [configure the task's classpath to include the developmentOnly configuration](#).

79.1. Diagnosing Classloading Issues

As described in the Restart vs Reload section, restart functionality is implemented by using two classloaders. For most applications, this approach works well. However, it can sometimes cause classloading issues, in particular in multi-module projects.

To diagnose whether the classloading issues are indeed caused by devtools and its two classloaders, [try disabling restart](#). If this solves your problems, [customize the restart classloader](#) to include your entire project.

79.2. Property Defaults

Several of the libraries supported by Spring Boot use caches to improve performance. For example, [template engines](#) cache compiled templates to avoid repeatedly parsing template files. Also, Spring MVC can add HTTP caching headers to responses when serving static resources.

While caching is very beneficial in production, it can be counter-productive during development, preventing you from seeing the changes you just made in your application. For this reason, `spring-boot-devtools` disables the caching options by default.

Cache options are usually configured by settings in your `application.properties` file. For example, Thymeleaf offers the `configprop:spring.thymeleaf.cache[]` property.

The same applies for tracing probability that's set to 100% as the default may not log all traces used for testing.

Rather than needing to set these properties manually, the `spring-boot-devtools` module automatically applies sensible development-time configuration.

The following table lists all the properties that are applied:

partial\$propertydefaults/devtools-property-defaults.adoc



If you do not want property defaults to be applied you can set `configprop:spring.devtools.add-properties[]` to `false` in your `application.properties`.

Because you need more information about web requests while developing Spring MVC and Spring WebFlux applications, developer tools suggests you to enable `DEBUG` logging for the `web` logging group. This will give you information about the incoming request, which handler is processing it, the response outcome, and other details. If you wish to log all request details (including potentially sensitive information), you can turn on the `configprop:spring.mvc.log-request-details[]` or `configprop:spring.http.codecs.log-request-details[]` configuration properties.

79.3. Automatic Restart

Applications that use `spring-boot-devtools` automatically restart whenever files on the classpath change. This can be a useful feature when working in an IDE, as it gives a very fast feedback loop for code changes. By default, any entry on the classpath that points to a directory is monitored for changes. Note that certain resources, such as static assets and view templates, do not need to restart the application.

Triggering a restart

As DevTools monitors classpath resources, the only way to trigger a restart is to update the classpath. Whether you're using an IDE or one of the build plugins, the modified files have to be recompiled to trigger a restart. The way in which you cause the classpath to be updated depends on the tool that you are using:

- In Eclipse, saving a modified file causes the classpath to be updated and triggers a restart.
- In IntelliJ IDEA, building the project (**Build -> Build Project**) has the same effect.
- If using a build plugin, running `mvn compile` for Maven or `gradle build` for Gradle will trigger a restart.



If you are restarting with Maven or Gradle using the build plugin you must leave the `forking` set to `enabled`. If you disable forking, the isolated application classloader used by devtools will not be created and restarts will not operate properly.



Automatic restart works very well when used with LiveReload. See the [using/devtools.html](#) section for details. If you use JRebel, automatic restarts are disabled in favor of dynamic class reloading. Other devtools features (such as LiveReload and property overrides) can still be used.



DevTools relies on the application context's shutdown hook to close it during a restart. It does not work correctly if you have disabled the shutdown hook (`SpringApplication.setRegisterShutdownHook(false)`).



DevTools needs to customize the `javadoc:org.springframework.core.io.ResourceLoader[]` used by the `javadoc:org.springframework.context.ApplicationContext[]`. If your application provides one already, it is going to be wrapped. Direct override of the `getResource` method on the `javadoc:org.springframework.context.ApplicationContext[]` is not supported.



Automatic restart is not supported when using AspectJ weaving.

Restart vs Reload

The restart technology provided by Spring Boot works by using two classloaders. Classes that do not change (for example, those from third-party jars) are loaded into a *base* classloader. Classes that you are actively developing are loaded into a *restart* classloader. When the application is restarted, the *restart* classloader is thrown away and a new one is created. This approach means that application restarts are typically much faster than “cold starts”, since the *base* classloader is already available and populated.

If you find that restarts are not quick enough for your applications or you encounter classloading issues, you could consider reloading technologies such as [JRebel](#) (<https://jrebel.com/software/jrebel/>) from ZeroTurnaround. These work by rewriting classes as they are loaded to make them more amenable to reloading.

79.3.1. Logging Changes in Condition Evaluation

By default, each time your application restarts, a report showing the condition evaluation delta is logged. The report shows the changes to your application's auto-configuration as you make changes such as adding or removing beans and setting configuration properties.

To disable the logging of the report, set the following property:

```
spring:
  devtools:
    restart:
      log-condition-evaluation-delta: false
```

79.3.2. Excluding Resources

Certain resources do not necessarily need to trigger a restart when they are changed. For example, Thymeleaf templates can be edited in-place. By default, changing resources in `/META-INF/maven`, `/META-INF/resources`, `/resources`, `/static`, `/public`, or `/templates` does not trigger a restart but does trigger a [live reload](#). If you want to customize these exclusions, you can use the `configprop:spring.devtools.restart.exclude[]` property. For example, to exclude only `/static` and `/public` you would set the following property:

```
spring:
  devtools:
    restart:
      exclude: "static/**,public/**"
```



If you want to keep those defaults and *add* additional exclusions, use the `configprop:spring.devtools.restart.additional-exclude[]` property instead.

79.3.3. Watching Additional Paths

You may want your application to be restarted or reloaded when you make changes to files that are not on the classpath. To do so, use the `configprop:spring.devtools.restart.additional-paths[]` property to configure additional paths to watch for changes. You can use the `configprop:spring.devtools.restart.exclude[]` property [described earlier](#) to control whether changes beneath the additional paths trigger a full restart or a [live reload](#).

79.3.4. Disabling Restart

If you do not want to use the restart feature, you can disable it by using the `configprop:spring.devtools.restart.enabled[]` property. In most cases, you can set this property in your `application.properties` (doing so still initializes the restart classloader, but it does not watch for file changes).

If you need to *completely* disable restart support (for example, because it does not work with a specific library), you need to set the `configprop:spring.devtools.restart.enabled[]` `javadoc:java.lang.System[]` property to `false` before calling `SpringApplication.run(...)`, as shown in the following example:

```
include-code::MyApplication[]
```

79.3.5. Using a Trigger File

If you work with an IDE that continuously compiles changed files, you might prefer to trigger restarts only at specific times. To do so, you can use a “trigger file”, which is a special file that must be modified when you want to actually trigger a restart check.



Any update to the file will trigger a check, but restart only actually occurs if Devtools has detected it has something to do.

To use a trigger file, set the `configprop:spring.devtools.restart.trigger-file[]` property to the name (excluding any path) of your trigger file. The trigger file must appear somewhere on your classpath.

For example, if you have a project with the following structure:

```
src
+- main
  +- resources
    +- .reloadtrigger
```

Then your `trigger-file` property would be:

```
spring:
  devtools:
    restart:
      trigger-file: ".reloadtrigger"
```

Restarts will now only happen when the `src/main/resources/.reloadtrigger` is updated.



You might want to set `spring.devtools.restart.trigger-file` as a [global setting](#), so that all your projects behave in the same way.

Some IDEs have features that save you from needing to update your trigger file manually. [Spring Tools for Eclipse](https://spring.io/tools) (<https://spring.io/tools>) and [IntelliJ IDEA \(Ultimate Edition\)](https://www.jetbrains.com/idea/) (<https://www.jetbrains.com/idea/>) both have such support. With Spring Tools, you can use the “reload” button from the console view (as long as your `trigger-file` is named `.reloadtrigger`). For IntelliJ IDEA, you can follow the [instructions in their documentation](https://www.jetbrains.com/help/idea/spring-boot.html#application-update-policies) (<https://www.jetbrains.com/help/idea/spring-boot.html#application-update-policies>).

79.3.6. Customizing the Restart Classloader

As described earlier in the Restart vs Reload section, restart functionality is implemented by using two classloaders. If this causes issues, you can diagnose the problem by using the `spring.devtools.restart.enabled` system property, and if the app works with restart switched off, you might need to customize what gets loaded by which classloader.

By default, any open project in your IDE is loaded with the “restart” classloader, and any regular `.jar` file is loaded with the “base” classloader. The same is true if you use `mvn spring-boot:run` or `gradle bootRun`: the project containing your `javadoc:org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` is loaded with the “restart” classloader, and everything else with the “base” classloader. The classpath is printed on the console when you start the app, which can help to identify any problematic entries. Classes used reflectively, especially annotations, can be loaded into the parent (fixed) classloader on startup before the application classes which use them, and this might lead to them not being detected by Spring in the application.

You can instruct Spring Boot to load parts of your project with a different classloader by creating a `META-INF/spring-devtools.properties` file. The `spring-devtools.properties` file can contain properties prefixed with `restart.exclude` and `restart.include`. The `include` elements are items that should be pulled up into the “restart” classloader, and the `exclude` elements are items that should be pushed down into the “base” classloader. The value of the property is a regex pattern that is applied to the classpath passed to the JVM on startup. Here is an example where some local class files are excluded and some extra libraries are included in the restart class loader:

```
restart:
  exclude:
    companycommonlibs: "/mycorp-common-[\\w\\d-\\.]/(build|bin|out|target)/"
  include:
    projectcommon: "/mycorp-myproj-[\\w\\d-\\.]+\\.jar"
```

PROPERTIES



All property keys must be unique. As long as a property starts with `restart.include.` or `restart.exclude.` it is considered.



All `META-INF/spring-devtools.properties` from the classpath are loaded. You can package files inside your project, or in the libraries that the project consumes. System properties can not be used, only the properties file.

79.3.7. Known Limitations

Restart functionality does not work well with objects that are deserialized by using a standard `java.io.ObjectInputStream[]`. If you need to deserialize data, you may need to use Spring's `org.springframework.core.ConfigurableObjectInputStream[]` in combination with `Thread.currentThread().getContextClassLoader()`.

Unfortunately, several third-party libraries deserialize without considering the context classloader. If you find such a problem, you need to request a fix with the original authors.

79.4. LiveReload

The `spring-boot-devtools` module includes an embedded LiveReload server that can be used to trigger a browser refresh when a resource is changed. LiveReload browser extensions are freely available for Chrome, Firefox and Safari. You can find these extensions by searching 'LiveReload' in the marketplace or store of your chosen browser.

If you want to start the LiveReload server when your application runs, you can set the `configprop:spring.devtools.livereload.enabled[]` property to `true`.



You can only run one LiveReload server at a time. Before starting your application, ensure that no other LiveReload servers are running. If you start multiple applications from your IDE, only the first has LiveReload support.



To trigger LiveReload when a file changes, [using/devtools.html](#) must be enabled.

79.5. Global Settings

You can configure global devtools settings by adding any of the following files to the `$HOME/.config/spring-boot` directory:

1. `spring-boot-devtools.properties`
2. `spring-boot-devtools.yaml`
3. `spring-boot-devtools.yml`

Any properties added to these files apply to *all* Spring Boot applications on your machine that use devtools. For example, to configure restart to always use a [trigger file](#), you would add the following property to your `spring-boot-devtools` file:

```
spring:
  devtools:
    restart:
      trigger-file: ".reloadtrigger"
```

By default, `$HOME` is the user's home directory. To customize this location, set the `SPRING_DEVTOOLS_HOME` environment variable or the `spring.devtools.home` system property.



If devtools configuration files are not found in `$HOME/.config/spring-boot`, the root of the `$HOME` directory is searched for the presence of a `.spring-boot-devtools.properties` file. This allows you to share the devtools global configuration with applications that are on an older version of Spring Boot that does not support the `$HOME/.config/spring-boot` location.



Profiles are not supported in devtools properties/yaml files.

Any profiles activated in `.spring-boot-devtools.properties` will not affect the loading of [profile-specific configuration files](#). Profile specific filenames (of the form `spring-boot-devtools-<profile>.properties`) and `spring.config.activate.on-profile` documents in both YAML and Properties files are not supported.

79.5.1. Configuring File System Watcher

`java.io.ObjectInputStream[]` works by polling the class changes with a certain time interval, and then waiting for a predefined quiet period to make sure there are no more changes. Since Spring Boot relies entirely on the IDE to compile and copy files into the location from where Spring Boot can read them, you might find that there are times when certain changes are not reflected when devtools restarts the application. If you observe such problems constantly, try increasing the `spring.devtools.restart.poll-interval` and `spring.devtools.restart.quiet-period` parameters to the values that fit your development environment:

```
spring:
  devtools:
    restart:
      poll-interval: "2s"
      quiet-period: "1s"
```

The monitored classpath directories are now polled every 2 seconds for changes, and a 1 second quiet period is maintained to make sure there are no additional class changes.

79.6. Remote Applications

The Spring Boot developer tools are not limited to local development. You can also use several features when running applications remotely. Remote support is opt-in as enabling it can be a security risk. It should only be enabled when running on a trusted network or when secured with SSL. If neither of these options is available to you, you should not use DevTools' remote support. You should never enable support on a production deployment.

To enable it, you need to make sure that `devtools` is included in the repackaged archive, as shown in the following listing:

```

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <includeOptional>true</includeOptional>
        <excludeDevtools>false</excludeDevtools>
      </configuration>
    </plugin>
  </plugins>
</build>

```



Optional dependencies are not included by default, which explains why `includeOptional` is also present.

Then you need to set the `configprop:spring.devtools.remote.secret[]` property. Like any important password or secret, the value should be unique and strong such that it cannot be guessed or brute-forced.

Remote devtools support is provided in two parts: a server-side endpoint that accepts connections and a client application that you run in your IDE. The server component is automatically enabled when the `configprop:spring.devtools.remote.secret[]` property is set. The client component must be launched manually.



Remote devtools is not supported for Spring WebFlux applications.

79.6.1. Running the Remote Client Application

The remote client application is designed to be run from within your IDE. You need to run `javac:org.springframework.boot.devtools.RemoteSpringApplication[]` with the same classpath as the remote project that you connect to. The application's single required argument is the remote URL to which it connects.

For example, if you are using Eclipse or Spring Tools and you have a project named `my-app` that you have deployed to Cloud Foundry, you would do the following:

- Select **Run Configurations...** from the **Run** menu.
- Create a new **Java Application** “launch configuration”.
- Browse for the `my-app` project.
- Use `javac:org.springframework.boot.devtools.RemoteSpringApplication[]` as the main class.
- Add `https://myapp.cfapps.io` to the **Program arguments** (or whatever your remote URL is).

A running remote client might resemble the following listing:

```
link:ROOT:example$remote-spring-application.txt[role=include]
```



Because the remote client is using the same classpath as the real application it can directly read application properties. This is how the `configprop:spring.devtools.remote.secret[]` property is read and passed to the server for authentication.



It is always advisable to use `https://` as the connection protocol, so that traffic is encrypted and passwords cannot be intercepted.



If you need to use a proxy to access the remote application, configure the `spring.devtools.remote.proxy.host` and `spring.devtools.remote.proxy.port` properties.

79.6.2. Remote Update

The remote client monitors your application classpath for changes in the same way as the local restart. Any updated resource is pushed to the remote application and (*if required*) triggers a restart. This can be helpful if you iterate on a feature that uses a cloud service that you do not have locally. Generally, remote updates and restarts are much quicker than a full rebuild and deploy cycle.

On a slower development environment, it may happen that the quiet period is not enough, and the changes in the classes may be split into batches. The server is restarted after the first batch of class changes is uploaded. The next batch can't be sent to the application, since the server is restarting.

This is typically manifested by a warning in the `javac:org.springframework.boot.devtools.RemoteSpringApplication[]` logs about failing to upload some of the classes, and a consequent retry. But it may also lead to application code inconsistency and failure to restart after the first batch of changes is uploaded. If you observe such problems constantly, try increasing the `spring.devtools.restart.poll-interval` and `spring.devtools.restart.quiet-period` parameters to the values that fit your development environment. See the [using/devtools.html](#) section for configuring these properties.



Files are only monitored when the remote client is running. If you change a file before starting the remote client, it is not pushed to the remote server.

80. Developing with Spring Boot

This section goes into more detail about how you should use Spring Boot. It covers topics such as build systems, auto-configuration, and how to run your applications. We also cover some Spring Boot best practices. Although there is nothing particularly special about Spring Boot (it is just another library that you can consume), there are a few recommendations that, when followed, make your development process a little easier.

If you are starting out with Spring Boot, you should probably read the [tutorial:first-application/index.html](#) tutorial before diving into this section.

81. Packaging Your Application for Production

Once your Spring Boot application is ready for production deployment, there are many options for packaging and optimizing the application. See the [packaging/index.html](#) section of the documentation to read about these features.

For additional "production ready" features, such as health, auditing, and metric REST or JMX end-points, consider adding `spring-boot-actuator` . See [how-to:actuator.html](#) for details.

82. Running Your Application

One of the biggest advantages of packaging your application as a jar and using an embedded HTTP server is that you can run your application as you would any other. The same applies to debugging Spring Boot applications. You do not need any special IDE plugins or extensions.



The options below are best suited for running an application locally for development. For production deployment, see [reference:using/packaging-for-production.html](#).



This section only covers jar-based packaging. If you choose to package your application as a war file, see your server and IDE documentation.

82.1. Running From an IDE

You can run a Spring Boot application from your IDE as a Java application. However, you first need to import your project. Import steps vary depending on your IDE and build system. Most IDEs can import Maven projects directly. For example, Eclipse users can select **Import...** → **Existing Maven Projects** from the **File** menu.

If you cannot directly import your project into your IDE, you may be able to generate IDE metadata by using a build plugin. Maven includes plugins for [Eclipse](https://maven.apache.org/plugins/maven-eclipse-plugin/) (<https://maven.apache.org/plugins/maven-eclipse-plugin/>) and [IntelliJ IDEA](https://maven.apache.org/plugins/maven-idea-plugin/) (<https://maven.apache.org/plugins/maven-idea-plugin/>). Gradle offers plugins for [{url-gradle-docs}/userguide.html](#) [various IDEs].



If you accidentally run a web application twice, you see a “Port already in use” error. Spring Tools users can use the **Relaunch** button rather than the **Run** button to ensure that any existing instance is closed.

82.2. Running as a Packaged Application

If you use the Spring Boot Maven or Gradle plugins to create an executable jar, you can run your application using `java -jar`, as shown in the following example:

```
$ java -jar target/myapplication-0.0.1-SNAPSHOT.jar
```

SHELL

It is also possible to run a packaged application with remote debugging support enabled. Doing so lets you attach a debugger to your packaged application, as shown in the following example:

```
$ java -agentlib:jdwp=server=y,transport=dt_socket,address=8000,suspend=n \
  -jar target/myapplication-0.0.1-SNAPSHOT.jar
```

SHELL

82.3. Using the Maven Plugin

The Spring Boot Maven plugin includes a `run` goal that can be used to quickly compile and run your application. Applications run in an exploded form, as they do in your IDE. The following example shows a typical Maven command to run a Spring Boot application:

```
$ mvn spring-boot:run
```

SHELL

You might also want to use the `MAVEN_OPTS` operating system environment variable, as shown in the following example:

```
$ export MAVEN_OPTS=-Xmx1024m
```

SHELL

82.4. Using the Gradle Plugin

The Spring Boot Gradle plugin also includes a `bootRun` task that can be used to run your application in an exploded form. The `bootRun` task is added whenever you apply the `org.springframework.boot` and `java` plugins and is shown in the following example:

```
$ gradle bootRun
```

SHELL

You might also want to use the `JAVA_OPTS` operating system environment variable, as shown in the following example:

```
$ export JAVA_OPTS=-Xmx1024m
```

SHELL

82.5. Hot Swapping

Since Spring Boot applications are plain Java applications, JVM hot-swapping should work out of the box. JVM hot swapping is somewhat limited with the bytecode that it can replace. For a more complete solution, [JRebel](https://www.jrebel.com/products/jrebel) (<https://www.jrebel.com/products/jrebel>) can be used.

The `spring-boot-devtools` module also includes support for quick application restarts. See the [how-to:hotswapping.html](#) section in “How-to Guides” for details.

83. Spring Beans and Dependency Injection

You are free to use any of the standard Spring Framework techniques to define your beans and their injected dependencies. We generally recommend using constructor injection to wire up dependencies and `javadoc:org.springframework.context.annotation.ComponentScan[format=annotation]` to find beans.

If you structure your code as suggested above (locating your application class in a top package), you can add `javadoc:org.springframework.context.annotation.ComponentScan[format=annotation]` without any arguments or use the `javadoc:org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` annotation which implicitly includes it. All of your application components (`javadoc:org.springframework.stereotype.Component[format=annotation]`, `javadoc:org.springframework.stereotype.Service[format=annotation]`, `javadoc:org.springframework.stereotype.Repository[format=annotation]`, `javadoc:org.springframework.stereotype.Controller[format=annotation]`, and others) are automatically registered as Spring Beans.

The following example shows a `javadoc:org.springframework.stereotype.Service[format=annotation]` Bean that uses constructor injection to obtain a required `RiskAssessor` bean:

```
include-code::singleconstructor/MyAccountService[]
```

If a bean has more than one constructor, you will need to mark the one you want Spring to use with `javadoc:org.springframework.beans.factory.annotation.Autowired[format=annotation]`:

```
include-code::multipleconstructors/MyAccountService[]
```



Notice how using constructor injection lets the `riskAssessor` field be marked as `final`, indicating that it cannot be subsequently changed.

84. Structuring Your Code

Spring Boot does not require any specific code layout to work. However, there are some best practices that help.



If you wish to enforce a structure based on domains, take a look at [Spring Modulith](https://spring.io/projects/spring-modulith#overview) (https://spring.io/projects/spring-modulith#overview).

84.1. Using the “default” Package

When a class does not include a `package` declaration, it is considered to be in the “default package”. The use of the “default package” is generally discouraged and should be avoided. It can cause particular problems for Spring Boot applications that use the `javax.annotation.processing.Generated`, `javax.persistence.Entity`, `javax.persistence.EntityScan`, `javax.persistence.EntityScan`, or `javax.persistence.EntityScan` annotations, since every class from every jar is read.



We recommend that you follow Java’s recommended package naming conventions and use a reversed domain name (for example, `com.example.project`).

84.2. Locating the Main Application Class

We generally recommend that you locate your main application class in a root package above other classes. The `@SpringBootApplication` annotation is often placed on your main class, and it implicitly defines a base “search package” for certain items. For example, if you are writing a JPA application, the package of the `javax.persistence.Entity` annotated class is used to search for `javax.persistence.Entity` items. Using a root package also allows component scan to apply only on your project.



If you do not want to use `javax.annotation.processing.Generated`, `javax.persistence.Entity`, `javax.persistence.EntityScan`, or `javax.persistence.EntityScan` annotations that it imports defines that behavior so you can also use those instead.

The following listing shows a typical layout:

```
com
+- example
  +- myapplication
    +- MyApplication.java
    |
    +- customer
    |   +- Customer.java
    |   +- CustomerController.java
    |   +- CustomerService.java
    |   +- CustomerRepository.java
    |
    +- order
    |   +- Order.java
    |   +- OrderController.java
    |   +- OrderService.java
    |   +- OrderRepository.java
```

The `MyApplication.java` file would declare the `main` method, along with the basic `javax.annotation.processing.Generated`, `javax.persistence.Entity`, `javax.persistence.EntityScan`, or `javax.persistence.EntityScan` annotations, as follows:

```
include-code::MyApplication[]
```

85. Using the @SpringBootApplication Annotation

Many Spring Boot developers like their apps to use auto-configuration, component scan and be able to define extra configuration on their "application class". A single `javax.annotation.processing.processor.Path@org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` annotation can be used to enable those three features, that is:

- `javax.annotation.processing.processor.Path@org.springframework.boot.autoconfigure.EnableAutoConfiguration[format=annotation]`: enable [Spring Boot's auto-configuration mechanism](#)
- `javax.annotation.processing.processor.Path@org.springframework.context.annotation.ComponentScan[format=annotation]`: enable `javax.annotation.processing.processor.Path@org.springframework.stereotype.Component[format=annotation]` scan on the package where the application is located (see [the best practices](#))
- `javax.annotation.processing.processor.Path@org.springframework.boot.SpringBootConfiguration[format=annotation]`: enable registration of extra beans in the context or the import of additional configuration classes. An alternative to Spring's standard `javax.annotation.processing.processor.Path@org.springframework.context.annotation.Configuration[format=annotation]` that aids [configuration detection](#) in your integration tests.

include-code::springapplication/MyApplication[]



`javax.annotation.processing.processor.Path@org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` also provides aliases to customize the attributes of `javax.annotation.processing.processor.Path@org.springframework.boot.autoconfigure.EnableAutoConfiguration[format=annotation]` and `javax.annotation.processing.processor.Path@org.springframework.context.annotation.ComponentScan[format=annotation]`.

None of these features are mandatory and you may choose to replace this single annotation by any of the features that it enables. For instance, you may not want to use component scan or configuration properties scan in your application:



include-code::individualannotations/MyApplication[]

In this example, **MyApplication** is just like any other Spring Boot application except that `javax.annotation.processing.processor.Path@org.springframework.stereotype.Component[format=annotation]`-annotated classes and `javax.annotation.processing.processor.Path@org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]`-annotated classes are not detected automatically and the user-defined beans are imported explicitly (see `javax.annotation.processing.processor.Path@org.springframework.context.annotation.Import[format=annotation]`).

86. Graceful Shutdown

Graceful shutdown is enabled by default with all three embedded web servers (Jetty, Reactor Netty, and Tomcat) and with both reactive and servlet-based web applications. It occurs as part of closing the application context and is performed in the earliest phase of stopping `org.springframework.context.SmartLifecycle[]` beans. This stop processing uses a timeout which provides a grace period during which existing requests will be allowed to complete but no new requests will be permitted.

To configure the timeout period, configure the `configprop:spring.lifecycle.timeout-per-shutdown-phase[]` property, as shown in the following example:

```
spring:
  lifecycle:
    timeout-per-shutdown-phase: "20s"
```



Shutdown in your IDE may be immediate rather than graceful if it does not send a proper **SIGTERM** signal. See the documentation of your IDE for more details.

86.1. Rejecting Requests During the Grace Period

The exact way in which new requests are not permitted varies depending on the web server that is being used. Implementations may stop accepting requests at the network layer, or they may return a response with a specific HTTP status code or HTTP header. The use of persistent connections can also change the way that requests stop being accepted.



To learn more about the specific method used with your web server, see the `shutDownGracefully` API documentation for `javadoc:org.springframework.boot.tomcat.TomcatWebServer#shutDownGracefully(org.springframework.boot.web.server.GracefulShutdownCallback[])`, `javadoc:org.springframework.boot.reactor.netty.NettyWebServer#shutDownGracefully(org.springframework.boot.web.server.GracefulShutdownCallback[])`, or `javadoc:org.springframework.boot.jetty.JettyWebServer#shutDownGracefully(org.springframework.boot.web.server.GracefulShutdownCallback[])`.

Jetty, Reactor Netty, and Tomcat will stop accepting new requests at the network layer.

86.2. Disabling Graceful Shutdown

To disable graceful shutdown, configure the `configprop:server.shutdown[]` property, as shown in the following example:

```
server:
  shutdown: "immediate"
```

87. Web

Spring Boot is well suited for web application development. You can create a self-contained HTTP server by using embedded Tomcat, Jetty, or Netty. Most web applications use the `spring-boot-starter-web` module to get up and running quickly. You can also choose to build reactive web applications by using the `spring-boot-starter-webflux` module.

If you have not yet developed a Spring Boot web application, you can follow the “Hello World!” example in the [Getting started](#) section.

88. Reactive Web Applications

Spring Boot simplifies development of reactive web applications by providing auto-configuration for Spring Webflux.

88.1. The “Spring WebFlux Framework”

Spring WebFlux is the new reactive web framework introduced in Spring Framework 5.0. Unlike Spring MVC, it does not require the servlet API, is fully asynchronous and non-blocking, and implements the [Reactive Streams](https://www.reactive-streams.org/) specification through [the Reactor project](https://projectreactor.io/).

Spring WebFlux comes in two flavors: functional and annotation-based. The annotation-based one is quite close to the Spring MVC model, as shown in the following example:

```
include-code::MyRestController[]
```

WebFlux is part of the Spring Framework and detailed information is available in its [{url-spring-framework-docs}/web/webflux.html](#) [reference documentation].

“WebFlux.fn”, the functional variant, separates the routing configuration from the actual handling of the requests, as shown in the following example:

```
include-code::MyRoutingConfiguration[]
```

```
include-code::MyUserHandler[]
```

“WebFlux.fn” is part of the Spring Framework and detailed information is available in its [{url-spring-framework-docs}/web/webflux-functional.html](#) [reference documentation].



You can define as many `javadoc:org.springframework.web.reactive.function.server.RouterFunction[]` beans as you like to modularize the definition of the router. Beans can be ordered if you need to apply a precedence.

To get started, add the `spring-boot-starter-webflux` module to your application.



Adding both `spring-boot-starter-web` and `spring-boot-starter-webflux` modules in your application results in Spring Boot auto-configuring Spring MVC, not WebFlux. This behavior has been chosen because many Spring developers add `spring-boot-starter-webflux` to their Spring MVC application to use the reactive `javadoc:org.springframework.web.reactive.function.client.WebClient[]`. You can still enforce your choice by setting the chosen application type to `SpringApplication.setWebApplicationType(WebApplicationType.REACTIVE)`.

88.1.1. Spring WebFlux Auto-configuration

Spring Boot provides auto-configuration for Spring WebFlux that works well with most applications.

The auto-configuration adds the following features on top of Spring’s defaults:

- Configuring codecs for `javadoc:org.springframework.http.codec.HttpMessageReader[]` and `javadoc:org.springframework.http.codec.HttpMessageWriter[]` instances (described [later in this document](#)).
- Support for serving static resources, including support for WebJars (described [later in this document](#)).

If you want to keep Spring Boot WebFlux features and you want to add additional [{url-spring-framework-docs}/web/webflux/config.html](#) [WebFlux configuration], you can add your own `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class of type `javadoc:org.springframework.web.reactive.config.WebFluxConfigurer[]` but **without** `javadoc:org.springframework.web.reactive.config.EnableWebFlux[format=annotation]`.

If you want to add additional customization to the auto-configured `javadoc:org.springframework.http.server.reactive.HttpHandler[]`, you can define beans of type `javadoc:org.springframework.boot.webflux.autoconfigure.WebHttpHandlerBuilderCustomizer[]` and use them to modify the `javadoc:org.springframework.web.server.adapter.WebHttpHandlerBuilder[]`.

If you want to take complete control of Spring WebFlux, you can add your own `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` annotated with `javadoc:org.springframework.web.reactive.config.EnableWebFlux[format=annotation]`.

88.1.2. Spring WebFlux Conversion Service

If you want to customize the `javadoc:org.springframework.core.convert.ConversionService[]` used by Spring WebFlux, you can provide a `javadoc:org.springframework.web.reactive.config.WebFluxConfigurer[]` bean with an `addFormatters` method.

Conversion can also be customized using the `spring.webflux.format.*` configuration properties. When not configured, the following defaults are used:

Property	DateTimeFormatter	Formats
configprop:spring.webflux.format.date[]	ofLocalizedDate(FormatStyle.SHORT)	java.util.Date and javadoc:java.time.LocalDate[]
configprop:spring.webflux.format.time[]	ofLocalizedTime(FormatStyle.SHORT)	java.time’s javadoc:java.time.LocalDateTime[] and javadoc:java.time.OffsetTime[]
configprop:spring.webflux.format.date-time[]	ofLocalizedDateTime(FormatStyle.SHORT)	java.time’s javadoc:java.time.LocalDateTime[], javadoc:java.time.OffsetDateTime[], and javadoc:java.time.ZonedDateTime[]

88.1.3. HTTP Codecs with HttpMessageReaders and HttpMessageWriters

Spring WebFlux uses the `javax.annotation.processing.processor.support.RouterFunctionMapping` and `javax.annotation.processing.processor.support.RouterFunctionMapping` interfaces to convert HTTP requests and responses. They are configured with `javax.annotation.processing.processor.support.RouterFunctionMapping` to have sensible defaults by looking at the libraries available in your classpath.

Spring Boot provides dedicated configuration properties for codecs, `spring.http.codecs.*`. It also applies further customization by using `javax.annotation.processing.processor.support.RouterFunctionMapping` instances. For example, `spring.jackson.*` configuration keys are applied to the Jackson codec.

If you need to add or customize codecs, you can create a custom `javax.annotation.processing.processor.support.RouterFunctionMapping` component, as shown in the following example:

```
include-code::MyCodecsConfiguration[]
```

You can also leverage [Boot's custom JSON serializers and deserializers](#).

88.1.4. Static Content

By default, Spring Boot serves static content from a directory called `/static` (or `/public` or `/resources` or `META-INF/resources`) in the classpath. It uses the `javax.annotation.processing.processor.support.RouterFunctionMapping` from Spring WebFlux so that you can modify that behavior by adding your own `javax.annotation.processing.processor.support.RouterFunctionMapping` and overriding the `addResourceHandlers` method.

By default, resources are mapped on `/**`, but you can tune that by setting the `configprop:spring.webflux.static-path-pattern` property. For instance, relocating all resources to `/resources/**` can be achieved as follows:

```
spring:
  webflux:
    static-path-pattern: "/resources/**"
```

You can also customize the static resource locations by using `spring.web.resources.static-locations`. Doing so replaces the default values with a list of directory locations. If you do so, the default welcome page detection switches to your custom locations. So, if there is an `index.html` in any of your locations on startup, it is the home page of the application.

In addition to the “standard” static resource locations listed earlier, a special case is made for [Webjars content](#) (<https://www.webjars.org/>). By default, any resources with a path in `/webjars/**` are served from jar files if they are packaged in the Webjars format. The path can be customized with the `configprop:spring.webflux.webjars-path-pattern` property.



Spring WebFlux applications do not strictly depend on the servlet API, so they cannot be deployed as war files and do not use the `src/main/webapp` directory.

88.1.5. Welcome Page

Spring Boot supports both static and templated welcome pages. It first looks for an `index.html` file in the configured static content locations. If one is not found, it then looks for an `index` template. If either is found, it is automatically used as the welcome page of the application.

This only acts as a fallback for actual index routes defined by the application. The ordering is defined by the order of `javax.annotation.processing.processor.support.RouterFunctionMapping` beans which is by default the following:

<code>org.springframework.web.reactive.function.server.support.RouterFunctionMapping</code>	Endpoints declared with <code>javax.annotation.processing.processor.support.RouterFunctionMapping</code> beans
<code>org.springframework.web.reactive.result.method.annotation.RequestMappingHandlerMapping</code>	Endpoints declared in <code>javax.annotation.processing.processor.support.Controller</code> beans
RouterFunctionMapping for the Welcome Page	The welcome page support

88.1.6. Template Engines

As well as REST web services, you can also use Spring WebFlux to serve dynamic HTML content. Spring WebFlux supports a variety of templating technologies, including Thymeleaf, FreeMarker, and Mustache.

Spring Boot includes auto-configuration support for the following templating engines:

- [FreeMarker](#) (<https://freemarker.apache.org/docs/>)
- [Thymeleaf](#) (<https://www.thymeleaf.org>)
- [Mustache](#) (<https://mustache.github.io/>)



Not all FreeMarker features are supported with WebFlux. For more details, check the description of each property.

When you use one of these templating engines with the default configuration, your templates are picked up automatically from `src/main/resources/templates`.

88.1.7. Error Handling

Spring Boot provides a `javax.annotation.processing.processor.support.RouterFunctionMapping` that handles all errors in a sensible way. Its position in the processing order is immediately before the handlers provided by WebFlux, which are considered last. For machine clients, it produces a JSON response with details of the error, the HTTP status, and the exception message. For browser clients, there is a “whitelabel” error handler that renders the same data in HTML format. You can also provide your own HTML templates to display errors (see the [next section](#)).

Before customizing error handling in Spring Boot directly, you can leverage the {url-spring-framework-docs}/web/webflux/ann-rest-exceptions.html[RFC 9457 Problem Details] support in Spring WebFlux. Spring WebFlux can produce custom error messages with the `application/problem+json` media type, like:

```
{
  "type": "https://example.org/problems/unknown-project",
  "title": "Unknown project",
  "status": 404,
  "detail": "No project found for id 'spring-unknown'",
  "instance": "/projects/spring-unknown"
}
```

This support can be enabled by setting `configprop:spring.webflux.problemdetails.enabled[]` to `true` .

The first step to customizing this feature often involves using the existing mechanism but replacing or augmenting the error contents. For that, you can add a bean of type `javadoc:org.springframework.boot.webflux.error.ErrorAttributes[]`.

To change the error handling behavior, you can implement `javadoc:org.springframework.boot.webflux.error.ErrorWebExceptionHandler[]` and register a bean definition of that type. Because an `javadoc:org.springframework.boot.webflux.error.ErrorWebExceptionHandler[]` is quite low-level, Spring Boot also provides a convenient `javadoc:org.springframework.boot.webflux.autoconfigure.error.AbstractErrorWebExceptionHandler[]` to let you handle errors in a WebFlux functional way, as shown in the following example:

include-code::MyErrorWebExceptionHandler[]

For a more complete picture, you can also subclass `javadoc:org.springframework.boot.webflux.autoconfigure.error.DefaultErrorWebExceptionHandler[]` directly and override specific methods.

In some cases, errors handled at the controller level are not recorded by web observations or the [metrics infrastructure](#). Applications can ensure that such exceptions are recorded with the observations by {url-spring-framework-docs}/integration/observability.html#observability.http-server:reactive[setting the handled exception on the observation context].

Custom Error Pages

If you want to display a custom HTML error page for a given status code, you can add views that resolve from `error/*` , for example by adding files to a `/error` directory. Error pages can either be static HTML (that is, added under any of the static resource directories) or built with templates. The name of the file should be the exact status code, a status code series mask, or `error` for a default if nothing else matches. Note that the path to the default error view is `error/error` , whereas with Spring MVC the default error view is `error` .

For example, to map `404` to a static HTML file, your directory structure would be as follows:

```
src/
+- main/
  +- java/
  |   + <source code>
  +- resources/
    +- public/
      +- error/
      |   +- 404.html
      +- <other public assets>
```

To map all `5xx` errors by using a Mustache template, your directory structure would be as follows:

```
src/
+- main/
  +- java/
  |   + <source code>
  +- resources/
    +- templates/
      +- error/
      |   +- 5xx.mustache
      +- <other templates>
```

88.1.8. Web Filters

Spring WebFlux provides a `javadoc:org.springframework.web.server.WebFilter[]` interface that can be implemented to filter HTTP request-response exchanges. `javadoc:org.springframework.web.server.WebFilter[]` beans found in the application context will be automatically used to filter each exchange.

Where the order of the filters is important they can implement `javadoc:org.springframework.core.Ordered[]` or be annotated with `javadoc:org.springframework.core.annotation.Order[format=annotation]`. Spring Boot auto-configuration may configure web filters for you. When it does so, the orders shown in the following table will be used:

Web Filter	Order
javadoc:org.springframework.security.web.server.WebFilterChainProxy[] (Spring Security)	-100
javadoc:org.springframework.boot.webflux.actuate.web.exchanges.HttpExchangesWebFilter[]	Ordered.LOWEST_PRECEDENCE - 10

88.1.9. API Versioning

Spring WebFlux supports API versioning which can be used to evolve an HTTP API over time. The same `@Controller` path can be mapped multiple times to support different versions of the API.

For more details see <https://docs.spring.io/spring-framework/docs/web/webflux/controller/ann-requestmapping.html#webflux-ann-requestmapping-version> [Spring Framework's reference documentation].

Once mappings have been added, you additionally need to configure Spring WebFlux so that it is able to use any version information sent with a request. Typically, versions are sent as HTTP headers, query parameters or as part of the path.

To configure Spring WebFlux, you can either use a `javadoc:org.springframework.web.reactive.config.WebFluxConfigurer[]` bean and override the `configureApiVersioning(...)` method, or you can use properties.

For example, the following will use an `X-Version` HTTP header to obtain version information and default to `1.0.0` when no header is sent.

```
spring:
  webflux:
    apiversion:
      default: 1.0.0
      use:
        header: X-Version
```

For more complete control, you can also define `javadoc:org.springframework.web.reactive.accept.ApiVersionResolver[]`, `javadoc:org.springframework.web.accept.ApiVersionParser[]` and `javadoc:org.springframework.web.reactive.accept.ApiVersionDeprecationHandler[]` beans which will be injected into the auto-configured Spring MVC configuration.



API versioning is also supported on the client-side with both `WebClient` and `RestClient`. See [io/rest-client.html](https://docs.spring.io/spring-framework/docs/web/rest-client.html) for details.

88.2. Embedded Reactive Server Support

Spring Boot includes support for the following embedded reactive web servers: Reactor Netty, Tomcat, and Jetty. Most developers use the appropriate starter to obtain a fully configured instance. By default, the embedded server listens for HTTP requests on port 8080.

88.2.1. Customizing Reactive Servers

Common reactive web server settings can be configured by using Spring `javadoc:org.springframework.core.env.Environment[]` properties. Usually, you would define the properties in your `application.properties` or `application.yaml` file.

Common server settings include:

- Network settings: Listen port for incoming HTTP requests (`server.port`), interface address to bind to (`server.address`), and so on.
- Error management: Location of the error page (`configprop:spring.web.error.path[]`) and so on.
- [SSL](#)
- [HTTP compression](#)

Spring Boot tries as much as possible to expose common settings, but this is not always possible. For those cases, dedicated namespaces such as `server.netty.*` offer server-specific customizations.



See the `javadoc:org.springframework.boot.web.server.autoconfigure.ServerProperties[]` class for a complete list.

Programmatic Customization

If you need to programmatically configure your reactive web server, you can register a Spring bean that implements the `javadoc:org.springframework.boot.web.server.WebServerFactoryCustomizer[]` interface. `javadoc:org.springframework.boot.web.server.WebServerFactoryCustomizer[]` provides access to the `javadoc:org.springframework.boot.web.server.reactive.ConfigurableReactiveWebServerFactory[]`, which includes numerous customization setter methods. The following example shows programmatically setting the port:

```
include-code::MyWebServerFactoryCustomizer[]
```

`javadoc:org.springframework.boot.jetty.reactive.JettyReactiveWebServerFactory[]`, `javadoc:org.springframework.boot.reactor.netty.NettyReactiveWebServerFactory[]`, and `javadoc:org.springframework.boot.tomcat.reactive.TomcatReactiveWebServerFactory[]` are dedicated variants of `javadoc:org.springframework.boot.web.server.reactive.ConfigurableReactiveWebServerFactory[]` that have additional customization setter methods for Jetty, Reactor Netty, and Tomcat respectively. The following example shows how to customize `javadoc:org.springframework.boot.reactor.netty.NettyReactiveWebServerFactory[]` that provides access to Reactor Netty-specific configuration options:

```
include-code::MyNettyWebServerFactoryCustomizer[]
```

Customizing ConfigurableReactiveWebServerFactory Directly

For more advanced use cases that require you to extend from `javadoc:org.springframework.boot.web.server.reactive.ReactiveWebServerFactory[]`, you can expose a bean of such type yourself.

Setters are provided for many configuration options. Several protected method “hooks” are also provided should you need to do something more exotic. See the `javadoc:org.springframework.boot.web.server.reactive.ConfigurableReactiveWebServerFactory[]` API documentation for details.



Auto-configured customizers are still applied on your custom factory, so use that option carefully.

88.3. Reactive Server Resources Configuration

When auto-configuring a Reactor Netty or Jetty server, Spring Boot will create specific beans that will provide HTTP resources to the server instance: `javadoc:org.springframework.http.client.ReactorResourceFactory[]` or `javadoc:org.springframework.http.client.reactive.JettyResourceFactory[]`.

By default, those resources will be also shared with the Reactor Netty and Jetty clients for optimal performances, given:

- the same technology is used for server and client
- the client instance is built using the `javadoc:org.springframework.web.reactive.function.client.WebClient$Builder[]` bean auto-configured by Spring Boot

Developers can override the resource configuration for Jetty and Reactor Netty by providing a custom `javadoc:org.springframework.http.client.ReactorResourceFactory[]` or `javadoc:org.springframework.http.client.reactive.JettyResourceFactory[]` bean - this will be applied to both clients and servers.

You can learn more about the resource configuration on the client side in the [io/rest-client.html](#) section.

89. Servlet Web Applications

If you want to build servlet-based web applications, you can take advantage of Spring Boot's auto-configuration for Spring MVC or Jersey.

89.1. The “Spring Web MVC Framework”

The {url-spring-framework-docs}/web/webmvc.html[Spring Web MVC framework] (often referred to as “Spring MVC”) is a rich “model view controller” web framework. Spring MVC lets you create special `javax.servlet.annotation.Controller` or `javax.servlet.annotation.RestController` beans to handle incoming HTTP requests. Methods in your controller are mapped to HTTP by using `javax.servlet.annotation.RequestMapping` annotations.

The following code shows a typical `javax.servlet.annotation.RestController` that serves JSON data:

```
include-code::MyRestController[]
```

“WebMvc.fn”, the functional variant, separates the routing configuration from the actual handling of the requests, as shown in the following example:

```
include-code::MyRoutingConfiguration[]
```

```
include-code::MyUserHandler[]
```

Spring MVC is part of the core Spring Framework, and detailed information is available in the {url-spring-framework-docs}/web/webmvc.html[reference documentation]. There are also several guides that cover Spring MVC available at spring.io/guides.



You can define as many `javax.servlet.function.RouterFunction` beans as you like to modularize the definition of the router. Beans can be ordered if you need to apply a precedence.

89.1.1. Spring MVC Auto-configuration

Spring Boot provides auto-configuration for Spring MVC that works well with most applications. It replaces the need for `javax.servlet.config.annotation.EnableWebMvc` and the two cannot be used together. In addition to Spring MVC's defaults, the auto-configuration provides the following features:

- Inclusion of `javax.servlet.view.ContentNegotiatingViewResolver` and `javax.servlet.view.BeanNameViewResolver` beans.
- Support for serving static resources, including support for WebJars (covered [later in this document](#)).
- Automatic registration of `javax.convert.converter.Converter`, `javax.convert.converter.GenericConverter`, and `javax.format.Formatter` beans.
- Support for `javax.http.converter.autoconfigure.HttpMessageConverters` (covered [later in this document](#)).
- Automatic registration of `javax.validation.MessageCodesResolver` (covered [later in this document](#)).
- Static `index.html` support.
- Automatic use of a `javax.bind.support.ConfigurableWebBindingInitializer` bean (covered [later in this document](#)).

If you want to keep those Spring Boot MVC customizations and make more {url-spring-framework-docs}/web/webmvc.html[MVC customizations] (interceptors, formatters, view controllers, and other features), you can add your own `javax.context.annotation.Configuration` class of type `javax.servlet.config.annotation.WebMvcConfigurer` but **without** `javax.servlet.config.annotation.EnableWebMvc`.

If you want to provide custom instances of `javax.servlet.mvc.method.annotation.RequestMappingHandlerMapping`, `javax.servlet.mvc.method.annotation.RequestMappingHandlerAdapter`, or `javax.servlet.mvc.method.annotation.ExceptionHandlerExceptionResolver`, and still keep the Spring Boot MVC customizations, you can declare a bean of type `javax.boot.autoconfigure.web.WebMvcRegistrations` and use it to provide custom instances of those components. The custom instances will be subject to further initialization and configuration by Spring MVC. To participate in, and if desired, override that subsequent processing, a `javax.servlet.config.annotation.WebMvcConfigurer` should be used.

If you do not want to use the auto-configuration and want to take complete control of Spring MVC, add your own `javax.context.annotation.Configuration` annotated with `javax.servlet.config.annotation.EnableWebMvc`. Alternatively, add your own `javax.servlet.config.annotation.Configuration`-annotated `javax.servlet.config.annotation.DelegatingWebMvcConfiguration` as described in the `javax.servlet.config.annotation.EnableWebMvc` API documentation.

89.1.2. Spring MVC Conversion Service

Spring MVC uses a different `javax.convert.ConversionService` to the one used to convert values from your `application.properties` or `application.yml` file. It means that `java.time.Period`, `java.time.Duration` and `javax.util.unit.DataSize` converters are not available and that `javax.boot.convert.DurationUnit` and `javax.boot.convert.DataSizeUnit` annotations will be ignored.

If you want to customize the `javax.convert.ConversionService` used by Spring MVC, you can provide a `javax.servlet.config.annotation.WebMvcConfigurer` bean with an `addFormatters` method. From this method you can register any converter that you like, or you can delegate to the static methods available on `javax.boot.convert.ApplicationConversionService`.

Conversion can also be customized using the `spring.mvc.format.*` configuration properties. When not configured, the following defaults are used:

Property	DateTimeFormatter	Formats
configprop:spring.mvc.format.date[]	ofLocalizedDate(FormatStyle.SHORT)	java.util.Date and javadoc:java.time.LocalDate[]
configprop:spring.mvc.format.time[]	ofLocalizedTime(FormatStyle.SHORT)	java.time's javadoc:java.time.LocalDateTime[] and javadoc:java.time.OffsetTime[]
configprop:spring.mvc.format.date-time[]	ofLocalizedDateTime(FormatStyle.SHORT)	java.time's javadoc:java.time.LocalDateTime[], javadoc:java.time.OffsetDateTime[], and javadoc:java.time.ZonedDateTime[]

89.1.3. [HttpMessageConverters](#)

Spring MVC uses the `javadoc:org.springframework.http.converter.HttpMessageConverter[]` interface to convert HTTP requests and responses. Sensible defaults are included out of the box. For example, objects can be automatically converted to JSON (by using the Jackson library) or XML (by using the Jackson XML extension, if available, or by using JAXB if the Jackson XML extension is not available). By default, strings are encoded in `UTF-8`.

Any `javadoc:org.springframework.http.converter.HttpMessageConverter[]` bean that is present in the context is added to the list of converters. You can also override default converters in the same way.

If you need to add or customize converters, you can declare one or more `javadoc:org.springframework.boot.http.converter.autoconfigure.ClientHttpMessageConvertersCustomizer[]` or `javadoc:org.springframework.boot.http.converter.autoconfigure.ServerHttpMessageConvertersCustomizer[]` as beans. There, you can choose whether converter instances should be added before default ones (`addCustomConverter`) or if they should override a specific default converter (like `withJsonConverter`).

See the following listing for an example:

```
include-code::MyHttpMessageConvertersConfiguration[]
```

89.1.4. [MessageCodesResolver](#)

Spring MVC has a strategy for generating error codes for rendering error messages from binding errors: `javadoc:org.springframework.validation.MessageCodesResolver[]`. If you set the `configprop:spring.mvc.message-codes-resolver-format[]` property `PREFIX_ERROR_CODE` or `POSTFIX_ERROR_CODE`, Spring Boot creates one for you (see the enumeration in `javadoc:org.springframework.validation.DefaultMessageCodesResolver$Format[]`).

89.1.5. [Static Content](#)

By default, Spring Boot serves static content from a directory called `/static` (or `/public` or `/resources` or `/META-INF/resources`) in the classpath or from the root of the `javadoc:jakarta.servlet.ServletContext[]`. It uses the `javadoc:org.springframework.web.servlet.resource.ResourceHttpRequestHandler[]` from Spring MVC so that you can modify that behavior by adding your own `javadoc:org.springframework.web.servlet.config.annotation.WebMvcConfigurer[]` and overriding the `addResourceHandlers` method.

In a stand-alone web application, the default servlet from the container is not enabled. It can be enabled using the `configprop:server.servlet.register-default-servlet[]` property.

The default servlet acts as a fallback, serving content from the root of the `javadoc:jakarta.servlet.ServletContext[]` if Spring decides not to handle it. Most of the time, this does not happen (unless you modify the default MVC configuration), because Spring can always handle requests through the `javadoc:org.springframework.web.servlet.DispatcherServlet[]`.

By default, resources are mapped on `/**`, but you can tune that with the `configprop:spring.mvc.static-path-pattern[]` property. For instance, relocating all resources to `/resources/**` can be achieved as follows:

```
spring:
  mvc:
    static-path-pattern: "/resources/**"
```

You can also customize the static resource locations by using the `configprop:spring.web.resources.static-locations[]` property (replacing the default values with a list of directory locations). The root servlet context path, `/`, is automatically added as a location as well.

In addition to the “standard” static resource locations mentioned earlier, a special case is made for [Webjars content](#) (<https://www.webjars.org/>). By default, any resources with a path in `/webjars/**` are served from jar files if they are packaged in the Webjars format. The path can be customized with the `configprop:spring.mvc.webjars-path-pattern[]` property.



Do not use the `src/main/webapp` directory if your application is packaged as a jar. Although this directory is a common standard, it works **only** with war packaging, and it is silently ignored by most build tools if you generate a jar.

Spring Boot also supports the advanced resource handling features provided by Spring MVC, allowing use cases such as cache-busting static resources or using version agnostic URLs for Webjars.

To use version agnostic URLs for Webjars, add the `org.webjars:webjars-locator-lite` dependency. Then declare your Webjar. Using jQuery as an example, adding `/webjars/jquery/jquery.min.js` results in `/webjars/jquery/x.y.z/jquery.min.js` where `x.y.z` is the Webjar version.

To use cache busting, the following configuration configures a cache busting solution for all static resources, effectively adding a content hash, such as `<link href="/css/spring-2a2d595e6ed9a0b24f027f2b63b134d6.css"/>`, in URLs:

```
spring:
  web:
    resources:
      chain:
        strategy:
          content:
            enabled: true
            paths: "/*"
```



Links to resources are rewritten in templates at runtime, thanks to a `javax.servlet.resource.ResourceUrlEncodingFilter` that is auto-configured for Thymeleaf and FreeMarker. You should manually declare this filter when using JSPs. Other template engines are currently not automatically supported but can be with custom template macros/helpers and the use of the `javax.servlet.resource.ResourceUrlProvider`.

When loading resources dynamically with, for example, a JavaScript module loader, renaming files is not an option. That is why other strategies are also supported and can be combined. A "fixed" strategy adds a static version string in the URL, without changing the file name, as shown in the following example:

```
spring:
  web:
    resources:
      chain:
        strategy:
          content:
            enabled: true
            paths: "/*"
        fixed:
          enabled: true
          paths: "/js/lib/"
          version: "v12"
```

With this configuration, JavaScript modules located under `/js/lib/` use a fixed versioning strategy (`/v12/js/lib/myModule.js`), while other resources still use the content one (`<link href="/css/spring-2a2d595e6ed9a0b24f027f2b63b134d6.css"/>`).

See `javax.servlet.config.WebProperties$Resources` for more supported options.



This feature has been thoroughly described in a dedicated [blog post](https://spring.io/blog/2014/07/24/spring-framework-4-1-handling-static-web-resources) (<https://spring.io/blog/2014/07/24/spring-framework-4-1-handling-static-web-resources>) and in Spring Framework's `{url-spring-framework-docs}/web/webmvc/mvc-config/static-resources.html` [reference documentation].

89.1.6. Welcome Page

Spring Boot supports both static and templated welcome pages. It first looks for an `index.html` file in the configured static content locations. If one is not found, it then looks for an `index` template. If either is found, it is automatically used as the welcome page of the application.

This only acts as a fallback for actual index routes defined by the application. The ordering is defined by the order of `javax.servlet.HandlerMapping` beans which is by default the following:

RouterFunctionMapping	Endpoints declared with <code>javax.servlet.function.RouterFunction</code> beans
RequestMappingHandlerMapping	Endpoints declared in <code>javax.servlet.stereotype.Controller[format=annotation]</code> beans
WelcomePageHandlerMapping	The welcome page support

89.1.7. Custom Favicon

As with other static resources, Spring Boot checks for a `favicon.ico` in the configured static content locations. If such a file is present, it is automatically used as the favicon of the application.

89.1.8. Path Matching and Content Negotiation

Spring MVC can map incoming HTTP requests to handlers by looking at the request path and matching it to the mappings defined in your application (for example, `javax.servlet.bind.annotation.GetMapping[format=annotation]` annotations on Controller methods).

Spring Boot chooses to disable suffix pattern matching by default, which means that requests like `GET /projects/spring-boot.json` will not be matched to `@GetMapping("/projects/spring-boot")` mappings. This is considered as a `{url-spring-framework-docs}/web/webmvc/mvc-controller/ann-requestmapping.html#mvc-ann-requestmapping-suffix-pattern-match` [best practice for Spring MVC applications]. This feature was mainly useful in the past for HTTP clients which did not send proper "Accept" request headers; we needed to make sure to send the correct Content Type to the client. Nowadays, Content Negotiation is much more reliable.

There are other ways to deal with HTTP clients that do not consistently send proper "Accept" request headers. Instead of using suffix matching, we can use a query parameter to ensure that requests like `GET /projects/spring-boot?format=json` will be mapped to `@GetMapping("/projects/spring-boot")`:

```
spring:
  mvc:
    contentnegotiation:
      favor-parameter: true
```

Or if you prefer to use a different parameter name:

```
spring:
  mvc:
    contentnegotiation:
      favor-parameter: true
      parameter-name: "myparam"
```

Most standard media types are supported out-of-the-box, but you can also define new ones:

```
spring:
  mvc:
    contentnegotiation:
      media-types:
        markdown: "text/markdown"
```

As of Spring Framework 5.3, Spring MVC supports two strategies for matching request paths to controllers. By default, Spring Boot uses the `javax.servlet.http.HttpServlet` strategy. `javax.servlet.http.HttpServlet` is an [optimized implementation](https://spring.io/blog/2020/06/30/url-matching-with-pathpattern-in-spring-mvc) (<https://spring.io/blog/2020/06/30/url-matching-with-pathpattern-in-spring-mvc>) but comes with some restrictions compared to the `javax.servlet.http.HttpServlet` strategy. `javax.servlet.http.HttpServlet` restricts usage of `{url-spring-framework-docs}/web/webmvc/mvc-controller/ann-requestmapping.html#mvc-ann-requestmapping-uri-templates[some path pattern variants]`. It is also incompatible with configuring the `javax.servlet.http.HttpServlet` with a path prefix (`configprop:spring.mvc.servlet.path[]`).

The strategy can be configured using the `configprop:spring.mvc.pathmatch.matching-strategy[]` configuration property, as shown in the following example:

```
spring:
  mvc:
    pathmatch:
      matching-strategy: "ant-path-matcher"
```

Spring MVC will throw a `javax.servlet.http.HttpServlet.NoHandlerFoundException[]` if a handler is not found for a request. Note that, by default, the [serving of static content](#) is mapped to `/**` and will, therefore, provide a handler for all requests. If no static content is available, `javax.servlet.http.HttpServlet.ResourceHttpRequestHandler[]` will throw a `javax.servlet.http.HttpServlet.NoResourceFoundException[]`. For a `javax.servlet.http.HttpServlet.NoHandlerFoundException[]` to be thrown, set `configprop:spring.mvc.static-path-pattern[]` to a more specific value such as `/resources/**` or set `configprop:spring.web.resources.add-mappings[]` to `false` to disable serving of static content entirely.

89.1.9. ConfigurableWebBindingInitializer

Spring MVC uses a `javax.servlet.http.HttpServlet.WebBindingInitializer[]` to initialize a `javax.servlet.http.HttpServlet.WebDataBinder[]` for a particular request. If you create your own `javax.servlet.http.HttpServlet.WebBindingInitializer[]` `javax.servlet.http.HttpServlet.WebDataBinder[]`, Spring Boot automatically configures Spring MVC to use it.

89.1.10. Template Engines

As well as REST web services, you can also use Spring MVC to serve dynamic HTML content. Spring MVC supports a variety of templating technologies, including Thymeleaf, FreeMarker, and JSPs. Also, many other templating engines include their own Spring MVC integrations.

Spring Boot includes auto-configuration support for the following templating engines:

- [FreeMarker](https://freemarker.apache.org/docs/) (<https://freemarker.apache.org/docs/>)
- [Groovy](https://docs.groovy-lang.org/docs/next/html/documentation/template-engines.html#_the_markuptemplateengine) (https://docs.groovy-lang.org/docs/next/html/documentation/template-engines.html#_the_markuptemplateengine)
- [Thymeleaf](https://www.thymeleaf.org) (<https://www.thymeleaf.org>)
- [Mustache](https://mustache.github.io/) (<https://mustache.github.io/>)



If possible, JSPs should be avoided. There are several [known limitations](#) when using them with embedded servlet containers.

When you use one of these templating engines with the default configuration, your templates are picked up automatically from `src/main/resources/templates`.



Depending on how you run your application, your IDE may order the classpath differently. Running your application in the IDE from its main method results in a different ordering than when you run your application by using Maven or Gradle or from its packaged jar. This can cause Spring Boot to fail to find the expected template. If you have this problem, you can reorder the classpath in the IDE to place the module's classes and resources first.

89.1.11. Error Handling

By default, Spring Boot provides an `/error` mapping that handles all errors in a sensible way, and it is registered as a “global” error page in the servlet container. For machine clients, it produces a JSON response with details of the error, the HTTP status, and the exception message. For browser clients, there is a “whitelabel” error view that renders the same data in HTML format (to customize it, add a `javax.servlet.http.HttpServlet.View[]` that resolves to `error`).

There are a number of `spring.web.error` properties that can be set if you want to customize the default error handling behavior. See the [Web Properties](#) section of the Appendix.

To replace the default behavior completely, you can implement `javax.servlet.http.HttpServlet.WebMvcError.ErrorController[]` and register a bean definition of that type or add a bean of type `javax.servlet.http.HttpServlet.WebMvcError.ErrorAttributes[]` to use the existing mechanism but replace the contents.



The `javax.servlet.http.HttpServlet` can be used as a base class for a custom `javax.servlet.http.HttpServlet`. This is particularly useful if you want to add a handler for a new content type (the default is to handle `text/html` specifically and provide a fallback for everything else). To do so, extend `javax.servlet.http.HttpServlet`, add a public method with a `javax.servlet.http.HttpServlet` that has a `produces` attribute, and create a bean of your new type.

As of Spring Framework 6.0, <https://springframework.org/docs/6.0.0/web/webmvc/mvc-ann-rest-exceptions.html> [RFC 9457 Problem Details] is supported. Spring MVC can produce custom error messages with the `application/problem+json` media type, like:

JSON

```
{
  "type": "https://example.org/problems/unknown-project",
  "title": "Unknown project",
  "status": 404,
  "detail": "No project found for id 'spring-unknown'",
  "instance": "/projects/spring-unknown"
}
```

This support can be enabled by setting `configprops:spring.mvc.problemdetails.enabled` to `true`.

You can also define a class annotated with `javax.servlet.http.HttpServlet` to customize the JSON document to return for a particular controller and/or exception type, as shown in the following example:

include-code::MyControllerAdvice[]

In the preceding example, if `MyException` is thrown by a controller defined in the same package as `SomeController`, a JSON representation of the `MyResponseBody` POJO is used instead of the `javax.servlet.http.HttpServlet` representation.

In some cases, errors handled at the controller level are not recorded by web observations or the [metrics infrastructure](#). Applications can ensure that such exceptions are recorded with the observations by `{url-spring-framework-docs}/integration/observability.html#observability.http-server.servlet` [setting the handled exception on the observation context].

Custom Error Pages

If you want to display a custom HTML error page for a given status code, you can add a file to an `/error` directory. Error pages can either be static HTML (that is, added under any of the static resource directories) or be built by using templates. The name of the file should be the exact status code or a series mask.

For example, to map 404 to a static HTML file, your directory structure would be as follows:

```
src/
+- main/
  +- java/
  |   + <source code>
  +- resources/
  |   +- public/
  |   |   +- error/
  |   |   |   +- 404.html
  |   |   +- <other public assets>
```

To map all 5xx errors by using a FreeMarker template, your directory structure would be as follows:

```
src/
+- main/
  +- java/
  |   + <source code>
  +- resources/
  |   +- templates/
  |   |   +- error/
  |   |   |   +- 5xx.ftlh
  |   +- <other templates>
```

For more complex mappings, you can also add beans that implement the `javax.servlet.http.HttpServlet` interface, as shown in the following example:

include-code::MyErrorViewResolver[]

You can also use regular Spring MVC features such as `{url-spring-framework-docs}/web/webmvc/mvc-servlet/exceptionhandlers.html` [`@ExceptionHandler` methods] and `{url-spring-framework-docs}/web/webmvc/mvc-controller/ann-advice.html` [`@ControllerAdvice`]. The `javax.servlet.http.HttpServlet` then picks up any unhandled exceptions.

Mapping Error Pages Outside of Spring MVC

For applications that do not use Spring MVC, you can use the `javax.servlet.http.HttpServlet` interface to directly register `javax.servlet.http.HttpServlet` instances. This abstraction works directly with the underlying embedded servlet container and works even if you do not have a Spring MVC `javax.servlet.http.HttpServlet`.

include-code::MyErrorPagesConfiguration[]



If you register an `javax.servlet.Filter` with a path that ends up being handled by a `javax.servlet.Filter` (as is common with some non-Spring web frameworks, like Jersey and Wicket), then the `javax.servlet.Filter` has to be explicitly registered as an **ERROR** dispatcher, as shown in the following example:

```
include-code::MyFilterConfiguration[]
```

Note that the default `org.springframework.boot.web.servlet.FilterRegistrationBean` does not include the **ERROR** dispatcher type.

Error Handling in a WAR Deployment

When deployed to a servlet container, Spring Boot uses its error page filter to forward a request with an error status to the appropriate error page. This is necessary as the servlet specification does not provide an API for registering error pages. Depending on the container that you are deploying your war file to and the technologies that your application uses, some additional configuration may be required.

The error page filter can only forward the request to the correct error page if the response has not already been committed. By default, WebSphere Application Server 8.0 and later commits the response upon successful completion of a servlet's service method. You should disable this behavior by setting `com.ibm.ws.webcontainer.invokeFlushAfterService` to `false`.

89.1.12. CORS Support

Cross-origin resource sharing (https://en.wikipedia.org/wiki/Cross-origin_resource_sharing) (CORS) is a **W3C specification** (<https://www.w3.org/TR/cors/>) implemented by **most browsers** (<https://caniuse.com/#feat=cors>) that lets you specify in a flexible way what kind of cross-domain requests are authorized, instead of using some less secure and less powerful approaches such as **IFRAME** or **JSONP**.

As of version 4.2, Spring MVC ([web/webmvc-cors.html](https://springframework.org/docs/4.2.x/web/webmvc-cors.html)) supports CORS. Using ([web/webmvc-cors.html#mvc-cors-controller](https://springframework.org/docs/4.2.x/web/webmvc-cors.html#mvc-cors-controller)) with `javax.servlet.Filter` annotations in your Spring Boot application does not require any specific configuration. ([web/webmvc-cors.html#mvc-cors-global](https://springframework.org/docs/4.2.x/web/webmvc-cors.html#mvc-cors-global)) can be defined by registering a `org.springframework.web.servlet.config.annotation.WebMvcConfigurer` bean with a customized `addCorsMappings(CorsRegistry)` method, as shown in the following example:

```
include-code::MyCorsConfiguration[]
```

89.1.13. API Versioning

Spring MVC supports API versioning which can be used to evolve an HTTP API over time. The same `@Controller` path can be mapped multiple times to support different versions of the API.

For more details see ([web/webmvc/mvc-controller/ann-requestmapping.html#mvc-ann-requestmapping-version](https://springframework.org/docs/4.2.x/web/webmvc/mvc-controller/ann-requestmapping.html#mvc-ann-requestmapping-version)) [Spring Framework's reference documentation].

One mappings have been added, you additionally need to configure Spring MVC so that it is able to use any version information sent with a request. Typically, versions are sent as HTTP headers, query parameters or as part of the path.

To configure Spring MVC, you can either use a `org.springframework.web.servlet.config.annotation.WebMvcConfigurer` bean and override the `configureApiVersioning(...)` method, or you can use properties.

For example, the following will use an `X-Version` HTTP header to obtain version information and default to `1.0.0` when no header is sent.

```
spring:
  mvc:
    api-version:
      default: 1.0.0
      use:
        header: X-Version
```

For more complete control, you can also define `org.springframework.web.accept.ApiVersionResolver`, `org.springframework.web.accept.ApiVersionParser` and `org.springframework.web.accept.ApiVersionDeprecationHandler` beans which will be injected into the auto-configured Spring MVC configuration.



API versioning is also supported with both **WebClient** and **RestClient**. See [io/rest-client.html](https://springframework.org/docs/4.2.x/web/rest-client.html) for details.

89.2. JAX-RS and Jersey

If you prefer the JAX-RS programming model for REST endpoints, you can use one of the available implementations instead of Spring MVC. **Jersey** (<https://jersey.github.io/>) and **Apache CXF** (<https://cxf.apache.org/>) work quite well out of the box. CXF requires you to register its `javax.servlet.Filter` or `javax.servlet.Filter` as a `org.springframework.context.annotation.Bean` in your application context. Jersey has some native Spring support, so we also provide auto-configuration support for it in Spring Boot, together with a starter.

To get started with Jersey, include the `spring-boot-starter-jersey` as a dependency and then you need one `org.springframework.context.annotation.Bean` of type `org.glassfish.jersey.server.ResourceConfig` in which you register all the endpoints, as shown in the following example:

```
include-code::MyJerseyConfig[]
```




Jersey's support for scanning executable archives is rather limited. For example, it cannot scan for endpoints in a package found in a [fully executable jar file](#) or in `WEB-INF/classes` when running an executable war file. To avoid this limitation, the `packages` method should not be used, and endpoints should be registered individually by using the `register` method, as shown in the preceding example.

For more advanced customizations, you can also register an arbitrary number of beans that implement `javax.ws.rs.core.ResourceConfigCustomizer`.

All the registered endpoints should be a `javax.ws.rs.core.Component[format=annotation]` with HTTP resource annotations (`@GET` and others), as shown in the following example:

```
include-code::MyEndpoint[]
```

Since the `javax.ws.rs.core.Endpoint[format=annotation]` is a Spring `javax.ws.rs.core.Component[format=annotation]`, its lifecycle is managed by Spring and you can use the `javax.ws.rs.core.Autowired[format=annotation]` annotation to inject dependencies and use the `javax.ws.rs.core.Value[format=annotation]` annotation to inject external configuration. By default, the Jersey servlet is registered and mapped to `/*`. You can change the mapping by adding `javax.ws.rs.ApplicationPath[format=annotation]` to your `javax.ws.rs.ResourceConfig[]`.

By default, Jersey is set up as a servlet in a `javax.ws.rs.core.Bean[format=annotation]` of type `javax.ws.rs.core.ServletRegistrationBean[]` named `jerseyServletRegistration`. By default, the servlet is initialized lazily, but you can customize that behavior by setting `spring.jersey.servlet.load-on-startup`. You can disable or override that bean by creating one of your own with the same name. You can also use a filter instead of a servlet by setting `spring.jersey.type=filter` (in which case, the `javax.ws.rs.core.Bean[format=annotation]` to replace or override is `jerseyFilterRegistration`). The filter has an `javax.ws.rs.core.annotation.Order[format=annotation]`, which you can set with `spring.jersey.filter.order`. When using Jersey as a filter, a servlet that will handle any requests that are not intercepted by Jersey must be present. If your application does not contain such a servlet, you may want to enable the default servlet by setting `configprop:server.servlet.register-default-servlet[]` to `true`. Both the servlet and the filter registrations can be given init parameters by using `spring.jersey.init.*` to specify a map of properties.

89.3. Embedded Servlet Container Support

For servlet application, Spring Boot includes support for embedded [Tomcat](https://tomcat.apache.org/) and [Jetty](https://www.eclipse.org/jetty/) servers. Most developers use the appropriate starter to obtain a fully configured instance. By default, the embedded server listens for HTTP requests on port `8080`.

89.3.1. Servlets, Filters, and Listeners

When using an embedded servlet container, you can register servlets, filters, and all the listeners (such as `javax.servlet.http.HttpSessionListener[]`) from the servlet spec, either by using Spring beans or by scanning for servlet components.

Registering Servlets, Filters, and Listeners as Spring Beans

Any `javax.servlet.Servlet[]`, `javax.servlet.Filter[]`, or servlet `*Listener` instance that is a Spring bean is registered with the embedded container. This can be particularly convenient if you want to refer to a value from your `application.properties` during configuration.

By default, if the context contains only a single Servlet, it is mapped to `/`. In the case of multiple servlet beans, the bean name is used as a path prefix. Filters map to `/*`.

If convention-based mapping is not flexible enough, you can use the `javax.ws.rs.core.ServletRegistrationBean[]`, `javax.ws.rs.core.FilterRegistrationBean[]`, and `javax.ws.rs.core.ServletListenerRegistrationBean[]` classes for complete control. If you prefer annotations over `javax.ws.rs.core.ServletRegistrationBean[]` and `javax.ws.rs.core.FilterRegistrationBean[]`, you can also use `javax.ws.rs.core.ServletRegistration[format=annotation]` and `javax.ws.rs.core.FilterRegistration[format=annotation]` as an alternative.

It is usually safe to leave filter beans unordered. If a specific order is required, you should annotate the `javax.servlet.Filter[]` with `javax.ws.rs.core.annotation.Order[format=annotation]` or make it implement `javax.ws.rs.core.Ordered[]`. You cannot configure the order of a `javax.servlet.Filter[]` by annotating its bean method with `javax.ws.rs.core.annotation.Order[format=annotation]`. If you cannot change the `javax.servlet.Filter[]` class to add `javax.ws.rs.core.annotation.Order[format=annotation]` or implement `javax.ws.rs.core.Ordered[]`, you must define a `javax.ws.rs.core.FilterRegistrationBean[]` for the `javax.servlet.Filter[]` and set the registration bean's order using the `setOrder(int)` method. Or, if you prefer annotations, you can also use `javax.ws.rs.core.FilterRegistration[format=annotation]` and set the `order` attribute. Avoid configuring a filter that reads the request body at `Ordered.HIGHEST_PRECEDENCE`, since it might go against the character encoding configuration of your application. If a servlet filter wraps the request, it should be configured with an order that is less than or equal to `OrderedFilter.REQUEST_WRAPPER_FILTER_MAX_ORDER`.



To see the order of every `javax.servlet.Filter[]` in your application, enable debug level logging for the `web logging.group` (`logging.level.web=debug`). Details of the registered filters, including their order and URL patterns, will then be logged at startup.



Take care when registering `javax.servlet.Filter[]` beans since they are initialized very early in the application lifecycle. If you need to register a `javax.servlet.Filter[]` that interacts with other beans, consider using a `javax.ws.rs.core.DelegatingFilterProxyRegistrationBean[]` instead.

89.3.2. Servlet Context Initialization

Embedded servlet containers do not directly execute the `javax.servlet.ServletContainerInitializer[]` interface or Spring's `javax.servlet.Web.WebApplicationInitializer[]` interface. This is an intentional design decision intended to reduce the risk that third party libraries designed to run inside a war may break Spring Boot applications.

If you need to perform servlet context initialization in a Spring Boot application, you should register a bean that implements the `javax.servlet.ServletContextInitializer` interface. The single `onStartup` method provides access to the `javax.servlet.ServletContext` and, if necessary, can easily be used as an adapter to an existing `javax.servlet.WebApplicationInitializer`.

Init Parameters

Init parameters can be configured on the `javax.servlet.ServletContext` using `server.servlet.context-parameters.*` properties. For example, the property `server.servlet.context-parameters.com.example.parameter=example` will configure a `ServletContext` init parameter named `com.example.parameter` with the value `example`.

Scanning for Servlets, Filters, and listeners

When using an embedded container, automatic registration of classes annotated with `javax.servlet.annotation.WebServlet` [format=annotation], `javax.servlet.annotation.WebFilter` [format=annotation], and `javax.servlet.annotation.WebListener` [format=annotation] can be enabled by using `javax.servlet.ServletComponentScan` [format=annotation].



`javax.servlet.ServletComponentScan` [format=annotation] has no effect in a standalone container, where the container's built-in discovery mechanisms are used instead.

89.3.3. The ServletWebServerApplicationContext

Under the hood, Spring Boot uses a different type of `javax.servlet.ServletContext` for embedded servlet container support. The `javax.servlet.ServletContext` is a special type of `javax.servlet.ServletContext` that bootstraps itself by searching for a single `javax.servlet.ServletWebServerFactory` bean. Usually a `javax.servlet.ServletWebServerFactory`, or `javax.servlet.ServletWebServerFactory` has been auto-configured.



You usually do not need to be aware of these implementation classes. Most applications are auto-configured, and the appropriate `javax.servlet.ServletContext` and `javax.servlet.ServletWebServerFactory` are created on your behalf.

In an embedded container setup, the `javax.servlet.ServletContext` is set as part of server startup which happens during application context initialization. Because of this beans in the `javax.servlet.ServletContext` cannot be reliably initialized with a `javax.servlet.ServletContext`. One way to get around this is to inject `javax.servlet.ServletContext` as a dependency of the bean and access the `javax.servlet.ServletContext` only when it is needed. Another way is to use a callback once the server has started. This can be done using an `javax.servlet.ServletContextListener` which listens for the `javax.servlet.ServletContextEvent` as follows:

```
include-code::MyDemoBean[]
```

89.3.4. Customizing Embedded Servlet Containers

Common servlet container settings can be configured by using Spring `javax.servlet.ServletContext` properties. Usually, you would define the properties in your `application.properties` or `application.yml` file.

Common server settings include:

- Network settings: Listen port for incoming HTTP requests (`server.port`), interface address to bind to (`server.address`), and so on.
- Session settings: Whether the session is persistent (`server.servlet.session.persistent`), session timeout (`server.servlet.session.timeout`), location of session data (`server.servlet.session.store-dir`), and session-cookie configuration (`server.servlet.session.cookie.*`).
- Error management: Location of the error page (`configprops:spring.web.error.path`) and so on.
- [SSL](#)
- [HTTP compression](#)

Spring Boot tries as much as possible to expose common settings, but this is not always possible. For those cases, dedicated namespaces offer server-specific customizations (see `server.tomcat`). For instance, [access logs](#) can be configured with specific features of the embedded servlet container.



See the `javax.servlet.ServletContext` class for a complete list.

SameSite Cookies

The `SameSite` cookie attribute can be used by web browsers to control if and how cookies are submitted in cross-site requests. The attribute is particularly relevant for modern web browsers which have started to change the default value that is used when the attribute is missing.

If you want to change the `SameSite` attribute of your session cookie, you can use the `configprops:server.servlet.session.cookie.same-site` property. This property is supported by auto-configured Tomcat and Jetty servers. It is also used to configure Spring Session servlet based `javax.servlet.SessionRepository` beans.

For example, if you want your session cookie to have a `SameSite` attribute of `None`, you can add the following to your `application.properties` or `application.yml` file:

```
server:
  servlet:
    session:
      cookie:
        same-site: "none"
```

If you want to change the `SameSite` attribute on other cookies added to your `javax.servlet.http.HttpServletResponse[]`, you can use a `javax.servlet.http.CookieSameSiteSupplier[]`. The `javax.servlet.http.CookieSameSiteSupplier[]` is passed a `javax.servlet.http.Cookie[]` and may return a `SameSite` value, or `null`.

There are a number of convenience factory and filter methods that you can use to quickly match specific cookies. For example, adding the following bean will automatically apply a `SameSite` of `Lax` for all cookies with a name that matches the regular expression `myapp.*`.

```
include-code::MySameSiteConfiguration[]
```

Character Encoding

The character encoding behavior of the embedded servlet container for request and response handling can be configured using the `server.servlet.encoding.*` configuration properties.

When a request's `Accept-Language` header indicates a locale for the request it will be automatically mapped to a charset by the servlet container. Each container provides default locale to charset mappings and you should verify that they meet your application's needs. When they do not, use the `configprop:server.servlet.encoding.mapping[]` configuration property to customize the mappings, as shown in the following example:

```
server:
  servlet:
    encoding:
      mapping:
        ko: "UTF-8"
```

In the preceding example, the `ko` (Korean) locale has been mapped to `UTF-8`. This is equivalent to a `<locale-encoding-mapping-list>` entry in a `web.xml` file of a traditional war deployment.

Programmatic Customization

If you need to programmatically configure your embedded servlet container, you can register a Spring bean that implements the `javax.servlet.ServletContextInitializer` interface. `javax.servlet.ServletContextInitializer` provides access to the `javax.servlet.ServletContext`, which includes numerous customization setter methods. The following example shows programmatically setting the port:

```
include-code::MyWebServerFactoryCustomizer[]
```

`javadoc.org.springframework.boot.tomcat.servlet.TomcatServletWebServerFactory[]`, and `javadoc.org.springframework.boot.jetty.servlet.JettyServletWebServerFactory[]` are dedicated variants of `javadoc.org.springframework.boot.web.server.servlet.ConfigurableServletWebServerFactory[]` that have additional customization setter methods for Tomcat, and Jetty respectively. The following example shows how to customize `javadoc.org.springframework.boot.tomcat.servlet.TomcatServletWebServerFactory[]` that provides access to Tomcat-specific configuration options:

```
include-code::MyTomcatWebServerFactoryCustomizer[]
```

Customizing ConfigurableServletWebServerFactory Directly

For more advanced use cases that require you to extend from `javax.servlet.ServletWebServerFactory[]`, you can expose a bean of such type yourself.

Settings are provided for many configuration options. Several protected method “hooks” are also provided should you need to do something more exotic. See the [javadoc.org.springframework.boot.web.server.servlet.ConfigurableServletWebServerFactory/](http://docs.spring.io/spring-framework/docs/current/javadoc/org.springframework.boot.web.server.servlet.ConfigurableServletWebServerFactory/) API documentation for details.



Auto-configured customizers are still applied on your custom factory, so use that option carefully.

89.3.5. JSP Limitations

When running a Spring Boot application that uses an embedded servlet container (and is packaged as an executable archive), there are some limitations in the JSP support.

- With Jetty and Tomcat, it should work if you use war packaging. An executable war will work when launched with `java -jar`, and will also be deployable to any standard container. JSPs are not supported when using an executable jar.
- Creating a custom `error.jsp` page does not override the default view for [error handling](#). [Custom error pages](#) should be used instead.
- If you run your application using `mvn spring-boot:run` or `gradle bootRun` and you deviate from the standard `src/main/webapp` directory structure you may need to set a `WAR_SOURCE_DIRECTORY` environment variable so that Spring Boot can find your JSPs.

90. Spring for GraphQL

If you want to build GraphQL applications, you can take advantage of Spring Boot's auto-configuration for `{url-spring-graphql-site}`[Spring for GraphQL]. The Spring for GraphQL project is based on [GraphQL Java](https://github.com/graphql-java/graphql-java) (<https://github.com/graphql-java/graphql-java>). You'll need the `spring-boot-starter-graphql` starter at a minimum. Because GraphQL is transport-agnostic, you'll also need to have one or more additional starters in your application to expose your GraphQL API over the web:

Starter	Transport	Implementation
<code>spring-boot-starter-web</code>	HTTP	Spring MVC
<code>spring-boot-starter-websocket</code>	WebSocket	WebSocket for Servlet apps
<code>spring-boot-starter-webflux</code>	HTTP, WebSocket	Spring WebFlux
<code>spring-boot-starter-rsocket</code>	TCP, WebSocket	Spring WebFlux on Reactor Netty

90.1. GraphQL Schema

A Spring GraphQL application requires a defined schema at startup. By default, you can write ".graphqls" or ".gqls" schema files under `src/main/resources/graphql/**` and Spring Boot will pick them up automatically. You can customize the locations with `configprop:spring.graphql.schema.locations[]` and the file extensions with `configprop:spring.graphql.schema.file-extensions[]`.



If you want Spring Boot to detect schema files in all your application modules and dependencies for that location, you can set `configprop:spring.graphql.schema.locations[]` to `"classpath*:graphql/**/"` (note the `classpath*` prefix).

In the following sections, we'll consider this sample GraphQL schema, defining two types and two queries:

```
link:ROOT:example$resources/graphql/schema.graphqls[role=include]
```

JSON



By default, [field introspection](https://spec.graphql.org/draft/#sec-Introspection) (<https://spec.graphql.org/draft/#sec-Introspection>) will be allowed on the schema as it is required for tools such as GraphiQL. If you wish to not expose information about the schema, you can disable introspection by setting `configprop:spring.graphql.schema.introspection.enabled[]` to `false`.

90.2. GraphQL RuntimeWiring

The GraphQL Java `javax.annotation.processing.processor.Path$Builder[]` can be used to register custom scalar types, directives, type resolvers, `javax.annotation.processing.processor.DataFetcher[]`, and more. You can declare `javax.annotation.processing.processor.RuntimeWiringConfigurer[]` beans in your Spring config to get access to the `javax.annotation.processing.processor.Path$Builder[]`. Spring Boot detects such beans and adds them to the `{url-spring-graphql-docs}/request-execution.html#execution.graphqlsource[GraphQLSource builder]`.

Typically, however, applications will not implement `javax.annotation.processing.processor.DataFetcher[]` directly and will instead create `{url-spring-graphql-docs}/controllers.html[annotated controllers]`. Spring Boot will automatically detect `javax.annotation.processing.processor.Controller[format=annotation]` classes with annotated handler methods and register those as `DataFetcher` s. Here's a sample implementation for our greeting query with a `javax.annotation.processing.processor.Controller[format=annotation]` class:

```
include-code::GreetingController[]
```

90.3. Querydsl and QueryByExample Repositories Support

Spring Data offers support for both Querydsl and QueryByExample repositories. Spring GraphQL can `{url-spring-graphql-docs}/data.html[configure Querydsl and QueryByExample repositories as javax.annotation.processing.processor.DataFetcher[]]`.

Spring Data repositories annotated with `javax.annotation.processing.processor.data.GraphQLRepository[format=annotation]` and extending one of:

- `javax.annotation.processing.processor.data.querydsl.QuerydslPredicateExecutor[]`
- `javax.annotation.processing.processor.data.querydsl.ReactiveQuerydslPredicateExecutor[]`
- `javax.annotation.processing.processor.data.repository.query.QueryByExampleExecutor[]`
- `javax.annotation.processing.processor.data.repository.query.ReactiveQueryByExampleExecutor[]`

are detected by Spring Boot and considered as candidates for `javax.annotation.processing.processor.DataFetcher[]` for matching top-level queries.

90.4. Transports

90.4.1. HTTP and WebSocket

The GraphQL HTTP endpoint is at HTTP POST `/graphql` by default. It also supports the `"text/event-stream"` media type over Server Sent Events for subscriptions only. The path can be customized with `configprop:spring.graphql.http.path[]`.



The HTTP endpoint for both Spring MVC and Spring WebFlux is provided by a `RouterFunction` bean with an `javax.annotation.processing.processor.annotation.Order[format=annotation]` of `0`. If you define your own `RouterFunction` beans, you may want to add appropriate `javax.annotation.processing.processor.annotation.Order[format=annotation]` annotations to ensure that they are sorted correctly.

The GraphQL WebSocket endpoint is off by default. To enable it:

- For a Servlet application, add the WebSocket starter `spring-boot-starter-websocket`
- For a WebFlux application, no additional dependency is required
- For both, the `spring.graphql.websocket.path[]` application property must be set

Spring GraphQL provides a [{url-spring-graphql-docs}/transports.html#server.interception\[Web Interception\]](#) model. This is quite useful for retrieving information from an HTTP request header and set it in the GraphQL context or fetching information from the same context and writing it to a response header. With Spring Boot, you can declare a `javax.ws.rs.container.ContainerRequestContext` bean to have it registered with the web transport.

[{url-spring-framework-docs}/web/webmvc-cors.html\[Spring MVC\]](#) and [{url-spring-framework-docs}/web/webflux-cors.html\[Spring WebFlux\]](#) support CORS (Cross-Origin Resource Sharing) requests. CORS is a critical part of the web config for GraphQL applications that are accessed from browsers using different domains.

Spring Boot supports many configuration properties under the `spring.graphql.cors.*` namespace; here's a short configuration sample:

```
spring:
  graphql:
    cors:
      allowed-origins: "https://example.org"
      allowed-methods: GET,POST
      max-age: 1800s
```

90.4.2. RSocket

RSocket is also supported as a transport, on top of WebSocket or TCP. Once the [RSocket server is configured](#), we can configure our GraphQL handler on a particular route using `spring.graphql.rsocket.mapping[]`. For example, configuring that mapping as `"graphql"` means we can use that as a route when sending requests with the `javax.ws.rs.client.RSocketGraphQLClient`.

Spring Boot auto-configures a `RSocketGraphQLClient.Builder` bean that you can inject in your components:

```
include-code::RSocketGraphQLClientExample[tag=builder]
```

And then send a request: `include-code::RSocketGraphQLClientExample[tag=request]`

90.5. Exception Handling

Spring GraphQL enables applications to register one or more `javax.ws.rs.container.ContainerRequestContext` components that are invoked sequentially. The Exception must be resolved to a list of `javax.ws.rs.container.ContainerRequestContext` objects, see [{url-spring-graphql-docs}/controllers.html#controllers.exception-handler\[Spring GraphQL exception handling documentation\]](#). Spring Boot will automatically detect `javax.ws.rs.container.ContainerRequestContext` beans and register them with the `javax.ws.rs.container.ContainerRequestContext`.

90.6. GraphQL and Schema Printer

Spring GraphQL offers infrastructure for helping developers when consuming or developing a GraphQL API.

Spring GraphQL ships with a default [GraphQL](#) (<https://github.com/graphql/graphql>) page that is exposed at `"/graphql"` by default. This page is disabled by default and can be turned on with the `spring.graphql.graphql.enabled[]` property. Many applications exposing such a page will prefer a custom build. A default implementation is very useful during development, this is why it is exposed automatically with [spring-boot-devtools](#) during development.

You can also choose to expose the GraphQL schema in text format at `/graphql/schema` when the `spring.graphql.schema.printer.enabled[]` property is enabled.

91. Spring HATEOAS

If you develop a RESTful API that makes use of hypermedia, Spring Boot provides auto-configuration for Spring HATEOAS that works well with most applications. The auto-configuration replaces the need to use `javax.annotation.processing.EnableHypermediaSupport(format=annotation)` and registers a number of beans to ease building hypermedia-based applications, including a `javax.annotation.processing.LinkDiscoverers[]` (for client side support) and an `javax.tools.jackson.databind.json.JsonMapper[]` configured to correctly marshal responses into the desired representation. The `javax.tools.jackson.databind.json.JsonMapper[]` is customized by setting the various `spring.jackson.*` properties or, if any exist, the `javax.annotation.processing.boot.jackson.autoconfigure.JsonMapperBuilderCustomizer[]` beans.

You can take control of Spring HATEOAS's configuration by using `javax.annotation.processing.EnableHypermediaSupport(format=annotation)`. Note that doing so disables the `javax.tools.jackson.databind.json.JsonMapper[]` customization described earlier.



`spring-boot-starter-hateoas` is specific to Spring MVC and should not be combined with Spring WebFlux. In order to use Spring HATEOAS with Spring WebFlux, you can add a direct dependency on `org.springframework.hateoas:spring-hateoas` along with `spring-boot-starter-webflux`.

By default, requests that accept `application/json` will receive an `application/hal+json` response. To disable this behavior set `configprop:spring.hateoas.use-hal-as-default-json-media-type[]` to `false` and define a `javax.annotation.processing.hateoas.config.HypermediaMappingInformation[]` or `javax.annotation.processing.hateoas.mediatype.hal.HalConfiguration[]` to configure Spring HATEOAS to meet the needs of your application and its clients.

92. Spring Security

If `{url-spring-security-site}[Spring Security]` is on the classpath, then web applications are secured by default. This includes securing Spring Boot's `/error` endpoint. Spring Boot relies on Spring Security's content-negotiation strategy to determine whether to use `httpBasic` or `formLogin`. To add method-level security to a web application, you can also add `javado:org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity[format=annotation]` with your desired settings. Additional information can be found in the `{url-spring-security-docs}/servlet/authorization/method-security.html[Spring Security Reference Guide]`.

The default `javado:org.springframework.security.core.userdetails.UserDetailsService[]` has a single user. The user name is `user`, and the password is random and is printed at `WARN` level when the application starts, as shown in the following example:

```
Using generated security password: 78fa095d-3f4c-48b1-ad50-e24c31d5cf35
```

```
This generated password is for development use only. Your security configuration must be updated before running your application in production.
```



If you fine-tune your logging configuration, ensure that the `org.springframework.boot.security.autoconfigure` category is set to log `WARN`-level messages. Otherwise, the default password is not printed.

You can change the username and password by providing a `spring.security.user.name` and `spring.security.user.password`.

The basic features you get by default in a web application are:

- A `javado:org.springframework.security.core.userdetails.UserDetailsService[]` (or `javado:org.springframework.security.core.userdetails.ReactiveUserDetailsService[]` in case of a WebFlux application) bean with in-memory store and a single user with a generated password (see `javado:org.springframework.boot.security.autoconfigure.SecurityProperties$User[]` for the properties of the user).
- Form-based login or HTTP Basic security (depending on the `Accept` header in the request) for the entire application (including actuator endpoints if actuator is on the classpath).
- A `javado:org.springframework.security.authentication.DefaultAuthenticationEventPublisher[]` for publishing authentication events.

You can provide a different `javado:org.springframework.security.authentication.AuthenticationEventPublisher[]` by adding a bean for it.

92.1. MVC Security

The default security configuration is implemented in `javado:org.springframework.boot.security.autoconfigure.SecurityAutoConfiguration[]` and `javado:org.springframework.boot.security.autoconfigure.UserDetailsServiceAutoConfiguration[]`. `javado:org.springframework.boot.security.autoconfigure.SecurityAutoConfiguration[]` imports `SpringBootWebSecurityConfiguration` for web security and `javado:org.springframework.boot.security.autoconfigure.UserDetailsServiceAutoConfiguration[]` for authentication.

To completely switch off the default web application security configuration, including Actuator security, or to combine multiple Spring Security components such as OAuth2 Client and Resource Server, add a bean of type `javado:org.springframework.security.web.SecurityFilterChain[]` (doing so does not disable the `javado:org.springframework.security.core.userdetails.UserDetailsService[]` configuration). To also switch off the `javado:org.springframework.security.core.userdetails.UserDetailsService[]` configuration, add a bean of type `javado:org.springframework.security.core.userdetails.UserDetailsService[]`, `javado:org.springframework.security.authentication.AuthenticationProvider[]`, or `javado:org.springframework.security.authentication.AuthenticationManager[]`.

The auto-configuration of a `javado:org.springframework.security.core.userdetails.UserDetailsService[]` will also back off when any of the following Spring Security modules is on the classpath:

- `spring-security-oauth2-client`
- `spring-security-oauth2-resource-server`
- `spring-security-saml2-service-provider`

To use `javado:org.springframework.security.core.userdetails.UserDetailsService[]` in addition to one or more of these dependencies, define your own `javado:org.springframework.security.provisioning.InMemoryUserDetailsManager[]` bean.

Access rules can be overridden by adding a custom `javado:org.springframework.security.web.SecurityFilterChain[]` bean. Spring Boot provides convenience methods that can be used to override access rules for actuator endpoints and static resources. `javado:org.springframework.boot.security.autoconfigure.actuate.web.servlet.EndpointRequest[]` can be used to create a `javado:org.springframework.security.web.util.matcher.RequestMatcher[]` that is based on the `configprop:management.endpoints.web.base-path[]` property. `javado:org.springframework.boot.security.autoconfigure.web.servlet.PathRequest[]` can be used to create a `javado:org.springframework.security.web.util.matcher.RequestMatcher[]` for resources in commonly used locations.

92.2. WebFlux Security

Similar to Spring MVC applications, you can secure your WebFlux applications by adding the `spring-boot-starter-security` dependency. The default security configuration is implemented in `javado:org.springframework.boot.security.autoconfigure.web.reactive.ReactiveWebSecurityAutoConfiguration[]` and `javado:org.springframework.boot.security.autoconfigure.ReactiveUserDetailsServiceAutoConfiguration[]`. In addition to reactive web applications, the latter is also auto-configured when `RSocket` is in use. `javado:org.springframework.boot.security.autoconfigure.web.reactive.ReactiveWebSecurityAutoConfiguration[]` imports `WebFluxSecurityConfiguration` for web security. `javado:org.springframework.boot.security.autoconfigure.ReactiveUserDetailsServiceAutoConfiguration[]` auto-configures authentication.

To completely switch off the default web application security configuration, including Actuator security, add a bean of type `javado:org.springframework.security.web.server.WebFilterChainProxy[]` (doing so does not disable the `javado:org.springframework.security.core.userdetails.ReactiveUserDetailsService[]` configuration). To also switch off the `javado:org.springframework.security.core.userdetails.ReactiveUserDetailsService[]` configuration, add a bean of type `javado:org.springframework.security.core.userdetails.ReactiveUserDetailsService[]` or `javado:org.springframework.security.authentication.ReactiveAuthenticationManager[]`.

The auto-configuration will also back off when any of the following Spring Security modules is on the classpath:

- `spring-security-oauth2-client`
- `spring-security-oauth2-resource-server`

To use `javado:org.springframework.security.core.userdetails.ReactiveUserDetailsService[]` in addition to one or more of these dependencies, define your own `javado:org.springframework.security.core.userdetails.MapReactiveUserDetailsService[]` bean.

Access rules and the use of multiple Spring Security components such as OAuth 2 Client and Resource Server can be configured by adding a custom `javado:org.springframework.security.web.server.SecurityWebFilterChain[]` bean. Spring Boot provides convenience methods that can be used to override access rules for actuator endpoints and static resources. `javado:org.springframework.boot.security.autoconfigure.actuate.web.reactive.EndpointRequest[]` can be used to create a `javado:org.springframework.security.web.server.util.matcher.ServerWebExchangeMatcher[]` that is based on the `configprop:management.endpoints.web.base-path[]` property.

`javado:org.springframework.boot.security.autoconfigure.web.reactive.PathRequest[]` can be used to create a `javado:org.springframework.security.web.server.util.matcher.ServerWebExchangeMatcher[]` for resources in commonly used locations.

For example, you can customize your security configuration by adding something like:

`include-code::MyWebFluxSecurityConfiguration[]`

92.3. OAuth2

[OAuth2](https://oauth.net/2/) (<https://oauth.net/2/>) is a widely used authorization framework that is supported by Spring.

92.3.1. Client

If you have `spring-security-oauth2-client` on your classpath, you can take advantage of some auto-configuration to set up OAuth2/Open ID Connect clients. This configuration makes use of the properties under `javado:org.springframework.boot.security.oauth2.client.autoconfigure.OAuth2ClientProperties[]`. The same properties are applicable to both servlet and reactive applications.

You can register multiple OAuth2 clients and providers under the `spring.security.oauth2.client` prefix, as shown in the following example:

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-login-client:
            client-id: "abcd"
            client-secret: "password"
            client-name: "Client for OpenID Connect"
            provider: "my-oauth-provider"
            scope: "openid,profile,email,phone,address"
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
            client-authentication-method: "client_secret_basic"
            authorization-grant-type: "authorization_code"

        my-client-1:
          client-id: "abcd"
          client-secret: "password"
          client-name: "Client for user scope"
          provider: "my-oauth-provider"
          scope: "user"
          redirect-uri: "{baseUrl}/authorized/user"
          client-authentication-method: "client_secret_basic"
          authorization-grant-type: "authorization_code"

        my-client-2:
          client-id: "abcd"
          client-secret: "password"
          client-name: "Client for email scope"
          provider: "my-oauth-provider"
          scope: "email"
          redirect-uri: "{baseUrl}/authorized/email"
          client-authentication-method: "client_secret_basic"
          authorization-grant-type: "authorization_code"

      provider:
        my-oauth-provider:
          authorization-uri: "https://my-auth-server.com/oauth2/authorize"
          token-uri: "https://my-auth-server.com/oauth2/token"
          user-info-uri: "https://my-auth-server.com/userinfo"
          user-info-authentication-method: "header"
          jwk-set-uri: "https://my-auth-server.com/oauth2/jwks"
          user-name-attribute: "name"
```

For OpenID Connect providers that support [OpenID Connect discovery](https://openid.net/specs/openid-connect-discovery-1_0.html) (https://openid.net/specs/openid-connect-discovery-1_0.html), the configuration can be further simplified. The provider needs to be configured with an `issuer-uri` which is the URI that it asserts as its Issuer Identifier. For example, if the `issuer-uri` provided is "https://example.com", then an "OpenID Provider Configuration Request" will be made to "https://example.com/well-known/openid-configuration". The result is expected to be an "OpenID Provider Configuration Response". The following example shows how an OpenID Connect Provider can be configured with the `issuer-uri`:

```
spring:
  security:
    oauth2:
      client:
        provider:
          oidc-provider:
            issuer-uri: "https://dev-123456.oktapreview.com/oauth2/default/"
```

By default, Spring Security's `javadoc:org.springframework.security.oauth2.client.web.OAuth2LoginAuthenticationFilter[]` only processes URLs matching `/login/oauth2/code/*`. If you want to customize the `redirect-uri` to use a different pattern, you need to provide configuration to process that custom pattern. For example, for servlet applications, you can add your own `javadoc:org.springframework.security.web.SecurityFilterChain[]` that resembles the following:

include-code::MyOAuthClientConfiguration[]



Spring Boot auto-configures an `javadoc:org.springframework.security.oauth2.client.InMemoryOAuth2AuthorizedClientService[]` which is used by Spring Security for the management of client registrations. The `javadoc:org.springframework.security.oauth2.client.InMemoryOAuth2AuthorizedClientService[]` has limited capabilities and we recommend using it only for development environments. For production environments, consider using a `javadoc:org.springframework.security.oauth2.client.JdbcOAuth2AuthorizedClientService[]` or creating your own implementation of `javadoc:org.springframework.security.oauth2.client.OAuth2AuthorizedClientService[]`.

OAuth2 Client Registration for Common Providers

For common OAuth2 and OpenID providers, including Google, Github, Facebook, and Okta, we provide a set of provider defaults (`google`, `github`, `facebook`, and `okta`, respectively).

If you do not need to customize these providers, you can set the `provider` attribute to the one for which you need to infer defaults. Also, if the key for the client registration matches a default supported provider, Spring Boot infers that as well.

In other words, the two configurations in the following example use the Google provider:

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-client:
            client-id: "abcd"
            client-secret: "password"
            provider: "google"
        google:
          client-id: "abcd"
          client-secret: "password"
```

92.3.2. Resource Server

If you have `spring-security-oauth2-resource-server` on your classpath, Spring Boot can set up an OAuth2 Resource Server. For JWT configuration, a JWK Set URI or OIDC Issuer URI needs to be specified, as shown in the following examples:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: "https://example.com/oauth2/default/v1/keys"

spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: "https://dev-123456.oktapreview.com/oauth2/default/"
```



If the authorization server does not support a JWK Set URI, you can configure the resource server with the Public Key used for verifying the signature of the JWT. This can be done using the `configprop:spring.security.oauth2.resourceserver.jwt.public-key-location[]` property, where the value needs to point to a file containing the public key in the PEM-encoded x509 format.

The `configprop:spring.security.oauth2.resourceserver.jwt.audiences[]` property can be used to specify the expected values of the `aud` claim in JWTs. For example, to require JWTs to contain an `aud` claim with the value `my-audience`:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          audiences:
            - "my-audience"
```

The same properties are applicable for both servlet and reactive applications. Alternatively, you can define your own `javadoc:org.springframework.security.oauth2.jwt.JwtDecoder[]` bean for servlet applications or a `javadoc:org.springframework.security.oauth2.jwt.ReactiveJwtDecoder[]` for reactive applications.

In cases where opaque tokens are used instead of JWTs, you can configure the following properties to validate tokens through introspection:

```
spring:
  security:
    oauth2:
      resourceserver:
        opaquetoken:
          introspection-uri: "https://example.com/check-token"
          client-id: "my-client-id"
          client-secret: "my-client-secret"
```

Again, the same properties are applicable for both servlet and reactive applications. Alternatively, you can define your own `javax.javadoc.org.springframework.security.oauth2.server.resource.introspection.OpaqueTokenIntrospector[]` bean for servlet applications or a `javax.javadoc.org.springframework.security.oauth2.server.resource.introspection.ReactiveOpaqueTokenIntrospector[]` for reactive applications.

92.3.3. Authorization Server

If you have `spring-security-oauth2-authorization-server` on your classpath, you can take advantage of some auto-configuration to set up a Servlet-based OAuth2 Authorization Server.

You can register multiple OAuth2 clients under the `spring.security.oauth2.authorizationserver.client` prefix, as shown in the following example:

```
spring:
  security:
    oauth2:
      authorizationserver:
        client:
          my-client-1:
            registration:
              client-id: "abcd"
              client-secret: "{noop}secret1"
              client-authentication-methods:
                - "client_secret_basic"
            authorization-grant-types:
              - "authorization_code"
              - "refresh_token"
            redirect-uris:
              - "https://my-client-1.com/login/oauth2/code/abcd"
              - "https://my-client-1.com/authorized"
            scopes:
              - "openid"
              - "profile"
              - "email"
              - "phone"
              - "address"
            require-authorization-consent: true
          token:
            authorization-code-time-to-live: 5m
            access-token-time-to-live: 10m
            access-token-format: "reference"
            reuse-refresh-tokens: false
            refresh-token-time-to-live: 30m
          my-client-2:
            registration:
              client-id: "efgh"
              client-secret: "{noop}secret2"
              client-authentication-methods:
                - "client_secret_jwt"
            authorization-grant-types:
              - "client_credentials"
            scopes:
              - "user.read"
              - "user.write"
          jwk-set-uri: "https://my-client-2.com/jwks"
          token-endpoint-authentication-signing-algorithm: "RS256"
```



The `client-secret` property must be in a format that can be matched by the configured `javax.javadoc.org.springframework.security.crypto.password.PasswordEncoder[]`. The default instance of `javax.javadoc.org.springframework.security.crypto.password.PasswordEncoder[]` is created via `PasswordEncoderFactories.createDelegatingPasswordEncoder()`.

The auto-configuration Spring Boot provides for Spring Authorization Server is designed for getting started quickly. Most applications will require customization and will want to define several beans to override auto-configuration.

The following components can be defined as beans to override auto-configuration specific to Spring Authorization Server:

- `javax.javadoc.org.springframework.security.oauth2.server.authorization.client.RegisteredClientRepository[]`
- `javax.javadoc.org.springframework.security.oauth2.server.authorization.settings.AuthorizationServerSettings[]`
- `javax.javadoc.org.springframework.security.web.SecurityFilterChain[]`
- `com.nimbusds.jose.jwk.source.JWKSource<com.nimbusds.jose.proc.SecurityContext>`
- `javax.javadoc.org.springframework.security.oauth2.jwt.JwtDecoder[]`



Spring Boot auto-configures an `javado:org.springframework.security.oauth2.server.authorization.client.InMemoryRegisteredClientRepository[]` which is used by Spring Authorization Server for the management of registered clients. The `javado:org.springframework.security.oauth2.server.authorization.client.InMemoryRegisteredClientRepository[]` has limited capabilities and we recommend using it only for development environments. For production environments, consider using a `javado:org.springframework.security.oauth2.server.authorization.client.JdbcRegisteredClientRepository[]` or creating your own implementation of `javado:org.springframework.security.oauth2.server.authorization.client.RegisteredClientRepository[]`.

Additional information can be found in the `{url-spring-security-docs}/servlet/oauth2/authorization-server/getting-started.html` [Getting Started] chapter of Spring Security Reference Documentation.

92.4. SAML 2.0

92.4.1. Relying Party

If you have `spring-security-saml2-service-provider` on your classpath, you can take advantage of some auto-configuration to set up a SAML 2.0 Relying Party. This configuration makes use of the properties under `javado:org.springframework.boot.security.saml2.autoconfigure.Saml2RelyingPartyProperties[]`.

A relying party registration represents a paired configuration between an Identity Provider, IDP, and a Service Provider, SP. You can register multiple relying parties under the `spring.security.saml2.relyingparty` prefix, as shown in the following example:

```
spring:
  security:
    saml2:
      relyingparty:
        registration:
          my-relying-party1:
            signing:
              credentials:
                - private-key-location: "path-to-private-key"
                  certificate-location: "path-to-certificate"
            decryption:
              credentials:
                - private-key-location: "path-to-private-key"
                  certificate-location: "path-to-certificate"
            singlelogout:
              url: "https://myapp/logout/saml2/slo"
              response-url: "https://remoteidp2.slo.url"
              binding: "POST"
          assertingparty:
            verification:
              credentials:
                - certificate-location: "path-to-verification-cert"
            entity-id: "remote-idp-entity-id1"
            sso-url: "https://remoteidp1.sso.url"

          my-relying-party2:
            signing:
              credentials:
                - private-key-location: "path-to-private-key"
                  certificate-location: "path-to-certificate"
            decryption:
              credentials:
                - private-key-location: "path-to-private-key"
                  certificate-location: "path-to-certificate"
          assertingparty:
            verification:
              credentials:
                - certificate-location: "path-to-other-verification-cert"
            entity-id: "remote-idp-entity-id2"
            sso-url: "https://remoteidp2.sso.url"
            singlelogout:
              url: "https://remoteidp2.slo.url"
              response-url: "https://myapp/logout/saml2/slo"
              binding: "POST"
```

For SAML2 logout, by default, Spring Security's `javado:org.springframework.security.saml2.provider.service.web.authentication.logout.Saml2LogoutRequestFilter[]` and `javado:org.springframework.security.saml2.provider.service.web.authentication.logout.Saml2LogoutResponseFilter[]` only process URLs matching `/logout/saml2/slo`. If you want to customize the `url` to which AP-initiated logout requests get sent to or the `response-url` to which an AP sends logout responses to, to use a different pattern, you need to provide configuration to process that custom pattern. For example, for servlet applications, you can add your own `javado:org.springframework.security.web.SecurityFilterChain[]` that resembles the following:

```
include-code::MySamlRelyingPartyConfiguration[]
```


93. Spring Session

Spring Boot provides [{url-spring-session-site}](#)[Spring Session] auto-configuration for a range of data stores. When building a servlet web application, the following stores can be auto-configured:

- Redis
- JDBC

Additionally, [{url-spring-boot-for-apache-geode-site}](#)[Spring Boot for Apache Geode] provides [{url-spring-boot-for-apache-geode-docs}#geode-session](#)[auto-configuration for using Apache Geode as a session store].

The servlet auto-configuration replaces the need to use `@Enable*HttpSession` .

If a single Spring Session module is present on the classpath, Spring Boot uses that store implementation automatically. If you have more than one implementation, Spring Boot uses the following order for choosing a specific implementation:

1. Redis
2. JDBC
3. If neither Redis nor JDBC are available, we do not configure a `javadoc:org.springframework.session.SessionRepository[]`.

When building a reactive web application, the Redis store can be auto-configured. This replaces the need to use `@EnableRedisWebSession` .

Each store has specific additional settings. For instance, it is possible to customize the name of the table for the JDBC store, as shown in the following example:

```
spring:
  session:
    jdbc:
      table-name: "SESSIONS"
```

For setting the timeout of the session you can use the `configprop:spring.session.timeout[]` property. If that property is not set with a servlet web application, the auto-configuration falls back to the value of `configprop:server.servlet.session.timeout[]`.

You can take control over Spring Session's configuration using `@Enable*HttpSession` (servlet) or `@EnableRedisWebSession` (reactive). This will cause the auto-configuration to back off. Spring Session can then be configured using the annotation's attributes rather than the previously described configuration properties.

95. Actuator

Spring Boot includes the Spring Boot Actuator. This section answers questions that often arise from its use.

95.1. Change the HTTP Port or Address of the Actuator Endpoints

In a standalone application, the Actuator HTTP port defaults to the same as the main HTTP port. To make the application listen on a different port, set the external property: `configprop:management.server.port[]`. To listen on a completely different network address (such as when you have an internal network for management and an external one for user applications), you can also set `management.server.address` to a valid IP address to which the server is able to bind.

For more detail, see the `javado:org.springframework.boot.actuate.autoconfigure.web.server.ManagementServerProperties[]` source code and [Customizing the Management Server Port](#) in the “Production-Ready Features” section.

95.2. Customizing Sanitization

To take control over the sanitization, define a `javado:org.springframework.boot.actuate.endpoint.SanitizingFunction[]` bean. The `javado:org.springframework.boot.actuate.endpoint.SanitizableData[]` with which the function is called provides access to the key and value as well as the `javado:org.springframework.core.env.PropertySource[]` from which they came. This allows you to, for example, sanitize every value that comes from a particular property source. Each `javado:org.springframework.boot.actuate.endpoint.SanitizingFunction[]` is called in order until a function changes the value of the sanitizable data.

95.3. Map Health Indicators to Micrometer Metrics

Spring Boot health indicators return a `javado:org.springframework.boot.health.contributor.Status[]` type to indicate the overall system health. If you want to monitor or alert on levels of health for a particular application, you can export these statuses as metrics with Micrometer. By default, the status codes “UP”, “DOWN”, “OUT_OF_SERVICE” and “UNKNOWN” are used by Spring Boot. To export these, you will need to convert these states to some set of numbers so that they can be used with a Micrometer `javado:io.micrometer.core.instrument.Gauge[]`.

The following example shows one way to write such an exporter:

```
include-code::MyHealthMetricsExportConfiguration[]
```

96. AOT Cache

This section includes information about using AOT cache with Spring Boot applications. For an overview of Spring Boot support for AOT cache, see [reference:packaging/aot-cache.html](#).

96.1. Packaging an Application Using AOT Cache and Buildpacks

Spring Boot's [support for Cloud Native Buildpacks](#) along with the [Paketo Java buildpack](#) (<https://paketo.io/docs/reference/java-reference>) and its [Spring Boot support](#) (<https://paketo.io/docs/reference/java-reference/#spring-boot-applications>) can be used to generate a Docker image containing an AOT Cache-optimized application.

To enable the AOT Cache in a generated Docker image, you have to use Java 25 or later, and the buildpack environment variable `BP_JVM_AOTCACHE_ENABLED` should be set to `true` when building the image as described in the [Maven plugin](#) and [Gradle plugin](#) documentation. This will cause the buildpack to do a training run of the application, save the AOT Cache in the image, and use the AOT Cache when launching the application.

The Paketo Buildpack for Spring Boot [documentation](#) (<https://github.com/paketo-buildpacks/spring-boot?tab=readme-ov-file#configuration>) has information on other configuration options that can be enabled with builder environment variables, like `TRAINING_RUN_JAVA_TOOL_OPTIONS` that allows to override the default `JAVA_TOOL_OPTIONS`, only for the AOT Cache training run.

96.2. Packaging an Application Using AOT Cache and Dockerfiles

If you don't want to use Cloud Native Buildpacks, it is possible to use AOT cache (or CDS) with a `Dockerfile`. For more information about that, please see the [Dockerfiles reference documentation](#).

96.3. Preventing Remote Services Interaction During the Training Run

When performing the training run, it may be needed to customize the Spring Boot application configuration to prevent connections to remote services that may happen before the Spring lifecycle is started. This can typically happen with early database interactions and can be handled via related configuration that can be applied by default to your application (or specifically to the training run) to prevent such interactions, see [related documentation](#)

(<https://github.com/spring-projects/spring-lifecycle-smoke-tests/blob/main/README.adoc#training-run-configuration>).

97. Ahead-of-Time Processing

A number of questions often arise when people use the ahead-of-time processing of Spring Boot applications. This section addresses those questions.

97.1. Conditions

Ahead-of-time processing optimizes the application and evaluates `javax.annotation.processing.Generated` annotations based on the environment at build time. Profiles are implemented through conditions and are therefore affected, too.

If you want beans that are created based on a condition in an ahead-of-time optimized application, you have to set up the environment when building the application. The beans which are created while ahead-of-time processing at build time are then always created when running the application and can't be switched off. To do this, you can set the profiles which should be used when building the application.

For Maven, this works by setting the `profiles` configuration of the `spring-boot-maven-plugin:process-aot` execution:

```
<profile>
  <id>native</id>
  <build>
    <pluginManagement>
      <plugins>
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-maven-plugin</artifactId>
          <executions>
            <execution>
              <id>process-aot</id>
              <configuration>
                <profiles>profile-a,profile-b</profiles>
              </configuration>
            </execution>
          </executions>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</profile>
```

XML

For Gradle, you need to configure the `ProcessAot` task:

```
tasks.withType(org.springframework.boot.gradle.tasks.aot.ProcessAot).configureEach {
    args('--spring.profiles.active=profile-a,profile-b')
}
```

GRADLE

Profiles which only change configuration properties that don't influence conditions are supported without limitations when running ahead-of-time optimized applications.

98. Spring Boot Application

This section includes topics relating directly to Spring Boot applications.

98.1. Create Your Own FailureAnalyzer

`javadoc:org.springframework.boot.diagnostics.FailureAnalyzer[]` is a great way to intercept an exception on startup and turn it into a human-readable message, wrapped in a `javadoc:org.springframework.boot.diagnostics.FailureAnalysis[]`. Spring Boot provides such an analyzer for application-context-related exceptions, JSR-303 validations, and more. You can also create your own.

`javadoc:org.springframework.boot.diagnostics.AbstractFailureAnalyzer[]` is a convenient extension of `javadoc:org.springframework.boot.diagnostics.FailureAnalyzer[]` that checks the presence of a specified exception type in the exception to handle. You can extend from that so that your implementation gets a chance to handle the exception only when it is actually present. If, for whatever reason, you cannot handle the exception, return `null` to give another implementation a chance to handle the exception.

`javadoc:org.springframework.boot.diagnostics.FailureAnalyzer[]` implementations must be registered in `META-INF/spring.factories`. The following example registers `ProjectConstraintViolationFailureAnalyzer`:

```
org.springframework.boot.diagnostics.FailureAnalyzer=\ncom.example.ProjectConstraintViolationFailureAnalyzer
```

PROPERTIES



If you need access to the `javadoc:org.springframework.beans.factory.BeanFactory[]` or the `javadoc:org.springframework.core.env.Environment[]`, declare them as constructor arguments in your `javadoc:org.springframework.boot.diagnostics.FailureAnalyzer[]` implementation.

98.2. Troubleshoot Auto-configuration

The Spring Boot auto-configuration tries its best to “do the right thing”, but sometimes things fail, and it can be hard to tell why.

There is a really useful `javadoc:org.springframework.boot.autoconfigure.condition.ConditionEvaluationReport[]` available in any Spring Boot `javadoc:org.springframework.context.ApplicationContext[]`. You can see it if you enable `DEBUG` logging output. If you use the `spring-boot-actuator` (see the [actuator.html](#) section), there is also a `conditions` endpoint that renders the report in JSON. Use that endpoint to debug the application and see what features have been added (and which have not been added) by Spring Boot at runtime.

Many more questions can be answered by looking at the source code and the API documentation. When reading the code, remember the following rules of thumb:

- Look for classes called `*AutoConfiguration` and read their sources. Pay special attention to the `@Conditional*` annotations to find out what features they enable and when. Add `--debug` to the command line or the System property `-Ddebug` to get a log on the console of all the auto-configuration decisions that were made in your app. In a running application with actuator enabled, look at the `conditions` endpoint (`/actuator/conditions` or the JMX equivalent) for the same information.
- Look for classes that are `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` (such as `javadoc:org.springframework.boot.web.server.autoconfigure.ServerProperties[]`) and read from there the available external configuration options. The `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` annotation has a `name` attribute that acts as a prefix to external properties. Thus, `javadoc:org.springframework.boot.web.server.autoconfigure.ServerProperties[]` has `prefix="server"` and its configuration properties are `server.port`, `server.address`, and others. In a running application with actuator enabled, look at the `configprops` endpoint.
- Look for uses of the `bind` method on the `javadoc:org.springframework.boot.context.properties.bind.Binder[]` to pull configuration values explicitly out of the `javadoc:org.springframework.core.env.Environment[]` in a relaxed manner. It is often used with a prefix.
- Look for `javadoc:org.springframework.beans.factory.annotation.Value[format=annotation]` annotations that bind directly to the `javadoc:org.springframework.core.env.Environment[]`.
- Look for `javadoc:org.springframework.boot.autoconfigure.condition.ConditionalOnExpression[format=annotation]` annotations that switch features on and off in response to SpEL expressions, normally evaluated with placeholders resolved from the `javadoc:org.springframework.core.env.Environment[]`.

98.3. Customize the Environment or ApplicationContext Before It Starts

A `javadoc:org.springframework.boot.SpringApplication[]` has `javadoc:org.springframework.context.ApplicationListener[]` and `javadoc:org.springframework.context.ApplicationContextInitializer[]` implementations that are used to apply customizations to the context or environment. Spring Boot loads a number of such customizations for use internally from `META-INF/spring.factories`. There is more than one way to register additional customizations:

- Programmatically, per application, by calling the `addListeners` and `addInitializers` methods on `javadoc:org.springframework.boot.SpringApplication[]` before you run it.
- Declaratively, for all applications, by adding a `META-INF/spring.factories` and packaging a jar file that the applications all use as a library.

The `javadoc:org.springframework.boot.SpringApplication[]` sends some special `javadoc:org.springframework.test.context.event.ApplicationEvents[]` to the listeners (some even before the context is created) and then registers the listeners for events published by the `javadoc:org.springframework.context.ApplicationContext[]` as well. See [reference:features/spring-application.html](#) in the “Spring Boot Features” section for a complete list.

It is also possible to customize the `javadoc:org.springframework.core.env.Environment[]` before the application context is refreshed by using `javadoc:org.springframework.boot.EnvironmentPostProcessor[]`. Each implementation should be registered in `META-INF/spring.factories`, as shown in the following example:

```
org.springframework.boot.EnvironmentPostProcessor=com.example.YourEnvironmentPostProcessor
```

The implementation can load arbitrary files and add them to the `javadoc:org.springframework.core.env.Environment[]`. For instance, the following example loads a YAML configuration file from the classpath:

```
include-code::MyEnvironmentPostProcessor[]
```



The `javado:org.springframework.core.env.Environment[]` has already been prepared with all the usual property sources that Spring Boot loads by default. It is therefore possible to get the location of the file from the environment. The preceding example adds the **custom-resource** property source at the end of the list so that a key defined in any of the usual other locations takes precedence. A custom implementation may define another order.



While using `javado:org.springframework.context.annotation.PropertySource[format=annotation]` on your `javado:org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` may seem to be a convenient way to load a custom resource in the `javado:org.springframework.core.env.Environment[]`, we do not recommend it. Such property sources are not added to the `javado:org.springframework.core.env.Environment[]` until the application context is being refreshed. This is too late to configure certain properties such as `logging.*` and `spring.main.*` which are read before refresh begins.

98.4. Build an ApplicationContext Hierarchy (Adding a Parent or Root Context)

You can use the `javado:org.springframework.boot.builder.SpringApplicationBuilder[]` class to create parent/child `javado:org.springframework.context.ApplicationContext[]` hierarchies. See [reference:features/spring-application.html](#) in the “Spring Boot Features” section for more information.

98.5. Create a Non-web Application

Not all Spring applications have to be web applications (or web services). If you want to execute some code in a `main` method but also bootstrap a Spring application to set up the infrastructure to use, you can use the `javado:org.springframework.boot.SpringApplication[]` features of Spring Boot. A `javado:org.springframework.boot.SpringApplication[]` changes its `javado:org.springframework.context.ApplicationContext[]` class, depending on whether it thinks it needs a web application or not. The first thing you can do to help it is to leave server-related dependencies (such as the servlet API) off the classpath. If you cannot do that (for example, if you run two applications from the same code base) then you can explicitly call `setWebApplicationType(WebApplicationType.NONE)` on your `javado:org.springframework.boot.SpringApplication[]` instance or set the `applicationContextClass` property (through the Java API or with external properties). Application code that you want to run as your business logic can be implemented as a `javado:org.springframework.boot.CommandLineRunner[]` and dropped into the context as a `javado:org.springframework.context.annotation.Bean[format=annotation]` definition.

99. Batch Applications

A number of questions often arise when people use Spring Batch from within a Spring Boot application. This section addresses those questions.

99.1. Specifying a Batch Data Source

Batch applications that store job details in an SQL database require a `javax.sql.DataSource[]` bean. A single `javax.sql.DataSource[]` bean is required by default. To have Spring Batch use a `javax.sql.DataSource[]` other than the application's main `javax.sql.DataSource[]`, declare a `javax.sql.DataSource[]` bean, annotating its `javax.annotation.AnnotationBean[format=annotation]` method with `javax.annotation.AnnotationBean[format=annotation]`. If you do so and want two data sources (for example by retaining the main auto-configured `javax.sql.DataSource[]`), set the `defaultCandidate` attribute of the `javax.annotation.AnnotationBean[format=annotation]` annotation to `false`.

99.2. Specifying a Batch Transaction Manager

Similar to [batch.html](#), you can define a `org.springframework.transaction.PlatformTransactionManager[]` for use in batch processing by annotating its `javax.annotation.AnnotationBean[format=annotation]` method with `javax.annotation.AnnotationBean[format=annotation]`. If you do so and want two transaction managers (for example by retaining the auto-configured `org.springframework.transaction.PlatformTransactionManager[]`), set the `defaultCandidate` attribute of the `javax.annotation.AnnotationBean[format=annotation]` annotation to `false`.

99.3. Specifying a Batch Task Executor

Similar to [batch.html](#), you can define a `org.springframework.core.task.TaskExecutor[]` for use in batch processing by annotating its `javax.annotation.AnnotationBean[format=annotation]` method with `javax.annotation.AnnotationBean[format=annotation]`. If you do so and want two task executors (for example by retaining the auto-configured `org.springframework.core.task.TaskExecutor[]`), set the `defaultCandidate` attribute of the `javax.annotation.AnnotationBean[format=annotation]` annotation to `false`.

99.4. Running From the Command Line

Spring Boot converts any command line argument starting with `--` to a property to add to the `org.springframework.core.env.Environment[]`, see [accessing command line properties](#). This should not be used to pass arguments to batch jobs. To specify batch arguments on the command line, use the regular format (that is without `--`), as shown in the following example:

```
$ java -jar myapp.jar someParameter=someValue anotherParameter=anotherValue
```

SHELL

If you specify a property of the `org.springframework.core.env.Environment[]` on the command line, it is ignored by the job. Consider the following command:

```
$ java -jar myapp.jar --server.port=7070 someParameter=someValue
```

SHELL

This provides only one argument to the batch job: `someParameter=someValue`.

99.5. Restarting a Stopped or Failed Job

To restart a failed `org.springframework.batch.core.job.Job[]`, all parameters (identifying and non-identifying) must be re-specified on the command line. Non-identifying parameters are **not** copied from the previous execution. This allows them to be modified or removed.



When you're using a custom `org.springframework.batch.core.job.parameters.JobParametersIncrementer[]`, you have to gather all parameters managed by the incrementer to restart a failed execution.

100. Build

Spring Boot includes build plugins for Maven and Gradle. This section answers common questions about these plugins.

100.1. Generate Build Information

Both the Maven plugin and the Gradle plugin allow generating build information containing the coordinates, name, and version of the project. The plugins can also be configured to add additional properties through configuration. When such a file is present, Spring Boot auto-configures a `javadoc:org.springframework.boot.info.BuildProperties[]` bean.

To generate build information with Maven, add an execution for the `build-info` goal, as shown in the following example:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <version>{version-spring-boot}</version>
      <executions>
        <execution>
          <goals>
            <goal>build-info</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

XML



See the [Spring Boot Maven Plugin documentation](#) for more details.

The following example does the same with Gradle:

```
springBoot {
    buildInfo()
}
```

GRADLE



See the [Spring Boot Gradle Plugin documentation](#) for more details.

100.2. Generate Git Information

Both Maven and Gradle allow generating a `git.properties` file containing information about the state of your `git` source code repository when the project was built.

For Maven users, the `spring-boot-starter-parent` POM includes a pre-configured plugin to generate a `git.properties` file. To use it, add the following declaration for the `{url-git-commit-id-maven-plugin}[Git Commit Id Plugin]` to your POM:

```
<build>
  <plugins>
    <plugin>
      <groupId>io.github.git-commit-id</groupId>
      <artifactId>git-commit-id-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

XML

Gradle users can achieve the same result by using the [gradle-git-properties](https://plugins.gradle.org/plugin/com.gorylenko.gradle-git-properties) (<https://plugins.gradle.org/plugin/com.gorylenko.gradle-git-properties>) plugin, as shown in the following example:

```
plugins {
    id "com.gorylenko.gradle-git-properties" version "2.4.1"
}
```

GRADLE

Both the Maven and Gradle plugins allow the properties that are included in `git.properties` to be configured.



The commit time in `git.properties` is expected to match the following format: `yyyy-MM-dd'T'HH:mm:ssZ`. This is the default format for both plugins listed above. Using this format lets the time be parsed into a `javadoc:java.util.Date[]` and its format, when serialized to JSON, to be controlled by Jackson's date serialization configuration settings.

100.3. Generate a CycloneDX SBOM

Both Maven and Gradle allow generating a CycloneDX SBOM at project build time.

For Maven users, the `spring-boot-starter-parent` POM includes a pre-configured plugin to generate the SBOM. To use it, add the following declaration for the `{url-cyclonedx-docs-maven-plugin}[cyclonedx-maven-plugin]` to your POM:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.cyclonedx</groupId>
      <artifactId>cyclonedx-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Gradle users can achieve the same result by using the `{url-cyclonedx-docs-gradle-plugin}[ cyclonedx-gradle-plugin ]` plugin, as shown in the following example:

```
plugins {
    id 'org.cyclonedx.bom' version '3.0.0'
}
```

100.4. Customize Dependency Versions

The `spring-boot-dependencies` POM manages the versions of common dependencies. The Spring Boot plugins for Maven and Gradle allow these managed dependency versions to be customized using build properties.



Each Spring Boot release is designed and tested against this specific set of third-party dependencies. Overriding versions may cause compatibility issues.

To override dependency versions with Maven, see [maven-plugin:using.html](#) in the Maven plugin's documentation.

To override dependency versions in Gradle, see [gradle-plugin:managing-dependencies.html](#) in the Gradle plugin's documentation.

100.5. Create an Executable JAR with Maven

The `spring-boot-maven-plugin` can be used to create an executable “fat” JAR. If you use the `spring-boot-starter-parent` POM, you can declare the plugin and your jars are repackaged as follows:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

If you do not use the parent POM, you can still use the plugin. However, you must additionally add an `<executions>` section, as follows:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <version>{version-spring-boot}</version>
      <executions>
        <execution>
          <goals>
            <goal>repackage</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

See the [plugin documentation](#) for full usage details.

100.6. Use a Spring Boot Application as a Dependency

Like a war file, a Spring Boot application is not intended to be used as a dependency. If your application contains classes that you want to share with other projects, the recommended approach is to move that code into a separate module. The separate module can then be depended upon by your application and other projects.

If you cannot rearrange your code as recommended above, Spring Boot's Maven and Gradle plugins must be configured to produce a separate artifact that is suitable for use as a dependency. The executable archive cannot be used as a dependency as the [executable jar format](#) packages application classes in `BOOT-INF/classes`. This means that they cannot be found when the executable jar is used as a dependency.

To produce the two artifacts, one that can be used as a dependency and one that is executable, a classifier must be specified. This classifier is applied to the name of the executable archive, leaving the default archive for use as a dependency.

To configure a classifier of `exec` in Maven, you can use the following configuration:

```

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <classifier>exec</classifier>
      </configuration>
    </plugin>
  </plugins>
</build>

```

100.7. Extract Specific Libraries When an Executable Jar Runs

Most nested libraries in an executable jar do not need to be unpacked in order to run. However, certain libraries can have problems. For example, JRuby includes its own nested jar support, which assumes that the `jruby-complete.jar` is always directly available as a file in its own right.

To deal with any problematic libraries, you can flag that specific nested jars should be automatically unpacked when the executable jar first runs. Such nested jars are written beneath the temporary directory identified by the `java.io.tmpdir` system property.



Care should be taken to ensure that your operating system is configured so that it will not delete the jars that have been unpacked to the temporary directory while the application is still running.

For example, to indicate that JRuby should be flagged for unpacking by using the Maven Plugin, you would add the following configuration:

```

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <requiresUnpack>
          <dependency>
            <groupId>org.jruby</groupId>
            <artifactId>jruby-complete</artifactId>
          </dependency>
        </requiresUnpack>
      </configuration>
    </plugin>
  </plugins>
</build>

```

100.8. Create a Non-executable JAR with Exclusions

Often, if you have an executable and a non-executable jar as two separate build products, the executable version has additional configuration files that are not needed in a library jar. For example, the `application.yaml` configuration file might be excluded from the non-executable JAR.

In Maven, the executable jar must be the main artifact and you can add a classified jar for the library, as follows:

```

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <plugin>
      <artifactId>maven-jar-plugin</artifactId>
      <executions>
        <execution>
          <id>lib</id>
          <phase>package</phase>
          <goals>
            <goal>jar</goal>
          </goals>
          <configuration>
            <classifier>lib</classifier>
            <excludes>
              <exclude>application.yaml</exclude>
            </excludes>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>

```

100.9. Remote Debug a Spring Boot Application Started with Maven

To attach a remote debugger to a Spring Boot application that was started with Maven, you can use the `jvmArguments` property of the [maven plugin](#).

See [this example](#) for more details.

100.10. Build an Executable Archive From Ant without Using spring-boot-antlib

To build with Ant, you need to grab dependencies, compile, and then create a jar or war archive. To make it executable, you can either use the `spring-boot-antlib` module or you can follow these instructions:

1. If you are building a jar, package the application's classes and resources in a nested `BOOT-INF/classes` directory. If you are building a war, package the application's classes in a nested `WEB-INF/classes` directory as usual.
2. Add the runtime dependencies in a nested `BOOT-INF/lib` directory for a jar or `WEB-INF/lib` for a war. Remember **not** to compress the entries in the archive.
3. Add the provided (embedded container) dependencies in a nested `BOOT-INF/lib` directory for a jar or `WEB-INF/lib-provided` for a war. Remember **not** to compress the entries in the archive.
4. Add the `spring-boot-loader` classes at the root of the archive (so that the `Main-Class` is available).
5. Use the appropriate launcher (such as `javadoc.org.springframework.boot.loader.launch.JarLauncher[]` for a jar file) as a `Main-Class` attribute in the manifest and specify the other properties it needs as manifest entries—principally, by setting a `Start-Class` property.

The following example shows how to build an executable archive with Ant:

```
<target name="build" depends="compile">
  <jar destfile="target/${ant.project.name}-${spring-boot.version}.jar" compress="false">
    <mappedresources>
      <fileset dir="target/classes" />
      <globmapper from="*" to="BOOT-INF/classes/*" />
    </mappedresources>
    <mappedresources>
      <fileset dir="src/main/resources" erroronmissingdir="false" />
      <globmapper from="*" to="BOOT-INF/classes/*" />
    </mappedresources>
    <mappedresources>
      <fileset dir="${lib.dir}/runtime" />
      <globmapper from="*" to="BOOT-INF/lib/*" />
    </mappedresources>
    <zipfileset src="${lib.dir}/loader/spring-boot-loader-jar-${spring-boot.version}.jar" />
    <manifest>
      <attribute name="Main-Class" value="org.springframework.boot.loader.launch.JarLauncher" />
      <attribute name="Start-Class" value="${start-class}" />
    </manifest>
  </jar>
</target>
```

XML

101. Data Access

Spring Boot includes a number of starters for working with data sources. This section answers questions related to doing so.

101.1. Configure a Custom DataSource

To configure your own `javax.sql.DataSource[]`, define a `org.springframework.context.annotation.Bean[format=annotation]` of that type in your configuration. Spring Boot reuses your `javax.sql.DataSource[]` anywhere one is required, including database initialization. If you need to externalize some settings, you can bind your `javax.sql.DataSource[]` to the environment (see [reference:features/external-config.html](#)).

The following example shows how to define a data source in a bean:

```
include-code::custom/MyDataSourceConfiguration[]
```

The following example shows how to define a data source by setting its properties:

```
app:
  datasource:
    url: "jdbc:h2:mem:mydb"
    username: "sa"
    pool-size: 30
```

Assuming that `SomeDataSource` has regular JavaBean properties for the URL, the username, and the pool size, these settings are bound automatically before the `javax.sql.DataSource[]` is made available to other components.

Spring Boot also provides a utility builder class, called `org.springframework.boot.jdbc.DataSourceBuilder[]`, that can be used to create one of the standard data sources (if it is on the classpath). The builder can detect which one to use based on what is available on the classpath. It also auto-detects the driver based on the JDBC URL.

The following example shows how to create a data source by using a `org.springframework.boot.jdbc.DataSourceBuilder[]`:

```
include-code::builder/MyDataSourceConfiguration[]
```

To run an app with that `javax.sql.DataSource[]`, all you need is the connection information. Pool-specific settings can also be provided. Check the implementation that is going to be used at runtime for more details.

The following example shows how to define a JDBC data source by setting properties:

```
app:
  datasource:
    url: "jdbc:mysql://localhost/test"
    username: "dbuser"
    password: "dbpass"
    pool-size: 30
```

However, there is a catch due to the method's `javax.sql.DataSource[]` return type. This hides the actual type of the connection pool so no configuration property metadata is generated for your custom `javax.sql.DataSource[]` and no auto-completion is available in your IDE. To address this problem, use the builder's `type(Class)` method to specify the type of `javax.sql.DataSource[]` to be built and update the method's return type. For example, the following shows how to create a `com.zaxxer.hikari.HikariDataSource[]` with `org.springframework.boot.jdbc.DataSourceBuilder[]`:

```
include-code::simple/MyDataSourceConfiguration[]
```

Unfortunately, this basic setup does not work because Hikari has no `url` property. Instead, it has a `jdbc-url` property which means that you must rewrite your configuration as follows:

```
app:
  datasource:
    jdbc-url: "jdbc:mysql://localhost/test"
    username: "dbuser"
    password: "dbpass"
    pool-size: 30
```

To address this problem, make use of `org.springframework.boot.jdbc.autoconfigure.DataSourceProperties[]` which will handle the `url` to `jdbc-url` translation for you. You can initialize a `org.springframework.boot.jdbc.DataSourceBuilder[]` from the state of any `org.springframework.boot.jdbc.autoconfigure.DataSourceProperties[]` object using its `initializeDataSourceBuilder()` method. You could inject the `org.springframework.boot.jdbc.autoconfigure.DataSourceProperties[]` that Spring Boot creates automatically, however, that would split your configuration across `spring.datasource.*` and `app.datasource.*`. To avoid this, define a custom `org.springframework.boot.jdbc.autoconfigure.DataSourceProperties[]` with a custom configuration properties prefix, as shown in the following example:

```
include-code::configurable/MyDataSourceConfiguration[]
```

This setup is equivalent to what Spring Boot does for you by default, except that the pool's type is specified in code and its settings are exposed as `app.datasource.configuration.*` properties. `org.springframework.boot.jdbc.autoconfigure.DataSourceProperties[]` takes care of the `url` to `jdbc-url` translation, so you can configure it as follows:

```

app:
  datasource:
    url: "jdbc:mysql://localhost/test"
    username: "dbuser"
    password: "dbpass"
    configuration:
      maximum-pool-size: 30

```

Note that, as the custom configuration specifies in code that Hikari should be used, `app.datasource.type` will have no effect.

As described in [reference:data/sql.html](#), `jdbc.DataSourceBuilder[]` supports several different connection pools. To use a pool other than Hikari, add it to the classpath, use the `type(Class)` method to specify the pool class to use, and update the `jdbc.DataSourceBuilder[]` method's return type to match. This will also provide you with configuration property metadata for the specific connection pool that you've chosen.



Spring Boot will expose Hikari-specific settings to `spring.datasource.hikari`. This example uses a more generic `configuration` sub namespace as the example does not support multiple datasource implementations.

See [reference:data/sql.html](#) and the `{code-spring-boot-jdbc-src}/autoconfigure/DataSourceAutoConfiguration.java` [`DataSourceAutoConfiguration`] class for more details.

101.2. Configure Two DataSources

To define an additional `jdbc.DataSource[]`, an approach that's similar to the previous section can be used. A key difference is that the `jdbc.DataSource[]` `jdbc.DataSourceBuilder[]` must be declared with `defaultCandidate=false`. This prevents the auto-configured `jdbc.DataSource[]` from backing off.



The [{url-spring-framework-docs}/core/beans/dependencies/factory-autowire.html#beans-factory-autowire-candidate](#) [Spring Framework reference documentation] describes this feature in more details.

To allow the additional `jdbc.DataSource[]` to be injected where it's needed, also annotate it with `jdbc.DataSourceBuilder[]` as shown in the following example:

```
include-code::MyAdditionalDataSourceConfiguration[]
```

To consume the additional `jdbc.DataSource[]`, annotate the injection point with the same `jdbc.DataSourceBuilder[]`.

The auto-configured and additional data sources can be configured as follows:

```

spring:
  datasource:
    url: "jdbc:mysql://localhost/first"
    username: "dbuser"
    password: "dbpass"
    configuration:
      maximum-pool-size: 30
app:
  datasource:
    url: "jdbc:mysql://localhost/second"
    username: "dbuser"
    password: "dbpass"
    max-total: 30

```

More advanced, implementation-specific, configuration of the auto-configured `jdbc.DataSource[]` is available through the `spring.datasource.configuration.*` properties. You can apply the same concept to the additional `jdbc.DataSource[]` as well, as shown in the following example:

```
include-code::MyCompleteAdditionalDataSourceConfiguration[]
```

The preceding example configures the additional data source with the same logic as Spring Boot would use in auto-configuration. Note that the `app.datasource.configuration.*` properties provide advanced settings based on the chosen implementation.

As with [configuring a single custom jdbc.DataSource\[\]](#), the type of one or both of the `jdbc.DataSource[]` beans can be customized using the `type(Class)` method on `jdbc.DataSourceBuilder[]`. See [reference:data/sql.html](#) for details of the supported types.

101.3. Use Spring Data Repositories

Spring Data can create implementations of `jdbc.Repository[]` interfaces of various flavors. Spring Boot handles all of that for you, as long as those `jdbc.Repository[]` implementations are included in one of the [auto-configuration packages](#), typically the package (or a sub-package) of your main application class that is annotated with `jdbc.SpringBootApplication[]` or `jdbc.EnableAutoConfiguration[]`.

For many applications, all you need is to put the right Spring Data dependencies on your classpath. There is a `spring-boot-starter-data-jpa` for JPA, `spring-boot-starter-data-mongodb` for MongoDB, and various other starters for supported technologies. To get started, create some repository interfaces to handle your `jdbc.Entity[]` objects.

Spring Boot determines the location of your `jdbc.Repository[]` implementations by scanning the [auto-configuration packages](#). For more control, use the `@Enable..Repositories` annotations from Spring Data.

For more about Spring Data, see the [Spring Data project page](#).

101.4. Separate @Entity Definitions from Spring Configuration

Spring Boot determines the location of your `javax.persistence.Entity` definitions by scanning the [auto-configuration packages](#). For more control, use the `org.springframework.boot.autoconfigure.EntityScan` annotation, as shown in the following example:

```
include-code::MyApplication[]
```

101.5. Filter Scanned @Entity Definitions

It is possible to filter the `javax.persistence.Entity` definitions using a `org.springframework.orm.jpa.persistenceunit.ManagedClassNameFilter` bean. This can be useful in tests when only a sub-set of the available entities should be considered. In the following example, only entities from the `com.example.app.customer` package are included:

```
include-code::MyEntityScanConfiguration[]
```

101.6. Configure JPA Properties

Spring Data JPA already provides some vendor-independent configuration options (such as those for SQL logging), and Spring Boot exposes those options and a few more for Hibernate as external configuration properties. Some of them are automatically detected according to the context so you should not have to set them.

The `spring.jpa.hibernate.ddl-auto` is a special case, because, depending on runtime conditions, it has different defaults. If an embedded database is used and no schema manager (such as Liquibase or Flyway) is handling the `javax.sql.DataSource`, it defaults to `create-drop`. In all other cases, it defaults to `none`.

The dialect to use is detected by the JPA provider. If you prefer to set the dialect yourself, set the `configprop:spring.jpa.database-platform` property.

The most common options to set are shown in the following example:

```
spring:
  jpa:
    hibernate:
      naming:
        physical-strategy: "com.example.MyPhysicalNamingStrategy"
    show-sql: true
```

In addition, all properties in `spring.jpa.properties.*` are passed through as normal JPA properties (with the prefix stripped) when the local `javax.persistence.EntityManagerFactory` is created.



You need to ensure that names defined under `spring.jpa.properties.*` exactly match those expected by your JPA provider. Spring Boot will not attempt any kind of relaxed binding for these entries.

For example, if you want to configure Hibernate's batch size you must use `spring.jpa.properties.hibernate.jdbc.batch_size`. If you use other forms, such as `batchSize` or `batch-size`, Hibernate will not apply the setting.



If you need to apply advanced customization to Hibernate properties, consider registering a `org.springframework.boot.hibernate.autoconfigure.HibernatePropertiesCustomizer` bean that will be invoked prior to creating the `javax.persistence.EntityManagerFactory`. This takes precedence over anything that is applied by the auto-configuration.

101.7. Configure Hibernate Naming Strategy

Hibernate uses `{url-hibernate-userguide}#naming` (two different naming strategies) to map names from the object model to the corresponding database names. The fully qualified class name of the physical and the implicit strategy implementations can be configured by setting the `spring.jpa.hibernate.naming.physical-strategy` and `spring.jpa.hibernate.naming.implicit-strategy` properties, respectively. Alternatively, if `org.hibernate.boot.model.naming.ImplicitNamingStrategy` or `org.hibernate.boot.model.naming.PhysicalNamingStrategy` beans are available in the application context, Hibernate will be automatically configured to use them.

By default, Spring Boot configures the physical naming strategy with `org.hibernate.boot.model.naming.CamelCaseToUnderscoresNamingStrategy`. Using this strategy, all dots are replaced by underscores and camel casing is replaced by underscores as well. Additionally, by default, all table names are generated in lower case. For example, a `TelephoneNumber` entity is mapped to the `telephone_number` table. If your schema requires mixed-case identifiers, define a custom `org.hibernate.boot.model.naming.CamelCaseToUnderscoresNamingStrategy` bean, as shown in the following example:

```
include-code::spring/MyHibernateConfiguration[]
```

If you prefer to use Hibernate's default instead, set the following property:

```
spring:
  jpa:
    hibernate:
      naming:
        physical-strategy: org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
```

Alternatively, you can configure the following bean:

```
include-code::standard/MyHibernateConfiguration[]
```


See `{code-spring-boot-hibernate-src}/autoconfigure/HibernateJpaAutoConfiguration.java` [`HibernateJpaAutoConfiguration`] and `{code-spring-boot-jpa-src}/autoconfigure/JpaBaseConfiguration.java` [`JpaBaseConfiguration`] for more details.

101.8. Configure Hibernate Second-Level Caching

Hibernate `{url-hibernate-userguide}#caching[second-level cache]` can be configured for a range of cache providers. Rather than configuring Hibernate to lookup the cache provider again, it is better to provide the one that is available in the context whenever possible.

To do this with JCache, first make sure that `org.hibernate.orm:hibernate-jcache` is available on the classpath. Then, add a `javadoc:org.springframework.boot.hibernate.autoconfigure.HibernatePropertiesCustomizer[]` bean as shown in the following example:

```
include-code::MyHibernateSecondLevelCacheConfiguration[]
```

This customizer will configure Hibernate to use the same `javadoc:org.springframework.cache.CacheManager[]` as the one that the application uses. It is also possible to use separate `javadoc:org.springframework.cache.CacheManager[]` instances. For details, see `{url-hibernate-userguide}#caching-provider-jcache` [the Hibernate user guide].

101.9. Use Dependency Injection in Hibernate Components

By default, Spring Boot registers a `javadoc:org.hibernate.resource.beans.container.spi.BeanContainer[]` implementation that uses the `javadoc:org.springframework.beans.factory.BeanFactory[]` so that converters and entity listeners can use regular dependency injection.

You can disable or tune this behavior by registering a `javadoc:org.springframework.boot.hibernate.autoconfigure.HibernatePropertiesCustomizer[]` that removes or changes the `hibernate.resource.beans.container` property.

101.10. Use a Custom EntityManagerFactory

To take full control of the configuration of the `javadoc:jakarta.persistence.EntityManagerFactory[]`, you need to add a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` named 'entityManagerFactory'. Spring Boot auto-configuration switches off its entity manager in the presence of a bean of that type.



When you create a bean for `javadoc:org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean[]` yourself, any customization that was applied during the creation of the auto-configured `javadoc:org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean[]` is lost. Make sure to use the auto-configured `javadoc:org.springframework.boot.jpa.EntityManagerFactoryBuilder[]` to retain JPA and vendor properties. This is particularly important if you were relying on `spring.jpa.*` properties for configuring things like the naming strategy or the DDL mode.

101.11. Using Multiple EntityManagerFactories

If you need to use JPA against multiple datasources, you likely need one `javadoc:jakarta.persistence.EntityManagerFactory[]` per datasource. The `javadoc:org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean[]` from Spring ORM allows you to configure an `javadoc:jakarta.persistence.EntityManagerFactory[]` for your needs. You can also reuse `javadoc:org.springframework.boot.jpa.autoconfigure.JpaProperties[]` to bind settings for a second `javadoc:jakarta.persistence.EntityManagerFactory[]`. Building upon [the example for configuring a second javadoc:javax.sql.DataSource\[\]](#), a second `javadoc:jakarta.persistence.EntityManagerFactory[]` can be defined as shown in the following example:

```
include-code::MyAdditionalEntityManagerFactoryConfiguration[]
```

The example above creates an `javadoc:jakarta.persistence.EntityManagerFactory[]` using the `javadoc:javax.sql.DataSource[]` bean qualified with `@Qualifier("second")`. It scans entities located in the same package as `Order`. It is possible to map additional JPA properties using the `app.jpa` namespace. The use of `@Bean(defaultCandidate=false)` allows the `secondJpaProperties` and `secondEntityManagerFactory` beans to be defined without interfering with auto-configured beans of the same type.



The `{url-spring-framework-docs}/core/beans/dependencies/factory-autowire.html#beans-factory-autowire-candidate` [Spring Framework reference documentation] describes this feature in more details.

You should provide a similar configuration for any more additional data sources for which you need JPA access. To complete the picture, you need to configure a `javadoc:org.springframework.orm.jpa.JpaTransactionManager[]` for each `javadoc:jakarta.persistence.EntityManagerFactory[]` as well. Alternatively, you might be able to use a JTA transaction manager that spans both.

If you use Spring Data, you need to configure `javadoc:org.springframework.data.jpa.repository.config.EnableJpaRepositories[format=annotation]` accordingly, as shown in the following examples:

```
include-code::OrderConfiguration[]
```

```
include-code::CustomerConfiguration[]
```

101.12. Use a Traditional persistence.xml File

Spring Boot will not search for or use a `META-INF/persistence.xml` by default. If you prefer to use a traditional `persistence.xml`, you need to define your own `javadoc:org.springframework.context.annotation.Bean[format=annotation]` of type `javadoc:org.springframework.orm.jpa.LocalEntityManagerFactoryBean[]` (with an ID of 'entityManagerFactory') and set the persistence unit name there.

See `{code-spring-boot-jpa-src}/autoconfigure/JpaBaseConfiguration.java` [`JpaBaseConfiguration`] for the default settings.

101.13. Use Spring Data JPA and Mongo Repositories

Spring Data JPA and Spring Data Mongo can both automatically create `javadoc:org.springframework.data.repository.Repository[]` implementations for you. If they are both present on the classpath, you might have to do some extra configuration to tell Spring Boot which repositories to create. The most explicit way to do that is to use the standard Spring Data `javadoc:org.springframework.data.jpa.repository.config.EnableJpaRepositories[format=annotation]` and

`javax.persistence.repository.config.EnableMongoRepositories` [format=annotation] annotations and provide the location of your `javax.persistence.repository.Repository` interfaces.

There are also flags (`spring.data.*.repositories.enabled` and `spring.data.*.repositories.type`) that you can use to switch the auto-configured repositories on and off in external configuration. Doing so is useful, for instance, in case you want to switch off the Mongo repositories and still use the auto-configured `javax.persistence.repository.config.MongoTemplate`.

The same obstacle and the same features exist for other auto-configured Spring Data repository types (Elasticsearch, Redis, and others). To work with them, change the names of the annotations and flags accordingly.

101.14. Customize Spring Data's Web Support

Spring Data provides web support that simplifies the use of Spring Data repositories in a web application. Spring Boot provides properties in the `spring.data.web` namespace for customizing its configuration. Note that if you are using Spring Data REST, you must use the properties in the `spring.data.rest` namespace instead.

101.15. Expose Spring Data Repositories as REST Endpoint

Spring Data REST can expose the `javax.persistence.repository.Repository` implementations as REST endpoints for you, provided Spring MVC has been enabled for the application.

Spring Boot exposes a set of useful properties (from the `spring.data.rest` namespace) that customize the

`javax.persistence.repository.config.RepositoryRestConfiguration`. If you need to provide additional customization, you should use a `javax.persistence.repository.config.RepositoryRestConfigurer` bean.



If you do not specify any order on your custom `javax.persistence.repository.config.RepositoryRestConfigurer`, it runs after the one Spring Boot uses internally. If you need to specify an order, make sure it is higher than 0.

101.16. Configure a Component that is Used by JPA

If you want to configure a component that JPA uses, then you need to ensure that the component is initialized before JPA. When the component is auto-configured, Spring Boot takes care of this for you. For example, when Flyway is auto-configured, Hibernate is configured to depend on Flyway so that Flyway has a chance to initialize the database before Hibernate tries to use it.

If you are configuring a component yourself, you can use an `javax.persistence.repository.config.EntityManagerFactoryDependsOnPostProcessor` subclass as a convenient way of setting up the necessary dependencies. For example, if you use Hibernate Search with Elasticsearch as its index manager, any `javax.persistence.repository.config.EntityManagerFactory` beans must be configured to depend on the `elasticsearchClient` bean, as shown in the following example:

```
include-code::ElasticsearchEntityManagerFactoryDependsOnPostProcessor[]
```

101.17. Configure jOOQ with Two DataSources

If you need to use jOOQ with multiple data sources, you should create your own `javax.persistence.repository.config.jooq.DSLContext` for each one. See `{code-spring-boot-jooq-src}/autoconfigure/JooqAutoConfiguration.java` [`JooqAutoConfiguration`] for more details.



In particular, `javax.persistence.repository.config.jooq.autoconfigure.ExceptionTranslatorExecuteListener` and `javax.persistence.repository.config.jooq.autoconfigure.SpringTransactionProvider` can be reused to provide similar features to what the auto-configuration does with a single `javax.persistence.repository.config.DataSource`.

102. Database Initialization

An SQL database can be initialized in different ways depending on what your stack is. Of course, you can also do it manually, provided the database is a separate process. It is recommended to use a single mechanism for schema generation.

102.1. Initialize a Database Using Hibernate

You can set `configprop:spring.jpa.hibernate.ddl-auto[]` to control Hibernate's database initialization. Supported values are `none`, `validate`, `update`, `create`, and `create-drop`. Spring Boot chooses a default value for you based on whether you are using an embedded database. An embedded database is identified by looking at the `javax.sql.Connection[]` type and JDBC url. `hsqldb`, `h2`, or `derby` (deprecated) are embedded databases and others are not. If an embedded database is identified and no schema manager (Flyway or Liquibase) has been detected, `ddl-auto` defaults to `create-drop`. In all other cases, it defaults to `none`.

Be careful when switching from in-memory to a 'real' database that you do not make assumptions about the existence of the tables and data in the new platform. You either have to set `ddl-auto` explicitly or use one of the other mechanisms to initialize the database.



You can output the schema creation by enabling the `org.hibernate.SQL` logger. This is done for you automatically if you enable the [debug mode](#).

In addition, a file named `import.sql` in the root of the classpath is executed on startup if Hibernate creates the schema from scratch (that is, if the `ddl-auto` property is set to `create` or `create-drop`). This can be useful for demos and for testing if you are careful but is probably not something you want to be on the classpath in production. It is a Hibernate feature (and has nothing to do with Spring).

102.2. Initialize a Database Using Basic SQL Scripts

Spring Boot can automatically create the schema (DDL scripts) of your JDBC `javax.sql.DataSource[]` or R2DBC `io.r2dbc.spi.ConnectionFactory[]` and initialize its data (DML scripts).

By default, it loads schema scripts from `optional:classpath*:schema.sql` and data scripts from `optional:classpath*:data.sql`. The locations of these schema and data scripts can be customized using `configprop:spring.sql.init.schema-locations[]` and `configprop:spring.sql.init.data-locations[]` respectively. The `optional:` prefix means that the application will start even when the files do not exist. To have the application fail to start when the files are absent, remove the `optional:` prefix.

In addition, Spring Boot processes the `optional:classpath*:schema-${platform}.sql` and `optional:classpath*:data-${platform}.sql` files (if present), where `${platform}` is the value of `configprop:spring.sql.init.platform[]`. This allows you to switch to database-specific scripts if necessary. For example, you might choose to set it to the vendor name of the database (`hsqldb`, `h2`, `oracle`, `mysql`, `postgresql`, and so on).

By default, SQL database initialization is only performed when using an embedded in-memory database. To always initialize an SQL database, irrespective of its type, set `configprop:spring.sql.init.mode[]` to `always`. Similarly, to disable initialization, set `configprop:spring.sql.init.mode[]` to `never`. By default, Spring Boot enables the fail-fast feature of its script-based database initializer. This means that, if the scripts cause exceptions, the application fails to start. You can tune that behavior by setting `configprop:spring.sql.init.continue-on-error[]`.

Script-based `javax.sql.DataSource[]` initialization is performed, by default, before any JPA `jakarta.persistence.EntityManagerFactory[]` beans are created. `schema.sql` can be used to create the schema for JPA-managed entities and `data.sql` can be used to populate it. While we do not recommend using multiple data source initialization technologies, if you want script-based `javax.sql.DataSource[]` initialization to be able to build upon the schema creation performed by Hibernate, set `configprop:spring.jpa.defer-datasource-initialization[]` to `true`. This will defer data source initialization until after any `jakarta.persistence.EntityManagerFactory[]` beans have been created and initialized. `schema.sql` can then be used to make additions to any schema creation performed by Hibernate and `data.sql` can be used to populate it.



The initialization scripts support `--` for single line comments and `/* */` for block comments. Other comment formats are not supported.

If you are using a [higher-level database migration tool](#), like Flyway or Liquibase, you should use them alone to create and initialize the schema. Using the basic `schema.sql` and `data.sql` scripts alongside Flyway or Liquibase is not recommended and support will be removed in a future release.

If you need to initialize test data using a higher-level database migration tool, please see the sections about [Flyway](#) and [Liquibase](#).

102.3. Initialize a Spring Batch Database

If you use Spring Batch, it comes pre-packaged with SQL initialization scripts for most popular database platforms. Spring Boot can detect your database type and execute those scripts on startup. If you use an embedded database, this happens by default. You can also enable it for any database type, as shown in the following example:

```
spring:
  batch:
    jdbc:
      initialize-schema: "always"
```

You can also switch off the initialization explicitly by setting `spring.batch.jdbc.initialize-schema` to `never`.

102.4. Use a Higher-level Database Migration Tool

Spring Boot supports two higher-level migration tools: [Flyway](#) (<https://flywaydb.org/>) and [Liquibase](#) (<https://www.liquibase.org/>).

102.4.1. Execute Flyway Database Migrations on Startup

To automatically run Flyway database migrations on startup, add the appropriate Flyway module to your classpath. In-memory and file-based databases are supported by `org.flywaydb:flyway-core`. Otherwise, a database-specific module is required. For example, use `org.flywaydb:flyway-database-postgresql` with PostgreSQL and `org.flywaydb:flyway-mysql` with MySQL. See [the Flyway Documentation](#) (<https://documentation.red-gate.com/fd/supported-databases-and-versions-143754067.html>) for further details.

Typically, migrations are scripts in the form `V<VERSION>__<NAME>.sql` (with `<VERSION>` an underscore-separated version, such as `'1'` or `'2_1'`). By default, they are in a directory called `classpath:db/migration`, but you can modify that location by setting `spring.flyway.locations`. This is a comma-separated list of one or more `classpath:` or `filesystem:` locations. For example, the following configuration would search for scripts in both the default classpath location and the `/opt/migration` directory:

```
spring:
  flyway:
    locations: "classpath:db/migration,filesystem:/opt/migration"
```

You can also add a special `{vendor}` placeholder to use vendor-specific scripts. Assume the following:

```
spring:
  flyway:
    locations: "classpath:db/migration/{vendor}"
```

Rather than using `db/migration`, the preceding configuration sets the directory to use according to the type of the database (such as `db/migration/mysql` for MySQL). The list of supported databases is available in `jdbc.org.springframework.boot.jdbc.DatabaseDriver[]`.

Migrations can also be written in Java. Flyway will be auto-configured with any beans that implement `jdbc.org.flywaydb.core.api.migration.JavaMigration[]`.

`jdbc.org.springframework.boot.flyway.autoconfigure.FlywayProperties[]` provides most of Flyway's settings and a small set of additional properties that can be used to disable the migrations or switch off the location checking. If you need more control over the configuration, consider registering a `jdbc.org.springframework.boot.flyway.autoconfigure.FlywayConfigurationCustomizer[]` bean.

Spring Boot calls `Flyway.migrate()` to perform the database migration. If you would like more control, provide a `jdbc.org.springframework.context.annotation.Bean[format=annotation]` that implements `jdbc.org.springframework.boot.flyway.autoconfigure.FlywayMigrationStrategy[]`.

Flyway supports SQL and Java [callbacks](https://documentation.red-gate.com/fd/callbacks-275218509.html) (<https://documentation.red-gate.com/fd/callbacks-275218509.html>). To use SQL-based callbacks, place the callback scripts in the `classpath:db/migration` directory. To use Java-based callbacks, create one or more beans that implement `jdbc.org.flywaydb.core.api.callback.Callback[]`. Any such beans are automatically registered with `jdbc.org.flywaydb.core.Flyway[]`. They can be ordered by using `jdbc.org.springframework.core.annotation.Order[format=annotation]` or by implementing `jdbc.org.springframework.core.Ordered[]`.

By default, Flyway autowires the (`@Primary`) `jdbc.org.javafx.sql.DataSource[]` in your context and uses that for migrations. If you like to use a different `jdbc.org.javafx.sql.DataSource[]`, you can create one and mark its `jdbc.org.springframework.context.annotation.Bean[format=annotation]` as `jdbc.org.springframework.boot.flyway.autoconfigure.FlywayDataSource[format=annotation]`. If you do so and want two data sources (for example by retaining the main auto-configured `jdbc.org.javafx.sql.DataSource[]`), remember to set the `defaultCandidate` attribute of the `jdbc.org.springframework.context.annotation.Bean[format=annotation]` annotation to `false`. Alternatively, you can use Flyway's native `jdbc.org.javafx.sql.DataSource[]` by setting `spring.flyway.[url,user,password]` in external properties. Setting either `spring.flyway.url` or `spring.flyway.user` is sufficient to cause Flyway to use its own `jdbc.org.javafx.sql.DataSource[]`. If any of the three properties has not been set, the value of its equivalent `spring.datasource` property will be used.

You can also use Flyway to provide data for specific scenarios. For example, you can place test-specific migrations in `src/test/resources` and they are run only when your application starts for testing. Also, you can use profile-specific configuration to customize `spring.flyway.locations` so that certain migrations run only when a particular profile is active. For example, in `application-dev.properties`, you might specify the following setting:

```
spring:
  flyway:
    locations: "classpath:/db/migration,classpath:/dev/db/migration"
```

With that setup, migrations in `dev/db/migration` run only when the `dev` profile is active.

102.4.2. Execute Liquibase Database Migrations on Startup

To automatically run Liquibase database migrations on startup, add the `org.liquibase:liquibase-core` to your classpath.



When you add the `org.liquibase:liquibase-core` to your classpath, database migrations run by default for both during application startup and before your tests run. This behavior can be customized by using the `configprop:spring.liquibase.enabled[]` property, setting different values in the `main` and `test` configurations. It is not possible to use two different ways to initialize the database (for example Liquibase for application startup, JPA for test runs).

By default, the master change log is read from `db/changeLog/db.changeLog-master.yaml`, but you can change the location by setting `spring.liquibase.change-log`. In addition to YAML, Liquibase also supports JSON, XML, and SQL change log formats.

By default, Liquibase autowires the (`@Primary`) `jdbc.org.javafx.sql.DataSource[]` in your context and uses that for migrations. If you need to use a different `jdbc.org.javafx.sql.DataSource[]`, you can create one and mark its `jdbc.org.springframework.context.annotation.Bean[format=annotation]` as `jdbc.org.springframework.boot.liquibase.autoconfigure.LiquibaseDataSource[format=annotation]`. If you do so and want two data sources (for example by retaining the main auto-configured `jdbc.org.javafx.sql.DataSource[]`), remember to set the `defaultCandidate` attribute of the `jdbc.org.springframework.context.annotation.Bean[format=annotation]` annotation to `false`. Alternatively, you can use Liquibase's native `jdbc.org.javafx.sql.DataSource[]` by setting `spring.liquibase.[driver-class-name,url,user,password]` in external properties. Setting either `spring.liquibase.url` or `spring.liquibase.user` is sufficient to cause Liquibase to use its own `jdbc.org.javafx.sql.DataSource[]`. If any of the three properties has not been set, the value of its equivalent `spring.datasource` property will be used.

See `jdbc.org.springframework.boot.liquibase.autoconfigure.LiquibaseProperties[]` for details about available settings such as contexts, the default schema, and others.

You can also use a `Customizer<Liquibase>` bean if you want to customize the `jdbc.org.liquibase-javafx/liquibase.Liquibase[]` instance before it is being used.

102.4.3. Use Flyway for Test-only Migrations

If you want to create Flyway migrations which populate your test database, place them in `src/test/resources/db/migration`. A file named, for example, `src/test/resources/db/migration/V9999__test-data.sql` will be executed after your production migrations and only if you're running the tests. You can use this file to create the needed test data. This file will not be packaged in your uber jar or your container.

102.4.4. Use Liquibase for Test-only Migrations

If you want to create Liquibase migrations which populate your test database, you can leverage [Liquibase contexts](https://docs.liquibase.com/reference-guide/changelog-attributes/what-are-contexts) (<https://docs.liquibase.com/reference-guide/changelog-attributes/what-are-contexts>). See also the related [blog post](https://www.liquibase.com/blog/context-vs-labels) (<https://www.liquibase.com/blog/context-vs-labels>).

In practical terms, this translates into adding a `context:@test` attribute to changesets containing test data, for example:

```
--liquibase formatted sql
--changeset alice:1 context:@test
insert into project (id, name) values (1, 'Spring Boot');
```

SQL

And using `spring.liquibase.contexts=test` in environments where you would like changesets containing test data to be applied.

102.5. Depend Upon an Initialized Database

Database initialization is performed while the application is starting up as part of application context refresh. To allow an initialized database to be accessed during startup, beans that act as database initializers and beans that require that database to have been initialized are detected automatically. Beans whose initialization depends upon the database having been initialized are configured to depend upon those that initialize it. If, during startup, your application tries to access the database and it has not been initialized, you can configure additional detection of beans that initialize the database and require the database to have been initialized.

102.5.1. Detect a Database Initializer

Spring Boot will automatically detect beans of the following types that initialize an SQL database:

- `javadoc:org.springframework.boot.jdbc.init.DataSourceScriptDatabaseInitializer[]`
- `javadoc:jakarta.persistence.EntityManagerFactory[]`
- `javadoc:org.flywaydb.core.Flyway[]`
- `javadoc:org.springframework.boot.flyway.autoconfigure.FlywayMigrationInitializer[]`
- `javadoc:org.springframework.boot.r2dbc.init.R2dbcScriptDatabaseInitializer[]`
- `javadoc:liquibase.integration.spring.SpringLiquibase[]`

If you are using a third-party starter for a database initialization library, it may provide a detector such that beans of other types are also detected automatically. To have other beans be detected, register an implementation of `javadoc:org.springframework.boot.sql.init.dependency.DatabaseInitializerDetector[]` in `META-INF/spring.factories`.

102.5.2. Detect a Bean That Depends On Database Initialization

Spring Boot will automatically detect beans of the following types that depends upon database initialization:

- `javadoc:org.springframework.orm.jpa.AbstractEntityManagerFactoryBean[]` (unless `configprop:spring.jpa.defer-datasource-initialization[]` is set to `true`)
- `javadoc:org.jooq.DSLContext[]` (jOOQ)
- `javadoc:jakarta.persistence.EntityManagerFactory[]` (unless `configprop:spring.jpa.defer-datasource-initialization[]` is set to `true`)
- `javadoc:org.springframework.jdbc.core.simple.JdbcClient[]`
- `javadoc:org.springframework.jdbc.core.JdbcOperations[]`
- `javadoc:org.springframework.jdbc.core.namedparam.NamedParameterJdbcOperations[]`

If you are using a third-party starter data access library, it may provide a detector such that beans of other types are also detected automatically. To have other beans be detected, register an implementation of `javadoc:org.springframework.boot.sql.init.dependency.DependsOnDatabaseInitializationDetector[]` in `META-INF/spring.factories`. Alternatively, annotate the bean's class or its `javadoc:org.springframework.context.annotation.Bean[format=annotation]` method with `javadoc:org.springframework.boot.sql.init.dependency.DependsOnDatabaseInitialization[format=annotation]`.

103. Deploying to the Cloud

Spring Boot's executable jars are ready-made for most popular cloud PaaS (Platform-as-a-Service) providers. These providers tend to require that you “bring your own container”. They manage application processes (not Java applications specifically), so they need an intermediary layer that adapts *your* application to the *cloud*'s notion of a running process.

Two popular cloud providers, Heroku and Cloud Foundry, employ a “buildpack” approach. The buildpack wraps your deployed code in whatever is needed to *start* your application. It might be a JDK and a call to `java`, an embedded web server, or a full-fledged application server. A buildpack is pluggable, but ideally you should be able to get by with as few customizations to it as possible. This reduces the footprint of functionality that is not under your control. It minimizes divergence between development and production environments.

Ideally, your application, like a Spring Boot executable jar, has everything that it needs to run packaged within it.

In this section, we look at what it takes to get the [application that we developed](#) in the “Getting Started” section up and running in the Cloud.

103.1. Cloud Foundry

Cloud Foundry provides default buildpacks that come into play if no other buildpack is specified. The Cloud Foundry [Java buildpack](https://github.com/cloudfoundry/java-buildpack) (<https://github.com/cloudfoundry/java-buildpack>) has excellent support for Spring applications, including Spring Boot. You can deploy stand-alone executable jar applications as well as traditional `.war` packaged applications.

Once you have built your application (by using, for example, `mvn clean package`) and have [installed the cf command line tool](https://docs.cloudfoundry.org/cf-cli/install-go-cli.html) (<https://docs.cloudfoundry.org/cf-cli/install-go-cli.html>), deploy your application by using the `cf push` command, substituting the path to your compiled `.jar`. Be sure to have [logged in with your cf command line client](https://docs.cloudfoundry.org/cf-cli/getting-started.html#login) (<https://docs.cloudfoundry.org/cf-cli/getting-started.html#login>) before pushing an application. The following line shows using the `cf push` command to deploy an application:

```
$ cf push acloudyspringtime -p target/demo-0.0.1-SNAPSHOT.jar
```

SHELL



In the preceding example, we substitute `acloudyspringtime` for whatever value you give `cf` as the name of your application.

See the [cf push documentation](https://docs.cloudfoundry.org/cf-cli/getting-started.html#push) (<https://docs.cloudfoundry.org/cf-cli/getting-started.html#push>) for more options. If there is a Cloud Foundry [manifest.yml](https://docs.cloudfoundry.org/devguide/deploy-apps/manifest.html) (<https://docs.cloudfoundry.org/devguide/deploy-apps/manifest.html>) file present in the same directory, it is considered.

At this point, `cf` starts uploading your application, producing output similar to the following example:

```
Uploading acloudyspringtime... OK
Preparing to start acloudyspringtime... OK
-----> Downloaded app package (8.9M)
-----&; Java Buildpack Version: v3.12 (offline) | https://github.com/cloudfoundry/java-buildpack.git#6f25b7e
-----> Downloading Open Jdk JRE
      Expanding Open Jdk JRE to .java-buildpack/open_jdk_jre (1.6s)
-----> Downloading Open JDK Like Memory Calculator 2.0.2_RELEASE from https://java-buildpack.cloudfoundry.org/memory-calculator/trusty/x86_64/memory-calculator-2.0.2_RELEASE.tar.gz (found in cache)
      Memory Settings: -Xss349K -Xmx681574K -XX:MaxMetaspaceSize=104857K -Xms681574K -XX:MetaspaceSize=104857K
-----> Downloading Container Certificate Trust Store 1.0.0_RELEASE from https://java-buildpack.cloudfoundry.org/container-certificate-trust-store/container-certificate-trust-store-1.0.0_RELEASE.jar (found in cache)
      Adding certificates to .java-buildpack/container_certificate_trust_store/truststore.jks (0.6s)
-----> Downloading Spring Auto Reconfiguration 1.10.0_RELEASE from https://java-buildpack.cloudfoundry.org/auto-reconfiguration/auto-reconfiguration-1.10.0_RELEASE.jar (found in cache)
Checking status of app 'acloudyspringtime'...
  0 of 1 instances running (1 starting)
  ...
  0 of 1 instances running (1 starting)
  ...
  0 of 1 instances running (1 starting)
  ...
  1 of 1 instances running (1 running)

App started
```

Congratulations! The application is now live!

Once your application is live, you can verify the status of the deployed application by using the `cf apps` command, as shown in the following example:

```
$ cf apps
Getting applications in ...
OK

name                requested state  instances  memory  disk  urls
...
acloudyspringtime   started          1/1        512M    1G    acloudyspringtime.cfapps.io
...
```

SHELL

Once Cloud Foundry acknowledges that your application has been deployed, you should be able to find the application at the URI given. In the preceding example, you could find it at <https://acloudyspringtime.cfapps.io/>.

103.1.1. Binding to Services

By default, metadata about the running application as well as service connection information is exposed to the application as environment variables (for example: `$VCAP_SERVICES`). This architecture decision is due to Cloud Foundry's polyglot (any language and platform can be supported as a buildpack) nature. Process-scoped environment variables are language agnostic.

Environment variables do not always make for the easiest API, so Spring Boot automatically extracts them and flattens the data into properties that can be accessed through Spring's `javax.org.springframework.core.env.Environment[]` abstraction, as shown in the following example:

```
include-code::MyBean[]
```

All Cloud Foundry properties are prefixed with `vcap`. You can use `vcap` properties to access application information (such as the public URL of the application) and service information (such as database credentials). See the `javax.org.springframework.boot.cloud.CloudFoundryVcapEnvironmentPostProcessor[]` API documentation for complete details.



The [Java CFEnv](https://github.com/pivotal-cf/java-cfenv) (<https://github.com/pivotal-cf/java-cfenv>) project is a better fit for tasks such as configuring a `DataSource`.

103.2. Kubernetes

Spring Boot auto-detects Kubernetes deployment environments by checking the environment for `"*_SERVICE_HOST"` and `"*_SERVICE_PORT"` variables. You can override this detection with the `configprop:spring.main.cloud-platform[]` configuration property.

Spring Boot helps you to [manage the state of your application](#) and export it with [HTTP Kubernetes Probes using Actuator](#).

103.2.1. Kubernetes Container Lifecycle

When Kubernetes deletes an application instance, the shutdown process involves several subsystems concurrently: shutdown hooks, unregistering the service, removing the instance from the load-balancer...Because this shutdown processing happens in parallel (and due to the nature of distributed systems), there is a window during which traffic can be routed to a pod that has also begun its shutdown processing.

You can configure a sleep execution in a `preStop` handler to avoid requests being routed to a pod that has already begun shutting down. This sleep should be long enough for new requests to stop being routed to the pod and its duration will vary from deployment to deployment. The delay should be at least as long as the longest time it takes to process an in-flight request. You should not rely on the Spring Boot graceful shutdown period alone, as the platform will not be getting any liveness data in the period that the app is shutting down.

If you're using Kubernetes 1.32 or up, the `preStop` handler can be configured by using the `PodSpec` in the pod's configuration file as follows:

```
spec:
  containers:
  - name: "example-container"
    image: "example-image"
    lifecycle:
      preStop:
        sleep:
          seconds: 10
```

YAML

If you're not on Kubernetes 1.32 yet, you can use an `exec` command to invoke `sleep`.

```
spec:
  containers:
  - name: "example-container"
    image: "example-image"
    lifecycle:
      preStop:
        exec:
          command: ["sh", "-c", "sleep 10"]
```

YAML



The container needs to have a shell for this to work.

Once the pre-stop hook has completed, `SIGTERM` will be sent to the container and [graceful shutdown](#) will begin, allowing any remaining in-flight requests to complete.



When Kubernetes sends a `SIGTERM` signal to the pod, it waits for a specified time called the termination grace period (the default for which is 30 seconds). If the containers are still running after the grace period, they are sent the `SIGKILL` signal and forcibly removed. If the pod takes longer than 30 seconds to shut down, which could be because you have increased `configprop:spring.lifecycle.timeout-per-shutdown-phase[]`, make sure to increase the termination grace period by setting the `terminationGracePeriodSeconds` option in the Pod YAML.

103.3. Heroku

Heroku is another popular PaaS platform. To customize Heroku builds, you provide a `Procfile`, which provides the incantation required to deploy an application. Heroku assigns a `port` for the Java application to use and then ensures that routing to the external URI works.

You must configure your application to listen on the correct port. The following example shows the `Procfile` for our starter REST application:

```
web: java -Dserver.port=$PORT -jar target/demo-0.0.1-SNAPSHOT.jar
```

Spring Boot makes `-D` arguments available as properties accessible from a Spring `javax.org.springframework.core.env.Environment[]` instance. The `server.port` configuration property is fed to the embedded Tomcat or Jetty instance, which then uses the port when it starts up. The `$PORT` environment variable is assigned to us by the Heroku PaaS.

This should be everything you need. The most common deployment workflow for Heroku deployments is to `git push` the code to production, as shown in the following example:

```
$ git push heroku main
```

SHELL

Which will result in the following:

```
Initializing repository, done.
Counting objects: 95, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (78/78), done.
Writing objects: 100% (95/95), 8.66 MiB | 606.00 KiB/s, done.
Total 95 (delta 31), reused 0 (delta 0)

-----> Java app detected
-----> Installing OpenJDK... done
-----> Installing Maven... done
-----> Installing settings.xml... done
-----> Executing: mvn -B -DskipTests=true clean install

[INFO] Scanning for projects...
Downloading: https://repo.spring.io/...
Downloaded: https://repo.spring.io/... (818 B at 1.8 KB/sec)
....
Downloaded: https://s3pository.heroku.com/jvm/... (152 KB at 595.3 KB/sec)
[INFO] Installing /tmp/build_0c35a5d2-a067-4abc-a232-14b1fb7a8229/target/...
[INFO] Installing /tmp/build_0c35a5d2-a067-4abc-a232-14b1fb7a8229/pom.xml ...
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 59.358s
[INFO] Finished at: Fri Mar 07 07:28:25 UTC 2014
[INFO] Final Memory: 20M/493M
[INFO] -----

-----> Discovering process types
Procfile declares types -> web

-----> Compressing... done, 70.4MB
-----> Launching... done, v6
https://agile-sierra-1405.herokuapp.com/ deployed to Heroku

To git@heroku.com:agile-sierra-1405.git
* [new branch] main -> main
```

Your application should now be up and running on Heroku. For more details, see [Deploying Spring Boot Applications to Heroku](https://devcenter.heroku.com/articles/deploying-spring-boot-apps-to-heroku) (<https://devcenter.heroku.com/articles/deploying-spring-boot-apps-to-heroku>).

103.4. OpenShift

[OpenShift](https://www.openshift.com/) (<https://www.openshift.com/>) has many resources describing how to deploy Spring Boot applications, including:

- [Using the S2I builder](https://blog.openshift.com/using-openshift-enterprise-grade-spring-boot-deployments/) (<https://blog.openshift.com/using-openshift-enterprise-grade-spring-boot-deployments/>)
- [Running as a traditional web application on Wildfly](https://blog.openshift.com/using-spring-boot-on-openshift/) (<https://blog.openshift.com/using-spring-boot-on-openshift/>)
- [OpenShift Commons Briefing](https://blog.openshift.com/openshift-commons-briefing-96-cloud-native-applications-spring-rhoar/) (<https://blog.openshift.com/openshift-commons-briefing-96-cloud-native-applications-spring-rhoar/>)

103.5. Amazon Web Services (AWS)

Amazon Web Services provides several options that are suitable for running Spring Boot-based applications, either as containers, traditional web applications (war), or self-contained executable jar files. Popular options are:

- Amazon Elastic Container Service (ECS)
- AWS Elastic Beanstalk

103.5.1. Amazon Elastic Container Service (ECS)

Official [Amazon ECS developer guide](https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html) (<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html>) provides comprehensive overview of platform's features and includes [getting started guide](https://docs.aws.amazon.com/AmazonECS/latest/developerguide/getting-started.html) (<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/getting-started.html>) that walks you through the steps needed to get your containers up and running.



Spring Boot applications can be packaged in Docker containers using techniques described in [reference:packaging/container-images/index.html](https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html).

In addition to the developer guide, AWS also provides a [topical guide](https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html) (<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html>) for deploying containerized Java services on Amazon ECS using AWS Fargate.



Spring Boot auto-detects AWS ECS deployment environments by checking the environment for the `AWS_EXECUTION_ENV` variable. You can override this detection with the `configprop:spring.main.cloud-platform[]` configuration property.

103.5.2. AWS Elastic Beanstalk

As described in the official [Elastic Beanstalk Java guide](https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/create_deploy_Java.html) (https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/create_deploy_Java.html), there are two main options to deploy a Java application. You can either use the “Tomcat Platform” or the “Java SE platform”.

Using the Tomcat Platform

This option applies to Spring Boot projects that produce a war file. Follow the official guide and [Java on Tomcat tutorial](https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/tomcat-quickstart.html) (<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/tomcat-quickstart.html>).



Creating deployable war for a Spring Boot application is described in [deployment/traditional-deployment.html](https://cloudcaptain.sh/docs/commandline/run.html).

Using the Java SE Platform

This option applies to Spring Boot projects that produce a jar file and run an embedded web container. Follow the official guide and [Java tutorial](https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/java-quickstart.html) (<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/java-quickstart.html>). There are also community provided tutorials such as [this one](https://www.baeldung.com/spring-boot-deploy-aws-beanstalk) (<https://www.baeldung.com/spring-boot-deploy-aws-beanstalk>).

103.6. CloudCaptain and Amazon Web Services

[CloudCaptain](https://cloudcaptain.sh/) (<https://cloudcaptain.sh/>) works by turning your Spring Boot executable jar or war into a minimal VM image that can be deployed unchanged either on VirtualBox or on AWS. CloudCaptain comes with deep integration for Spring Boot and uses the information from your Spring Boot configuration file to automatically configure ports and health check URLs. CloudCaptain leverages this information both for the images it produces as well as for all the resources it provisions (instances, security groups, elastic load balancers, and so on).

Once you have created a [CloudCaptain account](https://console.cloudcaptain.sh/) (<https://console.cloudcaptain.sh/>), connected it to your AWS account, installed the latest version of the CloudCaptain Client, and ensured that the application has been built by Maven or Gradle (by using, for example, `mvn clean package`), you can deploy your Spring Boot application to AWS with a command similar to the following:

```
$ boxfuse run myapp-1.0.jar -env=prod
```

SHELL

See the [boxfuse run documentation](https://cloudcaptain.sh/docs/commandline/run.html) (<https://cloudcaptain.sh/docs/commandline/run.html>) for more options. If there is a [boxfuse.conf](https://cloudcaptain.sh/docs/commandline/#configuration) (<https://cloudcaptain.sh/docs/commandline/#configuration>) file present in the current directory, it is considered.



By default, CloudCaptain activates a Spring profile named `boxfuse` on startup. If your executable jar or war contains an `application-boxfuse.properties` (<https://cloudcaptain.sh/docs/payloads/springboot.html#configuration>) file, CloudCaptain bases its configuration on the properties it contains.

At this point, CloudCaptain creates an image for your application, uploads it, and configures and starts the necessary resources on AWS, resulting in output similar to the following example:

```
Fusing Image for myapp-1.0.jar ...
Image fused in 00:06.838s (53937 K) -> axelfontaine/myapp:1.0
Creating axelfontaine/myapp ...
Pushing axelfontaine/myapp:1.0 ...
Verifying axelfontaine/myapp:1.0 ...
Creating Elastic IP ...
Mapping myapp-axelfontaine.boxfuse.io to 52.28.233.167 ...
Waiting for AWS to create an AMI for axelfontaine/myapp:1.0 in eu-central-1 (this may take up to 50 seconds) ...
AMI created in 00:23.557s -> ami-d23f38cf
Creating security group boxfuse-sg_axelfontaine/myapp:1.0 ...
Launching t2.micro instance of axelfontaine/myapp:1.0 (ami-d23f38cf) in eu-central-1 ...
Instance launched in 00:30.306s -> i-92ef9f53
Waiting for AWS to boot Instance i-92ef9f53 and Payload to start at https://52.28.235.61/ ...
Payload started in 00:29.266s -> https://52.28.235.61/
Remapping Elastic IP 52.28.233.167 to i-92ef9f53 ...
Waiting 15s for AWS to complete Elastic IP Zero Downtime transition ...
Deployment completed successfully. axelfontaine/myapp:1.0 is up and running at https://myapp-axelfontaine.boxfuse.io/
```

Your application should now be up and running on AWS.

See the blog post on [deploying Spring Boot apps on EC2](https://cloudcaptain.sh/blog/spring-boot-ec2.html) (<https://cloudcaptain.sh/blog/spring-boot-ec2.html>) as well as the [documentation for the CloudCaptain Spring Boot integration](https://cloudcaptain.sh/docs/payloads/springboot.html) (<https://cloudcaptain.sh/docs/payloads/springboot.html>) to get started with a Maven build to run the app.

103.7. Azure

This [Getting Started guide](https://spring.io/guides/gs/spring-boot-for-azure/) (<https://spring.io/guides/gs/spring-boot-for-azure/>) walks you through deploying your Spring Boot application to either [Azure Spring Cloud](https://azure.microsoft.com/en-us/services/spring-cloud/) (<https://azure.microsoft.com/en-us/services/spring-cloud/>) or [Azure App Service](https://docs.microsoft.com/en-us/azure/app-service/overview) (<https://docs.microsoft.com/en-us/azure/app-service/overview>).

103.8. Google Cloud

Google Cloud has several options that can be used to launch Spring Boot applications. The easiest to get started with is probably App Engine, but you could also find ways to run Spring Boot in a container with Container Engine or on a virtual machine with Compute Engine.

To deploy your first app to App Engine standard environment, follow [this tutorial](https://codelabs.developers.google.com/codelabs/cloud-app-engine-springboot#0) (<https://codelabs.developers.google.com/codelabs/cloud-app-engine-springboot#0>).

Alternatively, App Engine Flex requires you to create an `app.yaml` file to describe the resources your app requires. Normally, you put this file in `src/main/appengine`, and it should resemble the following file:


```

service: "default"

runtime: "java17"
env: "flex"

handlers:
- url: "/*"
  script: "this field is required, but ignored"

manual_scaling:
  instances: 1

health_check:
  enable_health_check: false

env_variables:
  ENCRYPT_KEY: "your_encryption_key_here"

```

You can deploy the app (for example, with a Maven plugin) by adding the project ID to the build configuration, as shown in the following example:

```

<plugin>
  <groupId>com.google.cloud.tools</groupId>
  <artifactId>appengine-maven-plugin</artifactId>
  <version>2.4.4</version>
  <configuration>
    <project>myproject</project>
  </configuration>
</plugin>

```

Then deploy with `mvn appengine:deploy` (you need to authenticate first, otherwise the build fails).

104. Deploying Spring Boot Applications

Spring Boot's flexible packaging options provide a great deal of choice when it comes to deploying your application. You can deploy Spring Boot applications to a variety of cloud platforms and to virtual or real machines.

This section covers some of the more common deployment scenarios.

105. Installing Spring Boot Applications

In addition to running Spring Boot applications by using `java -jar` directly, it is also possible to run them as services.

105.1. Installation as a systemd Service

`systemd` is the successor of the System V init system and is now being used by many modern Linux distributions. Spring Boot applications can be launched by using `systemd` 'service' scripts.

Assuming that you have a Spring Boot application packaged as an uber jar in `/var/myapp`, to install it as a `systemd` service, create a script named `myapp.service` and place it in `/etc/systemd/system` directory. The following script offers an example:

```
[Unit]
Description=myapp
After=syslog.target network.target

[Service]
User=myapp
Group=myapp

Type=exec
ExecStart=/path/to/java/home/bin/java -jar /var/myapp/myapp.jar
WorkingDirectory=/var/myapp
SuccessExitStatus=143

[Install]
WantedBy=multi-user.target
```



Remember to change the `Description`, `User`, `Group`, `ExecStart` and `WorkingDirectory` fields for your application.



The `ExecStart` field does not declare the script action command, which means that the `run` command is used by default.

The user that runs the application, the PID file, and the console log file are managed by `systemd` itself and therefore must be configured by using appropriate fields in the 'service' script. Consult the [service unit configuration man page](https://www.freedesktop.org/software/systemd/man/systemd.service.html) (<https://www.freedesktop.org/software/systemd/man/systemd.service.html>) for more details.

To flag the application to start automatically on system boot, use the following command:

```
$ systemctl enable myapp.service
```

SHELL

Run `man systemctl` for more details.

105.2. Microsoft Windows Services

A Spring Boot application can be started as a Windows service by using [winsw](https://github.com/kohsuke/winsw) (<https://github.com/kohsuke/winsw>).

A [separately maintained sample](https://github.com/snicoll/spring-boot-daemon) (<https://github.com/snicoll/spring-boot-daemon>) describes step-by-step how you can create a Windows service for your Spring Boot application.

106. Traditional Deployment

Spring Boot supports traditional deployment as well as more modern forms of deployment. This section answers common questions about traditional deployment.

106.1. Create a Deployable War File



Because Spring WebFlux does not strictly depend on the servlet API and applications are deployed by default on an embedded Reactor Netty server, War deployment is not supported for WebFlux applications.

The first step in producing a deployable war file is to provide a `javadoc:org.springframework.boot.web.servlet.support.SpringBootServletInitializer[]` subclass and override its `configure` method. Doing so makes use of Spring Framework's servlet 3.0 support and lets you configure your application when it is launched by the servlet container. Typically, you should update your application's main class to extend `javadoc:org.springframework.boot.web.servlet.support.SpringBootServletInitializer[]`, as shown in the following example:

```
include-code::MyApplication[]
```

The next step is to update your build configuration such that your project produces a war file rather than a jar file. If you use Maven and `spring-boot-starter-parent` (which configures Maven's war plugin for you), all you need to do is to modify `pom.xml` to change the packaging to war, as follows:

```
<packaging>war</packaging>
```

XML

If you use Gradle, you need to modify `build.gradle` to apply the war plugin to the project, as follows:

```
apply plugin: 'war'
```

GRADLE

The final step in the process is to ensure that the embedded servlet container does not interfere with the servlet container to which the war file is deployed.

For Maven, you need to mark the embedded servlet container dependency as being `provided`. For example:

```
<dependencies>
  <!-- ... -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
  </dependency>
  <!-- ... -->
</dependencies>
```

XML

If you use Gradle, you need to move only the runtime dependencies into the `providedRuntime` configuration. For example:

```
dependencies {
    // ...
    providedRuntime 'org.springframework.boot:spring-boot-starter-tomcat-runtime'
    // ...
}
```

GRADLE



`providedRuntime` is preferred to Gradle's `compileOnly` configuration. Among other limitations, `compileOnly` dependencies are not on the test classpath, so any web-based integration tests fail.

If you use the Spring Boot [build-tool-plugin:index.html](#), marking the embedded servlet container dependency as provided produces an executable war file with the provided dependencies packaged in a `lib-provided` directory. This means that, in addition to being deployable to a servlet container, you can also run your application by using `java -jar` on the command line.

106.2. Convert an Existing Application to Spring Boot

To convert an existing non-web Spring application to a Spring Boot application, replace the code that creates your `javadoc:org.springframework.context.ApplicationContext[]` and replace it with calls to `javadoc:org.springframework.boot.SpringApplication[]` or `javadoc:org.springframework.boot.builder.SpringApplicationBuilder[]`. Spring MVC web applications are generally amenable to first creating a deployable war application and then migrating it later to an executable war or jar.

To create a deployable war by extending `javadoc:org.springframework.boot.web.servlet.support.SpringBootServletInitializer[]` (for example, in a class called `Application`) and adding the Spring Boot `javadoc:org.springframework.boot.autoconfigure.SpringBootApplication[format=annotation]` annotation, use code similar to that shown in the following example:

```
include-code::MyApplication[tag=!main]
```

Remember that, whatever you put in the `sources` is merely a Spring `javadoc:org.springframework.context.ApplicationContext[]`. Normally, anything that already works should work here. There might be some beans you can remove later and let Spring Boot provide its own defaults for them, but it should be possible to get something working before you need to do that.

Static resources can be moved to `/public` (or `/static` or `/resources` or `/META-INF/resources`) in the classpath root. The same applies to `messages.properties` (which Spring Boot automatically detects in the root of the classpath).

Vanilla usage of Spring `javax.servlet.DispatcherServlet[]` and Spring Security should require no further changes. If you have other features in your application (for instance, using other servlets or filters), you may need to add some configuration to your `Application` context, by replacing those elements from the `web.xml`, as follows:

- A `javax.servlet.DispatcherServlet[]` of type `javax.servlet.Servlet[]` or `javax.servlet.FilterRegistrationBean[]` installs that bean in the container as if it were a `<servlet/>` and `<servlet-mapping/>` in `web.xml`.
- A `javax.servlet.FilterRegistrationBean[]` of type `javax.servlet.Filter[]` or `javax.servlet.FilterRegistrationBean[]` behaves similarly (as a `<filter/>` and `<filter-mapping/>`).
- An `javax.servlet.ApplicationContext[]` in an XML file can be added through an `javax.servlet.ImportResource[format=annotation]` in your `Application`. Alternatively, cases where annotation configuration is heavily used already can be recreated in a few lines as `javax.servlet.Bean[format=annotation]` definitions.

Once the war file is working, you can make it executable by adding a `main` method to your `Application`, as shown in the following example:

```
include-code::MyApplication[tag=main]
```



If you intend to start your application as a war or as an executable application, you need to share the customizations of the builder in a method that is both available to the `javax.servlet.support.SpringBootServletInitializer[]` callback and in the `main` method in a class similar to the following:

```
include-code::both/MyApplication[]
```

Applications can fall into more than one category:

- Servlet 3.0+ applications with no `web.xml`.
- Applications with a `web.xml`.
- Applications with a context hierarchy.
- Applications without a context hierarchy.

All of these should be amenable to translation, but each might require slightly different techniques.

Servlet 3.0+ applications might translate pretty easily if they already use the Spring Servlet 3.0+ initializer support classes. Normally, all the code from an existing `javax.servlet.WebApplicationInitializer[]` can be moved into a `javax.servlet.support.SpringBootServletInitializer[]`. If your existing application has more than one `javax.servlet.ApplicationContext[]` (for example, if it uses `javax.servlet.support.AbstractDispatcherServletInitializer[]`) then you might be able to combine all your context sources into a single `javax.servlet.SpringApplication[]`. The main complication you might encounter is if combining does not work and you need to maintain the context hierarchy. See the [entry on building a hierarchy](#) for examples. An existing parent context that contains web-specific features usually needs to be broken up so that all the `javax.servlet.context.ServletContextAware[]` components are in the child context.

Applications that are not already Spring applications might be convertible to Spring Boot applications, and the previously mentioned guidance may help. However, you may yet encounter problems. In that case, we suggest [asking questions on Stack Overflow with a tag of `spring-boot`](https://stackoverflow.com/questions/tagged/spring-boot) (https://stackoverflow.com/questions/tagged/spring-boot).

106.3. Deploying a WAR to WebLogic

To deploy a Spring Boot application to WebLogic, you must ensure that your servlet initializer **directly** implements `javax.servlet.WebApplicationInitializer[]` (even if you extend from a base class that already implements it).

A typical initializer for WebLogic should resemble the following example:

```
include-code::MyApplication[]
```

If you use Logback, you also need to tell WebLogic to prefer the packaged version rather than the version that was pre-installed with the server. You can do so by adding a `WEB-INF/weblogic.xml` file with the following contents:

```
<?xml version="1.0" encoding="UTF-8"?>
<wls:weblogic-web-app
  xmlns:wls="http://xmlns.oracle.com/weblogic/weblogic-web-app"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/ejb-jar_3_0.xsd
    http://xmlns.oracle.com/weblogic/weblogic-web-app
    http://xmlns.oracle.com/weblogic/weblogic-web-app/1.4/weblogic-web-app.xsd">
  <wls:container-descriptor>
    <wls:prefer-application-packages>
      <wls:package-name>org.slf4j</wls:package-name>
    </wls:prefer-application-packages>
  </wls:container-descriptor>
</wls:weblogic-web-app>
```

XML

107. Docker Compose

This section includes topics relating to the Docker Compose support in Spring Boot.

107.1. Customizing the JDBC URL

When using `javado:org.springframework.boot.jdbc.autoconfigure.JdbcConnectionDetails[]` with Docker Compose, the parameters of the JDBC URL can be customized by applying the `org.springframework.boot.jdbc.parameters` label to the service. For example:

```
services:
  postgres:
    image: 'postgres:15.3'
    environment:
      - 'POSTGRES_USER=myuser'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_DB=mydb'
    ports:
      - '5432:5432'
    labels:
      org.springframework.boot.jdbc.parameters: 'ssl=true&sslmode=require'
```

YAML

With this Docker Compose file in place, the JDBC URL used is `jdbc:postgresql://127.0.0.1:5432/mydb?ssl=true&sslmode=require`.

107.2. Sharing Services Between Multiple Applications

If you want to share services between multiple applications, create the `compose.yaml` file in one of the applications and then use the configuration property `configprop:spring.docker.compose.file[]` in the other applications to reference the `compose.yaml` file. You should also set `configprop:spring.docker.compose.lifecycle-management[]` to `start-only`, as it defaults to `start-and-stop` and stopping one application would shut down the shared services for the other still running applications as well. Setting it to `start-only` won't stop the shared services on application stop, but a caveat is that if you shut down all applications, the services remain running. You can stop the services manually by running `docker compose stop` on the command line in the directory which contains the `compose.yaml` file.

108. Hot Swapping

Spring Boot supports hot swapping. This section answers questions about how it works.

108.1. Reload Static Content

There are several options for hot reloading. The recommended approach is to use `spring-boot-devtools`, as it provides additional development-time features, such as support for fast application restarts and LiveReload as well as sensible development-time configuration (such as template caching). Devtools works by monitoring the classpath for changes. This means that static resource changes must be "built" for the change to take effect. By default, this happens automatically in Eclipse when you save your changes. In IntelliJ IDEA, the Make Project command triggers the necessary build. Due to the [default restart exclusions](#), changes to static resources do not trigger a restart of your application. They do, however, trigger a live reload.

Alternatively, running in an IDE (especially with debugging on) is a good way to do development (all modern IDEs allow reloading of static resources and usually also allow hot-swapping of Java class changes).

Finally, the [Maven and Gradle plugins](#) can be configured (see the `addResources` property) to support running from the command line with reloading of static files directly from source. You can use that with an external css/js compiler process if you are writing that code with higher-level tools.

108.2. Reload Templates without Restarting the Container

Most of the templating technologies supported by Spring Boot include a configuration option to disable caching (described later in this document). If you use the `spring-boot-devtools` module, these properties are [automatically configured](#) for you at development time.

108.2.1. Thymeleaf Templates

If you use Thymeleaf, set `spring.thymeleaf.cache` to `false`. See `{code-spring-boot-thymeleaf-src}/autoconfigure/ThymeleafAutoConfiguration.java` [`ThymeleafAutoConfiguration`] for other Thymeleaf customization options.

108.2.2. FreeMarker Templates

If you use FreeMarker, set `spring.freemarker.cache` to `false`. See `{code-spring-boot-freemarker-src}/autoconfigure/FreeMarkerAutoConfiguration.java` [`FreeMarkerAutoConfiguration`] for other FreeMarker customization options.



Template caching for FreeMarker is not supported with WebFlux.

108.2.3. Groovy Templates

If you use Groovy templates, set `spring.groovy.template.cache` to `false`. See `{code-spring-boot-groovy-templates-src}/autoconfigure/GroovyTemplateAutoConfiguration.java` [`GroovyTemplateAutoConfiguration`] for other Groovy customization options.

108.3. Fast Application Restarts

The `spring-boot-devtools` module includes support for automatic application restarts. While not as fast as technologies such as [JRebel](https://www.jrebel.com/products/jrebel) (https://www.jrebel.com/products/jrebel) it is usually significantly faster than a "cold start". You should probably give it a try before investigating some of the more complex reload options discussed later in this document.

For more details, see the [reference:using/devtools.html](#) section.

108.4. Reload Java Classes without Restarting the Container

Many modern IDEs (Eclipse, IntelliJ IDEA, and others) support hot swapping of bytecode. Consequently, if you make a change that does not affect class or method signatures, it should reload cleanly with no side effects.

109. HTTP Clients

Spring Boot offers a number of starters that work with HTTP clients. This section answers questions related to using them.

109.1. Configure RestTemplate to Use a Proxy

As described in [RestTemplate Customization](#), you can use a `javado:org.springframework.boot.restclient.RestTemplateCustomizer[]` with `javado:org.springframework.boot.restclient.RestTemplateBuilder[]` to build a customized `javado:org.springframework.web.client.RestTemplate[]`. This is the recommended approach for creating a `javado:org.springframework.web.client.RestTemplate[]` configured to use a proxy.

The exact details of the proxy configuration depend on the underlying client request factory that is being used.

109.2. Configure the TcpClient used by a Reactor Netty-based WebClient

When Reactor Netty is on the classpath a Reactor Netty-based `javado:org.springframework.web.reactive.function.client.WebClient[]` is auto-configured. To customize the client's handling of network connections, provide a `javado:org.springframework.http.client.reactive.ClientHttpConnector[]` bean. The following example configures a 60 second connect timeout and adds a `javado:io.netty.handler.timeout.ReadTimeoutHandler[]`:

include-code::MyReactorNettyClientConfiguration[]



Note the use of `javado:org.springframework.http.client.ReactorResourceFactory[]` for the connection provider and event loop resources. This ensures efficient sharing of resources for the server receiving requests and the client making requests.

110. How-to Guides

This section provides answers to some common ‘how do I do that...’ questions that often arise when using Spring Boot. Its coverage is not exhaustive, but it does cover quite a lot.

If you have a specific problem that we do not cover here, you might want to check stackoverflow.com (<https://stackoverflow.com/tags/spring-boot>) to see if someone has already provided an answer. This is also a great place to ask new questions (please use the `spring-boot` tag).

We are also more than happy to extend this section. If you want to add a ‘how-to’, send us a [pull request](#).

111. Jersey

111.1. Secure Jersey Endpoints with Spring Security

Spring Security can be used to secure a Jersey-based web application in much the same way as it can be used to secure a Spring MVC-based web application. However, if you want to use Spring Security's method-level security with Jersey, you must configure Jersey to use `setStatus(int)` rather than `sendError(int)`. This prevents Jersey from committing the response before Spring Security has had an opportunity to report an authentication or authorization failure to the client.

The `jersey.config.server.response.setStatusOverSendError` property must be set to `true` on the application's `javax:org.glassfish.jersey.server.ResourceConfig[]` bean, as shown in the following example:

```
include-code::JerseySetStatusOverSendErrorConfig[]
```

111.2. Use Jersey Alongside Another Web Framework

To use Jersey alongside another web framework, such as Spring MVC, it should be configured so that it will allow the other framework to handle requests that it cannot handle. First, configure Jersey to use a filter rather than a servlet by configuring the `configprop:spring.jersey.type[]` application property with a value of `filter`. Second, configure your `javax:org.glassfish.jersey.server.ResourceConfig[]` to forward requests that would have resulted in a 404, as shown in the following example.

```
include-code::JerseyConfig[]
```

112. Logging

Spring Boot has no mandatory logging dependency, except for the Commons Logging API, which is typically provided by Spring Framework's `spring-jcl` module. To use [Logback](https://logback.qos.ch) (<https://logback.qos.ch>), you need to include it and `spring-jcl` on the classpath. The recommended way to do that is through the starters, which all depend on `spring-boot-starter-logging`. For a web application, you only need `spring-boot-starter-web`, since it depends transitively on the logging starter. If you use Maven, the following dependency adds logging for you:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

XML

Spring Boot has a `javadoc:org.springframework.boot.logging.LoggingSystem[]` abstraction that attempts to configure logging based on the content of the classpath. If Logback is available, it is the first choice.

If the only change you need to make to logging is to set the levels of various loggers, you can do so in `application.properties` by using the "logging.level" prefix, as shown in the following example:

```
logging:
  level:
    org.springframework.web: "debug"
    org.hibernate: "error"
```

You can also set the location of a file to which the log will be written (in addition to the console) by using `logging.file.name`.

To configure the more fine-grained settings of a logging system, you need to use the native configuration format supported by the `javadoc:org.springframework.boot.logging.LoggingSystem[]` in question. By default, Spring Boot picks up the native configuration from its default location for the system (such as `classpath:logback.xml` for Logback), but you can set the location of the config file by using the `configprop:logging.config[]` property.

112.1. Configure Logback for Logging

If you need to apply customizations to logback beyond those that can be achieved with `application.properties`, you will need to add a standard logback configuration file. You can add a `logback.xml` file to the root of your classpath for logback to find. You can also use `logback-spring.xml` if you want to use the Spring Boot <reference:features/logging.html>.



The Logback documentation has a [dedicated section that covers configuration](https://logback.qos.ch/manual/configuration.html) (<https://logback.qos.ch/manual/configuration.html>) in some detail.

Spring Boot provides a number of logback configurations that can be included in your own configuration. These includes are designed to allow certain common Spring Boot conventions to be re-applied.

The following files are provided under `org/springframework/boot/logging/logback/`:

- `defaults.xml` - Provides conversion rules, pattern properties and common logger configurations.
- `console-appender.xml` - Adds a `javadoc:ch.qos.logback.core.ConsoleAppender[]` using the `CONSOLE_LOG_PATTERN`.
- `structured-console-appender.xml` - Adds a `javadoc:ch.qos.logback.core.ConsoleAppender[]` using structured logging in the `CONSOLE_LOG_STRUCTURED_FORMAT`.
- `file-appender.xml` - Adds a `javadoc:ch.qos.logback.core.rolling.RollingFileAppender[]` using the `FILE_LOG_PATTERN` and `ROLLING_FILE_NAME_PATTERN` with appropriate settings.
- `structured-file-appender.xml` - Adds a `javadoc:ch.qos.logback.core.rolling.RollingFileAppender[]` using the `ROLLING_FILE_NAME_PATTERN` with structured logging in the `FILE_LOG_STRUCTURED_FORMAT`.

In addition, a legacy `base.xml` file is provided for compatibility with earlier versions of Spring Boot.

A typical custom `logback.xml` file would look something like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <include resource="org/springframework/boot/logging/logback/defaults.xml"/>
  <include resource="org/springframework/boot/logging/logback/console-appender.xml" />
  <root level="INFO">
    <appender-ref ref="CONSOLE" />
  </root>
  <logger name="org.springframework.web" level="DEBUG"/>
</configuration>
```

XML

Your logback configuration file can also make use of System properties that the `javadoc:org.springframework.boot.logging.LoggingSystem[]` takes care of creating for you:

- `${PID}`: The current process ID.
- `${LOG_FILE}`: Whether `logging.file.name` was set in Boot's external configuration.
- `${LOG_PATH}`: Whether `logging.file.path` (representing a directory for log files to live in) was set in Boot's external configuration.
- `${LOG_EXCEPTION_CONVERSION_WORD}`: Whether `logging.exception-conversion-word` was set in Boot's external configuration.
- `${ROLLING_FILE_NAME_PATTERN}`: Whether `logging.pattern.rolling-file-name` was set in Boot's external configuration.

Spring Boot also provides some nice ANSI color terminal output on a console (but not in a log file) by using a custom Logback converter. See the `CONSOLE_LOG_PATTERN` in the `defaults.xml` configuration for an example.

If Groovy is on the classpath, you should be able to configure Logback with `logback.groovy` as well. If present, this setting is given preference.



Spring extensions are not supported with Groovy configuration. Any `logback-spring.groovy` files will not be detected.

112.1.1. Configure Logback for File-only Output

If you want to disable console logging and write output only to a file, you need a custom `logback-spring.xml` that imports `file-appender.xml` but not `console-appender.xml`, as shown in the following example:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <include resource="org/springframework/boot/logging/logback/defaults.xml" />
  <property name="LOG_FILE" value="${LOG_FILE:-${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/spring.log}" />
  <include resource="org/springframework/boot/logging/logback/file-appender.xml" />
  <root level="INFO">
    <appender-ref ref="FILE" />
  </root>
</configuration>
```

XML

You also need to add `logging.file.name` to your `application.properties` or `application.yaml`, as shown in the following example:

```
logging:
  file:
    name: "myapplication.log"
```

112.2. Configure Log4j for Logging

Spring Boot supports [Log4j 2](https://logging.apache.org/log4j/2.x/) (<https://logging.apache.org/log4j/2.x/>) for logging configuration if it is on the classpath. If you use the starters for assembling dependencies, you have to exclude Logback and then include Log4j 2 instead. If you do not use the starters, you need to provide (at least) `spring-jcl` in addition to Log4j 2.

The recommended path is through the starters, even though it requires some jiggling. The following example shows how to set up the starters in Maven:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-log4j2</artifactId>
</dependency>
```

XML

Gradle provides a few different ways to set up the starters. One way is to use a [url-gradle-docs/resolution_rules.html#sec:module_replacement](https://docs.gradle.org/5.6.2/userguide/module_replacement.html#sec:module_replacement) [module replacement]. To do so, declare a dependency on the Log4j 2 starter and tell Gradle that any occurrences of the default logging starter should be replaced by the Log4j 2 starter, as shown in the following example:

```
dependencies {
  implementation "org.springframework.boot:spring-boot-starter-log4j2"
  modules {
    module("org.springframework.boot:spring-boot-starter-logging") {
      replacedBy("org.springframework.boot:spring-boot-starter-log4j2", "Use Log4j2 instead of Logback")
    }
  }
}
```

GRADLE



The Log4j starters gather together the dependencies for common logging requirements (such as having Tomcat use `java.util.logging` but configuring the output using Log4j 2).



To ensure that debug logging performed using `java.util.logging` is routed into Log4j 2, configure its [JDK logging adapter](https://logging.apache.org/log4j/2.x/log4j-jul.html) (<https://logging.apache.org/log4j/2.x/log4j-jul.html>) by setting the `java.util.logging.manager` system property to `org.apache.logging.log4j.jul.LogManager`.

112.2.1. Use YAML or JSON to Configure Log4j 2

In addition to its default XML configuration format, Log4j 2 also supports YAML and JSON configuration files. To configure Log4j 2 to use an alternative configuration file format, add the appropriate dependencies to the classpath and name your configuration files to match your chosen file format, as shown in the following example:

Format	Dependencies	File names
YAML	<code>com.fasterxml.jackson.core:jackson-databind</code> + <code>com.fasterxml.jackson.dataformat:jackson-dataformat-yaml</code>	<code>log4j2.yaml</code> + <code>log4j2.yml</code>
JSON	<code>com.fasterxml.jackson.core:jackson-databind</code>	<code>log4j2.json</code> + <code>log4j2.jsn</code>

112.2.2. Use Composite Configuration to Configure Log4j 2

Log4j 2 has support for combining multiple configuration files into a single composite configuration. To use this support in Spring Boot, configure `configprop:logging.log4j2.config.override[]` with the locations of one or more secondary configuration files. The secondary configuration files will be merged with the primary configuration, whether the primary's source is Spring Boot's defaults, a standard location such as `log4j.xml`, or the location configured by the `configprop:logging.config[]` property.



Log4j2 override configuration file locations can be prefixed with `optional:`. For example, `optional:classpath:log4j2-override.xml` indicates that `log4j2-override.xml` should only be loaded if the resource exists.

113. Messaging

Spring Boot offers a number of starters to support messaging. This section answers questions that arise from using messaging with Spring Boot.

113.1. Disable Transacted JMS Session

If your JMS broker does not support transacted sessions, you have to disable the support of transactions altogether. If you create your own `javax.jms.MessageListenerContainerFactory`, there is nothing to do, since, by default it cannot be transacted. If you want to use the `org.springframework.boot.jms.autoconfigure.DefaultJmsListenerContainerFactoryConfigurer` to reuse Spring Boot's default, you can disable transacted sessions, as follows:

```
include-code::MyJmsConfiguration[]
```

The preceding example overrides the default factory, and it should be applied to any other factory that your application defines, if any.

114. Developing Your First GraalVM Native Application

There are two main ways to build a Spring Boot native image application:

- Using Spring Boot [support for Cloud Native Buildpacks](#) with the [Paketo Java Native Image buildpack](#) (<https://paketo.io/docs/reference/java-native-image-reference/>) to generate a lightweight container containing a native executable.
- Using GraalVM Native Build Tools to generate a native executable.



The easiest way to start a new native Spring Boot project is to go to start.spring.io (<https://start.spring.io>), add the **GraalVM Native Support** dependency and generate the project. The included **HELP.md** file will provide getting started hints.

114.1. Sample Application

We need an example application that we can use to create our native image. For our purposes, the simple "Hello World!" web application that's covered in the [tutorial: first-application/index.html](#) section will suffice.

To recap, our main application code looks like this:

```
include-code::MyApplication[]
```

This application uses Spring MVC and embedded Tomcat, both of which have been tested and verified to work with GraalVM native images.

114.2. Building a Native Image Using Buildpacks

Spring Boot supports building Docker images containing native executables, using Cloud Native Buildpacks (CNB) integration with both Maven and Gradle and the [Paketo Java Native Image buildpack](#) (<https://paketo.io/docs/reference/java-native-image-reference/>). This means you can just type a single command and quickly get a sensible image into your locally running Docker daemon. The resulting image doesn't contain a JVM, instead the native image is compiled statically. This leads to smaller images.



The CNB builder used for the images is **paketobuildpacks/builder-noble-java-tiny:latest**. It has a small footprint and reduced attack surface. It does not include a shell and contains a reduced set of system libraries. If you need more tools in the resulting image, you can use **paketobuildpacks/ubuntu-noble-run:latest** as the **run** image.



You have to build your application with at least JDK 25, because Buildpacks use the same GraalVM native-image version as the Java version used for compilation.

114.2.1. System Requirements

Docker should be installed. See [Get Docker](#) (<https://docs.docker.com/installation/#installation>) for more details. [Configure it to allow non-root user](#) (<https://docs.docker.com/engine/install/linux-postinstall/#manage-docker-as-a-non-root-user>) if you are on Linux.



You can run **docker run hello-world** (without **sudo**) to check the Docker daemon is reachable as expected. Check the [Maven](#) or [Gradle](#) Spring Boot plugin documentation for more details.



On macOS, it is recommended to increase the memory allocated to Docker to at least **8GB**, and potentially add more CPUs as well. See this [Stack Overflow answer](#) (<https://stackoverflow.com/questions/44533319/how-to-assign-more-memory-to-docker-container/44533437#44533437>) for more details. On Microsoft Windows, make sure to enable the [Docker WSL 2 backend](#) (<https://docs.docker.com/docker-for-windows/wsl/>) for better performance.

114.2.2. Using Maven

To build a native image container using Maven you should ensure that your **pom.xml** file uses the **spring-boot-starter-parent** and the **org.graalvm.buildtools:native-maven-plugin**. You should have a **<parent>** section that looks like this:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>{version-spring-boot}</version>
</parent>
```

XML

You additionally should have this in the **<build>** **<plugins>** section:

```
<plugin>
  <groupId>org.graalvm.buildtools</groupId>
  <artifactId>native-maven-plugin</artifactId>
</plugin>
```

XML

The **spring-boot-starter-parent** declares a **native** profile that configures the executions that need to run in order to create a native image. You can activate profiles using the **-P** flag on the command line.



If you don't want to use **spring-boot-starter-parent** you'll need to configure executions for the **process-aot** goal from Spring Boot's plugin and the **add-reachability-metadata** goal from the Native Build Tools plugin.

To build the image, you can run the **spring-boot:build-image** goal with the **native** profile active:


```
$ mvn -Pnative native:compile
```

SHELL

The native image executable can be found in the `target` directory.

114.3.3. Using Gradle

When the Native Build Tools Gradle plugin is applied to your project, the Spring Boot Gradle plugin will automatically trigger the Spring AOT engine. Task dependencies are automatically configured, so you can just run the standard `nativeCompile` task to generate a native image:

```
$ gradle nativeCompile
```

SHFI 1

The native image executable can be found in the `build/native/nativeCompile` directory.

114.3.4. Running the Example

At this point, your application should work. You can now start the application by running it directly:

Maven

```
$ target/myproject
```

SHELL

Gradle

```
$ build/native/nativeCompile/myproject
```

SHELI

You should see output similar to the following:

```
.
.\ \_/_\_\_\_(\_) \_ \_ \_ \_ \_
( ) \_\_| '\_| | '\_ \_| \ \ \ \
\ \ / \_\_| |\_| | | | | | (\_| | ))))
      |_\_| |_\_| |_\_| |_\_, | / / / /
=====|_|=====|_|_/_/_/_/
:: Spring Boot :: (v{version-spring-boot})
.....
..... . . . (log output here)
.....
..... Started MyApplication in 0.08 seconds (process running for 0.095)
```

SHELL



The startup time differs from machine to machine, but it should be much faster than a Spring Boot application running on a JVM.

If you open a web browser to `localhost:8080`, you should see the following output:

Hello World!

To gracefully exit the application, press `ctrl-c`.

115. GraalVM Native Applications

This section contains details on developing and testing Spring Boot applications as GraalVM native images.



For an overview of GraalVM native image concepts, see the [reference:packaging/native-image/introducing-graalvm-native-images.html](#) section.

116. Testing GraalVM Native Images

When writing native image applications, we recommend that you continue to use the JVM whenever possible to develop the majority of your unit and integration tests. This will help keep developer build times down and allow you to use existing IDE integrations. With broad test coverage on the JVM, you can then focus native image testing on the areas that are likely to be different.

For native image testing, you're generally looking to ensure that the following aspects work:

- The Spring AOT engine is able to process your application, and it will run in an AOT-processed mode.
- GraalVM has enough hints to ensure that a valid native image can be produced.

116.1. Testing Ahead-of-Time Processing With the JVM

When a Spring Boot application runs, it attempts to detect if it is running as a native image. If it is running as a native image, it will initialize the application using the code that was generated during at build-time by the Spring AOT engine.

If the application is running on a regular JVM, then any AOT generated code is ignored.

Since the `native-image` compilation phase can take a while to complete, it's sometimes useful to run your application on the JVM but have it use the AOT generated initialization code. Doing so helps you to quickly validate that there are no errors in the AOT generated code and nothing is missing when your application is eventually converted to a native image.

To run a Spring Boot application on the JVM and have it use AOT generated code you can set the `spring.aot.enabled` system property to `true`.

For example:

```
$ java -Dspring.aot.enabled=true -jar myapplication.jar
```

SHELL



You need to ensure that the jar you are testing includes AOT generated code. For Maven, this means that you should build with `-Pnative` to activate the `native` profile. For Gradle, you need to ensure that your build includes the `org.graalvm.buildtools.native` plugin.

If your application starts with the `spring.aot.enabled` property set to `true`, then you have higher confidence that it will work when converted to a native image.

You can also consider running integration tests against the running application. For example, you could use the Spring `javax.xml.ws.client.WebClient` to call your application REST endpoints. Or you might consider using a project like Selenium to check your application's HTML responses.

116.2. Testing With Native Build Tools

GraalVM Native Build Tools includes the ability to run tests inside a native image. This can be helpful when you want to deeply test that the internals of your application work in a GraalVM native image.

Generating the native image that contains the tests to run can be a time-consuming operation, so most developers will probably prefer to use the JVM locally. They can, however, be very useful as part of a CI pipeline. For example, you might choose to run native tests once a day.

Spring Framework includes ahead-of-time support for running tests. All the usual Spring testing features work with native image tests. For example, you can continue to use the `org.springframework.boot.test.context.SpringBootTest` annotation. You can also use Spring Boot [test slices](#) to test only specific parts of your application.

Spring Framework's native testing support works in the following way:

- Tests are analyzed in order to discover any `org.springframework.context.ApplicationContext` instances that will be required.
- Ahead-of-time processing is applied to each of these application contexts and assets are generated.
- A native image is created, with the generated assets being processed by GraalVM.
- The native image also includes the `org.junit.platform.engine.TestEngine` configured with a list of the discovered tests.
- The native image is started, triggering the engine which will run each test and report results.

116.2.1. Using Maven

To run native tests using Maven, ensure that your `pom.xml` file uses the `spring-boot-starter-parent`. You should have a `<parent>` section that looks like this:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>{version-spring-boot}</version>
</parent>
```

XML

The `spring-boot-starter-parent` defines a `nativeTest` profile that provides the necessary configuration for the Spring Boot and Native Build Tools plugins. First you need to add those two plugin in the module to opt-in for the feature. Your tests are executed in native mode only when the `nativeTest` is enabled. You can activate profiles using the `-P` flag on the command line.



If you don't want to use `spring-boot-starter-parent` you'll need to configure executions for the `process-test-aot` goal from the Spring Boot plugin and the `test` goal from the Native Build Tools plugin.

To build the image and run the tests, use the `test` goal with the `nativeTest` profile active:

```
$ mvn -PnativeTest test
```

SHELL

116.2.2. Using Gradle

The Spring Boot Gradle plugin automatically configures AOT test tasks when the GraalVM Native Image plugin is applied. You should check that your Gradle build contains a `plugins` block that includes `org.graalvm.buildtools.native`.

To run native tests using Gradle you can use the `nativeTest` task:

```
$ gradle nativeTest
```

SHELL

117. NoSQL

Spring Boot offers a number of starters that support NoSQL technologies. This section answers questions that arise from using NoSQL with Spring Boot.

117.1. Use Jedis Instead of Lettuce

By default, the Spring Boot starter (`spring-boot-starter-data-redis`) uses Lettuce (<https://github.com/redis/lettuce/>). You need to exclude that dependency and include the Jedis (<https://github.com/redis/jedis/>) one instead. Spring Boot manages both of these dependencies, allowing you to switch to Jedis without specifying a version.

The following example shows how to accomplish this in Maven:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
  <exclusions>
    <exclusion>
      <groupId>io.lettuce</groupId>
      <artifactId>lettuce-core</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>redis.clients</groupId>
  <artifactId>jedis</artifactId>
</dependency>
```

XML

The following example shows how to accomplish this in Gradle:

```
dependencies {
    implementation('org.springframework.boot:spring-boot-starter-data-redis') {
        exclude group: 'io.lettuce', module: 'lettuce-core'
    }
    implementation 'redis.clients:jedis'
    // ...
}
```

GRADLE

118. Properties and Configuration

This section includes topics about setting and reading properties and configuration settings and their interaction with Spring Boot applications.

118.1. Automatically Expand Properties at Build Time

Rather than hardcoding some properties that are also specified in your project's build configuration, you can automatically expand them by instead using the existing build configuration. This is possible in both Maven and Gradle.

118.1.1. Automatic Property Expansion Using Maven

You can automatically expand properties in the Maven project by using resource filtering. If you use the `spring-boot-starter-parent`, you can then refer to your Maven 'project properties' with `@.` placeholders, as shown in the following example:

```
app:
  encoding: "@project.build.sourceEncoding@"
  java:
    version: "@java.version@"
```



Only production configuration is filtered that way (in other words, no filtering is applied on `src/test/resources`).



If you enable the `addResources` flag, the `spring-boot:run` goal can add `src/main/resources` directly to the classpath (for hot reloading purposes). Doing so circumvents the resource filtering and this feature. Instead, you can use the `exec:java` goal or customize the plugin's configuration. See the [plugin usage page](#) for more details.

If you do not use the starter parent, you need to include the following element inside the `<build/>` element of your `pom.xml`:

```
<resources>
  <resource>
    <directory>src/main/resources</directory>
    <filtering>true</filtering>
  </resource>
</resources>
```

XML

You also need to include the following element inside `<plugins/>`:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-resources-plugin</artifactId>
  <version>2.7</version>
  <configuration>
    <delimiters>
      <delimiter>@</delimiter>
    </delimiters>
    <useDefaultDelimiters>>false</useDefaultDelimiters>
  </configuration>
</plugin>
```

XML



The `useDefaultDelimiters` property is important if you use standard Spring placeholders (such as `${placeholder}`) in your configuration. If that property is not set to `false`, these may be expanded by the build.

118.1.2. Automatic Property Expansion Using Gradle

You can automatically expand properties from the Gradle project by configuring the Java plugin's `processResources` task to do so, as shown in the following example:

```
tasks.named('processResources') {
  expand(project.properties)
}
```

GRADLE

You can then refer to your Gradle project's properties by using placeholders, as shown in the following example:

```
app:
  name: "${name}"
  description: "${description}"
```



Gradle's `expand` method uses Groovy's `SimpleTemplateEngine`, which transforms `${..}` tokens. The `${..}` style conflicts with Spring's own property placeholder mechanism. To use Spring property placeholders together with automatic expansion, escape the Spring property placeholders as follows: `\${..}`.

118.2. Externalize the Configuration of SpringApplication

A `javadoc:org.springframework.boot.SpringApplication[]` has bean property setters, so you can use its Java API as you create the application to modify its behavior. Alternatively, you can externalize the configuration by setting properties in `spring.main.*`. For example, in `application.properties`, you might have the following settings:

```
spring:
  main:
    web-application-type: "none"
    banner-mode: "off"
```

Then the Spring Boot banner is not printed on startup, and the application is not starting an embedded web server.

Properties defined in external configuration override and replace the values specified with the Java API, with the notable exception of the primary sources. Primary sources are those provided to the `javadoc:org.springframework.boot.SpringApplication[]` constructor:

```
include-code::application/MyApplication[]
```

Or to `sources(...)` method of a `javadoc:org.springframework.boot.builder.SpringApplicationBuilder[]`:

```
include-code::builder/MyApplication[]
```

Given the examples above, if we have the following configuration:

```
spring:
  main:
    sources: "com.example.MyDatabaseConfig,com.example.MyJmsConfig"
    banner-mode: "console"
```

The actual application will show the banner (as overridden by configuration) and use three sources for the `javadoc:org.springframework.context.ApplicationContext[]`. The application sources are:

1. `MyApplication` (from the code)
2. `MyDatabaseConfig` (from the external config)
3. `MyJmsConfig` (from the external config)

118.3. Change the Location of External Properties of an Application

By default, properties from different sources are added to the Spring `javadoc:org.springframework.core.env.Environment[]` in a defined order (see [reference:features/external-config.html](#) in the “Spring Boot Features” section for the exact order).

You can also provide the following System properties (or environment variables) to change the behavior:

- `configprop:spring.config.name[]` (`configprop:spring.config.name[format=envvar]`): Defaults to `application` as the root of the file name.
- `configprop:spring.config.location[]` (`configprop:spring.config.location[format=envvar]`): The file to load (such as a classpath resource or a URL). A separate `javadoc:org.springframework.core.env.Environment[]` property source is set up for this document and it can be overridden by system properties, environment variables, or the command line.

No matter what you set in the environment, Spring Boot always loads `application.properties` as described above. By default, if YAML is used, then files with the ‘.yaml’ and ‘.yml’ extensions are also added to the list.



If you want detailed information about the files that are being loaded you can [set the logging level](#) of `org.springframework.boot.context.config` to `trace`.

118.4. Use ‘Short’ Command Line Arguments

Some people like to use (for example) `--port=9000` instead of `--server.port=9000` to set configuration properties on the command line. You can enable this behavior by using placeholders in `application.properties`, as shown in the following example:

```
server:
  port: "${port:8080}"
```



If you inherit from the `spring-boot-starter-parent` POM, the default filter token of the `maven-resources-plugins` has been changed from `${*}` to `@` (that is, `@maven.token@` instead of `${maven.token}`) to prevent conflicts with Spring-style placeholders. If you have enabled Maven filtering for the `application.properties` directly, you may want to also change the default filter token to use [other delimiters](#) (<https://maven.apache.org/plugins/maven-resources-plugin/resources-mojo.html#delimiters>).



In this specific case, the port binding works in a PaaS environment such as Heroku or Cloud Foundry. On those two platforms, the `PORT` environment variable is set automatically and Spring can bind to capitalized synonyms for `javadoc:org.springframework.core.env.Environment[]` properties.

118.5. Use YAML for External Properties

YAML is a superset of JSON and, as such, is a convenient syntax for storing external properties in a hierarchical format, as shown in the following example:

```
spring:
  application:
    name: "cruncher"
  datasource:
    driver-class-name: "com.mysql.jdbc.Driver"
    url: "jdbc:mysql://localhost/test"
  server:
    port: 9000
```

Create a file called `application.yaml` and put it in the root of your classpath. Then add `snakeyaml` to your dependencies (Maven coordinates `org.yaml:snakeyaml`, already included if you use the `spring-boot-starter`). A YAML file is parsed to a Java `Map<String, Object>` (like a JSON object), and Spring Boot flattens the map so that it is one level deep and has period-separated keys, as many people are used to with javadoc:java.util.Properties[] files in Java.

The preceding example YAML corresponds to the following `application.properties` file:

```
spring.application.name=cruncher
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost/test
server.port=9000
```

See [reference:features/external-config.html](#) in the “Spring Boot Features” section for more information about YAML.

118.6. Set the Active Spring Profiles

The Spring javadoc:org.springframework.core.env.Environment[] has an API for this, but you would normally set a System property (`configprop:spring.profiles.active[]`) or an OS environment variable (`configprop:spring.profiles.active[format=envvar]`). Also, you can launch your application with a `-D` argument (remember to put it before the main class or jar archive), as follows:

```
$ java -jar -Dspring.profiles.active=production demo-0.0.1-SNAPSHOT.jar
```

In Spring Boot, you can also set the active profile in `application.properties`, as shown in the following example:

```
spring:
  profiles:
    active: "production"
```

A value set this way is replaced by the System property or environment variable setting but not by the `SpringApplicationBuilder.profiles()` method. Thus, the latter Java API can be used to augment the profiles without changing the defaults.

See [reference:features/profiles.html](#) in the “Spring Boot Features” section for more information.

118.7. Set the Default Profile Name

The default profile is a profile that is enabled if no profile is active. By default, the name of the default profile is `default`, but it could be changed using a System property (`configprop:spring.profiles.default[]`) or an OS environment variable (`configprop:spring.profiles.default[format=envvar]`).

In Spring Boot, you can also set the default profile name in `application.properties`, as shown in the following example:

```
spring:
  profiles:
    default: "dev"
```

See [reference:features/profiles.html](#) in the “Spring Boot Features” section for more information.

118.8. Change Configuration Depending on the Environment

Spring Boot supports multi-document YAML and Properties files (see [reference:features/external-config.html](#) for details) which can be activated conditionally based on the active profiles.

If a document contains a `spring.config.activate.on-profile` key, then the profiles value (a comma-separated list of profiles or a profile expression) is fed into the Spring Environment.`acceptsProfiles()` method. If the profile expression matches, then that document is included in the final merge (otherwise, it is not), as shown in the following example:

```
server:
  port: 9000
---
spring:
  config:
    activate:
      on-profile: "development"
server:
  port: 9001
---
spring:
  config:
    activate:
      on-profile: "production"
server:
  port: 0
```


In the preceding example, the default port is 9000. However, if the Spring profile called 'development' is active, then the port is 9001. If 'production' is active, then the port is 0.



The documents are merged in the order in which they are encountered. Later values override earlier values.

118.9. Discover Built-in Options for External Properties

Spring Boot binds external properties from `application.properties` (or YAML files and other places) into an application at runtime. There is not (and technically cannot be) an exhaustive list of all supported properties in a single location, because contributions can come from additional jar files on your classpath.

A running application with the Actuator features has a `configprops` endpoint that shows all the bound and bindable properties available through `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]`.

The appendix includes an `application.properties` example with a list of the most common properties supported by Spring Boot. The definitive list comes from searching the source code for `javadoc:org.springframework.boot.context.properties.ConfigurationProperties[format=annotation]` and `javadoc:org.springframework.beans.factory.annotation.Value[format=annotation]` annotations as well as the occasional use of `javadoc:org.springframework.boot.context.properties.bind.Binder[]`. For more about the exact ordering of loading properties, see [reference:features/external-config.html](#).

119. Security

This section addresses questions about security when working with Spring Boot, including questions that arise from using Spring Security with Spring Boot.

For more about Spring Security, see the [Spring Security project page](#).

119.1. Switch Off the Spring Boot Security Configuration

If you define a `javax.annotation.Configuration` with a `javax.servlet.FilterChain` bean in your application, this action switches off the default webapp security settings in Spring Boot.

119.2. Change the UserDetailsService and Add User Accounts

If you provide a `javax.annotation.Bean` of type `javax.servlet.FilterChain`, `javax.servlet.Filter`, or `javax.servlet.FilterChain`, the default `javax.servlet.FilterChain` for `javax.servlet.FilterChain` is not created. This means you have the full feature set of Spring Security available (such as [Spring Security docs](#)/servlet/authentication/index.html[various authentication options]).

The easiest way to add user accounts is by providing your own `javax.servlet.FilterChain` bean.

119.3. Enable HTTPS When Running Behind a Proxy Server

Ensuring that all your main endpoints are only available over HTTPS is an important chore for any application. If you use Tomcat as a servlet container, then Spring Boot adds Tomcat's own `javax.servlet.FilterChain` automatically if it detects some environment settings, allowing you to rely on the `javax.servlet.FilterChain` to report whether it is secure or not (even downstream of a proxy server that handles the real SSL termination). The standard behavior is determined by the presence or absence of certain request headers (`x-forwarded-for` and `x-forwarded-proto`), whose names are conventional, so it should work with most front-end proxies. You can switch on the valve by adding some entries to `application.properties`, as shown in the following example:

```
server:
  tomcat:
    remoteip:
      remote-ip-header: "x-forwarded-for"
      protocol-header: "x-forwarded-proto"
```

(The presence of either of those properties switches on the valve. Alternatively, you can add the `javax.servlet.FilterChain` by customizing the `javax.servlet.FilterChain` using a `javax.servlet.FilterChain` bean.)

To configure Spring Security to require a secure channel for all (or some) requests, consider adding your own `javax.servlet.FilterChain` bean that adds the following `javax.servlet.FilterChain` configuration:

include-code::MySecurityConfig[]

120. Spring MVC

Spring Boot has a number of starters that include Spring MVC. Note that some starters include a dependency on Spring MVC rather than include it directly. This section answers common questions about Spring MVC and Spring Boot.

120.1. Write a JSON REST Service

Any Spring `javax.annotation.processing.annotation.RestController` in a Spring Boot application should render JSON response by default as long as Jackson 3 is on the classpath, as shown in the following example:

```
include-code::MyController[]
```

As long as `MyThing` can be serialized by Jackson 3 (true for a normal POJO or Groovy object), then `localhost:8080/thing` serves a JSON representation of it by default. Note that, in a browser, you might sometimes see XML responses, because browsers tend to send accept headers that prefer XML.

120.2. Write an XML REST Service

If you have the Jackson XML extension (`jackson-dataformat-xml`) on the classpath, you can use it to render XML responses. The previous example that we used for JSON would work. To use the Jackson XML renderer, add the following dependency to your project:

```
<dependency>
  <groupId>tools.jackson.dataformat</groupId>
  <artifactId>jackson-dataformat-xml</artifactId>
</dependency>
```

If Jackson's XML extension is not available and JAXB is available, XML can be rendered with the additional requirement of having `MyThing` annotated as `jakarta.xml.bind.annotation.XmlRootElement` [format=annotation], as shown in the following example:

```
include-code::MyThing[]
```

You will need to ensure that the JAXB library is part of your project, for example by adding:

```
<dependency>
  <groupId>org.glassfish.jaxb</groupId>
  <artifactId>jaxb-runtime</artifactId>
</dependency>
```



To get the server to render XML instead of JSON, you might have to send an `Accept: text/xml` header (or use a browser).

120.3. Customize the Jackson JsonMapper

You can configure the `tools.jackson.databind.json.JsonMapper` by using the environment. Jackson provides an extensive suite of on/off features that can be used to configure various aspects of its processing. These features are described in several enums (in Jackson) that map onto properties in the environment:

Enum	Property	Values
<code>tools.jackson.databind.cfg.DateTimeFeature</code>	<code>spring.jackson.datatype.datetime.<feature_name></code>	<code>true</code> , <code>false</code>
<code>tools.jackson.databind.cfg.EnumFeature</code>	<code>spring.jackson.datatype.enum.<feature_name></code>	<code>true</code> , <code>false</code>
<code>tools.jackson.databind.cfg.JsonNodeFeature</code>	<code>spring.jackson.datatype.json-node.<feature_name></code>	<code>true</code> , <code>false</code>
<code>com.fasterxml.jackson.annotation.JsonInclude\$Include</code>	<code>configprop:spring.jackson.default-property-inclusion</code>	<code>always</code> , <code>non_null</code> , <code>non_absent</code> , <code>non_default</code> , <code>non_empty</code>
<code>tools.jackson.databind.DeserializationFeature</code>	<code>spring.jackson.deserialization.<feature_name></code>	<code>true</code> , <code>false</code>
<code>tools.jackson.core.json.JsonReadFeature</code>	<code>spring.jackson.json.read.<feature_name></code>	<code>true</code> , <code>false</code>
<code>tools.jackson.core.json.JsonWriteFeature</code>	<code>spring.jackson.json.write.<feature_name></code>	<code>true</code> , <code>false</code>
<code>tools.jackson.databind.MapperFeature</code>	<code>spring.jackson.mapper.<feature_name></code>	<code>true</code> , <code>false</code>
<code>tools.jackson.databind.SerializationFeature</code>	<code>spring.jackson.serialization.<feature_name></code>	<code>true</code> , <code>false</code>

For example, to enable pretty print, set `spring.jackson.serialization.indent_output=true`. Note that, thanks to the use of relaxed binding, the case of `indent_output` does not have to match the case of the corresponding enum constant, which is `INDENT_OUTPUT`.

This environment-based configuration is applied to the auto-configured `tools.jackson.databind.json.JsonMapper$Builder` bean and applies to any mappers created by using the builder, including the auto-configured `tools.jackson.databind.json.JsonMapper` bean.

To ease the migration when working on an application that previously used Jackson 2, the auto-configured `JsonMapper` can be configured to use defaults that are as close as possible to those that Spring Boot used for Jackson 2. To enable these defaults, set `configprop:spring.jackson.use-jackson2-defaults` to `true`.

The context's `javax.tools.jackson.databind.json.JsonMapper$Builder[]` can be customized by one or more `javax.org.springframework.boot.jackson.autoconfigure.JsonMapperBuilderCustomizer[]` beans. Such customizer beans can be ordered (Boot's own customizer has an order of 0), letting additional customization be applied both before and after Boot's customization.

Any beans of type `javax.tools.jackson.databind.JacksonModule[]` are automatically registered with the auto-configured `javax.tools.jackson.databind.json.JsonMapper$Builder[]` and are applied to any `javax.tools.jackson.databind.json.JsonMapper[]` instances that it creates. This provides an application-wide mechanism for contributing custom modules when you add new features to your application.

Any modules that participate in the Java `ServiceLoader` mechanism are, by default, found and added to the auto-configured `javax.tools.jackson.databind.json.JsonMapper$Builder[]`. To disable this behavior, set `configprop:spring.jackson.find-and-add-modules[]` to `false`.

If you want to replace the default `javax.tools.jackson.databind.json.JsonMapper[]` completely, either define a `javax.org.springframework.context.annotation.Bean[format=annotation]` of that type or, if you prefer the builder-based approach, define a `javax.tools.jackson.databind.json.JsonMapper$Builder[]` `javax.org.springframework.context.annotation.Bean[format=annotation]`. When defining an `javax.tools.jackson.databind.json.JsonMapper[]` bean, marking it as `javax.org.springframework.context.annotation.Primary[format=annotation]` is recommended as the auto-configuration's `javax.tools.jackson.databind.json.JsonMapper[]` that it will replace is `javax.org.springframework.context.annotation.Primary[format=annotation]`. Note that, in either case, doing so disables all auto-configuration of the `javax.tools.jackson.databind.json.JsonMapper[]`.

If you provide any `javax.org.springframework.context.annotation.Bean[format=annotation]`s of type `javax.org.springframework.http.converter.json.JacksonJsonHttpMessageConverter[]`, they replace the default value in the MVC configuration. Also, a convenience bean of type `javax.org.springframework.boot.http.converter.autoconfigure.HttpMessageConverters[]` is provided (and is always available if you use the default MVC configuration). It has some useful methods to access the default and user-enhanced message converters.

See the [spring-mvc.html](#) section and the `{code-spring-boot-webmvc-src}/autoconfigure/WebMvcAutoConfiguration.java[ WebMvcAutoConfiguration ]` source code for more details.

120.4. Customize the @ResponseBody Rendering

Spring uses `javax.org.springframework.boot.http.converter.autoconfigure.HttpMessageConverters[]` to render `javax.org.springframework.web.bind.annotation.ResponseBody[format=annotation]` (or responses from `javax.org.springframework.web.bind.annotation.RestController[format=annotation]`). You can contribute additional converters by adding beans of the appropriate type in a Spring Boot context. If a bean you add is of a type that would have been included by default anyway (such as `javax.org.springframework.http.converter.json.JacksonJsonHttpMessageConverter[]` for JSON conversions), it replaces the default value. A convenience bean of type `javax.org.springframework.boot.http.converter.autoconfigure.HttpMessageConverters[]` is provided and is always available if you use the default MVC configuration. It has some useful methods to access the default and user-enhanced message converters (For example, it can be useful if you want to manually inject them into a custom `javax.org.springframework.web.client.RestTemplate[]`).

As in normal MVC usage, any `javax.org.springframework.web.servlet.config.annotation.WebMvcConfigurer[]` beans that you provide can also contribute converters by overriding the `configureMessageConverters` method. However, unlike with normal MVC, you can supply only additional converters that you need (because Spring Boot uses the same mechanism to contribute its defaults). Finally, if you opt out of the default Spring Boot MVC configuration by providing your own `javax.org.springframework.web.servlet.config.annotation.EnableWebMvc[format=annotation]` configuration, you can take control completely and do everything manually by using `getMessageConverters` from `javax.org.springframework.web.servlet.config.annotation.WebMvcConfigurationSupport[]`.

See the `{code-spring-boot-webmvc-src}/autoconfigure/WebMvcAutoConfiguration.java[ WebMvcAutoConfiguration ]` source code for more details.

120.5. Handling Multipart File Uploads

Spring Boot embraces the servlet 5 `javax.jakarta.servlet.http.Part[]` API to support uploading files. By default, Spring Boot configures Spring MVC with a maximum size of 1MB per file and a maximum of 10MB of file data in a single request. You may override these values, the location to which intermediate data is stored (for example, to the `/tmp` directory), and the threshold past which data is flushed to disk by using the properties exposed in the `javax.org.springframework.boot.servlet.autoconfigure.MultipartProperties[]` class. For example, if you want to specify that files be unlimited, set the `configprop:spring.servlet.multipart.max-file-size[]` property to `-1`.

The multipart support is helpful when you want to receive multipart encoded file data as a `javax.org.springframework.web.bind.annotation.RequestParam[format=annotation]`-annotated parameter of type `javax.org.springframework.web.multipart.MultipartFile[]` in a Spring MVC controller handler method.

See the `{code-spring-boot-servlet-src}/autoconfigure/MultipartAutoConfiguration.java[ MultipartAutoConfiguration ]` source for more details.



It is recommended to use the container's built-in support for multipart uploads rather than introduce an additional dependency such as Apache Commons File Upload.

120.6. Switch Off the Spring MVC DispatcherServlet

By default, all content is served from the root of your application (`/`). If you would rather map to a different path, you can configure one as follows:

```
spring:
  mvc:
    servlet:
      path: "/mypath"
```

If you have additional servlets you can declare a `javax.org.springframework.context.annotation.Bean[format=annotation]` of type `javax.jakarta.servlet.Servlet[]` or `javax.org.springframework.boot.web.servlet.ServletRegistrationBean[]` for each and Spring Boot will register them transparently to the container. It is also possible to use `javax.org.springframework.boot.web.servlet.ServletRegistration[format=annotation]` as an annotation-based alternative to `javax.org.springframework.boot.web.servlet.ServletRegistrationBean[]`. Because servlets are registered that way, they can be mapped to a sub-context of the `javax.org.springframework.web.servlet.DispatcherServlet[]` without invoking it.

Configuring the `javax.annotation.processing.processor.Path[]` yourself is unusual but if you really need to do it, a `javax.annotation.processing.processor.Path[]` of type `javax.annotation.processing.processor.Path[]` must be provided as well to provide the path of your custom `javax.annotation.processing.processor.Path[]`.

120.7. Switch Off the Default MVC Configuration

The easiest way to take complete control over MVC configuration is to provide your own `javax.annotation.processing.processor.Path[]` with the `javax.annotation.processing.processor.Path[]` annotation. Doing so leaves all MVC configuration in your hands.

120.8. Customize ViewResolvers

A `javax.annotation.processing.processor.Path[]` is a core component of Spring MVC, translating view names in `javax.annotation.processing.processor.Path[]` to actual `javax.annotation.processing.processor.Path[]` implementations. Note that view resolvers are mainly used in UI applications, rather than REST-style services (a `javax.annotation.processing.processor.Path[]` is not used to render a `javax.annotation.processing.processor.Path[]`). There are many implementations of `javax.annotation.processing.processor.Path[]` to choose from, and Spring on its own is not opinionated about which ones you should use. Spring Boot, on the other hand, installs one or two for you, depending on what it finds on the classpath and in the application context. The `javax.annotation.processing.processor.Path[]` uses all the resolvers it finds in the application context, trying each one in turn until it gets a result. If you add your own, you have to be aware of the order and in which position your resolver is added.

`javax.annotation.processing.processor.Path[]` adds the following `javax.annotation.processing.processor.Path[]` beans to your context:

- An `javax.annotation.processing.processor.Path[]` named 'defaultViewResolver'. This one locates physical resources that can be rendered by using the `DefaultServlet` (including static resources and JSP pages, if you use those). It applies a prefix and a suffix to the view name and then looks for a physical resource with that path in the servlet context (the defaults are both empty but are accessible for external configuration through `spring.mvc.view.prefix` and `spring.mvc.view.suffix`). You can override it by providing a bean of the same type.
- A `javax.annotation.processing.processor.Path[]` named 'beanNameViewResolver'. This is a useful member of the view resolver chain and picks up any beans with the same name as the `javax.annotation.processing.processor.Path[]` being resolved. It should not be necessary to override or replace it.
- A `javax.annotation.processing.processor.Path[]` named 'viewResolver' is added only if there **are** actually beans of type `javax.annotation.processing.processor.Path[]` present. This is a composite resolver, delegating to all the others and attempting to find a match to the 'Accept' HTTP header sent by the client. There is a useful [blog about javax.annotation.processing.processor.Path\[\]](https://spring.io/blog/2013/06/03/content-negotiation-using-views) (<https://spring.io/blog/2013/06/03/content-negotiation-using-views>) that you might like to study to learn more, and you might also look at the source code for detail. You can switch off the auto-configured `javax.annotation.processing.processor.Path[]` by defining a bean named 'viewResolver'.
- If you use Thymeleaf, you also have a `javax.annotation.processing.processor.Path[]` named 'thymeleafViewResolver'. It looks for resources by surrounding the view name with a prefix and suffix. The prefix is `spring.thymeleaf.prefix`, and the suffix is `spring.thymeleaf.suffix`. The values of the prefix and suffix default to 'classpath:/templates/' and '.html', respectively. You can override `javax.annotation.processing.processor.Path[]` by providing a bean of the same name.
- If you use FreeMarker, you also have a `javax.annotation.processing.processor.Path[]` named 'freemarkerViewResolver'. It looks for resources in a loader path (which is externalized to `spring.freemarker.templateLoaderPath` and has a default value of 'classpath:/templates/') by surrounding the view name with a prefix and a suffix. The prefix is externalized to `spring.freemarker.prefix`, and the suffix is externalized to `spring.freemarker.suffix`. The default values of the prefix and suffix are empty and '.ftlh', respectively. You can override `javax.annotation.processing.processor.Path[]` by providing a bean of the same name. FreeMarker variables can be customized by defining a bean of type `javax.annotation.processing.processor.Path[]`.
- If you use Groovy templates (actually, if `groovy-templates` is on your classpath), you also have a `javax.annotation.processing.processor.Path[]` named 'groovyMarkupViewResolver'. It looks for resources in a loader path by surrounding the view name with a prefix and suffix (externalized to `spring.groovy.template.prefix` and `spring.groovy.template.suffix`). The prefix and suffix have default values of 'classpath:/templates/' and '.tpl', respectively. You can override `javax.annotation.processing.processor.Path[]` by providing a bean of the same name.
- If you use Mustache, you also have a `javax.annotation.processing.processor.Path[]` named 'mustacheViewResolver'. It looks for resources by surrounding the view name with a prefix and suffix. The prefix is `spring.mustache.prefix`, and the suffix is `spring.mustache.suffix`. The values of the prefix and suffix default to 'classpath:/templates/' and '.mustache', respectively. You can override `javax.annotation.processing.processor.Path[]` by providing a bean of the same name.

For more detail, see the following sections:

- `{code-spring-boot-webmvc-src}/autoconfigure/WebMvcAutoConfiguration.java[ WebMvcAutoConfiguration ]`
- `{code-spring-boot-thymeleaf-src}/autoconfigure/ThymeleafAutoConfiguration.java[ ThymeleafAutoConfiguration ]`
- `{code-spring-boot-freemarker-src}/autoconfigure/FreeMarkerAutoConfiguration.java[ FreeMarkerAutoConfiguration ]`
- `{code-spring-boot-groovy-templates-src}/autoconfigure/GroovyTemplateAutoConfiguration.java[ GroovyTemplateAutoConfiguration ]`

120.9. Customize the 'whitelabel' Error Page

Spring Boot installs a 'whitelabel' error page that you see in a browser client if you encounter a server error (machine clients consuming JSON and other media types should see a sensible response with the right error code).



Set `spring.web.error.whitelabel.enabled` to **false** to switch the default error page off. Doing so restores the default of the servlet container that you are using. Note that Spring Boot still tries to resolve the error view, so you should probably add your own error page rather than disabling it completely.

Overriding the error page with your own depends on the templating technology that you use. For example, if you use Thymeleaf, you can add an `error.html` template. If you use FreeMarker, you can add an `error.ftlh` template. In general, you need a `javax.annotation.processing.processor.Path[]` that resolves with a name of `error` or a `javax.annotation.processing.processor.Path[]` that handles the `/error` path. Unless you replaced some of the default configuration, you should find a

javadoc:org.springframework.web.servlet.view.BeanNameViewResolver[] in your javadoc:org.springframework.context.ApplicationContext[], so a javadoc:org.springframework.context.annotation.Bean[format=annotation] named `error` would be one way of doing that. See {code-spring-boot-webmvc-src}/autoconfigure/error/ErrorMvcAutoConfiguration.java[`ErrorMvcAutoConfiguration`] for more options. See also the section on [reference:web/servlet.html](#) for details of how to register handlers in the servlet container.

121. Testing

Spring Boot includes a number of testing utilities and support classes as well as a dedicated starter that provides common test dependencies. This section answers common questions about testing.

121.1. Testing With Spring Security

Spring Security provides support for running tests as a specific user. For example, the test in the snippet below will run with an authenticated user that has the `ADMIN` role.

```
include-code::MySecurityTests[]
```

Spring Security provides comprehensive integration with Spring MVC Test, and this can also be used when testing controllers using the `javadoc:org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest[format=annotation]` slice and `javadoc:org.springframework.test.web.servlet.MockMvc[]`.

For additional details on Spring Security's testing support, see Spring Security's url-spring-security-docs/servlet/test/index.html [reference documentation].

121.2. Structure `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` Classes for Inclusion in Slice Tests

Slice tests work by restricting Spring Framework's component scanning to a limited set of components based on their type. For any beans that are not created through component scanning, for example, beans that are created using the `javadoc:org.springframework.context.annotation.Bean[format=annotation]` annotation, slice tests will not be able to include/exclude them from the application context. Consider this example:

```
include-code::MyConfiguration[]
```

For a `javadoc:org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest[format=annotation]` for an application with the above `javadoc:org.springframework.context.annotation.Configuration[format=annotation]` class, you might expect to have the `javadoc:org.springframework.security.web.SecurityFilterChain[]` bean in the application context so that you can test if your controller endpoints are secured properly. However, `MyConfiguration` is not picked up by `@WebMvcTest`'s component scanning filter because it doesn't match any of the types specified by the filter. You can include the configuration explicitly by annotating the test class with `@Import(MyConfiguration.class)`. This will load all the beans in `MyConfiguration` including the `javadoc:com.zaxxer.hikari.HikariDataSource[]` bean which isn't required when testing the web tier. Splitting the configuration class into two will enable importing just the security configuration.

```
include-code::MySecurityConfiguration[]
```

```
include-code::MyDatasourceConfiguration[]
```

Having a single configuration class can be inefficient when beans from a certain domain need to be included in slice tests. Instead, structuring the application's configuration as multiple granular classes with beans for a specific domain can enable importing them only for specific slice tests.

122. Embedded Web Servers

Each Spring Boot web application includes an embedded web server. This feature leads to a number of how-to questions, including how to change the embedded server and how to configure the embedded server. This section answers those questions.

122.1. Use Another Web Server

Many Spring Boot starters include default embedded containers.

- For servlet stack applications, the `spring-boot-starter-web` includes Tomcat by including `spring-boot-starter-tomcat`, but you can use `spring-boot-starter-jetty` instead.
- For reactive stack applications, the `spring-boot-starter-webflux` includes Reactor Netty by including `spring-boot-starter-reactor-netty`, but you can use `spring-boot-starter-tomcat` or `spring-boot-starter-jetty` instead.

When switching to a different HTTP server, you need to swap the default dependencies for those that you need instead. To help with this process, Spring Boot provides a separate starter for each of the supported HTTP servers.

The following example shows how to exclude Tomcat and include Jetty for Spring MVC:

Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
  <exclusions>
    <!-- Exclude the Tomcat dependency -->
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<!-- Use Jetty instead -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

XML

Gradle

```
dependencies {
    implementation('org.springframework.boot:spring-boot-starter-webmvc') {
        // Exclude the Tomcat dependency
        exclude group: 'org.springframework.boot', module: 'spring-boot-starter-tomcat'
    }
    // Use Jetty instead
    implementation "org.springframework.boot:spring-boot-starter-jetty"
}
```

GRADLE

If you are creating a war file, you can use a similar approach, but you must indicate provided dependencies:

Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
  <exclusions>
    <!-- Exclude the Tomcat dependency -->
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<!-- Use Jetty instead -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
  <scope>provided</scope>
</dependency>
```

XML

Gradle


```
dependencies {
    implementation('org.springframework.boot:spring-boot-starter-webmvc') {
        // Exclude the Tomcat dependency
        exclude group: 'org.springframework.boot', module: 'spring-boot-starter-tomcat'
    }
    // Use Jetty instead
    implementation "org.springframework.boot:spring-boot-starter-jetty"
    providedRuntime "org.springframework.boot:spring-boot-starter-jetty-runtime"
}
```

122.2. Disabling the Web Server

If your classpath contains the necessary bits to start a web server, Spring Boot will automatically start it. To disable this behavior configure the `javadoc:org.springframework.boot.WebApplicationType[]` in your `application.properties`, as shown in the following example:

```
spring:
  main:
    web-application-type: "none"
```

122.3. Change the HTTP Port

In a standalone application, the main HTTP port defaults to `8080` but can be set with `configprop:server.port[]` (for example, in `application.properties` or as a System property). Thanks to relaxed binding of `javadoc:org.springframework.core.env.Environment[]` values, you can also use `configprop:server.port[format=envvar]` (for example, as an OS environment variable).

To switch off the HTTP endpoints completely but still create a `javadoc:org.springframework.web.context.WebApplicationContext[]`, use `server.port=-1` (doing so is sometimes useful for testing).

For more details, see [Customizing Embedded Servlet Containers](#) in the ‘Spring Boot Features’ section, or the `javadoc:org.springframework.boot.web.server.autoconfigure.ServerProperties[]` class.

122.4. Use a Random Unassigned HTTP Port

To scan for a free port (using OS natives to prevent clashes) use `server.port=0`.

122.5. Discover the HTTP Port at Runtime

You can access the port the server is running on from log output or from the `javadoc:org.springframework.boot.web.server.context.WebServerApplicationContext[]` through its `javadoc:org.springframework.boot.web.server.WebServer[]`. The best way to get that and be sure it has been initialized is to add a `javadoc:org.springframework.context.annotation.Bean[format=annotation]` of type `ApplicationListener<WebServerInitializedEvent>` and pull the container out of the event when it is published.

Tests that use `@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)` can also inject the actual port into a field by using the `javadoc:org.springframework.boot.test.web.server.LocalServerPort[format=annotation]` annotation, as shown in the following example:

```
include-code::MyWebIntegrationTests[]
```



`javadoc:org.springframework.boot.test.web.server.LocalServerPort[format=annotation]` is a meta-annotation for `@Value("${local.server.port}")`. Do not try to inject the port in a regular application. As we just saw, the value is set only after the container has been initialized. Contrary to a test, application code callbacks are processed early (before the value is actually available).

122.6. Build URI Against the Test Web Server

Tests that start a `javadoc:org.springframework.boot.web.server.WebServer[]` such as `@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)` can get information about the running server using `javadoc:org.springframework.boot.test.http.server.LocalTestWebServer[]`, as shown in the following example:

```
include-code::MyWebIntegrationTests[]
```

`javadoc:org.springframework.boot.test.http.server.LocalTestWebServer[]` provides access to a suitable `javadoc:org.springframework.web.util.UriBuilderFactory[]`. It can also be customized to build such an instance on a sub-path of your application.

122.7. Enable HTTP Response Compression

HTTP response compression is supported by Jetty, Tomcat and Reactor Netty. It can be enabled in `application.properties`, as follows:

```
server:
  compression:
    enabled: true
```

By default, responses must be at least 2048 bytes in length for compression to be performed. You can configure this behavior by setting the `configprop:server.compression.min-response-size[]` property.

By default, responses are compressed only if their content type is one of the following:

- `text/html`
- `text/xml`

- `text/plain`
- `text/css`
- `text/javascript`
- `application/javascript`
- `application/json`
- `application/xml`

To configure the mime types that will be compressed, replacing the defaults, set the `configprop:server.compression.mime-types[]` property. To keep the defaults and configure additional mime types, set the `configprop:server.compression.additional-mime-types[]` property instead.

122.8. Configure SSL

SSL can be configured declaratively by setting the various `server.ssl.*` properties, typically in `application.properties` or `application.yml`. See [javadoc:org.springframework.boot.web.server.Ssl\[\]](https://java-docs.org/springframework.boot/web.server.Ssl/) for details of all of the supported properties.

The following example shows setting SSL properties using a Java KeyStore file:

```
server:
  port: 8443
  ssl:
    key-store: "classpath:keystore.jks"
    key-store-password: "secret"
    key-password: "another-secret"
```

Using configuration such as the preceding example means the application no longer supports a plain HTTP connector at port 8080. Spring Boot does not support the configuration of both an HTTP connector and an HTTPS connector through `application.properties`. If you want to have both, you need to configure one of them programmatically. We recommend using `application.properties` to configure HTTPS, as the HTTP connector is the easier of the two to configure programmatically.

122.8.1. Using PEM-encoded files

You can use PEM-encoded files instead of Java KeyStore files. You should use PKCS#8 key files wherever possible. PEM-encoded PKCS#8 key files start with a `-----BEGIN PRIVATE KEY-----` or `-----BEGIN ENCRYPTED PRIVATE KEY-----` header.

If you have files in other formats, e.g., PKCS#1 (`-----BEGIN RSA PRIVATE KEY-----`) or SEC 1 (`-----BEGIN EC PRIVATE KEY-----`), you can convert them to PKCS#8 using OpenSSL:

```
openssl pkcs8 -topk8 -nocrypt -in <input file> -out <output file>
```

SHELL

The following example shows setting SSL properties using PEM-encoded certificate and private key files:

```
server:
  port: 8443
  ssl:
    certificate: "classpath:my-cert.crt"
    certificate-private-key: "classpath:my-cert.key"
    trust-certificate: "classpath:ca-cert.crt"
```

122.8.2. Using SSL Bundles

Alternatively, the SSL trust material can be configured in an [SSL bundle](#) and applied to the web server as shown in this example:

```
server:
  port: 8443
  ssl:
    bundle: "example"
```



The `server.ssl.bundle` property can not be combined with the discrete Java KeyStore or PEM property options under `server.ssl`.

The `configprop:server.ssl.ciphers[]`, `configprop:server.ssl.enabled-protocols[]`, `configprop:server.ssl.protocol[]` properties are also ignored when using a bundle. These properties should instead be defined using `spring.ssl.bundle.<type>.<name>.options` properties.

122.8.3. Configure Server Name Indication

Tomcat, and Netty can be configured to use unique SSL trust material for individual host names to support Server Name Indication (SNI). SNI configuration is not supported with Jetty, but Jetty can [automatically set up SNI](https://eclipse.dev/jetty/documentation/jetty-12/operations-guide/index.html#og-protocols-ssl-sni) (https://eclipse.dev/jetty/documentation/jetty-12/operations-guide/index.html#og-protocols-ssl-sni) if multiple certificates are provided to it.

Assuming [SSL bundles](#) named `web`, `web-alt1`, and `web-alt2` have been configured, the following configuration can be used to assign each bundle to a host name served by the embedded web server:

```
server:
  port: 8443
  ssl:
    bundle: "web"
    server-name-bundles:
      - server-name: "alt1.example.com"
        bundle: "web-alt1"
      - server-name: "alt2.example.com"
        bundle: "web-alt2"
```

The bundle specified with `server.ssl.bundle` will be used for the default host, and for any client that does support SNI. This default bundle must be configured if any `server.ssl.server-name-bundles` are configured.

122.9. Configure HTTP/2

You can enable HTTP/2 support in your Spring Boot application with the `configprop:server.http2.enabled[]` configuration property. Both `h2` (HTTP/2 over TLS) and `h2c` (HTTP/2 over TCP) are supported. To use `h2`, SSL must also be enabled. When SSL is not enabled, `h2c` will be used. You may, for example, want to use `h2c` when your application is running behind a proxy server that is performing TLS termination.

122.9.1. HTTP/2 With Tomcat

Spring Boot ships by default with Tomcat 11.0.x which supports `h2c` and `h2` out of the box. Alternatively, you can use `libtcnative` for `h2` support if the library and its dependencies are installed on the host operating system.

The library directory must be made available, if not already, to the JVM library path. You can do so with a JVM argument such as `-Djava.library.path=/usr/local/opt/tomcat-native/lib`. More on this in the [{url-tomcat-docs}/apr.html](#) [official Tomcat documentation].

122.9.2. HTTP/2 With Jetty

For HTTP/2 support, Jetty requires the additional `org.eclipse.jetty.http2:jetty-http2-server` dependency. To use `h2c` no other dependencies are required. To use `h2`, you also need to choose one of the following dependencies, depending on your deployment:

- `org.eclipse.jetty:jetty-alpn-java-server` to use the JDK built-in support
- `org.eclipse.jetty:jetty-alpn-conscrypt-server` and the [Conscrypt library](https://www.conscrypt.org/) (<https://www.conscrypt.org/>)

122.9.3. HTTP/2 With Reactor Netty

The `spring-boot-webflux-starter` is using by default Reactor Netty as a server. Reactor Netty supports `h2c` and `h2` out of the box. For optimal runtime performance, this server also supports `h2` with native libraries. To enable that, your application needs to have an additional dependency.

Spring Boot manages the version for the `io.netty:netty-tcnative-boringssl-static` "uber jar", containing native libraries for all platforms. Developers can choose to import only the required dependencies using a classifier (see [the Netty official documentation](https://netty.io/wiki/forked-tomcat-native.html) (<https://netty.io/wiki/forked-tomcat-native.html>)).

122.10. Configure the Web Server

Generally, you should first consider using one of the many available configuration keys and customize your web server by adding new entries in your `application.properties` or `application.yml` file. See [properties-and-configuration.html](#)). The `server.*` namespace is quite useful here, and it includes namespaces like `server.tomcat.*`, `server.jetty.*` and others, for server-specific features. See the list of [appendix:application-properties/index.html](#).

The previous sections covered already many common use cases, such as compression, SSL or HTTP/2. However, if a configuration key does not exist for your use case, you should then look at `javadoc:org.springframework.boot.web.server.WebServerFactoryCustomizer[]`. You can declare such a component and get access to the server factory relevant to your choice: you should select the variant for the chosen Server (Tomcat, Jetty, Reactor Netty) and the chosen web stack (servlet or reactive).

The example below is for Tomcat with the `spring-boot-starter-web` (servlet stack):

```
include-code::MyTomcatWebServerCustomizer[]
```



Spring Boot uses that infrastructure internally to auto-configure the server. Auto-configured `javadoc:org.springframework.boot.web.server.WebServerFactoryCustomizer[]` beans have an order of `0` and will be processed before any user-defined customizers, unless it has an explicit order that states otherwise.

Once you have got access to a `javadoc:org.springframework.boot.web.server.WebServerFactory[]` using the customizer, you can use it to configure specific parts, like connectors, server resources, or the server itself - all using server-specific APIs.

In addition Spring Boot provides:

Server	Servlet stack	Reactive stack
Tomcat	<code>javadoc:org.springframework.boot.tomcat.servlet.TomcatServletWebServerFactory[]</code>	<code>javadoc:org.springframework.boot.tomcat.reactive.TomcatReactiveWebServerFactory[]</code>
Jetty	<code>javadoc:org.springframework.boot.jetty.servlet.JettyServletWebServerFactory[]</code>	<code>javadoc:org.springframework.boot.jetty.reactive.JettyReactiveWebServerFactory[]</code>
Reactor	N/A	<code>javadoc:org.springframework.boot.reactor.netty.NettyReactiveWebServerFactory[]</code>

As a last resort, you can also declare your own `javadoc:org.springframework.boot.web.server.WebServerFactory[]` bean, which will override the one provided by Spring Boot. When you do so, auto-configured customizers are still applied on your custom factory, so use that option carefully.

122.11. Add a Servlet, Filter, or Listener to an Application

In a servlet stack application, that is with the `spring-boot-starter-web`, there are two ways to add `javax.servlet.Servlet`, `javax.servlet.Filter`, `javax.servlet.ServletContextListener`, and the other listeners supported by the Servlet API to your application:

- [webserver.html](#)
- [webserver.html](#)

122.11.1. Add a Servlet, Filter, or Listener by Using a Spring Bean

To add a `javax.servlet.Servlet`, `javax.servlet.Filter`, or servlet `*Listener` by using a Spring bean, you must provide a `javax.annotation.AnnotationBean` definition for it. Doing so can be very useful when you want to inject configuration or dependencies. However, you must be very careful that they do not cause eager initialization of too many other beans, because they have to be installed in the container very early in the application lifecycle. (For example, it is not a good idea to have them depend on your `javax.sql.DataSource` or JPA configuration.) You can work around such restrictions by initializing the beans lazily when first used instead of on initialization.

In the case of filters and servlets, you can also add mappings and init parameters by adding a `org.springframework.boot.web.servlet.FilterRegistrationBean` or a `org.springframework.boot.web.servlet.ServletRegistrationBean` instead of or in addition to the underlying component. You can also use `org.springframework.boot.web.servlet.ServletRegistration` and `org.springframework.boot.web.servlet.FilterRegistration` as an annotation-based alternative to `org.springframework.boot.web.servlet.ServletRegistrationBean` and `org.springframework.boot.web.servlet.FilterRegistrationBean`.



If no `dispatcherType` is specified on a filter registration, `REQUEST` is used. This aligns with the servlet specification's default dispatcher type.

Like any other Spring bean, you can define the order of servlet filter beans; please make sure to check the [reference:web/servlet.html](#) section.

Disable Registration of a Servlet or Filter

As [described earlier](#), any `javax.servlet.Servlet` or `javax.servlet.Filter` beans are registered with the servlet container automatically. To disable registration of a particular `javax.servlet.Filter` or `javax.servlet.Servlet` bean, create a registration bean for it and mark it as disabled, as shown in the following example:

```
include-code::MyFilterConfiguration[]
```

122.11.2. Add Servlets, Filters, and Listeners by Using Classpath Scanning

`javax.servlet.annotation.WebServlet`, `javax.servlet.annotation.WebFilter`, and `javax.servlet.annotation.WebListener` annotated classes can be automatically registered with an embedded servlet container by annotating a `org.springframework.context.annotation.Configuration` class with `org.springframework.boot.web.server.servlet.context.ServletComponentScan` and specifying the package(s) containing the components that you want to register. By default, `org.springframework.boot.web.server.servlet.context.ServletComponentScan` scans from the package of the annotated class.

122.12. Configure Access Logging

Access logs can be configured for Tomcat and Jetty through their respective namespaces.

For instance, the following settings log access on Tomcat with a `{url-tomcat-docs}/config/valve.html#Access_Logging[custom pattern]`.

```
server:
  tomcat:
    basedir: "my-tomcat"
    accesslog:
      enabled: true
      pattern: "%t %a %r %s (%D microseconds)"
```



The default location for logs is a `logs` directory relative to the Tomcat base directory. By default, the `logs` directory is a temporary directory, so you may want to fix Tomcat's base directory or use an absolute path for the logs. In the preceding example, the logs are available in `my-tomcat/logs` relative to the working directory of the application.

Finally, access logging for Jetty can also be configured as follows:

```
server:
  jetty:
    accesslog:
      enabled: true
      filename: "/var/log/jetty-access.log"
```

By default, logs are redirected to `java.lang.System#err`. For more details, see the Jetty documentation.

122.13. Running Behind a Front-end Proxy Server

If your application is running behind a proxy, a load-balancer or in the cloud, the request information (like the host, port, scheme...) might change along the way. Your application may be running on `10.10.10.10:8080`, but HTTP clients should only see `example.org`.

[RFC7239 "Forwarded Headers"](https://tools.ietf.org/html/rfc7239) defines the `Forwarded` HTTP header; proxies can use this header to provide information about the original request. You can configure your application to read those headers and automatically use that information when creating links and sending them to clients in HTTP 302 responses, JSON documents or HTML pages. There are also non-standard headers, like `X-Forwarded-Host`, `X-Forwarded-Port`, `X-Forwarded-Proto`, `X-Forwarded-Ssl`, and `X-Forwarded-Prefix`.

If the proxy adds the commonly used `X-Forwarded-For` and `X-Forwarded-Proto` headers, setting `server.forward-headers-strategy` to `NATIVE` is enough to support those. With this option, the Web servers themselves natively support this feature; you can check their specific documentation to learn about specific behavior.

If this is not enough, Spring Framework provides a [{url-spring-framework-docs}/web/webmvc/filters.html#filters-forwarded-headers](#) [ForwardedHeaderFilter] for the servlet stack and a [{url-spring-framework-docs}/web/webflux/reactive-spring.html#webflux-forwarded-headers](#) [ForwardedHeaderTransformer] for the reactive stack. You can use them in your application by setting `configprop:server.forward-headers-strategy[]` to `FRAMEWORK`.



If you are using Tomcat and terminating SSL at the proxy, `configprop:server.tomcat.redirect-context-root[]` should be set to `false`. This allows the `X-Forwarded-Proto` header to be honored before any redirects are performed.



If your application runs `javadoc:org.springframework.boot.cloud.CloudPlatform#enum-constant-summary` [in a supported Cloud Platform], the `configprop:server.forward-headers-strategy[]` property defaults to `NATIVE`. In all other instances, it defaults to `NONE`.

122.13.1. Customize Tomcat's Proxy Configuration

If you use Tomcat, you can additionally configure the names of the headers used to carry “forwarded” information, as shown in the following example:

```
server:
  tomcat:
    remoteip:
      remote-ip-header: "x-your-remote-ip-header"
      protocol-header: "x-your-protocol-header"
```

Tomcat is also configured with a regular expression that matches internal proxies that are to be trusted. See the [configprop:server.tomcat.remoteip.internal-proxies](#) [entry in the appendix] for its default value. You can customize the valve's configuration by adding an entry to `application.properties`, as shown in the following example:

```
server:
  tomcat:
    remoteip:
      internal-proxies: "192\\.168\\.\\.\\d{1,3}\\\\.\\.\\d{1,3}"
```



You can trust all proxies by setting the `internal-proxies` to empty (but do not do so in production).

You can take complete control of the configuration of Tomcat's `javadoc:org.apache.catalina.valves.RemoteIpValve[]` by switching the automatic one off (to do so, set `server.forward-headers-strategy=NONE`) and adding a new valve instance using a `javadoc:org.springframework.boot.web.server.WebServerFactoryCustomizer[]` bean.

122.14. Enable Multiple Connectors with Tomcat

You can add an `javadoc:org.apache.catalina.connector.Connector[]` to the `javadoc:org.springframework.boot.tomcat.servlet.TomcatServletWebServerFactory[]`, which can allow multiple connectors, including HTTP and HTTPS connectors, as shown in the following example:

```
include-code::MyTomcatConfiguration[]
```

122.15. Enable Tomcat's MBean Registry

Embedded Tomcat's MBean registry is disabled by default. This minimizes Tomcat's memory footprint. If you want to use Tomcat's MBeans, for example so that they can be used by Micrometer to expose metrics, you must use the `configprop:server.tomcat.mbeanregistry.enabled[]` property to do so, as shown in the following example:

```
server:
  tomcat:
    mbeanregistry:
      enabled: true
```

122.16. Create WebSocket Endpoints Using @ServerEndpoint

If you want to use `javadoc:jakarta.websocket.server.ServerEndpoint[format=annotation]` in a Spring Boot application that used an embedded container, you must declare a single `javadoc:org.springframework.web.socket.server.standard.ServerEndpointExporter[]` `javadoc:org.springframework.context.annotation.Bean[format=annotation]`, as shown in the following example:

```
include-code::MyWebSocketConfiguration[]
```

The bean shown in the preceding example registers any `javadoc:jakarta.websocket.server.ServerEndpoint[format=annotation]` annotated beans with the underlying WebSocket container. When deployed to a standalone servlet container, this role is performed by a servlet container initializer, and the `javadoc:org.springframework.web.socket.server.standard.ServerEndpointExporter[]` bean is not required.

124. Spring Boot AntLib Module

The Spring Boot AntLib module provides basic Spring Boot support for Apache Ant. You can use the module to create executable jars. To use the module, you need to declare an additional `spring-boot` namespace in your `build.xml`, as shown in the following example:

```
<project xmlns:ivy="antlib:org.apache.ivy.ant"
  xmlns:spring-boot="antlib:org.springframework.boot.ant"
  name="myapp" default="build">
  ...
</project>
```

You need to remember to start Ant using the `-lib` option, as shown in the following example:

```
$ ant -lib <directory containing spring-boot-antlib-{version-spring-boot}.jar>
```



The “Using Spring Boot” section includes a more complete example of [using Apache Ant with `spring-boot-antlib`](#).

124.1. Spring Boot Ant Tasks

Once the `spring-boot-antlib` namespace has been declared, the following additional tasks are available:

- [antlib.html](#)
- [antlib.html](#)

124.1.1. Using the “exejar” Task

You can use the `exejar` task to create a Spring Boot executable jar. The following attributes are supported by the task:

Attribute	Description	Required
<code>destfile</code>	The destination jar file to create	Yes
<code>classes</code>	The root directory of Java class files	Yes
<code>start-class</code>	The main application class to run	No (<i>the default is the first class found that declares a <code>main</code> method</i>)

The following nested elements can be used with the task:

Element	Description
<code>resources</code>	One or more {url-ant-docs}/Types/resources.html#collection[Resource Collections] describing a set of {url-ant-docs}/Types/resources.html[Resources] that should be added to the content of the created jar file.
<code>lib</code>	One or more {url-ant-docs}/Types/resources.html#collection[Resource Collections] that should be added to the set of jar libraries that make up the runtime dependency classpath of the application.

124.1.2. Examples

This section shows two examples of Ant tasks.

Specify start-class

```
<spring-boot:exejar destfile="target/my-application.jar"
  classes="target/classes" start-class="com.example.MyApplication">
  <resources>
    <fileset dir="src/main/resources" />
  </resources>
  <lib>
    <fileset dir="lib" />
  </lib>
</spring-boot:exejar>
```

Detect start-class

```
<exejar destfile="target/my-application.jar" classes="target/classes">
  <lib>
    <fileset dir="lib" />
  </lib>
</exejar>
```

124.2. Using the “findmainclass” Task

The `findmainclass` task is used internally by `exejar` to locate a class declaring a `main`. If necessary, you can also use this task directly in your build. The following attributes are supported:

Attribute	Description	Required
classesroot	The root directory of Java class files	Yes <i>(unless mainclass is specified)</i>
mainclass	Can be used to short-circuit the main class search	No
property	The Ant property that should be set with the result	No <i>(result will be logged if unspecified)</i>

124.2.1. Examples

This section contains three examples of using `findmainclass`.

Find and log

```
<findmainclass classesroot="target/classes" />
```

XML

Find and set

```
<findmainclass classesroot="target/classes" property="main-class" />
```

XML

Override and set

```
<findmainclass mainclass="com.example.MainClass" property="main-class" />
```

XML

125. Build Tool Plugins

Spring Boot provides build tool plugins for Maven and Gradle. The plugins offer a variety of features, including the packaging of executable jars. This section provides more details on both plugins as well as some help should you need to extend an unsupported build system. If you are just getting started, you might want to read [reference:using/build-systems.html](#) from the [reference:using/index.html](#) section first.

126. Supporting Other Build Systems

If you want to use a build tool other than Maven, Gradle, or Ant, you likely need to develop your own plugin. Executable jars need to follow a specific format and certain entries need to be written in an uncompressed form (see the [executable jar format](#) section in the appendix for details).

The Spring Boot Maven and Gradle plugins both make use of `spring-boot-loader-tools` to actually generate jars. If you need to, you may use this library directly.

126.1. Repackaging Archives

To repackage an existing archive so that it becomes a self-contained executable archive, use `javadoc:org.springframework.boot.loader.tools.Repackager[]`. The `javadoc:org.springframework.boot.loader.tools.Repackager[]` class takes a single constructor argument that refers to an existing jar or war archive. Use one of the two available `repackage()` methods to either replace the original file or write to a new destination. Various settings can also be configured on the repackager before it is run.

126.2. Nested Libraries

When repackaging an archive, you can include references to dependency files by using the `javadoc:org.springframework.boot.loader.tools.Libraries[]` interface. We do not provide any concrete implementations of `javadoc:org.springframework.boot.loader.tools.Libraries[]` here as they are usually build-system-specific.

If your archive already includes libraries, you can use `javadoc:org.springframework.boot.loader.tools.Libraries#NONE[]`.

126.3. Finding a Main Class

If you do not use `Repackager.setMainClass()` to specify a main class, the repackager uses [ASM](https://asm.ow2.io/) (<https://asm.ow2.io/>) to read class files and tries to find a suitable class with a `public static void main(String[] args)` method. An exception is thrown if more than one candidate is found.

126.4. Example Repackage Implementation

The following example shows a typical repackage implementation:

```
include-code::MyBuildTool[]
```


128. Spring Boot CLI

The Spring Boot CLI is a command line tool that you can use to bootstrap a new project from start.spring.io or encode a password.

129. Installing the CLI

The Spring Boot CLI (Command-Line Interface) can be installed manually by using SDKMAN! (the SDK Manager) or by using Homebrew or MacPorts if you are an OSX user. See [ROOT:installing.html](#) in the “Getting Started” section for comprehensive installation instructions.

130. Using the CLI

Once you have installed the CLI, you can run it by typing `spring` and pressing Enter at the command line. If you run `spring` without any arguments, a help screen is displayed, as follows:

```
$ spring
usage: spring [--help] [--version]
      <command> [<args>]

Available commands are:

  init [options] [location]
      Initialize a new project using Spring Initializr (start.spring.io)

  encodepassword [options] <password to encode>
      Encode a password for use with Spring Security

  shell
      Start a nested shell

Common options:

  --debug Verbose mode
      Print additional status information for the command you are running

See 'spring help <command>' for more information on a specific command.
```

SHELL

You can type `spring help` to get more details about any of the supported commands, as shown in the following example:

```
$ spring help init
spring init - Initialize a new project using Spring Initializr (start.spring.io)

usage: spring init [options] [location]

Option                                Description
-----                                -
-a, --artifact-id <String>            Project coordinates; infer archive name (for
                                     example 'test')
-b, --boot-version <String>            Spring Boot version (for example '1.2.0.RELEASE')
--build <String>                        Build system to use (for example 'maven' or
                                     'gradle') (default: maven)
-d, --dependencies <String>            Comma-separated list of dependency identifiers to
                                     include in the generated project
--description <String>                 Project description
-f, --force                             Force overwrite of existing files
--format <String>                       Format of the generated content (for example
                                     'build' for a build file, 'project' for a
                                     project archive) (default: project)
-g, --group-id <String>                 Project coordinates (for example 'org.test')
-j, --java-version <String>            Language level (for example '1.8')
-l, --language <String>                Programming language (for example 'java')
--list                                 List the capabilities of the service. Use it to
                                     discover the dependencies and the types that are
                                     available
-n, --name <String>                     Project name; infer application name
-p, --packaging <String>                Project packaging (for example 'jar')
--package-name <String>                Package name
-t, --type <String>                     Project type. Not normally needed if you use --
                                     build and/or --format. Check the capabilities of
                                     the service (--list) for more details
--target <String>                       URL of the service to use (default: https://start.
                                     spring.io)
-v, --version <String>                 Project version (for example '0.0.1-SNAPSHOT')
-x, --extract                           Extract the project archive. Inferred if a
                                     location is specified without an extension
```

SHELL

examples:

```
To list all the capabilities of the service:
$ spring init --list

To creates a default project:
$ spring init

To create a web my-app.zip:
$ spring init -d=web my-app.zip

To create a web/data-jpa gradle project unpacked:
$ spring init -d=web,jpa --build=gradle my-dir
```

The `version` command provides a quick way to check which version of Spring Boot you are using, as follows:

```
$ spring version
Spring CLI v{version-spring-boot}
```

SHELL

130.1. Initialize a New Project

The `init` command lets you create a new project by using `start.spring.io` without leaving the shell, as shown in the following example:

```
$ spring init --dependencies=web,data-jpa my-project
Using service at https://start.spring.io
Project extracted to '/Users/developer/example/my-project'
```

SHELL

The preceding example creates a `my-project` directory with a Maven-based project that uses `spring-boot-starter-web` and `spring-boot-starter-data-jpa`. You can list the capabilities of the service by using the `--list` flag, as shown in the following example:

```
$ spring init --list
=====
Capabilities of https://start.spring.io
=====

Available dependencies:
-----
actuator - Actuator: Production ready features to help you monitor and manage your application
...
web - Web: Support for full-stack web development, including Tomcat and spring-webmvc
websocket - WebSocket: Support for WebSocket development
ws - WS: Support for Spring Web Services

Available project types:
-----
gradle-build - Gradle Config [format:build, build:gradle]
gradle-project - Gradle Project [format:project, build:gradle]
maven-build - Maven POM [format:build, build:maven]
maven-project - Maven Project [format:project, build:maven] (default)

...
```

SHELL

The `init` command supports many options. See the `help` output for more details. For instance, the following command creates a Gradle project that uses Java 17 and `war` packaging:

```
$ spring init --build=gradle --java-version=17 --dependencies=websocket --packaging=war sample-app.zip
Using service at https://start.spring.io
Content saved to 'sample-app.zip'
```

SHELL

130.2. Using the Embedded Shell

Spring Boot includes command-line completion scripts for the BASH and zsh shells. If you do not use either of these shells (perhaps you are a Windows user), you can use the `shell` command to launch an integrated shell, as shown in the following example:

```
$ spring shell
Spring Boot (v{version-spring-boot})
Hit TAB to complete. Type 'help' and hit RETURN for help, and 'exit' to quit.
```

SHELL

From inside the embedded shell, you can run other commands directly:

```
$ version
Spring CLI v{version-spring-boot}
```

SHELL

The embedded shell supports ANSI color output as well as `tab` completion. If you need to run a native command, you can use the `!` prefix. To exit the embedded shell, press `ctrl-c`.

Appendix A: Common Application Properties

Various properties can be specified inside your `application.properties` file, inside your `application.yaml` file, or as command line switches. This appendix provides a list of common Spring Boot properties and references to the underlying classes that consume them.



Spring Boot provides various conversion mechanisms with advanced value formatting. Make sure to review [the properties conversion section](#).



Property contributions can come from additional jar files on your classpath, so you should not consider this an exhaustive list. Also, you can define your own properties.

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/actuator.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/cache.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/core.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/data-migration.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/data.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/devtools.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/docker-compose.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/integration.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/json.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/mail.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/rsocket.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/security.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/server.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/templating.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/testcontainers.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/testing.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/transaction.adoc[]

Unresolved directive in appendix.adoc - include::partial\$configuration-properties/web.adoc[]

Appendix B: Auto-configuration Classes

This appendix contains details of all of the auto-configuration classes provided by Spring Boot, with links to documentation and source code. Remember to also look at the conditions report in your application for more details of which features are switched on. (To do so, start the app with `--debug` or `-Ddebug` or, in an Actuator application, use the `conditions` endpoint).

132. Managed Dependency Coordinates

The following table provides details of all of the dependency versions that are provided by Spring Boot in its CLI (Command Line Interface), Maven dependency management, and Gradle plugin. When you declare a dependency on one of these artifacts without declaring a version, the version listed in the table is used.

Unresolved directive in appendix.adoc - include::partial\$dependency-versions/documented-coordinates.adoc[]

Appendix C: Dependency Versions

This appendix provides details of the dependencies that are managed by Spring Boot.

133. Version Properties

The following table provides all properties that can be used to override the versions managed by Spring Boot. Browse the `{code-spring-boot}/platform/spring-boot-dependencies/build.gradle[ spring-boot-dependencies build.gradle]` for a complete list of dependencies. You can learn how to customize these versions in your application in the [build-tool-plugin:index.html](#) documentation.

Unresolved directive in appendix.adoc - include::partial\$dependency-versions/documentated-properties.adoc[]

Appendix D: Deprecated Application Properties

The following deprecated properties can be specified inside your `application.properties` file, inside your `application.yaml` file, or as command line switches. Support for these properties will be removed in a future release and should you should migrate away from them.

Spring Boot includes a useful `spring-boot-properties-migrator` tool to help you migrate away from deprecated properties. To use the property migrator tool, add the following dependency to your project:

Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-properties-migrator</artifactId>
  <scope>runtime</scope>
</dependency>
```

XML

Gradle

```
runtimeOnly("org.springframework.boot:spring-boot-properties-migrator")
```

GRADLE

Once added as a dependency to your project, the tool will not only analyze your application's environment and print diagnostics at startup, but also temporarily migrate properties at runtime for you.

Remember to remove the dependency when your migration is complete.

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/actuator.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/cache.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/core.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/data-migration.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/data.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/devtools.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/docker-compose.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/integration.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/json.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/mail.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/rsocket.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/security.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/server.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/templating.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/testcontainers.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/testing.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/transaction.adoc[]

Unresolved directive in appendix.adoc - include::partial\$deprecated-configuration-properties/web.adoc[]

Appendix E: Test Auto-configuration Annotations

This appendix describes the `@...Test` auto-configuration annotations that Spring Boot provides to test slices of your application.

134. Test Slices

The following table lists the various `@...Test` annotations that can be used to test slices of your application and the auto-configuration that they import by default:

Unresolved directive in appendix.adoc - include::partial\$slices/documentated-slices.adoc[]

Last updated 2026-01-30 07:30:00 UTC